MPLAB® X IDE User's Guide

Notice to Development Tools Customers

Important:
All documentation becomes dated, and Development Tools manuals are no exception. Our tools and documentation are constantly evolving to meet customer needs, so some actual dialogs and/or tool descriptions may differ from those in this document. Please refer to our website (www.microchip.com/) to obtain the latest version of the PDF document.

Documents are identified with a DS number located on the bottom of each page. The DS format is DS<DocumentNumber><Version>, where <DocumentNumber> is an 8-digit number and <Version> is an uppercase letter.

For the most up-to-date information, find help for your tool at onlinedocs.microchip.com/.



© 2022 Microchip Technology Inc.
and its subsidiaries

Table of Contents

Notice to Development Tools Customers 1

1. Preface 6
   - 1.1. Conventions Used in This Guide 6
   - 1.2. Recommended Reading 7

2. What is MPLAB X IDE? 8
   - 2.1. Development Tools Ecosystem and MPLAB X IDE 8
   - 2.2. Embedded Systems and MPLAB X IDE 9
   - 2.3. MPLAB X IDE Support Features 24

3. Before You Begin 27
   - 3.1. Review Installation Requirements 27
   - 3.2. JRE and MPLAB X IDE 27
   - 3.3. Install the USB Device Drivers (for Hardware Tools) 27
   - 3.4. Connect to a Target (for Hardware Tools) 30
   - 3.5. Install the Language Tools 30
   - 3.6. Install MPLAB X IDE 31
   - 3.7. Launch the IDE and View the Desktop 31
   - 3.8. Shop the MPLAB X Store 32
   - 3.9. Launch Multiple Instances of the IDE 33
   - 3.10. Launch Multiple Versions of the IDE 35
   - 3.11. Launch using Startup Parameters 35

4. Tutorial 36
   - 4.1. Selecting a Device and Development Board 36
   - 4.2. Downloading the Example Code 36
   - 4.3. Installing and Setting Up the Software 37
   - 4.4. Connecting the Hardware 38
   - 4.5. Opening an Example Project in MPLAB X IDE 38
   - 4.6. Examining MCC Generated Files 40
   - 4.7. Setting Project Properties 41
   - 4.8. Running the Code 43
   - 4.9. Debugging the Code 44
   - 4.10. Setting Breakpoints 45
   - 4.11. Stepping Through Code 46
   - 4.12. Viewing Variable Values 47
   - 4.13. Watching Symbol Values Change 49
   - 4.14. Viewing I/O Registers 50
   - 4.15. Viewing Device Memory (including Configuration Bits) 52
   - 4.16. Programming a Device 54
   - 4.17. Other Examples 54

5. Basic Development and Debug Flow 56
   - 5.1. Create a New Project 56
   - 5.2. View Changes to Desktop 65
   - 5.3. Open Project Properties 66

5.4. View or Make Changes to Project Properties....67
5.5. Set Up or Change Debugger/Programmer Tool Options....68
5.6. Set Up or Change Language Tool Options....69
5.7. Set Language Tool Locations....70
5.8. Set Other Tool Options....72
5.9. Add Files to a Project....73
5.10. Set Build Properties....81
5.11. Build a Project....85
5.12. Run Code....85
5.13. Debug Code....86
5.14. Control Program Execution with Breakpoints....88
5.15. Step Through Code....94
5.16. Watch Symbol Values Change....94
5.17. Watch Local Variable Values Change....96
5.18. View Registers for the Project (I/O View)....96
5.19. View or Change Device Memory....98
5.20. Set Configuration Values in the Configuration Bits Window....100
5.21. Program a Device....103

6. Projects....105
6.1. Work with Packs....105
6.2. Project Types....109
6.3. Project Files....150
6.4. Work with Projects....156
6.5. Project Reference....160

7. Editor....165
7.1. Editor Usage....165
7.2. Editor Options....167
7.3. Hyperlinks in Code....170
7.4. Editor Red Bangs....171
7.5. Variable Values....171
7.6. Code Folding....172
7.7. C Code Refactoring....174
7.8. C/C++ Code Error Directive....175

8. Development and Debug Features....176
8.1. Use the Stopwatch....176
8.2. View the Disassembly Window....177
8.3. View the Call Stack....178
8.4. View the Call Graph....178
8.5. View the Dashboard Display....179
8.6. Visualize Data....182
8.7. Improve Your Code....182
8.8. Control Source Code using Local History....182
8.9. Control Source Code using a Revision Control System....183
8.10. Collaborate on Code Development and Error Tracking....186
8.11. Add Plugin Tools....187

8.12. Hardware Tool Connections and Debugging....191
8.13. Work with Third-Party Hardware Tools....193

9. Analysis....194
9.1. Compiler Advisor....194
9.2. Stack Guidance....202
9.3. MPLAB Analysis Tool Suite....203

10. Automation....206
10.1. IDE Scripting (autoload.py)....206
10.2. CI/CD Wizard....216

11. Troubleshooting....247
11.1. USB Driver Installation Issues....247
11.2. Operating System Issues....247
11.3. NetBeans Platform Issues....247
11.4. MPLAB X IDE Issues....248
11.5. Errors....250
11.6. Forums....254

12. Desktop Reference....255
12.1. Menus....255
12.2. Toolbars....267
12.3. Status Bar....273
12.4. Grayed Out or Missing Items and Buttons....273

13. MPLAB X IDE Windows and Dialogs....275
13.1. MPLAB X IDE Windows Behavior....275
13.2. Action Items Window....277
13.3. Breakpoints Window....278
13.4. New Breakpoint Dialog....279
13.5. Call Stack Window....283
13.6. Customize Toolbars Window....284
13.7. Dashboard Window....285
13.8. Disassembly Window....286
13.9. I/O View Window....287
13.10. Licenses Windows....289
13.11. Manage Network Tools Dialog....292
13.12. Memory Windows - 8- and 16-Bit Devices....293
13.13. Memory Windows - 32-Bit Devices....314
13.14. Memory Windows Associated Dialogs....329
13.15. Message Center....333
13.16. Output Window....333
13.17. Pack Manager Window....334
13.18. Project Properties Window....338
13.19. Projects Window....339
13.20. Tools Options Window, Embedded....343
13.21. Tools Options Window, Plugins....349
13.22. Trace Window....350

13.23. Variables Window....351
13.24. Watches Window....353
13.25. Wizards....355

14. NetBeans Windows and Dialogs....357
14.1. NetBeans Specific Windows and Window Menus....357
14.2. NetBeans Specific Dialogs....357

15. Configuration Settings Summary....358
15.1. AVR GCC Toolchain....358
15.2. Arm GCC Toolchain....359
15.3. XC Toolchains....360

16. Revision History....361
16.1. Revision F (January 2022)....361
16.2. Revision E (September 2019)....362
16.3. Revision D (September 2015)....363
16.4. Revision C (March 2014)....363
16.5. Revision B (October 2012)....364
16.6. Revision A (November 2011)....365

17. Glossary....366

The Microchip Website....384

Product Change Notification Service....384

Customer Support....384

Microchip Devices Code Protection Feature....384

Legal Notice....384

Trademarks....385

Quality Management System....386

Worldwide Sales and Service....387

1. Preface

MPLAB X IDE documentation and support information is discussed in this section.

1.1 Conventions Used in This Guide

The following conventions may appear in this documentation:

Table 1-1. Documentation Conventions

Description	Represents	Examples
Arial font:		
Italic characters	Referenced books	*MPLAB® IDE User's Guide*
	Emphasized text	...is the *only* compiler...
Initial caps	A window	the Output window
	A dialog	the Settings dialog
	A menu selection	select Enable Programmer
Quotes	A field name in a window or dialog	"Save project before build"
Underlined, italic text with right angle bracket	A menu path	*File>Save*
Bold characters	A dialog button	Click **OK**
	A tab	Click the **Power** tab
N'Rnnnn	A number in verilog format, where N is the total number of digits, R is the radix and n is a digit.	4'b0010, 2'hF1
Text in angle brackets < >	A key on the keyboard	Press <Enter>, <F1>
Courier New font:		
Plain Courier New	Sample source code	`#define START`
	Filenames	`autoexec.bat`
	File paths	`c:\mcc18\h`
	Keywords	`_asm, _endasm, static`
	Command-line options	`-Opa+, -Opa-`
	Bit values	`0, 1`
	Constants	`0xFF, 'A'`
Italic Courier New	A variable argument	*`file.o`*, where *`file`* can be any valid filename
Square brackets [ ]	Optional arguments	`mcc18 [options] file [options]`
Curly brackets and pipe character: { | }	Choice of mutually exclusive arguments; an OR selection	`errorlevel {0|1}`

.....continued		
Description	**Represents**	**Examples**
Ellipses...	Replaces repeated text	`var_name [, var_name...]`

	Represents code supplied by user	`void main (void)`
		`{ ...`
		`}`

1.2 Recommended Reading

This document describes how to use the MPLAB X IDE. Other useful documents are listed below. The following Microchip documents are available and recommended as supplemental reference resources.

Readme for MPLAB X IDE

For the latest information on using MPLAB X IDE, select *Help>Release Notes* on the IDE toolbar. The release notes contain update information and known issues that may not be included in this user's guide.

Readme Files

For the latest information on using other tools, read the tool-specific Readme files which also can be found by selecting *Help>Release Notes* . The Readme files contain update information and known issues that may not be included in the tool user's guide.

Online Files

Select *Help>Contents Help* or *Help>Tool Contents Help* on the IDE toolbar to find links to documentation for specific development tools. Or search the Online Technical Documentation webpage for MPLAB related documentation.

Device Data Sheets and Family Reference Manuals

See the Microchip web site, www.microchip.com, for complete and updated versions of Microchip device data sheets and related family reference manuals.

2. What is MPLAB X IDE?

MPLAB® X IDE is a software program that is used to develop applications for Microchip microcontrollers, microprocessors, and digital signal controllers.

This development tool is called an Integrated Development Environment, or IDE, because it provides a single integrated “environment” to develop code for embedded devices. MPLAB X IDE works seamlessly within the Development Tools Ecosystem of software and tools, many of which are completely free.

2.1 Development Tools Ecosystem and MPLAB X IDE

The Development Tools Ecosystem consists of software and hardware tools designed to help you move from product concept through release. While MPLAB X IDE is definitely a development tool, the IDE can work with tools at all stages through access or integration.

Figure 2-1. Development Tools Ecosystem

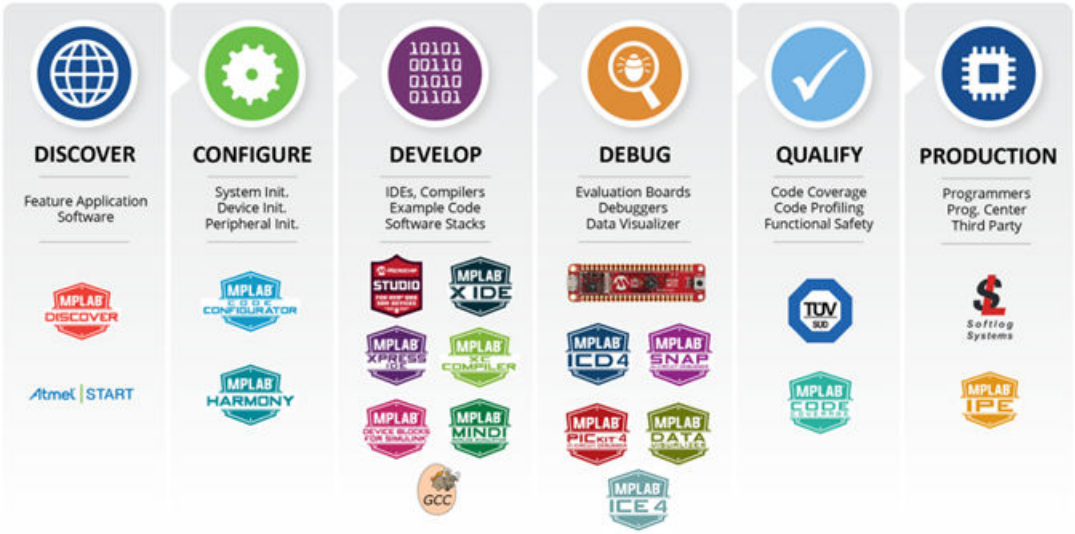


Table 2-1. Ecosystem Tools and MPLAB X IDE

Ecosystem Stage	Ecosystem Tool	MPLAB X IDE Connection	Extra features with license?
Discover	MPLAB Discover	Access from icon  . Download and open a project.	
Discover	Atmel START	Import a START project.	
Configure	MPLAB Code Configurator	Install MCC plugin. Enable from icon  .	
Configure	MPLAB Harmony	Install Launcher plugin. Enable from *Tools>Embedded*.	
Develop	MPLAB XC C Compilers	Use Built-in support.	yes
Develop	GNU C Compilers (GCC)	Use Built-in support.	
Develop	MPLAB Xpress	Download and open a project.	

.....continued			
Ecosystem Stage	**Ecosystem Tool**	**MPLAB X IDE Connection**	**Extra features with license?**
Develop	MPLAB Device Blocks for Simulink	Download and open a project.	
Develop	MPLAB MINDI Analog Simulator	Add to digital simulation by using MPLAB MINDI.	
Develop	Microchip Studio for AVR/SAM	Import a Studio project.	
Debug	Evaluation Boards	Use Built-in support.	
Debug	MPLAB Debuggers	Use Built-in support.	
Debug	MPLAB Data Visualizer	Use Built-in support. Enable from icon  .	
Qualify	MPLAB XC C Compilers (TUV SUD)	Use Built-in support.	yes
Qualify	MPLAB Analysis Tool Suite	Use Built-in support. Enable from *Tools>Analysis*.	yes
Production	MPLAB IPE	Get more programming features by using MPLAB IPE.	
Production	MPLAB Programmers	Use Built-in support.	
Production	Microchip Programming Center	Access from icon  .	
Production	Third-Party Programmers	Get more programming features by using 3rd party programmers.	

2.2 Embedded Systems and MPLAB X IDE

For those new to embedded system development, the following overview sections are recommended.

2.2.1 An Overview of Embedded Systems

An embedded system is typically a design that uses the power of a small microcontroller, like the Microchip PIC® or AVR® microcontroller (MCU). These microcontrollers combine a microprocessor unit (like the CPU in a personal computer) with some additional circuits called peripherals, plus some additional circuits, on the same chip to make a small control module requiring few other external devices. This single device can then be embedded into other electronic and mechanical devices for low-cost digital control.

2.2.1.1 Differences Between an Embedded Controller and a Personal Computer

The main difference between an embedded controller and a personal computer is that the embedded controller is dedicated to one specific task or set of tasks. A personal computer is designed to run many different types of programs and to connect to many different external devices. An embedded controller has a single program and, as a result, can be made cheaply to include just enough computing power and hardware to perform that dedicated task.

A personal computer has a relatively expensive generalized central processing unit (CPU) at its heart with many other external devices (memory, disk drives, video controllers, network interface circuits, etc.). An embedded system has a low-cost MCU for its intelligence, has many peripheral circuits on the same chip and has relatively few external devices.

Often an embedded system is an invisible part, or sub-module of another product, such as a cordless drill, refrigerator or garage door opener. The controller in these products does a tiny portion of the function of the whole device. The controller adds low-cost intelligence to some of the critical sub-systems in these devices.

An example of an embedded system is a smoke detector. Its function is to evaluate signals from a sensor and sound an alarm if the signals indicate the presence of smoke. A small program in the smoke detector either runs in an infinite loop, sampling the signal from the smoke sensor, or lies dormant in a low-power “Sleep” mode, being awakened by a signal from the sensor. The program then sounds the alarm. The program would possibly have a few other functions, such as a user test function and a low battery alert.

While a personal computer with a sensor and audio output could be programmed to do the same function, it would not be a cost-effective solution (nor would it run on a nine-volt battery, unattended for years). Embedded designs use inexpensive microcontrollers to put intelligence into the everyday things in our environment, such as smoke detectors, cameras, cell phones, appliances, automobiles, smart cards and security systems.

2.2.1.2 Components of a Microcontroller

The microcontroller (MCU) has on-chip program memory (Figure 2-2 or Figure 2-3) for the firmware, or coded instructions, to run a program (Figure 2-4 or Figure 2-5). A Program Counter (PC) is used to address program memory, including Reset and interrupt addresses. A hardware stack is used with call and return instructions in code, so it works with, but is not part of, program memory. Device data sheets describe the details of program memory operation, vectors and the stack.

Figure 2-2. PIC® MCU Data Sheet - Program Memory and Stack

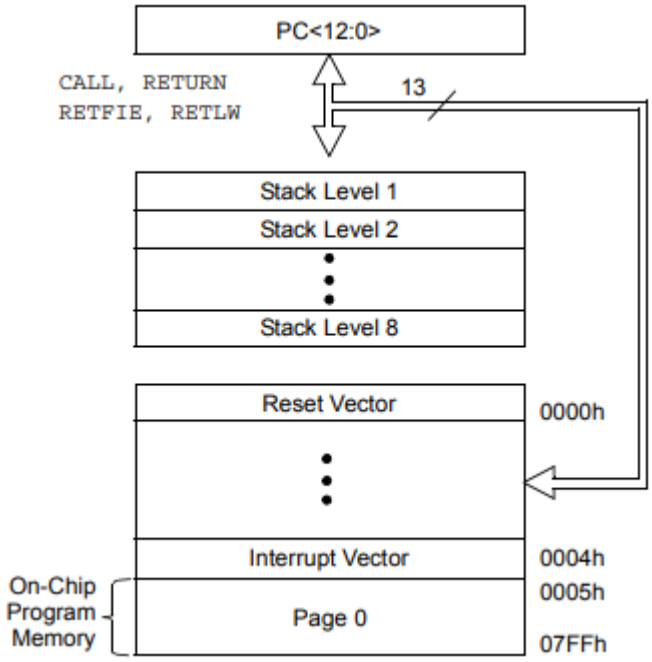


Figure 2-3. AVR® MCU Data Sheet - Program Memory

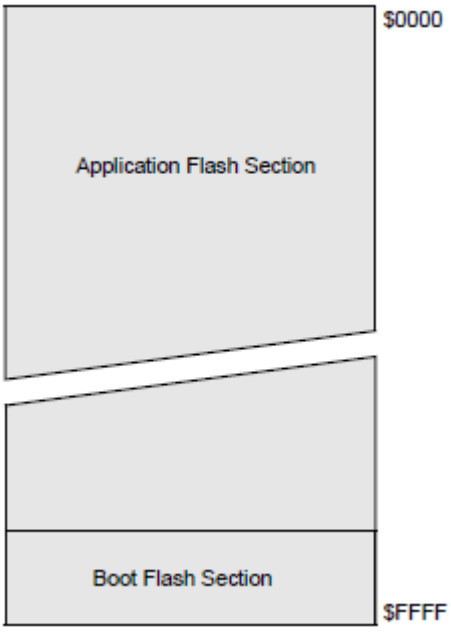


Figure 2-4. PIC® MCU Data Sheet – Instruction Set (Excerpt)

RRF	**Rotate Right f through Carry**
Syntax:	[*label*] RRF f,d
Operands:	$0 \leq f \leq 127$ $d \in [0,1]$
Operation:	See description below
Status Affected:	C
Description:	The contents of register 'f' are rotated one bit to the right through the Carry flag. If 'd' is '0', the result is placed in the W register. If 'd' is '1', the result is placed back in register 'f'.

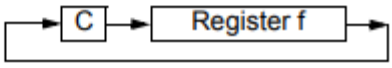


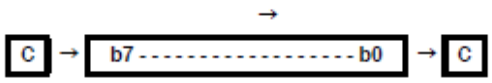
Figure 2-5. AVR® MCU Instruction Set (Excerpt)

ROR – Rotate Right through Carry

Description:

Shifts all bits in Rd one place to the right. The C Flag is shifted into bit 7 of Rd. Bit 0 is shifted into the C Flag. This operation, combined with ASR, effectively divides multi-byte signed values by two. Combined with LSR it effectively divides multi-byte unsigned values by two. The Carry Flag can be used to round the result.

Operation:



	Syntax:	Operands:	Program Counter:
(i)	ROR Rd	0 ≤ d ≤ 31	PC ← PC + 1

16-bit Opcode:

1001	010d	dddd	0111

Status Register (SREG) and Boolean Formula:

I	T	H	S	V	N	Z	C
–	–	–	⇔	⇔	⇔	⇔	⇔

S: N ⊕ V, For signed tests.

V: N ⊕ C (For N and C after the shift)

N: R7
Set if MSB of the result is set; cleared otherwise.

Z: $\overline{R7} \bullet \overline{R6} \bullet \overline{R5} \bullet \overline{R4} \bullet \overline{R3} \bullet \overline{R2} \bullet \overline{R1} \bullet \overline{R0}$
Set if the result is $00; cleared otherwise.

C: Rd0
Set if, before the shift, the LSB of Rd was set; cleared otherwise.

R (Result) equals Rd after the operation.

Example:

```
            lsr   r19          ; Divide r19:r18 by two
            ror   r18          ; r19:r18 is an unsigned two-byte integer
            brcc  zeroenc1     ; Branch if carry cleared
            asr   r17          ; Divide r17:r16 by two
            ror   r16          ; r17:r16 is a signed two-byte integer
            brcc  zeroenc2     ; Branch if carry cleared
            ...
zeroenc1:   nop                ; Branch destination (do nothing)
            ...

zeroenc1:   nop                ; Branch destination (do nothing)
```

Words: 1 (2 bytes)
Cycles: 1

The microcontroller also has data, or File Register, memory. This memory consists of Special Function Registers (SFRs) and General Purpose Registers (GPRs). SFRs are registers used by the CPU and peripheral functions for controlling the desired operation of the device. GPRs are for storage of variables that the program will need for computation or temporary storage. Some microcontrollers have additional data EEPROM memory. As with program memory, device data sheets describe the details of data memory use and operation.

Table 2-2. PIC® MCU Data Sheet – File Registers Example

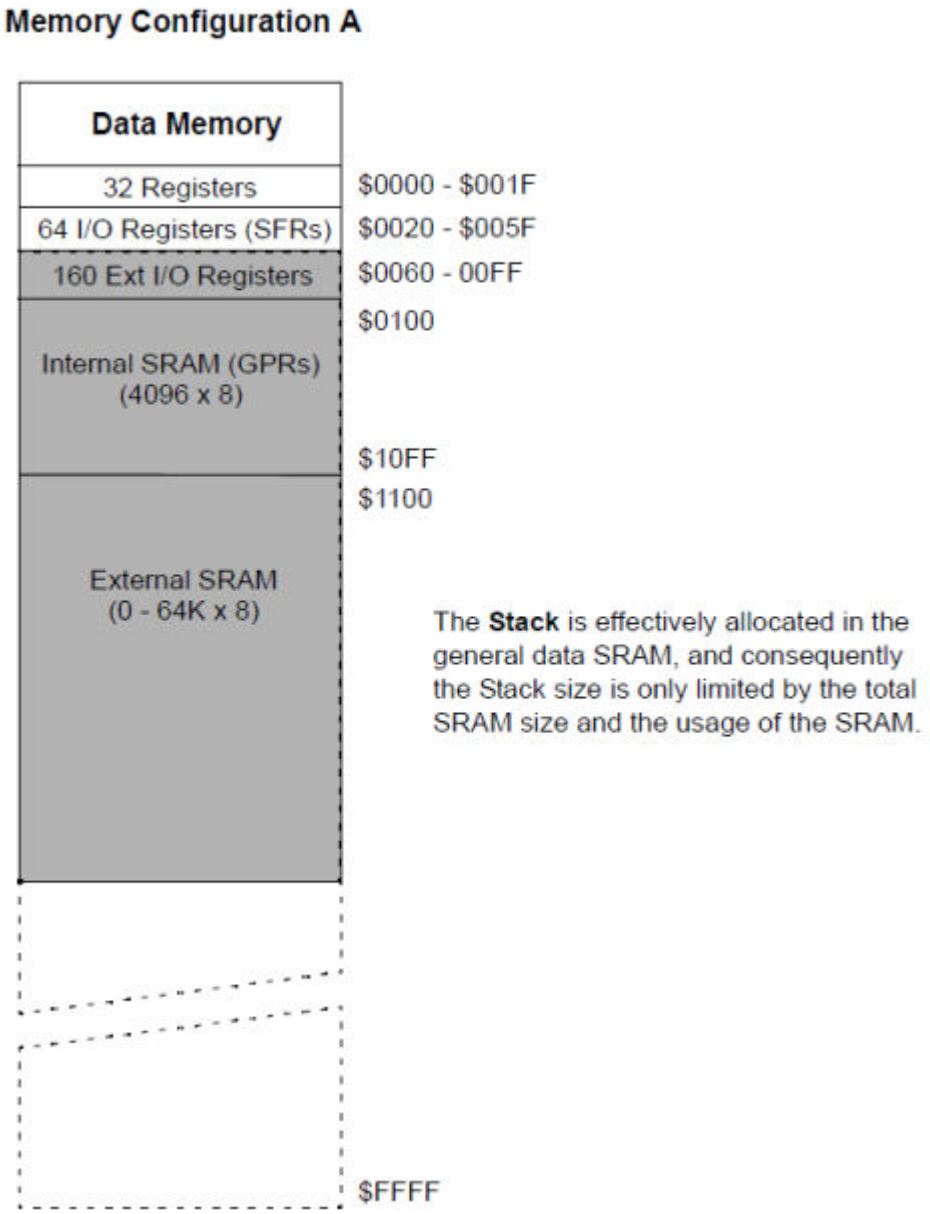
Bank 0	File Address	Bank 1	File Address	Bank 2	File Address	Bank 3	File Address
Indirect addr. (1)	00h	Indirect addr. (1)	80h	Indirect addr. (1)	100h	Indirect addr. (1)	180h
TMR0	01h	OPTION_REG	81h	TMR0	101h	OPTION_REG	181h
PCL	02h	PCL	82h	PCL	102h	PCL	182h
STATUS	03h	STATUS	83h	STATUS	103h	STATUS	183h
FSR	04h	FSR	84h	FSR	104h	FSR	184h
PORTA	05h	TRISA	85h	PORTA	105h	TRISA	185h
PORTB	06h	TRISB	86h	PORTB	106h	TRISB	186h
PORTC	07h	TRISC	87h	PORTC	107h	TRISC	187h
	08h		88h		108h		188h
	09h		89h		109h		189h
PCLATH	0Ah	PCLATH	8Ah	PCLATH	10Ah	PCLATH	18Ah
INTCON	0Bh	INTCON	8Bh	INTCON	10Bh	INTCON	18Bh
PIR1	0Ch	PIE1	8Ch	EEDAT	10Ch	EECON1	18Ch
PIR2	0Dh	PIE2	8Dh	EEADR	10Dh	EECON2(1)	18Dh
TMR1L	0Eh	PCON	8Eh	EEDATH	10Eh		18Eh
TMR1H	0Fh	OSCCON	8Fh	EEADRH	10Fh		18Fh
T1CON	10h	OSCTUNE	90h		110h		190h
TMR2	11h		91h		111h		191h
T2CON	12h	PR2	92h		112h		192h
SSPBUF	13h	SSPADD(2)	93h		113h		193h
SSPCON	14h	SSPSTAT	94h		114h		194h
CCPR1L	15h	WPUA	95h	WPUB	115h		195h
CCPR1H	16h	IOCA	96h	IOCB	116h		196h
CCP1CON	17h	WDTCON	97h		117h		197h
RCSTA	18h	TXSTA	98h	VRCON	118h		198h
TXREG	19h	SPBRG	99h	CM1CON0	119h		199h
RCREG	1Ah	SPBRGH	9Ah	CM2CON0	11Ah		19Ah
	1Bh	BAUDCTL	9Bh	CM2CON1	11Bh		19Bh
PWM1CON	1Ch		9Ch		11Ch		19Ch
ECCPAS	1Dh		9Dh		11Dh	PSTRCON	19Dh

.....continued

Bank 0	File Address	Bank 1	File Address	Bank 2	File Address	Bank 3	File Address
ADRESH	1Eh	ADRESL	9Eh	ANSEL	11Eh	SRCON	19Eh
ADCON0	1Fh	ADCON1	9Fh	ANSELH	11Fh		19Fh
General Purpose Register 96 Bytes	20h	General Purpose Register 80 Bytes	A0h	General Purpose Register 80 Bytes	120h		1A0h
			EFh		16Fh		
		accesses 70h-7Fh	F0h	accesses 70h-7Fh	170h	accesses 70h-7Fh	1F0h
	7Fh		FFh		17Fh		1FFh

Empty table cells: Unimplemented data memory locations, read as '0'.
Note 1: Not a physical register.
Note 2: Address 93h also accesses the SSP Mask (SSPMSK) register under certain conditions.

Figure 2-6. AVR® MCU Data Sheet – SRAM Data Memory and Stack



In addition to memory, the microcontroller has a number of peripheral device circuits on the same chip. Some peripheral devices are called input/output (I/O) ports. I/O ports are pins on the microcontroller that can be used as outputs and driven high or low to send signals, blink lights, drive speakers – just about anything that can be sent through a wire. Often these pins are bidirectional and can also be configured as inputs, allowing the program to respond to an external switch, a sensor, or to communicate with some external device.

© 2022 Microchip Technology Inc.
and its subsidiaries

Figure 2-7. PIC® MCU Data Sheet – Block Diagram (Excerpt)

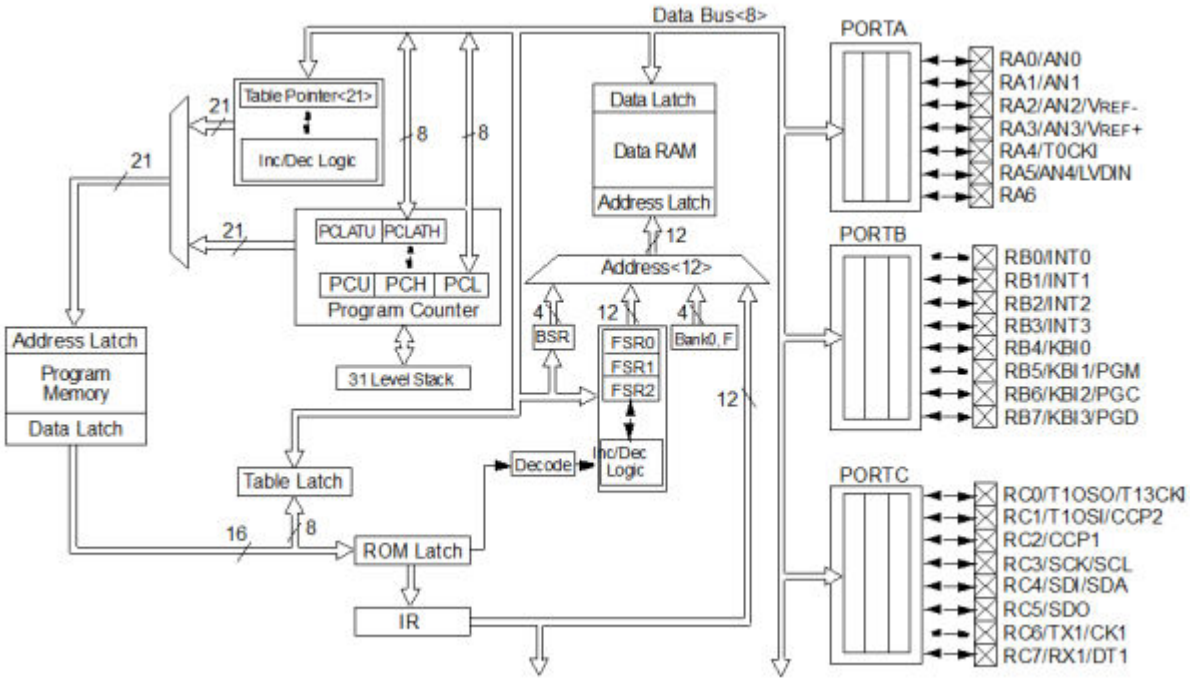
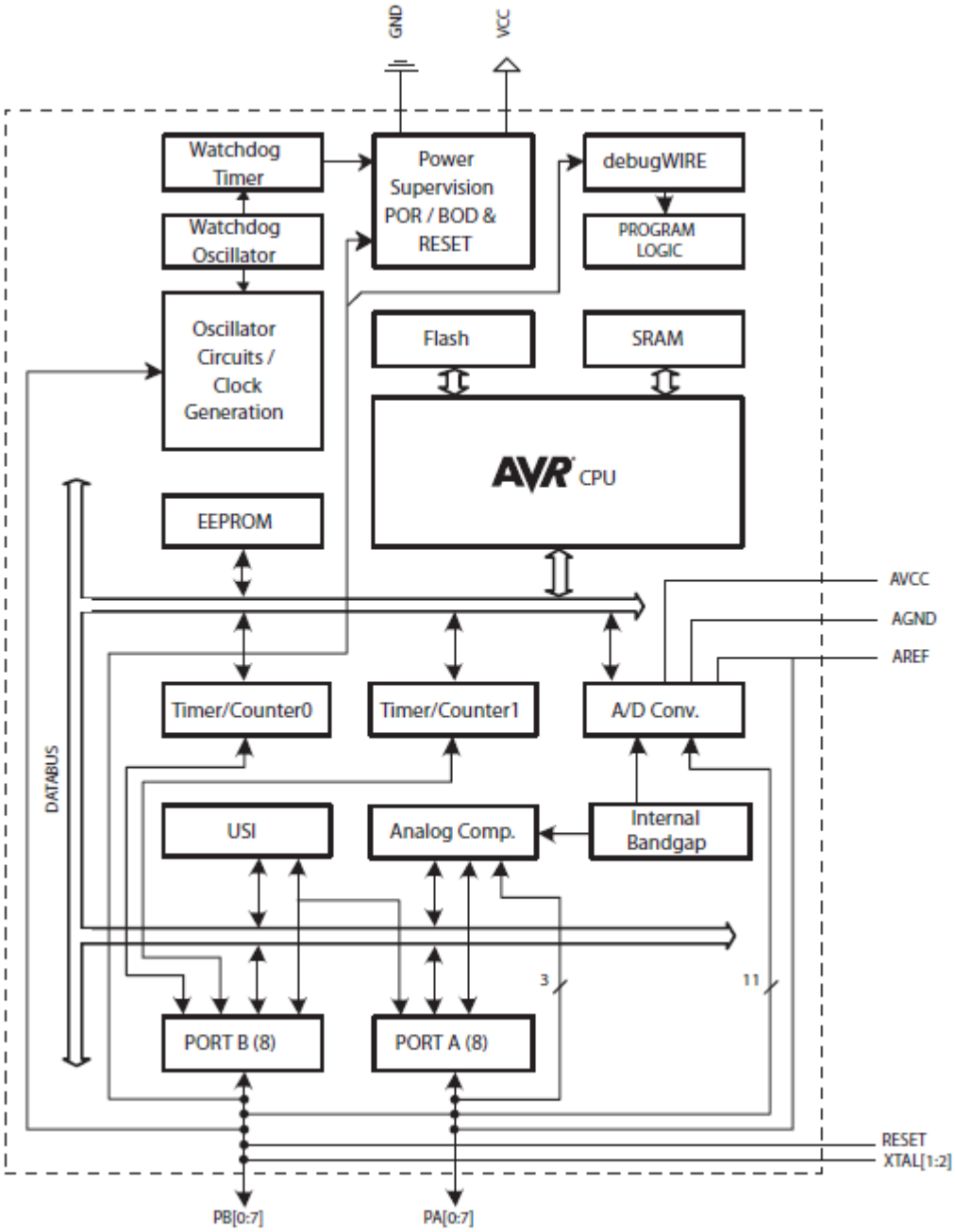


Figure 2-8. AVR® MCU Data Sheet – Block Diagram (Excerpt)



To design such a system, choose which peripherals are necessary for the application.

The following is a list of common peripherals:

- Analog-to-Digital Converters (ADCs) allow microcontrollers to connect to sensors and receive changing voltage levels.
- Serial communication peripherals that allow streaming communications over a few wires to another microcontroller, to a local network, or to the Internet.
- Peripherals on the PIC MCU called “timers” accurately measure signal events and generate and capture communications signals, produce precise waveforms, even automatically reset the microcontroller if it gets “hung” or lost due to a power glitch or hardware malfunction.

- Other peripherals detect when the external power is dipping to dangerous levels, so that the microcontroller can store critical information and safely shut down before power is completely lost.

The peripherals, and the amount of memory an application needs to run a program, largely determine which PIC MCU to use. Other factors could include the power consumed by the microcontroller and its “form factor,” i.e., the size and characteristics of the physical package that must reside on the target design.

Figure 2-9. PIC® MCU Device Package Example

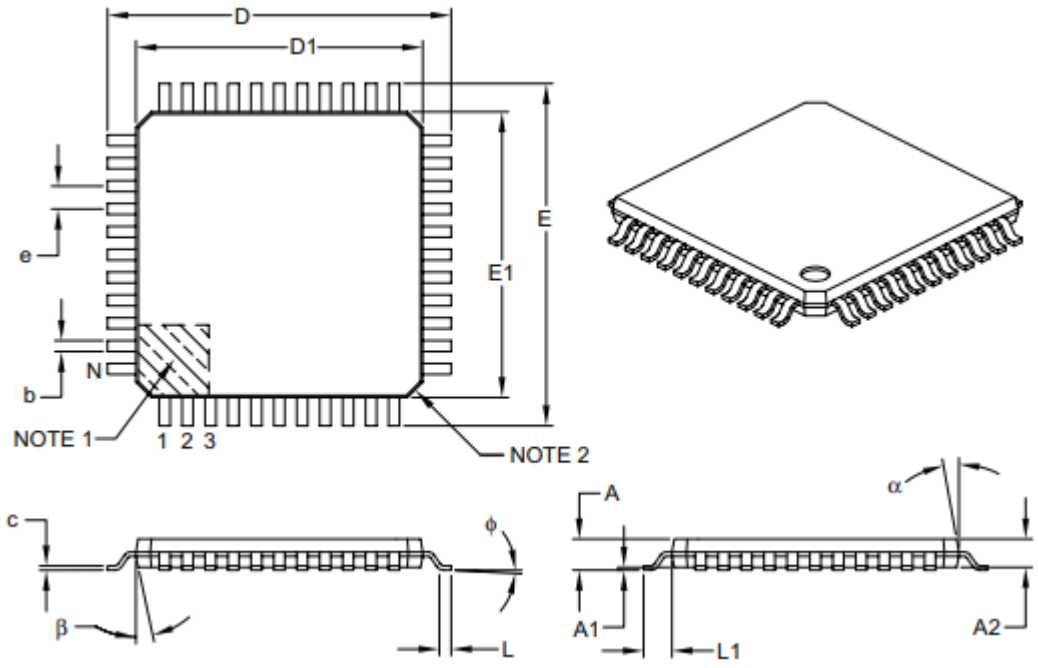
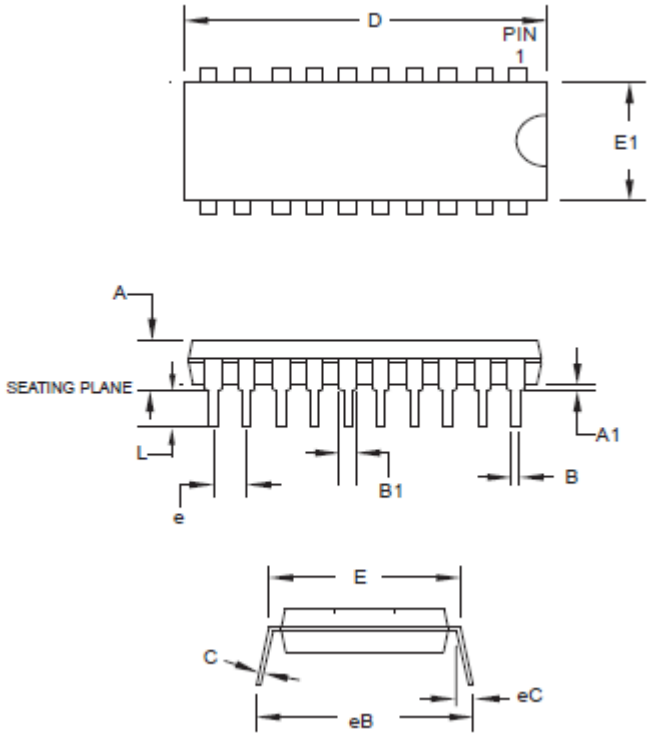


Figure 2-10. Example AVR® MCU Device Package



A microcontroller becomes active when power is applied and an oscillator begins generating a clocking signal. Depending on the microcontroller, there may be several internal and external oscillator operational modes.

Figure 2-11. PIC® MCU Data Sheet – Timing Diagram (Excerpt)

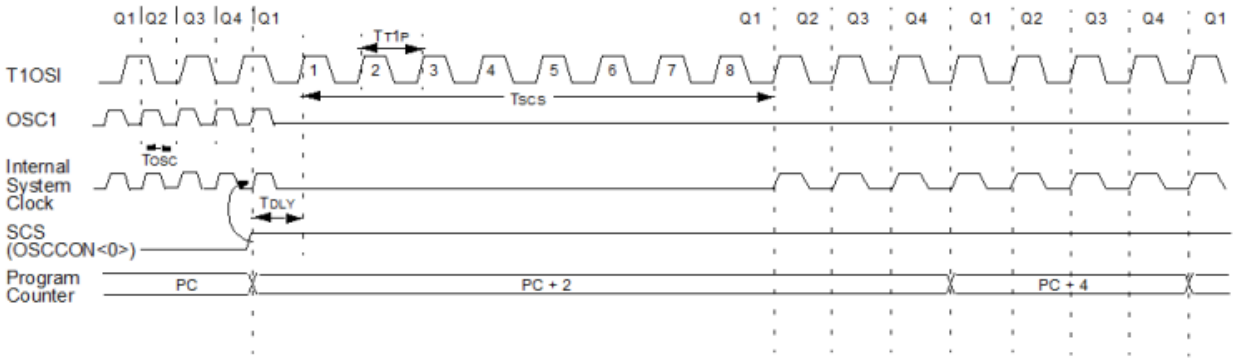
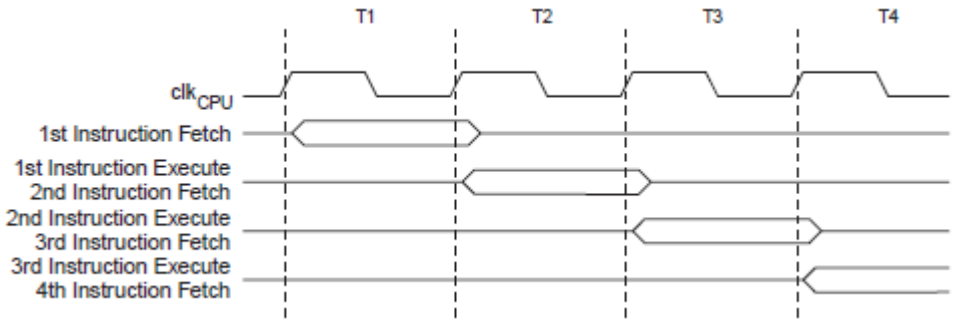


Figure 2-12. AVR® MCU Data Sheet – Timing (Excerpt)



2.2.1.3 Implementing an Embedded System Design with MPLAB X IDE

A development system for embedded controllers is a system of programs running on a computer that help to write, edit, debug and program code – which is the intelligence of embedded systems applications – into a microcontroller. MPLAB X IDE is such a system, it contains all the components needed to design and deploy embedded systems applications.

The typical tasks for developing an embedded controller application are:

1. Create the high level design. From the features and performance desired, decide which MCU is best suited to the application, then design the associated hardware circuitry. After determining which peripherals and pins control the hardware, write the firmware – the software that will control the hardware aspects of the embedded application. A language tool such as an assembler, which is directly translatable into machine code, or a compiler that allows a more natural language for creating programs, should be used to write and edit code. Assemblers and compilers help make the code understandable, allowing function labels to identify code routines with variables that have names associated with their use, and with constructs that help organize the code in a maintainable structure.
2. Compile, assemble and link the software using the assembler and/or compiler and linker to convert your code into “ones and zeros” – machine code for the MCUs. This machine code will eventually become the firmware (the code programmed into the microcontroller).
3. Test your code. Usually a complex program does not work exactly the way imagined, and “bugs” need to be removed from the design to get proper results. The debugger allows you to see the “ones and zeros” execute, related to the source code you wrote, with the symbols and function names from your program. Debugging allows you to experiment with your code to see the value of variables at various points in the program, and to do “what if” checks, changing variable values and stepping through routines.
4. “Burn” the code into a microcontroller and verify that it executes correctly in the finished application.

Of course, each of these steps can be quite complex. The important thing is to concentrate on the details of your own design, while relying upon MPLAB X IDE and its components to get through each step without continuously encountering new learning curves.

Step 1 is driven by the designer, although MPLAB X IDE can help in modeling circuits and code so that crucial design decisions can be made.

MPLAB X IDE really helps with steps 2 through 4. Its Programmer’s Editor helps write correct code with the language tools of choice. The editor is aware of the assembler and compiler programming constructs and automatically “color-keys” the source code to help ensure it is syntactically correct. The Project Manager enables you to organize the various files used in your application: source files, processor description header files and library files. When the code is built, you can control how rigorously code will be optimized for size or speed by the compiler and where individual variables and program data will be programmed into the device. You can also specify a “memory model” in order to make the best use of the microcontroller’s memory for your application. If the language tools run into errors when building the application, the offending line is shown and can be double clicked to go to the corresponding source file for immediate editing. After editing, you will rebuild and try your application again. Often this write-compile-fix loop is done many times for complex code as the sub-sections are written and tested. MPLAB X IDE goes through this loop with maximum speed, allowing you to get on to the next step.

When the code builds with no errors, it needs to be tested. MPLAB X IDE has components called “debuggers” and free software simulators for all MCUs to help test the code. Even if the hardware is not yet finished, you can begin testing the code with the simulator, a software program that simulates the execution of the microcontroller. The simulator can accept a simulated input (stimulus), in order to model how the firmware responds to external signals. The simulator can measure code execution time, single step through code to watch variables and peripherals, and trace the code to generate a detailed record of how the program ran.

When the hardware is in a prototype stage, a hardware debugger, such as an in-circuit emulator or an in-circuit debugger, can be used. These debug tools run the code in real time on your actual application by using special circuitry built into many devices with Flash program memory. They can “see into” the target microcontroller’s program and data memory, and stop and start program execution, allowing you to test the code with the microcontroller in place on the application.

After the application is running correctly, you can program a microcontroller with one of Microchip’s devices or development programmers. These programmers verify that the finished code is programmed correctly into the device.

© 2022 Microchip Technology Inc.
and its subsidiaries

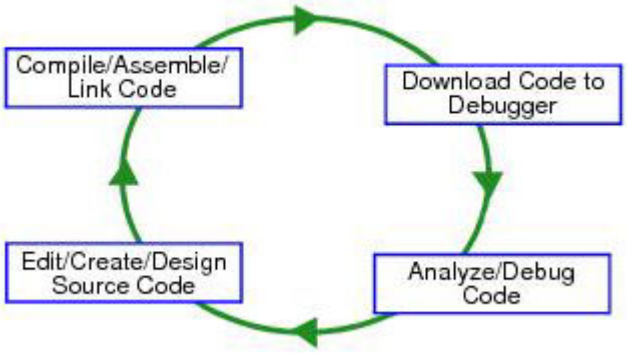
MPLAB® X IDE supports most PIC® MCUs, all dsPIC® DSCs, and a growing number of AVR® and SAM MCUs.

2.2.2 The Development Cycle

The process for writing an application is often described as a development cycle, since it is rare that all the steps from design to implementation can be done flawlessly the first time. More often code is written, tested and then modified to produce an application that performs correctly.

The Integrated Development Environment allows the embedded systems design engineer to progress through this cycle without the distraction of switching among an array of tools. By using MPLAB X IDE, all the functions are integrated, allowing the engineer to concentrate on completing the application without the interruption of separate tools and different modes of operation.

Figure 2-13. The Design Cycle



MPLAB X IDE is a “wrapper” that coordinates all the tools from a single graphical user interface, usually automatically. For instance, once code is written, it can be converted to executable instructions and downloaded into a microcontroller to see how it works. In this process multiple tools are needed: an editor to write the code, a project manager to organize files and settings, a compiler or assembler to convert the source code to machine code and some sort of hardware or software that either connects to a target microcontroller or simulates the operation of a microcontroller.

2.2.3 Project Manager

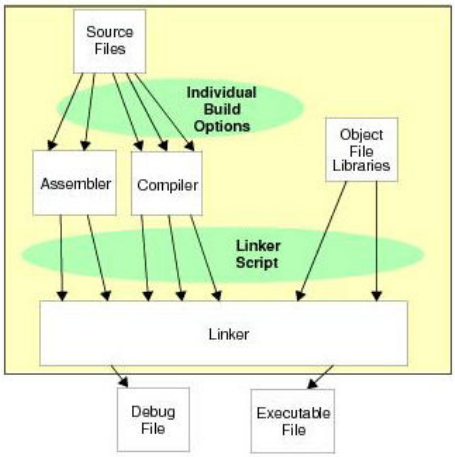
The project manager organizes the files to be edited and other associated files so they can be sent to the language tools for assembly or compilation, and ultimately to a linker.

The linker has the task of placing the object code fragments from the assembler, compiler and libraries into the proper memory areas of the embedded controller, and ensure that the modules function with each other (or are “linked”).

This entire operation from assembly and compilation through the link process is called a project “build.” Properties specified for the language tools can be invoked differently for each file, if desired, and a build process integrates all of the language tools’ operations.

© 2022 Microchip Technology Inc.
and its subsidiaries

Figure 2-14. MPLAB® X IDE Project Manager



The source files are text files that are written conforming to the rules of the assembler or compiler. The assembler and compiler convert them into intermediate modules of machine code and placeholders for references to functions and data storage.

The linker resolves these placeholders and combines all the modules into a file of executable machine code. The linker also produces a debug file which allows MPLAB X IDE to relate the executing machine codes back to the source files.

A text editor is used to write the code. It recognizes the constructs in the text and uses color coding to identify various elements, such as instruction mnemonics, C language constructs and comments. The editor supports operations commonly used in writing source code. After the code is written, the editor works with the other tools to display code execution in the debugger. Breakpoints (which stop or “break” the execution of code) can be set in the editor and the values of variables can be inspected by hovering the mouse pointer over the variable name. Names of variables can be dragged from source text windows and then dropped into a Watches window where their changing values can be watched after each breakpoint or during code execution.

2.2.4 Language Tools

Language tools are programs such as cross-assemblers and cross-compilers. Most people are familiar with some of the language tools that run on a computer, e.g., Visual Basic or C compilers.

When using language tools for embedded systems, a “cross-assembler” or “cross-compiler” is used. These tools differ from typical compilers in that they run on a computer, but they produce code to run on another microprocessor (or microcontroller).

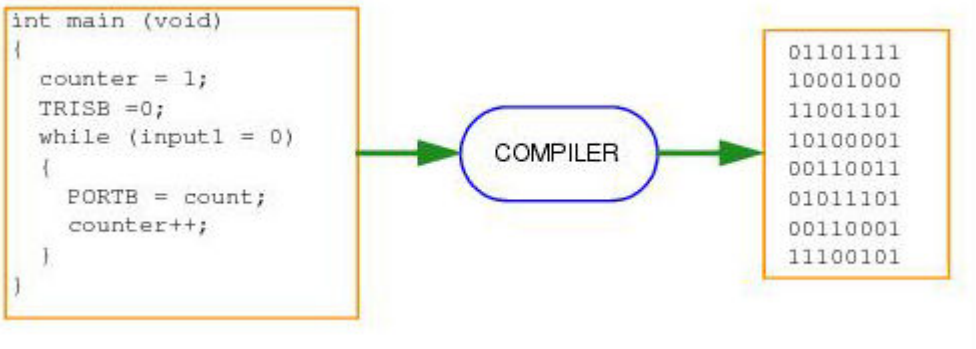
Language tools also produce a debug file that MPLAB X IDE uses to correlate the machine instructions and memory locations with the source code. This bit of integration allows the MPLAB X IDE editor to set breakpoints, allows Watches windows to view variable contents, and lets you single step through the source code, while watching the application execute.

Embedded system language tools also differ somewhat from compilers that run and execute on a computer because they must be very space conscious. The smaller the code produced, the better, because that provides the smallest possible memory usage for the target, which reduces cost. This means that techniques to optimize and enhance the code, using machine-specific knowledge, are desirable.

The size of programs for computers typically extends into the megabytes for moderately complex programs. The size of simple embedded systems programs may be as small as a thousand bytes or less. A medium size embedded system might need 32K or 64K of code for relatively complex functions. Some embedded systems use megabytes of storage for large tables, user text messages or data logging.

© 2022 Microchip Technology Inc. and its subsidiaries

Figure 2-15. A Compiler Converts Source Code Into Machine Instructions



2.2.5 Target Debugging

In an integrated development environment (IDE), the execution of the code is tested on a debugger. The debugger can be a software program that simulates the operation of the microcontroller for testing, or it can be a hardware instrument to analyze the program as it executes in the application.

The IDE and Debugging

Debugging usually becomes urgent near the end of the project design cycle. As deadlines loom, getting the application to function as originally designed is the last step before going into deployment of the product and often has the most influence on producing delays in getting a product out. That's where an integrated development environment is most important. Doing fine "tweaks" to the code, recompiling, downloading and testing all require time. Using all tools within a single environment will reduce the time around the "cycle." These last steps, where critical bugs are worked out, are a test for the embedded systems designer. The right tool can save time. With MPLAB X IDE, many tools can be selected, but they all will have a similar interface and the learning curve from simulator to low-cost in-circuit debugger to powerful in-circuit emulator is small.

Software Debuggers

Simulators are built into MPLAB X IDE so a program can be tested without any additional hardware. A simulator is a software debugger, and the debugger functions for the simulator are almost identical to the hardware debuggers, allowing a new tool to be learned with ease. Usually a simulator runs somewhat slower than an actual microcontroller since the CPU in the computer is being used to simulate the operations of the microcontroller.

Hardware Debuggers

There are two types of hardware that can be used with MPLAB X IDE: programmers and hardware debuggers. A programmer simply burns the machine code from the PC into the internal memory of the target microcontroller. The microcontroller can then be plugged into the application and, hopefully, it will run as designed.

Usually, however, the code does not function exactly as anticipated, and the engineer is tasked with reviewing the code and its operation in the application to determine how to modify the original source code to make it execute as desired. This process is called debugging. As noted previously, the simulator can be used to test how the code will operate, but once a microcontroller is programmed with the firmware, many things outside the scope of the simulator come into play. Using just a programmer, the code could be changed, reprogrammed into the microcontroller and plugged into the target for retest, but this could be a long, laborious cycle if the code is complex, and it is difficult to understand exactly what is going wrong in the hardware.

This is where a hardware debugger is useful. Hardware debuggers can be in-circuit emulators or in-circuit debuggers, which use microcontrollers that have special built-in debugging features. A hardware debugger, like a simulator, allows the engineer to inspect variables at various points in the code, and single step to follow instructions as the hardware interacts with its specialized circuitry.

2.2.6 Device Programming

After the application has been debugged and is running in the development environment, it needs to be tested on its own. A device can be programmed with an in-circuit emulator, an in-circuit debugger, a development programmer, or a device programmer. MPLAB X IDE can be set to the programmer function and the part can be "burned." The target application can now be observed in its nearly final state. Engineering prototype programmers allow quick prototypes

© 2022 Microchip Technology Inc. and its subsidiaries

to be made and evaluated. Some applications can be programmed after the device is soldered on the target PC board. Using In-Circuit Serial Programming™ (ICSP™) programming capability, the firmware can be programmed into the application at the time of manufacture, allowing updated revisions to be programmed into an embedded application later in its life cycle. Devices that support in-circuit debugging can even be plugged back into an in-circuit debugger after manufacturing for quality tests and development of next generation firmware.

Production programming can be accomplished using a production programmer and the MPLAB IPE, which is installed with MPLAB X IDE.

2.3 MPLAB X IDE Support Features

Several helpful support features are available within, and a few outside of, MPLAB X IDE.

2.3.1 Microchip Website

Microchip Technology provides online support of Development Tools via our website at:

http://www.microchip.com/development-tools/

Related documentation such as device datasheets, programming specifications, and product pages can all be found on the website.

2.3.2 MPLAB X IDE Documentation

In addition to the 2.3.1. Microchip Website, documentation and support links can be found in the MPLAB X IDE interface.

2.3.2.1 Help Menu

Help and additional documentation for MPLAB X IDE supported development tools may be found under the 12.1.12. Help Menu.

Help>Help Contents and Help>Tool Help Contents

MPLAB X IDE is built upon the NetBeans platform. Therefore, many of the NetBeans functions are part of MPLAB X IDE. For more on NetBeans Help topics, see also:

NetBeans Developing Applications with NetBeans IDE Release 8.0

MPLAB X IDE, MPLAB X Simulator, MPLAB IPE and newer tools have links to online WebHelp:

onlinedocs.microchip.com

Help for older tools is available under the Help menu.

Help>Release Notes

In addition to tool release notes, this also links you to:

- User's Guides
- Device Support Lists
- Debug Features Support
- Reserved Resources Lists

2.3.2.2 How Do I...

Enter text in the box next to "How do I?" to look up topics in Developer Help.

Microchip Developer Help is a training website that provides information on using development tools, microcontroller products, and exploring functional solutions.

microchipdeveloper.com/

2.3.2.3 Start Page

Links to other documentation, videos, forums, and wikis are featured on the Start Page.

© 2022 Microchip Technology Inc. and its subsidiaries



2.3.3 Microchip Store

Starting with MPLAB X IDE v3.xx, you may access the Microchip store for development tools software and hardware products via the IDE.

- MPLAB X Store tab – next to the Start tab is this tab, which provides links to products.
- *Help>MPLAB X Store* – under the Help menu is a link to open the MPLAB X Store tab.
- MPLAB X Store toolbar icon  – an icon on the MPLAB X Store toolbar opens the MPLAB X Store tab.

2.3.4 Programming Center

Add code to your device or order free samples pre-programmed using the Microchip Programming Center. Click the icon on the toolbar to order this value-added service  .

2.3.5 MPLAB X IDE Updates

MPLAB X IDE is an evolving program with thousands of users. Microchip Technology is continually designing new microcontrollers with new features. Many new MPLAB X IDE features come from customer requests and from internal usage. Continued new designs and the release of new microcontrollers ensure that MPLAB X IDE will continue to evolve.

MPLAB X IDE is updated as needed to provide support for new device features. Updates for new device and new tool support is now provided in releases of Device Family Packs (DFPs) and Tool Packs (TPs) respectively. Unless new device or tool support requires a feature update, the existing version of the IDE can be updated using the MPLAB Pack Manager.

For projects that are midway through development when a new version of a pack or MPLAB X IDE is released, it is considered “best practice” to not update to the new release unless there is a compelling reason to do so, such as a bug fix on a bug that inhibits the current efforts. The start of a new project is the best time to update to a new release.

The MPLAB X IDE User's Guide PDF may not be updated for every release, so the WebHelp version is the best source of up-to-date information about the UI.

To be notified of updates to MPLAB X IDE and its components, subscribe to the Development Tools section of myMICROCHIP Personalized Notification Service on:

http://www.microchip.com/pcn

2.3.6 Working Outside the IDE

MPLAB X IDE is designed to help you write, debug and release applications for embedded systems. However, your company may have requirements that make code development outside the IDE necessary.

To find out how you can build your code outside MPLAB X IDE, see:

microchipdeveloper.com/mplabx:work-outside

© 2022 Microchip Technology Inc.
and its subsidiaries

3. Before You Begin

Before you use MPLAB X IDE, read the following topics to ensure proper setup.

3.1 Review Installation Requirements

Installation requirements, as well as other installation issues, are specified in the “Release Notes for MPLAB X IDE.”

To view this file before installation, go to the MPLAB X IDE webpage and download “MPLAB X IDE Release Notes”.

To view this file after installation, either:

- Go to the `docs` folder of the MPLAB X IDE install.
- Launch the IDE and select *Help>Release Notes*.

3.2 JRE and MPLAB X IDE

Note: OpenJDK now being used for MPLAB X IDE 5.40 and later.

A license-free version of Java will be distributed with MPLAB X IDE starting in version 5.40. MPLAB X IDE versions up to and including 5.35 will use the last free JRE version 8 from Oracle.

Note: No Oracle® JRE license will be required to use any version of MPLAB X IDE.

3.3 Install the USB Device Drivers (for Hardware Tools)

The MPLAB X IDE installer contains a preinstaller that will place USB device drivers for supported hardware tools onto your system. After MPLAB X IDE installation, when a hardware tool is plugged into the computer, the operating system (OS) should associate the driver with the tool (i.e., finish the driver installation). However, additional steps may be needed for correct association, for some OSs and tools.

USB Driver Installation for macOS® or Linux® OS

If you are planning to install MPLAB X IDE on a Mac or Linux computer, no additional steps are needed.

USB Driver Installation for Windows® OS

If you are planning to install MPLAB X IDE on a personal computer that uses the Windows operating system, follow the instructions below.

Note: The USB hardware tool drivers for MPLAB X IDE are not the same as those for MPLAB IDE v8.

The list below shows which tools are affected:

- MPLAB® ICE 4, MPLAB ICD 4 or MPLAB PICkit™ 4
- MPLAB REAL ICE™ in-circuit emulator or MPLAB ICD 3 in-circuit debugger
- MPLAB PM3 device programmer
- PIC32 Starter Kit

The list below shows which tools are NOT affected:

- PICkit 2 or PICkit 3
- Other MPLAB Starter Kits

Complete the instructions in the following sections to determine your installation method.

3.3.1 Before You Install MPLAB X IDE

Be aware that if your Windows OS version of WinUSB drivers is older than the IDE preinstaller version, the system drivers will be replaced. If you want to keep your version of WinUSB drivers, rename these files before installing MPLAB X IDE.

The 64-bit WinUSB driver files are located at the following locations:

`C:\Windows\SysWOW64\WinUSB.dll` **(32-bit)**

`C:\Windows\system32\WinUSB.dll` **(64-bit)**

`C:\Windows\system32\drivers\WinUSB.sys` **(64-bit)**

3.3.2 If MPLAB IDE v8.xx is NOT Installed on Your System

You do not need to do anything; the USB drivers will be preinstalled when MPLAB X IDE is installed. Once you plug your tool into a computer USB port, a "New Hardware Found" notification should appear. Then, either the install will proceed automatically or you will have to follow a wizard and choose to "Automatically select driver." However, if either procedure fails to install the drivers (associate the preinstalled driver with your tool), you will need to install/associate the drivers manually. For instructions and driver locations, see 3.3.3.4. If You Need to Manually Switch the Drivers.

3.3.3 If MPLAB IDE v8.xx is Installed on Your System

If MPLAB IDE v8.xx or earlier is already installed on your computer, you may need to run the MPLAB Device Driver Switcher to switch from MPLAB IDE v8 drivers to MPLAB X IDE drivers.

To determine which drivers are associated with the your tool:

1. Plug your desired tool into the USB connector on your personal computer.
2. Open the computer's Device Manager window.
   a. For MPLAB IDE v8.xx, the driver should be under "Custom USB Device>Microchip Custom USB Device."
   b. For MPLAB X IDE, the driver should be under "Microchip Tools>Microchip WinUSB Device."
3. If you have an MPLAB IDE v8.xx driver, you will need to use the Switcher to switch to the MPLAB X IDE drivers.

To switch to MPLAB X IDE preinstalled drivers:

1. If You Have Windows XP 64, Manually Switch
2. If You Have Windows 7 or 8, Use Administrator Mode
3. Switch USB Device Drivers (for Hardware Tools)

If the Switcher fails to switch the drivers, see:

1. If You Need to Manually Switch the Drivers
2. Tool Communication Issues

3.3.3.1 If You Have Windows XP 64, Manually Switch

Note: Microsoft Windows XP Professional SP3 is no longer supported. However, MPLAB X IDE may continue to function on this operating system.

If you are using Windows XP 64-bit OS on your computer, you will not be able to use the MPLAB Device Driver Switcher to switch from MPLAB IDE v8 drivers to MPLAB X IDE drivers; you will have to switch the drivers manually. See 3.3.3.4. If You Need to Manually Switch the Drivers.

3.3.3.2 If You Have Windows 7, 8 or 10, Use Administrator Mode

To run the MPLAB Device Driver Switcher, you must be in Administrator mode.

To run the Switcher utility as administrator, right click on the executable and select 'Run as Administrator.'

The executable is located at:

`C:\Program Files\Microchip\MPLABX\vx.xx\Switcher\64Bit\MPDDSwitch.exe`

It is recommended that you use the Switcher GUI utility first to switch the drivers. If this is problematic, you may switch the drivers using command-line applications.

To run the command-line application – `mchpdds64.exe` – as administrator, first open the command prompt as administrator: *Start>All Programs>Accessories>Command Prompt*, right click and select 'Run as Administrator'. This will open up the Administrator: Command Prompt. After this, the instructions provided in the `ReadMe64.txt` file may be followed to complete the driver switching.

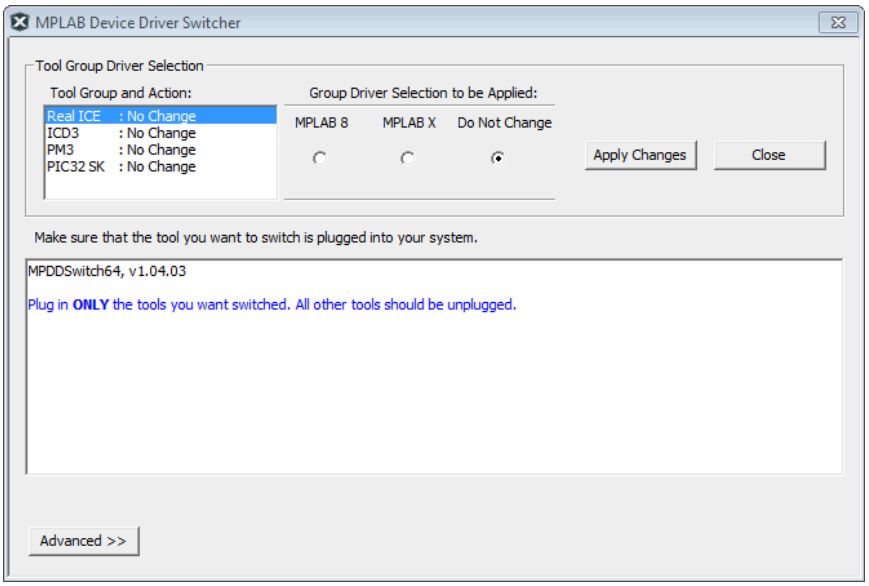
3.3.3.3 Switch USB Device Drivers (for Hardware Tools)

The MPLAB Device Driver Switcher is a GUI application that enables you to switch between MPLAB IDE v8 device drivers and MPLAB X IDE device drivers. Click on the desktop icon to launch the Switcher utility.

Figure 3-1. MPLAB Driver Switcher Icon



Figure 3-2. Switcher Utility



Switcher Operation

To switch USB device drivers:

1. Click to select the connected tool for which you wish to switch drivers under “Tool Group and Action.”
   Note: If the tool(s) are not connected when Switcher is run, the driver(s) will not be installed or switched for those particular tools.
2. Click the radio button for either “MPLAB 8” or “MPLAB X.”
3. Click the **Apply Changes** button. Switcher progress will be shown in the large text window. This may take some time.
4. Once the program/batch completes, view the name(s) of the driver(s) in the Device Manager window. For MPLAB X IDE, the driver(s) should be under “Microchip Tools>Microchip WinUSB Device.” For MPLAB IDE v8.xx, the driver(s) should be under “Custom USB Device>Microchip Custom USB Device.”

Switcher Troubleshooting

1. If the GUI fails to switch the drivers, check the paths to the driver files by clicking the **Advanced** button. Run the Switcher again.
2. If the GUI still fails to switch the drivers, you will need to switch the drivers manually. For instructions and driver locations, see 3.3.3.4. If You Need to Manually Switch the Drivers.

Switcher Executable Location

You can find the Switcher executable file in the MPLAB X IDE install folder, Switcher subfolder, kept in the following location (by default):

`C:\Program Files\Microchip\MPLABX\vx.xx\Switcher`

where `vx.xx` is the MPLAB X IDE version.

3.3.3.4 If You Need to Manually Switch the Drivers

If you need to switch or associate the USB device drivers manually:

1. Open the Device Manager (under Control Panel). Look under “Microchip Tools” for your tool or, if you cannot find it there, under “Other Devices” for “Unknown Device.”
2. Right click on your tool name or “Unknown Device” and select “Update Driver Software.”
3. In the Update Driver Software dialog, select “Browse my computer for driver software.”
   Note: DO NOT select “Search automatically for updated driver software.” This will associate a Windows default driver with your tool, which will probably not work. If you accidentally select this, back out or exit and repeat these steps to switch to the correct driver.
4. Locate the correct device driver for your system.
   For MPLAB X IDE, the default locations for the device drivers are:

   ```
   C:\Program Files\Microchip\MPLABX\vx.xx\Switcher\64Bit\
   winusb\amd64\MCHPWinUSBDevice.inf
   ```

   where `vx.xx` is the version number of MPLAB X IDE.
5. If a Windows Security dialog pops up, select “Install this driver software anyway” to proceed to switch to the selected drivers.

3.3.3.5 Tool Communication Issues

1. If you are using a docking station or hub and have issues after plugging in the tool, you may need to plug the tool directly into a USB port on your computer.
2. If you need to switch to a device driver manually, you will need to point to the `INF` file in the `32Bit` or `64Bit` folder. For details, see 3.3.3.4. If You Need to Manually Switch the Drivers.

3.4 Connect to a Target (for Hardware Tools)

For in-circuit debuggers and emulators, refer to the following to determine how to connect your hardware tool to a target:

- Development Tools Design Advisory
- If you are using a tool that supports debug headers: PEPs and Debug Headers or EEPs and Emulation Headers
- Debug Tool documentation

For dedicated programmers, refer to your tool documentation for connection information.

If you are using a Microchip demonstration board, evaluation kit or reference design as your target, please refer to the accompanying documentation for set up information.

3.5 Install the Language Tools

When you install MPLAB X IDE v5.40 and later, no language tools are installed. You will need to select, download and install a toolchain.

The following C compiler toolchains (compiler, assembler, linker, etc.) can be used with MPLAB X IDE. To select a compiler toolchain, consider which device you wish to use and then choose a toolchain that supports that device.

- MPLAB XC C Compilers
- AVR and Arm C Compilers
- AVRASM2 - installed with Atmel Studio

MPLAB XC compilers can be licensed as Free or PRO (full-featured, code-optimized) compilers. To install and license these compilers, view the document *Installing and Licensing MPLAB XC C Compilers* (DS50002059). MPLAB X IDE includes the option to license your installed compiler and roam in and out for network licenses. See 13.10. Licenses Windows.

AVR and Arm GNU C compilers (GCCs) are always free. Follow the installer screens to install on you computer.

Installation Path

Ensure these compilers are installed where MPLAB X IDE can search for them, e.g., where MPLAB XC C compilers are installed under `C:\Program Files\Microchip`. Otherwise you will have to specify in MPLAB X IDE where to find them (see 5.7. Set Language Tool Locations).

Installation Media - Linux OS

Expanding a compressed file (tar, tar.gz) onto a file system that does not honor symbolic links or does keep executable permissions intact may create an uncompressed directory structure that cannot be used as a compiler or recognized by MPLAB X IDE. This situation can happen when you expand a compressed file onto a removable USB stick (FAT file system).

3.6 Install MPLAB X IDE

Note: Operating system support for Windows, macOS and Linux will move to 64-bit only starting with MPLAB X IDE version 5.40.

Note: Product support releases will be delivered independently from MPLAB X IDE starting in version 5.30 by moving to a Device File Pack (DFP) system. A DFP can be applied to MPLAB X IDE version 5.30 or 5.35 to extend the life of the IDE for 32-bit operating systems. Not all new devices added by a Device File Pack are guaranteed to work in these versions of the IDE.

Download MPLAB X IDE from the MPLAB X IDE webpage and launch the installer.

Note: The following steps reflect the installation wizard at the time this was written.

1. **Welcome** - The version about to be installed is shown.
2. **License Agreement** - Accept the agreement to continue or reject to end the installation.
3. **Installation Options** - The default installation directory is shown but may be changed. “Use System Proxy Settings” is selected by default to allow access to the internet. You may select “No Proxy” for secure environments or if MPLAB X IDE hangs after launch (see “Known Issues” in the release notes for MPLAB X IDE). You may choose data sharing items to (1) allow microchip to collect usage information and/or (2) allow MPLAB X IDE to share data with MyMicrochip. The second item will enable Product Change Notices.
4. **Select Applications** - Select the applications you want to install. The MPLAB X IDE installer may be used to install MPLAB X IDE and/or MPLAB IPE. Then select all or some device support. Targeted device support results in a smaller install.
5. **Ready to Install** - Click **Next** to install MPLAB X IDE.
6. **Installing** - The status bar lets you see installation progress.
7. **Completing the MPLAB X IDE Setup** - Check to open web pages about MPLAB XC C compilers, MPLAB Harmony framework (for 32-bit devices), and MPLAB Code Configurator for 8-, 16- and 32-bit devices.

3.7 Launch the IDE and View the Desktop

Double click on the MPLAB X IDE icon to launch the program.

Figure 3-3. MPLAB X IDE Icon



MPLAB X IDE is built upon the NetBeans platform. If you are familiar with the NetBeans IDE, then the MPLAB X IDE desktop will look familiar. However, the tabs listed below are specific to MPLAB X IDE.

- Start Page
- MPLAB X Store

On the Start Page, there are 3 tabs with links. If you don’t want the Start Page displayed on start-up, uncheck “Show on Startup” beneath any tab.

Figure 3-4. MPLAB X IDE Desktop



3.8 Shop the MPLAB X Store

Click on the MPLAB X Store tab to purchase tools from the Microchip store. The tab is formatted to showcase some of the latest items for sale pertaining to MPLAB X IDE. You can also browse categories to find other items.

This tab may be accessed directly from the Help menu or from a toolbar icon.

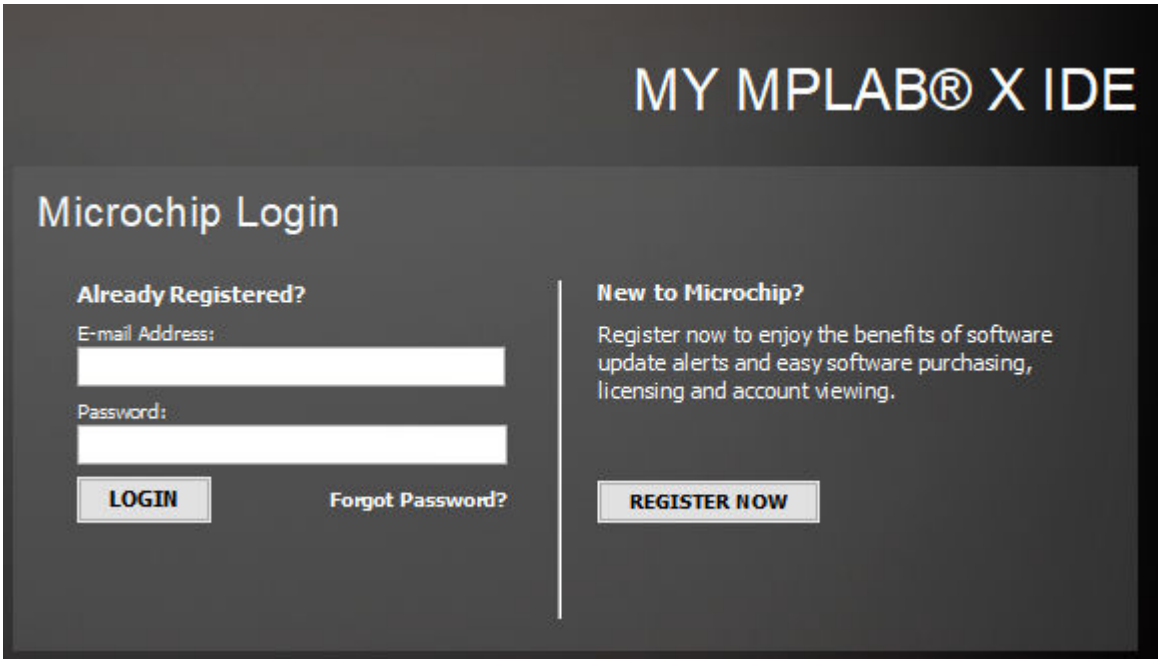
Figure 3-5. MPLAB X Store



A “Purchasing & Client Services” login is available from the Start Page, My MPLAB X IDE. You will then be able to:

- See alerts on software (like HPA expiring)
- Access to your mySoftware account, where you can register and download licenses and renew HPA

Figure 3-6. Start Page Login



3.9 Launch Multiple Instances of the IDE

Some set up is required before using hardware tools (MPLAB PICkit 4, etc.) with an instance of MPLAB X IDE. After any hardware tool setup, an instance of the IDE may be invoked from its own directory.

3.9.1 Setting Up Hardware Tools to Work with Multiple Instances

By default, you can work with up to five (5) instances of the IDE. If you want to have more instances, you will need to modify the “mchpdefport” file manually.

Use and Format of “mchpdefport” File

The “mchpdefport” file provides the information necessary for tool hot-plug use to both the IDE and to the low-level USB library (DLL, so, or dylib file). The format for this file is as follows:

```
localhost
30000
30002
30004
30006
30008
```

The first line indicates the host name on which the IDE is running.

The other lines represent port or socket numbers through which the low-level library communicates with the upper-level IDE. Each instance of the IDE will be assigned to a different port or socket. All communications between the instances of the IDE should be hidden from the user.

For up to five (5) instances of MPLAB X IDE, you do not need to alter this file. If you want more than five instances, you can edit the file to add more port or socket numbers.

Location of the “mchpdefport” File

Within a default installation of the IDE, the “mchpdefport” file can be found in the following place, depending on the operating system:

Operating System	Location

Windows OS	`C:\Windows\system32`
	and
	`C:\Windows\SysWOW64`
	Note: Both occurrences of “mchpdefport” must be modified.
	Note: If you are on a 64-bit system and use a 32-bit editor to edit “mchpdefport” in `C:\Windows\system32`, you will actually be modifying the “mchpdefport” file in `SysWOW64`. You must use a 64-bit editor to edit the file in `system32`.

Linux OS	`/etc/.mplab_ide`
macOSX	`/etc/.mplab_ide`

3.9.2 Invoking Instances of the IDE

MPLAB X IDE requires that each instance has its own user directory. Therefore, preferences that are set or plugins that are added to one instance will not be reflected in another instance.

In order to invoke multiple instances, launch the IDE with the `--userdir` option and specify a directory.

Window OS

Create a shortcut with the `--userdir` option. For example, on Windows 7 OS:

1. Right click on the desktop and select *New>Shortcut*.
2. Browse to the installed (default) MPLAB X IDE executable:
   `C:\Program Files\Microchip\MPLABX\vx.xx\mplab_platform\bin\mplab_ide.exe` where `vx.xx` is the MPLAB X IDE version.
3. At the end of that line, type the location of the user directory. You can either place your directory under the default location for this type of MPLAB X IDE information:
   `--userdir "C:\Users\MyFiles\AppData\Roaming\.mplab_ide\dev\anydir"`
   or you can place it where ever you like:
   `--userdir anydir`
4. Click OK.

Linux OS

The installed version, initiated without any parameters (clicking on the desktop icon), will run with a user directory of `$(HOME)/.mplab_ide`. To change the user directory, run the `$InstallationDir/mplab_platform/bin/mplab_ide` shell script, passing the argument `--userid anydir`. For example, to run MPLAB X IDE in two different instances:

```
$ /opt/microchip/mplabx/vx.xx/mplab_platform/bin/mplab_ide --userdir ~/.anydir1 &
$ /opt/microchip/mplabx/vx.xx/mplab_platform/bin/mplab_ide --userdir ~/.anydir2 &
```

where `vx.xx` is the MPLAB X IDE version.

You can create desktop icons that have the user ID embedded, too.

macOS

Open a Shell window and type the command line listed below to execute your installation of MPLAB X IDE in an alternate user directory. You can either place your directory under the default location for this type of MPLAB X IDE information:

```
$/bin/sh /Applications/microchip/mplabx/vx.xx/mplab_ide.app/Contents/
Resources/mplab_platform/bin/mplab_ide --userdir "${HOME}/Library/
Application Support/mplab_ide/dev/anydir"
```

or you can place it wherever you like:

```
$/bin/sh /Applications/microchip/mplabx/vx.xx/mplab_ide.app/Contents/
Resources/mplab_platform/bin/mplab_ide --userdir anydir
```

3.10 Launch Multiple Versions of the IDE

Before the release of MPLAB X IDE v3.00, you could have different versions of the IDE on your PC by installing to different directories. Beginning with MPLAB X IDE v3.00, this is done by default, for example:

```
C:\Program Files\Microchip\MPLABX\v3.00
```

You can launch and run different versions at the same time. Each version has its own user directory.

You can launch a version of MPLAB X IDE and a version of MPLAB IDE (v8 or earlier) on the same PC, but keep in mind of the USB hardware driver limitations mentioned in “**Install the USB Device Drivers (for Hardware Tools)**.”

3.11 Launch using Startup Parameters

Many command-line options are from the NetBeans platform. See the “Release Notes for MPLAB X IDE” to view a table of options.

In addition, a list of MPLAB® X IDE options may be found by using the `--help` option. For example, on a 64-bit Windows® system:

`Program Files\Microchip\MPLABX\v`*`x.xx`*`\mplab_platform\bin\mplab_ide64 --help`

4. Tutorial

This tutorial provides a guided example of working with an MPLAB X IDE project.

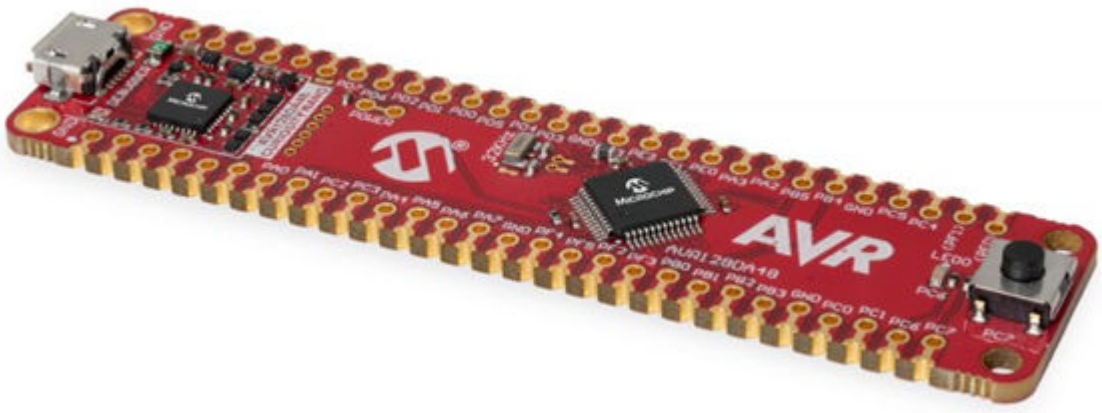
The example project will use a switch to control the LED flashing rate. After downloading and opening the project in MPLAB X IDE, how to set/change project properties and run the project is explained. Debug features such as setting breakpoints, stepping through code, viewing variable values, and viewing/changing register and memory values are demonstrated.

4.1 Selecting a Device and Development Board

The first step is to decide on a device for your application project. Then determine what hardware development tool(s) you will need to debug code on that device. For this tutorial, the following will be used:

- Device: AVR128DA48 MCU
- Hardware Tool: AVR128DA48 Curiosity Nano Evaluation Kit

Figure 4-1. AVR128DA48 Curiosity Nano



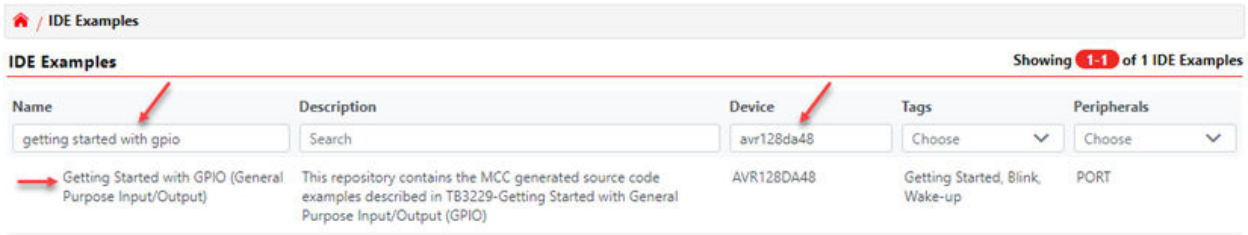
4.2 Downloading the Example Code

Once a device and development tool have been selected, example code can be found at MPLAB Discover.

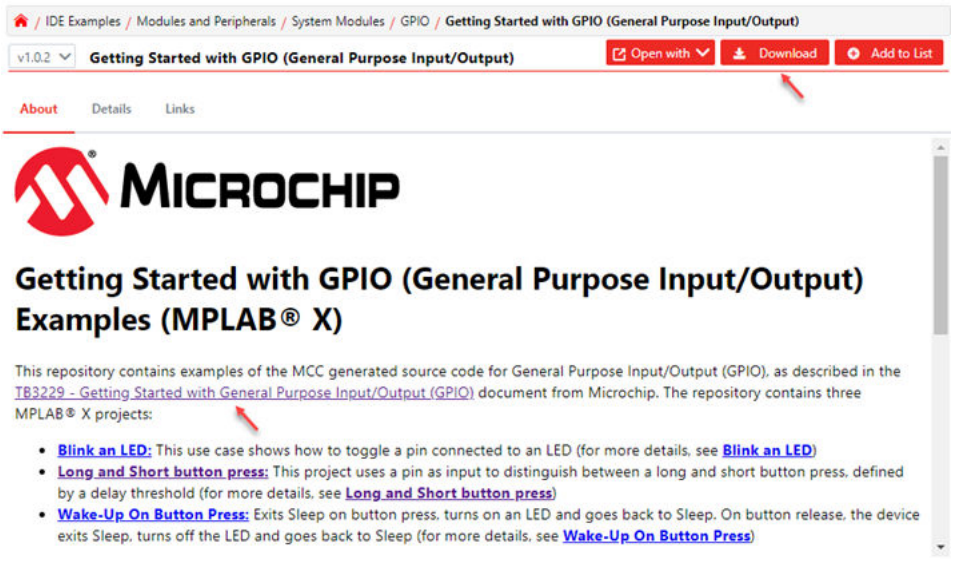
Filter for examples:

- Under **Name**, enter “getting started with gpio.”
- Under **Device**, enter “avr128da48.”

For this tutorial, select “Getting Started with GPIO (General Purpose Input/Output)” by clicking on the name (see figure below).



The page that opens contains information about the example code (see figure below). There is a link to a Technical Brief (TB3229) as well. To download all example code, click the **Download** button in the top right. A zip file containing the code will download to your computer. Unzip the files into your `MPLABXProjects` folder.

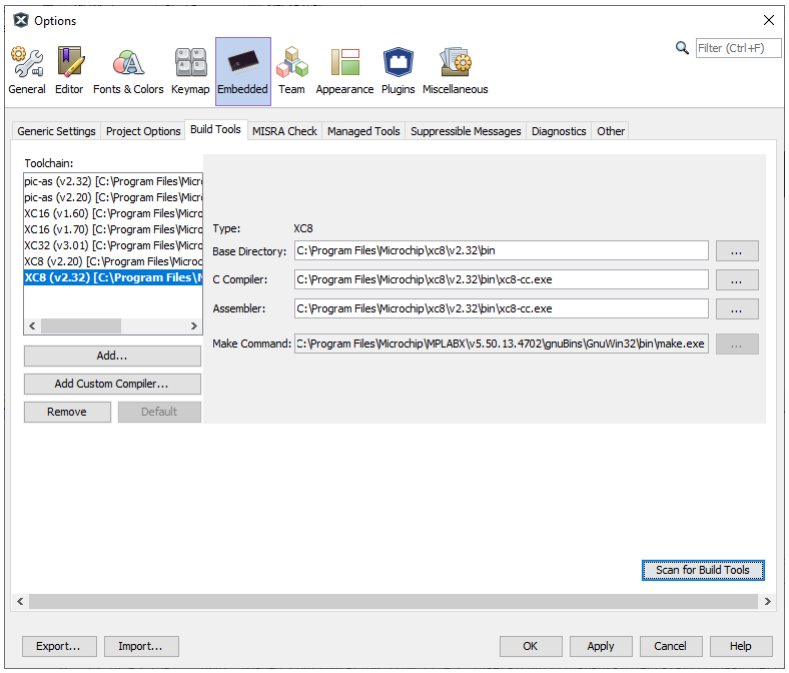


4.3 Installing and Setting Up the Software

To run and debug the downloaded example code, MPLAB® X IDE and an MPLAB XC C compiler will need to be downloaded and installed. Refer to section 3. Before You Begin for installation details.

- Install MPLAB X IDE. Download the free IDE at www.microchip.com/mplabx.
- Install MPLAB XC8 C compiler. Download a free MPLAB XC compiler at www.microchip.com/xc.

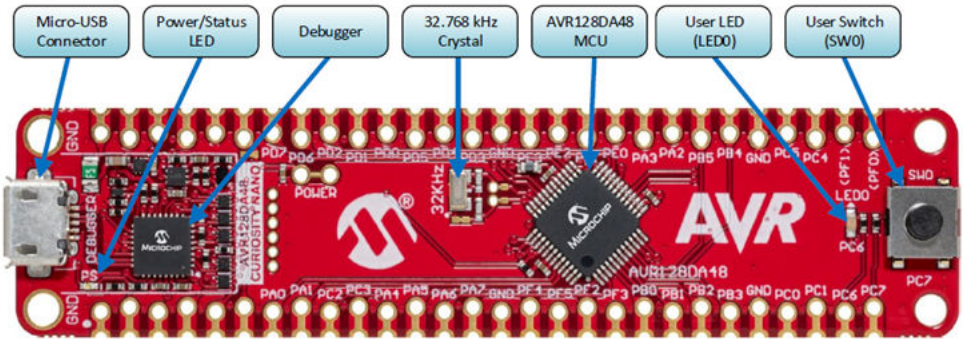
Launch MPLAB X IDE. Ensure that the compiler is shown in the IDE window *Tools>Options>Embedded>Build Tools*. If not, click the **Add** button to find the compiler install location manually.



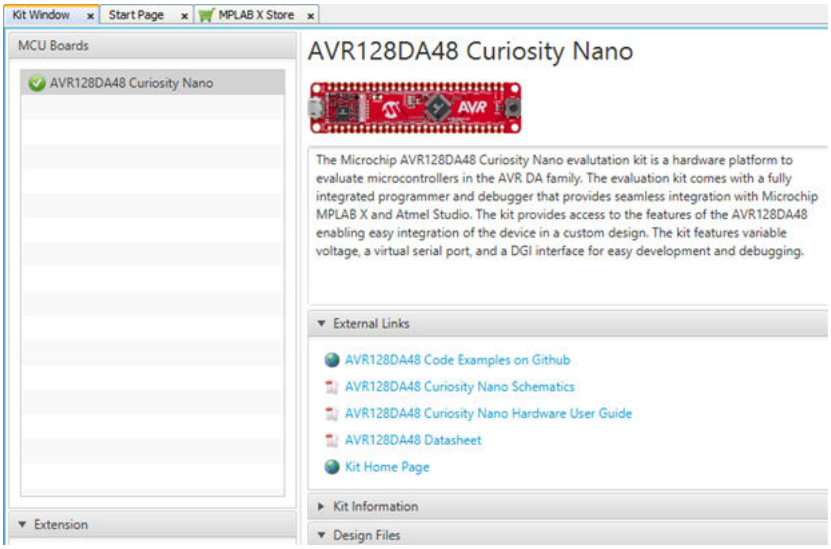
4.4 Connecting the Hardware

For a supported USB cable, plug the micro USB connector into the evaluation board (see figure below) and plug the USB connector into your computer. Drivers will install automatically.

The USB cable will provide both power and communication. Once the board is powered, the green status LED will be lit.



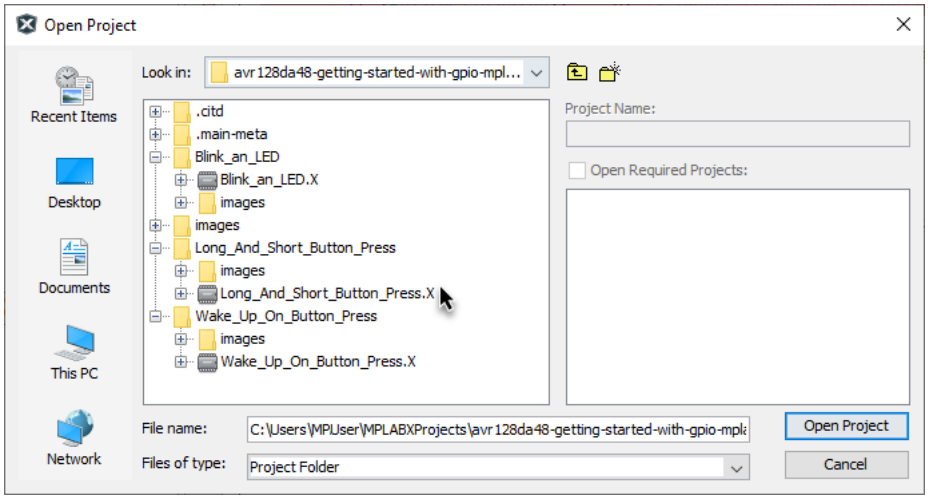
MPLAB® X IDE will auto-detect that the evaluation board is connected. The Kit Window in MPLAB X IDE will present relevant information like data sheets and board documentation (see figure below). The AVR128DA48 device on the AVR128DA48 Curiosity Nano board is programmed and debugged by the on-board debugger and, therefore, no external programmer or debugger tool is required.



4.5 Opening an Example Project in MPLAB X IDE

In MPLAB X IDE, click the “Open Project” icon  . This should open the viewer to the `MPLABXProjects` folder. Search for the downloaded example code and select the project `Long_And_Short_Button_Press.X`. Then click **Open Project**.

Figure 4-2. Open Project



View the Projects and Files Windows

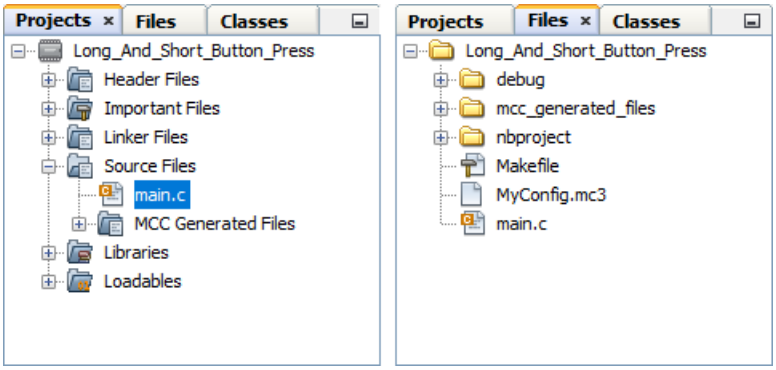
The Projects window displays the project tree with files grouped by category. For more information on these categories, see 6.5.1. Projects Window View.

Click on the Files window to see a file manager view of project files. See 6.5.2. Files Window View.

If you double click on a file name in either window, the related file will open in an Editor window. To close the tab, click on the “x” next to the file name.

Right click on the project name in the Projects window to view the pop-up (context) menu. You can do the same on the project’s subfolders and window background.

Figure 4-3. Projects and Files Windows

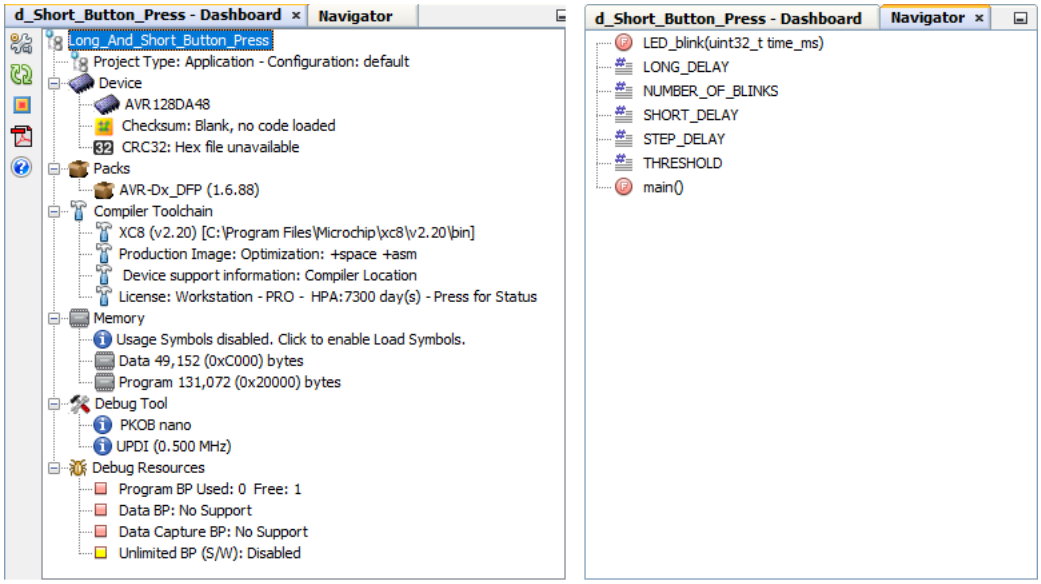


View the Dashboard and Navigator Windows

The Dashboard window shows details about the project. For more information, see 13.7. Dashboard Window.

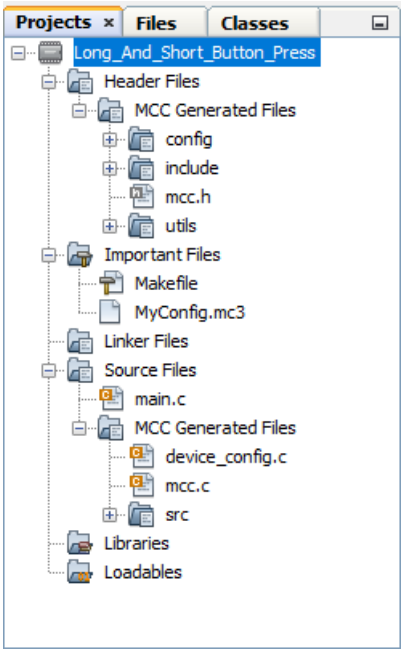
The Navigator window provides a compact view of the file that is currently selected and simplifies navigation between different parts of the file.

Figure 4-4. Dashboard Window on Project Open



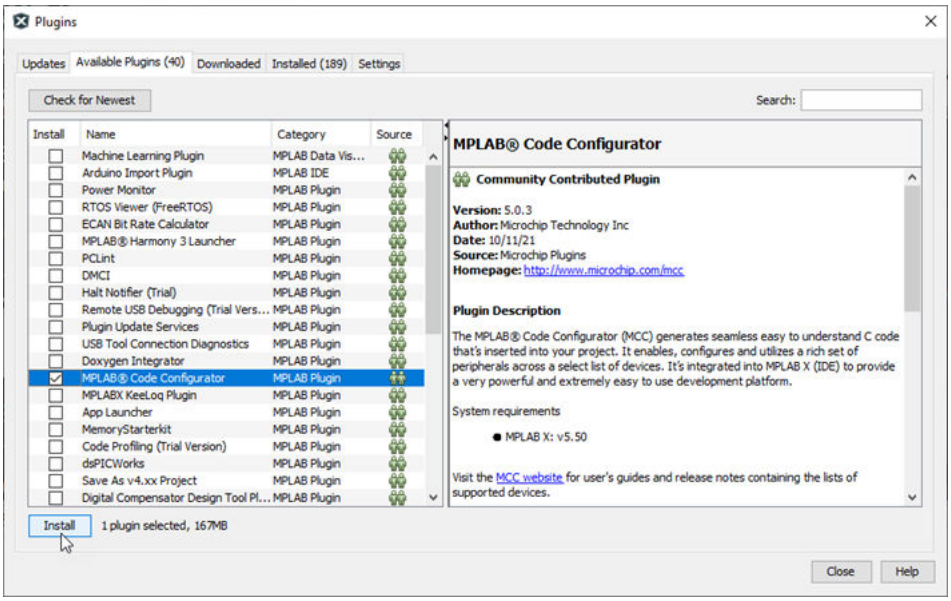
4.6 Examining MCC Generated Files

Expand the project tree to see project files in `MCC Generated Files` folders. This means that the project was generated using the MPLAB Code Configurator, or MCC.

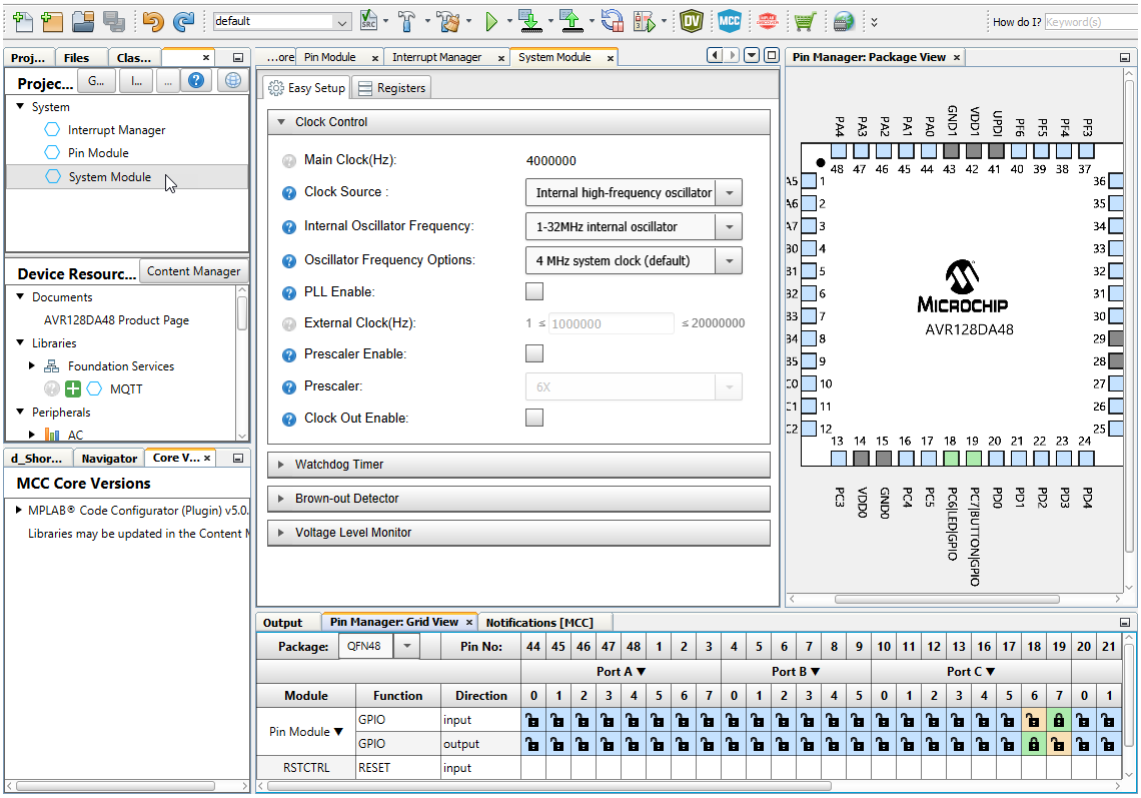


To see the MCC configuration that generated these files, you can install the MCC plugin by going to *Tools>Plugins>Available Plugins*, checking “MPLAB Code Configurator” and clicking **Install**.

Note: You will need to restart.

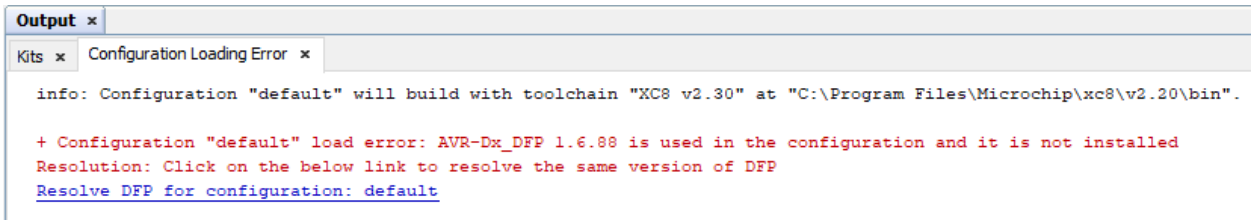


Click the MCC icon  that is now on the toolbar to launch MCC. The configuration information shown is from the project `MyConfig.m3` file. To close MCC, click the icon again.



4.7 Setting Project Properties

When the project was opened, you may have noticed information in the Output window about compiler and device family pack (DFP) versions (see figure below). You can update these versions in the Project Properties window.

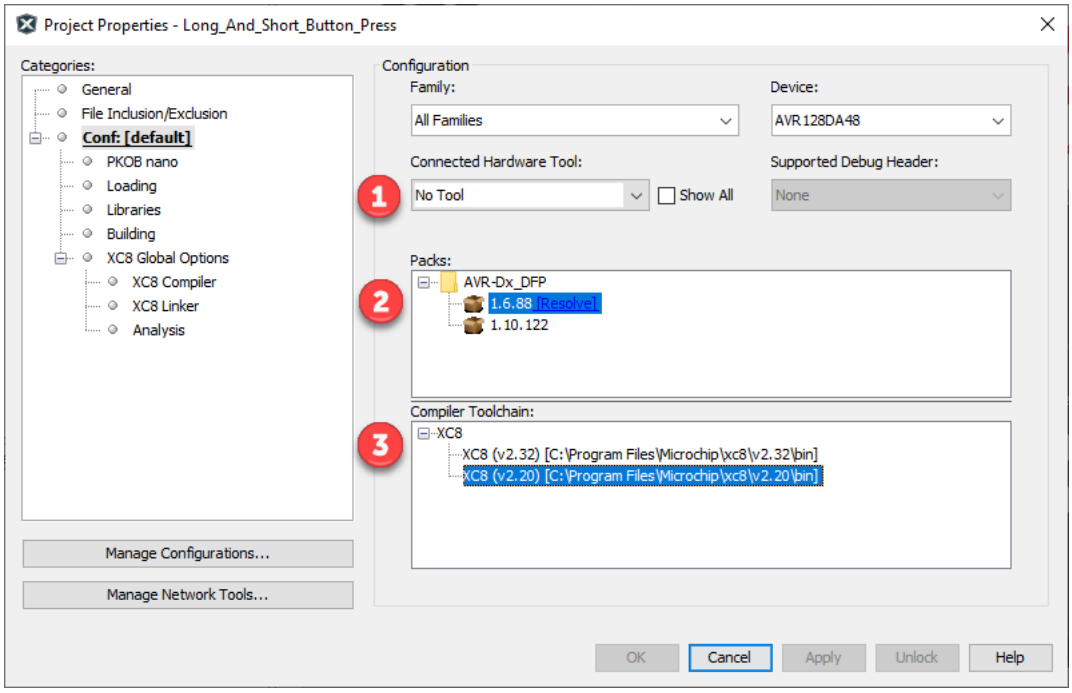


In the Projects window, right click on the project name to drop down a menu and select “Properties.” When the Project Properties window opens you may see something similar to the figure below, depending on your release of MPLAB X IDE, compiler, and DFP.

The following items need to be addressed:

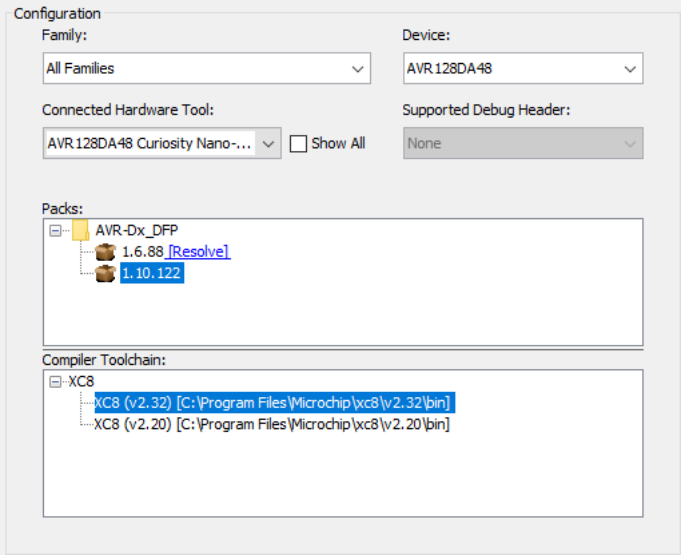
1. Select the AVR128DA43 Curiosity Nano from the “Connected Hardware Tool” drop-down list.
2. If the version of the DFP used to develop this example is not installed in your MPLAB X IDE, you may install it (click on Resolve) or choose to use a newer version of the DFP.
3. If you have a newer version of the MPLAB XC8 installed on your system, you may choose to use that version. If you don’t have the version used to develop the example (2.20), you can either download it or use a newer version.

Figure 4-5. Project Properties On Open



The project setting used in the example is shown in the figure below.

Figure 4-6. Project Properties Updates



4.8 Running the Code

Click on the Make and Program Device icon  to make, program and run the project code. Progress will be visible in the Output window (see figures below). Once complete, quickly click the button on the evaluation board to see the LED flash quickly, then press and hold the button before releasing to see the LED flash slowly.

Figure 4-7. Output Window Status - Build, Load,

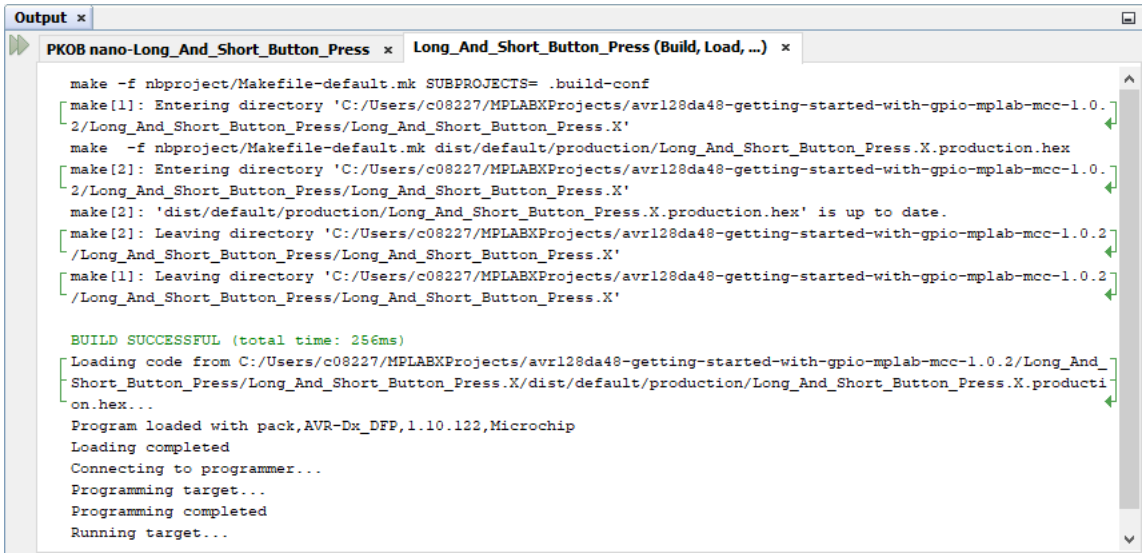
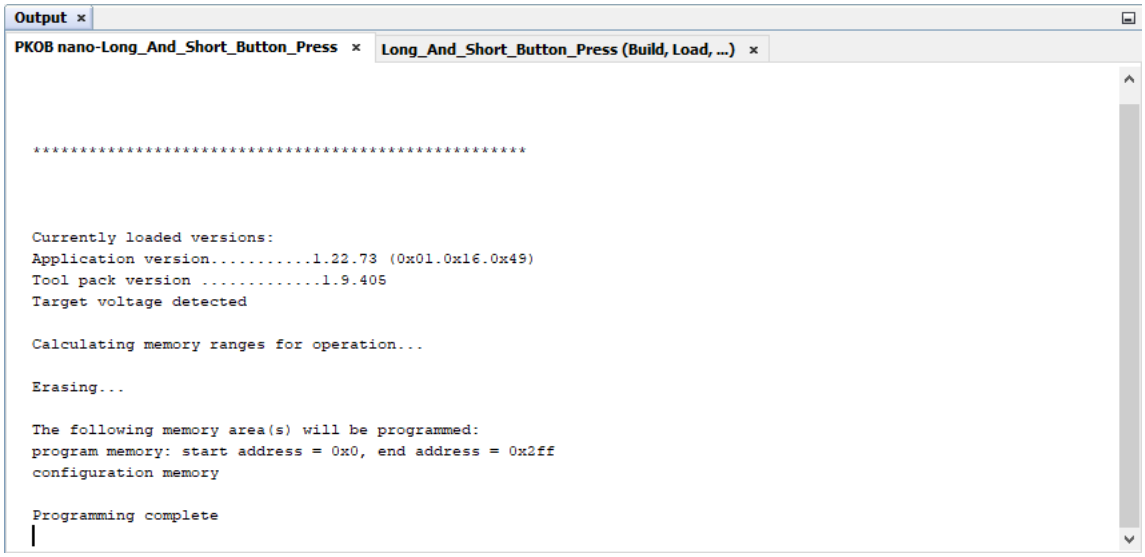
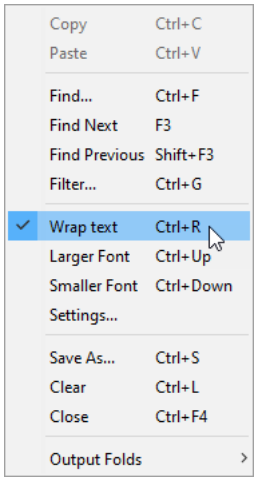


Figure 4-8. Output Window Status - Program Device



To wrap text in the Output window, right click in the window and select “Wrap text.”

Figure 4-9. Context Menu



4.9 Debugging the Code

For this tutorial, the code used has been tested and runs successfully. However, your own code may need to be debugged as you develop your application.

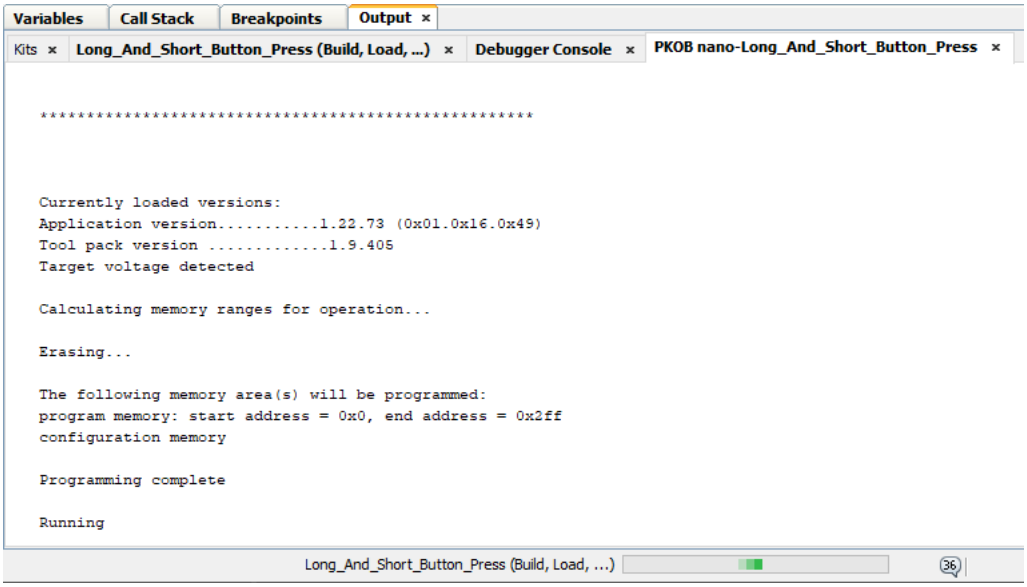
Click on the “Debug Project” icon  to begin a debug session. Debug progress will be visible in the Output window (see figure).

To halt your application code		Pause
To run your code again		Continue
To end execution of your code		Finish Debugger Session

For more on debugging C/C++ code projects, see also NetBeans Help:

Debugging C/C++ Projects Tutorial

Figure 4-10. Ouptut Window - Code Running in Debug Mode



4.10 Setting Breakpoints

When debugging, it can be useful to suspend execution at a specific location in code so that variable values can be examined. To do this, use breakpoints.

For more information on breakpoints, see also NetBeans Help:

Debugging C/C++ Projects Tutorial>Creating Breakpoints

4.10.1 Set Breakpoint

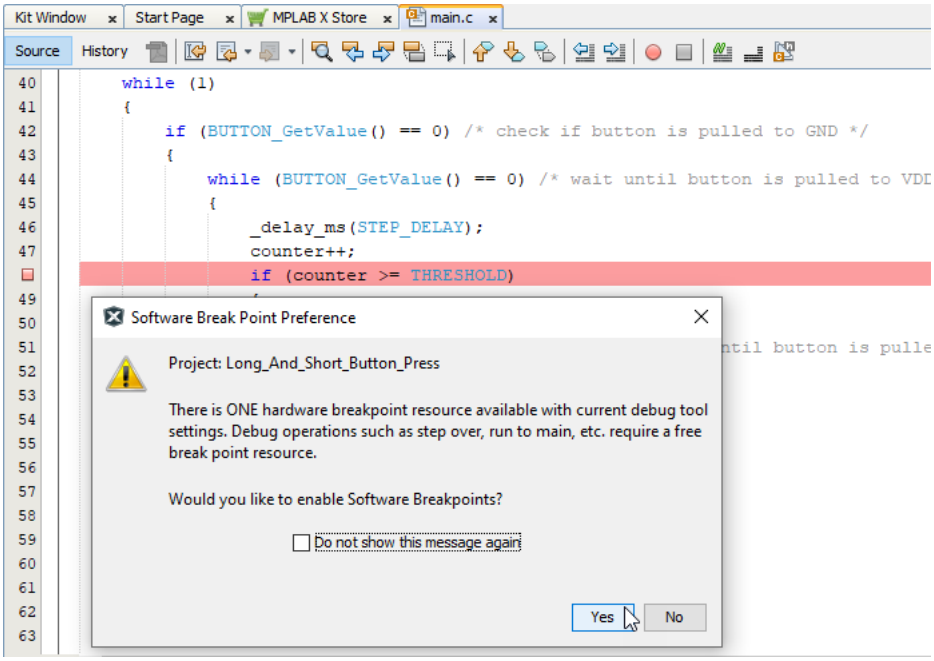
If your code is running, end execution by clicking the “Finish Debugger Session” icon  .

In the Projects window, under “Source Files,” double click on the file `main.c` to open it in an Editor window. Set a breakpoint on the line in the figure by clicking in the left margin of the line.

For the device used in this tutorial, there is only one hardware breakpoint but software breakpoints are available. A notebox will pop up asking about your choice of breakpoint type. Since stepping through code in the next section requires another breakpoint, click “Yes” to use software breakpoints.

For more on breakpoint type selection, see 5.14.1. Choose Hardware or Software Breakpoints.

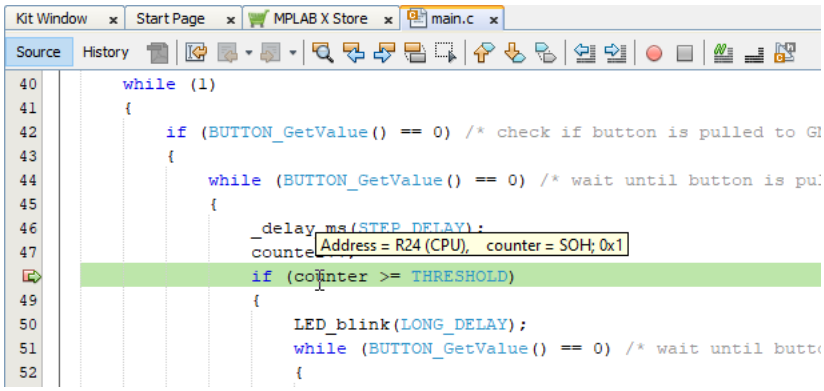
Figure 4-11. Breakpoint Set in Code



4.10.2 Halt at Breakpoint

Click the “Debug Project” icon  to execute the program in debug mode. As the program contains a loop waiting for a button press, you will need to press and release the button on the evaluation board once to enter the loop and halt at the breakpoint (see figure below). Hover over the variable in the figure to see its current value.

Figure 4-12. Program Execution Halted at Breakpoint



4.11 Stepping Through Code

Stepping allows you to examine changes in variable values or determine if the program flow is correct. Available step functions are shown in the table below.



Step Over – Executes one source line of a program. If the line is a function call, it executes the entire function and stops.



Step Into – Executes one source line of a program. If the line is a function call, it executes the program up to the function’s first statement and stops.



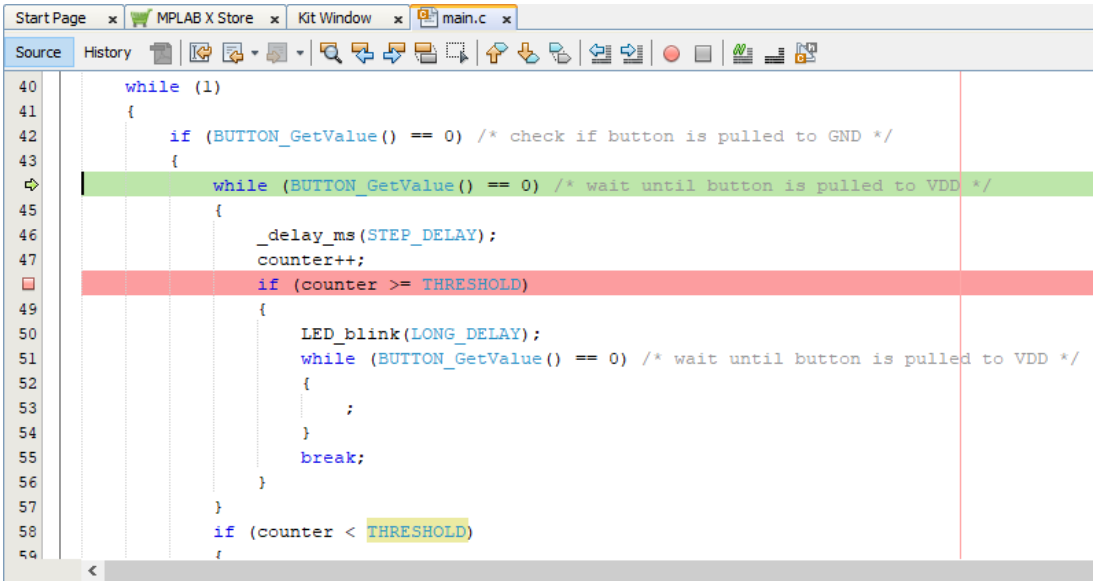
Step Out – Executes one source line of a program. If the line is a function call, it executes the functions and returns control to the caller.



Run to Cursor – Runs the current project to the cursor's location in the file and stops program execution.

Use Step Over or Step Into to execute the code on the breakpoint line. As the break stopped the counter at 1, the condition executes as false and control will return to and halt at the previous while statement (see figure below).

Figure 4-13. Code During Step



4.12 Viewing Variable Values

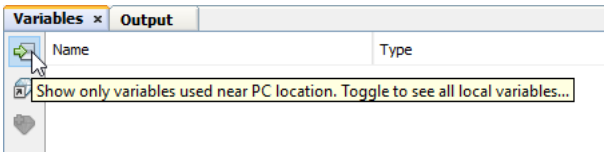
You can view the changing values of variables in the Variables window. For more information on this window, see 5.17. Watch Local Variable Values Change.

To see different values of the variable `counter`, remove the breakpoint set in the previous topics by clicking on it in the left margin. Then set a new breakpoint as shown in the figure below.

```
39
40      while (1)
41      {
42          if (BUTTON_GetValue() == 0) /* check if button is pulled to GND */
43          {
44              while (BUTTON_GetValue() == 0) /* wait until button is pulled to VDD */
45              {
46                  _delay_ms(STEP_DELAY);
47                  counter++;
48                  if (counter >= THRESHOLD)
49                  {
50                      LED_blink(LONG_DELAY);
51                      while (BUTTON_GetValue() == 0) /* wait until button is pulled to VD
52                      {
53                          ;
54                      }
55                      break;
56                  }
57              }
              if (counter < THRESHOLD)
59              {
60                  LED_blink(SHORT_DELAY);
61              }
62              counter = 0;
63          }
64      }
65  }
```

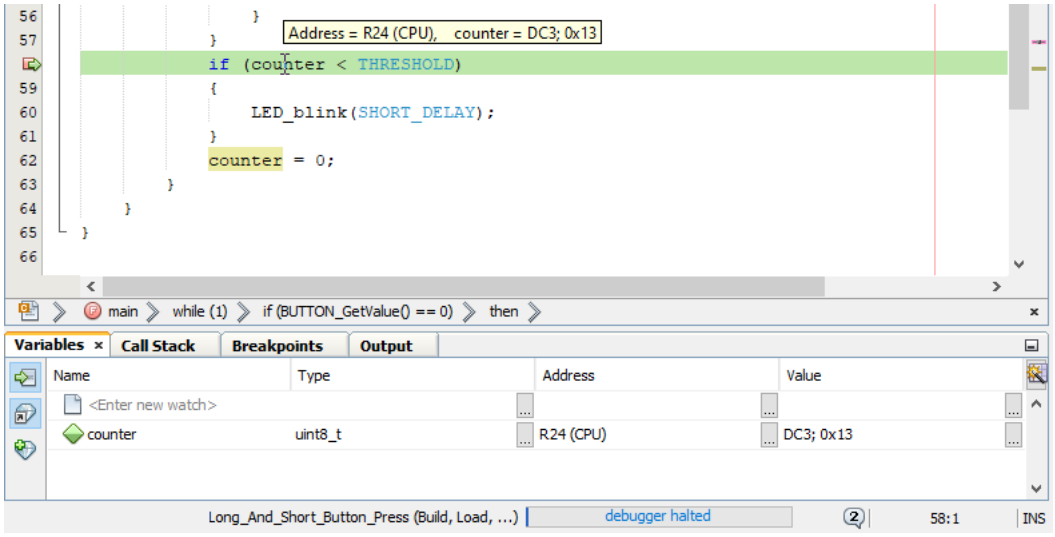
Click the “Debug Project” icon  . When the code starts to run, quickly press and release the button on the evaluation board. The code will halt at the new breakpoint.

Open the Variables window by selecting *Window>Debugging>Variables*. To view the variable `counter` in the Variables window, click on the Show Variables button (see figure below).



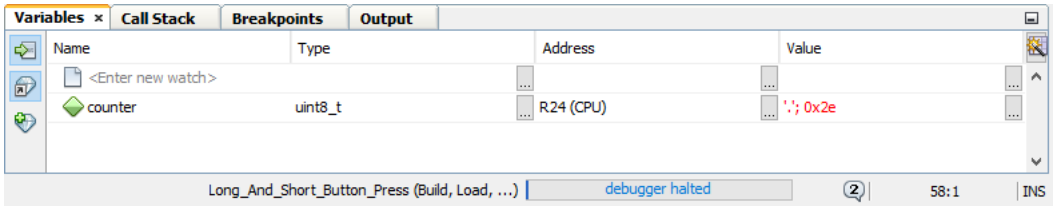
`counter` information will be shown in the Variables window but may also be seen by hovering over the variable in code.

Figure 4-14. Variables Window - First Halt



Click the “Continue” icon  to continue code execution. Press and release the button on the evaluation board a bit slower. When the breakpoint halts the code again, view the Variables window to see that the value has changed (red text).

Figure 4-15. Variables Window - Second Halt



4.13 Watching Symbol Values Change

Watch the values of global symbols or SFRs change in the Watches window. Determining if these values are as expected during program execution will help you to debug your code.

For more on this window, see 13.24. Watches Window.

4.13.1 Create a New Watch

Click the “Finish Debugger Session” icon  . Then remove the breakpoint from the previous discussion by clicking in the left margin.

To create a new watch:

1. Select *Debug>New Watch*. The New Watch dialog will open.
2. Enter a Watch expression, in this case `PORTC.IN`, and then click **OK**. The Watches window will now appear on the desktop with the symbol.

Figure 4-16. New Watch Symbol

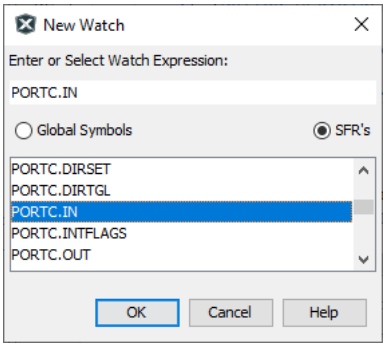
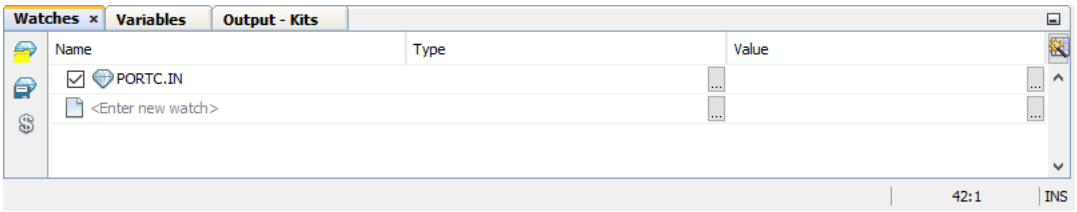


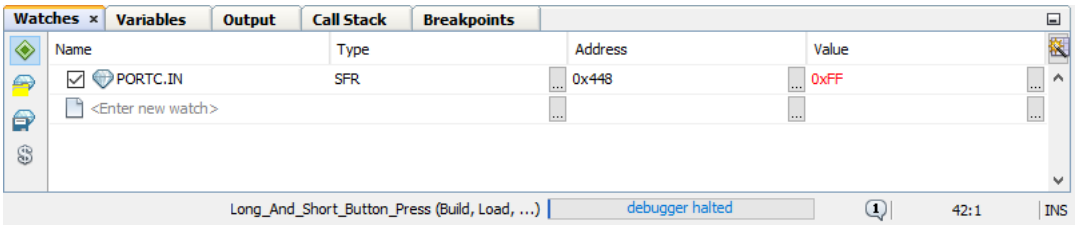
Figure 4-17. New Watch in Window



4.13.2 View Value Changes in Watches Window

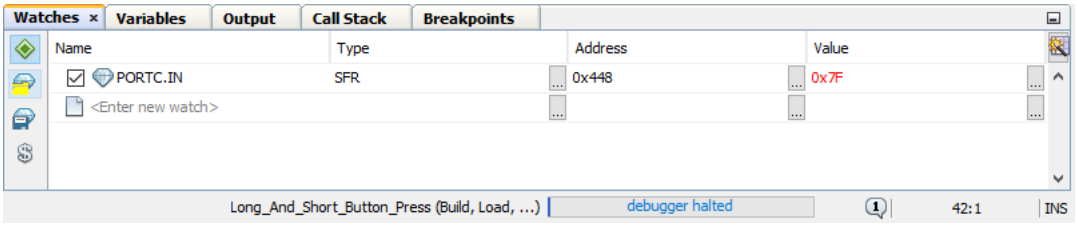
Click the “Debug Project” icon  to begin code execution. Halt the execution by clicking the “Pause” icon  . Click on the Watches tab to see the value of `PORTC.IN` after port initialization.

Figure 4-18. Watches Window - First Halt



Click the “Continue” icon  . This time press and hold the button on the evaluation board and then click the “Pause” icon. View the Watches window to see that the value has changed.

Figure 4-19. Watches Window - Second Halt



Click the “Finish Debugger Session” icon  .

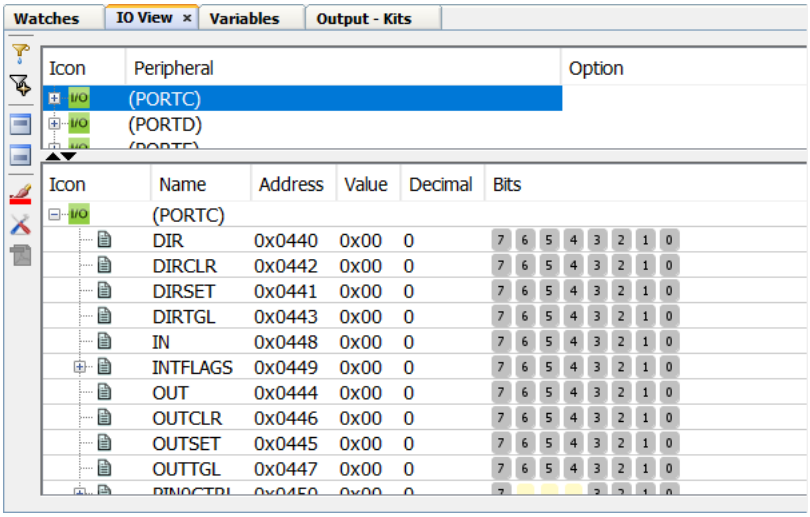
4.14 Viewing I/O Registers

The I/O View window provides a graphical view of the I/O memory map of the device associated with the active project. This debug tool will display the actual register content when debugging, allowing verification of peripheral configurations. It can also be used to modify the content of a register without having to recompile.

View I/O Register

Open the I/O View window by selecting *Window>Debugging>IO View*. In the Peripheral (upper) part of the window, click on “PORTC.” The registers associated with this peripheral will be shown in the Register (lower) part.

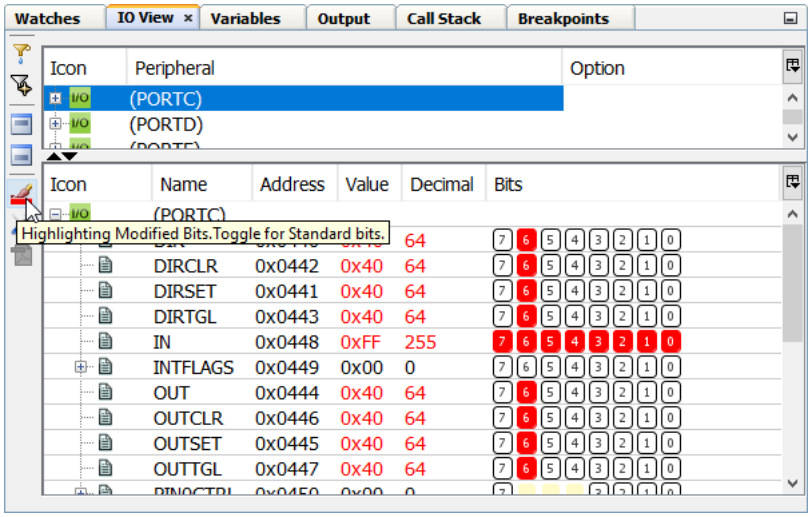
Figure 4-20. IO View Window



Click the “Debug Project” icon  to begin code execution. Halt the execution by clicking the “Pause” icon  .

Click on the **IO View** tab to view the changes. The contents of columns that have changed when the program initialized the port are shown in red (see figure below). This is selectable by the paintbrush button on the left side of the window.

Figure 4-21. IO View Window - First Halt

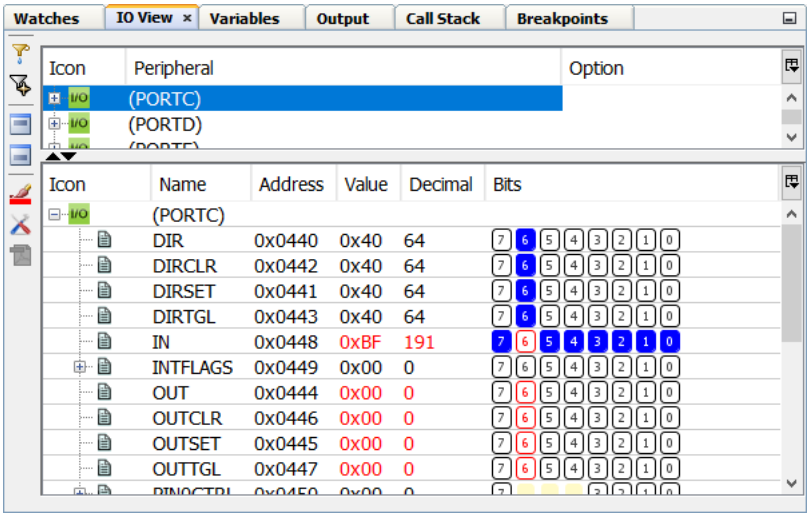


Click the “Continue” icon  and then “Pause” again. Nothing changed so the text will be black and the Bits will be blue.

View Register Changes

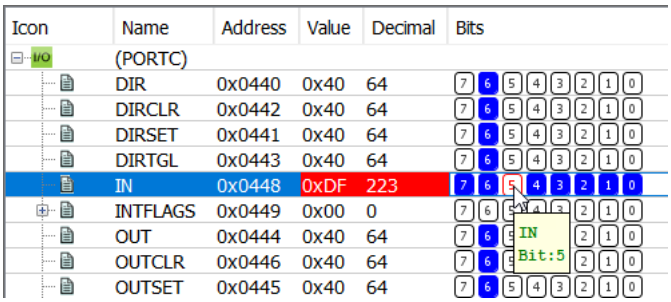
Click the “Continue” icon again. To see values change in IO View, press and hold the button on the evaluation board until you see the LED light and then click “Pause.” PC6 has been driven to ground (0) to activate the LED.

Figure 4-22. IO View Window - Bit Change



Make Changes to Register

You may also change register values directly by clicking on them when program execution is halted.



For more information on this window, see the following sections:

- 5.18. View Registers for the Project (I/O View)
- 13.9. I/O View Window

4.15 Viewing Device Memory (including Configuration Bits)

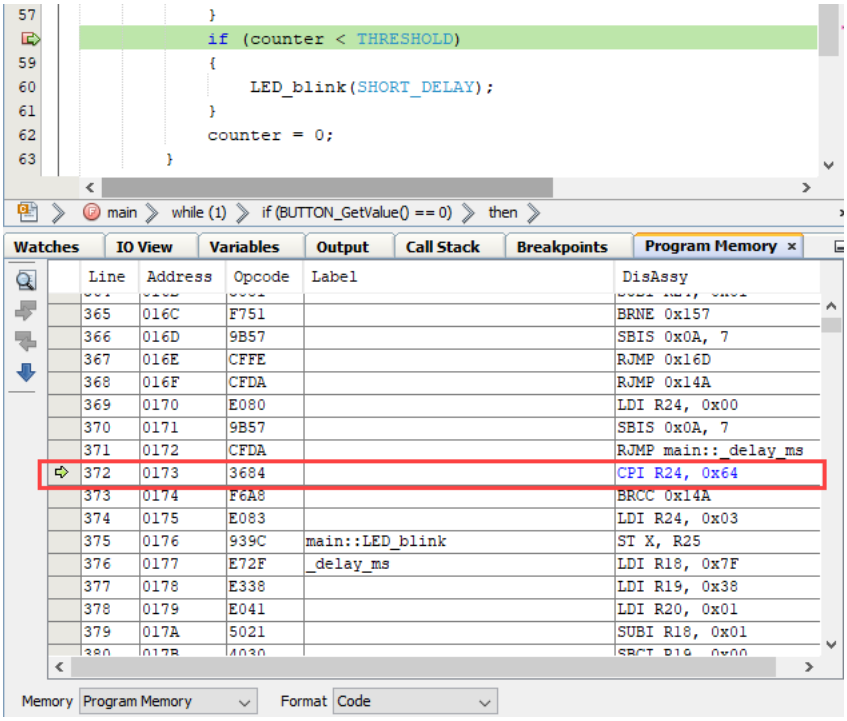
MPLAB X IDE has flexible, abstracted memory windows that provide a customizable view of differing types of device memory. Select *Window>Target Memory Views* to see a list of device-specific memory windows.

For example, to view the Flash memory:

1. Put a breakpoint at this line of code: `if (counter < THRESHOLD).`
2. Click the “Debug Project” icon  . When the code starts to run, quickly press and release the button on the evaluation board. The code will halt at the breakpoint.
3. Select *Window>Target Memory Views>Program Memory*.
4. The Program Memory window will open showing the last halt location.

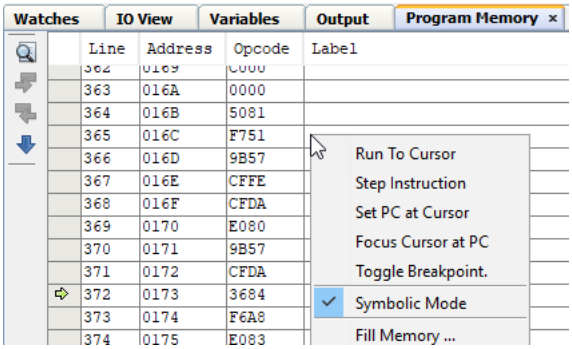
© 2022 Microchip Technology Inc. and its subsidiaries

Figure 4-23. Memory Window Content at Breakpoint Halt



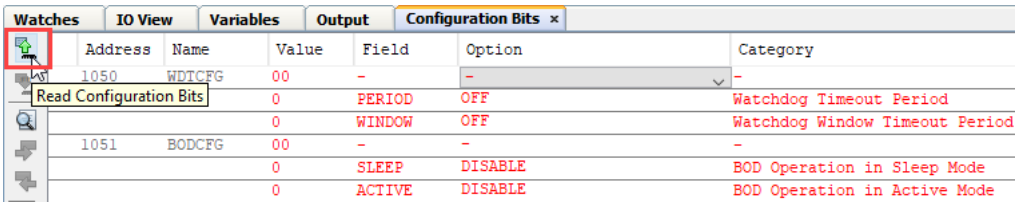
To set Memory window options, right click in the Memory window to pop up a menu with various options such as display options, fill memory, table import/export, and output to file. The content of the menu depends on the window. See 13. MPLAB X IDE Windows and Dialogs.

Figure 4-24. Memory Context Menu



To refresh some windows, a read of device memory is needed. For AVR devices, the Configuration Bits memory window needs to be read before it can be edited.

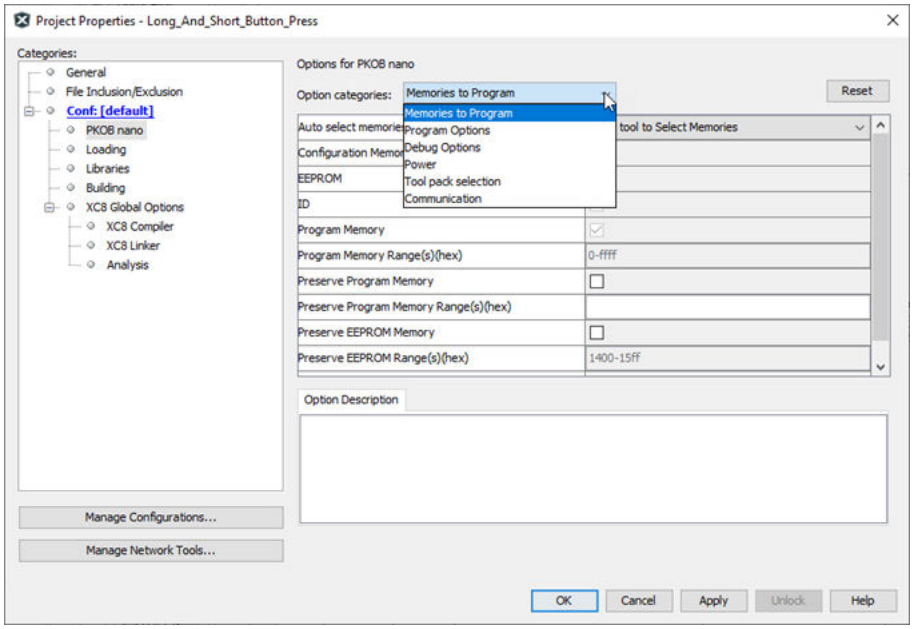
1. Click the “Finish Debugger Session” icon  .
2. Click on the “Read Device Memory” icon, either on the window (see image below) or the toolbar (). In the context of this window, it is “Read Configuration Bits.”
3. Once memory has been read, the red text will change to black.



4.16 Programming a Device

After your code is debugged, you can program it onto a target device.

First, check the programming options in the Project Properties window. For this tutorial, no options need to be changed.



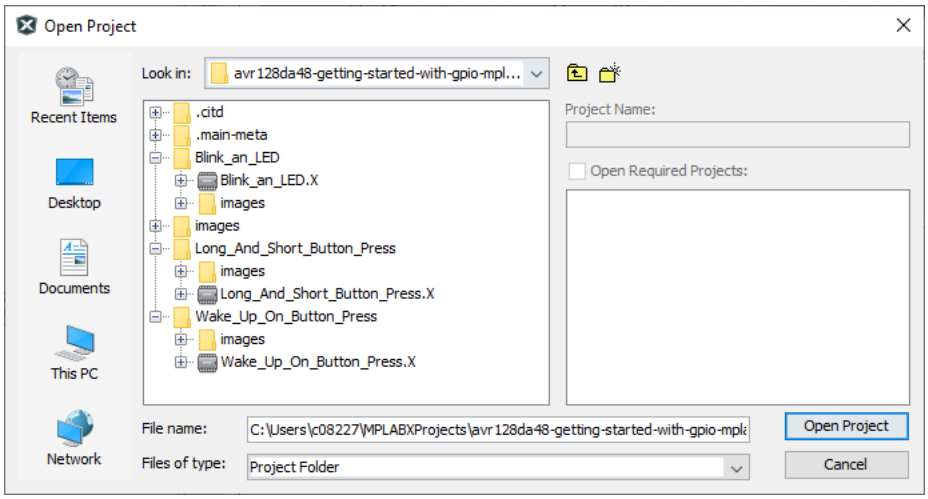
Finally, program the device by clicking the “Make and Program” icon  .

Note: Not all programming functions are in the MPLAB X IDE. For additional programming support, see the MPLAB IPE included with the MPLAB X IDE installation.

4.17 Other Examples

The example used in this tutorial is one of three. Select “Open Project” to work with the other downloaded projects.

© 2022 Microchip Technology Inc. and its subsidiaries



For different types of projects using the device in this tutorial, return to MPLAB Discover and filter based on:

- Under **Name**, enter “getting started.”
- Under **Device**, enter “avr128da48.”

Or filter for different parameters.

In addition, examples for different devices and development tools may be found here:

Developer Help Projects

5. Basic Development and Debug Flow

Before You Begin, install MPLAB X IDE, set up any hardware tools (install USB drivers and properly connect to your target) and install language tools for your selected device. Then launch MPLAB X IDE.

Use the following sections as a guide to basic development and debug of an application in MPLAB X IDE.

5.1 Create a New Project

A *Project* is a group of source files and associated information about how to compile and link the source files and run the resulting program. The IDE stores project information in a project folder that includes a makefile and metadata files. Your source directories do not need to be physically located in the project folder, though it is recommended for project portability.

In the IDE, you always work inside a project even if your program is contained in a single source file. Each project must have a makefile so the IDE can build the project. A project's makefile can be generated by the IDE or you can use a makefile that was previously created outside the IDE.

Projects with IDE-generated makefiles are called **Managed Projects** and come in several varieties. When you create a project in MPLAB X IDE using the New Project Wizard, you will be able to choose from available types of project.

Projects that use makefiles, created outside the IDE, are called **User Makefile Projects**. For more information on creating this type of makefile, see Creating Makefiles Outside of MPLAB® X IDE.

You work with projects through the IDE's **13.19. Projects Window**.

You can change a Standalone (Application) project to a Library project by changing the Configuration Type. For details, see 5.10.1. Change Project Configuration Type.

5.1.1 MPLAB X IDE v5 New Project Format

The new project format in v5.xx and greater supports packs which contain versioned device and tool information. Packs can be found under:

```
<MPLAB X IDE install directory>\v5.xx\packs
```

Managing Projects

Projects created in or updated to v5.00 or greater are not backward compatible to v4.xx.

When you open a project created prior to v5.00, a message displays, announcing that it will upgrade the project to the current project version.

- If you choose **Yes**, the project will be upgraded to the current version and the project can no longer be opened with the earlier version of the IDE.
- If you choose **No**, you cannot save any modifications to the project in v5.00 or greater. You must make any changes in v4.xx.

If you upgraded a project to v5.xx but need to revert a project back to v4.xx, install the plugin in MPLAB X IDE found under *Tools>Plugins>Available Plugins>Save As v4.xx Project*. Once that plugin is installed, you can save your project in the older version by selecting *Tools>Embedded>Save as MPLAB X v4.xx Project*.

About Packs

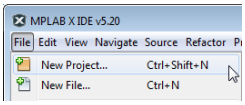
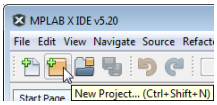
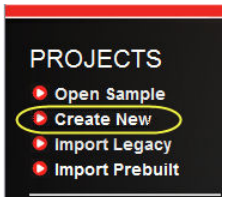
Previously (v4.20 and earlier versions), supported device information was built into each MPLAB X IDE version. You could not update the devices supported in this version; you had to wait until the next MPLAB X IDE version for device support.

Now (v5.00 and latest versions), this information for each device is grouped into versioned packs, so you can obtain new device support without updating your MPLAB X IDE version. For more on packs, see:

6.1. Work with Packs

5.1.2 Launch New Project Wizard

New Projects can be launched in several ways.

	*File>New Project* (or Ctrl+Shift+N)
	New Project icon (on the File toolbar)
	**Start Page**, **Learn & Discover** or **My MPLAB X IDE** tab, "Projects," > "Create New"

5.1.3 Step 1: Choose Project

Choose a project category. In most cases you will choose a project type from "Microchip Embedded."

The following options are available to choose for a project category:

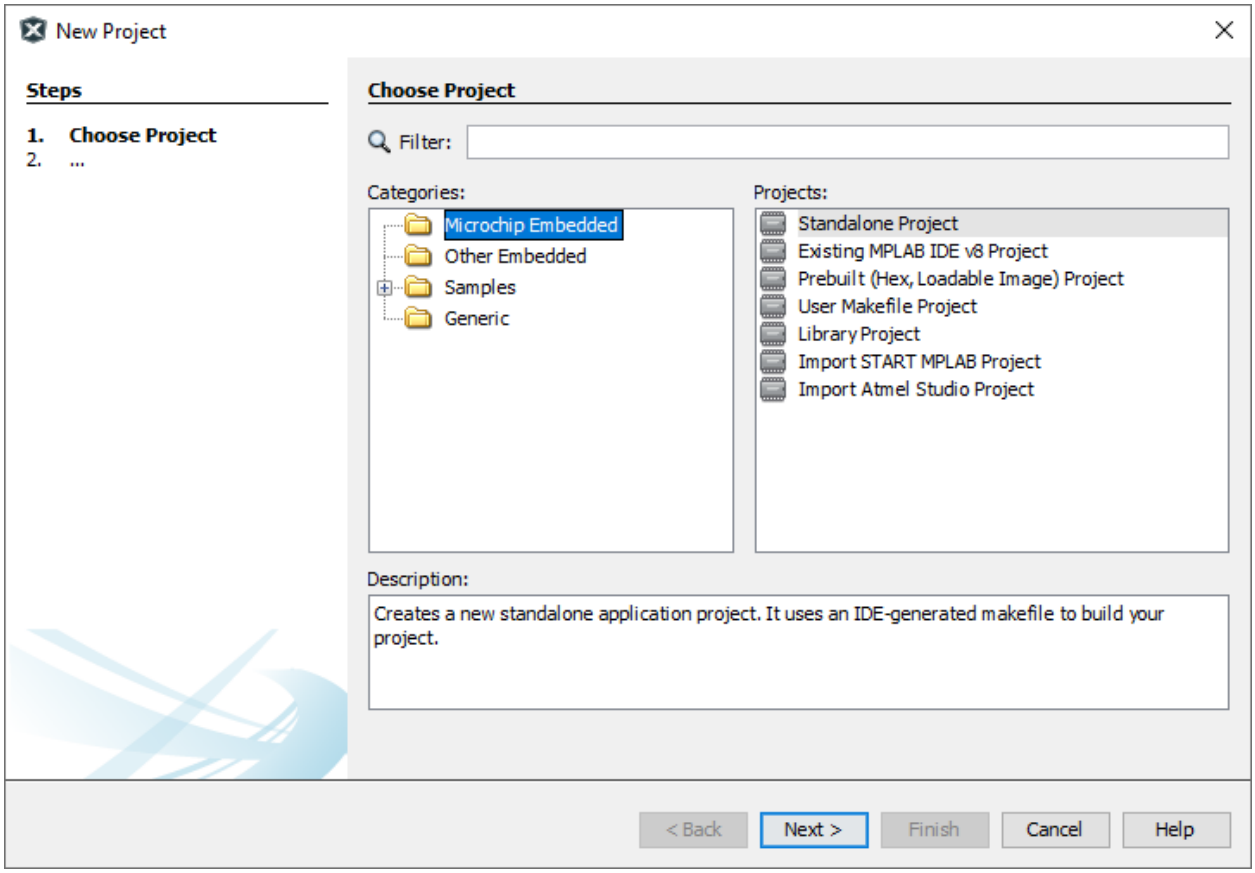
- Standalone Project – Create a new C and/or assembly code project. This type of project is shown in this section.
- Existing MPLAB IDE v8 Project – Convert your existing MPLAB IDE v8 project into an MPLAB X IDE project. For details see 6.2.2. Import an Existing MPLAB IDE v8 Project.
- Prebuilt (Hex, Loadable Image) Project – Load an existing project image into MPLAB X IDE. For details, see 6.2.4. Prebuilt Projects.
- User Makefile Project – Create a project that makes use of an external makefile. For details, see 6.2.9. Create User Makefile Projects.
- Library Project – Create a new C and/or assembly code project that will build into a library, instead of an executable hex file. For details, see 6.2.7. Library Projects.
- Import START MPLAB Project and Import Atmel Studio Project - Import these Atmel projects. For details, see 6.2.3. Import Atmel Studio 7 or Atmel START Project.

Other options are also available:

- 6.2.12. Other Embedded Projects – projects from other vendors.
- 6.2.14. Sample Projects – includes ready-to-use projects for different device families and project templates for different device families.
- Generic - Create a group of projects, for example an application and bootloader.

After you have made your selections, click **Next>** to move to the next dialog.

Figure 5-1. Project Wizard – Choose Project



5.1.4 Step 2: Select Device (and Tool)

Select the device you will be using in your application from the “Device” drop-down list. To narrow your selection list, chose a “Family” first.

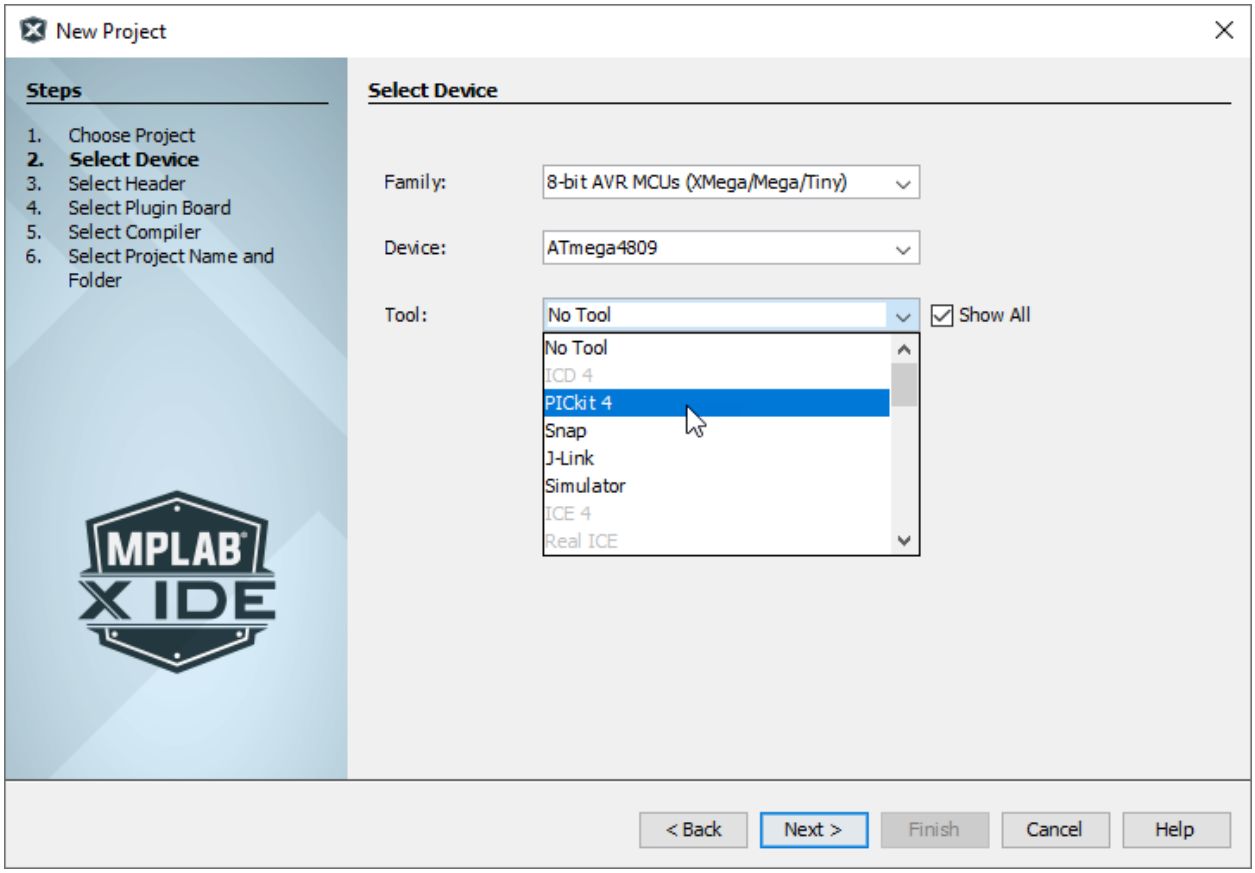
For LF devices, be aware that you will need to set up your physical device to be powered by a lower voltage than “F” type devices, typically 3.3 V instead of 5.0 V. If a higher voltage will damage your device, MPLAB X IDE will display a Warning dialog to remind you.

Select the debug tool that supports the selected device. Tools that support the device should be listed in the drop-down list. To see all tools supported in MPLAB X IDE, check “Show All.”

For hardware tools, you may notice that a serial number (SN) is specified below any tool that is connected to your computer. This allows you to select from several connected hardware tools.

Click **Next>**.

Figure 5-2. Project Wizard – Select Device



5.1.5 Step 3: Select Header

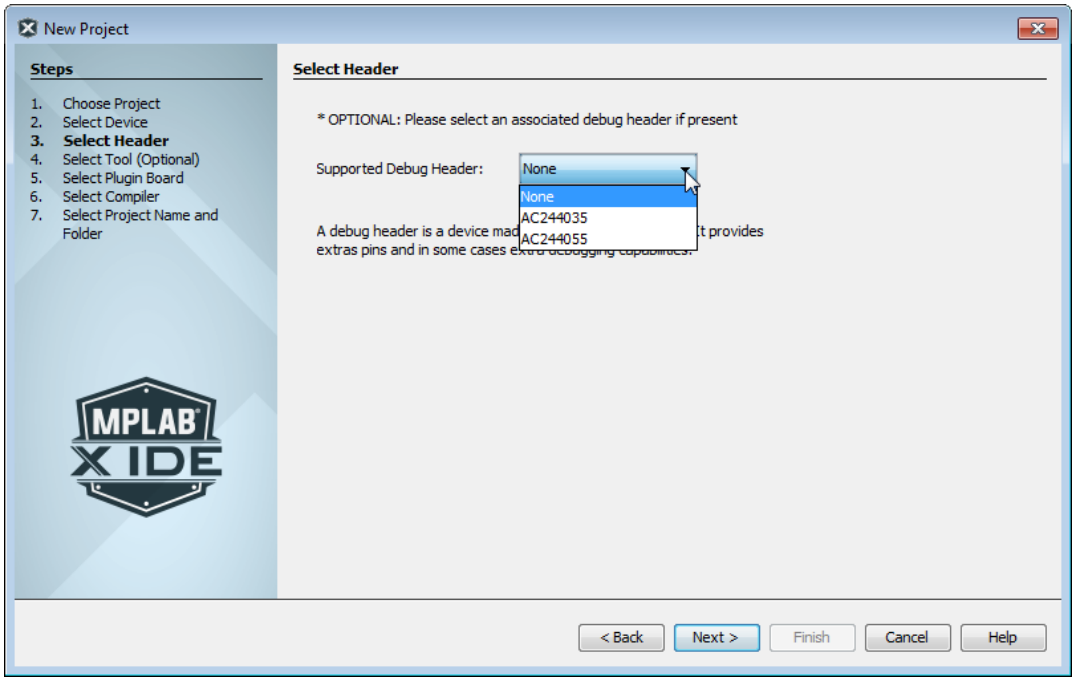
Select a header for your project device. If one is not available, MPLAB X IDE will not show this step. If one is available, consult the Developer Help topics below for more information:

- Processor Extension Paks and Debug Headers
- Emulation Extension Paks and Emulation Headers

Click **Next>** when done.

Note: You can select a header later (if one is available) using the Project Properties window.

Figure 5-3. Project Wizard – Select Header



5.1.6 Step 5: Select Plugin Board

Select the MPLAB REAL ICE in-circuit emulator plugin board. If the emulator is not chosen as your debug tool, MPLAB X IDE will not show this step.

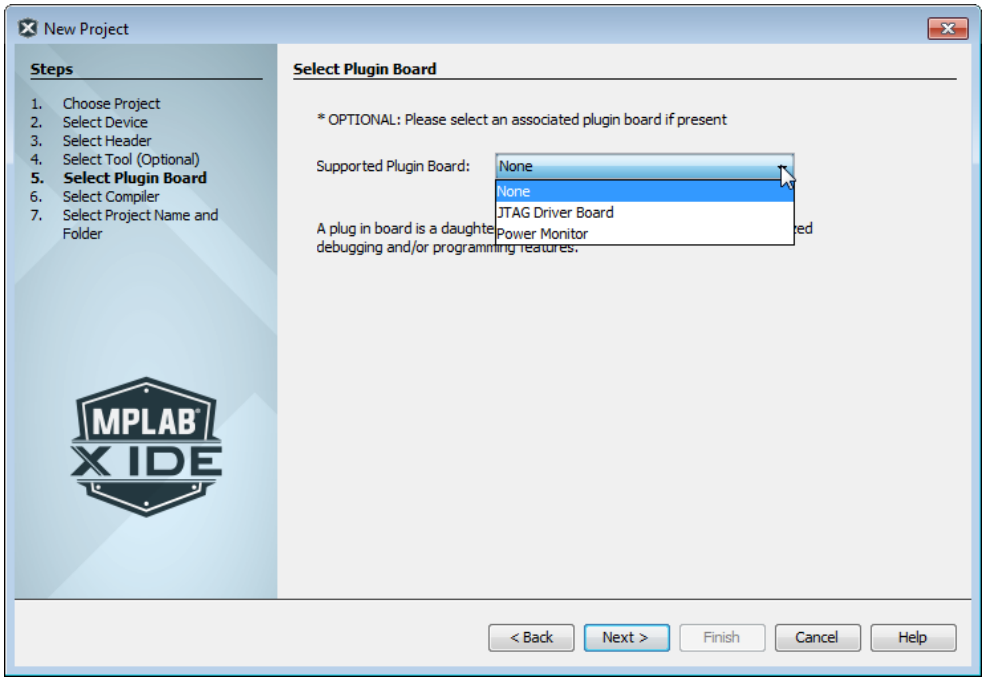
A plugin board is a circuit board that is inserted into the emulator's driver board slot to provide or add debug functionality.

Table 5-1. Emulator Plugin Boards

Supported Plugin Board	Board Description
None	Standard Communications driver board
None	High-Speed Communications driver board
JTAG Driver Board	JTAG Adapter board
Power Monitor Board	Power Monitor board (also inserts into logic probe connector)

Select your tool and then click **Next>**.

Figure 5-4. Project Wizard – Select Plugin



5.1.7 Step 6: Select Compiler

Select the language tool, either a C compiler or an assembler. Selectable language tools are show as:

ToolAbbreviation (ToolVersion) [ToolExecutablePath]

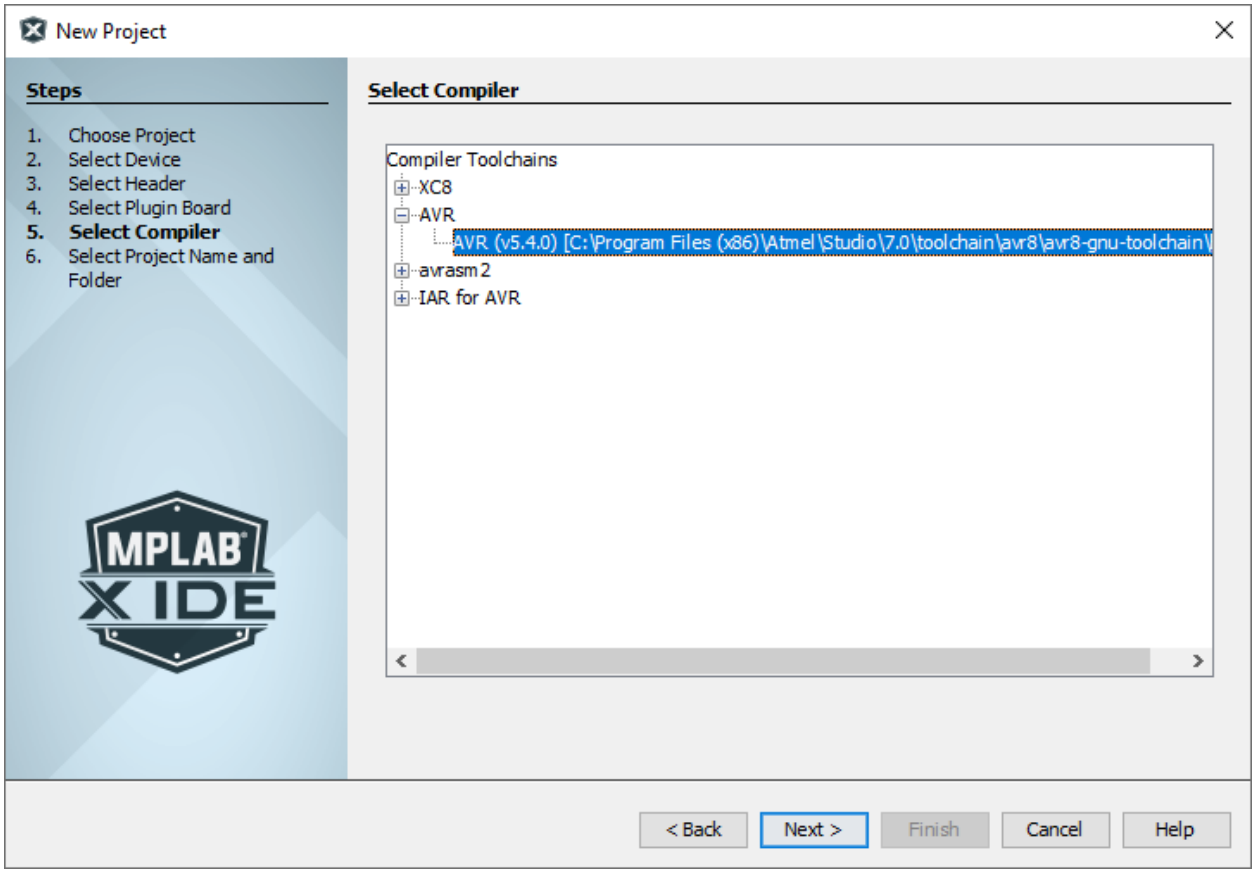
If you do not see your language tool listed or you do not see any support, ensure you have installed the tool. See 3.5. Install the Language Tools.

Note: For AVR GNU and Arm GNU toolchains, ensure the compilers are installed where MPLAB X IDE can search for them, e.g., for Windows OS, under `C:\Program Files\Microchip`. Otherwise you will have to enter the path. See 5.7. Set Language Tool Locations.

If your tool is listed under Build Tools but you do not see support, then your project device may not be supported by your tool or tool version. Consider selecting or installing another language tool that supports the device. Alternately, you can also use “Download Latest” to download and install the latest version of an MPLAB XC compiler, which may support your device.

Select your tool and then click **Next>**.

Figure 5-5. Project Wizard – Select Compiler (or Other Language Tool)



5.1.7.1 Supported Language Tools

MPLAB X IDE v5.40 (and later) is now a 64-bit only application. Therefore a language tool must be 64-bit to be supported in MPLAB X IDE.

Notes: MPASM assembler is a 32-bit windows application and will no long be supported or installed. Therefore:

- The 64-bit PIC Assembler will be available with the XC8 toolchain but will require migration.
- On first opening MPLAB X IDE, you will see a message that no toolchain is installed.

Table 5-2. Microchip Language Tools – Current

Toolchain	Full Name	Device Support
XC8	MPLAB® XC8 C Compiler for PIC MCUs	8-bit PIC® MCUs
	PIC Assembler	
	MPLAB XC8 C Compiler for AVR MCUs	8-bit AVR® MCUs
XC16	MPLAB XC16 C Compiler	16-bit PIC MCUs, dsPIC® DSCs
XC32	MPLAB XC32 C/C++ Compiler for PIC32M MCUs	32-bit PIC32M MCUs
	MPLAB XC32 C/C++ Compiler for PIC32C/SAM MCUs	32-bit PIC32C and SAM MCUs
Arm® GNU	Arm GNU C Compiler	32-bit Arm MCUs
AVR® GNU	AVR GNU C Compiler	8- and 32-bit AVR MCUs
AVRASM2	AVR Assembler	8-bit AVR MCUs

For more on each language tool, consult the language tool documentation.

For third-party language toolchains see the *Help>Release Notes>Release Notes for Third Party Tools*.

5.1.7.2 Download MPLAB XC Compilers

If your device is not supported by your compiler, download and install the latest version from the MPLAB XC C compiler webpage. On this page you will find the latest versions of the MPLAB XC C compilers, licensing information, and documentation.

If you would like to use an earlier version of an MPLAB XC C compiler, go to the Downloads Archive.

Once the compiler is downloaded, you can license your compiler by accessing your MySoftware account from the MPLAB X IDE desktop. Click the “My MPLAB X IDE” tab to find the Microchip Login area.

5.1.8 Step 7: Select Project Name and Folder

Select the project name, location and other project features.

Enter the project name. The name will be appended with `.X` by convention. See the “Project Folder” text box.

Browse to a folder location. You can create a new project folder if you need one. By default, projects will be placed in:

- Windows 7 and newer – `C:\Users\UserName\MPLABXProjects`
- Linux – `/home/UserName/MPLABXProjects`
- Mac – `/Users/UserName/MPLABXProjects`

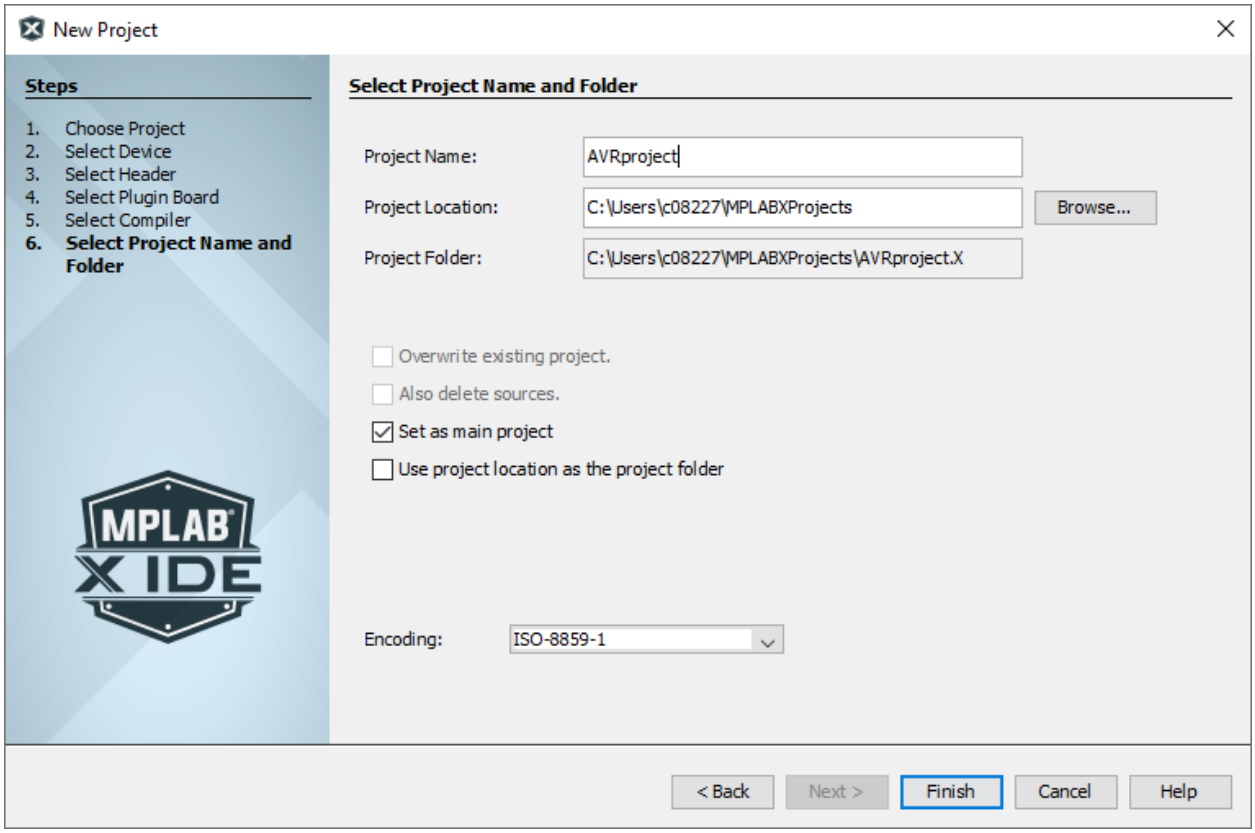
Table 5-3. Other Features

Set as main project	Specify this project as your main project
Use project location as the project folder	Create the project in the same folder as the project location. This is useful if you are importing an MPLAB IDE v8 project and want to maintain the same folder for your MPLAB X IDE project (as when you want the build to be the same). MPLAB X IDE uses a folder to store project information whereas MPLAB IDE v8 uses a `.mcp` file. Therefore you can only have one MPLAB X IDE project share a folder with an MPLAB IDE v8 project (`.mcp` file).
Encoding	Select the encoding for the project. The default is ISO-8859-1 (Latin 1) character set. This selection will specify the code syntax coloring, which can be edited under *Tools>Options* (*MPLAB X IDE>Preferences* for macOS), **Fonts and Colors** button, **Syntax** tab.

When you are done, click **Finish** to complete new project creation.

Note: You can change a Standalone (Application) project to a Library project by changing the Configuration Type. For details, see 5.10.1. Change Project Configuration Type.

Figure 5-6. Project Wizard – Select Project Name and Folder



5.1.9 Changing the Project Configuration Type

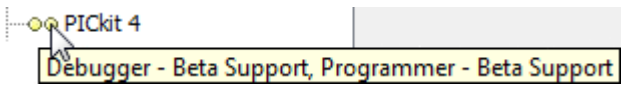
You can change a Standalone (Application) project to a Library project by changing the Configuration Type. For details, see 5.10.1. Change Project Configuration Type.

5.1.10 Support Lights - IDE v5.30 and Before

For earlier versions of MPLAB X IDE, project tool support for the project device was signified by the colored circles (lights) in front of the tool name.

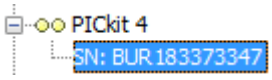
Light	Color	Support
●	Green	Full (Implemented and fully tested)
●	Yellow	Beta (Implemented but not fully tested)
●	Red	None

If you could not see the colors, mouse over a light to pop up text about support.



For hardware tools that could be used as either a debugger or a programmer, there were two lights next to the tool name, where the first (left-most) light signified Debugger Support and the second light signified Programmer Support.

Also for hardware tools, you would notice that a serial number (SN) specified below any tool that was connected to your computer. This allowed you to select from several connected hardware tools.



If you were using a third-party hardware tool, you needed to ensure you had properly installed that tool if you did not see it listed.

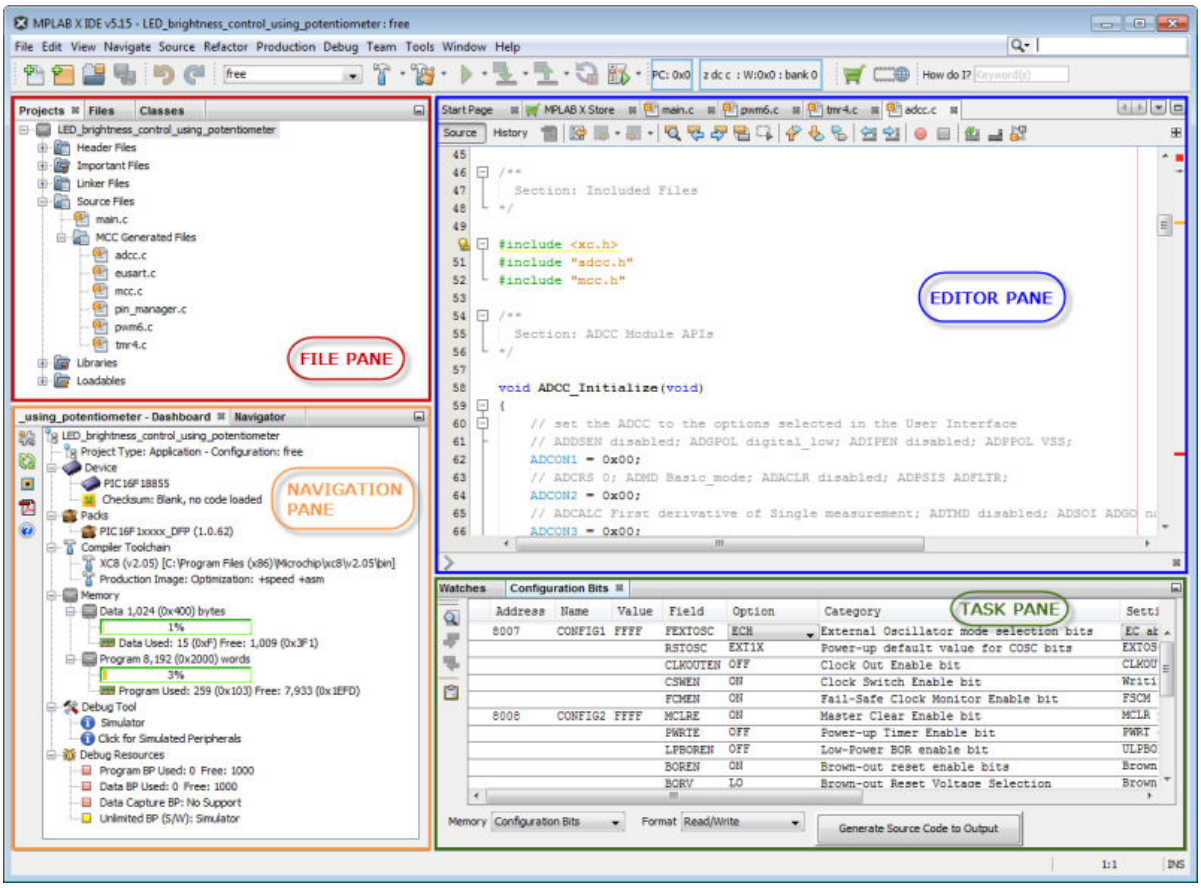
5.2 View Changes to Desktop

The MPLAB X IDE desktop is divided into four **panes**. When you have created your project, several **windows** will open in these panes.

Desktop Panes

- **File pane** – displays file-related information.
  - The Projects window displays the project tree with files grouped by category.
  - The Files window displays the project files according to the folder organization on your computer.
  - The Classes window displays any classes and their functions, variables and constants in the code. Double click on an item to see its declaration.
- **Navigation pane** – displays information on items in the File pane.
  - The Dashboard window displays information about the selected project.
  - The Navigator windows displays information on the symbols and variables in the project file selected.
- **Editor pane** – for viewing and editing project files. The Start Page and MPLAB X Store are also visible here.
- **Task pane** – displays task output from building, debugging, or running an application.

Figure 5-7. MPLAB X IDE Desktop - Panes



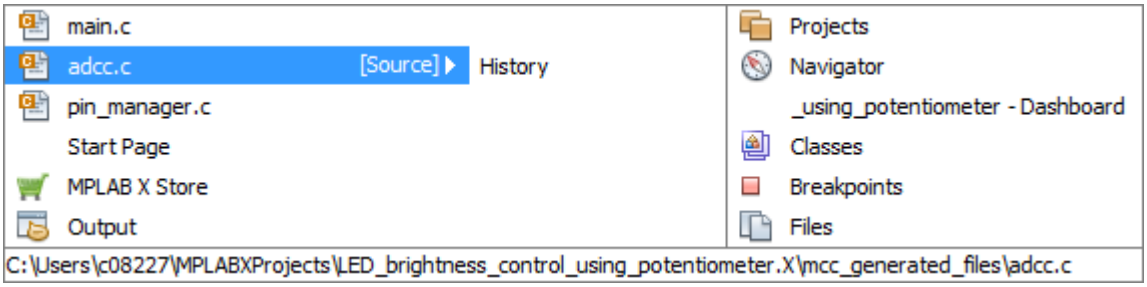
Desktop Windows

Although panes are useful for describing sections of the IDE interface, windows may be moved from one pane to another. So panes do not always represent the window content. Therefore, only windows are discussed in most documentation.

To move around between windows on the desktop:

- Click on a window to make it in focus.
- While in one window, you can change focus to any other open window by pressing in holding <Ctrl> and then pressing <Tab>. Select the window you want from a pop-up list.

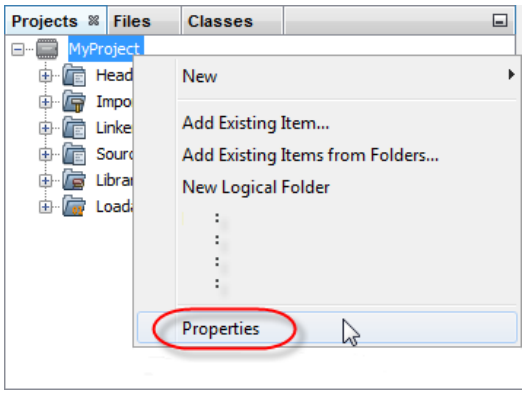
Figure 5-8. Switch Window Focus



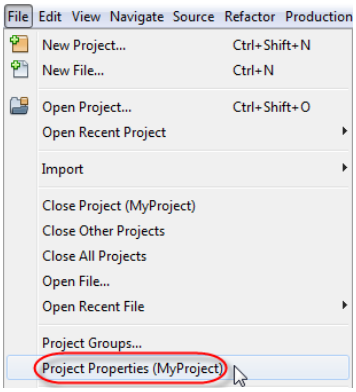
5.3 Open Project Properties

Once a project has been created, you can view or change the project properties in the Project Properties window. Access this window by performing one of the following actions.

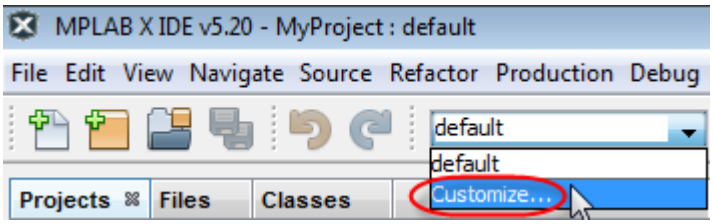
1. Right click on the project name in the Projects window and select “Properties.”



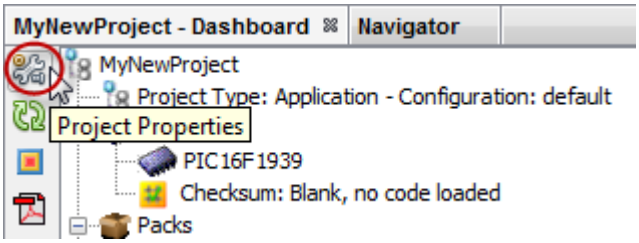
2. Click on the project name in the Projects window and then select *File>Project Properties*.



3. Click on the project name, in the Projects window, then click on the “Set Project Configuration” drop down menu to select "Customize."



4. Click on the “Project Properties” icon on the Dashboard window.



5.4 View or Make Changes to Project Properties

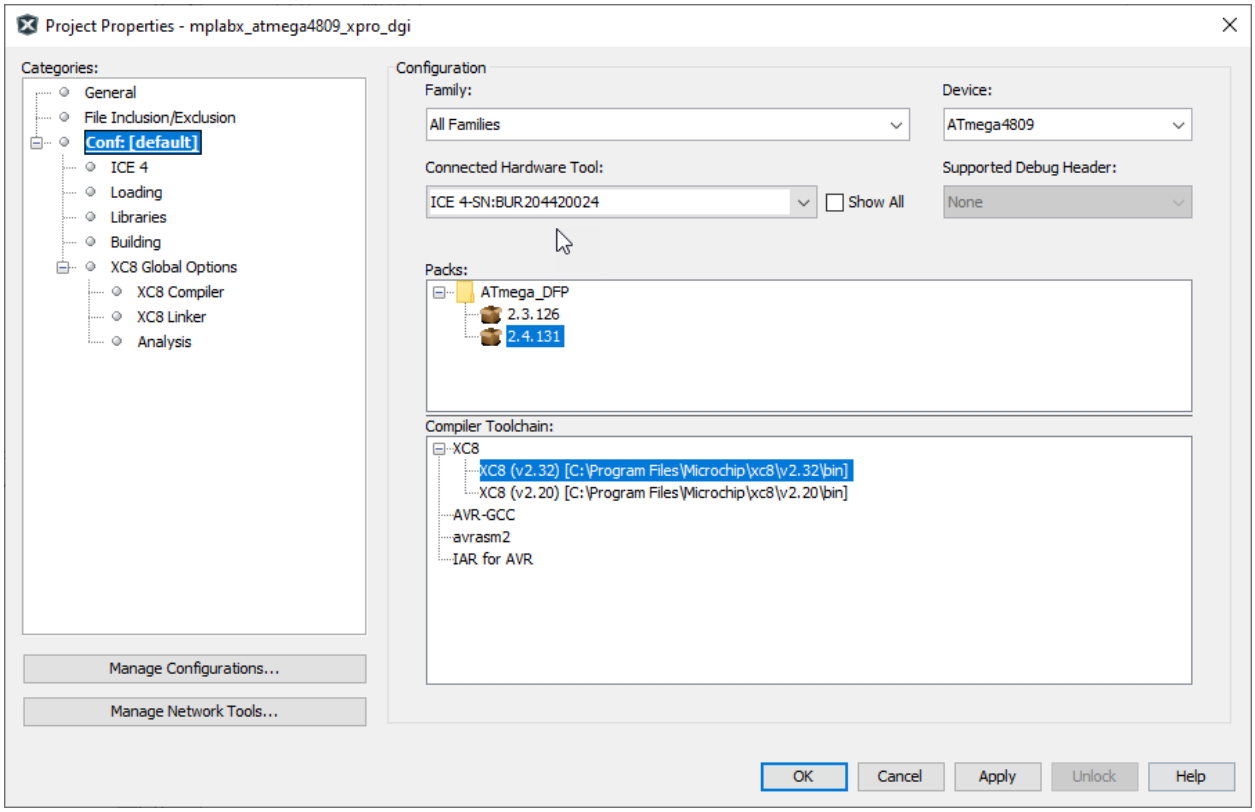
Once a project has been created, you can view or change the project properties in the Project Properties dialog. Access this dialog by performing either of the following actions:

- Right click on the project name in the Projects window and select “Properties.”
- Click on the project name in the Projects window and then select File>Project Properties.

Click the “Conf: [default]” category to reveal the general project configuration. Your may change the configuration by selecting a different item, if available.

Family	Device Family name. Select a device family to narrow the selection in the “Device” drop-down list.
Device	Name of the project device. Care should be taken when changing the device, especially to one from a different family, as the pack, hardware tool and compiler will need to be updated.
Connected Hardware Tool	Name of the connected hardware tool(s). To see all hardware tools that support the project device, check “Show All.”
Supported Debug Header	Number of supported header. Not all devices have/need a debug header.
Supported Plugin Board	Name of supported plugin board (Real ICE tool only).
Packs	Name and versions of each pack containing the device information file. The one previously selected is highlighted. See 6.1. Work with Packs.
Compiler Toolchain	Name(s) of supported toolchain(s) based on the project device. The one previously selected is highlighted. To get the latest version of an MPLAB XC C compiler, click the “Download Latest” link. See 5.1.7.2. Download MPLAB XC Compilers.

Figure 5-9. Project Properties Window - Default Configuration

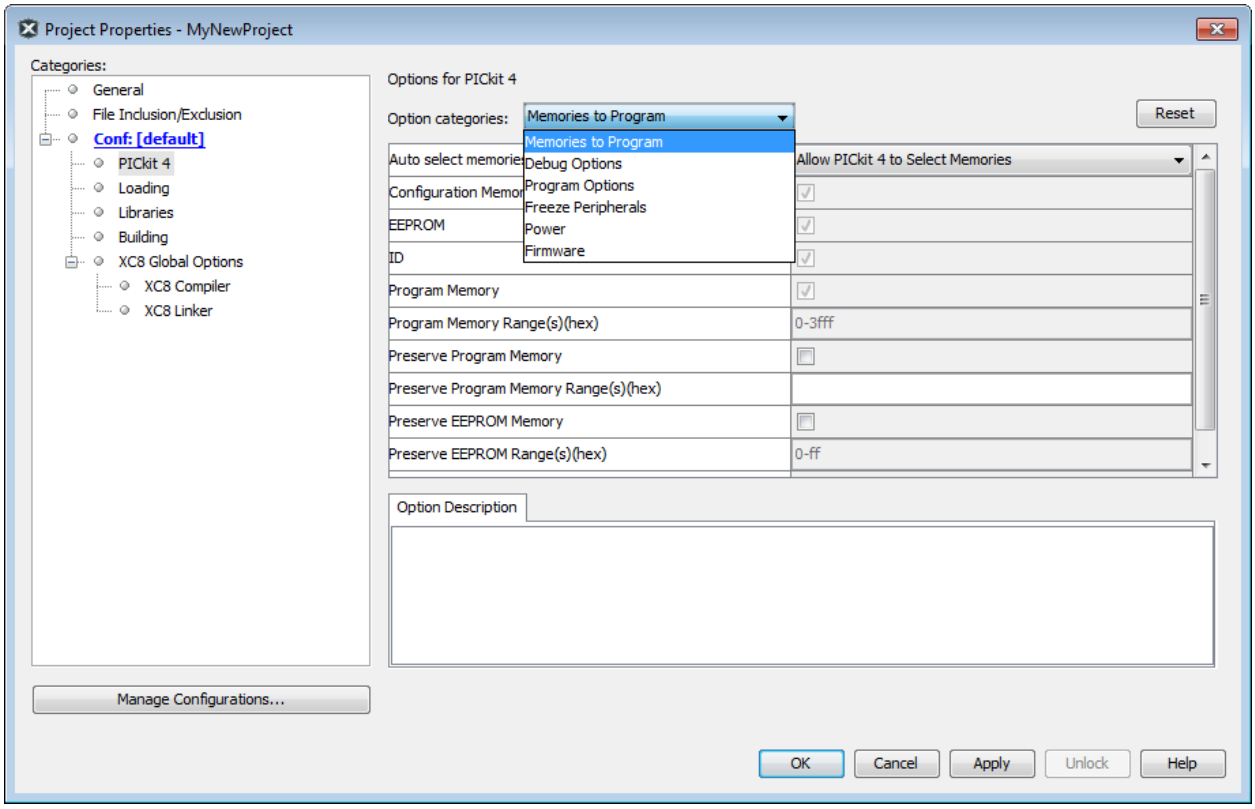


5.5 Set Up or Change Debugger/Programmer Tool Options

Set options for your tools in the Project Properties window.

Click on your hardware tool or simulator (beneath Conf:[default]) to see related setup options. For more on what these options mean, see your tool documentation.

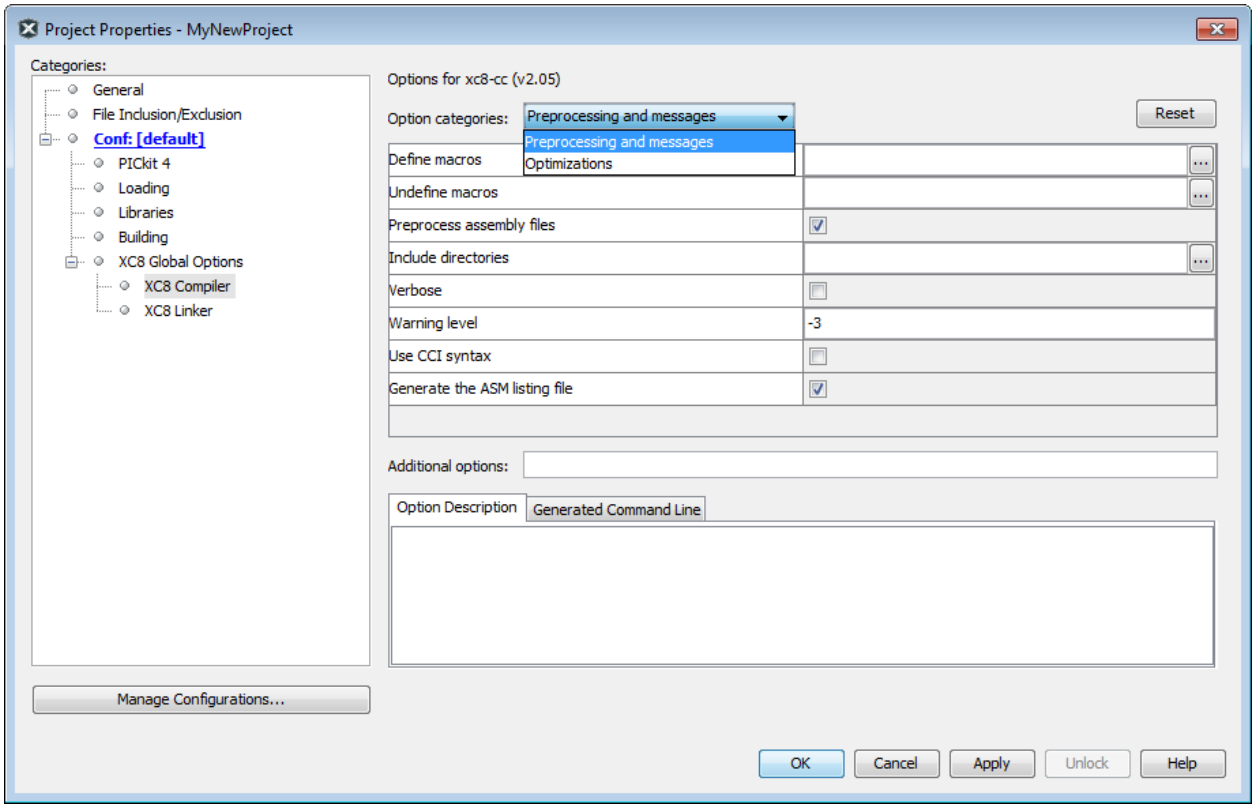
Figure 5-10. Example Tool Setup Page



5.6 Set Up or Change Language Tool Options

Click on your language tool to see related setup options in the Project Properties window. For more on what these options mean, see your language tool documentation.

Figure 5-11. Language Tool Setup Page



IDE Options vs. Command Line Options

Options entered into the IDE may differ in format from those entered on the command line. Click on an option name to see more information on that option in the “Option Description” tab. Also, after entering or selecting option data, click the “Generated Command Line” tab to view its contents and ensure the command line code generated by the option is what you expected.

Global Options

Set global options that apply to all language tools in the language toolsuite. Also use the “User Comments” tab to enter information about why certain options were selected.

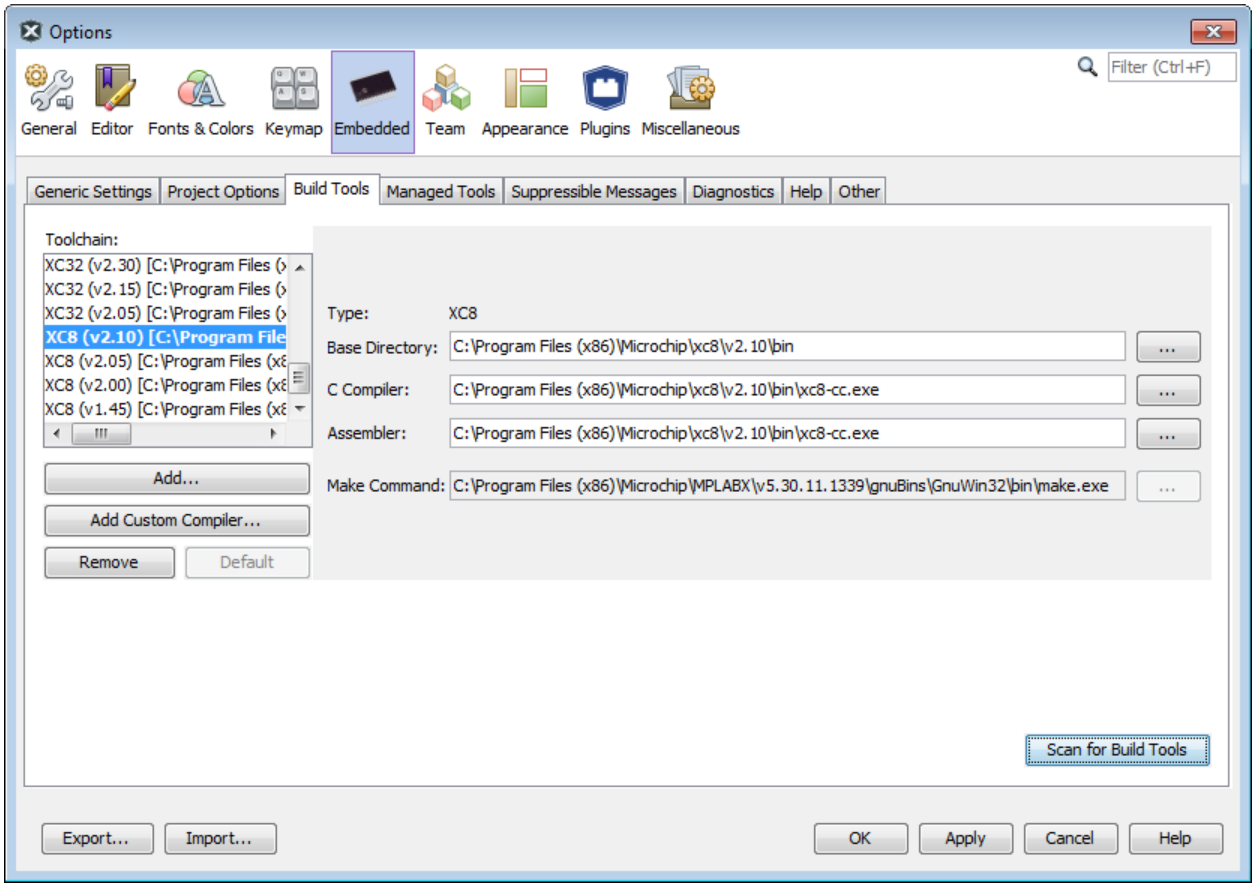
5.7 Set Language Tool Locations

To see which language tools are available to MPLAB X IDE, and to view or change their paths, use the following instructions:

- For macOS – Access the build tools from:
  MPLAB X IDE>Preferences>Embedded>Build Tools from the main menu bar.
- For Other OS – Access the build tools from:
  Tools>Options>Embedded>Build Tools.

The window should automatically populate with all installed toolchains. See also 13.20.3. Build Tools Tab.

Figure 5-12. Language Tool (Compiler) Locations



5.7.1 Add or Change a Toolchain

If you do not see your tool listed under “Toolchain,” try the following features:

- Scan for Build Tools – scan the environment path and list the language tools installed on the computer.
- Add – manually add the tool to the list by entering the path to the directory containing the tool executable(s), i.e., base directory. Typically, this is the bin subdirectory in the tool installation directory.

If you have more than one version of a compiler available, select one from the list.

To change the path of a tool, enter the new path or browse for it.

5.7.2 Remove a Toolchain

To remove an existing toolchain:

1. Click to select the toolchain from the list.
2. Click the Remove button.

This will NOT uninstall the compiler from your computer; it will only remove MPLAB X IDE awareness of this compiler. To add the toolchain back, see Section 4.7.1 “Add or Change a Toolchain.”

If you want to uninstall a compiler from your computer, remove the reference to the compiler in MPLAB X IDE first, as specified above. Then uninstall the compiler. If you uninstall the compiler first and then start MPLAB X IDE, the toolchain paths will appear in red. To remove the reference to this uninstalled compiler, follow the steps above.

© 2022 Microchip Technology Inc. and its subsidiaries

5.7.3 About Toolchain Paths

MPLAB X IDE searches for toolchains under default installation paths and the `PATH` environment variable. An example default path for MPLAB XC16 under a 64-bit Windows operating system is:

```
C:\Program Files\Microchip\xc16\vx.xx\bin
```

When you install a compiler, you have the choice to determine the location. You can either:

- install at the default location

OR

- install at a different location and add the location to the `PATH`

If you are installing at a location different from the default, you should add that location to the `PATH`.

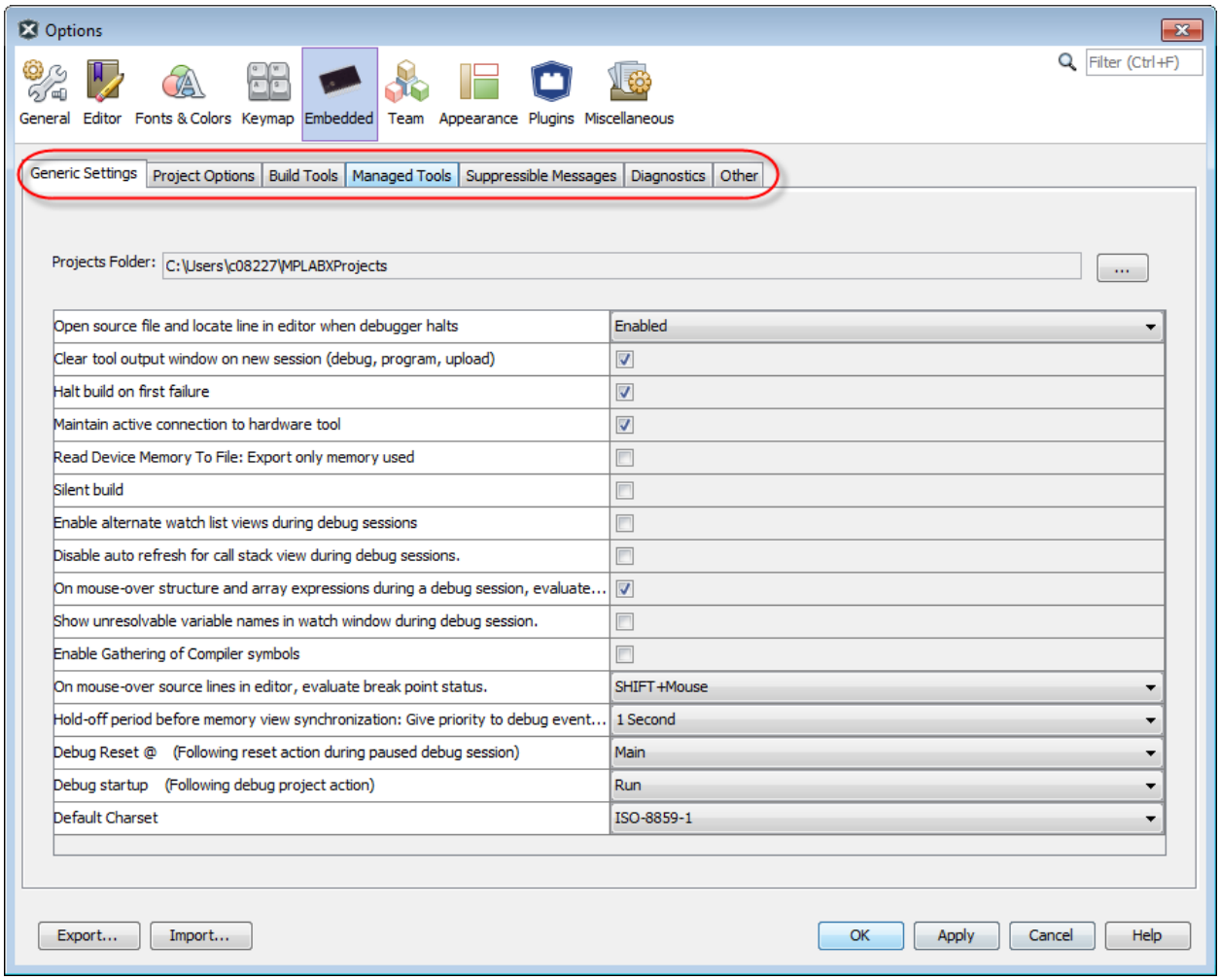
If you do not choose to have the installer add a new location to the `PATH`, you can point MPLAB X IDE to the compiler location in Tools>Options, Embedded button, Build Tools tab.

Finally, you can manually add the new location to the `PATH` variable.

If the toolchain is known to MPLAB X IDE, but the path cannot be found, the path will appear in red text.

5.8 Set Other Tool Options

In addition to the build paths, you may set up other Embedded options. Select from the tabs in the Options window, Embedded category. See 13.20. Tools Options Window, Embedded.



5.9 Add Files to a Project

Once you have created a new project, you will need to add files to it to create your application. You can create new, empty files in which to write your code or you can import or reference existing files. The editor can be used to create or update any file contents, making use of language-tool specific color-coding and other features.

File Link Order

The order in which you add files to the project will determine their link order. If you have imported a project from MPLAB IDE v8, the order will be determined by the `.mcp` file. A new file added to the project will be linked last. If you remove a file and then re-add it, it moves to the end of the link list.

Library and Object File Linking

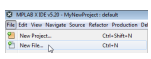
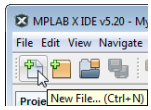
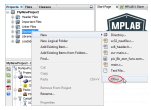
You can reference library and object files to be used by the linker.

Exclude Files from the Build

You can set file or folder level options to exclude files from the build.

5.9.1 Launch New File Wizard

To create a new, empty file to add code to your project, you can use the New File wizard. The wizard can be launched in several ways.

	*File>New File* (or Ctrl+N)
	New File icon (on the File toolbar)
	Right click on a project folder (e.g., “Source Files”) in the Projects/Files window and select *New>Other*.

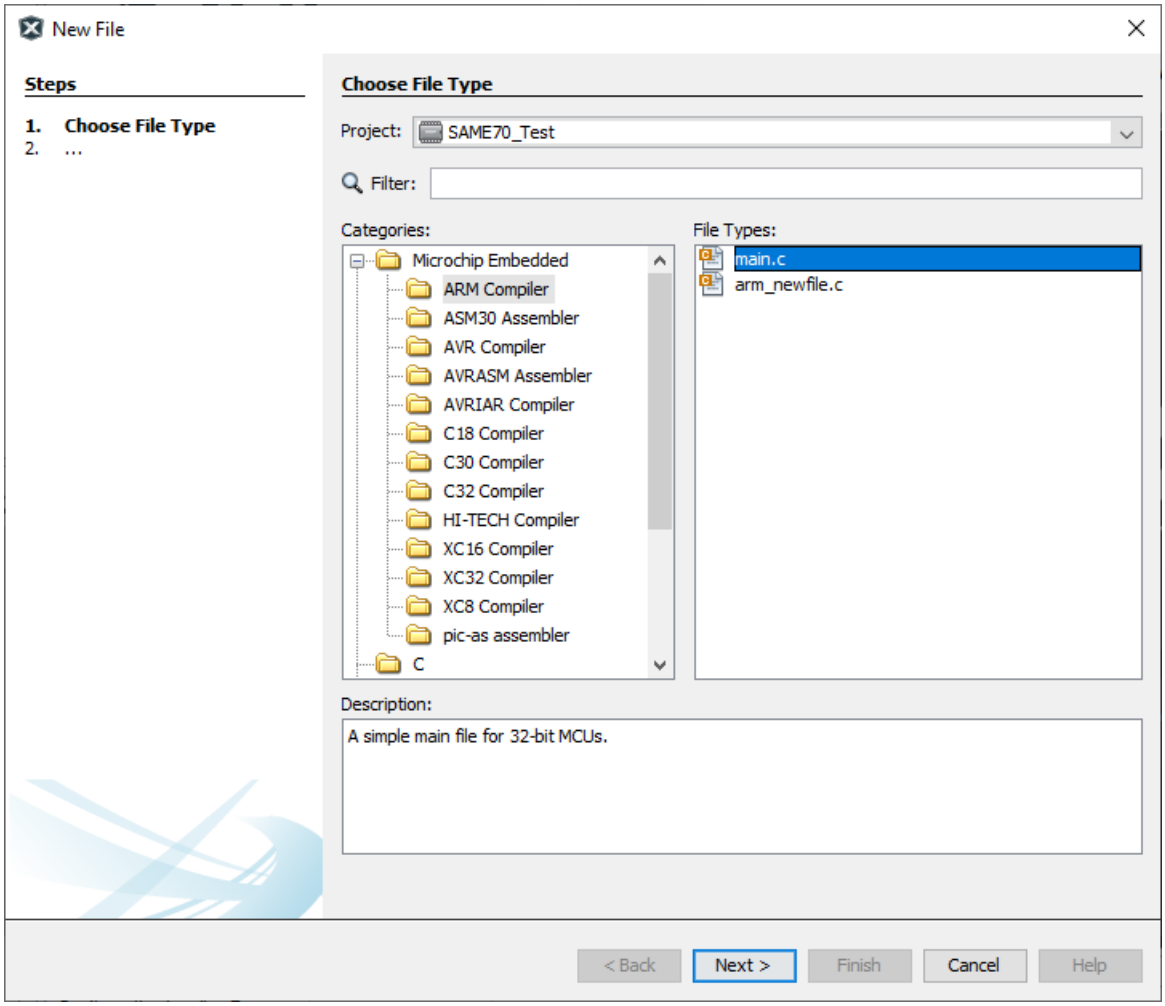
5.9.2 Create a New File

Create a new file by following the steps in the New File wizard.

Step 1. Choose File Type

Choose the file category by expanding “Microchip Embedded” to find an appropriate selection. Then select a file type. Click **Next>**.

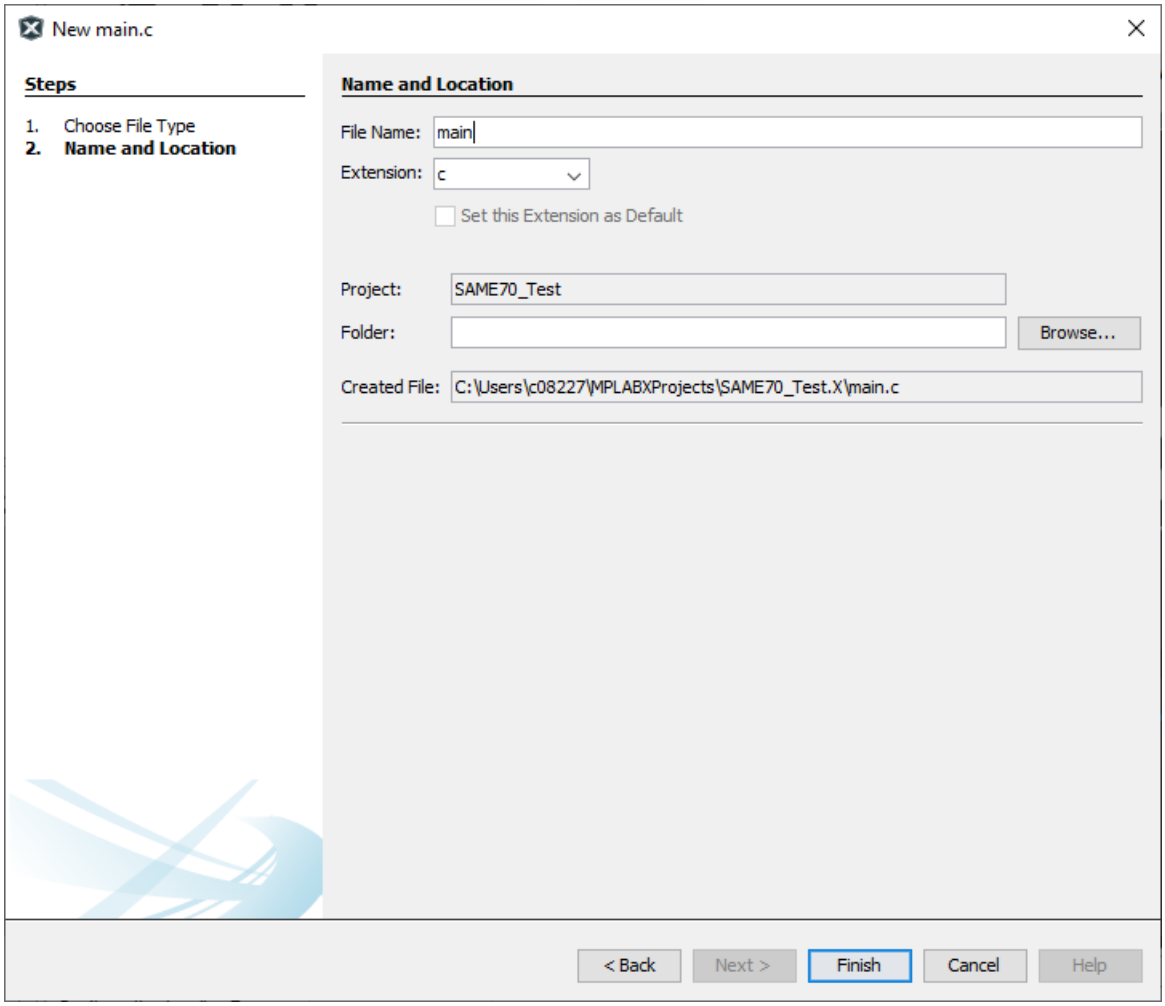
Figure 5-13. File Wizard – Choose Category and Type



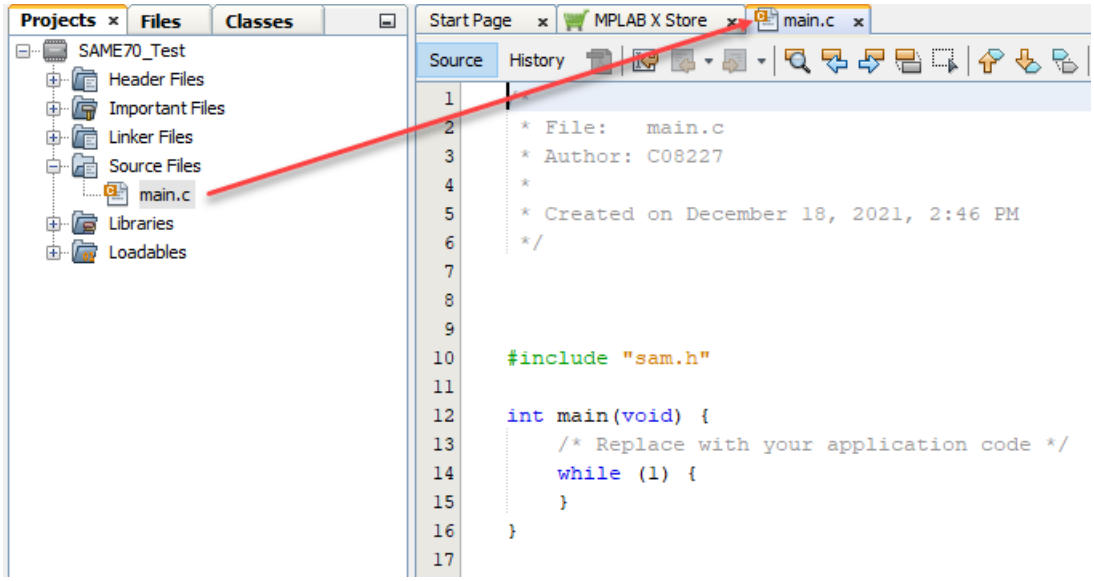
Step 2. Name and Location

Name the file and place it in the project folder. Click **Finish**.

Figure 5-14. File Wizard – Choose Associated Project



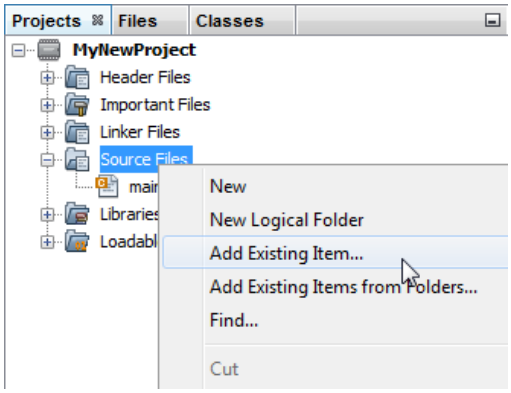
An Editor tab will appear with the file's name. Enter your file content under this tab. The text in the file will have syntax color based on its file type.



5.9.3 Add Existing Files to a Project

Existing files can be added to a project by doing one of the following:

- Right click on the project in the Project/File window and select “Add Existing Item.” Once added, you can drag and drop the file into a folder if you wish.
- Right click on a folder (e.g., “Source Files”) in the Project/File window and select “Add Existing Item.”



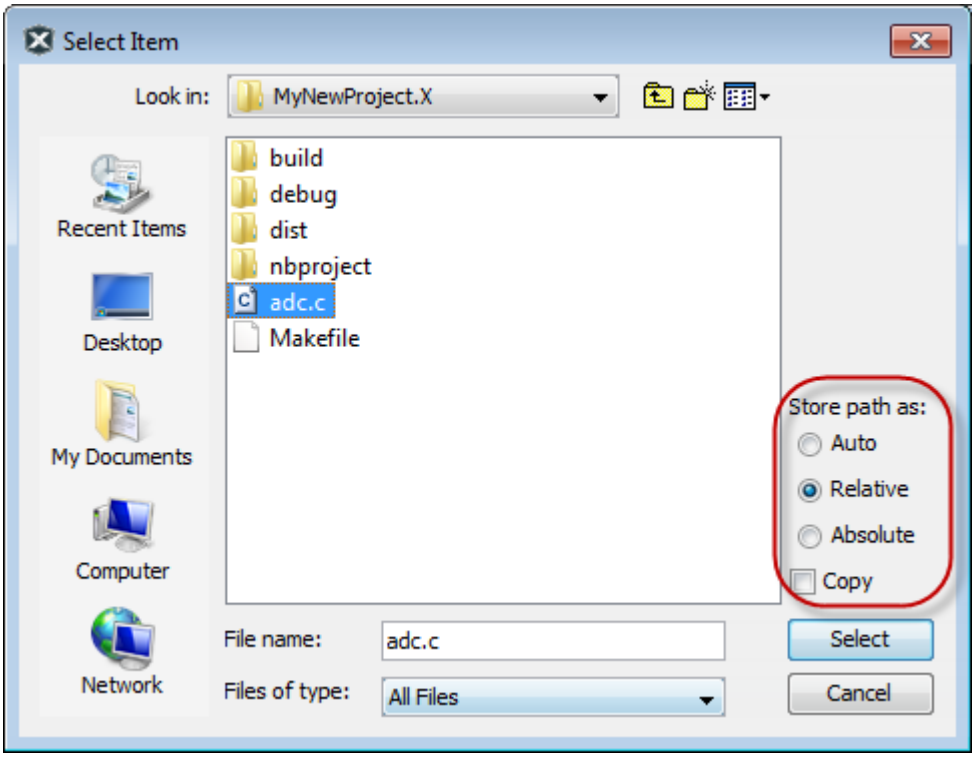
5.9.3.1 Add Files in the Project Folder

When adding a file that is located under the main project folder, you can choose whether to add it as:

- Auto – let MPLAB X IDE decide how best to locate the file
- Relative – specify the file location relative to the project (the most portable project)
- Absolute – specify the file location by an absolute path
- Copy – copy the specified file to the project folder

Once you select a path mode, MPLAB X IDE will remember this setting. To change it, see “File Path Mode” under in 13.20.2. Project Options Tab.

The file will appear in the Projects window. An Editor tab with the file’s name will appear also.



5.9.3.2 Add Files Outside the Project Folder

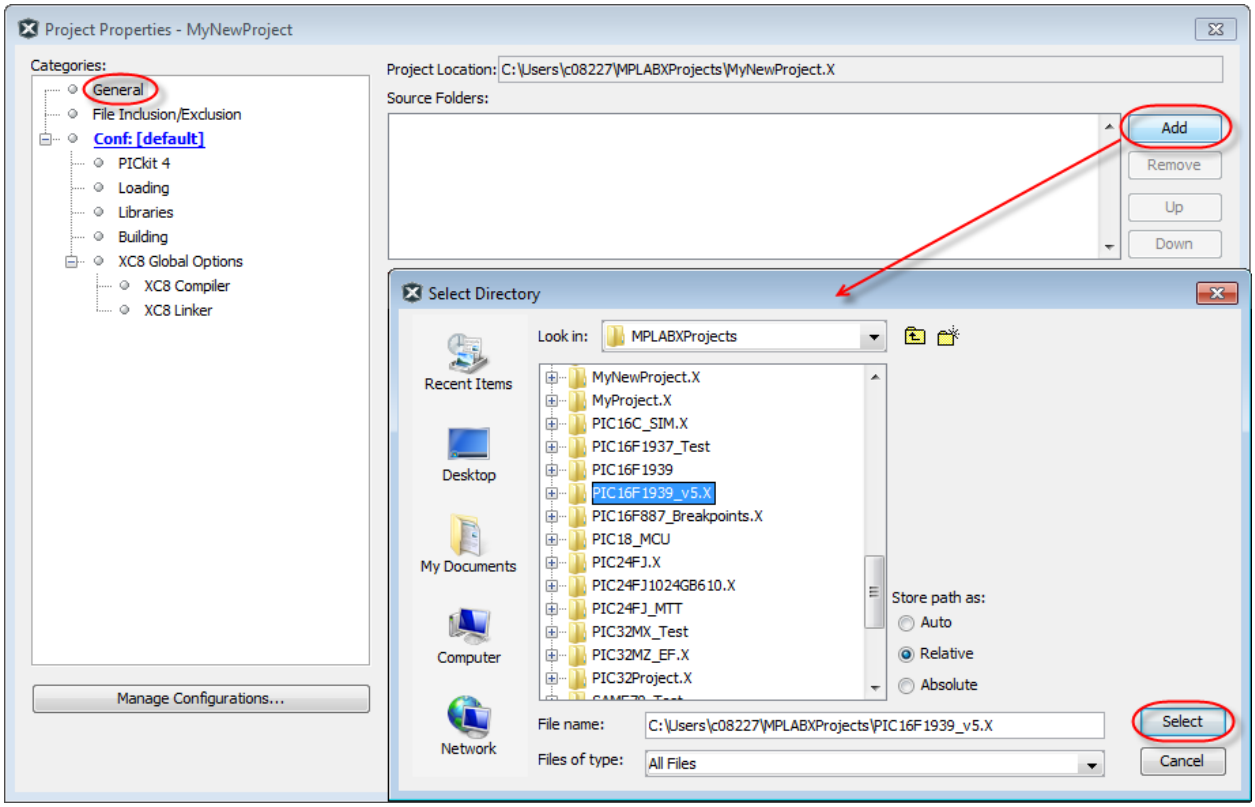
When adding a source file that is not in the project folder, add the file (store the path) as “Relative.” This creates an external folder (`_ext`) so the project can find the file when it is building.

To make use of navigation features such as the `//TODO` action item and file context menus, you must tell the project where the files are located. To do this:

1. In the Projects window, right click on the project name and select “Properties.”
2. Under “Categories,” click “General.”
3. Next to “Source Folders,” click Add.
4. Browse to the path of the external file(s) you have added to the project. Click the **Select** button.
5. Click **Apply** or **OK**. Then rebuild the project.

When you import an MPLAB IDE v8 project, the source files are not in the project folder. Use *File>Import>MPLAB IDE v8 Project* to automatically import the files.

Figure 5-15. Specify External File Path



5.9.4 Edit Code in Files

To create new code or change existing code, use the NetBeans editor. For more information about the editor, see the NetBeans Help topic:

netbeans.org/features/java/editor.html

Also see MPLAB X IDE-related information about this editor:

7. Editor

5.9.5 Add and Set Up Library and Object Files

You can reference library files to be used by the linker in the following locations:

- the Libraries folder in the Projects window
- the Project Properties window

Additional library file set up can be done in the language tool librarian.

5.9.5.1 Libraries Folder

In the Projects window, right click on the Libraries folder to see these options:

- Add Library Project
  Make a project that produces a library as its output required for your project, i.e., establish a dependency. For more on this, see the NetBeans Help topic:

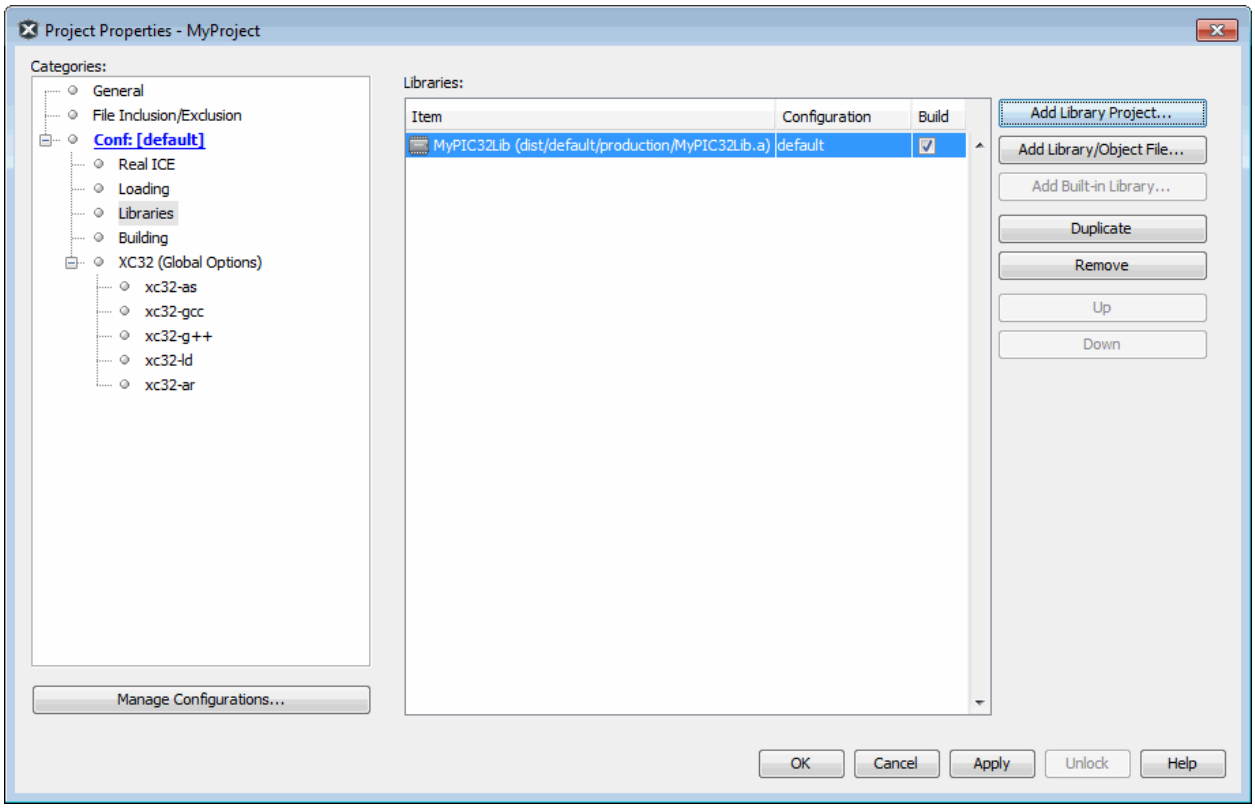
  netbeans.org/kb/73/java/project-setup.html#projects-dependencies.
- Add Library/Object File
  Add an existing library file or pre-built (object) file to the project.
- Properties
  Open the project's Properties window to select additional options.

5.9.5.2 Project Properties Window: Libraries Category

Open the Project Properties window and click on the "Libraries" category. Any files added to the Libraries folder in the Projects window will be visible here. You may add additional files and manage these files using the buttons on the right.

The order of files in the library list determines link order, which is important when the libraries included in the project reference each other. Libraries referencing objects from a secondary library should be placed higher in the link order list. An incorrect link order may result in an "undefined reference to" error during linking. For details on link order, see language tool documentation.

Figure 5-16. Manage Library/Object Files and Set Up Link Order

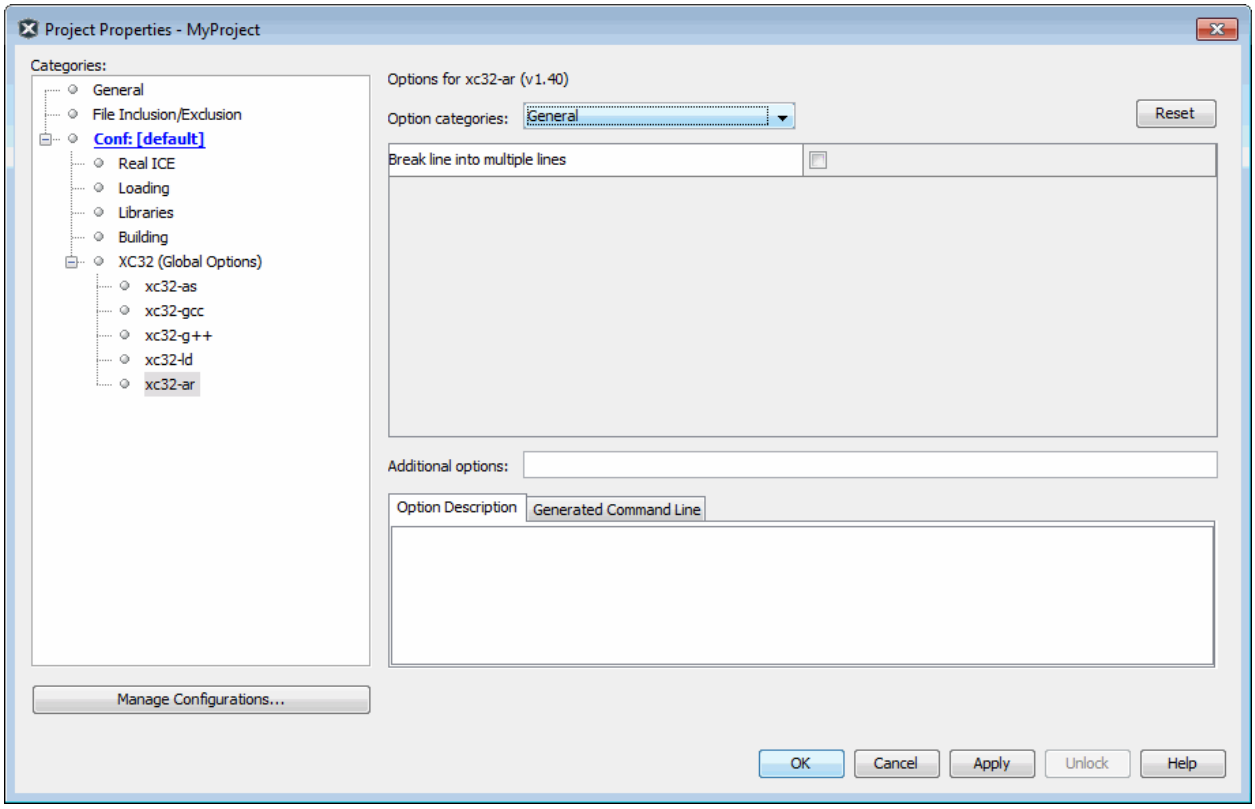


5.9.5.3 Project Properties Window: Librarian Category

Open the Project Properties window and click on the librarian/archiver under your language tool category. Consult your language tool documentation to determine the executable name of your librarian.

In the left pane, click on the name of the linker. Then, in the right pane, select the option "Libraries." Choose library options from this list.

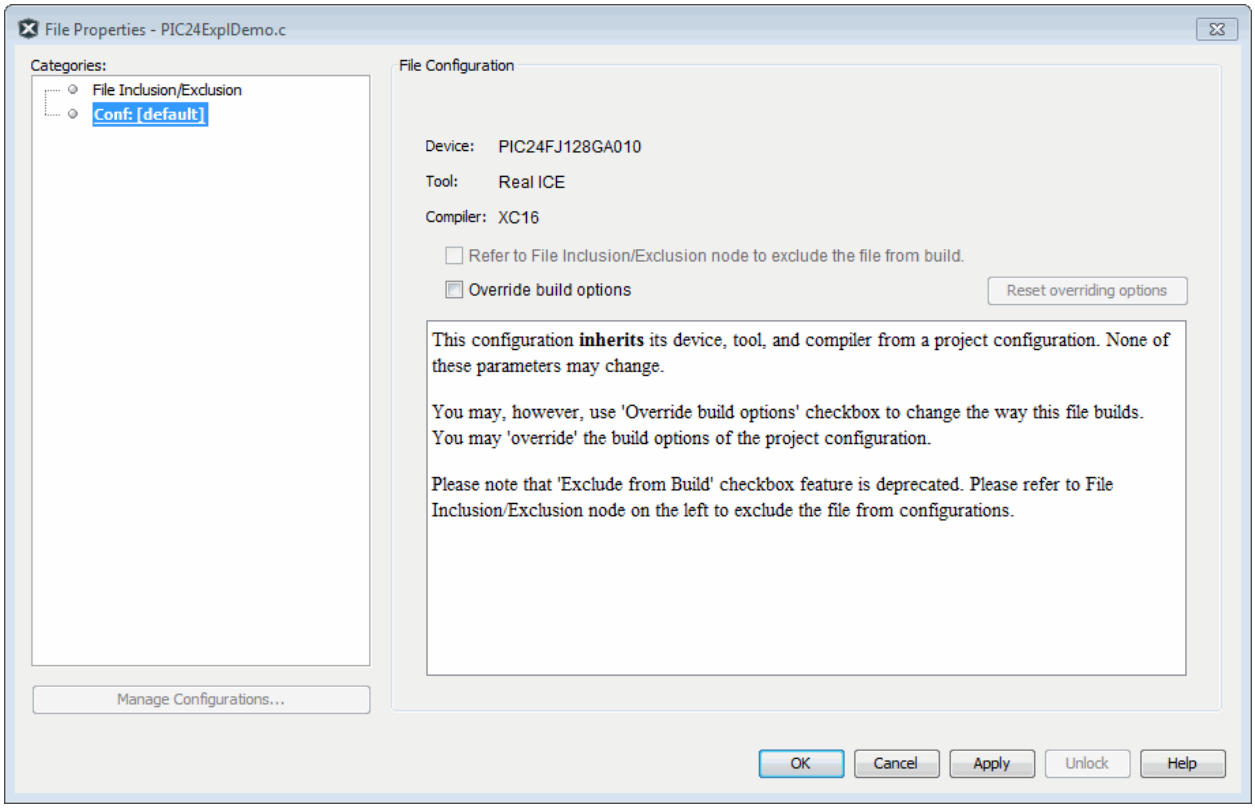
Figure 5-17. Set Up Libraries Options in the Librarian



5.9.6 Set File and Folder Properties

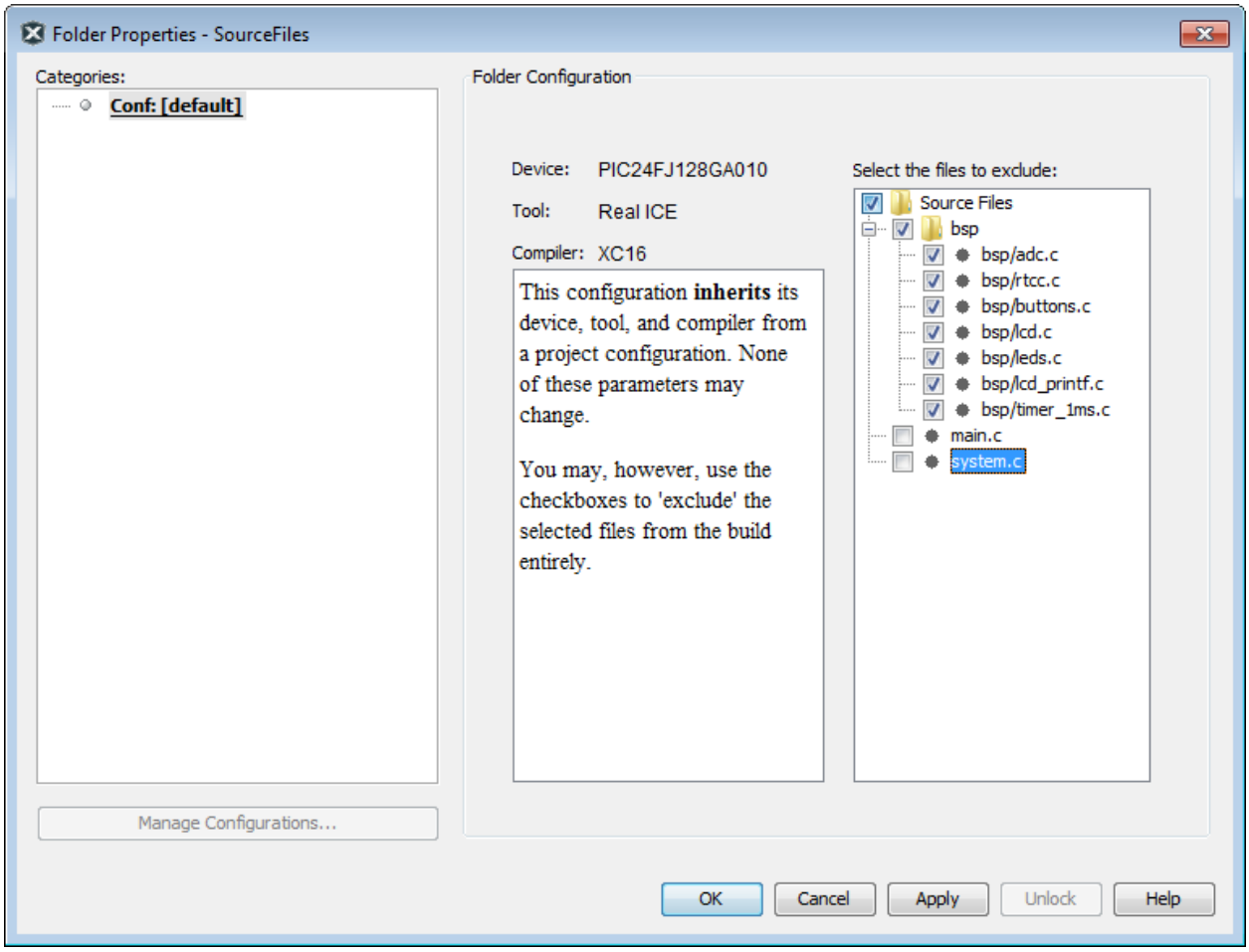
A project file can be built differently from other files in the project using the File Properties dialog. Access this dialog by right clicking on the file name in the Projects window and selecting “Properties.” Select to exclude the file from the project build or override the project build options with the ones selected here. To override build options, check the checkbox and then click “Apply” to see selection options appear under “Categories.”

Figure 5-18. File Properties



An entire (virtual) folder can be excluded from the build process by right clicking on the folder name in the Projects window and selecting “Properties.” Select to exclude folders or individual files from the build. Selecting a folder that is above other folders will select the entire contents of that folder. You can deselect files or other folders as you wish.

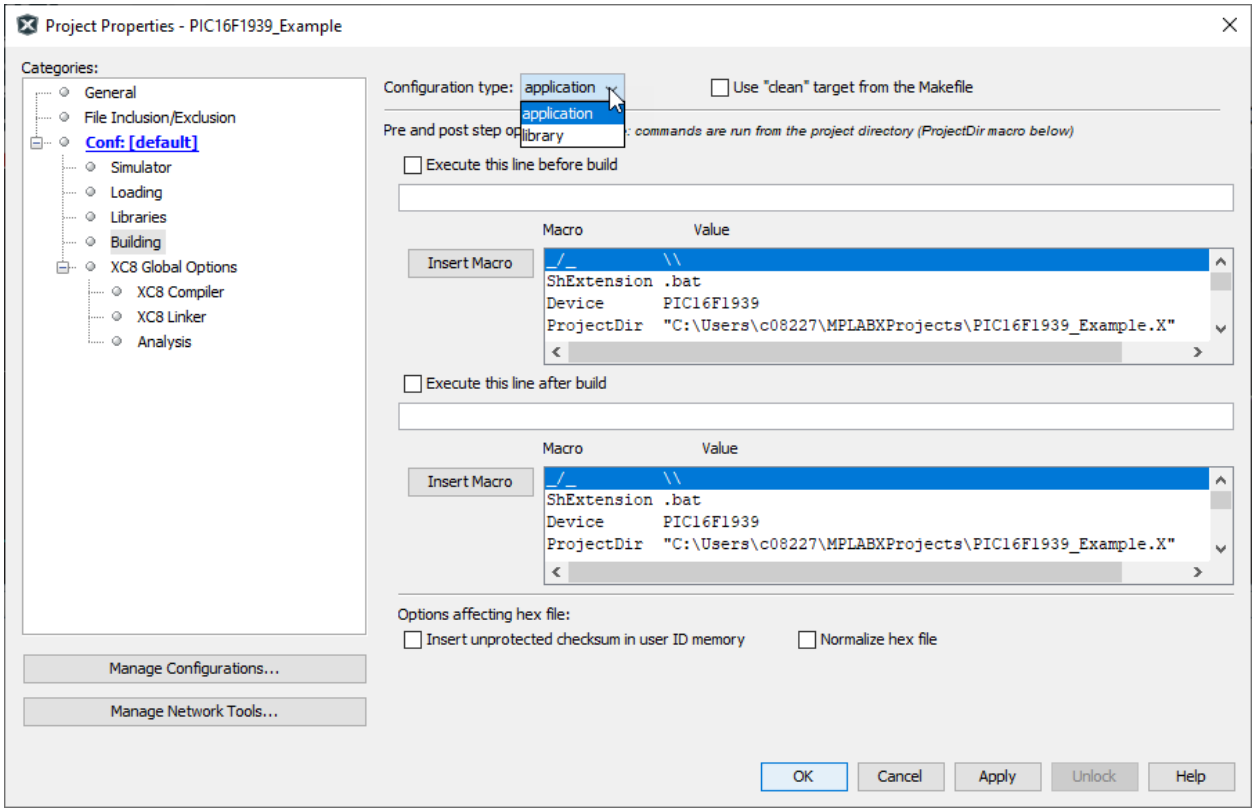
Figure 5-19. Folder Properties



5.10 Set Build Properties

Before building the project, you may want to set up additional build properties in the Project Properties window, Building category.

Figure 5-20. Build Properties



5.10.1 Change Project Configuration Type

As of MPLAB X IDE v2.15, when you create a Standalone or Library project in the New Project wizard, you are creating a project with either a configuration type “application” or “library” respectively. This means you can switch the configuration type of your existing project if you wish, with the caveats specified in the following sections.

Prebuilt and User Makefile projects are unaffected by this feature (i.e., they may not be changed to a different type of project once created).

Switch from a Standalone to a Library Project

When switching project configuration type from “application” to “library,” it is your responsibility to remove `main()`. Also, any loadables in the “application” project will not be built/linked once the configuration type is “library.” For more on loadables, see 6.2.5. Loadable Projects, Files and Symbols.

If the toolchain (i.e., XC8) does not support libraries, then the combo box will be disabled and you cannot switch the configuration to “library.”

Switch from a Library to a Standalone Project

When switching project configuration type from “library” to “application,” it is your responsibility to add `main()`.

5.10.2 Execute this line before/after the build

Type in a command to be executed at the very beginning or at the very end of the build process. These commands are inserted into the `nbproject/Makefile-$CONF.mk` file and allow you to customize the build process. If you need to refer to some of the project-related items (like the image name) in the script or program you are calling, use the supplied macros.

You can type the macros yourself or click **Insert Macro** to copy the macro name into the current position in the edit box. Commands are run in the make process with the current directory being set to the MPLAB X IDE project directory. The project directory is defined as the project that contains the `nbproject` folder.

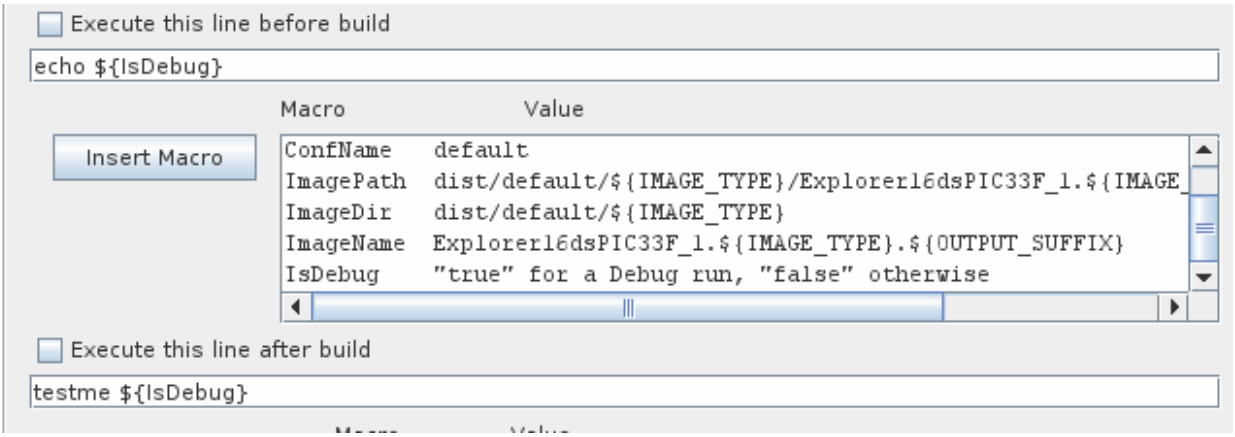
Table 5-4. Macros

Name*	Function
_/_	path delimiter based on PC operating system
ShExtension	shell extension based on PC operating system
Device	device for the currently-selected project configuration
ProjectDir	location of the project files on the PC
ProjectName	name of the currently-selected project
ConfName	name of the currently-selected project configuration
ImagePath	path to the build image
ImageDir	directory containing the build image
ImageName	name of the build image
IsDebug	“true” for Debug mode; “false” otherwise
* Additional macros may be available, depending on which compiler is used.	

Example: Execute a Process Only During Debug

Click the “IsDebug” macro (as shown in the window below) to select it, then click **Insert Macro** to insert it into a script.

Figure 5-21. Build Properties – Pre/Post Build



Check the value of `$1` (Linux or macOS) or `%1` (Windows OS) in the script being run.

Bash Code Example

```
#!/bin/bash
echo is $1
if [ "$1" == "true" ]; then
echo We are in debug
else
echo We are in production
fi
```

Batch Code Example

```
@echo off
if "%1" == "true" goto debug
:production
@echo We are in production
```

```
goto end
:debug
@echo We are in debug
:end
```

5.10.3 Insert unprotected checksum in user ID memory

If it is supported by the device (not grayed out), click the check box to use the checksum number that was generated by the build as the user ID.

For the PIC32 family of devices, the checksum is dependent on the USER ID. Therefore, this feature is not supported under PIC32 devices.

5.10.4 Normalize hex file

A hex file is normalized when line addresses are in incremental order and data is buffered to be one size (16 bytes), where possible.

For example, the following lines (records) are from the file `myfile.hex`, output from the MPLAB XC16 C compiler/linker:

```
:0801f800361e0000361e000057
:020000040000fa
:10020800361e0000361e0000361e0000361e000096
```

When this file is normalized, the file data looks like this:

```
:10022800361E0000361E0000361E0000361E000076
:10023800361E0000361E0000361E0000361E000066
:10024800361E0000361E0000361E0000361E000056
```

which is of the format:

```
:bbaaaarrdd...ddcc
```

where:

:	Start record
bb	Byte count
aaaa	Address
rr	Record type
dd	Data
cc	Checksum

In MPLAB X IDE, the application Hexmate is used to normalize Intel hex files. When the “Normalize hex file” option is checked, the following is called after a build:

```
hexmate myfile.hex -omyfile.hex
```

Essentially Hexmate unpacks the entire hex file and arranges the data at the addresses specified by the hex file. It then repackages the resulting memory image into a new hex file. In the resulting file, all the data is in ascending order and contiguous. If there is a gap in the addresses, then there will also be a gap in the output file (there is no filling of unused addresses).

For more information on using Hexmate, see MPLAB XC8 documentation or *Hexmate User’s Guide* (DS50003033) on the Microchip website.

A normalized hex file is useful for programming code (such as a bootloader) over a serial connection, as the variation of record bytes is minimized.

© 2022 Microchip Technology Inc. and its subsidiaries

5.11 Build a Project

For MPLAB X IDE, it is not necessary to build the project first and then run or debug. Building is part of the run and debug processes. For initial development or major changes, however, you may want to make sure that the project builds before attempting to run or debug.

To build a project:

- In the Projects window, right click on the project name and select “Build.” You may also select “Clean and Build” to remove intermediary files before building.
- Click on the “Build Project” or “Clean and Build Project” toolbar icon.

Build progress will be visible in the Output window.

Available build functions are:

Table 5-5. Build Options on Toolbar Buttons

Button Icon	Function	Description
	Build Project	Make all the files in the project.
	Build for Debugging	Make all the files in the project and add a debug executive to the built image.
	Build with PRO Comparison	If you use an MPLAB XC C compiler in free mode, you can build in PRO mode as well and see a comparison output of differences, if any exist. "See also "Compiler Advisor in MPLAB X IDE User's Guide (DS-50003215)."
	Clean and Build	Remove previous build files and make all the files in the project.
	Clean and Build for Debugging	Remove previous build files and make all the files in the project. Add a debug executive to the built image.
	Clean and Build with PRO Comparison	Remove previous build files and then Build with PRO Comparison.

To view errors in the Output window:

1. Right click in the Output window and select “Filter.”
2. In the Filter dialog, check “Match Case” and enter “: error” to show only errors in the Output window that stopped the build.
3. Toggle the filter on and off using <Ctrl>+<G>.

See you language tool documentation for a discussion of errors.

To view checksum information:

Open the Dashboard window (see 8.5. View the Dashboard Display) to see the checksum after a build.

5.12 Run Code

After the code builds successfully, you can run the application.

How Run Works

When you click the “Run Project” toolbar icon  , the following occurs:

- A build is done, if the make process determines that it is necessary.
- For in-circuit debuggers/emulators and programmers, the target device is automatically programmed with the image (without the debug executive) and the device is released to run, i.e., no breakpoints or other debug features are enabled.

Note: To hold a device in Reset after programming, click the “Hold in Reset” icon  instead of “Run Project.”

- For simulators, the application executes, without debugging capability.

Run progress is visible in the Output window.

Running Your Application Code

1. In the Projects window, select your project or make it the main project (right click on the project and select “Set as Main Project”).
2. Run your program by either:
   - Clicking on the “Run Program” icon  .
   - Clicking on the “Make and Program Device” icon  .

Run progress is visible in the Output window.

5.12.1 Run Considerations

When running your program, consider that MPLAB X IDE operation connects to the hardware tool at runtime (Run or Debug). Consequently, settings made in MPLAB X IDE are passed to the tool only at runtime. Settings changed during a halt are updated only when runtime is initiated again.

To always be connected to the hardware tool (like MPLAB IDE v8), see *Tools>Options* (*MPLAB X IDE>Preferences* for macOS), **Embedded** button, **Generic Settings** tab, “Maintain active connection to hardware tool” checkbox.

See also 5.21. Program a Device.

5.13 Debug Code

If your code does not run successfully, you should debug it.

How Debug Works

When you click the “Debug Project” toolbar icon  , the following occurs:

- A build is done, if the make process determines that it is necessary.
- For in-circuit debuggers/emulators, the target device or header is automatically programmed with the image (including the debug executive) and a debug session is started.
- For simulators, a debug session is started.

Debug progress is visible in the Output window.

Debugging Your Application Code

To debug your application code:

1. In the Projects window, select your project or make it the main project (right click on the project and select “Set as Main Project”).
2. Click on the “Debug Project” icon  or “Step Into” icon to begin debugging.

To halt your application code:

Click on the “Pause” icon  to halt your program execution.

To run your code again:

Click on the “Continue” icon  to start your program execution again.

To end execution of your code:

Click on the “Finish Debugger Session” icon  to end your program execution.

To launch the debugger:

If your code is built for debugging and you simply want to launch the debug tool, you can do so by selecting the down arrow next to the “Debug Project” icon and select “Launch Debugger.”

Table 5-6. Debug Icons

Icon	Function	Related Menu Item
	Debug Project	Debug>Debug Project
	Step Into	Debug>Step Into
	Pause	Debug>Pause
	Continue	Debug>Continue
	Finish Debug Session	Debug>Finish Debugger Session

5.13.1 Debug Macros Generated

MPLAB X IDE generates debug macros for use with Microchip language tools. Macros passed to Microchip compilers and assemblers are shown in the table below.

Table 5-7. Microchip Tools Debug Macros

Macro Name	Assoc. Tool	Function
__DEBUG	All	specifies that this is a debug build
__MPLAB_<toolname>__ where <toolname> is: • REAL_ICE • ICD4 • ICD3 • PK4 • PK3 • PICKIT2	XC8	specifies the hardware debug tool in use
__MPLAB_DEBUGGER_<toolname> where <toolname> is the same as for XC8.	XC16, XC32	specifies the hardware debug tool in use
__MPLAB_DEBUGGER_<skname> where <skname> represents the starter kit, i.e., PIC32MXSK.	XC32	specifies the starter kit in use

You can use these macros in your own code. For example:

```
#ifdef __DEBUG
fprintf(stderr,"This is a debugging message\n");
#endif
```

5.13.2 Debug Considerations

When debugging your code, consider the following issues:

- You need to be in a debug session (debug mode) to activate many debugging features – for example, to view variable values in the watch or memory windows.
- MPLAB X IDE operation is always connected to the hardware tool. To connect only at runtime, see *Tools>Options* (*MPLAB X IDE>Preferences* for macOS), **Embedded** button, **Generic Settings** tab, and uncheck “Maintain active connection to hardware tool.”
  When unchecked, settings made in MPLAB X IDE will be passed to the tool only at runtime. Settings changed during a halt are updated only when runtime is initiated again.
- For some applications you may need to break down the debug steps for independent execution. To do this, use the steps under *Debug>Discrete Debugger Operation*.
- Disable compiler optimizations when debugging or use a ‘debug’ optimization if available for your compiler. In the Project Properties window, select the language tool executable under “Categories.” Then select “Optimizations” from “Options categories” and choose the lowest level, i.e., “0” if available.

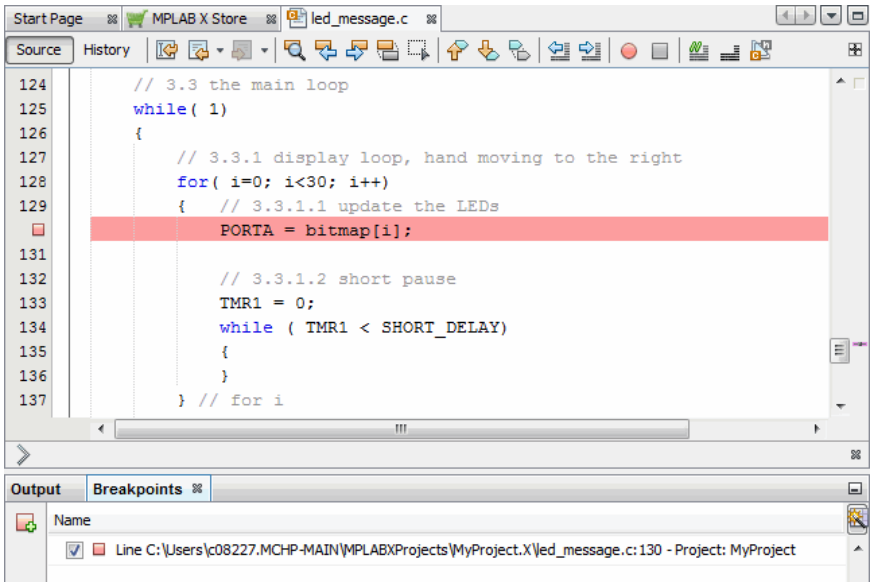
5.14 Control Program Execution with Breakpoints

When debugging code, it can be useful to suspend execution at a specific location in code so that variable values can be examined. To do this, use breakpoints.

For more on breakpoints, see the NetBeans Help topic:

netbeans.org/kb/docs/cnd/debugging.html#breakpoints

Figure 5-22. Breakpoint and Breakpoints Window



Related Links

13.3. Breakpoints Window

13.4. New Breakpoint Dialog

5.14.1 Choose Hardware or Software Breakpoints

Hardware breakpoints use device resources, that is internal registers, to perform a code break or halt. Therefore the number of breakpoints varies by device.

Software breakpoints replace an opcode instruction with one that causes a break or halt. Therefore the number of breakpoints is unlimited.

To help you decide which type of breakpoints to use (hardware or software) the following table compares the features of each.

Table 5-8. Hardware vs. Software Breakpoints

Feature	Hardware Breakpoints	Software Breakpoints
Number of breakpoints	Limited	Unlimited
Breakpoints written to*	Internal debug registers	Flash Program Memory
Breakpoints applied to**	Program Memory/Data Memory	Program Memory only
Time to set breakpoints	Minimal	Dependent on oscillator speed, time to program Flash Memory and page size
Breakpoint skidding	Most devices. See the “Breakpoints” section under General Debug Limitations for details.	No
* Where information about the breakpoint is written in the device. ** What kind of device feature applies to the breakpoint. This is where the breakpoint is set.		

To select hardware or software breakpoints:

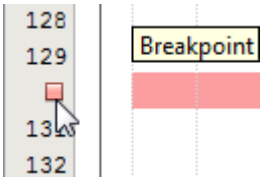
1. Right click on the project name and select “Properties.”
2. In the Project Properties dialog, select the hardware tool (PKOB nano) under “Categories.”
3. Under “Option Categories” select “Debug Options.”
4. Check “Use software breakpoints” to use software breakpoints. Uncheck to use hardware breakpoints.
   Note: Using software breakpoints for debug impacts device endurance. Therefore, it is recommended that devices used in this manner not be used as production parts.

5.14.2 Set or Clear a Simple Line Breakpoint

To set or clear a breakpoint on a line, do one of the following:

- Click the left margin of the line in the Source Editor (see figure).
- Click in the line and then press Ctrl+F8.
- Select *Debug>Toggle Line Breakpoint.*

Figure 5-23. Click to Enable/Disable Breakpoint



5.14.3 Set Breakpoints with the Breakpoint Dialog

To open the New Breakpoint dialog, complete the following steps:

1. Click on the “Create New Breakpoint” icon  in the upper left of the Breakpoints window.
2. Select *Debug>New Breakpoint*. Choose a breakpoint type and set up options.

Table 5-9. Breakpoint Types

Selection	Description
Line	Break on a line of code in an Editor window.
Data	Break on Reads/Writes to an address in data (RAM) memory. This includes SFR and symbol locations.
Address	Break on the execution of an address in program (Flash) memory.
Event	Break on a specific event, such as a reset, sleep/wake, watchdog timer time-out, etc.
Function	Break on entry into a specified function.

For options available for each breakpoint type, see 13.4. New Breakpoint Dialog.

5.14.4 Set or Clear Breakpoints in the Breakpoints Window

To view and toggle the breakpoint in the Breakpoints window, complete the following steps:

1. Toggle the breakpoint by checking/unchecking the checkbox.
2. Select *Window>Debugging>Breakpoints*.

5.14.5 Set a Breakpoint Sequence (Device Dependent)

A breakpoint sequence is a list of breakpoints that execute, but do not halt, until the last breakpoint is executed. Sequenced breakpoints can be useful when there is more than one execution path leading to a certain instruction and you only want to exercise one specific path.

To **create a Breakpoint Sequence**, complete the following steps:

1. Right click on an existing breakpoint or shift click to select a group of existing breakpoints and right click on the group.
2. From the pop-up menu, go to “Complex Breakpoint” and select “Add a New Sequence.”
3. Enter a name for your sequence in the dialog box and click OK.
4. The breakpoint(s) will appear under the new sequence.
5. To add additional existing breakpoints to the sequence, right click on the breakpoint and select *Complex Breakpoint>Add to Name*, where *Name* is the name of the sequence.
6. To add new breakpoints to the sequence, right click on the sequence and select “New Breakpoint.”

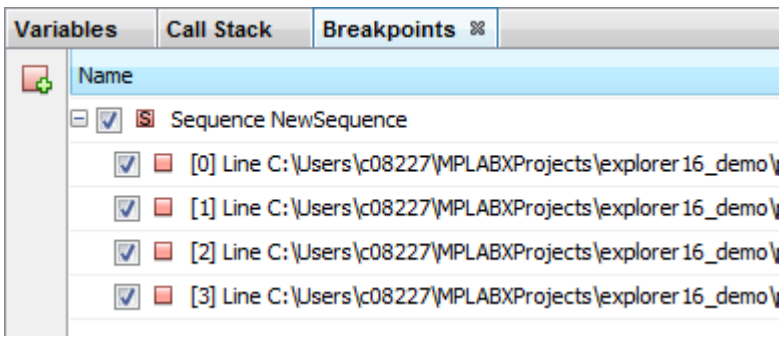
To **select the Sequence Order**, complete the following steps:

1. Expand on a sequence to see all items.
2. Right click on an item and select *Complex Breakpoints>Move Up or Complex Breakpoints>Move Down*. Sequence execution of breakpoints is bottom-up; the last breakpoint in the sequence occurs first.

To **Remove a Sequence** or Breakpoint, either:

- Right click on the item and select “Disable” to temporarily remove the item.
- Right click on the item and select “Delete” to permanently remove the item.

Figure 5-24. Breakpoint Sequence Example



5.14.6 Set a Breakpoint Tuple (Device Dependent)

For MPLAB X IDE, a tuple represents an ANDed list of breakpoints. ANDed breakpoints can be useful when a variable is modified in more than one location and you need to break only when that variable is modified in one particular location.

Only two breakpoints can be ANDed and these must consist of one program memory breakpoint and one data memory breakpoint. Breakpoint 1 AND breakpoint 2 must occur at the same time before a program halt.

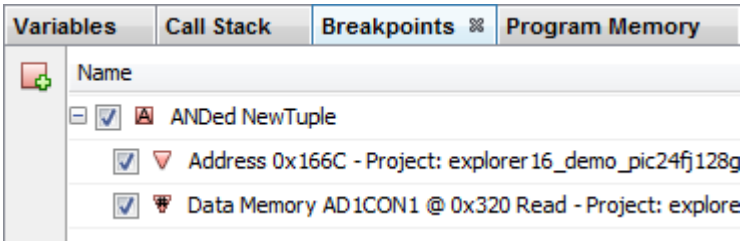
To **Create a Breakpoint Tuple**:

1. Click on the “Create New Breakpoint” icon  in the upper left of the Breakpoints window to open the New Breakpoint dialog.
2. Create an address breakpoint. Click **OK** to add it to the Breakpoints window.
3. Repeat steps 1 and 2 to create a data breakpoint.
4. Right click on one breakpoint and select *Complex Breakpoint>Add to New Tuple*.
5. Enter a name for your tuple in the dialog box and click **OK**.
6. The breakpoint will appear under the new tuple.
7. Right click on the other breakpoint and select *Complex Breakpoint>Move to Name*, where *Name* is the name of the tuple.

To **Remove a Tuple** or Breakpoint, do one of the following:

- Right click on the item and select “Disable” to temporarily remove the item.
- Right click on the item and select “Delete” to permanently remove the item.

Figure 5-25. Breakpoint Tuple Example



5.14.7 See Breakpoint Address(es)

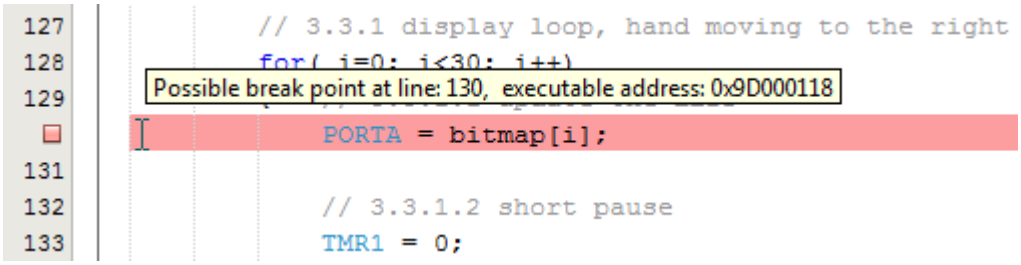
To see the address of a breakpoint before and during a debug session, complete the following steps:

1. Under *Tools>Options*, **Embedded** button, **Generic Settings** tab, select an action for “On mouseover source lines in editor, evaluate break point status.”
2. Set a line breakpoint.
3. Click the Debug Project icon to debug your code.
4. Place the cursor between a breakpoint icon and the beginning of source code, and perform the action of step 1 to see the mouseover.

Single Breakpoint Address

In many cases, you will see a single address. Open a program/execution memory window (*Window>Target Memory Views*) to see the memory address and instructions at that location.

Figure 5-26. See Breakpoint Address

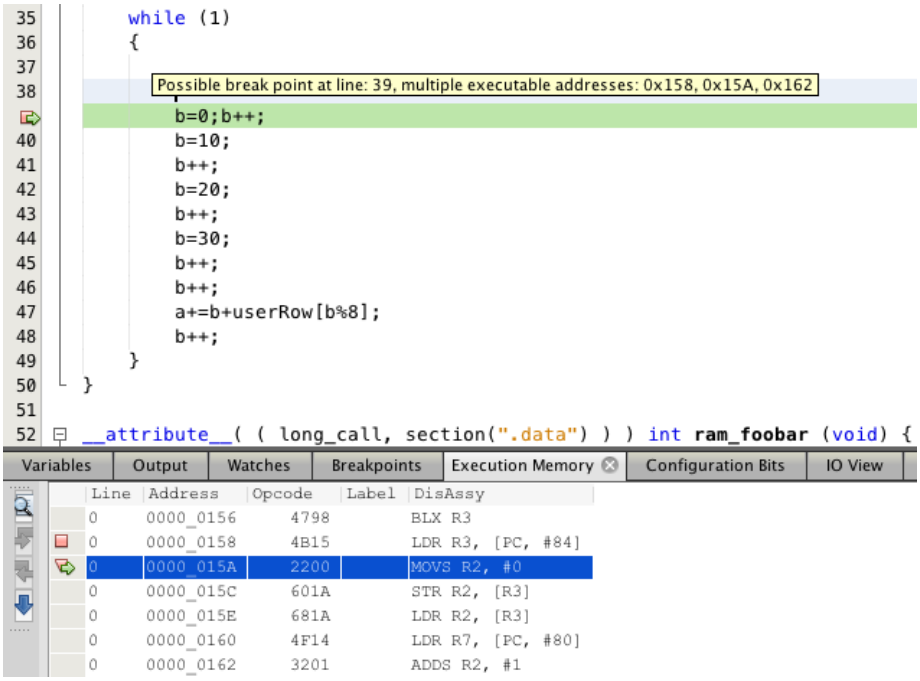


Multiple Breakpoint Addresses

In some cases, you will see multiple possible breakpoint locations. A possible cause for this could be higher compiler optimizations (-Os). Previous to MPLAB X IDE v5.20, only the first address was shown. Now all addresses are.

Open a program/execution memory window (*Window>Target Memory Views*) to see each memory address and instructions at that location. If your breakpoint is not working, you can select other locations in the memory window. When the breakpoint hits, it will still show at the same line number in code.

Figure 5-27. See Multiple Breakpoint Addresses



5.14.8 Hardware Breakpoint Usage

If you must be halted at a line of code to:

- switch from a software breakpoint to a hardware breakpoint
- set a hardware breakpoint

Be aware that debugging will cause a halt on the same line of code. Debug again to continue program execution or perform a single step to execute the code on the breakpoint line.

5.14.9 Dual Partition Devices and Breakpoints

Due to the nature of debugging, dual partition device breakpoints must be set in the active partition once the debugger has halted there. So starting in partition 1, you cannot set breakpoints in partition 2. You need to run up to the switch, switch to partition 2, and then set breakpoints that will reside in partition 2.

© 2022 Microchip Technology Inc. and its subsidiaries

5.14.10 Breakpoint Applications

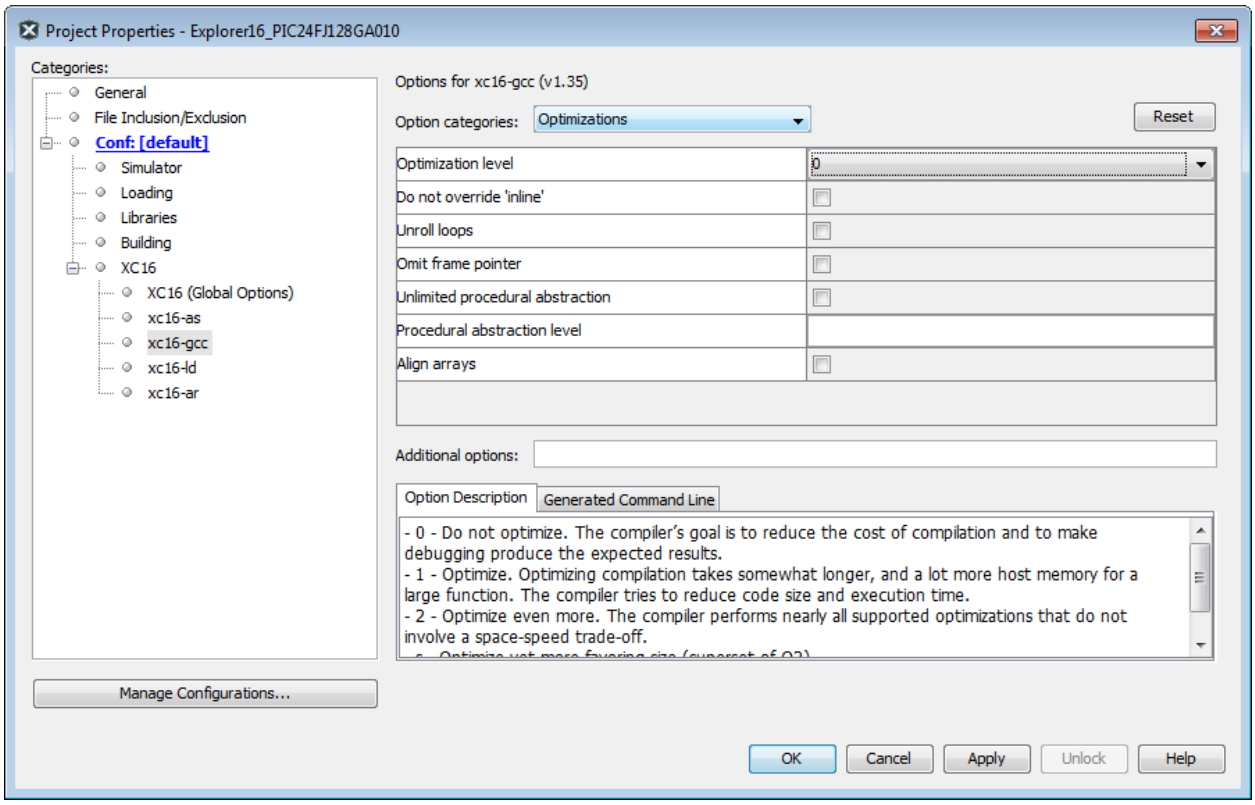
To determine the timing between breakpoints use the stopwatch (see 8.1. Use the Stopwatch).

To determine breakpoint resources open the Dashboard window (see 8.5. View the Dashboard Display) to see the number of available and used breakpoints and whether software breakpoints are supported.

5.14.11 Compiler Optimizations and Breakpoints

Compiler optimizations can change how a program is executed. This can affect memory address mapping of breakpoints, and produce multiple address possibilities. When debugging code, it is recommended that you use the lowest level optimizations to avoid breakpoint issues.

Figure 5-28. Lowest Optimization Level Example



5.14.12 Breakpoint Resource Usage

Hardware debuggers (e.g., in-circuit debug tools) support a limited number of breakpoints. The number of breakpoints available is dependent on the device selected. To see the number of breakpoints available and keep track of the number you have used, see the Dashboard window (8.5. View the Dashboard Display).

The following MPLAB X IDE features use breakpoints to accomplish their functions.

	Step Over
	Step Out
	Run to Cursor
	Reset to Main

If you attempt to use one of these features when no breakpoints are available, a dialog will be displayed telling you that all resources are used.

5.14.13 Breakpoint Awareness

Using tool tips and icons, breakpoint information is presented in and out of a debug session:

- When out of a debug session and no build has occurred - if you try to set a breakpoint, a mouse tool tip displays with a blue torn icon stating no debug information is available.
- When out of a debug session and a build has occurred - if a line has been optimized out, a breakpoint can either be possible or not possible.
- When in a debug session - the tool tip shows whether a breakpoint can be set or not.

The option to show a tool tip can be disabled if you do not want it (*Tools>Options*, **Embedded**, “Generic Settings”).

5.14.14 Broken Breakpoint Icon

There are occasions when the breakpoint icon will appear broken ( ):

- When out of a debug session and no build has occurred, if you try to set a breakpoint a mouse tool tip will occur with a torn icon stating no debug information is available.
- When in a debug session, source files may not be able to be found due to path, file or folder naming restrictions (see Path, File and Folder Name Restrictions).

5.15 Step Through Code

Use one of the stepping functions on the Debug menu and Debug toolbar to move through code either from the beginning of code or after a breakpoint halt. Examine changes in variable values (see next section) or determine if the program flow is correct.

There are several ways to step through code:

	**Step Over** – Executes one source line of a program. If the line is a function call, it executes the entire function and stops.
	**Step Into** – Executes one source line of a program. If the line is a function call, it executes the program up to the function’s first statement and stops.
	**Step Out** – Executes one source line of a program. If the line is a function call, it executes the functions and returns control to the caller.
	**Run to Cursor** – Runs the current project to the cursor’s location in the file and stops program execution.

In addition to the Editor window, you can single-step through code in the Disassembly window (8.2. View the Disassembly Window) and Program/Execution Memory window.

5.16 Watch Symbol Values Change

Watch the values of symbols that you have selected change in the Watches window. Knowing whether these values are set as expected during program execution will help you to debug your code.

The following symbols may be added to a Watches window:

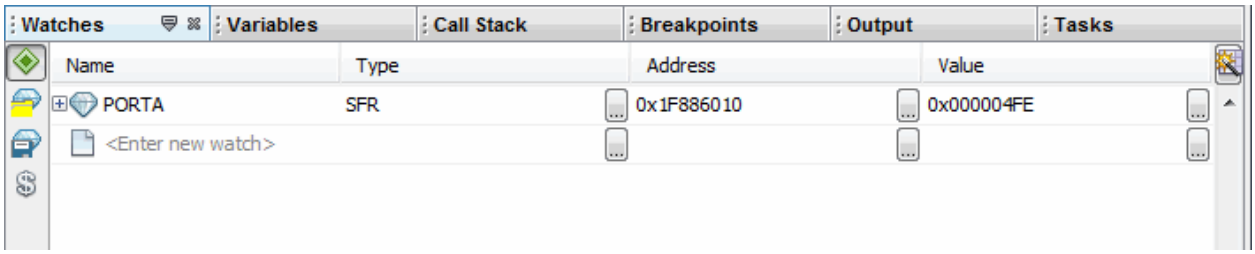
- Global symbols – visible after a build
- SFRs – special function registers (device dependent)
- Absolute addresses

In general, you must Pause from debugging to be able to see updated values. However, some tools allow runtime updates (you can see the values change as your program is running). Check your tool documentation to see if it supports this feature.

For all devices except PIC32 MCUs, symbols used in a runtime watch must be sized to match the device memory. That is, you need 8-bit symbols when using an 8-bit device.

For C-language enumerated types, you may enter either the enum label (text) or integer value in the window. For labels, be aware that they are case sensitive.

Figure 5-29. Watches Window – Program Pause



To view the Watches window do one of the following:

- Select *Window>Debugging>Watches* to open the window.
- If the window is already open, click the Watches tab in the Output window.

To create a new watch directly:

You can add a symbol directly to the Watches window by doing one of the following:

- Double click in the name column and type in a global symbol, SFR, or absolute address (0x300).
- Right click on a global symbol or SFR in the Editor window and select “New Watch.”
- Select the global symbol or SFR in the Editor window and drag-and-drop it into the Watches window.

To create a new watch using the New Watch dialog:

You can add a symbol or SFR to the Watches window by doing one of the following:

- Right click in the Watches window and select “New Watch” or select *Debug>New Watch*. Click the selection buttons to see either Global Symbols or SFRs. Click on a name from the list and then click **OK**.
- Select the symbol or SFR name in the Editor window and then select “New Watch” from the right click menu. The name will be populated in the window. Click **OK**.

To create a new runtime watch:

Before you add a runtime watch to the Watches window, you need to set up the clock:

1. Right click on the project name and select “Properties.”
2. Click on the debug tool name (e.g., PICkit 4), and select the “Clock” option category.
3. Set the runtime instruction speed.

To add a global symbol or SFR as a runtime watch, follow the instructions under To create a new watch using the New Watch dialog:, except select “New Runtime Watch” instead of “New Watch.”

For all devices except PIC32 MCUs, symbols used in a runtime watch must be sized to match the device memory. That is, you need 8-bit symbols when using an 8-bit device.

To view symbol changes:

1. Debug and then Pause your program.
2. Click the Watches tab to make the window active.
3. For watch symbols, continue the debug session and Pause to see changing values. For runtime watch symbols, continue Debug and watch the values change as the program executes.

You must be in a debug session to see the values of symbols – global symbols, SFRs, arrays, register bitfields, etc.

Note: You cannot view the value of macros in this way. Use the Macro Expansion Window by right clicking on the macro in the editor during debug.

To change the radix of a watch symbol:

Right click in the line of the symbol and select “Display Value As.”

To perform other tasks:

For more on watches, see 13.24. Watches Window.

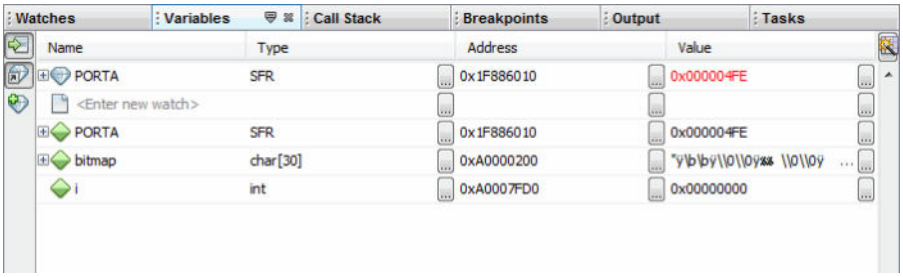
5.17 Watch Local Variable Values Change

Watch the values of local variables change in the Variables window. Determining if these values are as expected during program execution will help you to debug your code.

To view the Variables window, either:

- Select *Window>Debugging>Variables* to open the window.
- Click the Variables tab in the Output window if the window is already open.

Figure 5-30. Variables Window – Program Pause



View Variable Changes

1. Debug and then Pause your program.*
2. Click the “Variables” tab to view the window and see the local variable value.

* Some tools allow runtime updates, meaning that you can see the value change as the program executes. Check your tool documentation to see if it supports this feature.

Change the Radix of a Variable:

Right click in the line of the variable and select “Display Value As.”

Understanding the Optimized Descriptor

The Variables window may show (Optimized) for local variables. This is not related to compiler settings but rather implies the variable value is not available on the current stack frame at the scope where it is evaluated. It is most commonly applicable to function `param` variables which are not used by a function at the current scope.

5.18 View Registers for the Project (I/O View)

Use the I/O View window (*Window>Debugging>IO View*) to see an overview of registers of the target device for the current project. It serves as a quick reference during design and is capable of displaying register values when the project is in debug mode.

The I/O View is a split pane view with peripherals in top pane and registers of the selected peripheral in bottom pane.

- The top pane can show registers with named editing values in combo, like Config combo.
- The bottom pane shows register words with custom bit control for editing and changed colored bits.

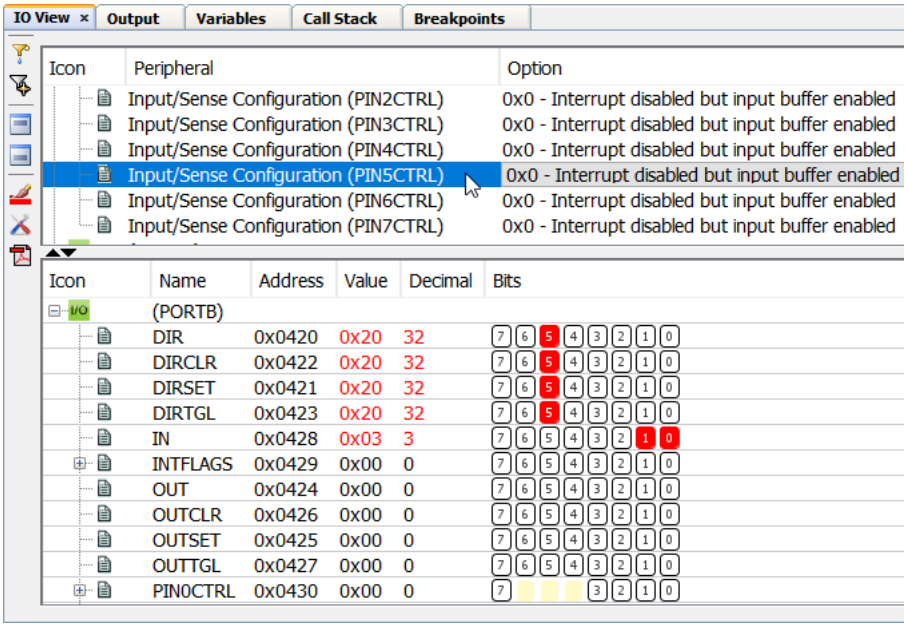
For more on this window, see 13.9. I/O View Window.

Edit During Debug Pause

When debug mode has been paused, you can edit values in the window.

© 2022 Microchip Technology Inc. and its subsidiaries

Figure 5-31. I/O View for Debug Pause



To change a bit, simply click it, and the value will be toggled. Each bit in the register is displayed in a separate column. Bits which are set will have a dark color by default, while cleared bits will have no color (default white).

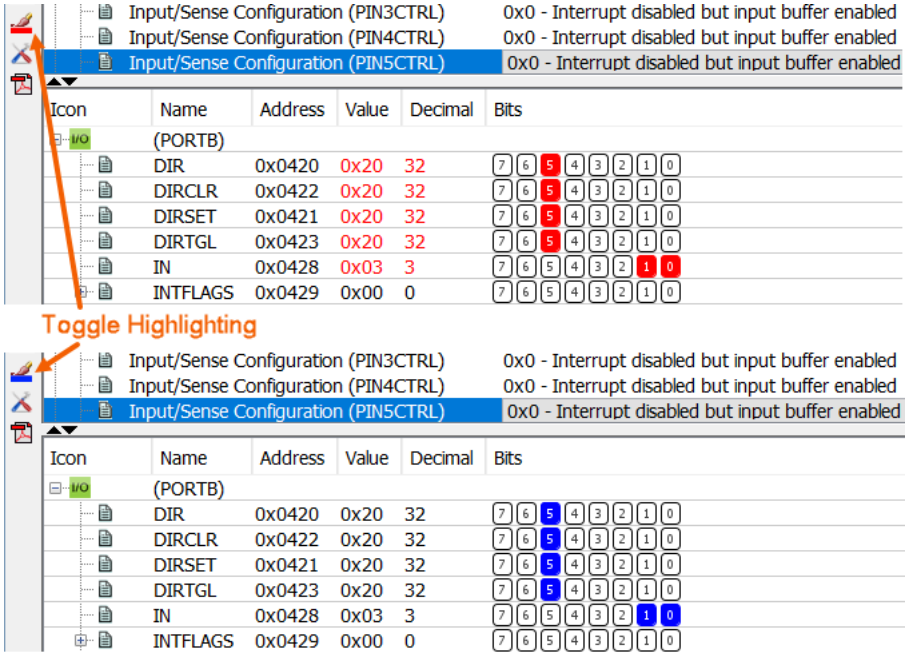
Any value can be changed by clicking the value field and writing a new value.

Some values and bits cannot be modified as they are read-only and some bits may be write-only. See the documentation for each device for more information. When a bit or value is set, it is immediately read back from the device, ensuring that the I/O View only displays actual values from the device. If a new value is set, but the I/O view does not update as expected, the register might be write-only or simply not accessible.

Change Colors

When a register has changed since last time it was displayed, it will indicate so with a red colored value and bits in the display. If a bit has been set since last time, it will be solid red. If it has been cleared it will simply have a red border. This feature can be toggled on or off in the toolbar.

Figure 5-32. I/O View - Toggle Highlighting of Modified Bits



5.19 View or Change Device Memory

MPLAB X IDE has flexible, abstracted memory windows that provide a more customized view of the different types of device memory during debug.

5.19.1 View Device Memory

To view device memory:

1. Click a window in a pane to make the pane active. The memory window will open in this pane.
2. Select a memory view from *Window>Target Memory Views*. The available choices are shown in the following tables. Not all memory types are available for all devices.

Table 5-10. Memory Views – 8- and 16-Bit Devices

Type	Description
Program Memory	all program memory (ROM) on the device
File Registers	all file register (RAM) memory on the device
SFRs	all Special Function Registers (SFRs)
Peripherals	all SFRs by Peripheral
Configuration Bits	all Configuration registers
EE Data Memory	all EE Data memory on the device
User OTP Memory	User OTP memory
User ID Memory	User ID memory

Table 5-11. Memory Views – 32-Bit Devices

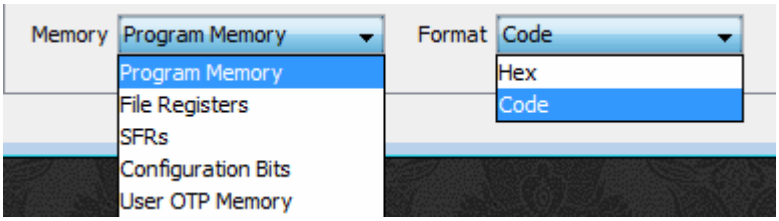
Type	Description
Execution Memory	all Flash memory on the device

.....continued

Type	Description
Data Memory	all RAM memory on the device
Peripherals	all Special Function Registers (SFRs) related to peripherals
Configuration Bits	all Configuration registers
CPU Memory	all CPU memory
User ID Memory	User ID memory

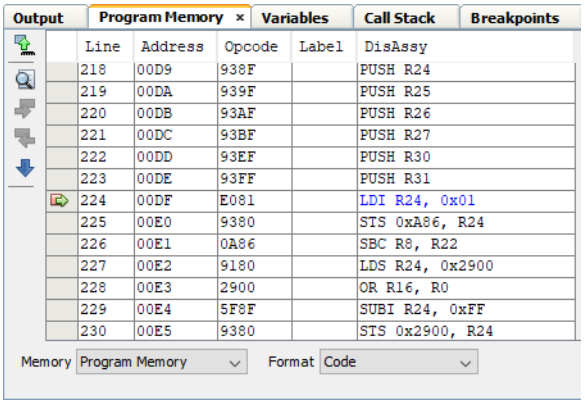
3. When a Memory window is open, you may change your view by selecting a different memory in the Memory drop-down box. Or, you can change the format of the current memory display in the Format drop-down box. The format choices depend on the type of memory chosen and the device.

Figure 5-33. Memory and Memory Format Selection



4. After Debug and then Pause, the window will populate with the memory chosen.
5. Close the window by clicking the “x” on that window’s tab.

Figure 5-34. Memory Window Content



5.19.2 Change Device Memory

You must Debug your code to change memory values. You cannot change memory when code is running.

Note: The data will change only during Debug. Your application code is not changed.

Use the following information to change memory values:

- Change a value in the Memory window by clicking in the appropriate column and selecting or entering new data. For some windows, the text will be red to show a change.
- The Fill memory feature is found on the context (right click) menu of most Memory windows.
- For program memory, you must rebuild to see the changes. Use *Debug>Discrete Debugger Operation* to program the target and launch the debugger with the changed data.

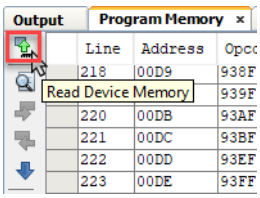
5.19.3 Set Memory Window Options with Context Menu

Right clicking in the Memory window will pop up a context menu with various options such as display options, fill memory, table import/export and output to file. The content of the menu depends on the window. See 13. MPLAB X IDE Windows and Dialogs.

5.19.4 Refresh Selected Memory Window

For Memory windows showing program, EEPROM, user ID, or configuration bits memory, you can refresh the view by doing the following:

1. If you are debugging, end your debug session unless:
   a. Your device and tool support Debug Reads (see below).
   b. Your device is a SAM MCU and your debug tool supports reading on pause (see below).
2. Click on the icon named “Read Device Memory”, either on the window (see image below) or the toolbar ().



Debug Reads

For most devices, you must halt your program (Finish Debugger Session) before you can read device memory. For some devices, you can read while in debug mode (Debug Read). You will know that this is available as the “Read Device Memory” icon will not be grayed out when you are debugging.

Currently, only MPLAB REAL ICE in-circuit emulator and MPLAB ICD 3 support Debug Reads.

Debug Reads are done at target oscillator speeds so if the target is running very slow, a read may take a long time. You can force a fast ICSP read by finishing the debug session and then doing a read since ICSP reads will always be done when not in a debug session.

Read on Debug Pause

For a SAM project device and one of the following debug tools:

- Xplained or Curiosity boards (with EDBG/nEDBG/mEDGB)
- Atmel-ICE
- PowerDebugger/
- J-32

A Pause to debug execution will allow a read of device memory, using the “Read Device Memory” button on the memory window or debug toolbar.

5.20 Set Configuration Values in the Configuration Bits Window

Select “Configuration Bits” from *Window>Target Memory Views* to open the Configuration Bits window. For details on using memory windows, see 5.19. View or Change Device Memory.

The Configuration Bits window can aid you in developing your Configuration bit settings. However, you must set Configuration bits in code for production.

5.20.1 Working with the Configuration Bits Window

You can temporarily change Configuration bits during a debug session in the Configuration Bits window (see 5.19. View or Change Device Memory.) Once you have the settings you want, you have several options to preserve these settings as discussed in the following sections.

You cannot edit the Configuration bits if you are not in a debug session. Also, if you rebuild, all changes in the Configuration Bits window will be lost.

For a summary of Configuration bits settings for different devices, see 15. Configuration Settings Summary.

Note: For AVR or SAM devices, red text in this window indicates that an action is needed. Hover your mouse over to see pop up text for instructions.

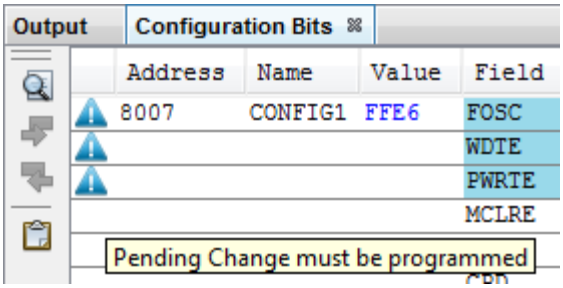
Edit Configuration Settings

Editing values in this window is always active, unlike other memory windows where you must be in debug mode. However, if you rebuild, all changes in the Configuration Bits window will be lost.

Default values of the editable fields will occur on initial entry to Configuration Bits view and also on Reset to Defaults menu action. When configuration bits are expressed in source code, the value of all editable fields will be initialized to the value assigned in source code. This occurs following the build action and after a successful load.

When you change a default value in the configuration window, you are reminded that this is a temporary change that will not be permanent until programmed into the target device.

Figure 5-35. Pending Change



You can change values as follows:

- **Selection Box Values**: Selection box values are available in the “Option” or “Setting” column. Simply click on the column value for the field that you want and select from the list to change.
- **Manual Entry Values:** Manual entry values may be available in the “Option” or “Setting” column. The “Option” text will state that there is a user selectable range, e.g., “User range: 0x0 - 0x3F.” The “Setting” text will state that a value needs to be entered, e.g., “Enter Hexadecimal value.”

  Once you click on either the “Option” or “Setting” text for the field you want, the corresponding “Value” column will become editable. The value entered will be validated against absolute (raw) values, so there is no need to be concerned about the actual position with regard to where an edited field value will be applied. Attempts to ‘0’ pad entered values to approximate actual mask position will be filtered and corrected to an absolute value to the extent possible.

Generate Source Code to Output

Click on the button Generate Source Code to Output on the bottom of the Configuration Bits window (first figure below). This will generate Configuration Bit Settings in the Output window that you can then copy into your code (second figure below).

Figure 5-36. Configuration Bits Window - Generate Source Code in Output

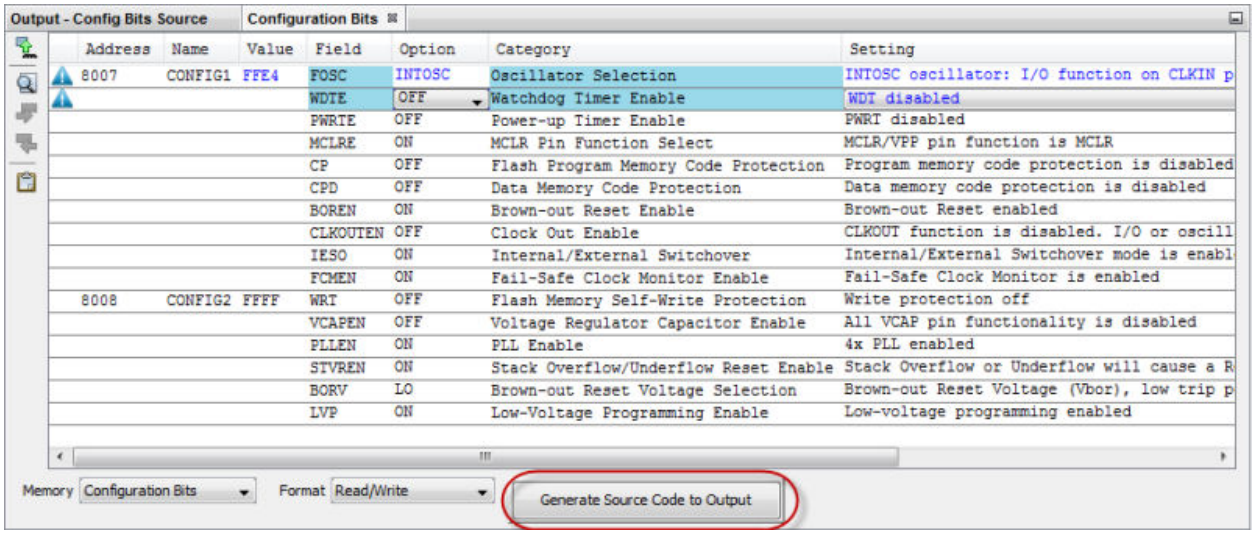
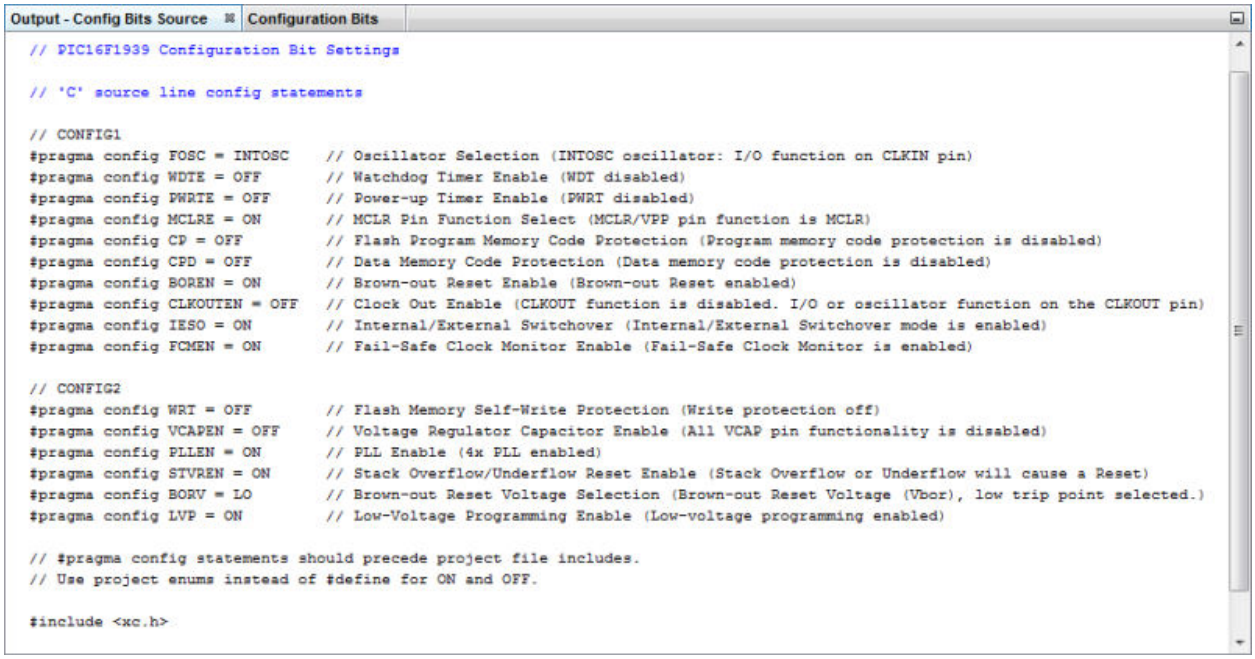


Figure 5-37. Output Window - Generated Source Code



Insert Source Code in Editor

You can select to “Insert Source Code In Editor” from the Configuration Bits window context menu or side bar “clipboard” icon (first figure below). Open or create a main project file in the editor, place the cursor where you want the configuration bits settings to go, then click on the icon (second figure below).

Figure 5-38. Configuration Bits Window - Insert Source Code in Editor

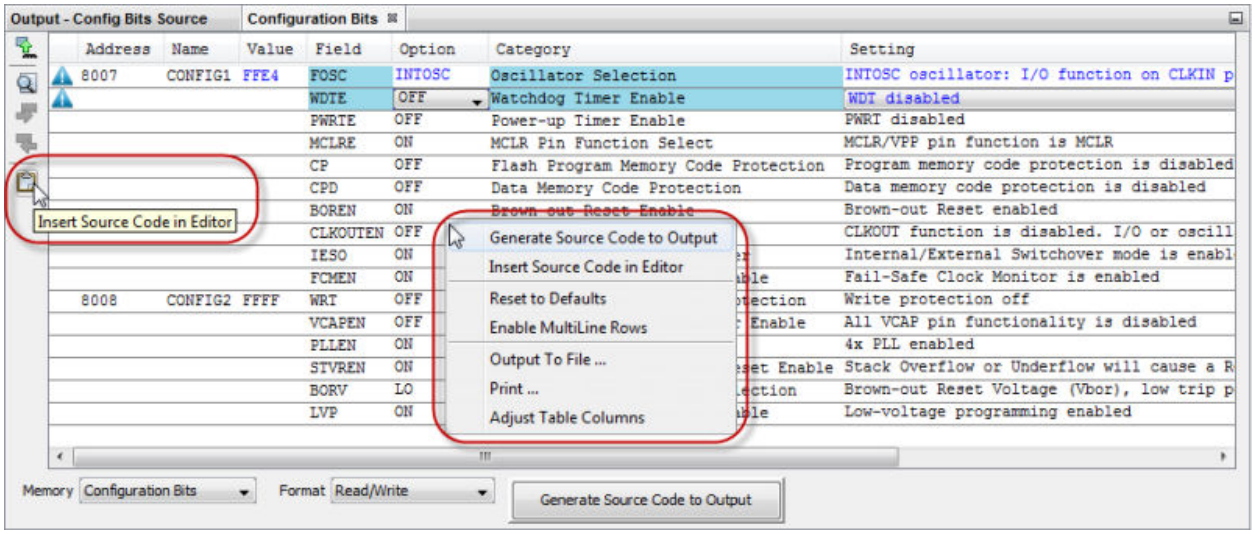
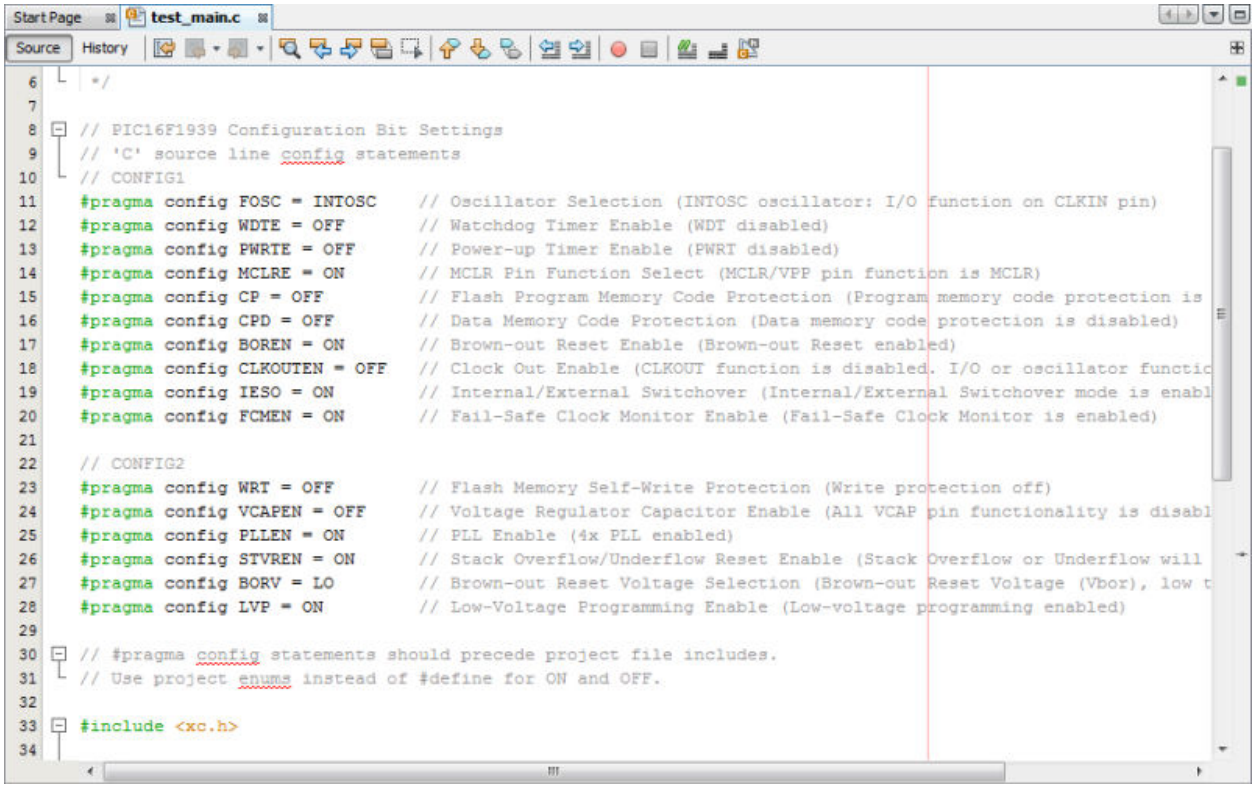


Figure 5-39. Editor Window - Generated Source Code



Program Device for Debugging

Click on the IDE toolbar icon “Program device for debugging” to program the current values of the Configuration Bits window into a device. See 5.21. Program a Device.

5.20.2 Setting Configuration Bits in Assembly Projects

For MPLAB XC assembly projects, you can still use the Configuration Bits window. Add a simple C file to your project to contain the generated `#pragma config` macros. Then you will not have to hand-enter assembly `config` words.

There are no differences between the assembly `config` words and `#pragma config`; they both write a single program word and result in the same program size. However, the `#pragma config` syntax is more flexible and allows for improved validation, formatting, and descriptive comments.

5.21 Program a Device

Once your code is debugged, you can program it onto a target device.

Set Project Programming Properties

Set up programming options in the Project Properties window:

1. Right click on the project name in the Projects window and select “Properties.”
2. Under “Categories,” click on the hardware tool you will use to program your code, e.g., PM3.
3. Review the settings under the “Memories to Program” options category.
   If you wish to use a Preserve Memory option, ensure that your code is not code protected. Code is preserved when the programmer reads the section it needs to save, performs a bulk erase of the device, reprograms the device and then rewrites the area that is preserved with what was saved.
4. Review the settings under the “Program Options” category.
5. Depending on your hardware tool, there may be other programming option categories. Review each one to ensure the settings are correct for your project.

6. When using RTDM with DMCI, Motor Control applications etc., you will need to check the checkbox “Load symbols when programming or building for production (slows process)” under the “Loading” category.

For more information about programming options, please consult your hardware tool documentation.

After the programming options are set up as you desire, you may proceed to program the device.

Perform Programming

To program your target device with debugged code, click the toolbar button Make and Program Device Project.

Other programming-related functions are shown in the Table 4-9. The first function is activated by clicking on the button. For other functions, click on the down arrow next to the button icon.

Table 5-12. Programming Functions on Toolbar Buttons

Button Icon	Function	Details
	Make and Program Device	The project is built (if necessary) and the device is programmed. The program will immediately begin execution on completion of programming.
	Program Device for Debugging	The device is programmed from a debug image. The program will immediately begin execution on completion of programming.
	Program Device for Production	The device is programmed from a production image. The program will immediately begin execution on completion of programming.
	Programmer to Go PICkit 3	Use the Programmer to Go feature of PICkit 3.
	Read Device Memory	Transfer what is in target memory to MPLAB X IDE.
	Read Device Memory to File	Transfer what is in target memory to the specified file.
	Read EE/Flash Data Memory to a File	Transfer what is in target data memory to the specified file.
	Hold In Reset	Toggle the device between Reset and Run.

Program Using the MPLAB Integrated Production Environment

Not all programming functions are in the MPLAB X IDE. For additional programming support, see the MPLAB IPE included with the MPLAB X IDE installation.

	Desktop Icon for MPLAB IPE

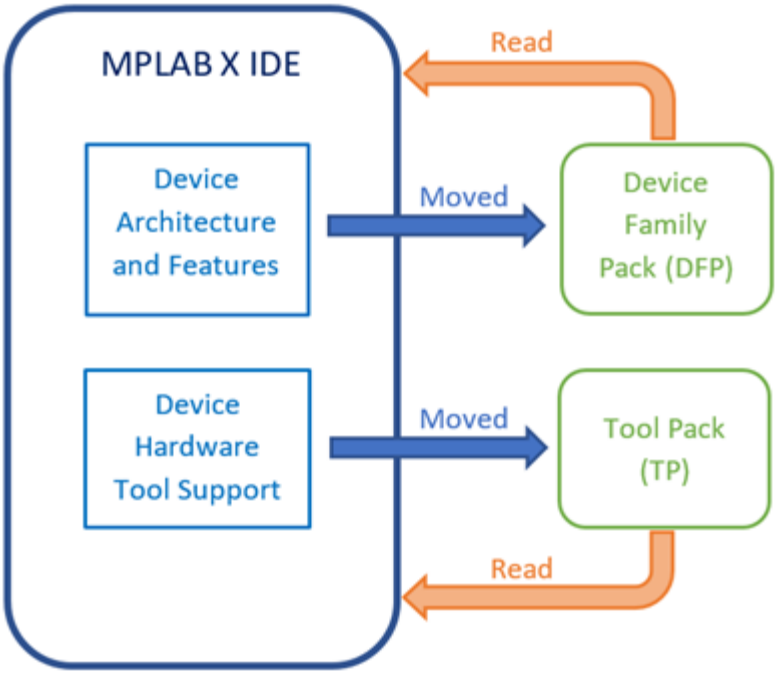
6. Projects

Projects are at the core of developing applications in MPLAB X IDE. Projects contain the application code and development-related information such as the device on which the embedded code will run, the compiler used, and the debug tool used. Projects provide a compact and portable structure for application development.

6.1 Work with Packs

MPLAB X IDE requires specific information about supported devices in order to simulate or emulate these devices. In previous releases (4.20 and earlier), all of this information was built into the IDE. Therefore, if you wanted to support for a new device, you had to wait until the next release.

In releases of 5.00 and later, this device information has been removed from MPLAB X IDE and grouped into versioned packs. This means you can obtain support for new devices in your current version of MPLAB X IDE by updating to the latest pack containing information on this device. MPLAB X IDE will read and use this updated information.



About Packs

Device family packs (DFPs) contain information on each supported device grouped by family. This information includes data on device power requirements, programming methods, architecture, etc. *Tool packs (TPs)* contain information on hardware debug tool firmware for supported devices, as well as other hardware-related information needed by the IDE to work with the tool.

When you update a DFP, the related TP will also be updated so all your hardware tools will work with the new devices in the DFP.

6.1.1 Get Packs

When you install MPLAB X IDE, the latest packs will automatically be installed for that IDE version.

Getting the Latest Pack

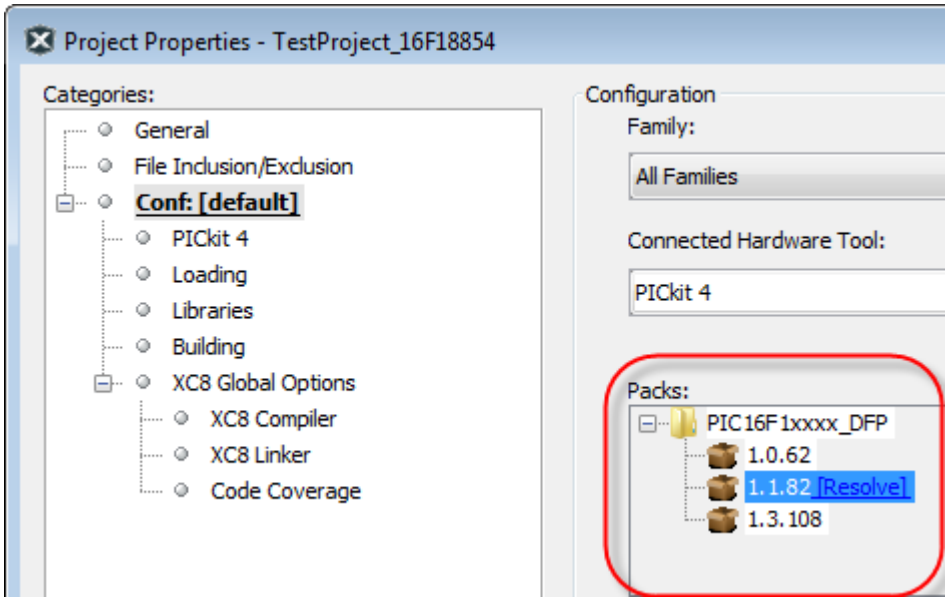
When you open a project, MPLAB X IDE will automatically look for DFP updates. If it finds a newer DFP version, it will notify you. Click on the provided link to download the latest DFP and related tool pack (TP) if needed. Once complete, your project will now have the latest pack selected in the Project Properties.

Using an Older Pack

Note: As of v5.40 and future versions, you can update/switch device and tool support by updating/switching device family packs (DFPs) and related tool packs (TPs). Because of changes to the DFPs and TPs, you cannot switch between v5.40 and v5.35 or previous versions.

When you open an older v5 project that uses a pack not installed, you will be prompted to install that pack. Click on the provided link to go to the Project Properties window to decide if you want to download the older pack or select on another pack already installed.

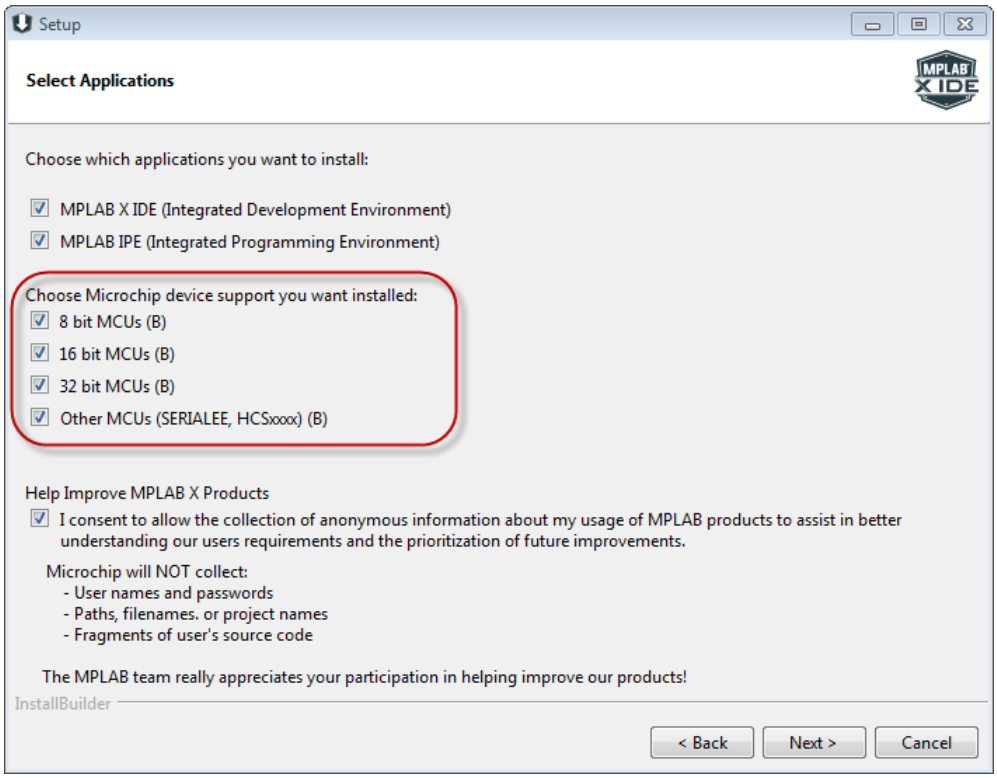
Note: Packs followed by a “[Resolve]” link are not included in that version of MPLAB X IDE.



6.1.2 Select Packs during MPLAB X IDE Install

When installing MPLAB X IDE (as of v5.20), you may "Choose Microchip device support you want installed." This will install only the packs related to the device architecture(s) selected. Installing only what you need saves disk space and speeds installation.

This is also called a distributed installer.



Information on installing MPLAB X IDE may be found in the MPLAB X IDE Readme.

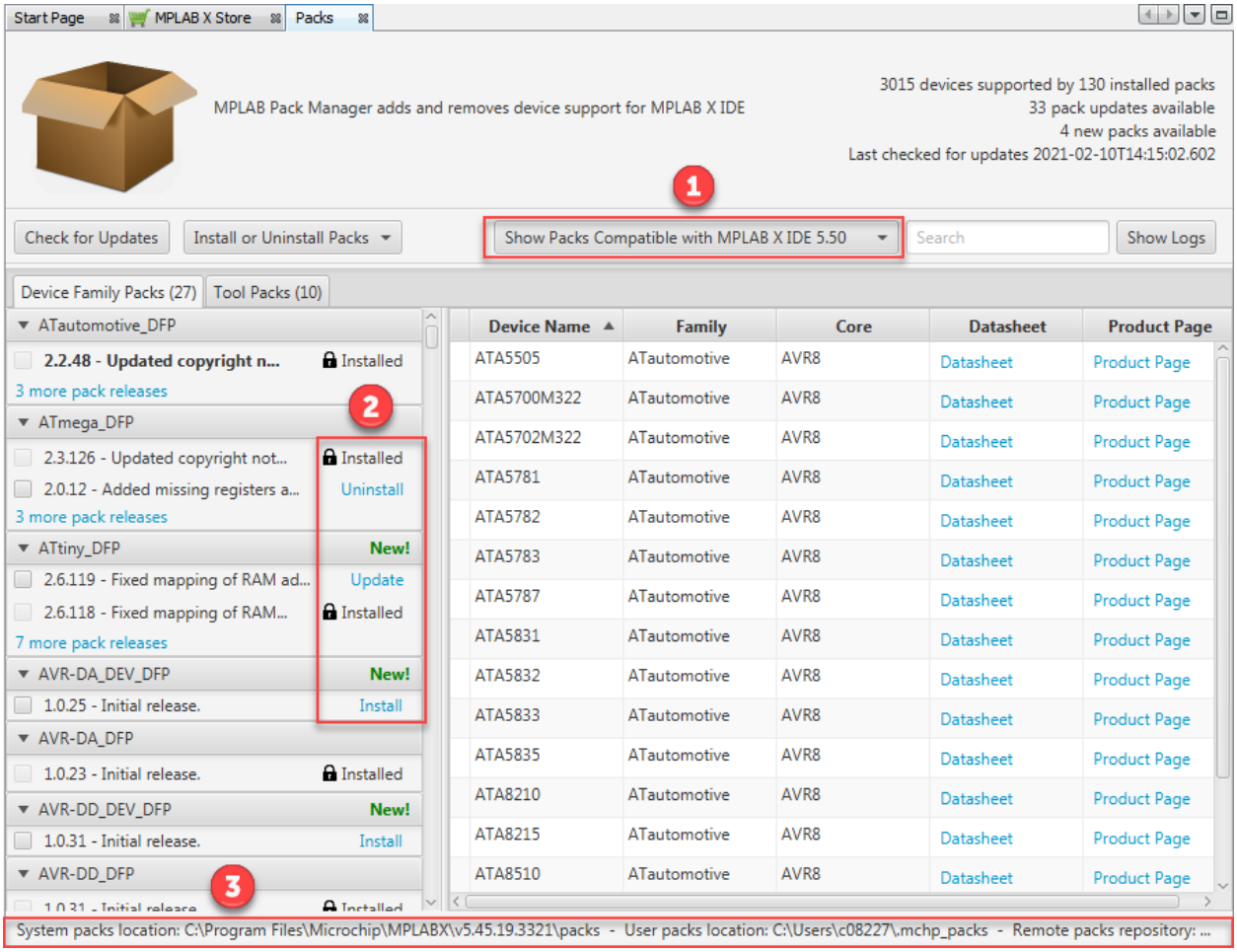
6.1.3 Select Packs after MPLAB X IDE Install

Select *Tools>Packs* to open the MPLAB Pack Manager and find a list of versioned packs available.

1. When first opened, the list is filtered for packs known to be compatible with the current IDE release. For other options, click on the down arrow on “Show Packs Compatible with MPLAB X IDE x.xx” and select from the list.
2. On the “Device Family Pack” or “Tool Pack” tabs, click to “Update,” "Install" or "Uninstall" versions. The number on the tab reflects the number of new DFPs or TPs.
3. At the bottom are pack locations for reference. For more on "System packs location" and "User Packs location," see 6.1.4. Installed Pack Locations. For more on "Remote Packs repository," see 6.1.5. Packs on the Web.

Note: Upcoming versions of MPLAB XC C compilers will use DFPs to provide a more complete debugging experience. Check compiler documentation.

Figure 6-1. MPLAB Pack Manager



Related Links

13.17. Pack Manager Window

6.1.4 Installed Pack Locations

Packs are placed in a `packs` subdirectory of the MPLAB X IDE installation. For example, in Windows 7 OS:

```
C:\Program Files\Microchip\MPLABX\vx.xx\packs
```

In this subdirectory, packs have to following naming conventions:

- Device Family Packs: `<pack_name>_DFP`, e.g., `ATmega_DFP`
- Tool Pack: `<hardware_tool_abbreviated name>`, e.g., `icd4`

Packs may be in a user directory if they are placed after an MPLAB X IDE install. For example, in Window 7 OS:

```
C:\User\Microchip\UserName\.mchp-packs
```

Note: The file structure inside a pack can change at any time. For more on pack structure, see: www.keil.com/pack/doc/CMSIS/Pack/html/index.html.

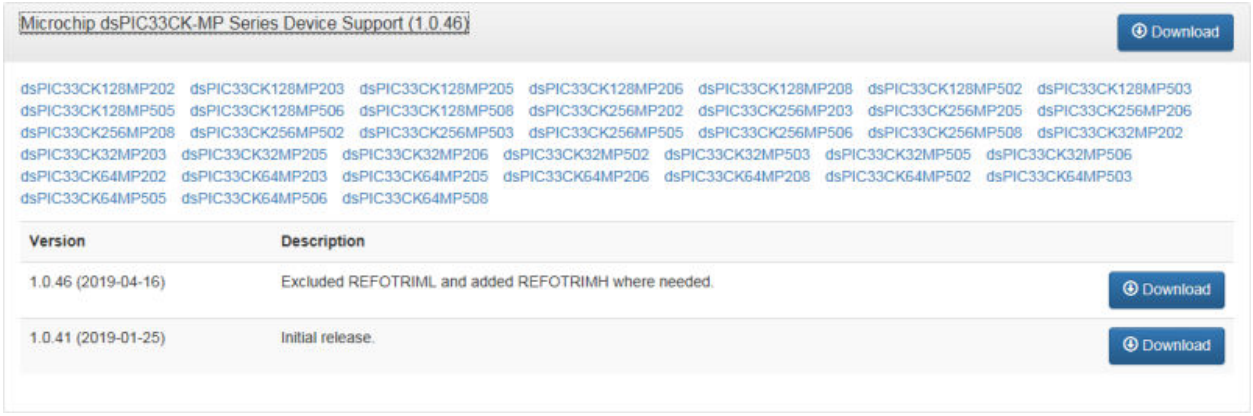
6.1.5 Packs on the Web

To find and download pack versions from the web, go to:

packs.download.microchip.com/

Click on the name of a pack to see pack information, such as supported devices and pack version history.

Figure 6-2. Microchip Packs Repository



6.2 Project Types

Enter a short description of your topic here (optional).

This is the start of your topic.

6.2.1 Open an Existing MPLAB X IDE Project

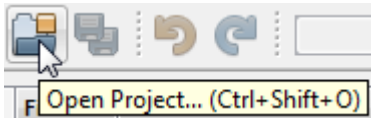
Once you have an existing project, either one that you created or a downloaded example, there are a several ways to open this project.

Open from IDE Menus

Select *File>Open Project* or *File>Open Recent Project*.

Open by Icon

Click on the **Open Project** icon.



Open by Drag-and-Drop of Project Folder

Select a project folder in your file manager window. Drag and drop the folder into the **Editor Pane** (see 5.2. View Changes to Desktop). The project will open in the Projects window.

Figure 6-3. Drag and Drop Folder

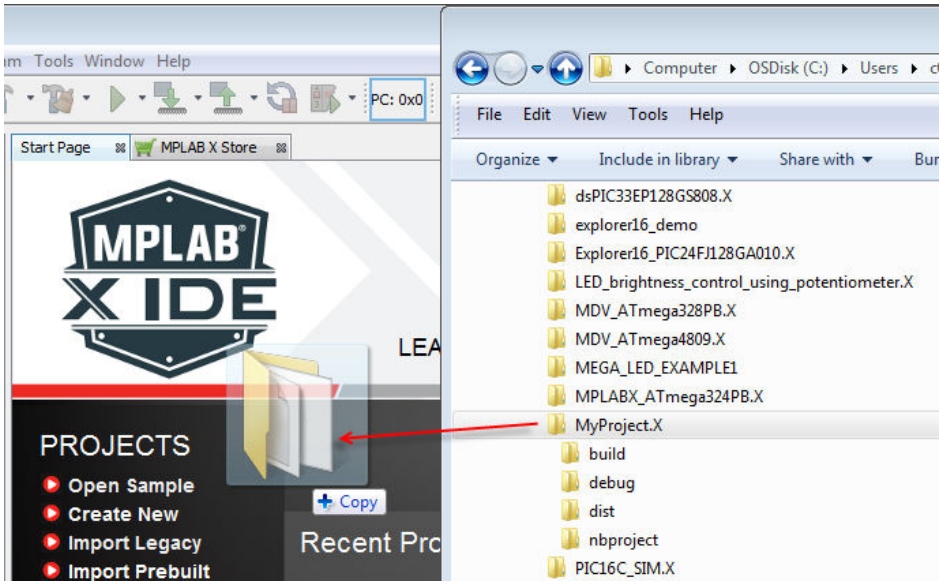
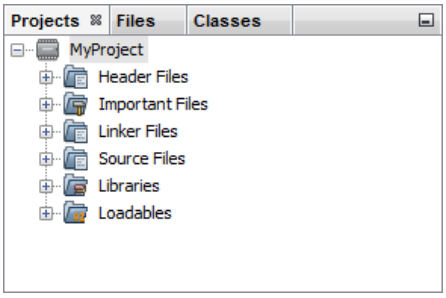


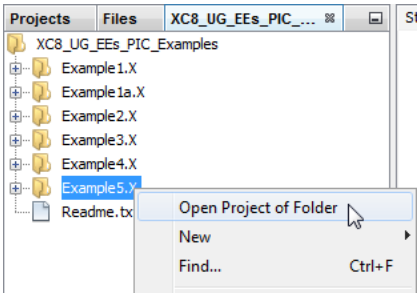
Figure 6-4. Opened Project



Open by Drag-and-Drop of Folder containing Projects

To open a folder containing projects, select the folder in your file manager window. Drag and drop the folder into the **Editor Pane** (see previous step). A window with the name of the folder on the tab will open in the **File Pane**. The folder and related projects will be shown in this window. To open a project in this folder, right click on the project name and select “Open Project of Folder.” The project will open in the Projects window.

Figure 6-5. Opened Folder containing Projects



6.2.2 Import an Existing MPLAB IDE v8 Project

Use the New Project wizard to import an MPLAB IDE v8 project into an MPLAB X IDE project with the following considerations:

MPLAB IDE v8 workspace settings not transferred

Settings that were saved in a workspace in MPLAB IDE v8 (such as tool settings) will not be transferred to the new MPLAB X IDE project. Refer to the MPLAB IDE v8 help for what is stored in a workspace (*MPLAB IDE Reference>Operational Reference>Saved Information*).

Project settings, such as compiler, linker, and assembler options, will be transferred to the new MPLAB X IDE project.

MPLAB IDE v8 project possible file changes

An MPLAB IDE v8 project using the "MPLAB C Compiler for PIC24 MCUs and dsPIC DSCs" (aka MPLAB C30) and a COFF debug file format may have changes.

- The MPLAB X IDE project will be converted to the ELF/DWARF debug file format unless the project uses COFF libraries, in which case the project format will continue to be COFF.
- MPLAB IDE v8 is case insensitive to file extensions such as `.c` and `.C`. However, MPLAB X IDE is case sensitive and associates `.c` to C code files and `.C` to C++ code files. Therefore, if you import an MPLAB IDE v8 project with C code specified as `.C`, MPLAB X IDE will rename the `.C` files to `.c` to avoid incorrect compiler behavior.

MPLAB IDE v8 project possible build changes

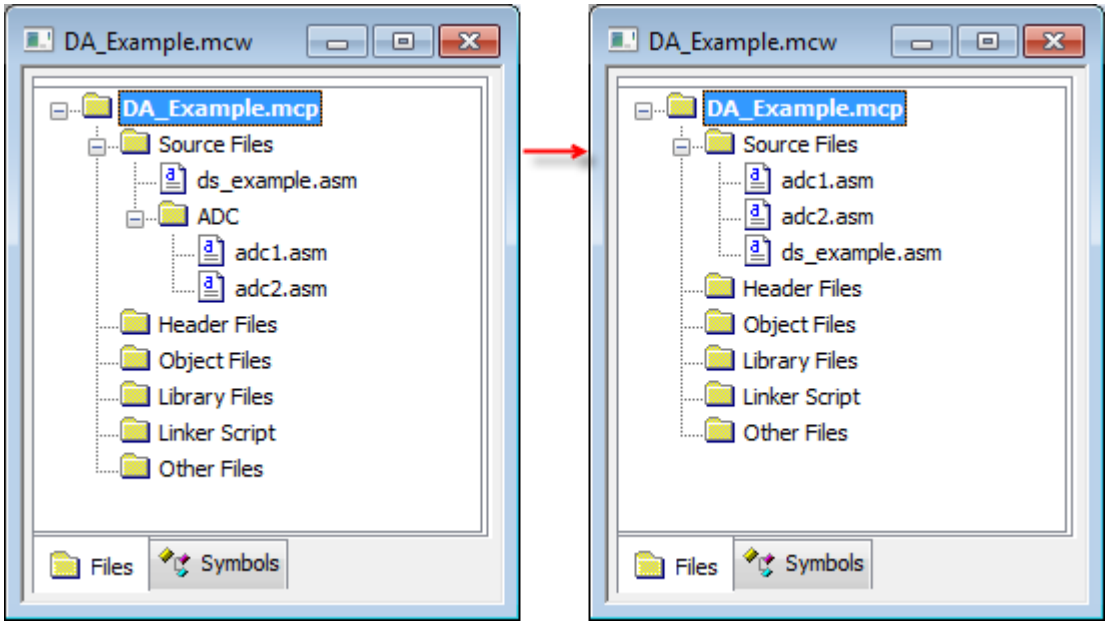
An MPLAB IDE v8 project imported into MPLAB X IDE may not build the same as the original project.

- Do not use `__FILE__`, `assert()`, `__TIME__`, or `__DATE__` in your project code or the build (and checksum) will be different.
- An MPLAB IDE v8 project that uses logical subfolders will not have the same checksum when imported into MPLAB X IDE. To build with the same checksum, move the files in the logical subfolders into the main logical folder (i.e., Source Files, Header Files, etc.) and then import into MPLAB X IDE.

To remove a logical subfolder under Source Files:

- Drag to files under Source Files.
- Select the folder and hit <Delete>.
- Alternately, you can right click on the folder and select "Remove from Project" to remove the folder from the project. But it exists on the PC, it will not be deleted.

Figure 6-6. Move Files and Delete Subfolder



6.2.2.1 Importing an MPLAB IDE v8 Project – Relative Paths

For an MPLAB IDE v8 project located at:

```
C:\MyProjects\mplab8project
```

On importing to MPLAB X IDE you will get:

```
C:\MyProjects\mplab8project\mplab8project.X
```

The MPLAB X IDE imported project is placed by default under the MPLAB IDE v8 project to preserve maintainability of both projects. However, you can place the MPLAB X IDE project folder anywhere. Also, the name of the project will be set as `mplab8project.X` but you can rename it if you wish. There is a convention to finish the name of an MPLAB X IDE project with `.X` but this is just a convention.

The new project will not copy the source files into its folder, but instead will reference the location of the files in the v8 folder. To create an independent MPLAB X IDE project, create a new project and copy the MPLAB IDE v8 source files to it.

For details, see 6.2.2. Import an Existing MPLAB IDE v8 Project.

6.2.2.2 Open the Import MPLAB IDE v8 Project Wizard

There are two ways to open the wizard, described below.

Import Legacy Project Option

To open the Import Legacy Project wizard, do one of the following:

- On the **Start Page**, click on the **Learn & Discover** or **My MPLAB X IDE** tab, “Projects” section, “Import Legacy” link.
- Select *File>Import>MPLAB IDE v8 Project*.

New Project Option

To open the New Project wizard, do one of the following:

- On the **Start Page**, click on the **Learn & Discover** or **My MPLAB X IDE** tab, “Projects” section, “Create New” link.
- Select *File>New Project* (or Ctrl+Shift+N).
- Click on the “New Project” icon on the toolbar. 

A wizard will launch to guide you through new project set up.

Step 1. Choose Project: select the “Microchip Embedded” category and choose from the project type “Existing MPLAB IDE v8 Project.” Click **Next**.

6.2.2.3 Use the Import MPLAB IDE v8 Project Wizard

Follow the steps below to import your MPLAB IDE v8 project. The steps are numbered according to the “Import Legacy Project” Option (see 6.2.2.2. Open the Import MPLAB IDE v8 Project Wizard.)

Click **Next** to move to the next step.

Step 1. Locate MPLAB IDE v8 Project *.mcp File. Enter or browse to the legacy project.

Step 2. Select Device (and Tool). The device from the imported project will be shown if still supported. It is recommended that you keep this device selected until the project is successfully imported. However, you can select a different device from the “Device” drop-down list. To narrow your selection, choose a “Family” first.

Select a “Tool” that supports the device. To see all debug tools, check “Show all”. Tools grayed out are not supported.

Step 3. Select Header. This step will appear if a header is available for your selected device. To determine if a header is required for debug or if your device has on-board debug circuitry, consult one of the documents below. Then choose whether or not to use a header.

- Processor Extension Pak and Debug Header Specification
- Emulation Extension Pak and Emulation Header User’s Guide

Step 5. Select Plugin Board. Select a plugin board, if using one.

Step 6. Select Compiler. Select the language tool (compiler) you will be using to develop your application from the list. Grayed out tools do not support the project device.

© 2022 Microchip Technology Inc. and its subsidiaries

Note: If the project used the MPASM assembler, you will have to select a different language tool as this tool is no longer supported. The suggested alternative is the PIC Assembler (included with MPLAB XC8).

Step 7. Select Project Name and Folder. It is recommended that you do not change the default name and location to preserve maintainability of both projects. See example in the figure below.

File Locations:

The new project will not copy the source files into its folder, but instead will reference the location of the files in the v8 folder.

To create an independent MPLAB X IDE project, create a new project and copy the MPLAB IDE v8 source files to it.

Main Project:

Check the checkbox to make this project the main project on import.

Project Location:

The MPLAB X IDE project location is not in the MPLAB IDE v8 project folder, so this should be unchecked.

File Formatting:

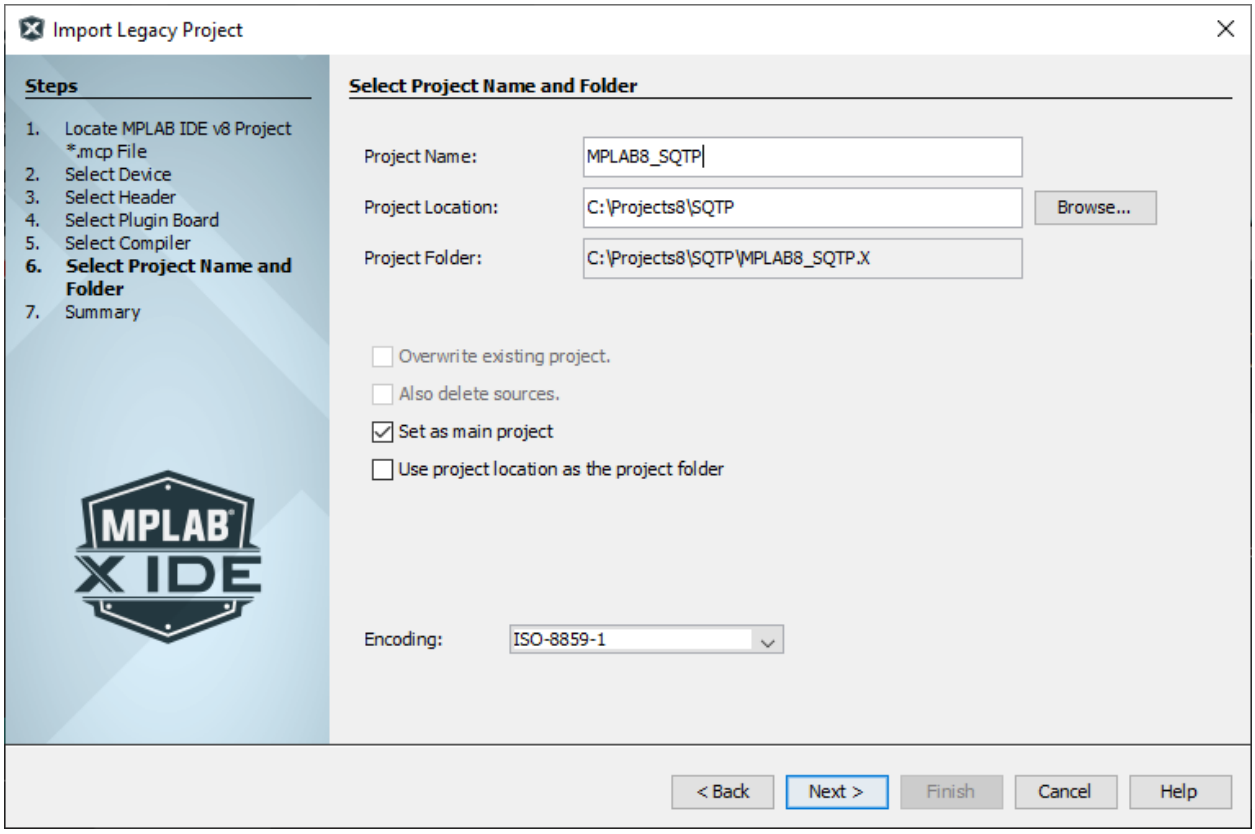
"ISO-8859-1" is the default character encoding used when importing a project from MPLAB IDE v8.

You should select the encoding that matches the one that is used in the imported project. For example, if the MPLAB IDE v8 format is "950 (ANSI/OEM – Traditional Chinese Big5)", then select "Big5" from the drop-down list.

Step 8. Summary. Review the summary before clicking **Finish**. If anything is incorrect, use the **Back** button to go back and change it.

The legacy project will open in the Projects window

Figure 6-7. Step 7 Dialog Example



6.2.3 Import Atmel Studio 7 or Atmel START Project

Use the New Project or Import wizard to import either an Atmel Studio 7 Project or an Atmel START export file for MPLAB X IDE.

There are two ways to open the wizard, described below.

Import Project Option

To open the Import Atmel Studio or START Project wizard, select either:

- `File>Import>Atmel Studio Project`
- `File>Import>START MPLAB Project`

New Project Option

To open the New Project wizard, do one of the following:

- On the Start Page, click on the **Learn & Discover** or **My MPLAB X IDE** tab, “Projects” section, “Create New” link.
- Select *File>New Project* (or Ctrl+Shift+N).
- Click on the “New Project” icon on the toolbar.

A wizard will launch to guide you through new project set up.

Step 1. Choose Project: select the “Microchip Embedded” category and choose from the project type either “Import Atmel Studio Project” or “Import Atmel START MPLAB Project.” Click **Next**.

6.2.3.1 Import Atmel Project - Locate Project File

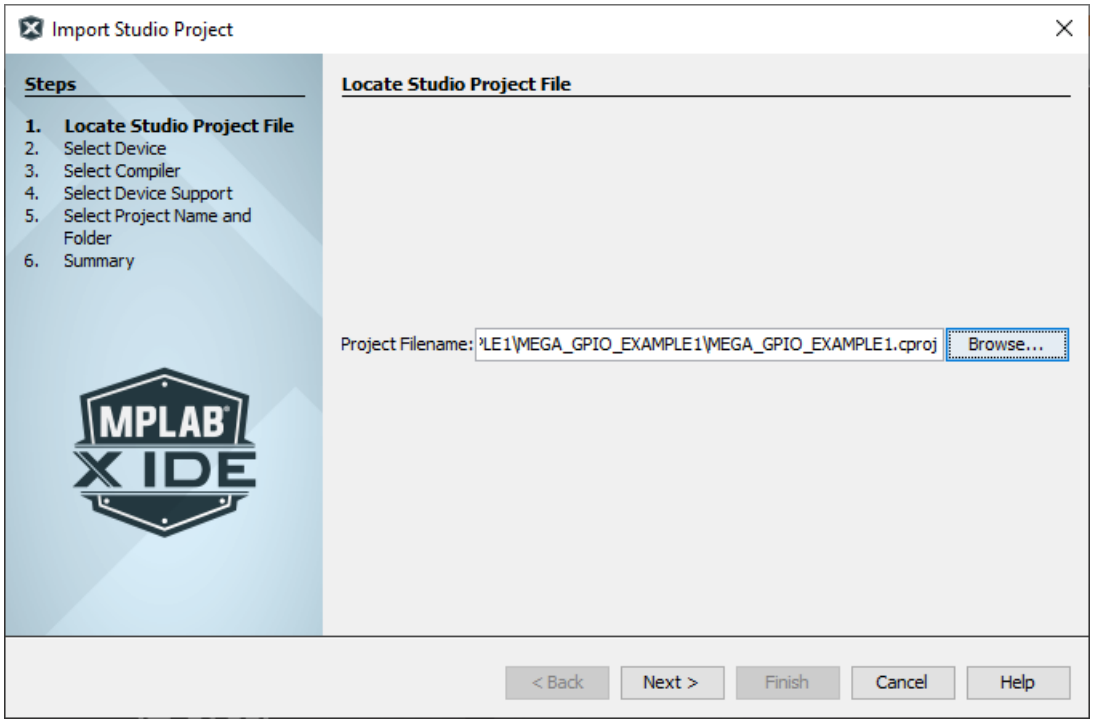
Step 1 or 2. Locate Atmel Studio or START Project File

Enter or browse to the requested file. For example, on Windows OS, project files may be found under:

`C:\Users\`*`UserName`*`\Documents\Atmel Studio\7.0\`

For Atmel Studio, find the project file `*.cproj` or `*.cppproj`. For Atmel START, find the project file `*.atzip`.

Figure 6-8. Import Atmel Studio Project



6.2.3.2 Import Atmel Project - Select Device

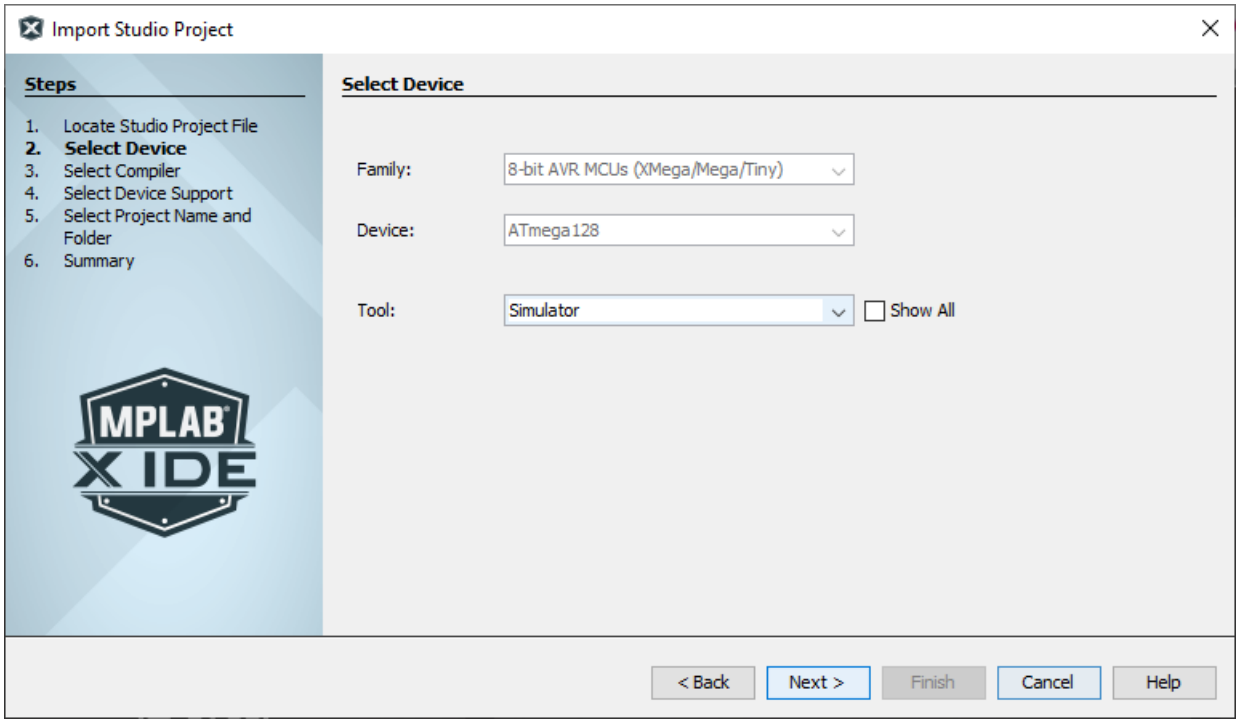
Step 2 or 3. Select Device (and Tool)

This window will be populated with the device and tool information from the imported project.

If the tool is not available in MPLAB X IDE, you may select a different tool from the drop-down list, but you may want to keep the original project tool until after import to ensure your project is working correctly. Then you can select a different tool in the Project Properties dialog.

If you do not see a your tool listed, check the “Show All” box to see if the tool supports the device but is not connected (shows as gray.) Consult the hardware user’s guide to connect the tool correctly.

Figure 6-9. Import Atmel Project - Select Device

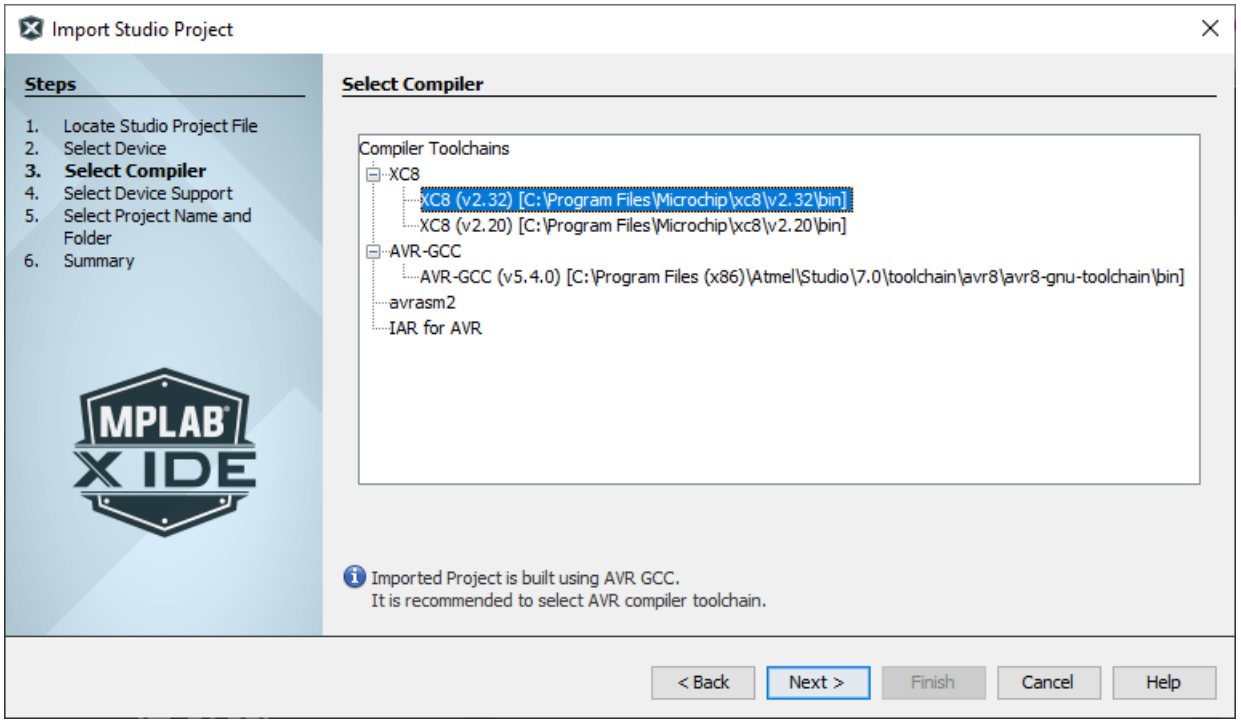


6.2.3.3 Import Atmel Project - Select Compiler

Step 3 or 4. Select Compiler

This window will highlight the compiler from the imported project, if MPLAB X IDE can locate it. If not, you may select a compiler from the list. However, if it is not the original project compiler, you will notified.

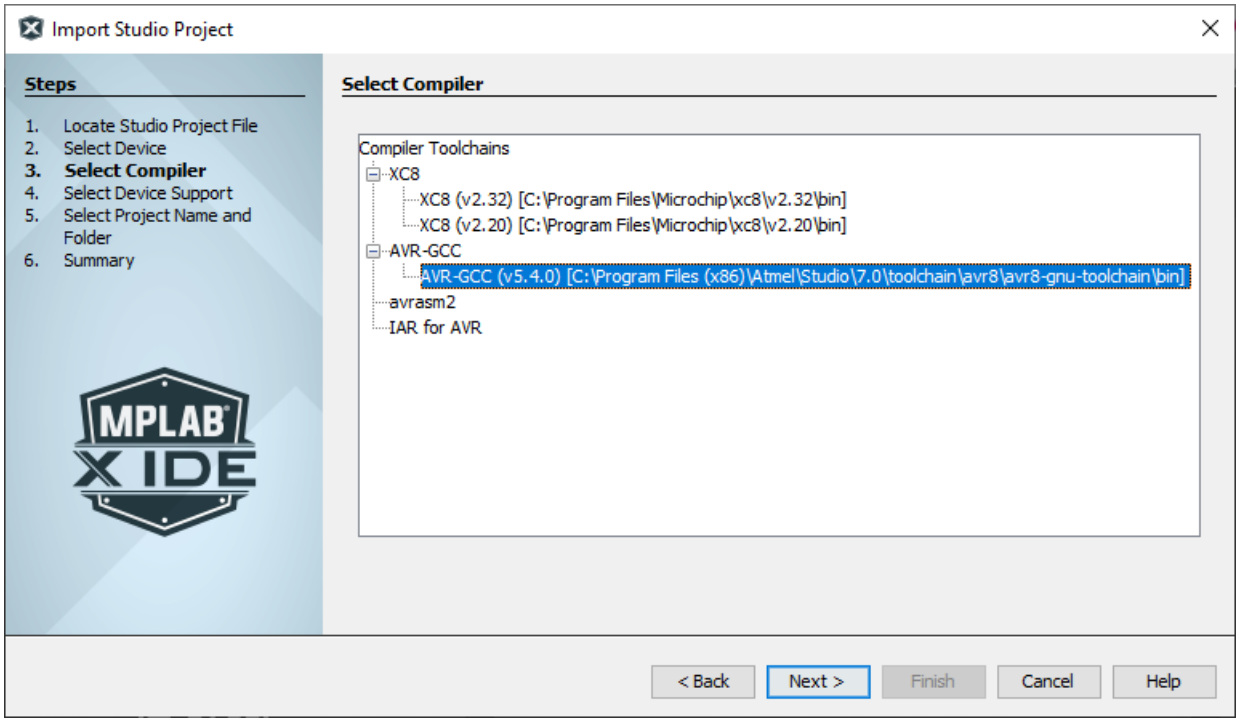
Figure 6-10. Import Atmel Project - Different Compiler Selected



It is suggested that you keep the original project compiler until after import to ensure your project is working correctly. Then you can select a different compiler in the Project Properties dialog.

If you do not see a compatible compiler listed, click **Cancel** and follow the directions under “Create a New Project,” 5.1.7. Step 6: Select Compiler.

Figure 6-11. Import Atmel Project - Imported Project Compiler Selected



6.2.3.4 Import Atmel Project - Select Device Support

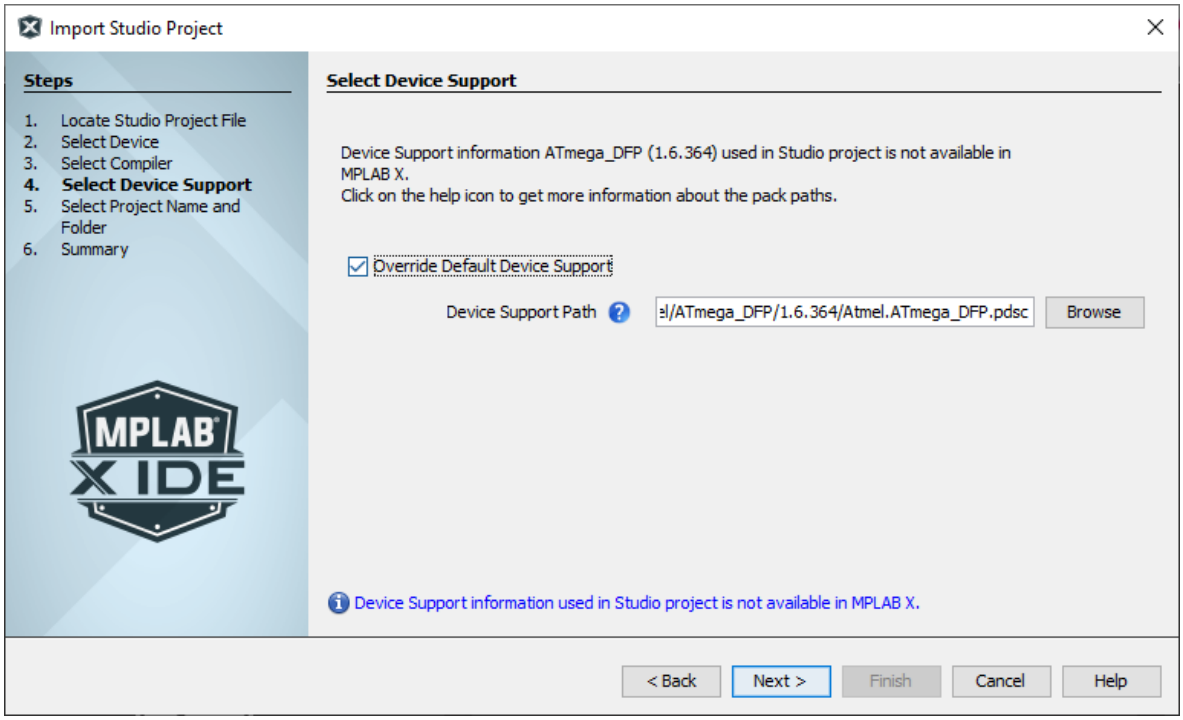
Step 4 or 5. Select Device Support

As of MPLAB X IDE v5.00, devices are supported in versioned device packs.

For “Override Default Device Support” checked:

1. If the Device Support Path is populated, then the project pack was found in the Atmel Studio pack directory. Use this pack.

Figure 6-12. Import Atmel Project - Device Support Selected



2. If the Device Support Path is NOT populated, then you can download the pack support by clicking on the help icon (question mark) and following the instructions. When you download the pack, it will install in Atmel Studio. Browse to this location to select the pack. For Windows OS: `C:\Program Files (x86)\Atmel\Studio\7.0\packs`

Figure 6-13. Import Atmel Project - Select Device Support



For “Override Default Device Support” unchecked, you can use the MPLAB X IDE device pack version (the latest) in the project. For example, the MPLAB X IDE project packs would be located under:

```
C:\Program Files\Microchip\MPLABX\v5.xx\packs
```

6.2.3.5 Import Atmel Project - Select Project Name and Folder

Step 5 or 6. Select Project Name and Folder

It is recommended that you do not change the default name and location to preserve maintainability of both projects.

Project Files Location:

For Atmel Start, source files will be copied into the MPLAB X IDE project.

For Atmel Studio, the new project will not copy the source files into its folder, but instead will reference the location of the files in the Atmel project folder.

To create an independent MPLAB X IDE project, create a new project and copy the source files to it.

Main Project:

Check “Set as main project” to make this project the main project on import.

Project Location:

For Atmel Studio, it is not recommended to check “Use project location as the project folder,” but to keep the MPLAB X IDE project with the Atmel project. For Windows OS systems, if the Atmel project path is too long (see warning below), change the path of this project first before importing into MPLAB X IDE.

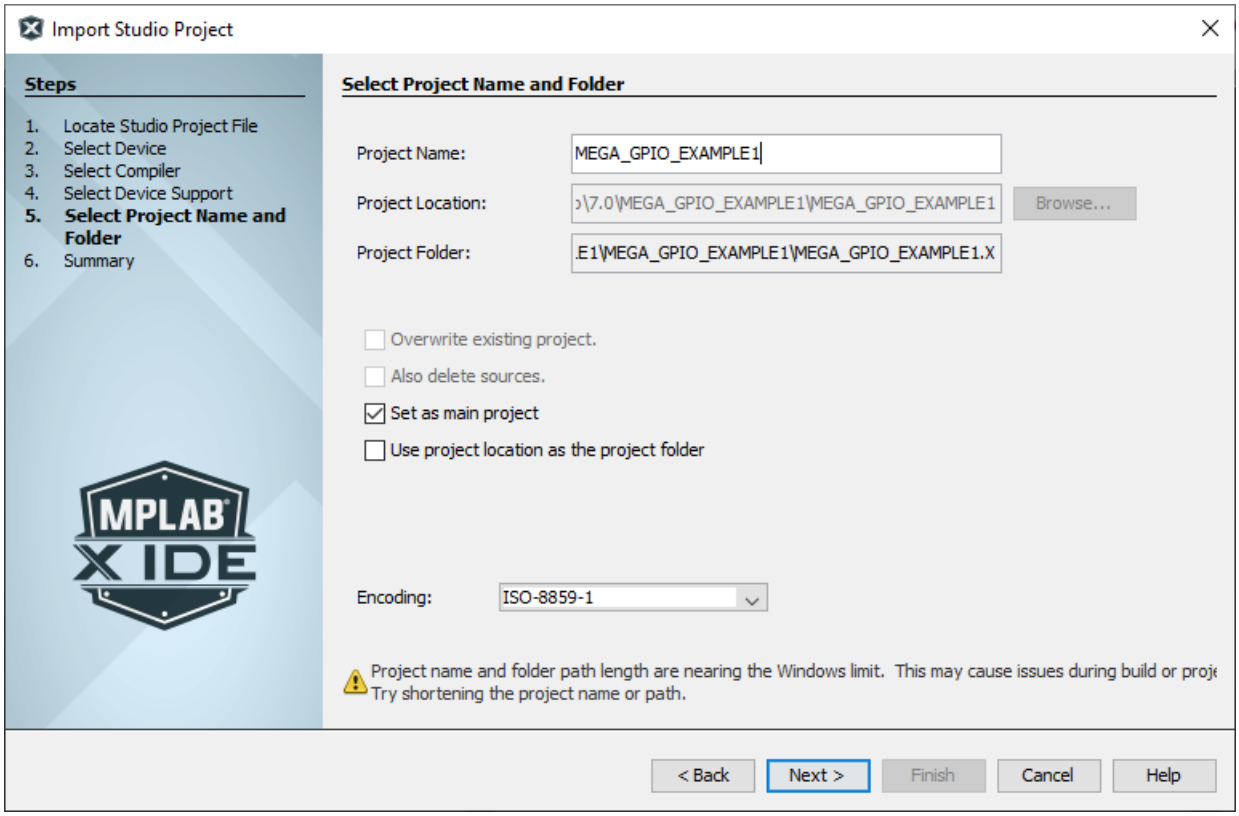
File Formatting/Encoding:

You should select the encoding that matches the one that is used in the imported project. For example, if the format is “950 (ANSI/OEM – Traditional Chinese Big5),” then select “Big5” from the drop-down list.

Path Length (Window OS):

If the path to your project is too long, you may need to shorten it to avoid issues during build.

Figure 6-14. Import Atmel Project – Select Project Name and Folder



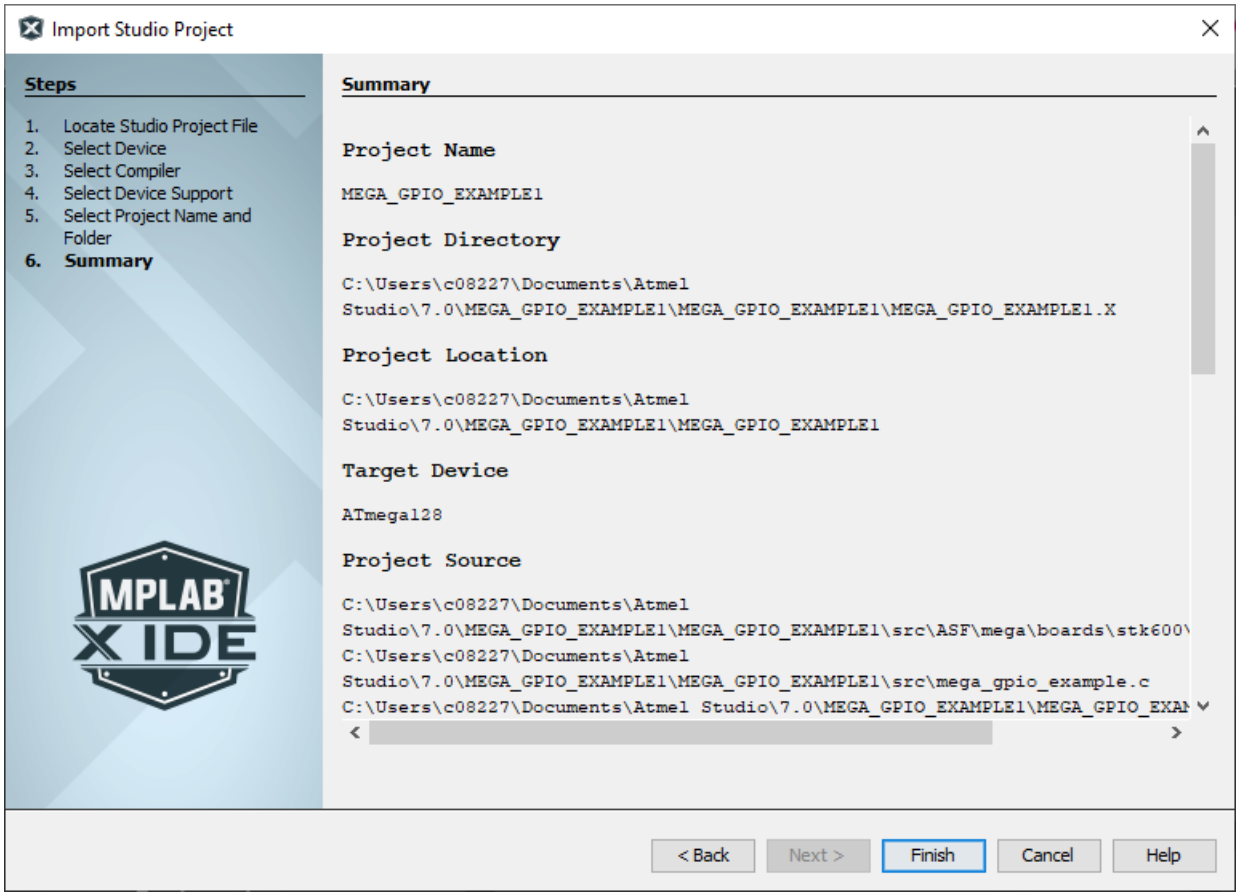
6.2.3.6 Import Atmel Project - Summary

Step 6 or 7. Summary

Review the summary before clicking **Finish**. If anything is incorrect, use the **Back** button to go back and change it.

When the project is being imported, observe any warnings or other messages in the Output window. You may need to make changes so your new project will build.

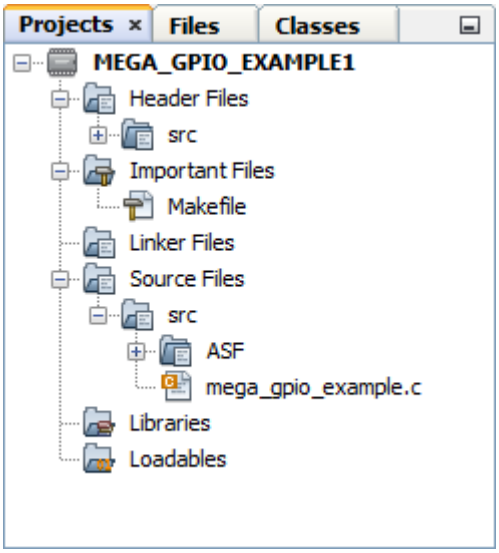
Figure 6-15. Import Atmel Project – Summary



The new project will open in the Projects window.

Note: The source files from the original Atmel project are linked, as opposed to copied. So modifications to the source files in either the Atmel or MPLAB X IDE projects will be noticed in both.

Figure 6-16. Import Atmel Project – Projects Window



6.2.4 Prebuilt Projects

Create a project from a prebuilt loadable image (Hex, ELF or COF files) by using the Import Image File wizard.

Note: To program a prebuilt image into a device, you will click the **Make and Program Device** icon even though it will only program the device (no make).



There are two ways to open this wizard: the Start Page and the New Project options.

Import Prebuilt Project Option

To open the “Import Image File” wizard, do one of the following:

- On the **Start Page**, click on the **Learn & Discover** or **My MPLAB X IDE** tab, “Projects” section, “Import Prebuilt” link.
- Select *File>Import>Hex/ELF (Prebuilt) File*.

New Project Option

For ways to open the New Project wizard, see 5.1.2. Launch New Project Wizard.

The wizard will launch to guide you through new project set up.

- **Step 1. Choose Project**: select the “Microchip Embedded” category and choose from the project type “Prebuilt (Hex, Loadable Image) Project.” Click **Next**.
- The “Import Image File” wizard opens.

Import Image File Wizard

Follow the steps below to import your image file. The steps are numbered according to the “Import Prebuilt Project” option.

Click **Next** to move to the next step.

Step 1. Import Image File. Select the name and location of your image file. You may browse to a location.

Step 2. Select Device. Select the device you will be using in your application from the “Device” drop-down list. To narrow your selection list, choose a “Family” first.

Step 3. Select Header. This step will appear if a header is available for your selected device. To determine if a header is required for debug or if your device has on-board debug circuitry, consult one of the documents below. Then choose whether or not to use a header.

- Processor Extension Pak and Debug Header Specification
- Emulation Extension Pak and Emulation Header User’s Guide

Step 4. Select Tool. Select the development tool you will be using to develop your application from the list. Grayed out tools do not support the project device.

Step 5. Select Project Name and Folder. Select a name and location for your new project. You may browse to a location. Click **Finish** when done.

The new project will open in the Projects window.

For information on exporting a project as hex, see the “Project Menu” in 13.19. Projects Window.

6.2.5 Loadable Projects, Files and Symbols

Use loadable projects and files to combine projects, combine hex files, combine projects and hex files or replace the project hex file. The Hexmate application is used to merge project or loaded hex files into one file. For more information on using Hexmate, see MPLAB XC8 documentation or *Hexmate User’s Guide* (DS50003033) on the Microchip website.

Use loadable symbols to load debug symbols during a production build or program, as when using RTDM with DMCI, Motor Control applications, etc.

Loadable projects or files are useful for creating combined bootloader and application code. See 6.2.6. Loadable Projects and Files: Bootloaders.

© 2022 Microchip Technology Inc. and its subsidiaries

The combinations of current projects and loadables are listed below.

Table 6-1. Loadable Combinations

Current Project	Loadable	Caveat
Standalone, Existing MPLAB IDE v8, Library	Standalone	None
	Hex file	None
	ELF or COF file	Can debug, but not build. An error will be displayed in the Output window.
Prebuilt (Hex)	Hex file	No auto checking for overlapping memory areas. Can debug, but not build. Build button will be disabled.
	ELF or COF file	
Prebuilt (ELF or COF)	Hex file	
	ELF or COF file	

The options are listed below.

Table 6-2. Loadable Options

Add Loadable Project(s)	Load one or more existing projects into your current project. When you build your current project, all projects will be built and the hex files will be combined into one. All debug files will be combined as well (ELF or COF).
Add Loadable File(s)	Load one or more existing hex files into your current project. When you build your current project, the hex file will be combined with the other hex files into one file. Note: You will no longer be able to debug the project that contains hex file(s). Use Loadable Projects for debugging.
Add Alternate File	Load an alternate hex file to be used. This options provides a post-build step where you may copy or move your project hex file to another location, use a tool such as Hexmate to merge your file with another hex file, and then load the file back into the IDE.

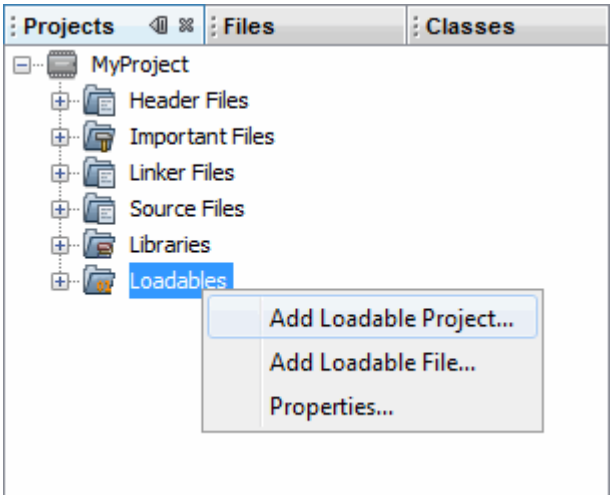
6.2.5.1 Projects Window – Loadables Setup

Right Click on the “Loadables” folder in the Projects window (see figure) and select an option:

- Add Loadable Project – select to add an existing project to your current project. Repeat to add additional projects.
- Add Loadable Files – select to add an existing hex file to your current project. Repeat to add additional hex files.
- Properties – Open the Project Properties window for Loading. See 6.2.5.2. Project Properties Window – Loading Setup.

Build your current project to build all projects and combine hex files into one. Any debug files will also be combined.

Figure 6-17. PROJECTS WINDOW – LOADABLES FOLDER

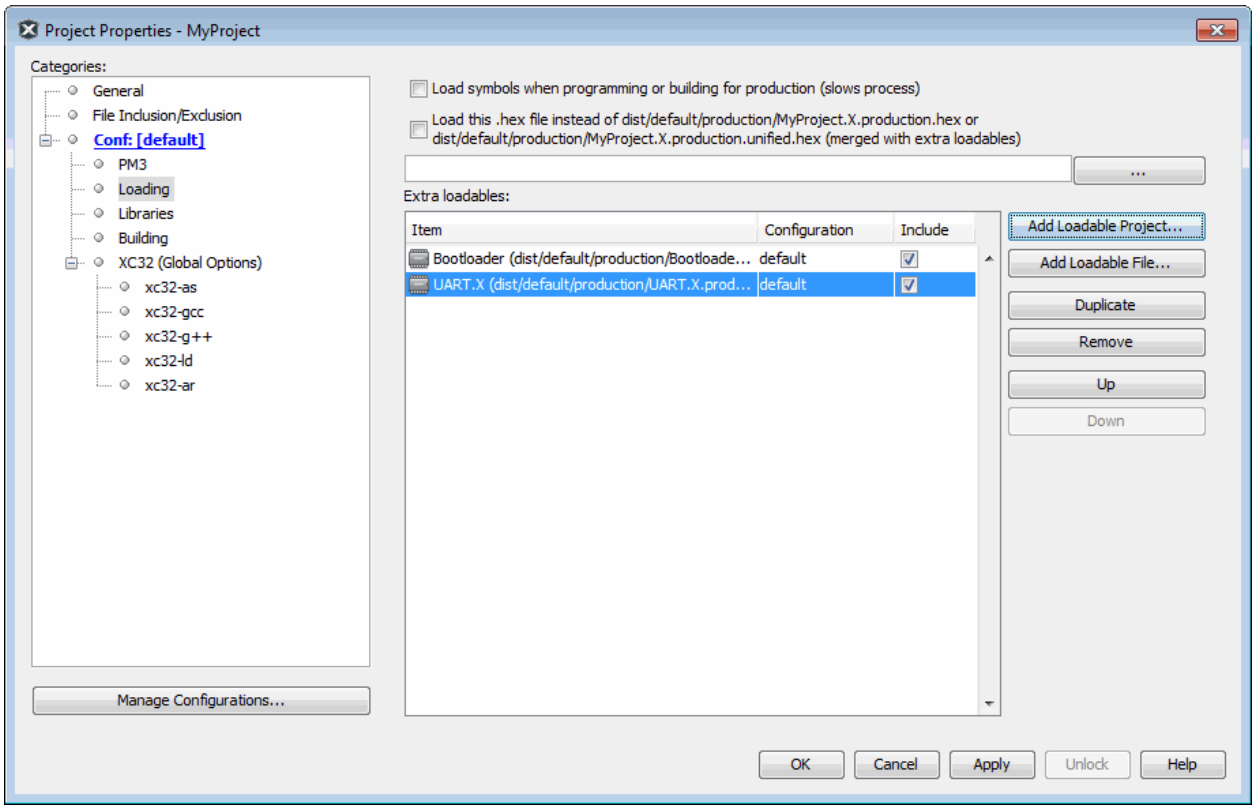


6.2.5.2 Project Properties Window – Loading Setup

Open the Project Properties window (*File>Project Properties*) and click on "Loading."

- Combining the Current Project with Other Projects
- Combining the Current Project Hex File with Other Hex Files
- Loading an Alternative Hex File
- Loading Debug Symbols During Program/Build

Figure 6-18. Project Properties – Loading



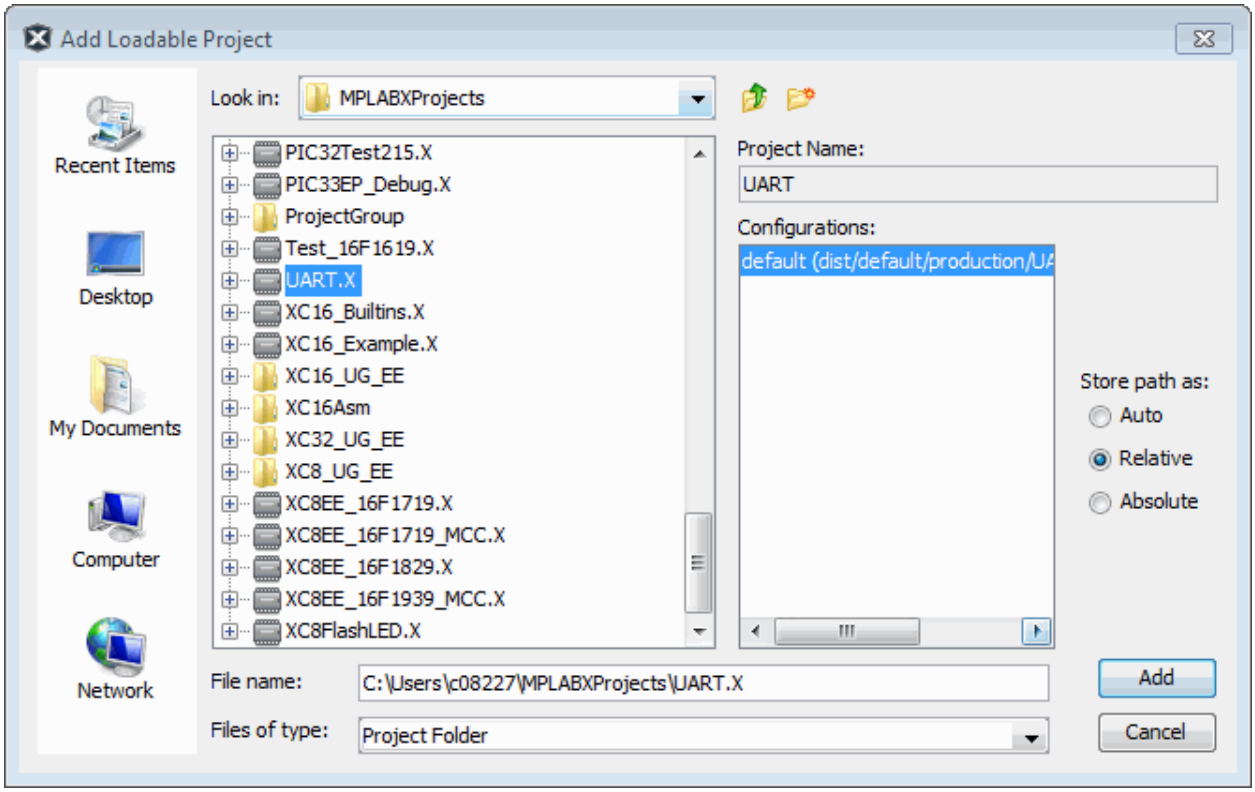
Combining the Current Project with Other Projects

1. Click **Add Loadable Project**. The Add Loadable Project dialog will appear.

2. Browse to a project and select it.
3. Under “Configurations” select a build configuration to use for this project. If you have not added different configurations to this project, you will only see “default.”
4. Under “Store path as” select how to store information about the path to the loadable project. Choose from:
   a. Auto – allow MPLAB IDE to determine whether the loadable project path should be relative or absolute based upon the its location.
   If the main project directory contains the loadable project, the reference is a relative path. Otherwise, the reference is an absolute path.
   b. Relative – reference the loadable project with a relative (to the main project) path.
   This is the preferred way to add paths.
   c. Absolute – reference the loadable project with an absolute path.
   There are cases when this mode is useful, but these paths often require correction when a project is moved from one system to another.
5. Click **Add** to close the dialog.
6. In the Project Properties window, ensure the “Include” checkbox is checked if you want to build this project when you build the current project.
7. If you have added more than one project, the order shown here will determine the order in which the hex files will be added to the current project’s hex file. Use **Up** and **Down** to change the order.
8. Click **Apply** or **OK** to accept the changes.

The next time you build the current project, the projects listed here will also be built (if “Include” is checked) and their hex files will be combined with the current project’s hex file to create a single output hex file. Any debug files will also be combined.

Figure 6-19. Add Loadable Project



Combining the Current Project Hex File with Other Hex Files

1. Click **Add Loadable File**. The Add Loadable File dialog will appear.
2. Browse to a hex file and select it.
3. Under “Store path as” select how to store information about the path to the loadable file. Choose from the following options:

a. Auto – Allow MPLAB IDE to determine whether the loadable file path should be relative or absolute based upon the its location.
   If the main project directory contains the loadable file, the reference is a relative path. Otherwise, the reference is an absolute path.
b. Relative – Reference the loadable file with a relative (to the main project) path.
   This is the preferred way to add paths.
c. Absolute – Reference the loadable file with an absolute path.
   There are cases when this mode is useful, but these paths often require correction when a project is moved from one system to another.

4. Click **Add** to close the dialog.
5. In the Project Properties window, if you have added more than one file, the order shown here will determine the order in which the hex files will be added to the current project's hex file. Use **Up** and **Down** to change the order.
6. Click **Apply** or **OK** to accept the changes.

The next time you build the current project, the hex files listed here will be combined with the current project's hex file (`CurrentProject.X.Production.hex`) to create a single output hex file. (`CurrentProject.X.unified.hex`) in the same folder as the current project hex file.

Loading an Alternative Hex File

1. Click to check "Load this .hex file instead of ..."
2. Click the ellipses (...) button to open the Add Loadable File dialog. For details on adding the hex file using this dialog, see the previous topic.

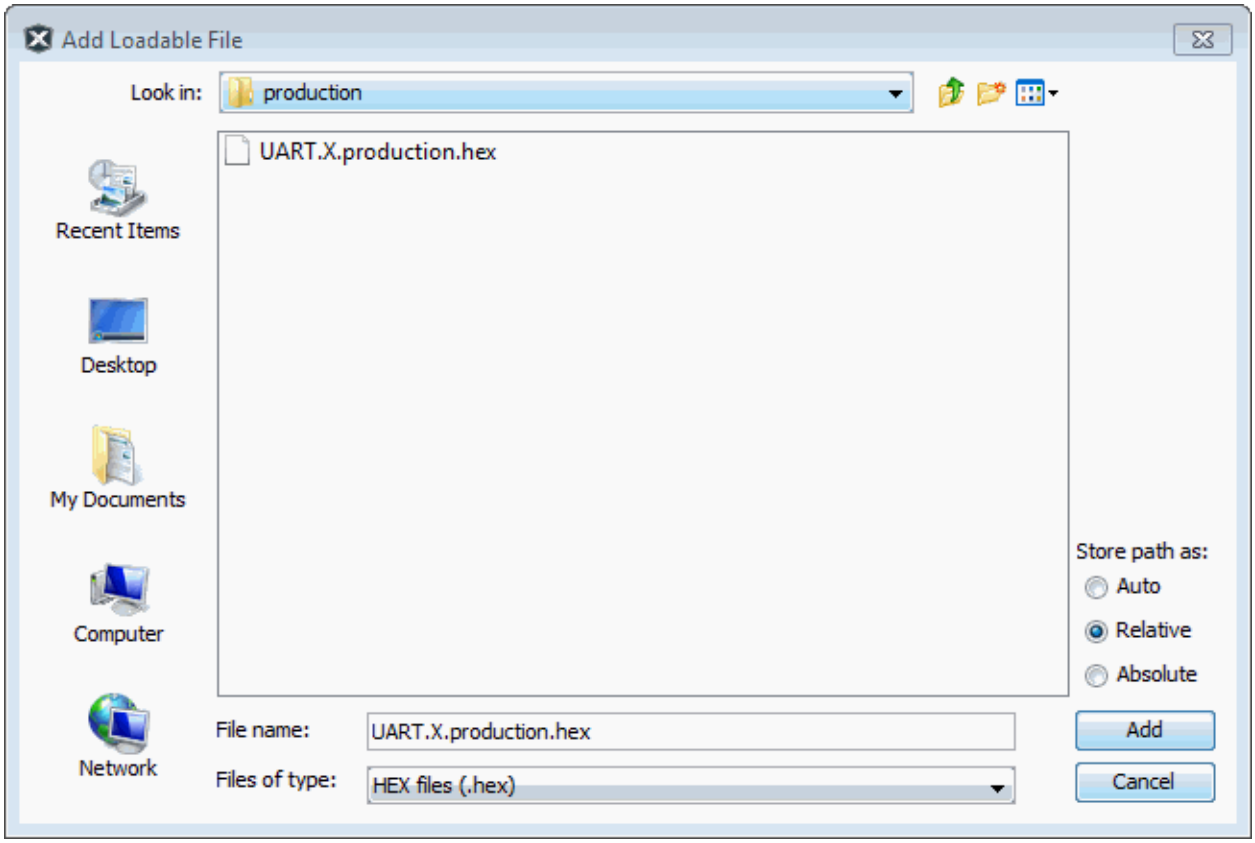
The next time you build the current project, the alternative hex file will load on build complete.

Loading Debug Symbols During Program/Build

The checkbox to "Load symbols when programming or building for production (slows process)" allows you to load debug symbols when programming, as when using RTDM with DMCI, Motor Control applications etc. Otherwise the checkbox should be left unchecked to speed up programming and building (default).

© 2022 Microchip Technology Inc.
and its subsidiaries

Figure 6-20. Add Loadable File



6.2.5.3 The Preferred Method to Use Loadables

The recommended way to use loadables is:

1. Select the project that sets the device configuration bits and initialization as the main project (right click and select “Set as Main Project”). The loadable should not have device configuration settings as this will conflict with the main project.
2. Add the other projects as loadables to this main project. If the project being loaded has more than one project configuration, be sure to specify that when loading.

Specifying the project containing the loadables as the main project ensures that a change in any loadable will be picked up by the build when the Build button is clicked.

6.2.6 Loadable Projects and Files: Bootloaders

To combine a bootloader with application code:

1. Create one project for your application and one project for your bootloader.
2. Load the bootloader project or hex file into the application project. See 6.2.5.1. Projects Window – Loadables Setup or 6.2.5.2. Project Properties Window – Loading Setup for how to do this.

The next time you build your application project, the resulting hex file will be a combined bootloader/application hex file. Any debug files will also be combined.

For build errors, see 6.2.5.3. The Preferred Method to Use Loadables or the sections below.

MPLAB XC8 – PIC18 MCU Example

The following Microchip webinar details how to combine a bootloader with application code for a PIC18 MCU using MPLAB XC8 and MPLAB X IDE:
Linking PIC18 Bootloaders and Applications

MPLAB XC16 Bootloader Example

See the “*MPLAB XC16 C Compiler User’s Guide*” (DS50002071) or Help file for an example bootloader.

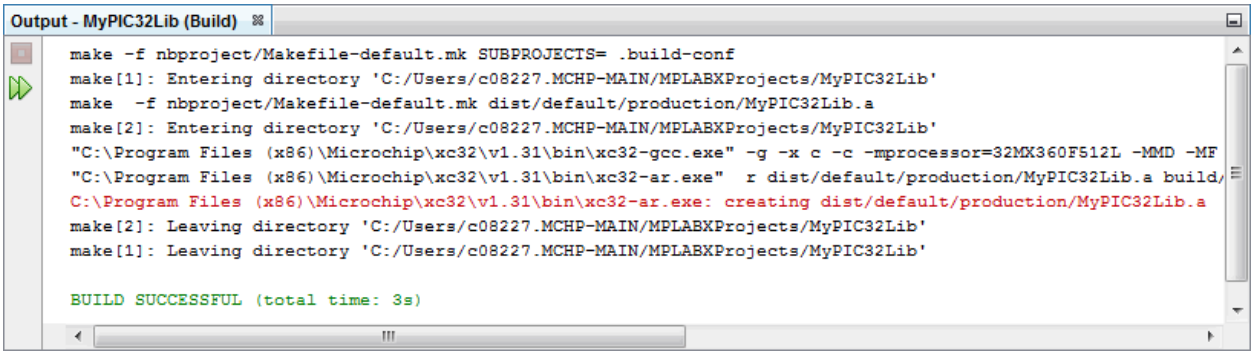
MPLAB XC32 Bootloader Example

See the “*PIC32 Bootloader Application Note*” (DS01388) on the Microchip website.

6.2.7 Library Projects

Create a project that will generate a library file (`*.lib` or `*.a` depending on which compiler you use) instead of a hex file (see text in red). This project will not be able to run on its own as it will not have a `main()` function. The resulting library will be used in another project that has its own `main()` function.

Figure 6-21. Library Project Example



For ways to open the New Project wizard, see 5.1.2. Launch New Project Wizard.

The wizard will launch to guide you through new project set up. Click **Next** to move to the next step.

Step 1. Choose Project. Select the “Microchip Embedded” category and choose from the project type “Library Project.”

Step 2. Select Device. Select the device you will be using in your application from the “Device” drop-down list. To narrow your selection list, chose a Family first.

Step 3. Select Header. This step will appear if a header is available for your selected device. To determine if a header is required for debug or if your device has on-board debug circuitry, consult either:

- Processor Extension Paks and Debug Headers
- Emulation Extension Paks and Emulation Headers

Step 4. Select Tool. Select the development tool you will be using to develop your application from the list. Grayed out tools do not support the project device.

Step 5. Select Compiler. Select the language tool (compiler) you will be using to develop your application from the list. Grayed out compilers do not support the project device.

Step 6. Select Project Name and Folder. Select a name and location for your new project. You may browse to a location. Click **Finish** when done.

The new project will open in the Projects window.

You can change your Library project to a Standalone (Application) project if you wish. For details, see 5.10.1. Change Project Configuration Type.

6.2.8 Create Dual Core Projects

A dual core device (e.g., dsPIC33CH DSCs) has a Master core and Slave core that can operate independently and can be programmed and debugged separately during application development. Both processor (Master and Slave) subsystems have their own interrupt controllers, clock generators, ICD, port logic, I/O MUXes and PPS. The device is equivalent to having two complete dsPIC® DSCs on a single die. For more details on dual core devices and how to debug dual core code, see:

AN2721 Getting Started with Dual Core

Dual core devices have several modes of operation which correspond to different ways of creating and using projects (see the sections below).

Master Only Programming and Debugging

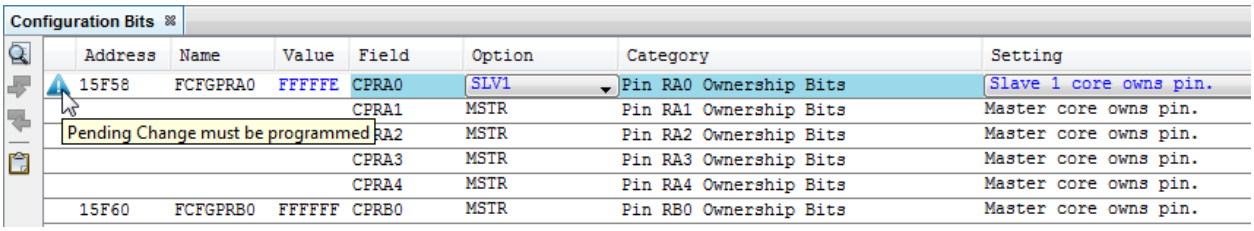
A dual core device is by default in Master Only mode; the Master Slave Interface (MSI) has the MSI1 Master Control Register (MSI1CON) Slave Enable bit (SLVEN) set to '0'.

A project for this device mode is no different from a project for any Microchip device. For details, see 5.1. Create a New Project.

Slave Only Programming

For the Slave project to work independently, the ports for the Slave need to be assigned. Therefore a Master project (e.g., MasterStub) is necessary that programs the Master with the configuration bit settings (like which PORTS are assigned to Slave). For details, see 5.20. Set Configuration Values in the Configuration Bits Window.

Figure 6-22. Master Configuration Bits



Two projects are required for Slave Only Programming mode:

1. Create a MasterStub project per "Create a New Project" and be sure to enable the Slave (SLVEN = '1'). Set the configuration bits as described previously.
2. Create a SlaveOnly project per "Create a New Project" but use the "S1" version of the device, e.g., dsPIC33CH128MP508S1.

MasterStub must be programmed first and then SlaveOnly.

Slave Only Debugging

To debug the SlaveOnly project in the previous section, enable background debug (`S1DEBUG`) in the configuration settings:

```
// FS1ICD
#pragma config S1ICS = PGD2 // ICD Communication Channel Select bits
                            // (Communicate on PGEC2 and PGED2)
#pragma config S1DEBUG = ON // Background Debug (Enabled)
 :
```

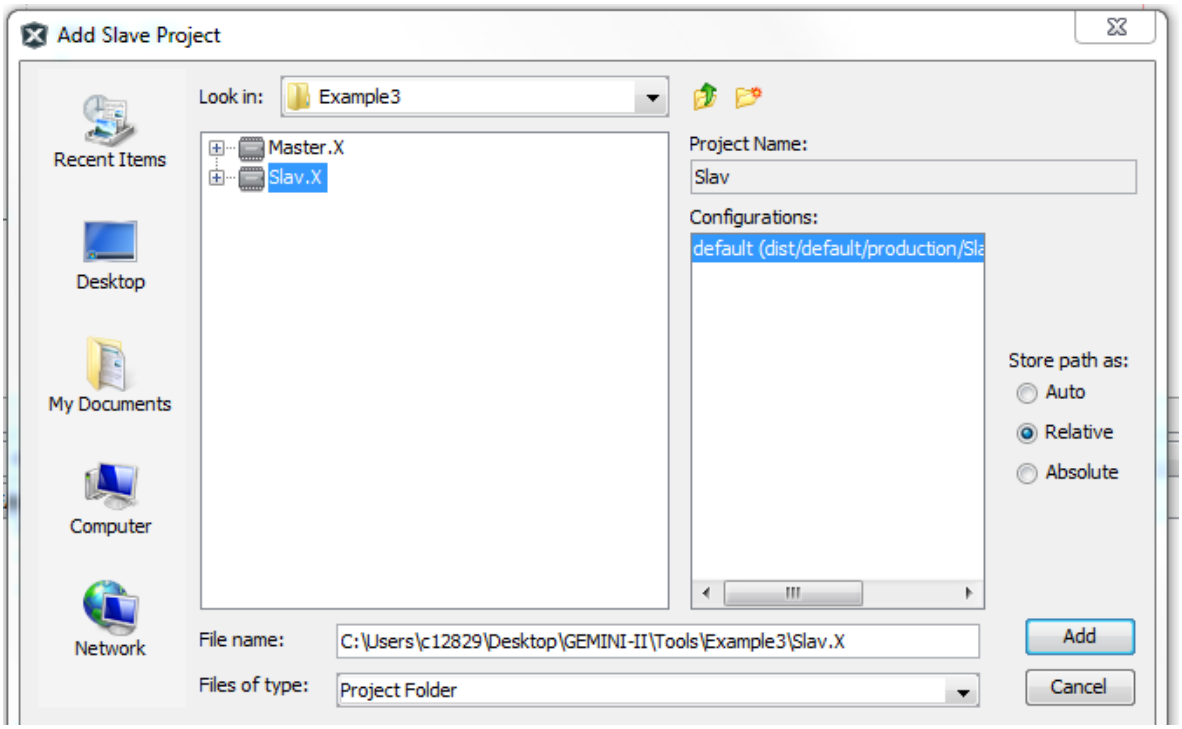
Master and Slave Programming and Debugging

In this mode, the Master project will be linked to the Slave project. That is, the Slave code resides in Master and Master must transfer the code to Slave.

1. Create Master and Slave projects as in "Slave Only Programming."
2. In the Projects window, the Master project will have a folder named "Slaves." Right click on the folder and click "Add Slave Project." Then select the Slave project in the dialog and click **Add**.

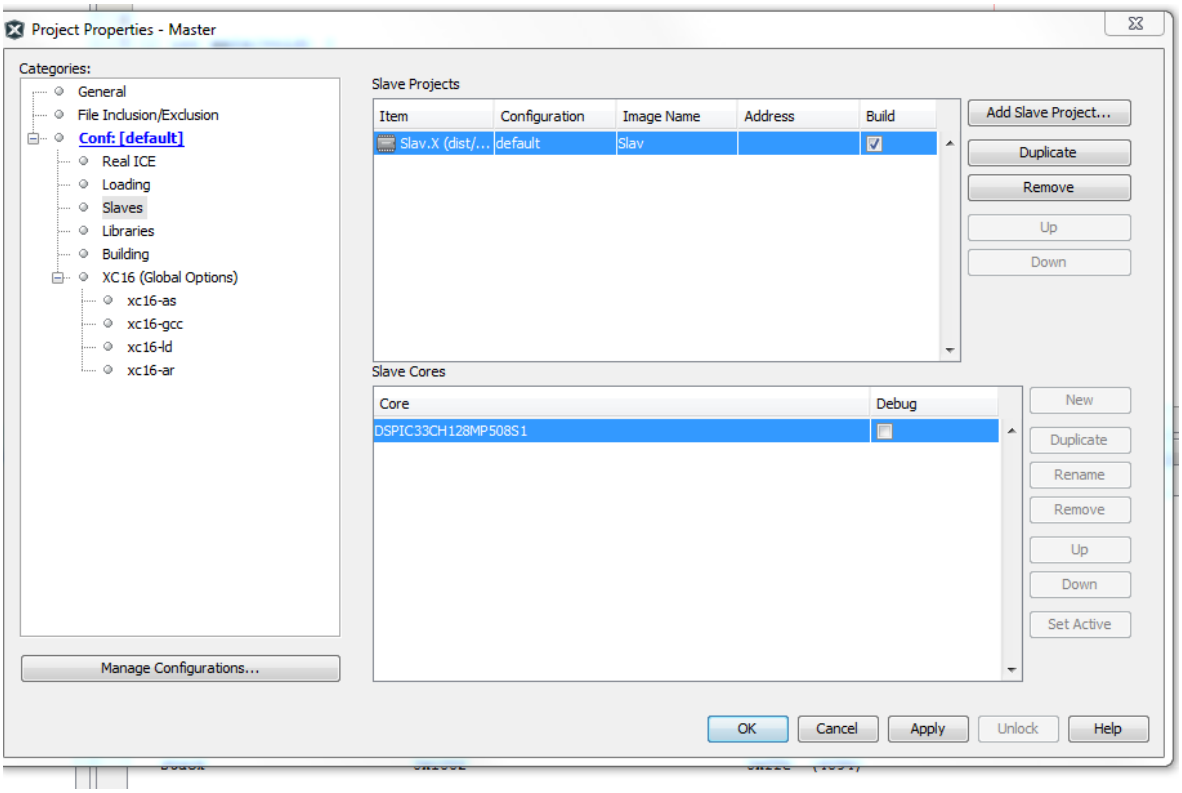
© 2022 Microchip Technology Inc. and its subsidiaries

Figure 6-23. Add Slave Project



3. Right click on the Master project and select “Properties” to open the Project Properties window. Click on the “Slaves” category and make sure to check the “Build” checkbox. Click **Apply**.

Figure 6-24. Master with Slave Project



4. In the Master code add:

```
#include "Slav.h"
//The function to transfer the code from Slave to Master and start the Slave is below.
 _program_slave(1,0,Slav);
 _start_slave();
```

5. By building and programming Master, both Master and Slave projects will be built and both Master and Slave cores will be programmed.

6.2.9 Create User Makefile Projects

Create a project that makes use of an external makefile. This is useful if you have a project that was built outside of MPLAB X IDE, but now you want to use MPLAB X IDE to do debugging.

To create the makefile project, use the New Project wizard.

6.2.9.1 New Project - User Makefile

For ways to open the New Project wizard, see 5.1.2. Launch New Project Wizard.

The wizard will launch to guide you through new project setup.

Step 1. Choose Project: Select the “Microchip Embedded” category and choose from the project type “User Makefile Project.”

Step 2. Select Device: Select the device used in your project from the “Device” drop-down list. To narrow your selection list, choose a Family first.

Step 3. Select Header: This step will appear if a header is available for your selected device. To determine if a header is required for debug, or if your device has on-board debug circuitry, consult one of the documents below. Then choose whether or not to use a header.

- Processor Extension Pak and Debug Header Specification
- Emulation Extension Pak and Emulation Header User’s Guide

Step 4. Select Tool: Select the development tool you will be using to debug your application from the list.
The level of tool support for the selected device will be displayed next to the tool name. That is, green for full support, yellow for beta support and red for no support, yet.

Step 5. Select Plugin Board: This step will appear if a plugin board is available for your selected device. Drop down the list under “Supported Plugin Board” to see what it available.

Step 6. Select Project Name and Folder: Select a name and location for your new project. See Table 1.

Step 7. Create User Makefile Project: Enter data to set up your makefile project (See Table 2). Instructions for General Data Entry in this window are:

- Forward (/) slashes are recommended in your path. However, you may use backward (\) slashes instead, though you may need to escape them (\\).
- A path and file name without spaces is recommended. However, if you do have spaces, use escape spaces (backward slash before the space) when you enter a path and file name in this window. GNU Make is known to cause weird issues with spaces as spaces are their separators.

Step 8. Finish: The new project will open in the Projects window.

Change Makefile project settings in the Project Properties window, “Makefile” category, after the project is created.

For information on exporting a project as hex, see “Project Menu” in 13.19. Projects Window.

Table 6-3. Table 1: Select Project Name and Folder Options

Item	Description
Project Name	Specify a name for your project.
Project Location	Enter or browse to a location for your project.
Project Folder	View the folder location based on the Project Name and Project Location. For information on the relation of the project folder to the working folder, see 6.2.9.2. User Makefile Project Folder and Working Folder.

.....continued	
Item	**Description**
Set as main project	Set project created as the main project. Checked by default.
Overwrite existing project	If project name has been used already, this check box will be active.
Also delete sources	If project exists already and contains source code, this check box will be active.
Use project location as the project folder	Create the project in the same folder as the project location. This is useful if you are importing an MPLAB IDE v8 project and want to maintain the same folder for your MPLAB X IDE project (as when you want the build to be the same). MPLAB X IDE uses a folder to store a project information whereas MPLAB IDE v8 uses an .mcp file. Therefore you can only have one MPLAB X IDE project share a folder with an MPLAB IDE v8 project (.mcp file).
Encoding	By default set as ISO-8859-1. There is usually no need to change this setting.

Table 6-4. Table 2: Create User Makefile Project Options

Item	Description
Working Directory	Specify the location of the external project. This is the default location from where the build, debug build, and clean commands will be executed. For information on the relation of the project folder to the working folder, see 6.2.9.2. User Makefile Project Folder and Working Folder. Browsing to the directory is recommended as this will take care of formatting issues for you*.
Build command	When the command to Run is requested in MPLAB X IDE (icon or menu), perform the build according to this command instead. Build path*: For Windows, use quotes. For Linux or macOS, any spaces must be escaped.
Debug build command	When the command Debug is requested in MPLAB X IDE (icon or menu), perform the debug build according to this command, instead. Debug build path*: For Windows, use quotes. For Linux or macOS, any spaces must be escaped. Also see: microchipdeveloper.com/mplabx:work-outside-compile-for-debug
Clean command	When the command to Clean is requested in MPLAB X IDE (icon or menu), perform the clean according to this command, instead. Clean path*: For Windows, use quotes. For Linux or macOS, any spaces must be escaped.
Image name	Enter the path and name of the production image file (hex). Image path*: Any spaces must be escaped.
Debug image name	Enter the path and name of the debug image file (ELF or COF). Debug image path*: Any spaces must be escaped.
User Include Paths	Enter or browse to user include file(s). Manage include paths in the dialog (Figure 1).
User Macros	Add or parse-and-add macros. Edit: Enter a macro or browse to a file containing one. Manage macros in the dialog (Figure 2). Parse: Automatically parse a macro (MPLAB XC compilers only). Follow the instructions on the dialog. Once complete, choose to Replace any current macro(s) listed or Append to any current list of macros. See 6.2.9.3. Example of a Makefile Project for an example of how to use this dialog.

Figure 6-25. Select Include Paths Dialog

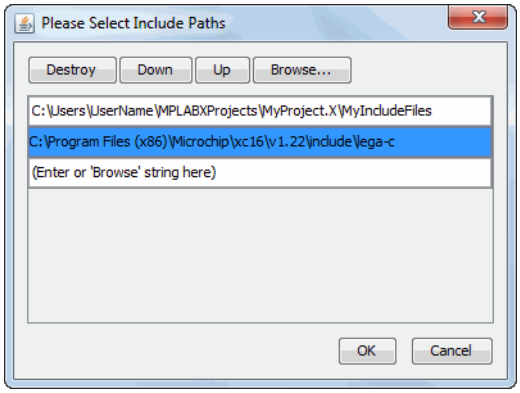
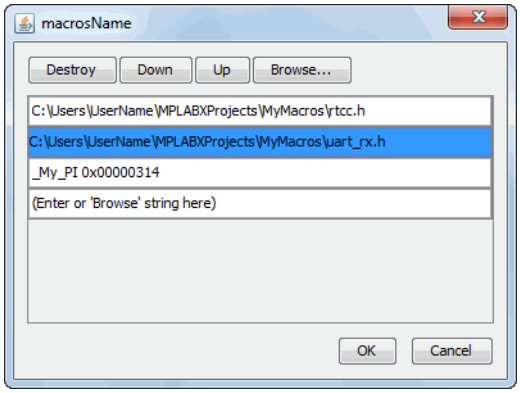


Figure 6-26. macrosName Dialog



6.2.9.2 User Makefile Project Folder and Working Folder

The project folder contains its own `Makefile` and `nbproject/Makefiles`. MPLAB X IDE will run the makefile in the project folder, then change to the working folder (directory) and run the “Build command” or “Debug build command.” In effect, GNU Make is run and then whatever is specified as a build command is run.

The project folder may be located anywhere, with relation to the working folder, if absolute paths are used to define locations. However, this will make your project less portable. Make the working folder relative to the project folder (e.g., one level below the project folder) for your project to be more portable.

Note: Package project is not supported for makefile projects.

6.2.9.3 Example of a Makefile Project

This example shows how to create a user makefile project in MPLAB X IDE that will run either a batch file or make on user code that was created outside of the IDE. The user code consists of a single file `t.c`. The simple code contained in this file is listed below.

Example: `t.c` Code

```
#include <xc.h>
int main(void) {
while(1) {
}
return 0;
}
```

For production the code is linked into `t_production.hex` and for debug the code is linked into `t_debug.elf`.

For this example, `t.c`, the batch file `makeme.bat` and the user makefile `Makefile` are assumed to be in the (Windows OS) directory:

```
C:\Users\MCHP\MPLABXProjects\makestuff\myOwnCodeWithItsOwnMakefile
```

Substitute your *UserName* for MCHP to recreate the example on your PC.

- User Makefile Project – Create Code from a Batch File
- User Makefile Project – Create Code from User Makefile
- User Makefile Project – Complete
- User Makefile Project – Parse Macros

6.2.9.3.1 User Makefile Project – Create Code from a Batch File

To compile the code, the simple batch file makeme.bat takes as arguments production, debug or clean. If production is passed then t_production.hex is built. If debug is passed then t_debug.elf is built. If clean is passed, then the .elf and .hex files are erased.

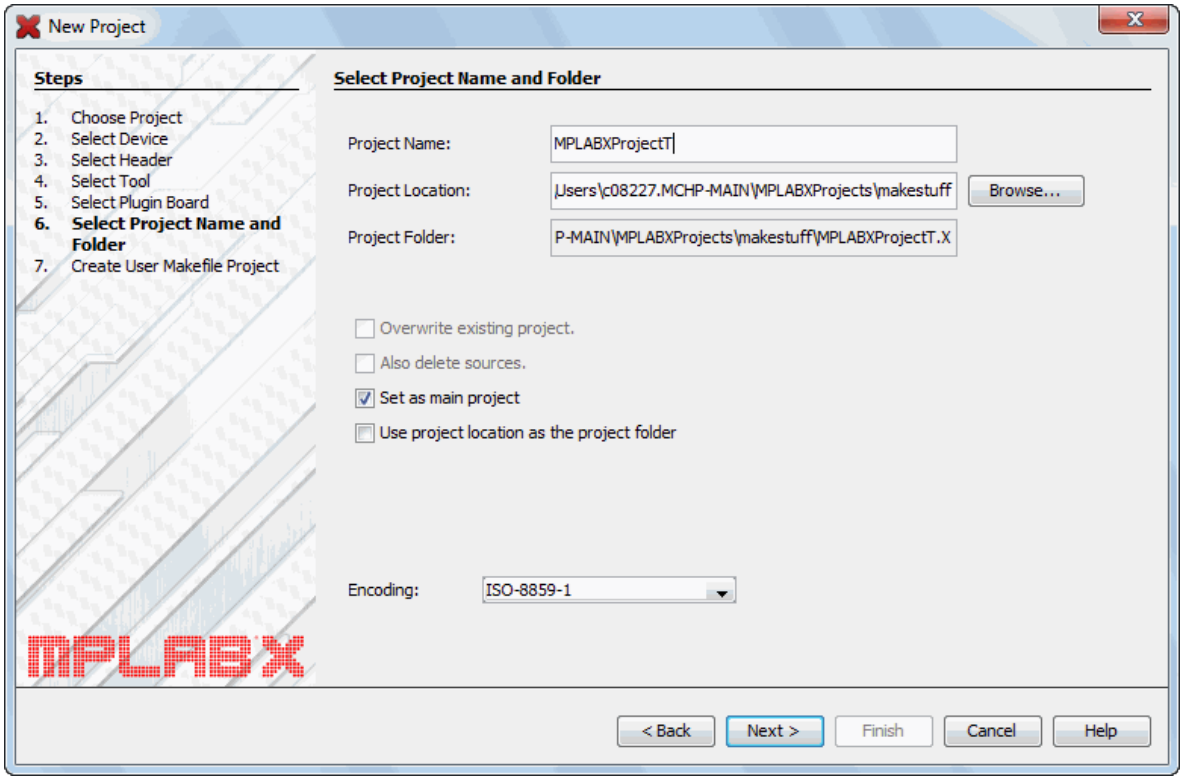
Example: makeme.bat Code

```
::assumes xc32-gcc and xc32-bin2hex are on the path
rem @echo off
set COMPILER=xc32-gcc
set BIN2HEX=xc32-bin2hex
set PROCESSOR=32MX360F512L
if "%1" == "production" goto production
if "%1" == "clean" goto clean
:debug
%COMPILER% -mprocessor=%PROCESSOR% -g -D_DEBUG -o t_debug.elf t.c
goto end
:production
%COMPILER%  -mprocessor=%PROCESSOR% -o t_production.elf t.c
%BIN2HEX% t_production.elf
goto end
:clean
del /F *.elf
del /F *.hex
goto end
:end
```

To create a new user makefile project in MPLAB X IDE, follow the steps in 6.2.9.1. New Project - User Makefile:

1. **Choose Project:** Select “Microchip Embedded,” “User Makefile Project.”
2. **Select Device:** Select PIC32MX360F512L as the device.
3. **Select Header:** None available for this device.
4. **Select Tool:** Select the hardware tool by SN or the Simulator. For this example, the Simulator was used.
5. **Select Plugin Board:** None used for this example.
6. **Select Project Name and Folder:** For this example, the Project Folder will be a sibling to myOwnCodeWithItsOwnMakefile. Therefore the Project Location will be: C:\Users\MCHP\MPLABXProjects\makestuff. The Project Name is MPLABXProjectT.

Figure 6-27. User Makefile Project – Select Project Nane and Folder



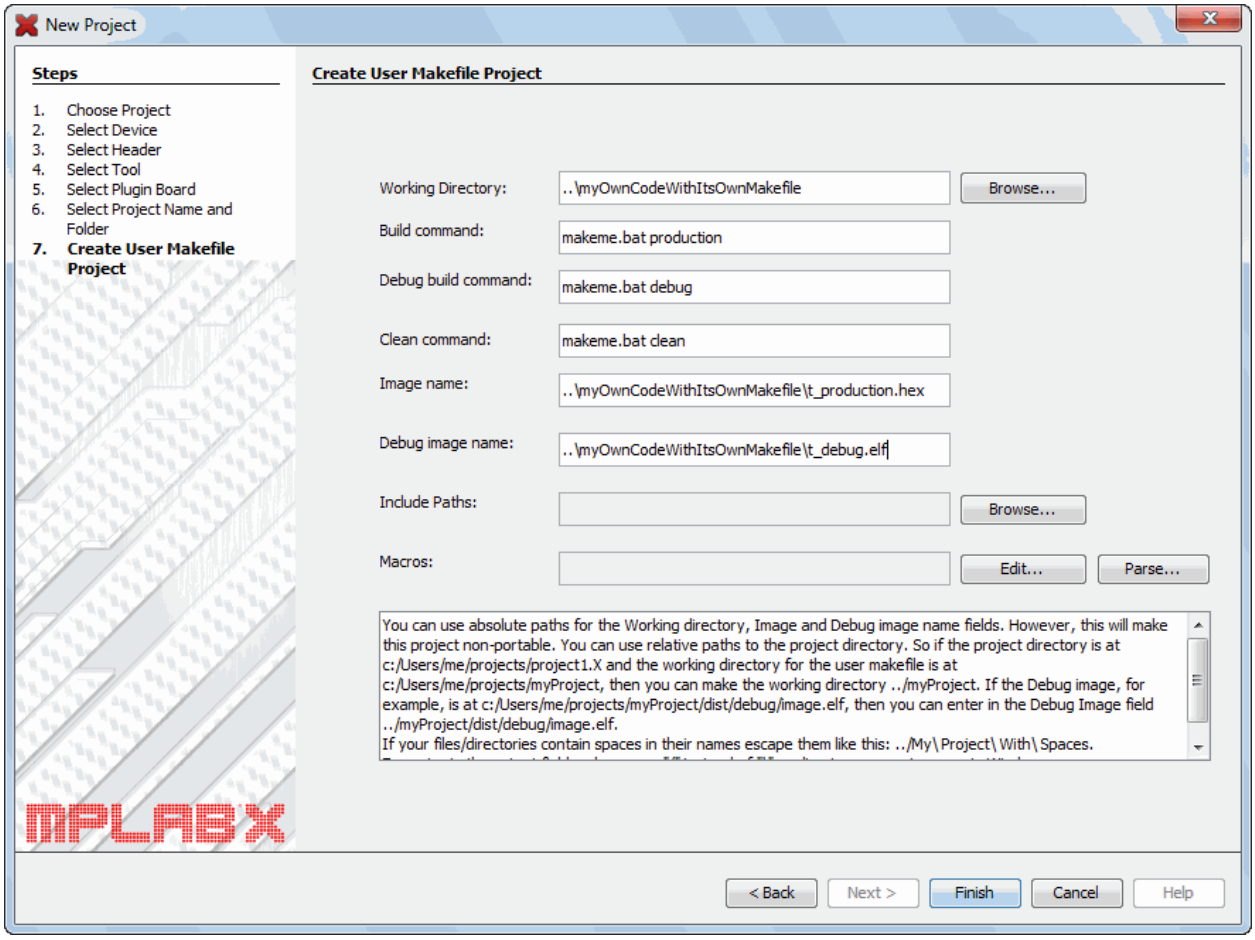
7. **Create User Makefile Project:** Enter information to set up your makefile project.
   The Working Directory is where the batch file build, debug build and clean commands will be run. This directory could be set as the absolute path:

   ```
   C:\Users\MCHP\MPLABXProjects\makestuff\myOwnCodeWithItsOwnMakefile.
   ```

   Doing this, however, makes the MPLAB X IDE project less portable. Therefore the Working Directory, Image name and the Debug image name will be entered as relative paths with respect to the MPLAB X IDE project directory.

   Click **Finish** to create the project.

Figure 6-28. User Makefile Project – Create from Batch



6.2.9.3.2 User Makefile Project – Create Code from User Makefile

To compile the code, the simple user makefile `Makefile` takes as arguments `production`, `debug` or `clean`. If `production` is passed then `t_production.hex` is built. If `debug` is passed then `t_debug.elf` is built. If `clean` is passed then the `.elf` and `.hex` files are erased.

Example: User Makefile Code

```
# Assumes xc32gcc and xc32bin2hex are on the path
COMPILER=xc32gcc
BIN2HEX=xc32bin2hex
PROCESSOR=32MX795F512L
production: t_production.hex
debug: t_debug.elf
t_debug.elf: t.c
$(COMPILER) mprocessor=$(PROCESSOR) -g -D_DEBUG -o t_debug.elf
t.c
t_production.hex: t_production.elf
$(BIN2HEX) t_production.elf
t_production.elf: t.c
$(COMPILER) mprocessor=$(PROCESSOR) o t_production.elf t.c
clean:
del /F *.hex
del /F *.elf
```

To create a new user makefile project in MPLAB X IDE, follow steps 1-6 from "User Makefile Project – Create Code From a Batch File". For Step 7, set up as per figure below.

The Working Directory is where the user Makefile build, debug build and clean commands will be run. The Working Directory, Image name and the Debug image name are entered as relative paths with respect to the MPLAB X IDE project directory.

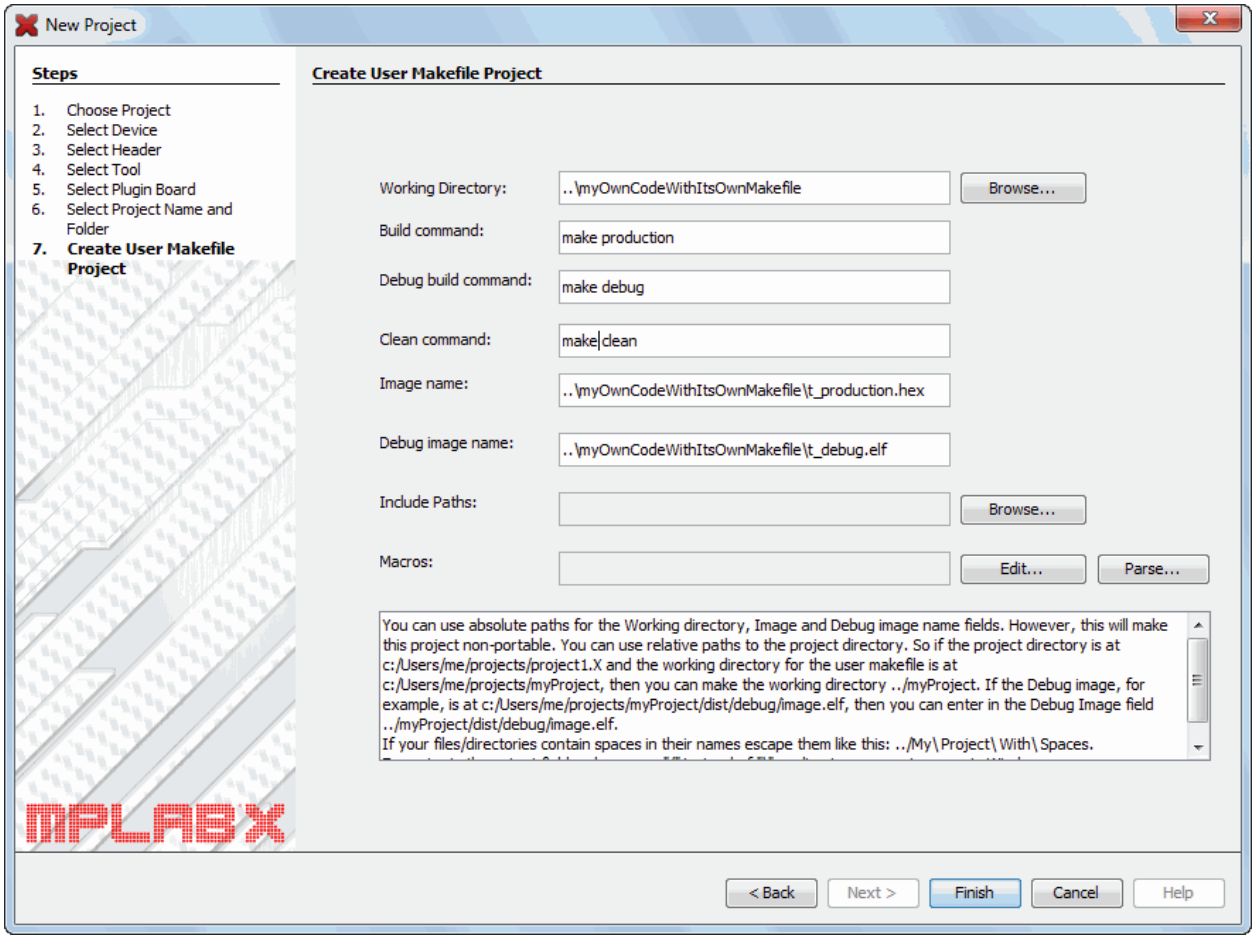
MPLAB X IDE uses GNU Make to build, debug build and clean. As per "User Makefile Project Folder and Working Folder", MPLAB X IDE will run the Makefile in the project folder, which will change directories to the working folder. There, make will look for a makefile and find the user-defined Makefile, which will define how to build, debug build, and clean.

Note: Another way to define commands, for example the Build command, would be to use the `-f` option to specify the user makefile, as in:

```
make -f Makefile production.
```

Click **Finish** to create the project.

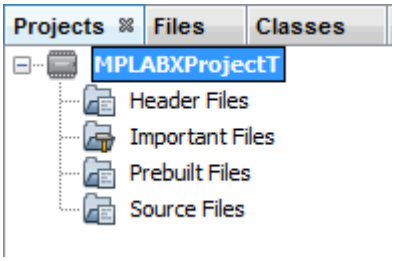
Figure 6-29. User Makefile Project – Create from Makefile



6.2.9.3.3 User Makefile Project – Complete

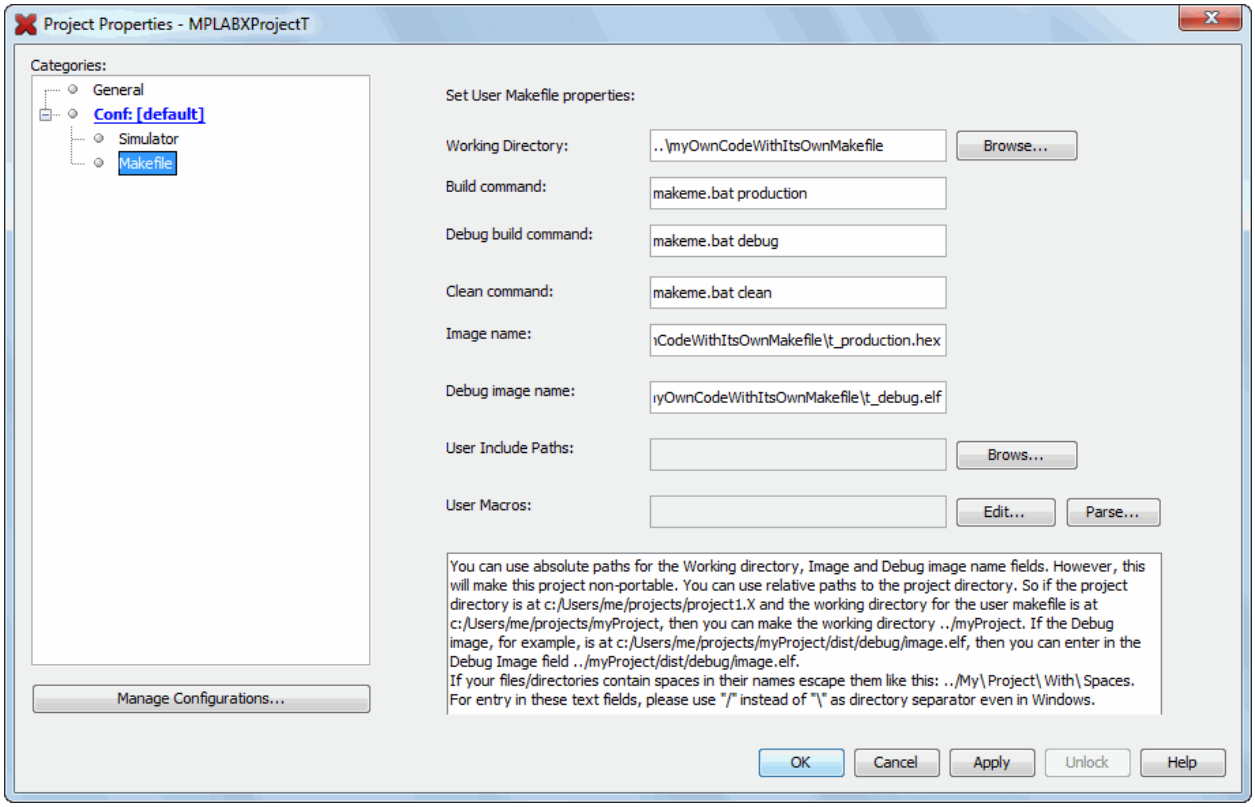
Once the project has been created, you should see the following in the Projects window. You can right click on the project name to build the project.

Figure 6-30. User Makefile Project – Project Tree



If you wish to change settings, right click on the project name and select “Properties.”

Figure 6-31. User Makefile Project – Project Properties



For this example, the results of build (`t_production.hex`) and debug build (`t_debug.elf`) are found in the `myOwnCodeWithItsOwnMakefile` working directory. As only the MPLABXProjectT project directory is visible in MPLAB X IDE, you will not see any changes in the IDE Projects window after a build.

6.2.9.3.4 User Makefile Project – Parse Macros

You can parse and add macros for MPLAB XC C compilers to your makefile project. To do this you need to call the compiler on the command line with the relevant arguments and copy the whole output to the first panel of the “Parse macros” dialog.

To run the compiler on the command line, you will need a C file. For this example, use the code below and paste into a file named `xcmain.c`:

```
#include <xc.h>
void main(void) {
return;
}
```

Ensure that your computer knows the Path to your MPLAB XC compiler. Then execute the following on the command line:

```
xc32-gcc -dM -E xcmain.c > macros32.txt
```

The MPLAB XC32 C compiler is used, not the MPLAB XC32++, so xc32-gcc is the executable. The option `-dM` tells the preprocessor to output only a list of the macro definitions that are in effect at the end of preprocessing. The option `-E` stops execution after the preprocessing stage. The redirect (>) character sends the output into the file `macros.txt`.

For other MPLAB XC C compilers, you can use:

```
xc16-gcc -dM -E xcmain.c > macros16.txt
xc8 --pre -v --chip=8BitMCU xcmain.c > macros8.txt
```

where `8BitMCU` should be replaced with your 8-bit device. The `macros8.txt` file will also require additional editing to isolate the macros from other text in the file.

To open the “Parse macros” dialog:

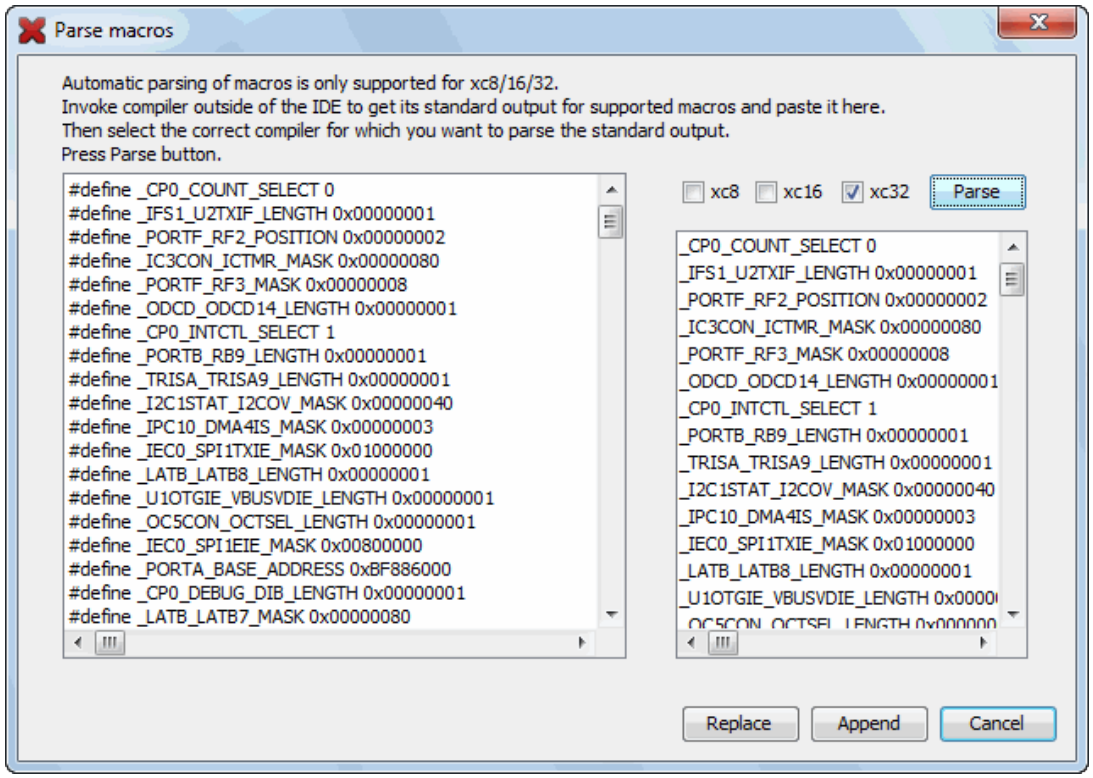
1. Right click on the project name and select “Properties.”
2. Under “Categories,” select “Makefile.”
3. Under “Set User Makefile properties,” go to “User Macros” and click on the **Parse** button.

To parse the macros:

1. Copy and paste the contents of `macros.txt` into the dialog left panel.
2. Check the checkbox for your MPLAB XC C compiler, in this case “xc32.”
3. Click the **Parse** button.

Click a button to **Replace** the current contents of the User Macros text box, **Append** to existing text in the text box, or **Cancel** to leave the existing text unchanged.

Figure 6-32. Parse Macros Dialog

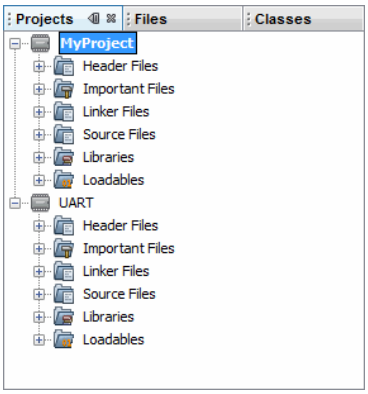


6.2.10 Work with Multiple Projects

MPLAB X IDE allows you to work with more than one project.

Multiple projects can be opened in MPLAB X IDE and viewed in the Projects window. For more about this window, see 13.19. Projects Window.

Figure 6-33. Multiple Projects in the Projects Window



Open Multiple Projects

You can create (*File>New Project*) or open (*File>Open Project*) more than one project in MPLAB X IDE directly.

Alternatively, you can use the `--open` directive in a short cut, as in:

```
"C:\Program Files\Microchip\MPLABX\vx.xx\mplab_platform\bin\
mplab_ide.exe" --open "C:\MPLAB_X_Projects\MyProject1"
```

Then you can create multiple short cuts to open multiple projects from many different locations.

Define Project Type

Each project can be made active, or the focus of development, as described below. However there should only be one main project at any time.

Main Project

A single project can be selected as the main project by right clicking the project name and selecting “Set as Main Project.” The main project name will then appear in bold. In this case all the toolbar functions will be executed on the one main project.

Active Projects

You can also work without setting a main project. Active projects are the projects in focus when there is no main project.

Projects may be made active by clicking on them in the Projects window. In that case, the IDE will know which project you are working on by context, i.e., if you set the editor focus on a file, then the project that owns that file will become the active project. The active project receives all actions, e.g., when you click the Debug Project icon.

Develop Project Code

To develop and debug your code for each project:

1. Start debugging a project by clicking on the project in the Projects window and then selecting *Debug>Debug Project*.
2. If you are running multiple simultaneous debug sessions, open the Sessions window (*Window>Debugging>Sessions*) and you can switch between any currently running debug session.

When one debug session is switched to another, the Watches window and variables plus the memory will switch to show the currently selected project being debugged. The Status bits should also follow the debug project. The Dashboard will follow the last selected whether it is a debug project or a project in the project window. This is by design.

© 2022 Microchip Technology Inc. and its subsidiaries

If you have the option “Maintain active connection to hardware tool” selected under *Tools>Options* (*MPLAB X IDE>Preferences* for macOS), **Embedded** button, **Generic Settings** tab, a reconnect **will be** performed when you switch sessions/projects. This is by design.

Group Multiple Projects

Another way to work with multiple projects is in groups. Select *File>Project Group* and pick or create a project group using the Create New Group dialog box.

6.2.11 Work with Multiple Configurations

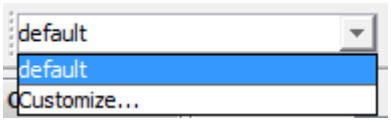
MPLAB X IDE allows multiple build configurations for the same project. This may be useful for code that can be compiled on multiple platforms (such as the Microchip Application Libraries Demo Projects.)

When you create a new project, a “default” configuration is created. To create additional configurations, follow the steps in these sections:

6.2.11.1 Manage Configurations Dialog

To create your own configuration, start by doing one of the following:

- Use the drop down menu on the toolbar and select “Customize”. The Project Properties window will open.



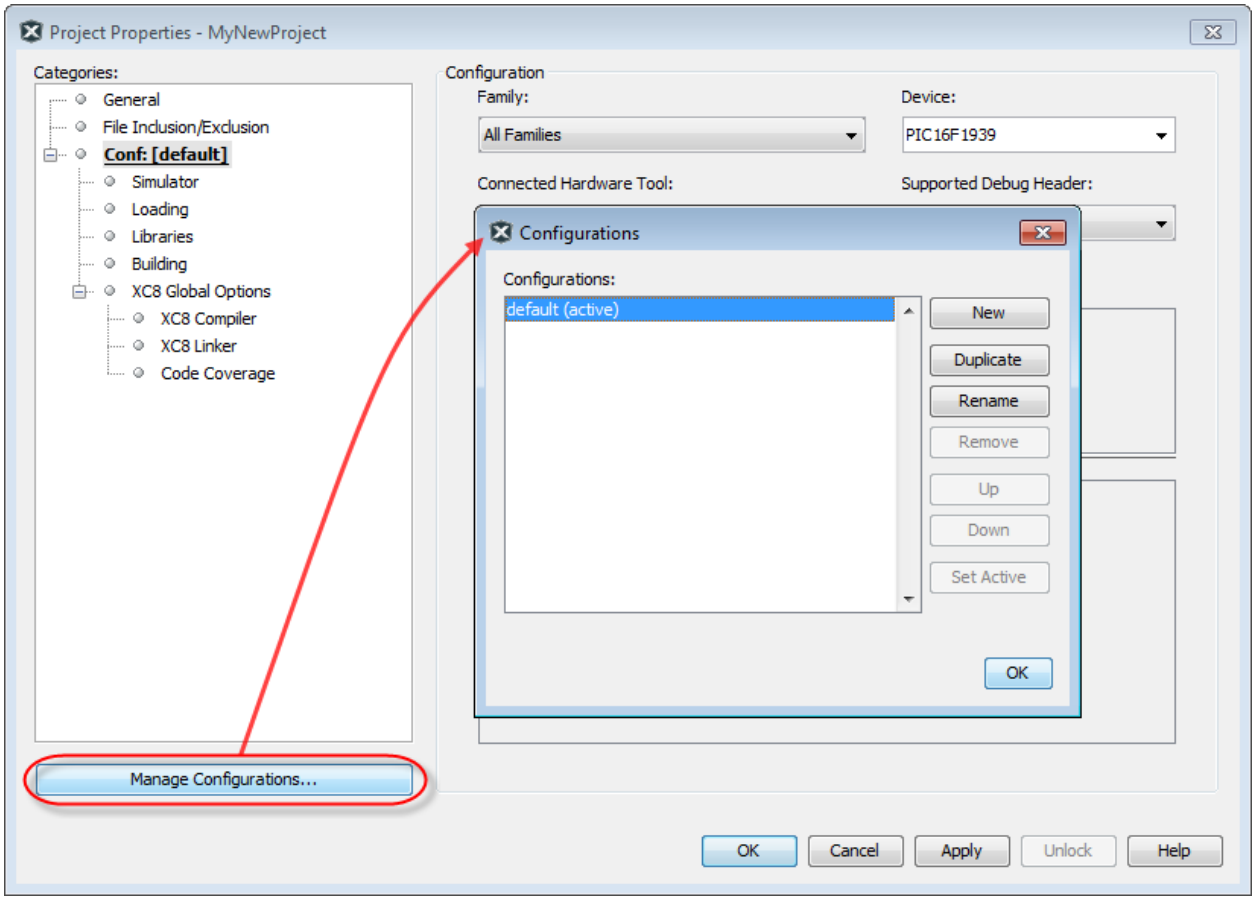
- Open the Project Properties window by right clicking on the project name and selecting “Properties”.

In the Project Properties window, click **Manage Configurations** to open the Configurations dialog. Existing configurations can be renamed or a new configuration can be added or duplicated from an existing one. Any spaces in the name will be replaced with underscores.

When more than one configuration is created for a project, the active one can be selected from **Manage Configurations** or from the drop-down menu.

© 2022 Microchip Technology Inc. and its subsidiaries

Figure 6-34. Project Properties – Configurations Dialog

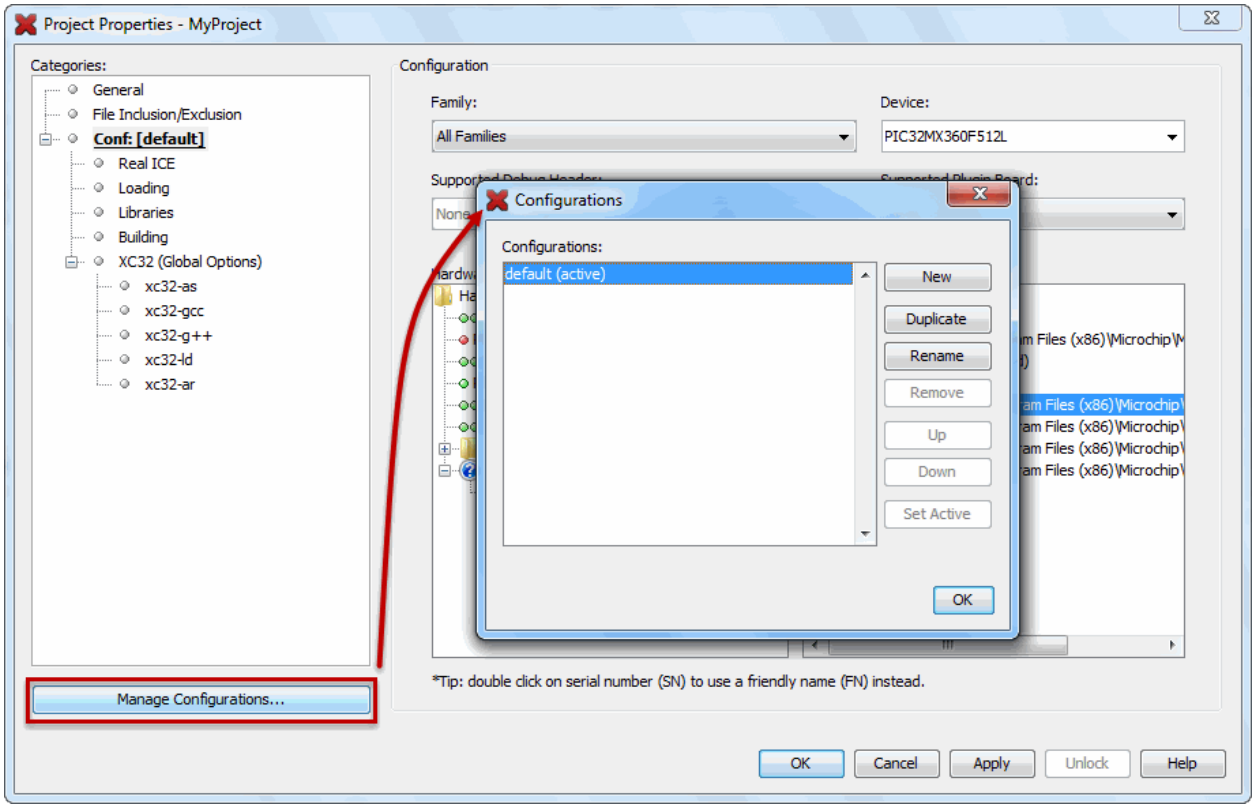


6.2.11.2 Add a New Configuration

To add a new configuration to your project:

1. Click **Manage Configurations** in the Project Properties window.
2. In the Configurations dialog, select a project configuration and click **New**.
3. Enter a name in the New Configuration Name dialog, such as “My Test” (see figure below). Click **OK** to close the dialog. Under “Configurations,” the name will be changed to “My_Test” since spaces are not allowed.
4. Click **OK** to return to the Project Properties window. The debug configuration (Conf: My_Test) should now be visible.

Figure 6-35. Create New Configuration



Once a new configuration is added, you must select the configuration options in the Project Properties window (see figure below).

- Device – You must select this first to see the hardware tool and compiler support.
- Hardware Tool and any supporting Debug Headers and/or Plugin Boards.
- Packs - Select the pack version you want to use.
- Compiler Toolchain - Select the compiler version you want to use.

Once you have made your selections, click **Apply** or **OK** to finish the new configuration.

Figure 6-36. New Configuration - Empty

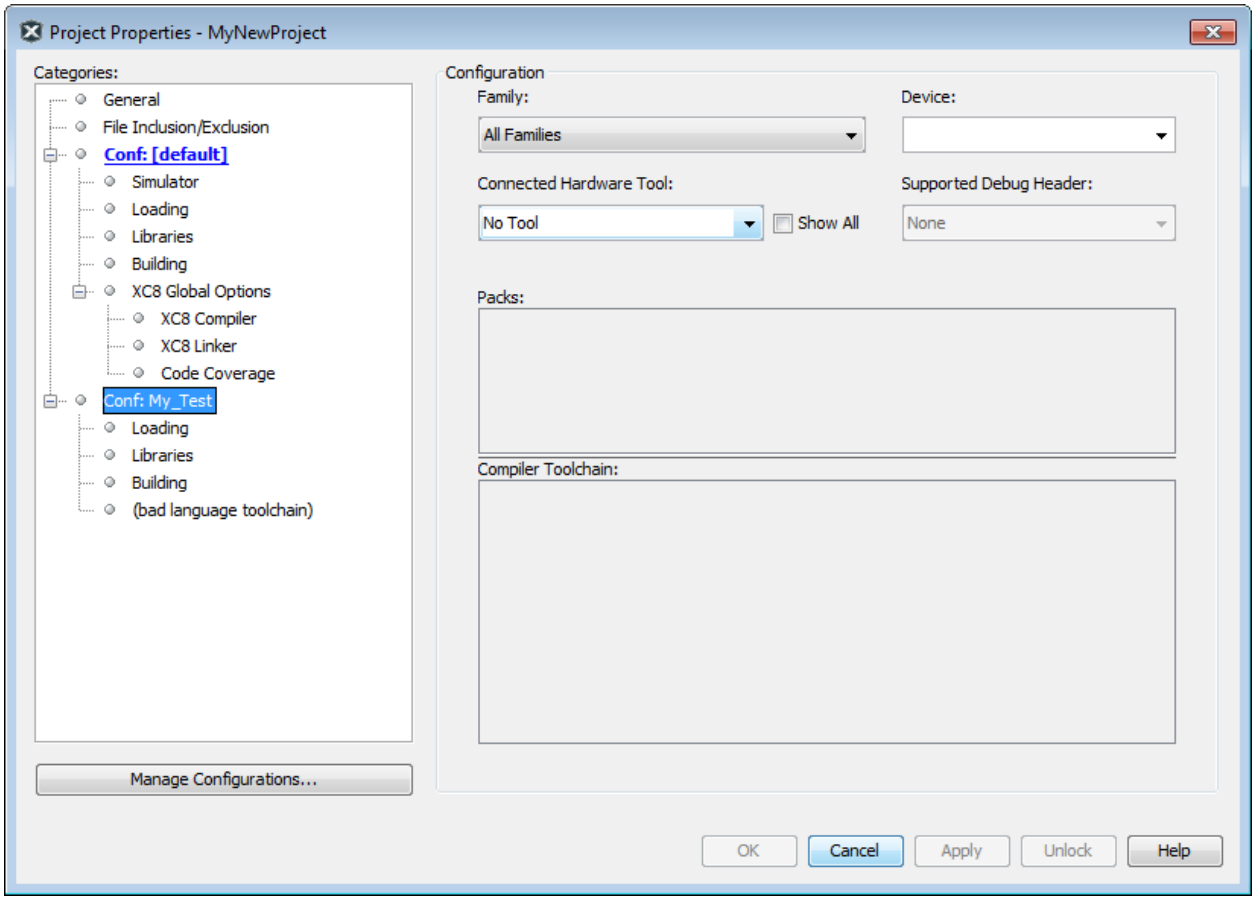
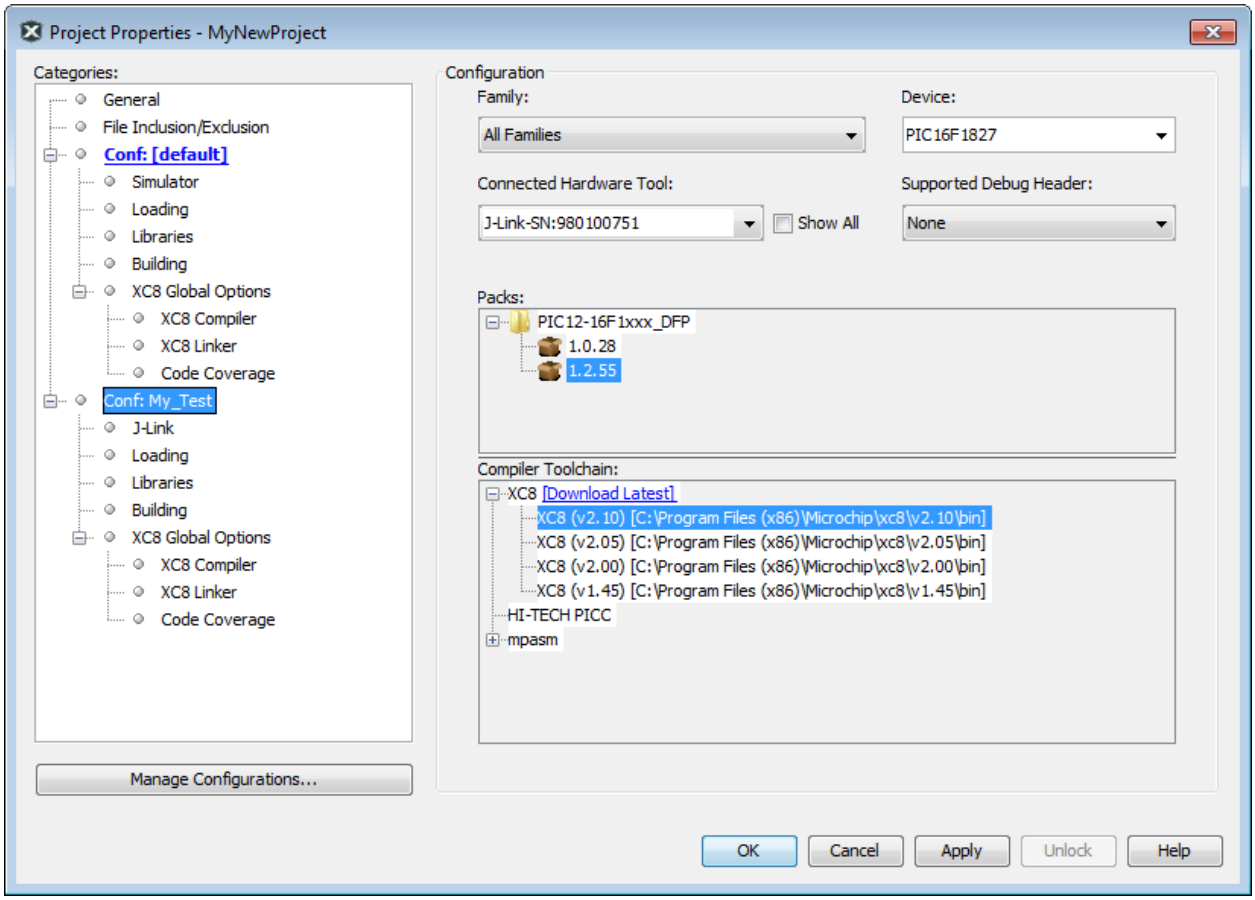


Figure 6-37. New Configuration - Apply



6.2.11.3 Add a Duplicate Configuration

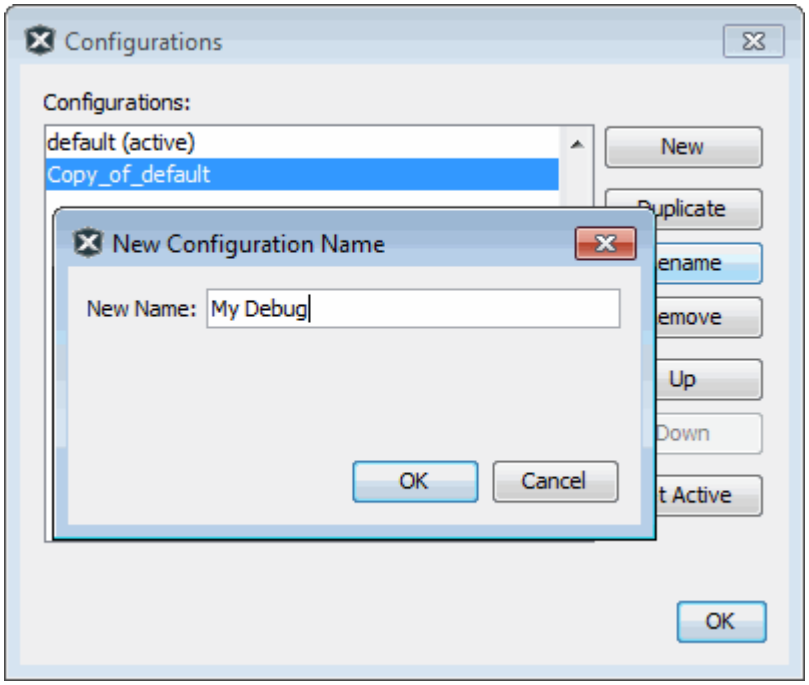
You can add a configuration that is the duplicate of an existing one and then make edits to it.

One reason to make a duplicate configuration is to create your own debug configuration. Although MPLAB X IDE provides debug macros for use with Microchip tools (see “Debug Macros Generated” in 5.13. Debug Code), you may want to use your own debug macros or you may want to set up the same debug capabilities with third-party tools.

To set up a **debug configuration**:

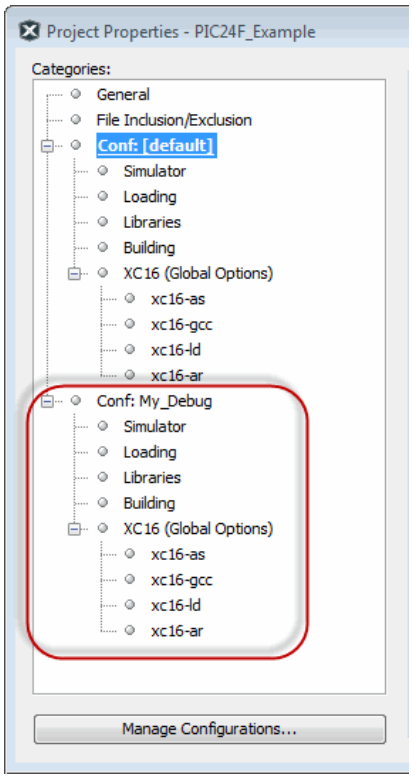
1. Click **Manage Configurations** in the Project Properties window.
2. In the Configurations dialog, select a project configuration and click **Duplicate**.
3. Click **Rename** and enter a name in the New Configuration Name dialog, such as “My Debug” (see figure below). Click **OK** to close the dialog. Under “Configurations,” the name will be changed to “My_Debug” since spaces are not allowed.

Figure 6-38. Create Debug Configuration



4. Click **OK** to return to the Project Properties window. The debug configuration (Conf: My_Debug) should now be visible (see figure below).

Figure 6-39. Debug Configuration



To use macros in code to switch to other configurations, like debug, see 6.2.11.6. Configuration Macros.

6.2.11.4 Manage Files in Configurations

You can manage your project files by assigning them to individually or as a group to the configurations you have created.

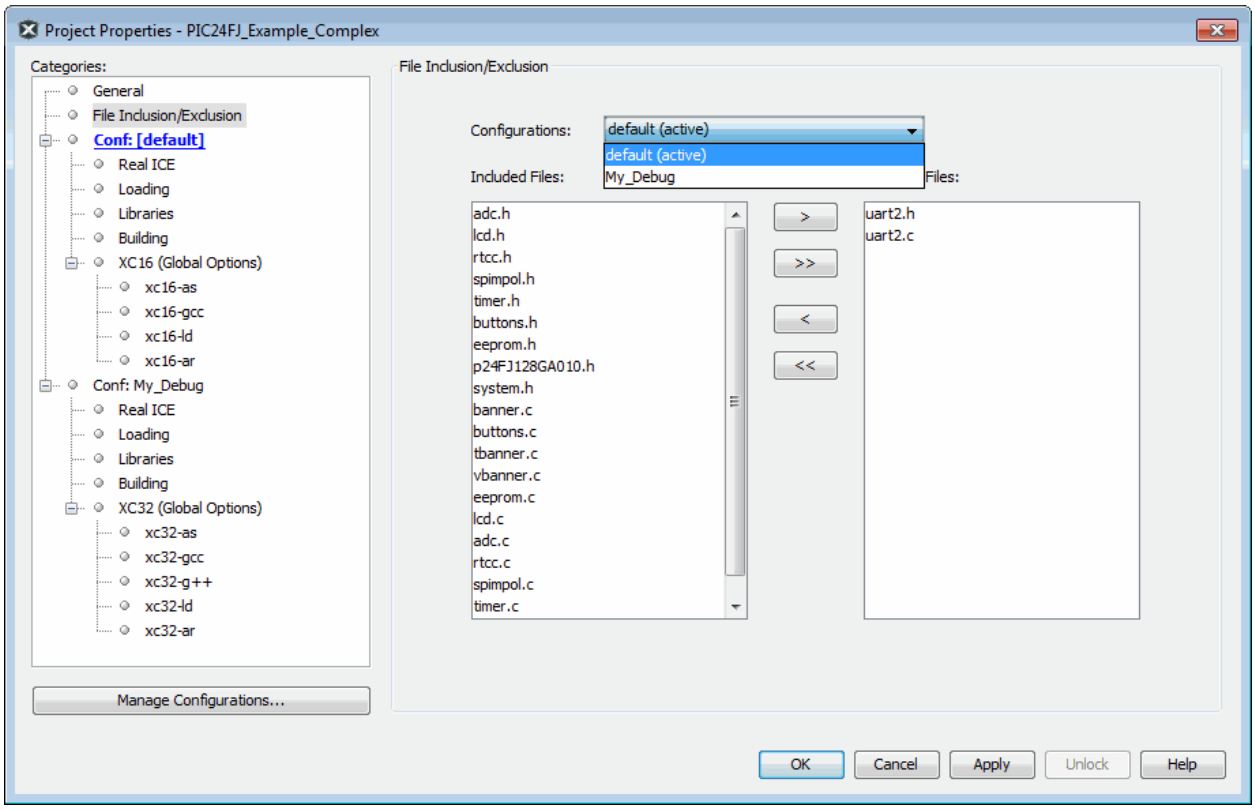
Projects Window

Select the configuration you want in the “Set Project Configuration” drop-down box on the toolbar. Then select the file or files (Shift-Click or Ctrl-Click) and right click on one of them to pop up a context menu. Select “Exclude file(s) from current configuration.” To include a file or files again, repeat to see on the context menu “Include file(s) from current configuration.”

Project Properties Window

To set the configuration you want to apply to many files, right click on the project name in the Projects window to open the Project Properties window. Select the “File Inclusion/Exclusion” category. Select a configuration from the Configurations drop down box and then specify which files to include or exclude from this configuration.

Figure 6-40. Project Properties Configuration



6.2.11.5 Configuration Names

Some special characters cannot be used in configuration names. This is due to being able to use the configuration name within the source code as a define macro (“Configuration Macros”).

Special characters like a hyphen (-), ampersand (&), etc., will be automatically replaced with an underscore (_).

6.2.11.6 Configuration Macros

When you add a new or duplicate configuration, you can create your own configuration-related macros or use autogenerated macros for #ifdef or other statements in code. These are MPLAB XC C compiler preprocessor macros.

User-Defined Macros

To define your own preprocessor macro:

1. In the Project Properties window, click on a project configuration to make it active. Under that configuration, click on the compiler in the toolchain.

© 2022 Microchip Technology Inc. and its subsidiaries

2. Under the “Preprocessing and messages” options category, locate the option that allows you to define a macro and click on the associated text box.
3. In the pop-up dialog, enter a macro name and click **OK**. This macro is now associated with the active configuration.
   You can name the macro the same as a configuration (such as “My_Debug” from 6.2.11.3. Add a Duplicate Configuration) or you can give it any other name (such as “DebugConfig”).

   Macro names (and strings, discussed below) are case-sensitive.

You can now use the preprocessor macro in conditional text:

```
// If My_Debug configuration is active
#ifdef DebugConfig
fprintf(stderr,"This is a debugging message\n");
#endif
```

In addition, you can assign a string value that matches the configuration name to your macro(s) for use in regular code:

```
#ifdef DebugConfig
#define CFG_NAME “My_Debug”;
// other defines
#endif
#ifdef DefaultConfig
#define CFG_NAME “My_Default”;
// other defines
#endif
:
int main(int argc, char** argv)
{
printf(CFG_NAME);
return{EXIT_SUCCESS};
}
```

The `CFG_NAME` macro will always hold the active configuration's name.

Autogenerated Macros

When a new or duplicate configuration is created, for example `My_Test`, a related macro is also created, for example `XPRJ_My_Test`. The macro will have a string value which matches the name of the configuration (“My_Test”).

You can use the macro for `#ifdef` statements as in the previous section, or you can use the value of the macros at compile time as follows:

```
int main(int argc, char** argv)
{
printf(CFG_NAME);
return{EXIT_SUCCESS};
}
```

The `CFG_NAME` macro will automatically hold the active configuration's name.

6.2.12 Other Embedded Projects

MPLAB X IDE can create a project from selected, other embedded projects.

1. Select *File>New Project*.
2. Click on “Other Embedded” under “Categories” and select from a list of available embedded projects.
3. Continue to create an MPLAB X IDE project.

This feature imports your existing files into an MPLAB X IDE project.

IAR Embedded Workbench®

The device names used in an IAR project may be slightly different from the device names used in an MPLAB X IDE project. Please verify that the suggested device is correct.

For an MPLAB XC C compiler selected as the toolchain, the following settings in the IAR project are imported into the resulting MPLAB X IDE project:

- Project files
- Include directories
- Defined symbols (called "Preprocessor macros" in the MPLAB X IDE project)

You must configure all other compiler settings manually. Consult the *MPLAB XC8 C Compiler User's Guide for AVR MCU* or *MPLAB XC32 C/C++ Compiler User's Guide for PIC32C/SAM MCUs* for more information.

Additional Information

For information on how to work with MPLAB X IDE, see:

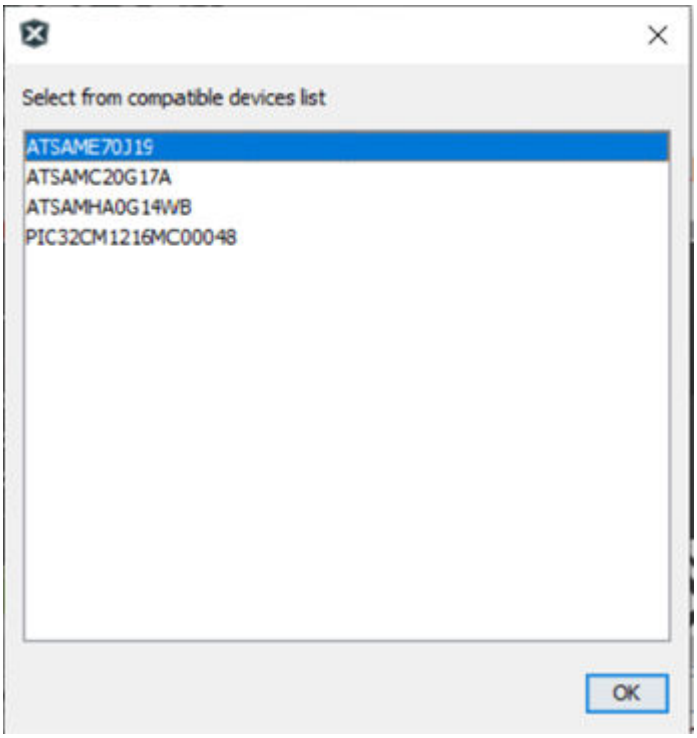
- Tutorial
- Basic Tasks

For information on available compilers, see:

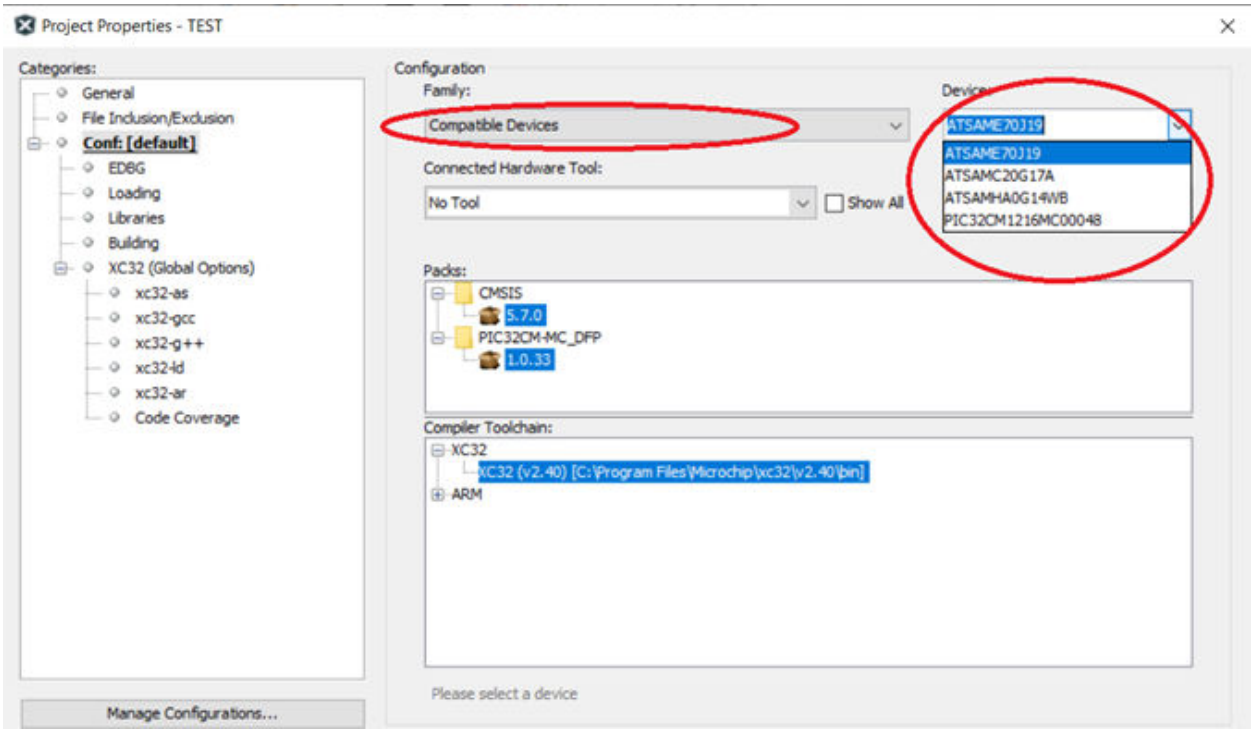
www.microchip.com/mplab/compilers

6.2.13 Use a Compatible Device Project

Some projects are configured to list a set of the devices that are supported. When opening such a project for the first time, a window will pop up requesting that you "select from compatible device list" to choose a project device. This window will not appear again after the selection.

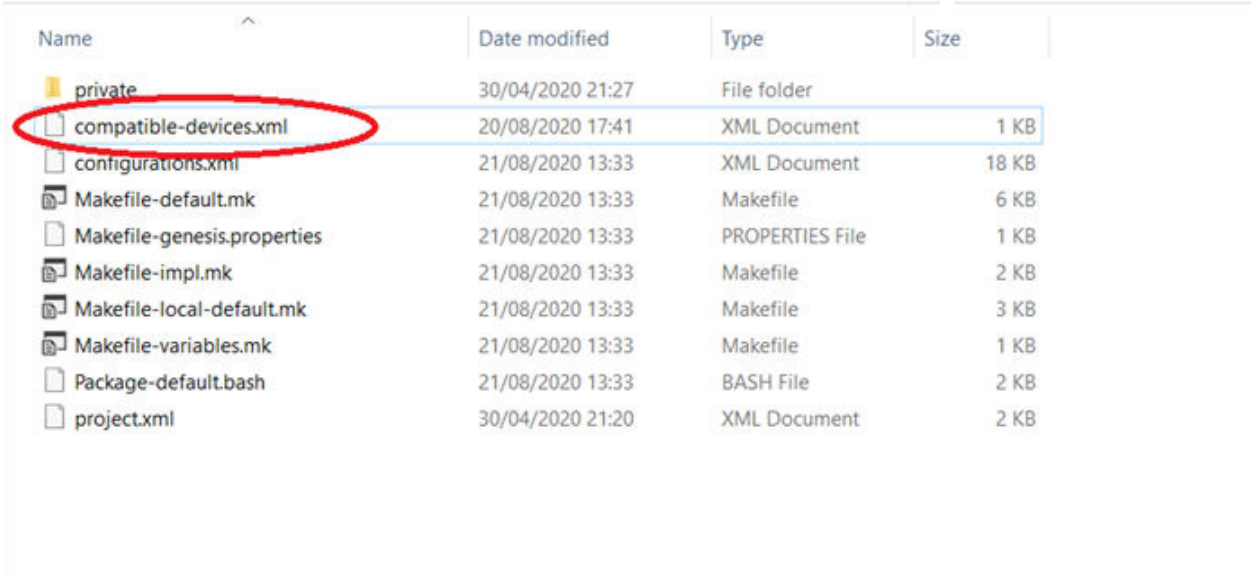


To change the project device at a later time, open the Project Properties window. Under "Configuration" and "Family," select "Compatible Devices" from the drop-down list. Then under "Device" you will again be able to select from a list of compatible devices.



Device List File

The device list is in an XML file in the project (see figure below). Each device has been tested with the example code to ensure it builds.



An example of the file format would be:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<compatibleDevices>
  <default>ATSAME70J19</default>
  <selectable>true</selectable>
  <device>ATSAME70J19</device>
  <device>ATSAMC20G17A</device>
  <device>ATSAMHA0G14WB</device>
  <device>PIC32CM1216MC00048</device>
</compatibleDevices>
```

6.2.14 Sample Projects

Create a sample project to help you learn about Microchip devices, tools and MPLAB X IDE.

1. Select *File>New Project*.
2. Click on “Samples>Microchip Embedded” under “Categories” and select from a list of available embedded projects (that blink demo board lights) or template projects. Read the Description for more information.

Part numbers for demo boards are as follows:

- Explorer 16 Demo Board: DM240001
- PICDEM 2 Plus: DM163022-1

6.3 Project Files

Enter a short description of your topic here (optional).

This is the start of your topic.

6.3.1 Work with Other Types of Files

When selecting *File>New File*, you are presented with many types of files. Using Microchip compiler files has been explored previously. However, there are other types of files you can select depending on your project language tool or need to create a specific file.

Table 6-5. File Types

Category	File Type
Microchip Embedded	Files for language tools that are supported in MPLAB X IDE Select your compiler folder to see the available file types.
C	Generic C files
C++	Generic C++ files
Assembler	Generic assembly files
Shell Scripts	Shell script files: Bash, C, Korn, etc.
Makefiles	Makefile files
XML	XML files
Other	Other types of files, such as HTML, JavaScript, etc. If you do not see the type of file you want listed here, select “Empty File.” In the next window, name the file with the desired extension.

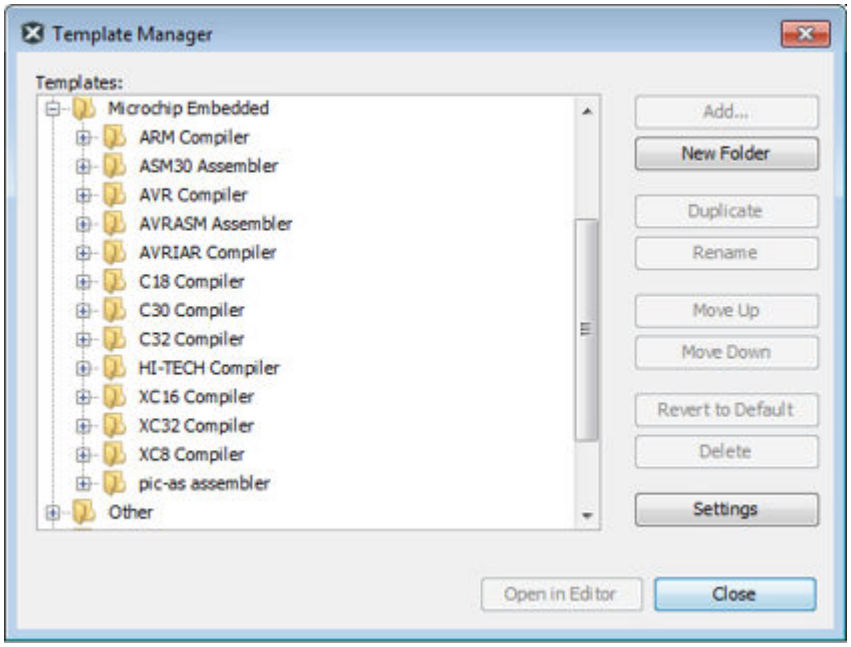
6.3.2 Modify or Create Code Templates

When you create a file to add to your project (5.9.2. Create a New File), a template is used for the new file. To change this template, select *Tools>Templates* and then “Open in Editor” to edit a template. You may also use “Add” or “Duplicate” in this dialog to create new templates.

“New Folder” can be used to create a new folder to hold templates. Be aware that MPLAB X IDE filters out all but Microchip Embedded, Shell Scripts, Makefiles and Other, so files or folders should be created under those folders.

© 2022 Microchip Technology Inc. and its subsidiaries

Figure 6-41. Template Manager



You may set template options by selection *Tools>Options* (*MPLAB X IDE>Preferences* for macOS), **Editor** button, **Code Templates** tab (as shown below).

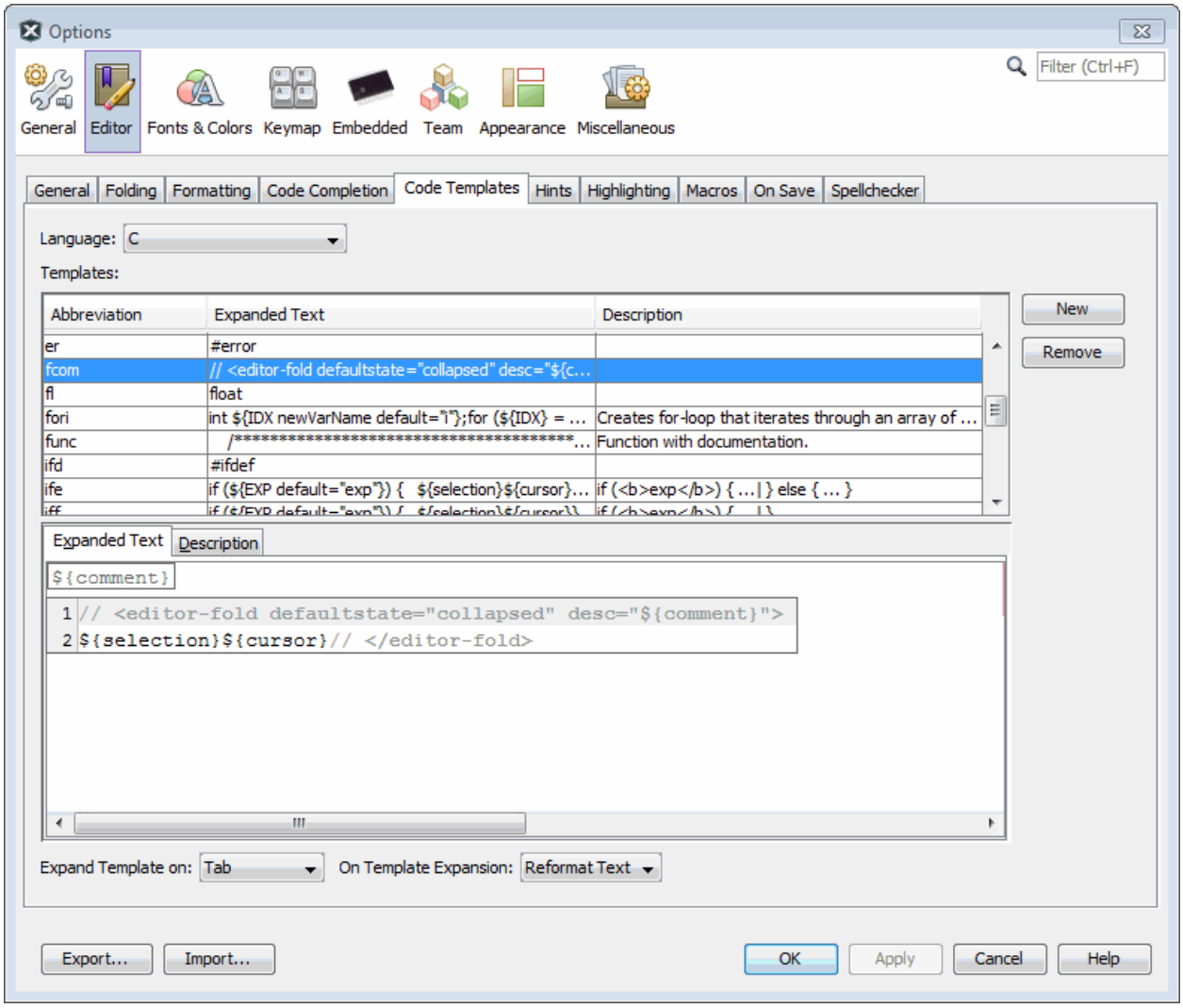
An option of note for C code is the “fcom” option. In an Editor window, type “fcom” and then press “Tab” to insert the following text into the source code:

```
// <editor-fold defaultstate="collapsed" desc="${comment}">
${selection}${cursor}// </editor-fold>
```

This option allows you to hide/view sections of code.

In the **Code Template** tab, selecting “fcom” will display “${comment}” in the “Expanded Text” section. Hover over this text to see the full text.

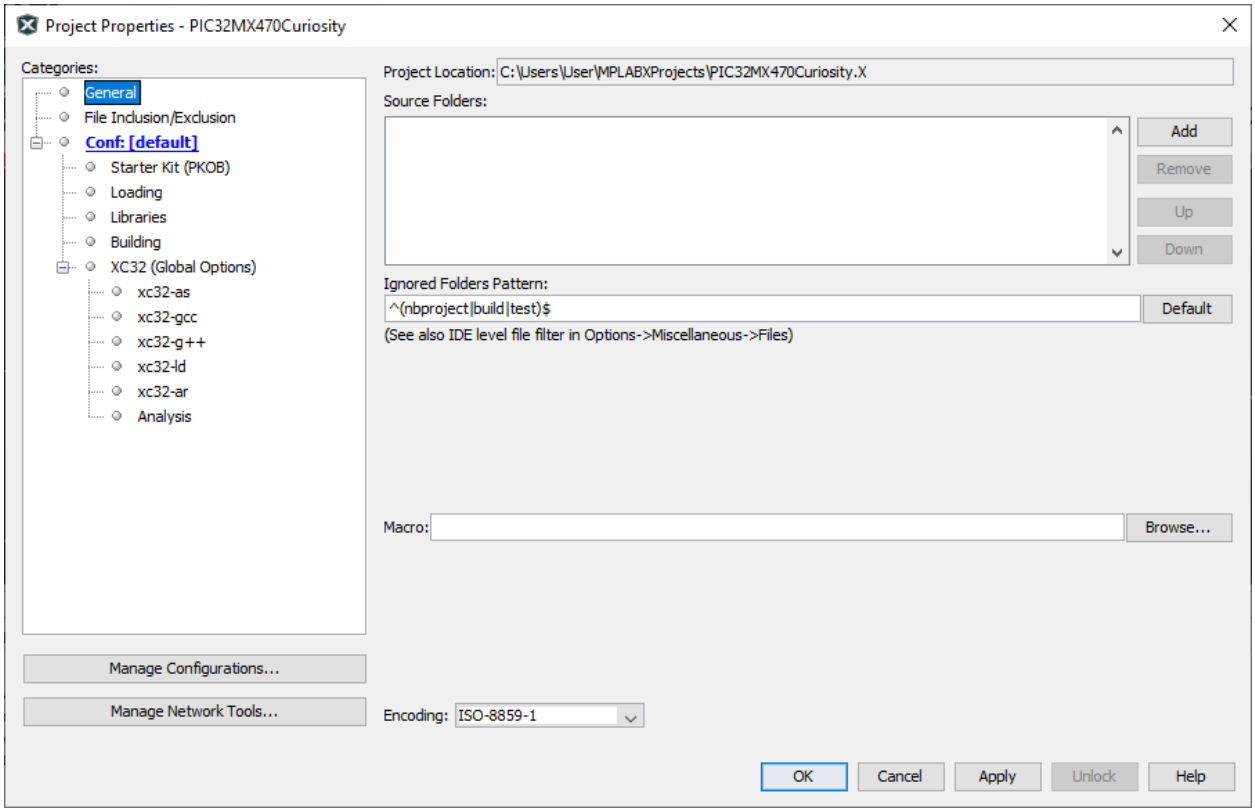
Figure 6-42. Template Options



6.3.3 Modify Project Folders and Encoding

When you created your project, you specified the project folder and encoding. Once the project is created, you may add or ignore project folders and change the project encoding using the “General” category in the Project Properties window.

Figure 6-43. Project Properties – General



Project Location
View the current project location. See 6.5.7. Moving, Copying or Renaming a Project to change the project location.

Source Folders
Add folders for MPLAB X IDE to search when looking for project files.

Note: Adding files outside the project folder may make the project less portable.

Ignored Folders Pattern
Ignore folders in the project folder according to the regular expression pattern specified.

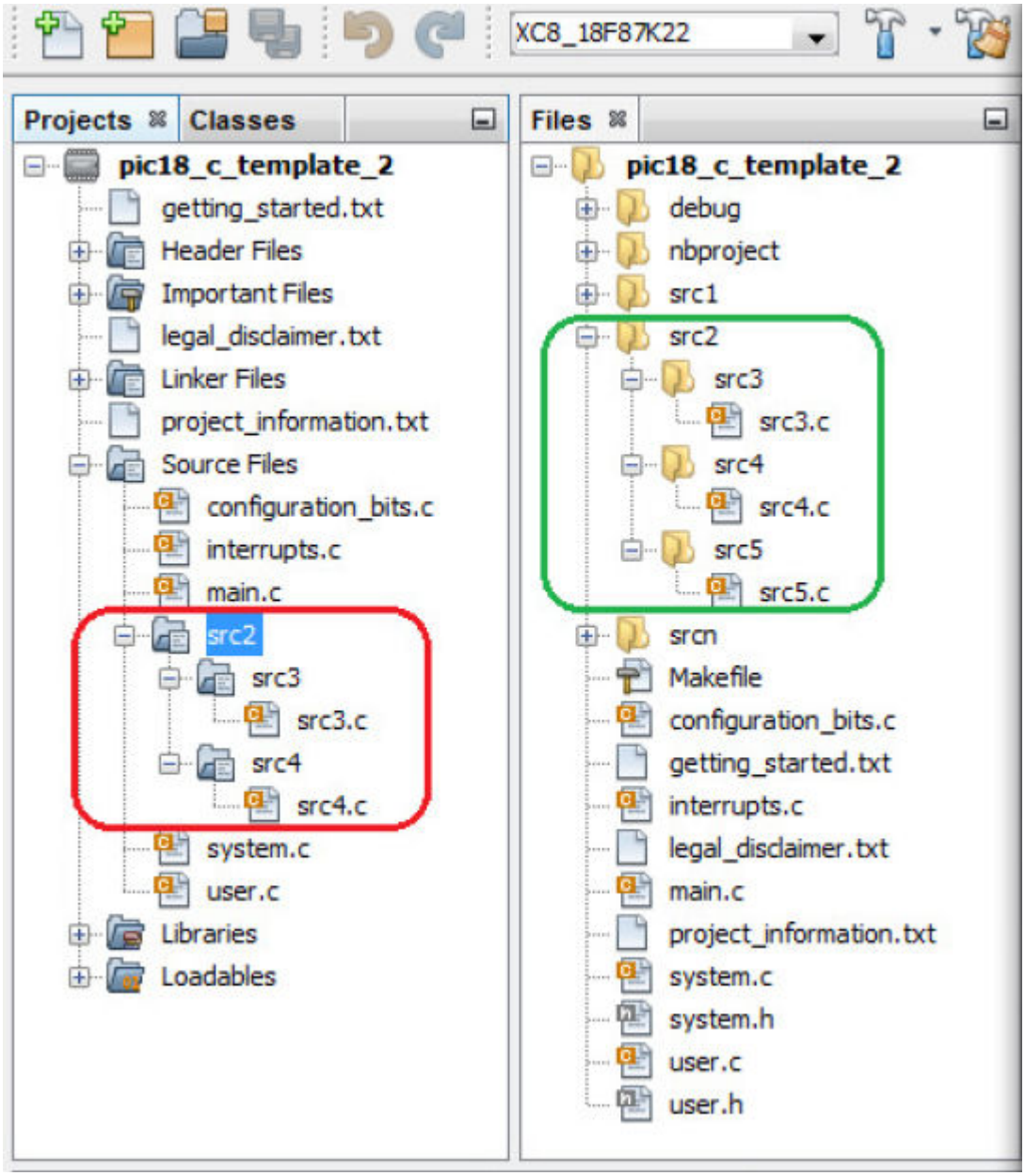
***Example*:** Exclude folder `src5`

To exclude the `src5` folder and its contents from the project, enter the following regular expression in the “Ignored Folders Pattern” text box:

```
^(src5|nbproject|build|test)$
```

View the Projects window to see no `src5` folder. View the Files window to see it does exist, but has been excluded.

Figure 6-44. Exclude Folder from Project



When the project is built, ensure that there is no mention of the `src5` folder or `scr5.c` file contained in the Build tab of the Output Window.

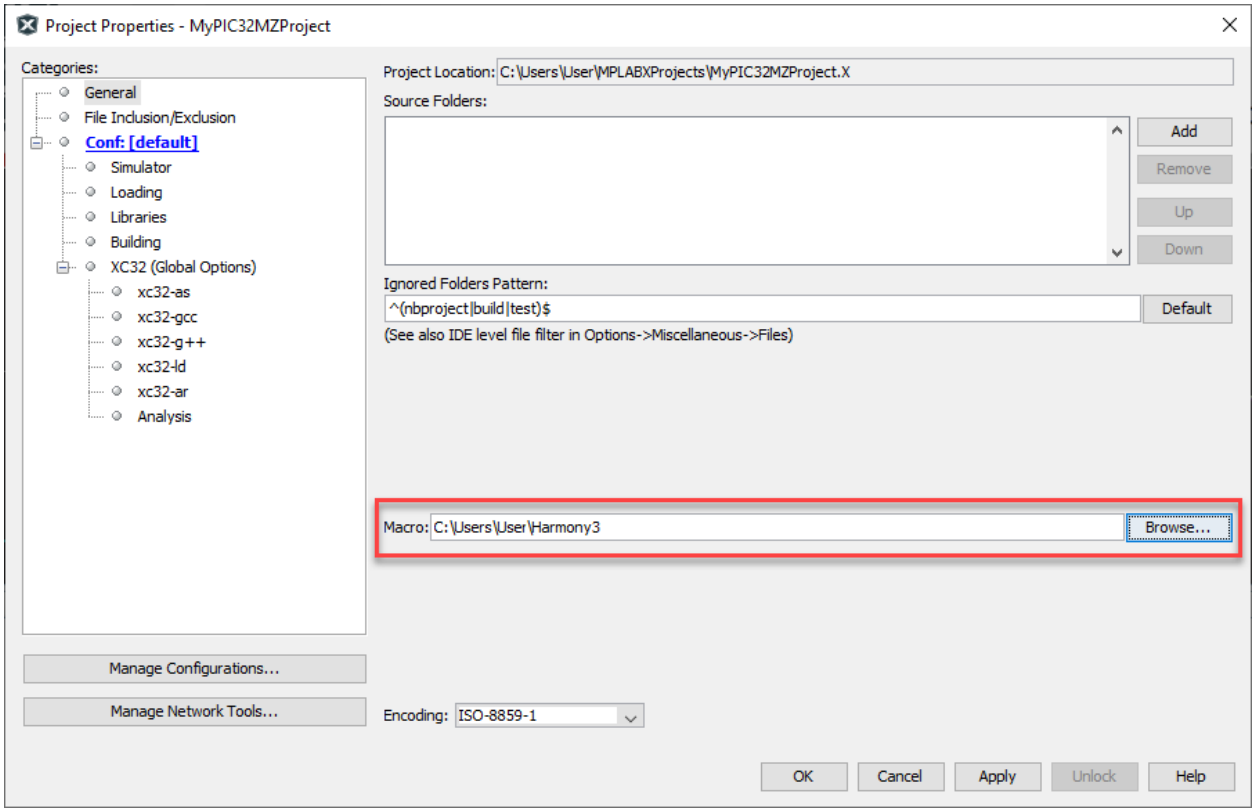
Macro

Enter a path to source files to be used in the “Add Existing Items” selection dialog to link to those specified source files.

Example: MPLAB Harmony Files

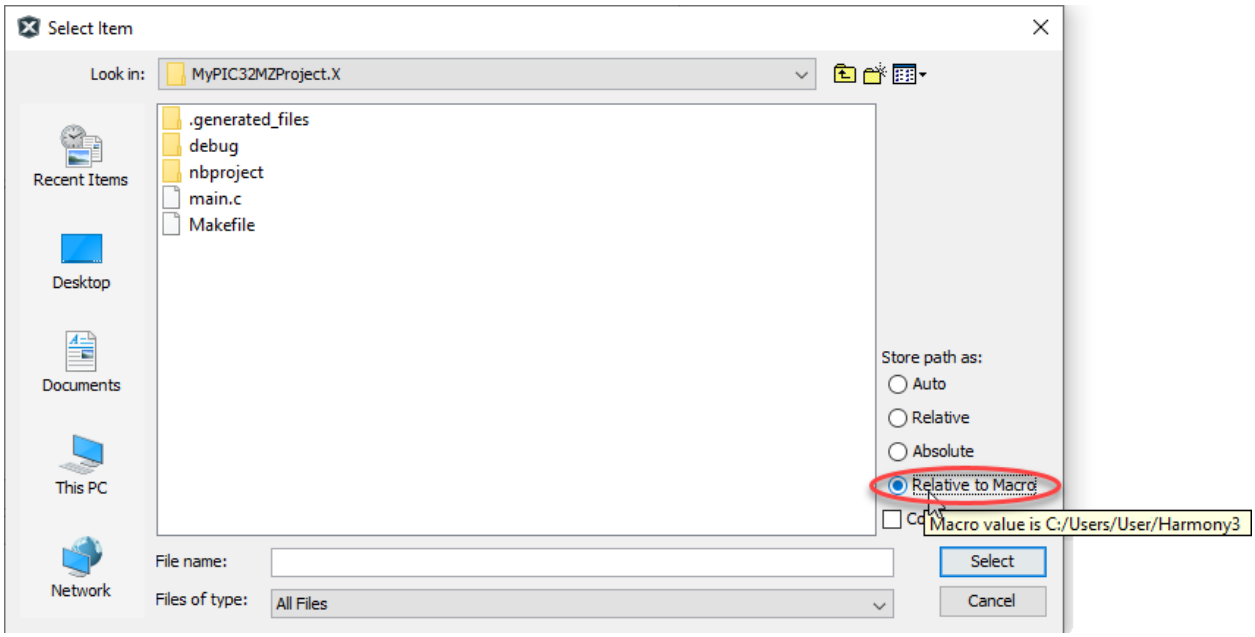
Select a path to MPLAB Harmony source files.

Figure 6-45. Enter Path to be Used for Macro



Go to the Projects window, right click on the “Source Files” folder and select one of the options for “Add Existing Items.” In the dialog is an option to choose files from the Macro. Hover over “Relative to Macro” to see macro value.

Figure 6-46. Choose to Select Files from Macro



Encoding

Change the project encoding. This selection will specify what character encoding is used to read and save the text-based information in the project. The current default is ISO 8859-1.

6.3.4 Use Linked Resources for Source File Folders

A macro can be used to point to a source base (or library version) and changed to move to another source base. This is particularly useful in MPLAB Harmony applications.

Open a project in MPLAB X IDE and select *File>Project Properties*. Click on the “General” category and click “Browse” next to “Macro.” Find and Open the folder containing the source base. Click **OK**. In the project, open file(s) from this source base as “Relative to Macro.”

To change to another source base, browse to the folder that contains the new source base under “Macro” and select it. Any files from the old source base that match the new source base will now refer to the new source base.

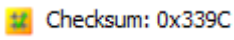
6.4 Work with Projects

The following topics presents ways to add to the MPLAB X IDE experience.

6.4.1 Checksums

A checksum is a hexadecimal number derived from the application code and other settings of an MCU or DSC. Refer to your device programming specification for details on how the checksum is generated.

A checksum is created when a project is built. You can view the generated checksum in the Dashboard window: 8.5. View the Dashboard Display



Checksums are used to detect errors between builds. For example, consider a project built on one computer with one checksum. If that project is copied to another computer and built there, an identical checksum means the project was successfully copied while a different checksum means the copying failed.

Care should be taken when developing project code if the checksum is used for error detection. Do not use the compiler macros `__TIME__` or `__DATE__`. Also, do no use the macros `__FILE__` or `assert()` if file paths will change.

For some device, the checksum may be used as the user ID: 5.10.3. Insert unprotected checksum in user ID memory.

6.4.2 Speed Up MPLAB X IDE

If MPLAB X IDE is operating too slowly, consider the suggestions in the follow sections.

Increase the Computer Heap

You can modify the amount of memory allocated to MPLAB X IDE in the file `mplab_ide.conf`. We recommend you back up this file before you start editing it. If you change the contents of this file, the changes will be operative the next time you run MPLAB X IDE.

- ***Windows OS*** - `C:\Program Files\Microchip\MPLABX\vx.xx\mplab_platform\etc`
- ***Linux OS*** - `/opt/microchip/mplabx/vx.xx/mplab_platform/etc`
- ***macOS*** - `/Applications/microchip/mplabx/vx.xx/Contents/Resources/mplab_platform/etc`

where `vx.xx` is the MPLAB X IDE version.

The following line contains the current default values, formatted for ease of reading:

```
default_options="-J-Dmymicrochip.connect=false
                 -J-Dstatistics.connect=false
                 -J-Dprism.allowhidpi=false
                 --branding mplab
                 -J-Dcrownking.stream.verbosity=very-quiet
                 -J-Xss2m -J-Xms256m -J-Xmx4026m
                 -J-Dapple.awt.graphics.UseQuartz=true
                 -J-XX:+UseConcMarkSweepGC
                 -J-XX:+CMSClassUnloadingEnabled
                 -J-Dapt.limit.expanded.params=1000
                 -J-Dplugin.manager.check.interval=EVERY_STARTUP
                 -J-Dsun.java2d.noddraw=true
```

```
                -J-Dorg.netbeans.modules.extbrowser.UseDesktopBrowse=true
                -J-Dnetbeans.importclass=org.netbeans.upgrade.AutoUpgrade"
```

The bolded areas are:

-Xms256m	Tells the JVM to start with at least 256 MB for the heap.
-Xmx4026m	Tells the JVM to allocate as much as 4026 MB for the heap, but no more.

In general, the most important area is `-Xmx4026m`; this limits the maximum amount of heap that MPLAB X IDE will use. It might seem that having a large amount of heap could be helpful. However, memory used for MPLAB X IDE means less memory for other applications and system functions.

You can monitor how much memory the IDE is using by enabling the Memory monitor. Either right click on an empty space in the toolbar area and select memory, or select *View>Toolbars>Memory*.



The upper number will not go above 4026 MB unless you change the value in `mplab_ide.conf`. It is recommended that you change the value by setting in 128 MB increments. If you have a lot of memory, you can increase it by more than 128 MB, but make sure you leave enough memory for the rest of the system.

Change Debug Window Usage

If using debug tools slows down MPLAB X IDE, try the following:

- Only display windows you need to view. The MPLAB X IDE debugger will examine each open window when a debug tool is used.
- Reduce updates in the Stack window during debug stepping. In the *Tools>Options* (*MPLAB X IDE>Preferences* in macOS) window, **Embedded** button, **Generic Settings** tab, check “Disable auto refresh for call stack view during debug sessions” and “On mouse over structure and array expressions, evaluate integral members only.”

6.4.3 Speed Up Build Times

Depending on the configuration of your computer, you may be able to use parallel make (see 13.20.2. Project Options Tab) to speed up your project build times. Not all language tools support parallel make.

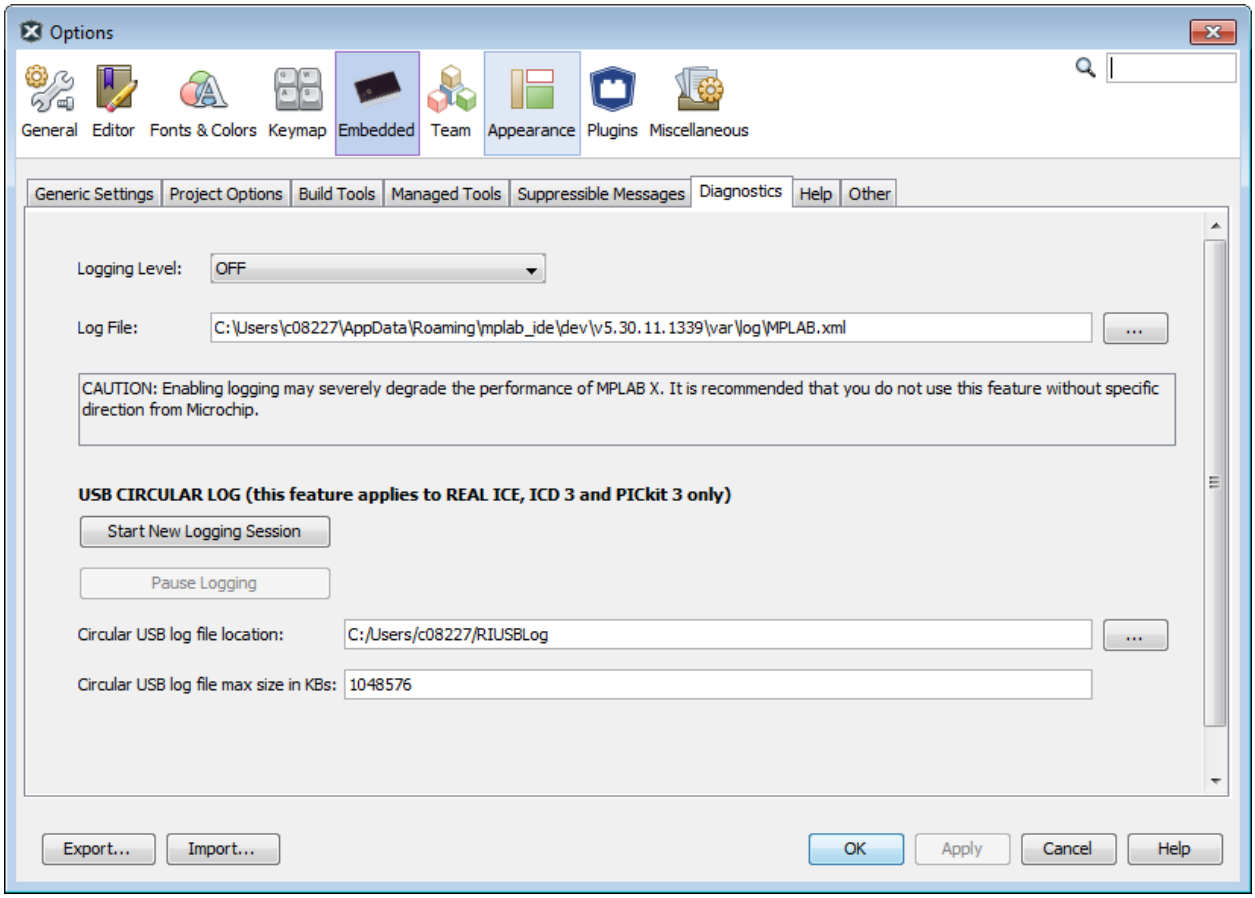
Another option is to consider your operating system (OS). Some OSs have faster file accessing. MPLAB X IDE supports Windows OS, Linux OS and macOS. Research which one might be right for you.

6.4.4 Log Data

Log files are useful for capturing your program execution and debugging problems.

When you have an MPLAB X IDE error or issue and need to contact technical support, you may be asked to capture data in two log files: MPLAB X IDE log file and the NetBeans platform log file. Other supporting information will be requested as well.

Figure 6-47. Data Logging



6.4.4.1 MPLAB® X IDE Log File

The MPLAB X IDE log file generates data based on the Java Logger class.

To set up a log file:

1. Select *Tools>Options* (*MPLAB X IDE>Preferences* for macOS), **Embedded** button, **Diagnostics** tab.
2. Select a logging level from the drop-down box.
   Note: The higher the logging level, the more data is collected, but the slower your application will run.
3. Select a location (path) for the log file.

To log data:

1. Set up the log file using logging level “Finest.”
2. Take note of the log file name and location.
3. Repeat the steps to cause the error or issue.
4. Find the log file(s) and send to technical support along with other requested information.

6.4.4.2 NetBeans Platform Log File

The NetBeans log file generates information on the executing NetBeans Platform.

To log data:

1. Open the log file the Output window by selecting *View>IDE log*.
2. Right click in the window and select “Clear.”
3. Repeat the steps to cause the error or issue.
4. Right click in the window and select “Save As” to save the text to a file.
5. Send the file to technical support with other requested information.

© 2022 Microchip Technology Inc. and its subsidiaries

6.4.4.3 Additional Supporting Information

Although logs are useful, other information will be required and requested by technical support to help them resolve your issue. This information includes:

- The steps performed that correspond to the log file.
- Details about the project tools, such as which debug and language tools were used and which versions of these tools were selected.
- Other MPLAB X IDE project data as necessary.

6.4.4.4 Logging Considerations

When logging:

- Keep the log files as short and focused as possible. That is, instead of running code and logging for a long time till an error occurs, attempt to capture a more focused run just before the error.
- For hardware tools, a circular log is useful in determining USB communication issues.

6.4.5 Package an MPLAB X IDE Project

Right click on your project and select “Package” to package your project into a zip file (`.zip`) file. While MPLAB X IDE has the ability to zip files up, it cannot unzip them. So a separate program will be required to unzip the project.

To package the project, this feature examines the project file to determine the location of the project files to include. Only files contained in the project folder and those using relative, not absolute, paths will be included.

Items included in the package are the source files, `makefile` and `nbproject` directory.

Header files are not included unless they have been added to the project. Therefore, the unzipped project may not build if any user-generated header files are required.

6.4.6 Configurations

The term “configuration” is used in many different contexts. The following table presents the terms, and definitions of them.

Table 6-6. Configurations Context

Term	Definition
Configuration Bits Configuration Fuses Configuration Words Configuration Bytes Configuration Registers	All of these terms refer to the physical section of device memory used to configure the device. See your device data sheet for available configuration settings.
Configuration Bits Window	The MPLAB X IDE window which displays the configuration settings available for the project device. For more on this window, see 5.20. Set Configuration Values in the Configuration Bits Window.
Project Configuration	Build settings for the project. When a project is created, the configuration is “[default].” This name may be changed and multiple configurations may be created for the project. For more information, see 6.2.11. Work with Multiple Configurations. This data is stored in the file `configurations.xml`.
Project Configuration Type	Settings for a specific type of project. This applies only to Standalone projects configured as “application” and Library projects configured as “library.” For details, see 5.10.1. Change Project Configuration Type.
User Configuration Data	MPLAB X IDE configuration settings created by the user. See 6.5.6. Viewing User Configuration Data.

6.5 Project Reference

MPLAB X IDE has several windows, usually in the File Pane, for working with project files and folders, namely the Projects, Files, Favorites and Classes windows. Other project file and folder related topics are provided as references.

6.5.1 Projects Window View

The Projects window is the main entry point to your projects. It shows a logical view of important project contents. For more on this window, see 13.19. Projects Window.

You will, at very least, need to add source files to your project. MPLAB X IDE will find many default files for you (header files, linker scripts). You may add library and precompiled object files, as well as edited header and linker script files. Files that will not be included in the build may be placed under “Important Files.”

Figure 6-48. Projects Window – Simple C Code Project

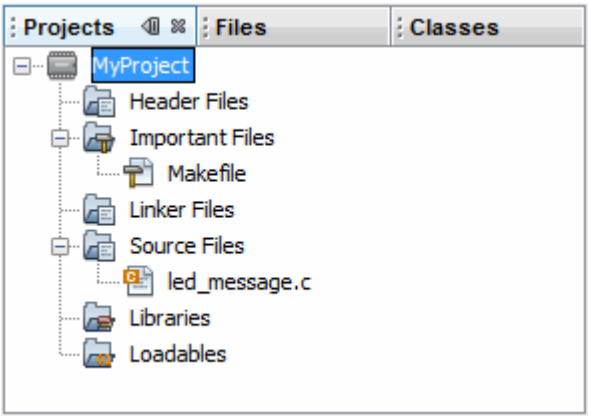


Table 6-7. Projects Window Definitions

Virtual Folder	Files Contained
Header Files	Header files (`.h` or `.inc`)
Important Files	Important files, such as makefiles . Other documents can be placed here, such as data sheet PDFs.
Linker Files	Linker files (`.ld`, `.gld` or `.lkr`)
Source Files	Source files (`.c`, `.cpp` or `.asm`)
Libraries	Library files (`.a` or `.lib`)
Loadables	Precompiled object files (`.o`)

See also the NetBeans help topic. Select *Help>Help Contents* and in the Search tab enter “Projects Window C or C++.” Find the first occurrence of “Projects Window” in the list.

6.5.2 Files Window View

The Files window allows you to view all of your project contents in a directory-based view, even files and folders not shown in the Projects window.

© 2022 Microchip Technology Inc. and its subsidiaries

Figure 6-49. Files Window – Simple C Code Project

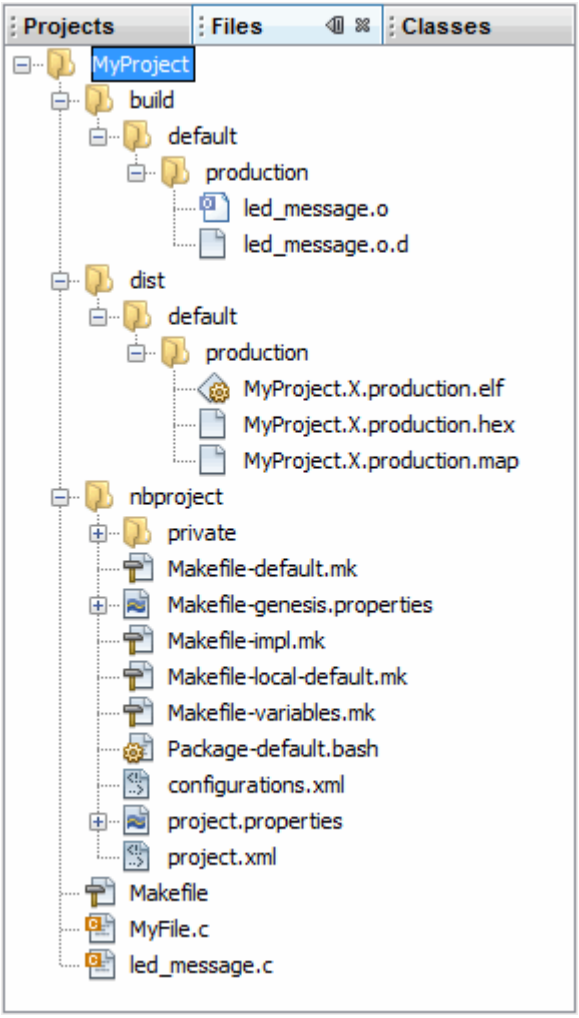


Table 6-8. Files Window Definitions

Folder	Description
MyProject	The project folder, which contains the Makefile file and C code or assembly application files. Makefile is the main makefile for the project. This file is generated at project creation time and it is not touched after that (it will not be regenerated). You can make changes to this file if you are familiar with GNU Make. But MPLAB X IDE provides ways to add a pre-step and post-step (Project Properties) which can be used instead of modifying the Makefile itself.
build(1)	The intermediate files folder. Files are contained in subfolders depending on project configuration, usage and location. The build files are: • Run files (`.o`) • Dependency files (`.o.d`) • HI-TECH® intermediate files (`.p1`)

.....continued

Folder	Description
dist(1)	The output files folder. Files are contained in subfolders depending on project configuration, usage and location. The distribution files are: • Executable files (`.hex`) • ELF or COF object files (`.elf` or `.cof`) • library files (`.a` or `.lib`)
nbproject	The makefile and metadata folder. Contains these files: • The private folder contains the user and computer specific settings of the project • Project makefile • Configuration-specific makefiles • `project.xml` IDE-generated metadata file • `configurations.xml` metadata file
default, MyConfig(2)	The project configurations folders. If no configurations are created by the developer, all code is in default.
production, debug(2)	The production and debug versions folders.
_ext(2)	The external project files folder. If a file is referenced outside the project folder, it is listed here.

1. You do not need to check these folders into source control. A build will create them. See also 8.9. Control Source Code using a Revision Control System.
2. These items are not shown in the figure above.

See also the NetBeans help topic. Select *Help>Help Contents* and in the Search tab enter “Files Window C and C++”. Find the first occurrence of “Files Window” in the list.

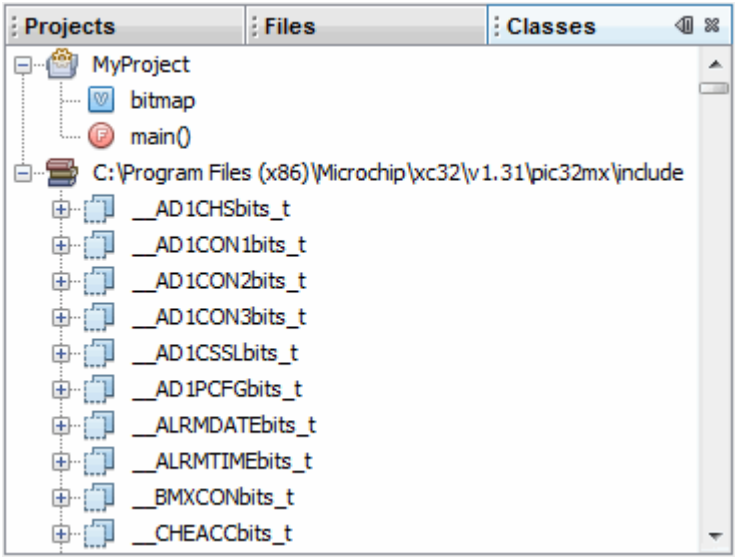
6.5.3 Classes Window View

For C or C++ code, you can see all of the classes (functions), and the members and fields for each class, in the Classes window (*Window>Classes*).

See also the NetBeans help topic. Select *Help>Help Contents* and in the Search tab enter “Using the Classes Window.”

© 2022 Microchip Technology Inc. and its subsidiaries

Figure 6-50. Classes Window – Simple C Code Project



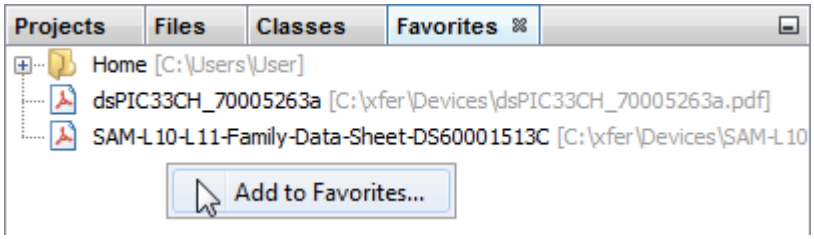
6.5.4 Favorites Window View

The Favorites window (*Window>Favorites*) enables you to access any file or folder on your computer or network, whether or not it is in a project.

When you first open the Favorites window, it only contains your computer's home directory.

- To add a file or folder, you can right click in the Favorites window and choose “Add to Favorites.”
- To add a file, you can right click on the file name in the Projects window and select *Tools>Add to Favorites*.

Figure 6-51. Favorites Window with Examples



See also the NetBeans help topic. Select *Help>Help Contents* and in the Search tab enter “Favorites Window.”

6.5.5 Path, File and Folder Name Restrictions

You should only use the following ASCII characters for path, file and folder names:

- A through Z
- a through z
- 0 through 9
- underscore: _
- dash: -
- period: .

Makefile

If you have files or directories within the project directory using either the `()` or `[]` characters (e.g., `t (1).c`), or if you use absolute paths to directories with these characters, the characters will cause the "no rule to make target" problem.

The workaround is to avoid doing it at this time. System libraries can be added through the project properties and therefore, do not need to be added to the project.

© 2022 Microchip Technology Inc. and its subsidiaries

Cross-Platform Operating Systems

If you plan on using MPLAB X IDE on different platforms (Windows OS, Linux OS or macOS), be aware of these issues:

- Use the forward slash "/" in relative paths. The backslash "\" works only on Windows OS platforms. Example: `#include headers/myheader.h`.
- Linux OS is case-sensitive, so `generictypedefs.h` is not the same as `GenericTypeDefs.h`.

Windows Operation System

Windows operating systems have a maximum path length of 260 characters. An explanation from Microsoft is found here:

msdn.microsoft.com/en-us/library/windows/desktop/aa365247(v=vs.85).aspx

Although Windows will attempt to stop you from creating path lengths that are too long, it is possible to do this (from certain applications, cutting-and-pasting, etc.) If a project source file uses such a path, the project will not build correctly. Additionally, there is a command-line limitation for Windows OS of 8191 characters.

For workarounds, see 11.5. Errors.

6.5.6 Viewing User Configuration Data

The MPLAB X IDE user directory (userdir) stores user configuration data such as window layouts, editor settings, menu and toolbar customizations, and various module settings such as the list of known compilers.

When you install plugins (*Tools>Plugins*), the plugin information (code) is stored in the userdir, not in the MPLAB X installation directory. However, you may change this location under *Tools>Plugins>Settings>Advanced* by selecting "Force install into shared directories" from the "Plugin Install Location" drop down menu.

To find out where your user directory is located, select *Help>About* and find the path next to **Userdir**. This information is created and managed by MPLAB X IDE.

6.5.7 Moving, Copying or Renaming a Project

After you have created a project, you may realize you need to make changes. Using the commands on the context (right click) project menu, you can move, copy or rename your project from within MPLAB X IDE. There is also an option to delete the project. For details, see "Project Menu" under 13.19. Projects Window.

You can also use an external tool; the project file structure does not require you to use MPLAB X IDE.

6.5.8 Deleting a Project

To delete a project in MPLAB X IDE:

1. In the Projects window, right click on the project name and select "Delete."
2. In the "Delete Project" dialog, you can delete:
   a. The project file from the computer - some source files will remain.
   b. The project file with all of the source files (via check box).

The project is deleted so that MPLAB X IDE will no longer see it.

Note: MPLAB X IDE does not release all resources assigned to a project when it closes. MPLAB X IDE runs on Java so releasing resources does not occur immediately. Under Java it is best to let the JRE decide when to call the garbage collection (GC) within the software code. However, you can force GC by enabling the memory toolbar and clicking on it.

7. Editor

MPLAB X IDE is built upon the Netbeans platform, which provides a built-in editor to create new code or change existing code.

For information on the NetBeans Source Editor, see:

docs.oracle.com/cd/E50453_01/doc.80/e50452/working_nbeans.htm#A1151635

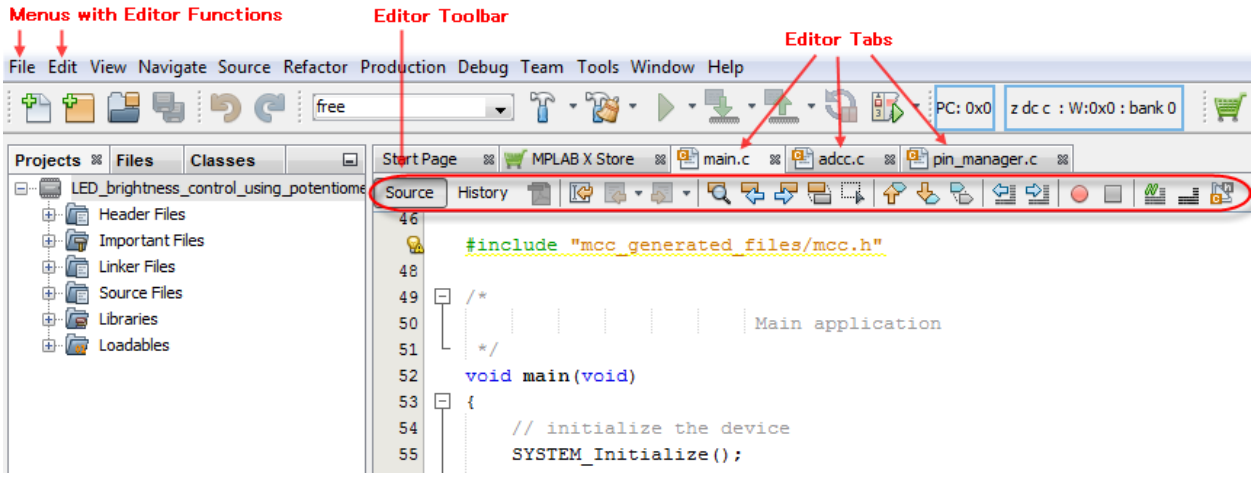
C compiler information regarding the editor is available under the NetBeans help topic:

netbeans.org/kb/docs/cnd/navigating-editing.html

7.1 Editor Usage

To begin using the Editor, create or open an existing file using *File>New File* or *File>Open File*, respectively. Additional controls and features are described in the following sections.

Figure 7-1. Editor GUI



Desktop Controls

The following desktop items are associated with the Editor:

- File menu – opens a file in an Editor window (see 12.1.1. File Menu).
- Edit menu – accesses edit commands (see 12.1.2. Edit Menu).
- Editor toolbar – also accesses edit commands – located at the top of each file's Editor window (see 12.2.10. Editor Toolbar).
- Window right-click (context) menu – for additional commands.

Display Controls

Basic display controls are shown in the table below. Additional formatting control may be found under 7.2. Editor Options.

Action	Description
Maximize the Source Editor.	Do one of the following: • Double click a file's tab in the Source Editor. • Make sure that the Source Editor window has focus and then press Shift-Escape. • Choose *Window>Configure Window>Maximize*.

.....continued

Action	Description
Revert a maximized Source Editor to its previous size.	Do one of the following: • Double click a file's tab in the Source Editor. • Press <Shift> + <Escape>. • Choose *Window>Configure Window>Restore*.
Display line numbers.	Choose *View>Show Line Numbers*.
View two files simultaneously.	1. Open two or more files. 2. Click the tab of one of the files and drag it to the side of the window where you want the file to be placed. When a red preview box appears to show you where the window will be placed, release the mouse button to drop the window. The window can be split horizontally or vertically, depending on where you drag the tab.
Split the view of a single file.	1. Right click the document's tab in the Source Editor and choose “Clone Document.” 2. Click the tab of the cloned document and drag it to the part of the window where you want the copy to be placed.
Move between open files	1. Press and hold <Ctrl>. 2. Press <Tab>. A selection box will appear. 3. Select the file you want to move to.
Format code automatically.	Right click in the Source Editor and choose “Format.” If any text is selected, only that text will be formatted. If no text is selected, then the whole file is formatted.

Basic Editor Features

The following table summarizes some of the more frequently-used features of the editor.

Editor Feature	Reference
Unicode is supported.	By default, newly-created projects in the IDE use ISO-8859-1 character encoding. To change this: Right click the project name in the Projects window and choose Properties. In the left column under “Categories,” select “General.” On the bottom of the page, find “Encoding” and change.
Code is colored based on syntax.	*Tools>Options* (*MPLAB X IDE>Preferences* for macOS), **Fonts and Colors** button, **Syntax** tab. Based on Encoding set during Project creation.
Errors are flagged as code is typed.	See the Netbeans Help topic Using Hints.
Colored markers provide quick access to multiple symbols, errors, etc.	*Tools>Options* (*MPLAB X IDE>Preferences* for macOS), **Fonts and Colors** button, **Annotations** tab
Smart code completion makes suggestions and provides hints.	*Tools>Options* (*MPLAB X IDE>Preferences* for macOS), **Editor** button, **Code Completion** tab

© 2022 Microchip Technology Inc. and its subsidiaries

.....continued

Editor Feature	Reference
Mouseover symbols in debug mode to see values.	On halt when debugging, mouse over symbols to see values. **Note:** You cannot view the value of macros in this way. Use the Macro Expansion Window by right clicking (or Cmd+Alt+Click for macOS) on the macro in the editor during debug.
Assembly and C code may be collapsed and expanded	See 7.6. Code Folding.
Right clicking on a function (`delay(x)`) finds usages. This can limit find within a function (e.g., a local `i` variable).	If selected, all usages of the method are displayed.
Right clicking on a function (`delay(x)`) shows a call graph . The call graph has buttons on the side to switch order, etc.	See 8.4. View the Call Graph.
Create comments with keywords (example: `// TODO`) in the source code and action items will be scanned and added automatically to the 13.2. Action Items Window.	See the Netbeans Help topic: netbeans.org/features/ide/index.html regarding the Options Window, Team: Action Items.
File history is available to view recent changes and revert, even without a version control system.	8.8. Control Source Code using Local History
Navigation is simplified with items such as “Go to file,” “Go to type,” “Go to symbol,” “Go to header,” and “Go to declaration.”	See “Navigate Menu.”
Refactor options (rename functions and variables, find all functions, etc.) for improving code structure.	See 7.7. C Code Refactoring.

Editor Troubleshooting

For errors in code highlighted in the editor, see your compiler documentation. For information on error highlighting, see the NetBeans help topic:

netbeans.org/kb/docs/java/editor-codereference.html#coloring

Specific errors may be found under 11.5. Errors.

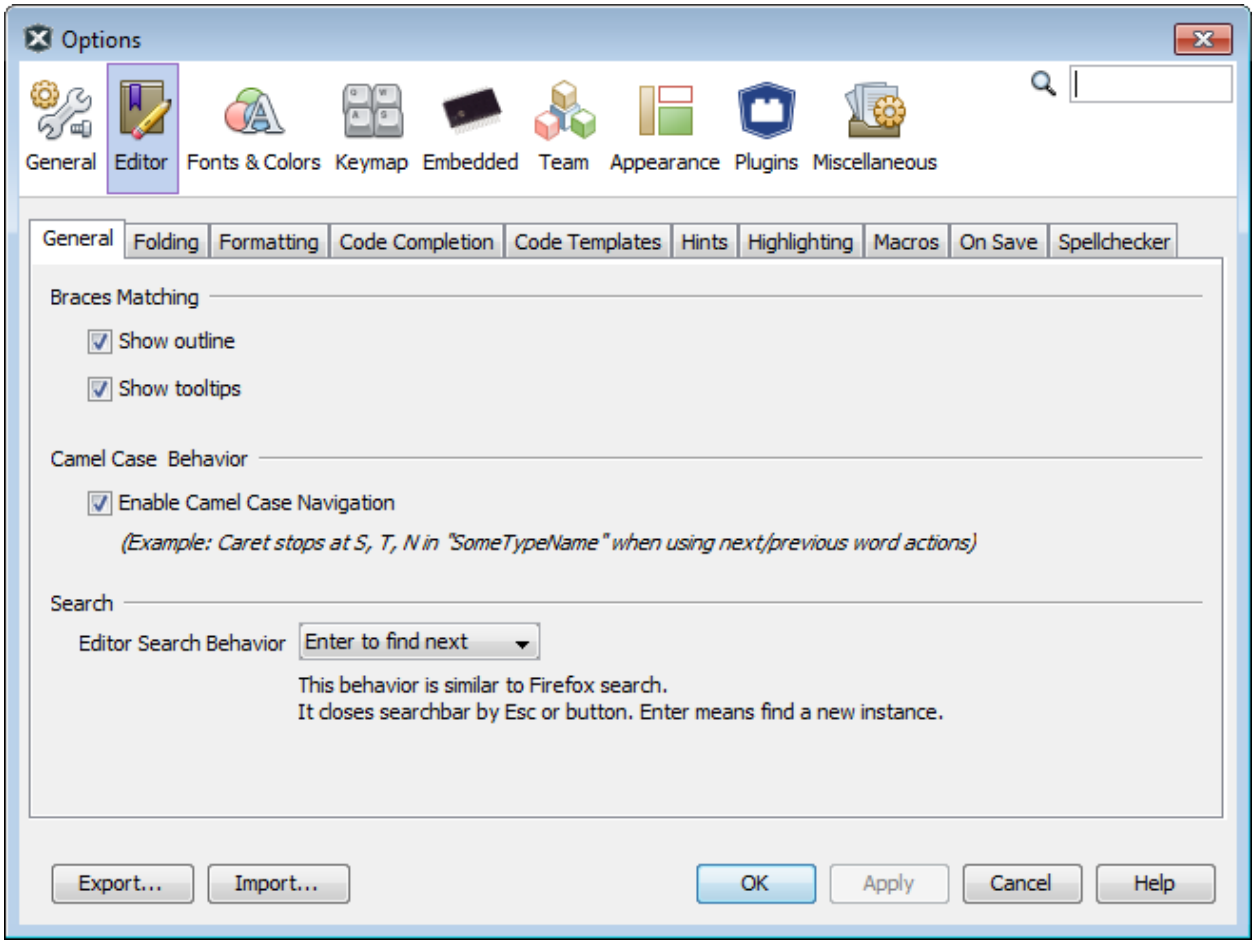
7.2 Editor Options

To set editor options:

1. Select *Tools>Options* (*MPLAB X IDE>Preferences* for macOS) to open the Options dialog.
2. Click on the **Editor** button. Then click on a tab to set up editor features.

© 2022 Microchip Technology Inc. and its subsidiaries

Figure 7-2. Editor Options



Each tab and its options are listed below.

Table 7-1. General Tab

Item	Description
Brace Matching	Check “Show outline” to see a line in the left-most margin connecting the matching braces. Check “Show tooltips” to see a tooltip containing a hidden matching brace with context information.
Camel Case Behavior	Check to enable camel case navigation.
Search	Select Editor search behavoir, either next search or final search.

Table 7-2. Folding Tab

Item	Description
Code Folding	Check to enable Code Folding and select which types of code to fold. For more on Code Folding, see 7.6. Code Folding. See also the NetBeans help topic: netbeans.org/kb/docs/java/editor-codereference.html
Language	Select the programming language to which the options will apply.
Collapse by default	Select which types of code to fold/collapse.

.....continued	
Item	**Description**
Display Options	Contents Preview: Show preview of collapsed contents Show Summary: Show summary of collapsed contents

Table 7-3. Formatting Tab

Item	Description
Language	Select the programming language to which the options will apply.
Category	Select a formatting category to edit.
Style	Select from a current formatting style. Click Manage to add or delete styles.
Override Global Options	Check to override global options with the ones selected in this dialog.

Table 7-4. Code Completion Tab

Item	Description
Language	Select the programming language to which options will apply.
Completion options	Check to enable code completion options. For more on code completion, see the NetBeans help topic: netbeans.org/kb/docs/java/editor-codereference.html

Table 7-5. Code Templates Tab

Item	Description
Language	Select the programming language to which the options will apply.
Templates	Enter template information for the language specified above. ***Abbreviation***: Enter an abbreviation to type into the editor. ***Expanded Text***: After typing the abbreviation and the “Expand template on” character(s), expand the abbreviation to this text in the editor. ***Description***: Add an optional description of the template item. For more on code templates, see the NetBeans help topic: netbeans.org/kb/docs/java/editor-codereference.html
Expand template on	Select character(s) that will be typed into the editor to expand the abbreviated text to the expanded text.
On Template Expansion	Select an action for when the template is expanded.

Table 7-6. Hints Tab

Item	Description
Language	Select the programming language to which the options will apply.
Hint Window	In the Hint window, select an occurrence. For more on using hints, see the NetBeans help topic: netbeans.org/kb/docs/java/editor-codereference.html
Show As	Specify the “hint” you wish to receive in the event of the occurrence: Error, Warning or Warning on current line.

© 2022 Microchip Technology Inc. and its subsidiaries

.....continued	
Item	**Description**
Description	View a description of the hint.

Table 7-7. Highlighting Tab

Item	Description
Language	Select the programming language to which the options will apply.
Hintlighting Checklist	Select the options for language syntax highlighting.

Table 7-8. Macros Tab

Item	Description
Macros	Add, delete and define editor macros. ***To create a new macro***, click the **New** button, enter the name for the new macro, then select the new macro in the list and enter the code, in quotes, in the Macro Code editor. ***To set a keyboard shortcut for a macro***, select a macro from the list, click the **Set Shortcut** button and enter the shortcut in the dialog box. Use this shortcut to run your macro. ***To remove a macro***, select a macro from the list and click the **Remove** button. ***To modify the macro code***, select a macro from the list and edit the code in the Macro Code editor.
Macro Code	Click on the macro name above and then type in your macro code in quotes. For a list of macro keywords, see: wiki.netbeans.org/FaqEditorMacros **Note:** Generally, it is easier to add a new macro by recording it than by adding one manually in the Macro Code editor. Use *Edit>Start/Stop Macro Recording*.

Table 7-9. On Save Tab

Item	Description
Language	Select the programming language to which the settings will apply when a document is saved.
Use all Language Settings	Check to save all language settings. Uncheck to specify Reformat and Trailing Whitespace options.

Table 7-10. Spellchecker Tab

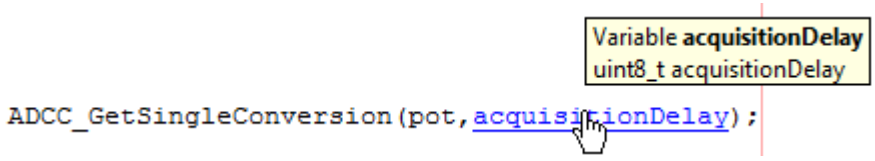
Item	Description
Use in	Specify the programming language(s) to be referenced for spellchecking.
Dictionaries	Specfy the dictionaries to be referenced for spellchecking.
Default Locale	Specify the default locale to be referenced for spellchecking.

7.3 Hyperlinks in Code

Hyperlinks in code help you to navigate to information relevant to the link content.

Hyperlinks in C Code

Hyperlink navigation lets you jump from the invocation of a function, variable, or constant to its declaration.



To use a hyperlink, do one of the following:

- Mouse over a function, variable, or constant while pressing <Ctrl> (Windows and Linux) or <Command> (Mac). A hyperlink appears, along with a tooltip with information about the element. Click the hyperlink and the editor jumps to the declaration. Press <Alt> + <Left Arrow> to jump back to the invocation.
- Mouse over an identifier and press <Ctrl> + <B> or <Command> + <B>. The editor jumps to the declaration. Press <Alt> + <Left Arrow> to jump back to the invocation.

Press <Alt> + <Left Arrow> and <Alt> + <Right Arrow> to move backward and forward through the history of the cursor position.

Hyperlinks in ASM Code

To navigate to header files included in assembly source files, press <Ctrl> (Windows and Linux) or <Command> (Mac) while placing the mouse cursor over the file name referenced by a `#include` statement. Then click the mouse select button to open the include file in its own file tab in the editor.

7.4 Editor Red Bangs

The Source Editor highlights syntax and semantic errors, such as missing semicolons and unresolved identifiers, in your source code. Errors are shown in red underline and a red bang “glyph” is displayed in the left margin of the editor window next to each line that contains an error. An error message is displayed when you mouse over the glyph.

Note: Not all errors in the editor are actual errors, but may be unrecognized symbols. Try building your code to determine if it will build anyway. For more details, see the section “Errors Caused by Unrecognized Symbols.”

Error Conditions and Additional Marking

Errors may occur when you do one of the following:

- Type in new code
- Edit existing code
- Add an existing file to your project
- Move a file to a different location in your project
- Change the properties of the project or one of its source files

If there are any errors in a file, red marks are displayed in the error stripe on the right side of the editor window. The error stripe represents the whole file, not just the lines currently displayed. Double click a mark in the error stripe to jump to the line the mark represents.

Errors Caused by Unrecognized Symbols

A symbol that is unrecognized by a compiler preparser or parser could be marked as an error but may not actually be an error (the code will build). Therefore, if you suspect this type of error, try to build the project anyway.

For examples, see the following issued under 11.5. Errors:

- For 8-bit code, #asm and #endasm cause red bangs in the Editor window.
- For 16-bit code, MPLAB XC16 packed attribute statement causes red bangs in the Editor window.

7.5 Variable Values

Editor mouse-overs may show (Optimized) for local variables. This is not related to compiler settings but rather implies the variable value is not available on the current stack frame at the scope where it is evaluated. It is most commonly applicable to function `param` variables which are not used by a function at the current scope.

7.6 Code Folding

Code folding allows you to hide some sections of code so you can focus on others. Sections of C or assembly code may be collapsed (hidden) or expanded (displayed), depending on your selected options.

Code folding is enabled by default. For most C or assembly code, sections of code that may be folded are signified by "-" and "+" icons to the left of the code.

Enable and Set Up Code Folding

Use the following steps to enable and set up code folding:

1. Select *Tools>Options* (*MPLAB X IDE>Preferences* for macOS) to open the Options dialog.
2. Click on the **Editor** button.
3. Click on the **General** tab and check "Use code folding."
4. Check other options to specify sections to fold. If you do not see the type of code you want to fold listed there, you can do a custom fold.

Expand/Collapse Code Folding

Within the editor, click the "-" icon next to a method and it folds. Click the "+" icon next to a folded method and it expands. Folded code is denoted by an ellipsis box. Hold the cursor over the ellipsis box and the IDE displays the collapsed method.

Figure 7-3. Expanded Code

```
54   int main(void)
55   {
56       // <editor-fold defaultstate="collapsed" desc="initialization">
57       _display_state = DISP_HELLO; // Start from displaying of PIC24 banners
58       AD1PCFG = 0xffff; // Setup PortA IOs as digital
59       SPIMPolInit(); // Setup SPI to communicate to EEPROM
60       EEPROMInit(); // Setup EEPROM IOs
61       UART2Init(); // Setup the UART
62       TimerInit(); // Setup the timer
63       mLCDInit(); // Setup the LCD
64       BtnInit(); // Setup debounce processing
65       ADCInit(); // Setup the ADC
66       BannerStart(); // Setup the banner processing
67       RTCCInit(); // Setup the RTCC
68       // </editor-fold>
69           while (1) {
70                   LCDProcessEvents();
71                   ADCProcessEvents();
```

Figure 7-4. Collapsed Code

```
54   int main(void)
55   {
56       // <editor-fold defaultstate="collapsed" desc="initialization">
57       _display_state = DISP_HELLO; // Start from displaying of PIC24 banners
58       AD1PCFG = 0xffff; // Setup PortA IOs as digital
59       SPIMPolInit(); // Setup SPI to communicate to EEPROM
60       EEPROMInit(); // Setup EEPROM IOs
61       UART2Init(); // Setup the UART
62       TimerInit(); // Setup the timer
63       mLCDInit(); // Setup the LCD
64       BtnInit(); // Setup debounce processing
65       ADCInit(); // Setup the ADC
66       BannerStart(); // Setup the banner processing
67       RTCCInit(); // Setup the RTCC
68       // </editor-fold>
69           while (1) {
70                   LCDProcessEvents();
71                   ADCProcessEvents();
```

Custom Code Folding – C

To custom fold C code, do one of the following:

- Type the following comments around your code:

```
// <editor-fold defaultstate="collapsed" desc="user-description">
C code block to fold
// </editor-fold>
```

OR

- Type `fcom` and press the <Tab> key to automatically enter the comment text above.

After the comment has been entered, customize it for your code:

`defaultstate`	Enter either `collapsed` or `expanded`,
`desc`	Enter a description of the code. Once collapsed, this description can still be seen for reference.

An example of custom code folding is shown in “Expand/Collapse Code Folding” above.

For additional information see the NetBeans help topic:

ui.netbeans.org/docs/ui/code_folding/cf_uispec.html

Custom Code Folding – Assembly

To custom fold assembly code, do the following:

- For MPLAB XC16 assembler code, type the following comments around your code:

```
; <editor-fold defaultstate="collapsed" desc="user-description">
ASM code block to fold
; </editor-fold>
```

- For MPLAB XC32 assembler code, type the following comments around your code:

```
// <editor-fold defaultstate="collapsed" desc="user-description">
ASM code block to fold
// </editor-fold>
```

OR

- For MPLAB XC16 assembler code, type `fcom`; and press the <Tab> key to automatically enter the relevant comment text above.
- For MPLAB XC32 assembler code, type `fcom//` and press the <Tab> key to automatically enter the relevant comment text above.

Assembly code will fold in the same way that C code folds. For an example of code folding, see the C code example above.

Expand/Collapse Custom Code Folding

To expand/collapse code blocks, go to the View menu, or right-click in the Editor window, and select one of the following:

1. *Code Folds>Collapse Fold* to hide the block of code.
2. *Code Folds>Expand Fold* to display the block of code.
3. *Code Folds>Collapse All* to hide all folding blocks of code.
4. *Code Folds>Expand All* to display all folding blocks of code.

An example of custom code folding is shown in “Expand/Collapse Code Folding” above.

7.7 C Code Refactoring

Refactoring is the use of small transformations to restructure code without changing any program behavior. Just as you factor an expression to make it easier to understand or modify, you refactor code to make it easier to read, simpler to understand, and faster to update. And just as a refactored expression must produce the same result, the refactored program must be functionally equivalent with the original source.

Some common motivations for refactoring code include:

- Making the code easier to change or easier to add a new feature
- Reducing complexity for better understanding
- Removing unnecessary repetition
- Enabling use of the code for other needs or more general needs
- Improving the performance of your code

The IDE’s refactoring features simplify code restructuring by evaluating the changes that you want to make, showing you the parts of your application that are affected, and making all necessary changes to your code. For example, if you use the Rename operation to change a class name, the IDE finds every usage of that name in your code and offers to change each occurrence of that name for you.

Refactor Menu

When you use the IDE’s refactoring operations, you can change the structure of your code and have the rest of your code updated to reflect the changes you have made.

Refactoring operations are available on the Refactor menu (see 12.1.6. Refactor Menu).

Undoing/Redoing Refactoring Changes

You can undo (or redo) any changes that you made using the commands in the Refactor menu by using the buttons on the Undo/Redo toolbar menu.

	**Undo** - The IDE rolls back all the changes in all the files that were affected by the refactoring.
	**Redo** - The IDE repeats the refactoring changes in all the files that were affected.

If any of the affected files have been modified since the refactoring took place, the refactoring Undo/Redo is not available.

Finding Function Usages

You can use the Find Usages command to determine each place a function is used in your project’s source code.

To find where a function is used in your project:

1. In the Navigator window or the Source Editor window, right click the function name and choose Find Usages - <Alt>+<F7> (or <Ctrl>+<F7> for macOS).
2. The Find Usages command displays the code lines that call the function. In the Find Usages dialog box, click Find.
   The Usages window displays the file name and the line of code for each usage found in that file.

To jump to a specific occurrence of the function, do one of the following actions:

- In the Usages window, use the up and down arrow buttons in the left pane to navigate from one occurrence of the function to the next.
- Double click a line of code to open the file and position the cursor on that particular line of code.

Additional IDE Find Mechanisms

The other IDE tools that enable you to search for all the places where specific text is used in a project include:

- ***Finding and Replacing Text***. Searches for all the places where specific text is used in a source file that is open in the Editor. Choose *Edit>Find* to open the Find dialog box, or choose *Edit>Replace* to open the Replace dialog box. These commands finds all matching strings, regardless of whether the string is a C code element.
- ***Find in Projects***. As with the Find command, the Find in Projects command searches for matching strings, regardless of whether the string is a function name. Choose *Edit>Find in Projects* to open the "Find in Projects" dialog box and then type the string of text that you are looking for.

To find where a function is declared in a source file, you can double click the function in the Navigator window. If the function is declared in a different source file, right click the function and choose *Navigate>Go To Declaration* from the contextual menu.

Renaming a Function or Parameter

To rename a function or parameter and update references to it throughout your project:

1. In the Source Editor, right click the function or parameter and choose *Refactor>Rename* from the context menu.
2. In the Rename dialog box, type the New Name.
3. Optionally, click **Preview**. In the Refactoring window, at the bottom of the Source Editor, review the lines of code that will be changed and clear the check box of any code that you do not want changed.
4. Click **Do Refactoring** to apply the selected changes.

For quick in-place renaming, place the cursor in the item that you want to rename and press Ctrl-R. Then type the new name. To finish renaming, press **Escape**.

Moving, Copying and Safely Deleting C Code

These functions are specific to C++ code. See 12.1.6. Refactor Menu.

7.8 C/C++ Code Error Directive

When using a `#error` directive in code, it is best to enclose the statement in parentheses, as in:

```
(#error This line shown only when there is an error.)
```

Otherwise, the syntax parser honors the `#error` directive in the same manner as a C compiler and stops in its tracks. Therefore the text immediately following the directive will be grayed in the Editor to signify this.

© 2022 Microchip Technology Inc. and its subsidiaries

8. Development and Debug Features

MPLAB X IDE provides many features useful for code development and debug. Additionally there are tools to help manage code, such as source code control features, and add functionality, such as with plugins.

To see some features, refer to the **Start Page**, **My MPLAB X IDE** tab, “Extend MPLAB” section, “Selecting Simple or Full-Featured Menus” topic.

8.1 Use the Stopwatch

Use the stopwatch to determine the timing between two breakpoints.

To use the Stopwatch:

1. Add a breakpoint where you want to start the stopwatch.
2. Add another breakpoint where you want to stop the stopwatch.
3. Select *Window>Debugging>Stopwatch*. Click on the **Properties** icon on the left of the window and select the start and stop breakpoints.
4. Debug the program again to get the stopwatch timing result.

Figure 8-1. Stopwatch Setup

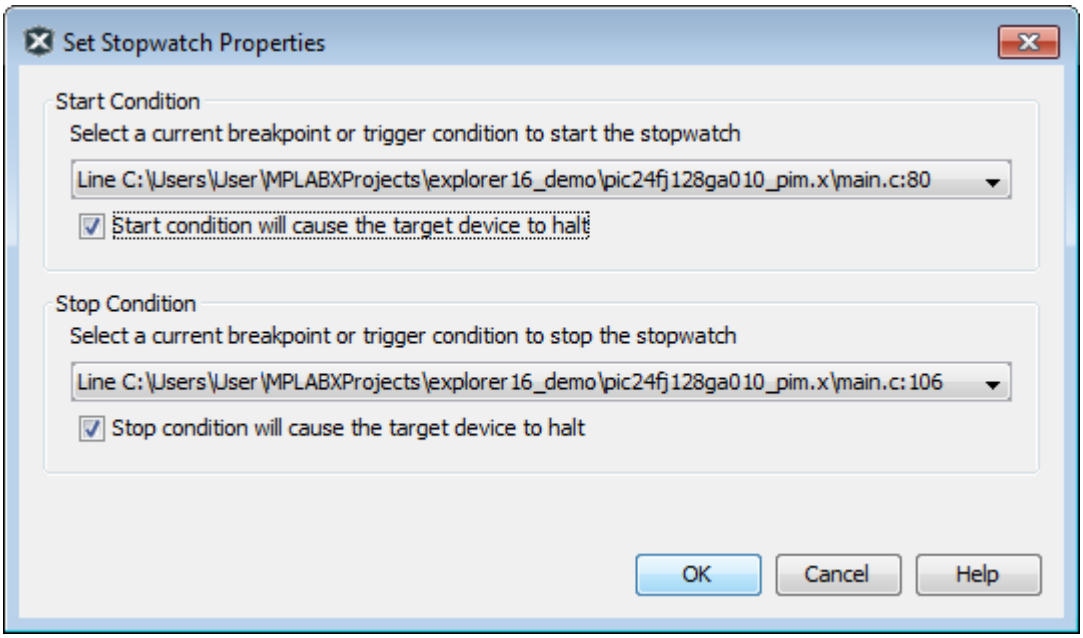
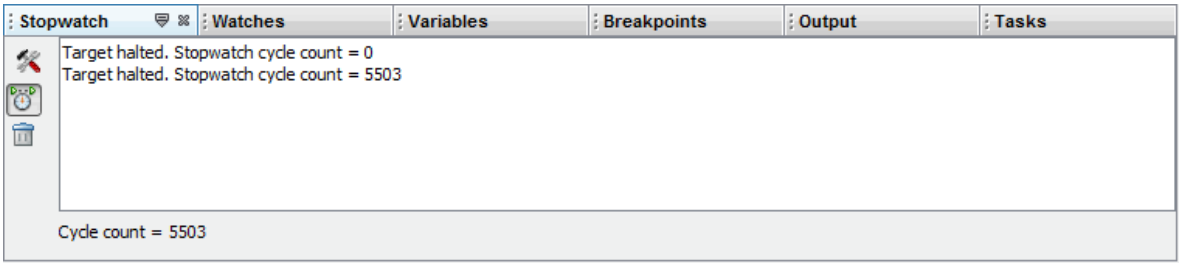


Figure 8-2. Stopwatch Window with Content



The stopwatch has the following icons on the left side of the window:

Table 8-1. Stopwatch Icons

Icon	Icon Text	Description
	Properties	Set stopwatch properties. Select one current breakpoint or trigger to start the stopwatch and one to stop the stopwatch.
	Reset Stopwatch on Run	Reset the stopwatch time to zero at the start of a run.
	Clear History	Clear the stopwatch window.
	Clear Stopwatch	(Simulator Only) Reset the stopwatch after you reset the device.

8.2 View the Disassembly Window

View disassembled code in this window. Select *Window>Debugging>Output>Disassembly* to open the window. You must Pause (halt) in a debug session (*Debug>Debug Project*) to see the window contents.

To view the linker-generated disassembly listing file, right click in the Disassembly window and select “Disassembly Listing File.”

Note: The Disassembly window will disassemble each instruction, but has no history of banking associated with the instruction. Therefore, SFR names displayed in the window will be for Bank 0.

Figure 8-3. Disassembly Windows with Variable Mouseover

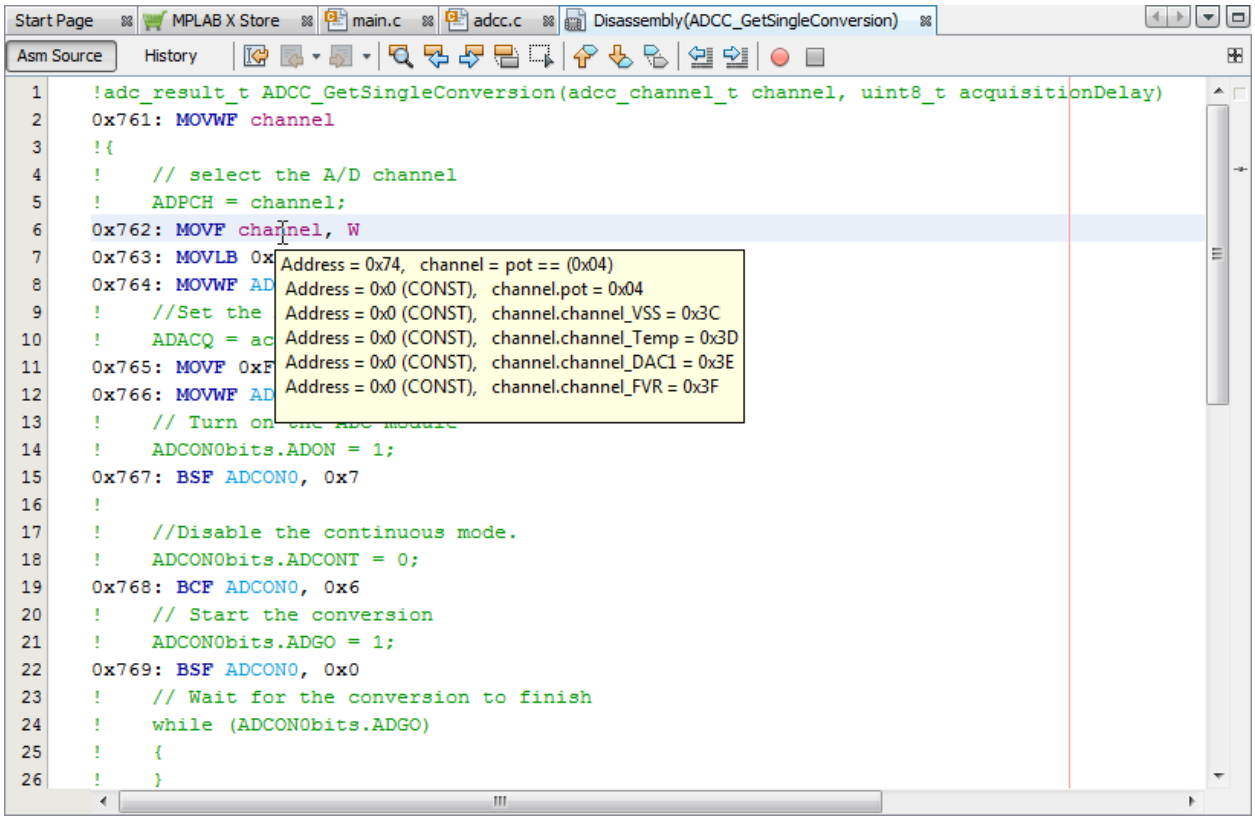


Table 8-2. Disassembly Window Context Menu

Menu Item(s)	Description
Step Instruction, Step Over, Run to Cursor	See 5.15. Step Through Code.

.....continued	
Menu Item(s)	**Description**
Cut, Copy, Paste	Cut, copy, or paste selected text in the window.
Select in Projects	Opens Projects window and selects the document containing selection.
Disassembly Listing File	See the contents of the linker-generated disassembly listing file. See also Disassembly Window Types.

8.3 View the Call Stack

For 16- and 32-bit devices, a software Call Stack window is available to view CALLs and GOTOs in executing C code. This window is not applicable for assembly code (it is recommended that code optimization be turned off when using the call stack).

The Call Stack window displays functions and their arguments listed in the order in which they were called in the executing program.

To view the call stack:

1. Debug and then Pause your program.
2. Select *Window>Debugging>Call Stack*. A Call Stack window will open. See also 13.5. Call Stack Window.

Call Stack Limitations

The MPLAB X IDE call stack function is capable of unwinding calls to functions on the stack for bare metal C programs. Once the execution context switches to control of a RTOS, other tools are required to view the state of all tasks and control objects. MPLAB X IDE does not have RTOS specific integrated support for presenting call-stack state of tasks controlled by the RTOS.

Call Stack Video

For more on the call stack, see the NetBeans video:

www.youtube.com/watch?v=iS8OOUPmTUc

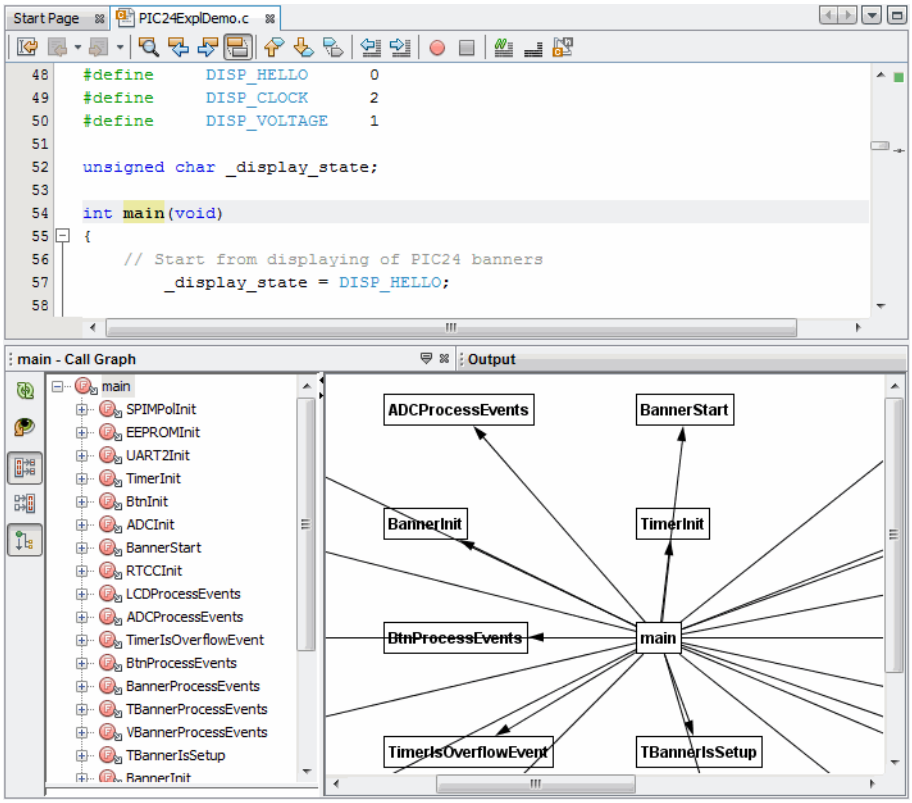
8.4 View the Call Graph

The Call Graph window displays a tree view of either the functions called from a selected function, or the functions that call that function.

To view the call graph:

- Right click on a function and select “Show Call Graph” from the drop-down menu.

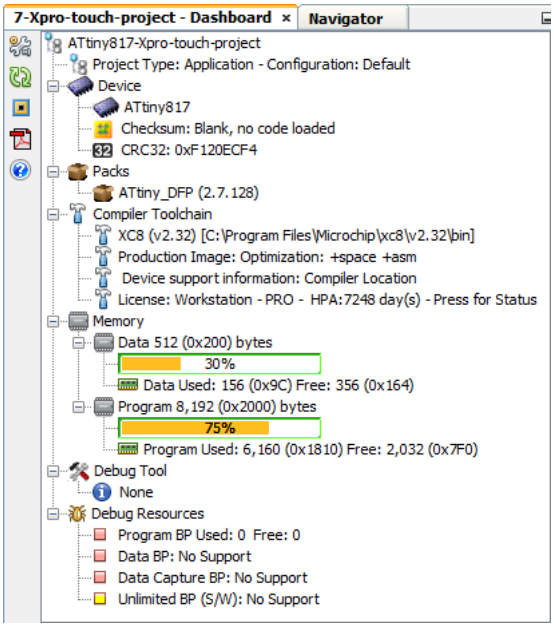
Figure 8-4. Call Graph of Main Function



8.5 View the Dashboard Display

Select *Window>Dashboard* to display project information.

Figure 8-5. Dashboard Display



The project displayed will either be:

- The active project in the Projects window, if no main project is selected. Click on a project in the Projects window to make it active.
- The main project. Right click a project in the Projects window and select “Set as Main Project.”
- No other project, active or inactive, will be displayed.

8.5.1 Dashboard Groups

Dashboard window project information is grouped into specific sections.

Table 8-3. Dashboard Groups

Group	Definition and Content
 Project Name	• Name of the **active/main project.** • **Project type** (application or library) and the **project configuration** (default or user-defined).
 Device	• Project device. • Any **status flags** generated during Run or Debug. • **Checksum** of what is loaded into device memory. You must build to see this. Refer to 6.4.1. Checksums.
 Packs	Packs containing versioned device information `-.PIC` file and other files, depending on device.
 Compiler Toolchain	• **Project toolchain name** (e.g., XC8), the tool **version number** in ( ) and the **path** to the executable files in [ ]. • **Image type** (Production or Debug) and toolchain **optimization** level, which corresponds to license type. For more on optimizations, see your language tool documentation. • Information on the **compiler license***: – license type: Workstation or Network – license mode: Free, Standard or PRO (Standard no longer available) – license seats: total number of seats available – licenses available: the number of licenses available – HPA – days until HPA ends – Expire – days until the license expires – Press for status – See status in Output window * You must have at least MPLAB XC8 v1.34, MPLAB XC16 v1.25, or MPLAB XC23 v1.40 for this to be visible. For more on compiler licenses, see: www.microchip.com/mplab/compilers

.....continued	
Group	**Definition and Content**
Memory	• Usage symbols disabled. Click on message to enable Load Symbols. In the Project Properties window, Loading category, check “Load symbols when programming or building for production (slows process).” • **Type of memory** (Data or Program) and the **amount available** on the device. • **Memory Used** should be a guide to the amount of memory remaining. The compilers output a Memory Summary that details usage for Program Space, Configuration Bits, ID location, and EEPROM (if on the device). The sum of these memory spaces, allowing for word sizes, should agree with the Dashboard Program Used. The Map file should be examined when your device has smaller amounts of memory. **Note:** The open source AVR GCC compiler does not provide an accurate report of memory usage to display within the IDE. Due to this, the memory usage when using this compiler are estimates and are not exact.
Debug Tool	**Hardware Debug Tools** • Debug tool **name** and serial number. • Debug tool **connection**. By default, the tool connection is active at all times. To change this, go to *Tools>Options* (*MPLAB X IDE>Preferences* for macOS), **Embedded** button, **Generic Settings** tab, “Maintain active connection to hardware tool.” • Click the **Refresh Debug Tool Status** button to see hardware debug tool firmware versions and current voltage levels. **Simulator** • Debug tool **name**, i.e., Simulator. • **Click for Simulated Peripherals** to see a list of supported device peripherals.
Debug Resources	**Hardware Breakpoints** The number of breakpoints currently being **used** and the number of breakpoints **free** for the project device. **Software Breakpoints** Whether software breakpoints **supported** on the project device. **Debug Tool Accessories** Which, if any, debug tool accessories are used in this project, such as debug headers.

8.5.2 Dashboard Icons

Icons on the left side of the Dashboard window provide access to functions.

Table 8-4. Sidebar Icons

Icon	Function
	**Project Properties** Displays the Project Properties window.
	**Refresh Debug Tool Status** Click this to see hardware debug tool details.

.....continued	
Icon	**Function**
	**Toggle Software Breakpoint – Enabled/Disabled** Click to alternately enable or disable software breakpoints. The state of this feature is shown in the icon: • Red center: Disabled • Green center: Enabled • Black center: Not supported – when you have exceeded the number of break points available for your device or tool.
	**Open Device Data sheets** Get a device data sheet from the Microchip web site: www.microchip.com/ Click to either open a saved, local data sheet or open a browser to go to the Microchip web site to search for a data sheet.
	**Compiler Help** Click to open the Master Index (if available) for documents in the docs folder of the compiler installation directory.

8.6 Visualize Data

The MPLAB Data Visualizer plugin in now part of MPLAB X IDE as of v5.50. For details on this product and documentation, see the MPLAB Data Visualizer webpage.

To access the data visualizer in MPLAB X IDE, select *Window>Debugging>Data Visualizer* or *Window>Debugging>Power Monitoring* to open the data visualizer for power monitoring.

To see if an updated plugin version is available, go to *Tools>Plugins>Updates* and click **Check for Updates**.

8.7 Improve Your Code

Improve your code by using code refactoring and/or profiling.

Refactoring code is a method of making code simpler without changing its functionality. Currently you can do the following with C code:

- Find function usages throughout files
- Rename functions and parameters throughout files

For more information, see 7.7. C Code Refactoring.

Profiling code is an examination of CPU Usage, Memory Usage, and Thread Usage tools, all observed while the program is running. The profiling tools run automatically whenever you run your C project.

8.8 Control Source Code using Local History

MPLAB X IDE has a built-in local file history feature, a benefit of the NetBeans platform. This feature provides built-in versioning support for local projects and files that is similar to conventional version control systems. Available tools include a local DIFF and file restoration.

To see local history for a file, either:

- Click on the **History** button on an Editor window
- right click on the file in the Projects or Files window and select *History>Show History*

Any past changes to the file should be listed there.

Figure 8-6. Local History in Editor Window

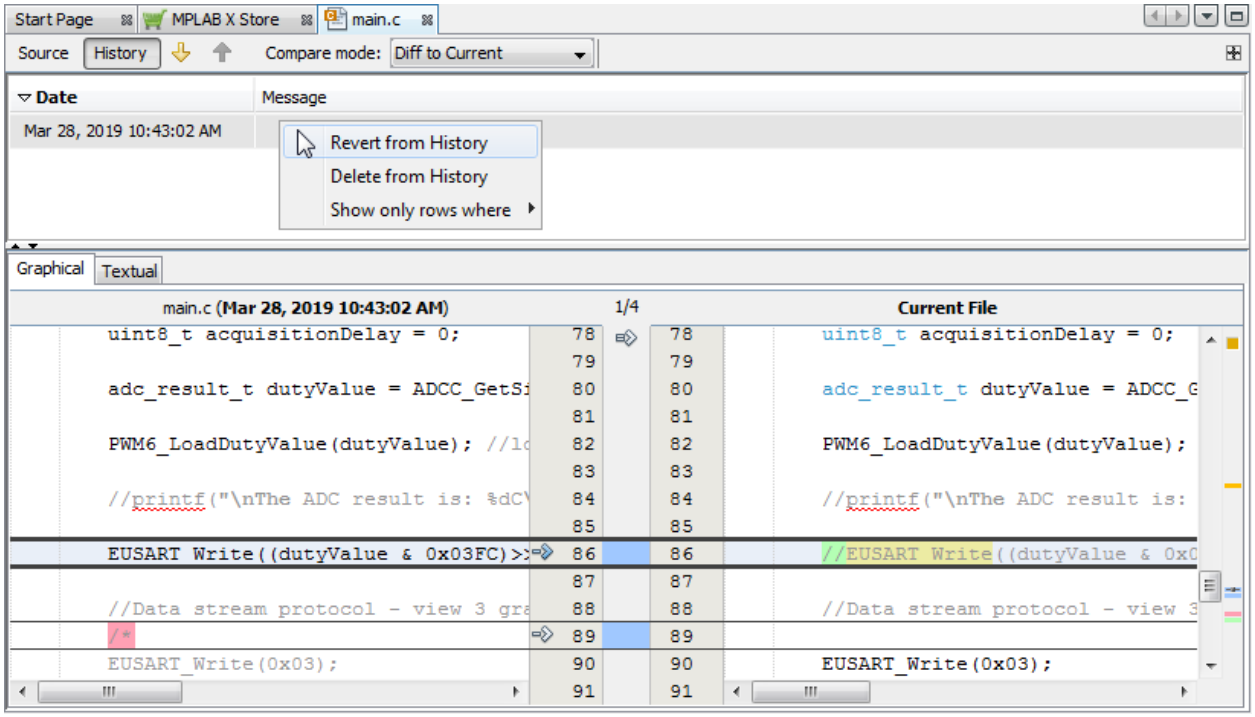


Table 8-5. Local History Context Menu

Menu Item	Description
Revert from History	To revert to a previous version. The Revert to dialog opens with any previous versions of the document. Select one and click **OK** to revert to that version.
Delete from History	To delete changed from history. To revert items deleted from history, right click on the file in the Projects or Files window and select *History>Revert Deleted*.
Show only rows where	Right click under the Date column to select date filtering (Date == or Date < >). Right click under the Message column to select message filtering (Message == or Message < >).

8.9 Control Source Code using a Revision Control System

When developing a complex application that consists of many files, especially as part of a team, controlling source code changes becomes a necessity. MPLAB X IDE provides a built-in method of source file versioning and supports external revision (source or version) control programs.

Several revision control systems (RCSs) may be used with MPLAB X IDE. Once set up, revision control actions can be accessed through a context menu.

Currently supported source/version control systems are Git, Subversion and Mercurial. Other systems may be supported through plugins (e.g., CVS).

When using a revision control system for your MPLAB X IDE projects:

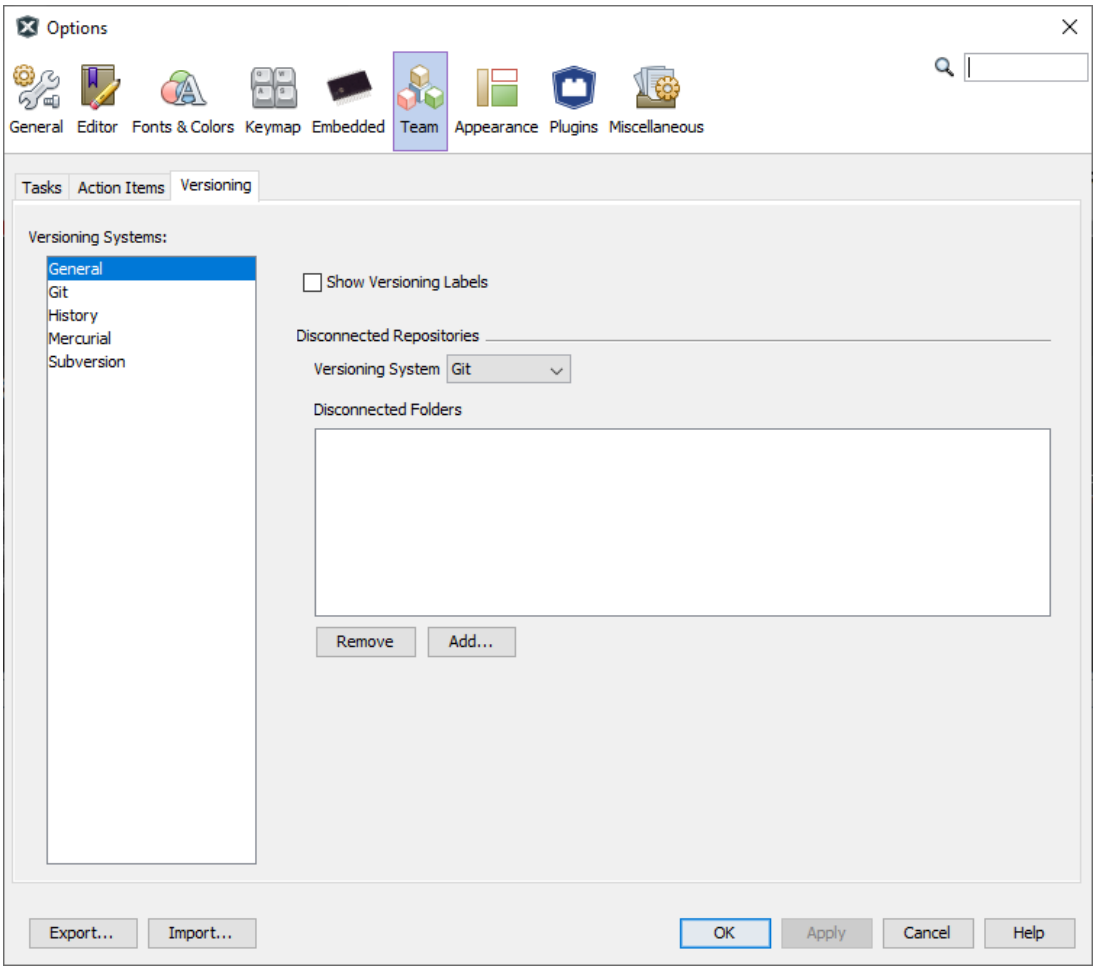
1. Try to keep all project files within the project. Files outside the project will also need to be included in the repository so the project can find them.
2. In MPLAB X IDE, commit only the files from the project. This is important as inclusion of unwanted files can prevent the project from compiling on other machines. See “Saving Project Files.”

3. Resolved any conflicts when checking in using the Merge Conflicts Resolver window. See “Resolving Conflicts in Revision Controlled Files.”
4. Check read-only files in per “Building a Project with Read-Only Files.”

To set up source/version control:

1. Team menu – select a version control program from the submenus and set up that version control program (see the figure below).
2. *Tools>Options* (*MPLAB X IDE>Preferences* for macOS), Team, Versioning – set up version control options.
3. *Window>Versioning* – open version control windows.

Figure 8-7. Version Control Options



Working with Git

Follow these links for more on Git:

- Git Website: git-scm.com
- Wikipedia: Git: en.wikipedia.org/wiki/Git
- Git Tutorial: git-scm.com/docs/gittutorial
- Using Git Support in NetBeans IDE: netbeans.org/kb/docs/ide/git.html

Working with Subversion

Follow these links for more on Subversion:

- Apache™ Subversion® Website: subversion.apache.org/
- Wikipedia: Apache™ Subversion®: en.wikipedia.org/wiki/Apache_Subversion
- Version Control with Subversion®: svnbook.red-bean.com/

- Using Subversion Support in NetBeans IDE: netbeans.org/kb/docs/ide/subversion.html

Working with Mercurial

Follow these links for more on Mercurial:

- Mercurial Website: www.mercurial-scm.org/
- Wikipedia: Mercurial: en.wikipedia.org/wiki/Mercurial
- Mercurial Tutorial: www.mercurial-scm.org/wiki/Tutorial
- Using Mercurial Support in NetBeans IDE: netbeans.org/kb/docs/ide/mercurial.html

Working with CVS

To use CVS with MPALB X IDE, you must install the CVS plugin. See 8.11. Add Plugin Tools.

- CVS - Concurrent Versions System: www.nongnu.org/cvs/
- Wikipedia: Concurrent Versions System: en.wikipedia.org/wiki/Concurrent_Versions_System
- CVS Manual: www.gnu.org/software/trans-coord/manual/cvs/cvs.html
- Using CVS Support in NetBeans IDE: netbeans.org/kb/docs/ide/cvs.html

Working with Other Revision Control Systems

Other revision control systems may be added as plugins. See 8.11. Add Plugin Tools.

8.9.1 Saving Project Files

There are project files that need to be saved into a repository.

The following table lists project files that either need or do not need to be committed to a version control repository.

Table 8-6. Project Files Saved to Repository

Directory or File(s)	Commit?
Project directory	
.generated_files directory	✗
`Makefile`	✓
Source files	✓
build directory	✗
dist directory	✗
nbproject directory	
`configurations.xml`	✓
`project.xml`	✓
`Makefile-*`	✗
`Package-*`	✗
private directory	✗

.....continued

Directory or File(s)	Commit?
Green Check: required to generate the project image Red X: these directories/files are regenerated and therefore do not need to be saved	

See "Files Window View" for more on the project structure.

8.9.2 Resolving Conflicts in Revision Controlled Files

Inevitably, conflicts will arise between a file you are trying to check in and the same file in the revision system repository that others have modified.

To understand how these issues can be resolved, see the example below.

Example: Conflict in `configurations.xml`

To resolve conflicts on the file `configurations.xml`:

1. Conflicts will be announced in:
   a. warning dialogs
   b. the output window
2. Click on the **Files** tab in the Projects window. Expand the `nbproject` folder. Find the file named `configurations.xml`. It should be in red font, signifying conflicts.
3. Right click on configurations.xml and select *RCS>Resolve Conflicts*, where *RCS* is your revision control system. This opens the Merge Conflicts Resolver window.
4. The Merge Conflicts Resolver window allows you to fix conflicts.
5. After fixing the errors, close the project. If the right click context menu for the project does not work, close the project using *File>Close Project*.
6. Reopen the project. The conflicts should be resolved.

8.9.3 Building a Project with Read-Only Files

To build a project where files have been checked into a repository and some may then be read only:

1. Commit the minimum number of files to source control. This is explained in 8.9.1. Saving Project Files.
2. Make the files `nbproject/configurations.xml` and `nbproject/project.xml` writeable.

8.10 Collaborate on Code Development and Error Tracking

Collaborate on code development with your group using a team server supported inside MPLAB X IDE. Use the "Team" menu to log into your account, create or open your project, share your project, get resources, send a chat message, or show your contact list. Collaborate on tracking bugs by using issue tracking systems.

Supporting menu items include:

- *Window>Services* – The Services window is the main entry point to your runtime resources. It shows a logical view of important runtime resources such as the servers, databases, and web services that are registered with the IDE. Search for "Services Window" under *Help>Help Contents* for more details.
- *Team>Find Task* – In the "Find Tasks" window, select a project task, select criteria, and click **Search**.
- *Team>Report Task* – in the "Report a New Task" window, select the new project task, specify the issue details, and click **Submit**.

For more on team projects and issue tracking, see the NetBeans Help topic:

netbeans.org/kb/docs/ide/team-servers.html

To find out more about these tools, see the following:

- Git – git-scm.com
- Mercurial – mercurial.selenic.com/
- Subversion – subversion.apache.org/

8.11 Add Plugin Tools

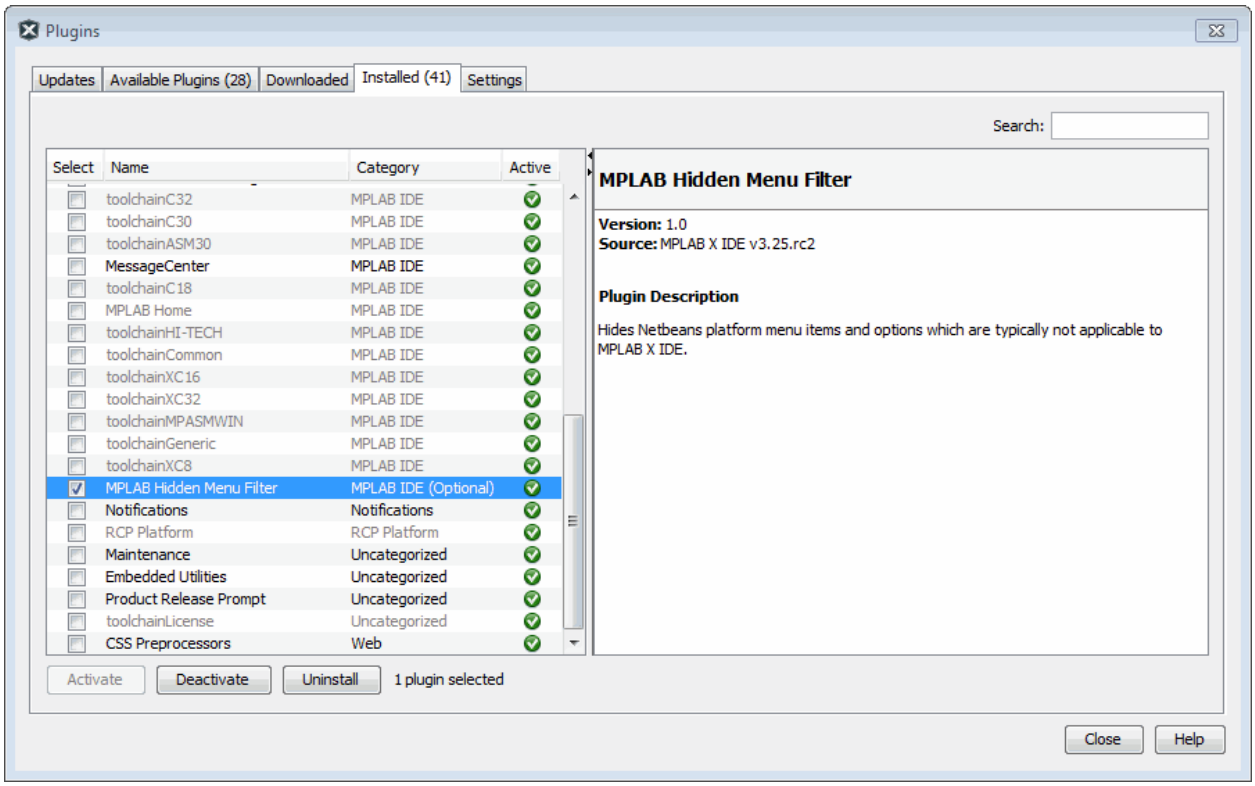
MPLAB X IDE plugin tools, like DMCI, are available from either the Plugin Manager in the IDE or the Embedded Code Source web site.

Viewing Installed Plugins

MPLAB X IDE comes with a number of plugins installed. Although ungrayed items may be deactivated or uninstalled, this is not recommended unless the category is listed as optional.

To view installed plugins, select *Tools>Plugins* and click on the **Installed** tab.

Figure 8-8. Installed Microchip Plugin Tools



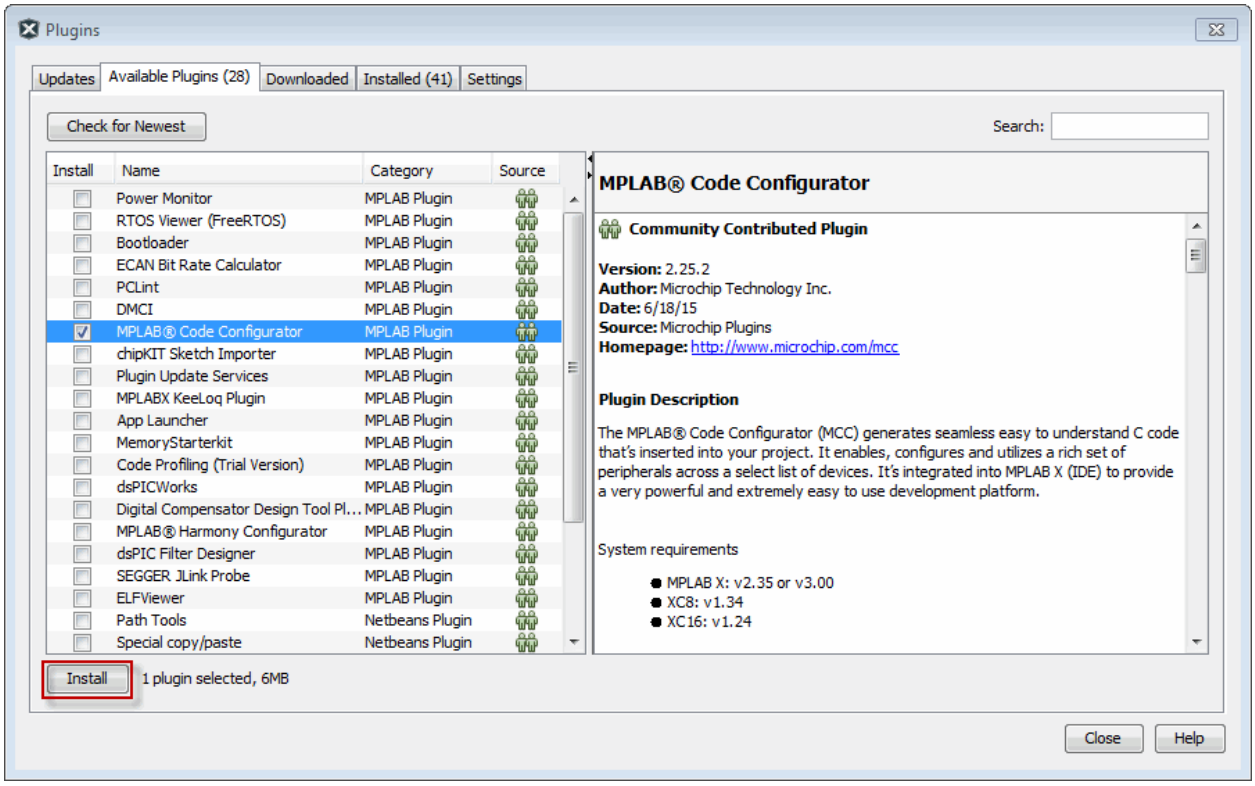
Add Plugins from the Plugin Manager

To view and install available plugins:

1. In MPLAB X IDE, select *Tools>Plugins* and click on the **Available Plugins** tab.
2. Select your plugin by checking its check box and then click Install (see figure below).
3. Follow the on-screen instructions to download and install your plugin.
   Note: Some plugins may be dependent on the modules in other plugins for their functionality to be implemented. The Plugins Manager warns you when this is the case.
4. Once installed, the plugin will be listed under the Installed tab, where it can be deactivated, reactivated or uninstalled.
5. Look for your tool under Tools>Embedded. If you do not see it, you may need to close and re-open MPLAB X IDE.

Click the **Help** button to read more about installing plugins.

Figure 8-9. Available Mlicrochip Plugin Tools

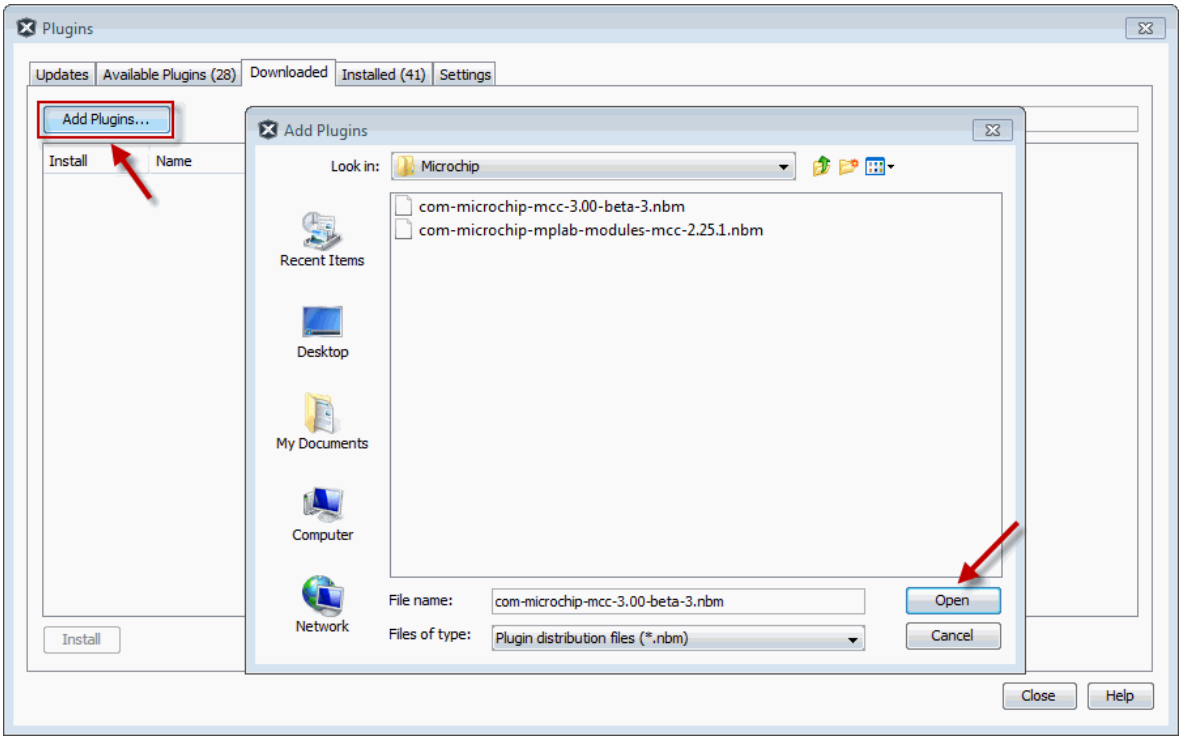


Add Plugins from Embedded Code Source

To view, download and install available plugins:

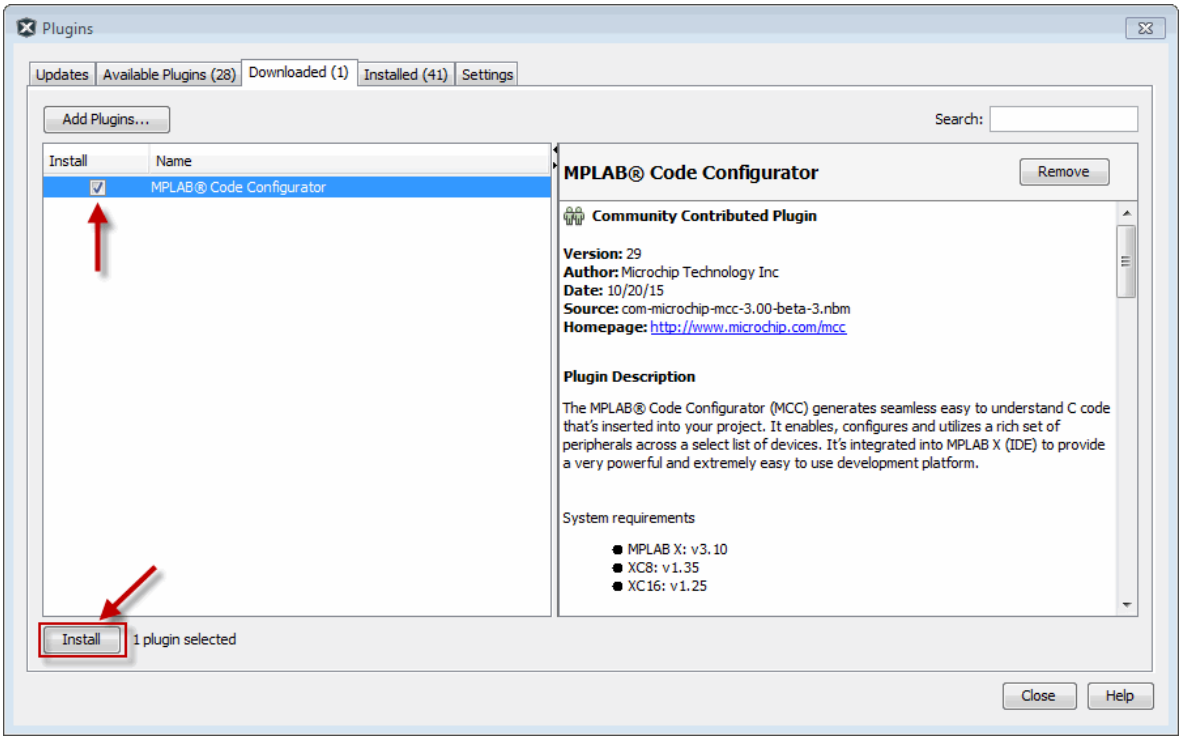
1. Go to the Embedded Code Source web site: www.embeddedcodesource.com/codedeveloper/microchip-technology
2. Select a plugin and follow the instructions on the web site to download to a folder on your computer.
3. Extract the `.nbm` file from the downloaded ZIP file.
4. In MPLAB X IDE, select *Tools>Plugins* and click on the **Downloaded** tab.
5. Click **Add Plugins**. Find, select, and open the `.nbm` file.

Figure 8-10. Downloaded Microchip Plugin Tools



6. Click Install to install the plugin. Ensure that the check box next to the plugin name is checked.

Figure 8-11. Install Downloaded Microchip Plugin Tools



7. Follow the on-screen instructions to download and install your plugin.
   Note: Some plugins may be dependent on modules in other plugins in order for the functionality to be implemented. The Plugins Manager warns you when this is the case.

8. Once installed, the plugin will be listed under the Installed tab, where it can be deactivated, reactivated or uninstalled.
9. Look for your tool under *Tools>Embedded*. If you do not see it, you may need to close and re-open MPLAB X IDE.

Upgrade Plugins

To determine if there is an update for your plugin:

1. In MPLAB X IDE, select *Tools>Plugins* and click on the Updates tab.
2. Click **Check for Updates**.
3. If any updates appear, ensure that the check box next to the plugin name is checked and then click **Update**.
4. Follow the on-screen instructions to update your plugin.

Installing a new version of MPLAB X IDE will NOT update your installed plugins. Plugins are tested against a versioned interface of a release. Not all plugins can be migrated between versions so they are not carried over. This is true for NetBeans, also. So, you may need to reinstall your plugins when you install a new version of MPLAB X IDE.

Configure Update Centers

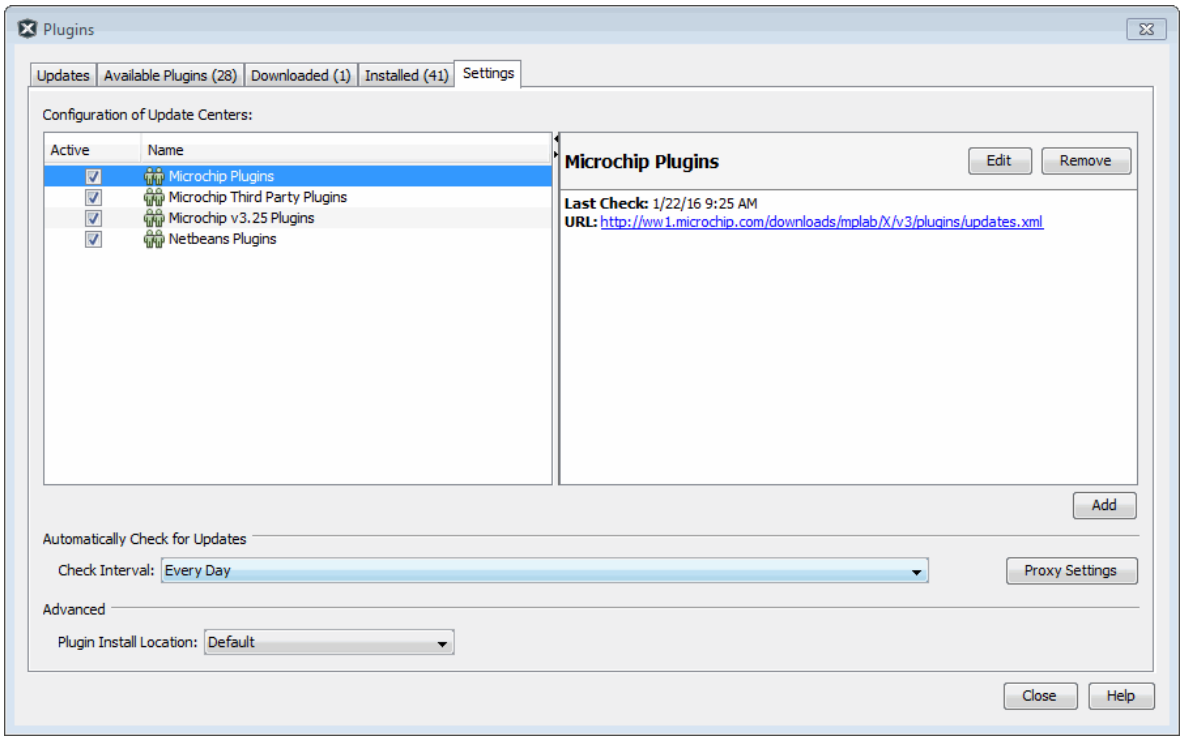
To see available plugins in the Plugins window, you must have one or more update centers configured. MPLAB X IDE comes with two Microchip Update Centers already configured:

- lww1.microchip.com/downloads/mplab/X/plugins/updates.xml
- ww1.microchip.com/downloads/mplab/X/thirdpartyplugins/updates.xml

To configure another Update Center in the Plugin Manager:

1. Select *Tools>Plugins* and click the Settings tab.
2. Click the Add button to open the Update Center Customizer dialog.
3. Enter a name for the update center.
4. Enter a URL for the update center.
5. Click **OK**.

Figure 8-12. Configure Microchip Update Center



Configure Installed Plugin

Once a plugin is installed in MPLAB X IDE, you may need to configure it. Plugin configuration options are available under *Tools>Options*, Plugins. See “Tools Options Plugins Window.”

Plugin Code Location

Plugin code is stored with MPLAB X IDE user configuration data. See 6.5.6. Viewing User Configuration Data.

8.12 Hardware Tool Connections and Debugging

The connection between your hardware debug tool and target can determine what debug features are available, in addition to the device-related debug features.

Table 8-7. Hardware Tool to Target Connections - PIC MCUs

Connection	Hardware Support(1)	Debug Support (2,3,4)	Trace Support(1)	Support Speed
Standard (ICSP) Communications	**RI**, **ICD4**, **ICD3**: Modular cable (6-pin RJ-11); **Snap**, **PK4**, **PK3**: Ribbon cable (6-pin SIL)	Basic and Advanced	**RI**: Native Trace	15 MIPS or less PIC32: Device Dependent
High-Speed (LVDS) Communications	**RI**: Performance Pak (AC244002)	Basic and Advanced	**RI**: Native Trace, **RI**: SPI Trace	Greater than 15 MIPS PIC32: Device Dependent
	RI: Performance Pak + Isolator Unit (AC244005)	Basic and Advanced	None	Greater than 15 MIPS PIC32: Device Dependent
JTAG	**RI**: JTAG Adapter (AC244007), **SJL**	Basic	None	PIC32: Device Dependent
Logic Port	**RI**: Logic Port Probes (4 pins)	N/A	**RI**: I/O Port Trace	Device Dependent
	RI:Trace Interface Kit (AC244006)	N/A	**RI**: Instruction Trace (PIC32 MCUs, some Emulation Headers)	Device Dependent
Standard (ICSP) Communications + Logic Port	**RI**: Power Monitor (AC244008)	Basic	None	15 MIPS or less PIC32: Device Dependent

1. For tool abbreviations, see last table.
2. Support is device dependent, i.e., some debug features are not available on all devices.
3. *Basic Debug Support*: Run, Halt, Reset, Step Into, Step Over, Software/Hardware Breakpoints.
4. *Advanced Debug Support*: Data Capture, Runtime Watch, etc.

© 2022 Microchip Technology Inc. and its subsidiaries

Table 8-8. Hardware Tool to Target Connections - AVR MCUs

Connection	Hardware Support[1,2]	Device Support[3]	Debug Support[4,5,6]	Trace Support	Support Speed
JTAG	AI, PK4	All AVR MCUs	Basic	No	32kHz to 7.5MHz
aWire	AI	AVR 32-bit MCUs	Basic	No	7.5kbit/s to 7Mbit/s
PDI 2-wire	AI, PK4	AVR XMEGA® MCUs	Basic	No	32kHz to 7.5MHz
debugWIRE	AI, PK4	AVR 8-bit MCUs	Basic	No	4kbit/s to 0.5Mbit/s
UPDI	AI, PK4	AVR 8-bit MCUs	Basics	No	Up to 750kbit/s
SPI	AI, PK4	AVR 8-bit MCUs	Programming only	N/A	8kHz to 5MHz
TPI	AI, PK4	tinyAVR 8-bit MCUs	Programming only	N/A	Device dependend

1. For tool abbreviations, see last table.
2. ICD4 and PK4 require the AC102015 Debugger Adapter Board.
3. Device support is on-going and may not be currently available.
4. Support is device dependent, i.e., some debug features are not available on all devices.
5. *Basic Debug Support*: Run, Halt, Software/Hardware Breakpoints.
6. *Programming Only*: No debugging with this connection.

Table 8-9. Hardware Tool to Target Connections - SAM MCUs

Connection	Hardware Support[1,2]	Device Support[3]	Debug Support[4,5,6]	Trace Support	Support Speed
JTAG	ICD4	SAM MCUs	Basic	No	32kHz to 7.5MHz
SWD	ICD4	SAM MCUs	Basic and Advanced	ITM - 3MB/s	32kHz to 10MHz

1. For tool abbreviations, see last table.
2. ICD4 and PK4 require the AC102015 Debugger Adapter Board.
3. Device support is on-going and may not be currently available.
4. Support is device dependent, i.e., some debug features are not available on all devices.
5. Basic Debug Support: Run, Halt, Software/Hardware Breakpoints.
6. Advanced Debug Support: Additional features over Basic; device dependent.

Table 8-10. Tool Abbreviations

Abbreviation	Tool Name
Snap	MPLAB Snap in-circuit debugger/production programmer
PK4	MPLAB PICkit 4 in-circuit debugger/production programmer
PK3	PICkit 3 in-circuit debugger/development programmer
ICD4	MPLAB ICD 4 in-circuit debugger/production programmer
ICD3	MPLAB ICD 3 in-circuit debugger/production programmer
RI	MPLAB REAL ICE in-circuit emulator/production programmer
AI	Atmel-ICE in-circuit debugger/programmer
SJL	SEGGER J-Link debug probes (Third Party)

For more on device-dependent debug features, see:

<MPLAB X IDE install>docs/FeatureSupport/HWToolDebugFeatures.html

8.13 Work with Third-Party Hardware Tools

When working with third-party hardware tools, these basic installation requirements apply:

- Acquire and install any related USB drivers for the hardware. USB drivers are usually available from the third-party site, along with installation instructions.
- Install the MPLAB X IDE plugin for this tool. For details, see 8.11. Add Plugin Tools.

When installation is complete, you should be able to see and select the hardware tool in the Project Properties window (see 5.4. View or Make Changes to Project Properties).

© 2022 Microchip Technology Inc. and its subsidiaries

9. Analysis

MPLAB X IDE supports MPLAB XC C compiler features that help you analyze:

- MISRA rules conformance
- how well your code is optimized
- how well your code is tested
- how to minimize your stack usage

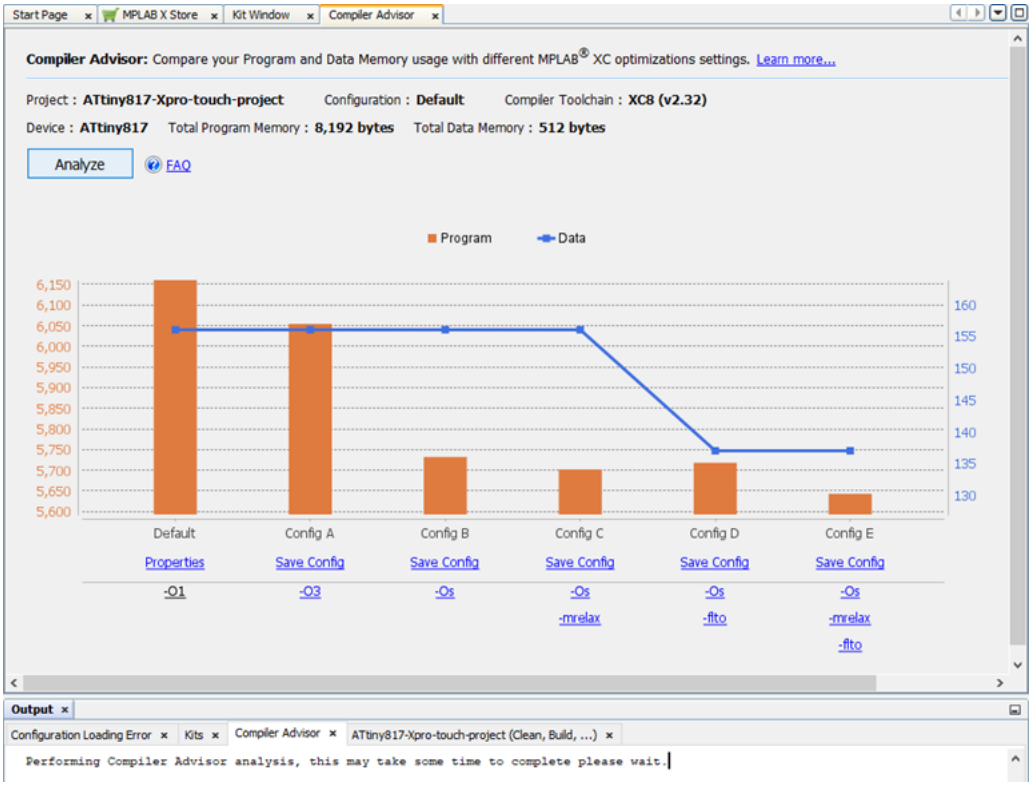
Currently available analysis features are in the following sections.

9.1 Compiler Advisor

Note: This content is also in the “Compiler Analysis in MPLAB X IDE User Guide” (DS-5000xxxx).

Compiler Advisor displays a graphical comparison of sets, with carefully selected available compiler optimizations using project code.

Figure 9-1. Compiler Advisor Example



This MPLAB X IDE plug-in can be useful in:

- Providing information on available compiler optimizations for each compiler type (XC8, XC16, XC32).
- Demonstrating the advantages each optimization provides for a project in an easy-to-read, graphical form for program and data memory size.
- Saving desired configurations.
- Providing links to optimization definitions for each configuration.

Compiler Support

Supported compiler versions:

- MPLAB XC8 v2.30 and later
- MPLAB XC16 v1.26 and later

- MPLAB XC32 v3.01 and later

No license is required for use. However, the number of optimizations for a free compiler will be less than for a licensed compiler.

MPLAB X IDE and Device Support

All devices supported in MPLAB X IDE will be supported in Compiler Advisor. Updated Device Family Packs (DFPs) will add device support.

9.1.1 Perform Project Analysis

To use the Compiler Advisor to analyze your project for different combinations of optimizations, follow the procedures in the following sections.

9.1.1.1 Select Project for Analysis

In MPLAB X IDE, open a project and in the Projects window either click on the project name to make it active or right click on the project name and select “Set as Main Project.”

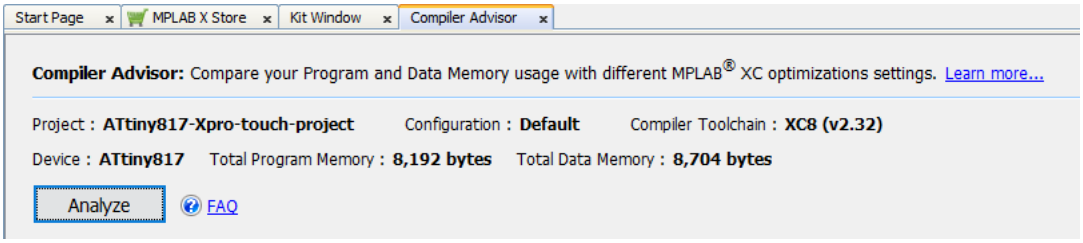
The project code, configuration, compiler and device will be used for the analysis. Therefore ensure that the compiler and device pack versions are supported as specified in 9.1. Compiler Advisor.

Note: You will be warned in Compiler Advisor before analysis if the compiler and device pack versions are not correct.

9.1.1.2 Open Compiler Advisor

Open the Compiler Advisor. Select *Analysis>Compiler Advisor* either by right clicking on the project or by using the Tools menu. Information about the selected project will be loaded into the Compiler Advisor and displayed at the top of the window (see figure below). Additionally, there are links to learn more about the Compiler Advisor or view Frequently Asked Questions.

Figure 9-2. Compiler Advisor with Project Information

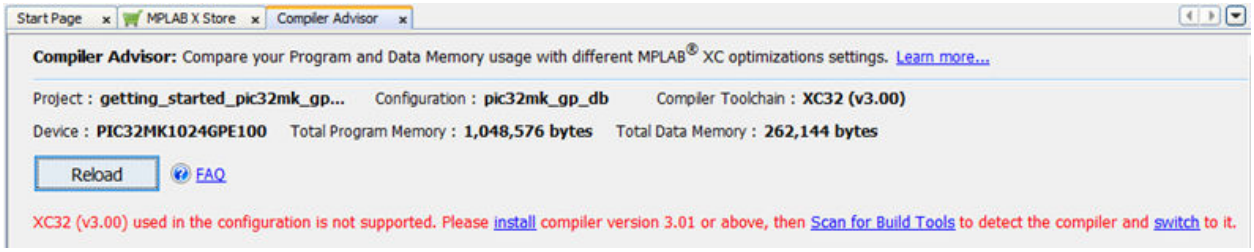


Verify that the project name, project configuration, compiler toolchain and device are correct for analysis. If you do not have a supported compiler or device pack version selected for your project, a note will be displayed. For example, a note about unsupported compiler versions will have links to help you (see figure below):

- Click “install” to open the MPLAB XC C Compiler webpage where you can download or purchase an updated compiler version.
- Click “Scan for Build Tools” to open the *Tools>Options>Embedded>Build Tools* tab where you can scan your system for existing compiler versions.
- Click “switch” to open project properties for compiler version selection.

Once you have completed any needed update, the Compiler Advisor will detect the change and request that you click **Reload**. Clicking this button will update the project information.

Figure 9-3. Note on Unsupported Compiler Version



If you make other changes to the project, such as changing the configuration, you will also need to **Reload**.

9.1.1.3 Analyze the Project

Once any project modifications are complete and loaded into the Compiler Advisor, click **Analyze**. The Compiler Advisor will build the project code several times using different sets of optimizations.

Note: Depending on code size, this may take some time.

When the analysis is complete, a graph will appear showing program and data memory used for each of the different configurations (see figures below). For a compiler in Free mode, the last column will show a PRO compiler comparison. To buy a PRO license, click the “Buy License” link to go to the MPLAB XC Compiler webpage to select the type of PRO license to buy.

The analysis information is saved in the project folder.

For details on the chart, see 9.1.2. Understand Analysis Results in Chart.

Figure 9-4. Free License Example

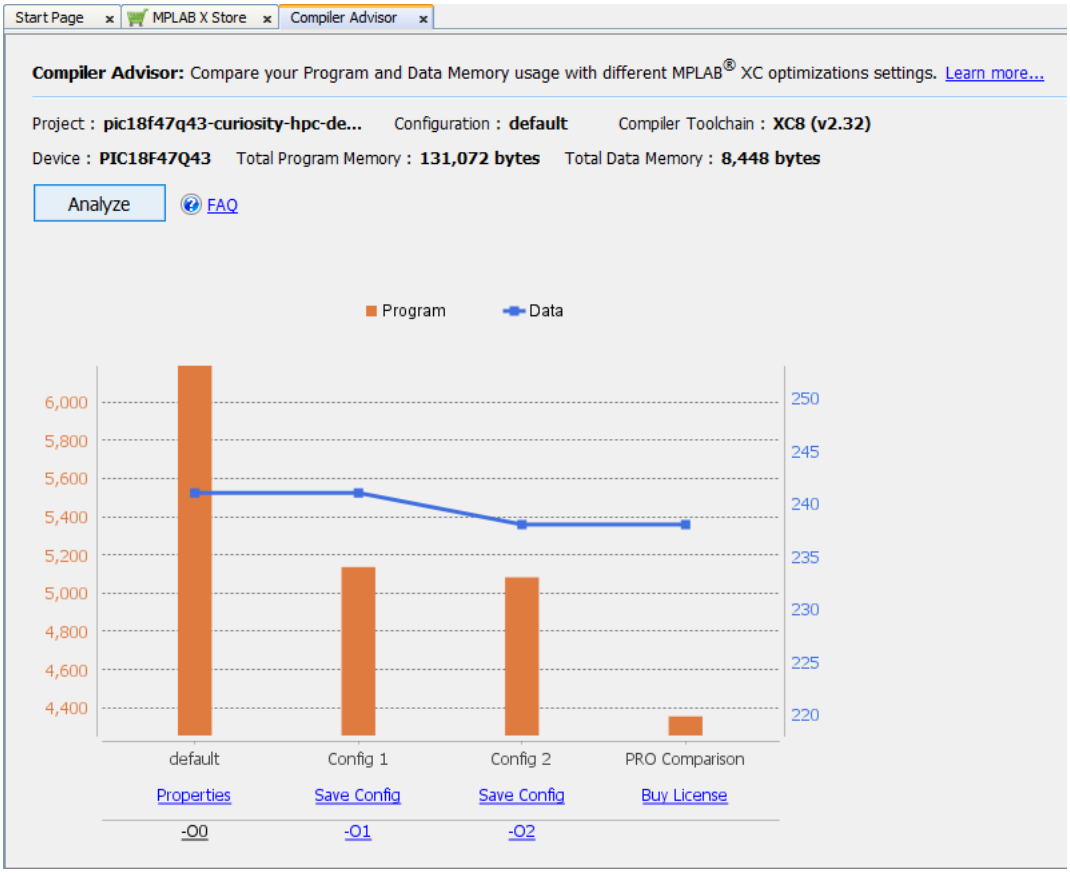
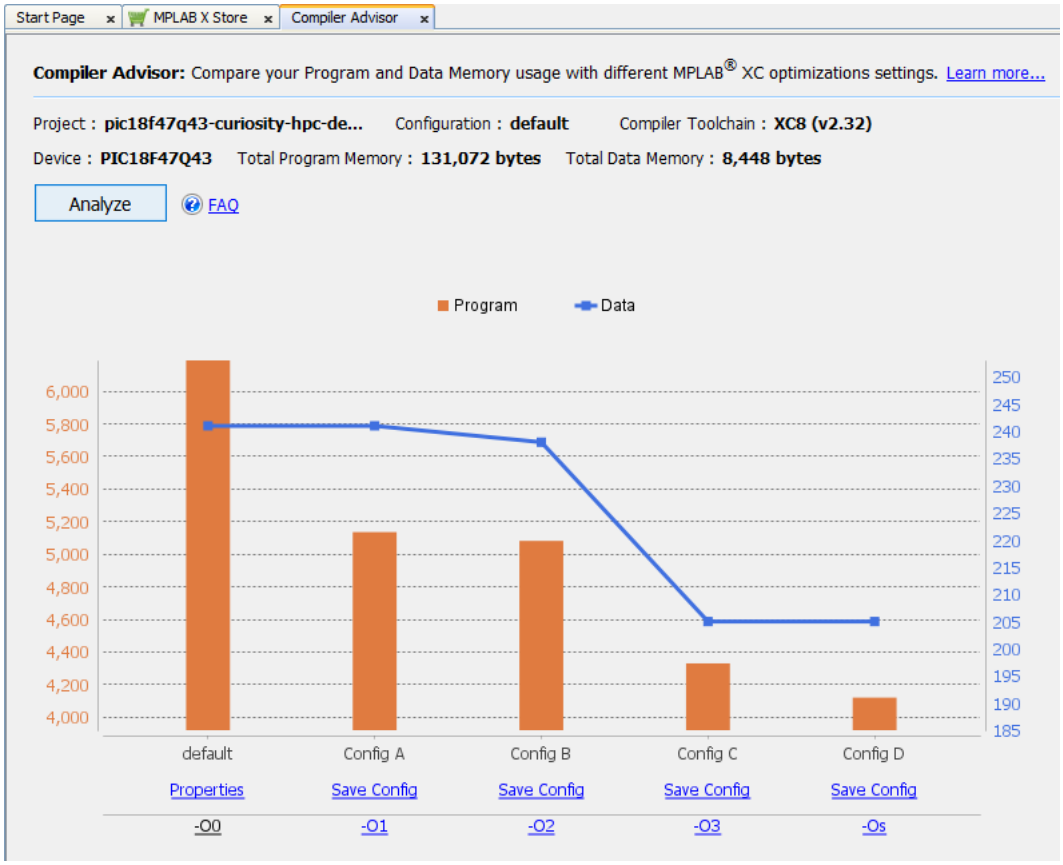


Figure 9-5. PRO License Example

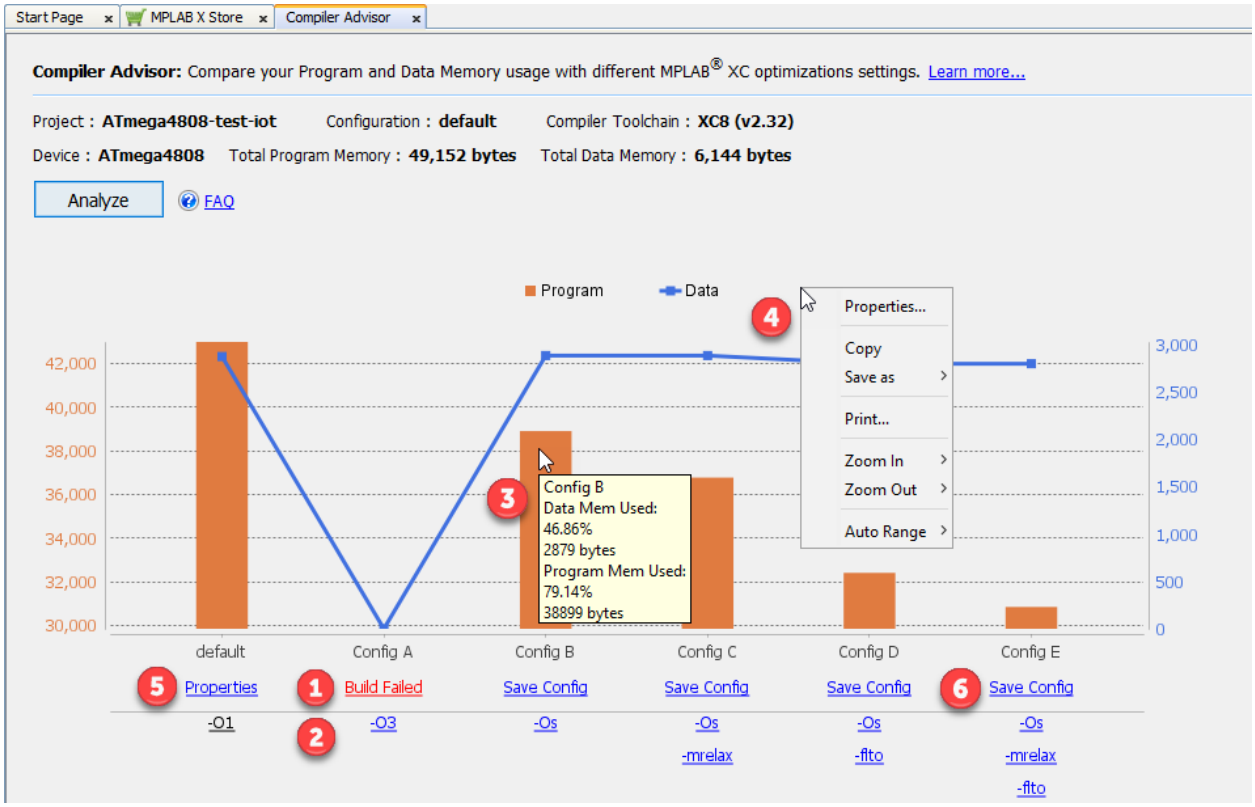


9.1.2 Understand Analysis Results in Chart

The chart generated after analysis has several features explained in the following sections. Use these features to determine if another configuration is right for your application.

1. 9.1.2.1. Find Build Failures
2. 9.1.2.2. View Configuration Optimizations
3. 9.1.2.3. View Configuration Data
4. 9.1.2.4. Use Context Menu Functions
5. 9.1.2.5. View Initial Configuration
6. 9.1.2.6. Save Configuration to Project

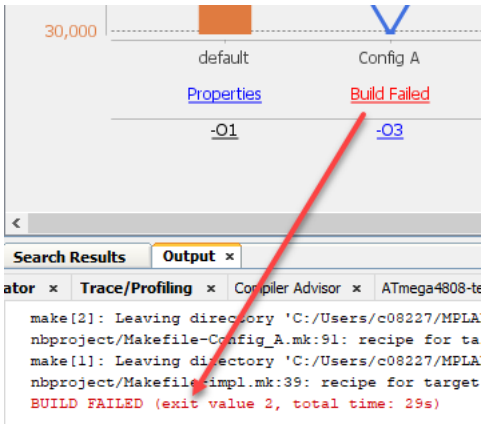
Figure 9-6. Annotated Chart Features



9.1.2.1 Find Build Failures

When a build fails due to certain optimization selections, you can click on Build Failed to go to where the error(s) are in the Output window.

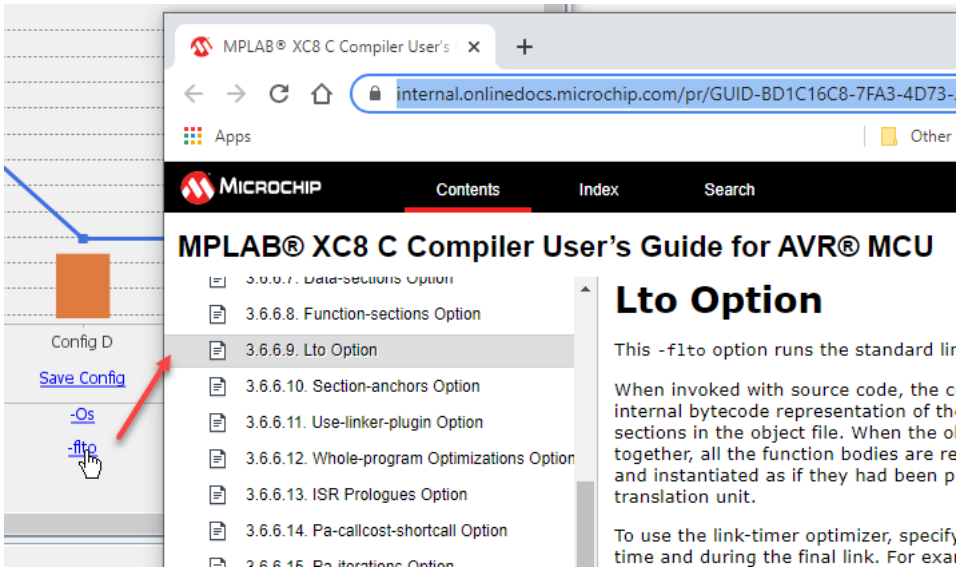
Figure 9-7. Build Failed Link



9.1.2.2 View Configuration Optimizations

Click the link of an optimization (e.g., `-Os`) used in a configuration to get more information. The link will take you to a description of the optimization in the compiler online documentation.

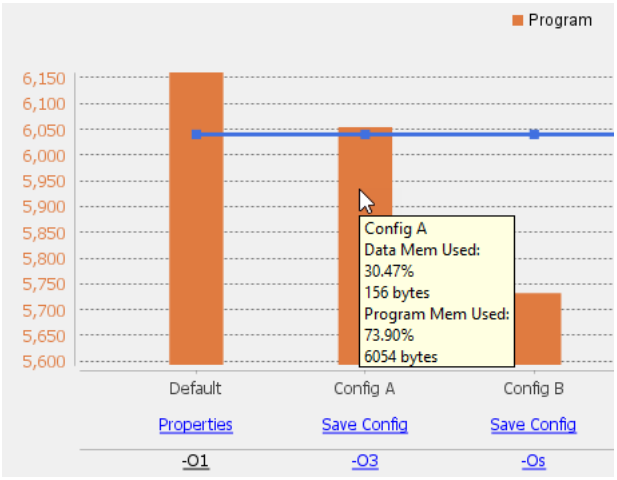
Figure 9-8. Click to See Optimization Description



9.1.2.3 View Configuration Data

To see the percentage and bytes of program and data memory used for each build configuration, mouseover a program memory bar for MCUs (see figure) and a data memory point for MPUs.

Figure 9-9. MCU Mouseover for Tooltip



9.1.2.4 Use Context Menu Functions

Right click on the chart to pop up the context menu with items listed in the table below.

Table 9-1. Compiler Analysis Context Menu

Menu Item	Description
Properties	Open the Chart Properties dialog. Add a title, format the plot or select other drawing options.
Copy	Copy an image of the chart to the clipboard. You may need to alter the Properties.
Save As	Save the chart as an image. You may need to alter the Properties.
Print	Print an image of the chart. You may need to alter the Properties.
Zoom In/Zoom Out	Zoom in or zoom out on the selected chart axes.
Auto Range	Automatically adjust the range of the selected axes for the data in the chart.

9.1.2.5 View Initial Configuration

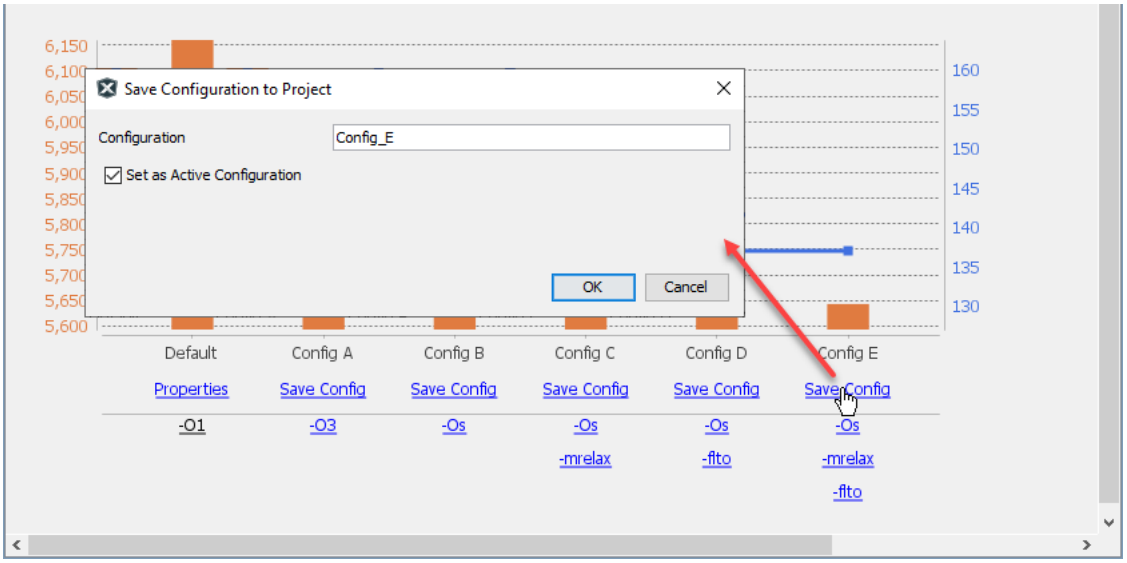
To view the initial project configuration used, click on “Properties” to open the Project Properties window.



9.1.2.6 Save Configuration to Project

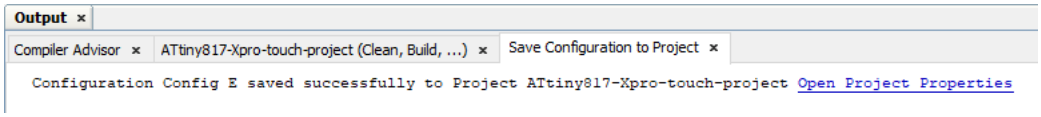
Click on the “Save Config” link under a configuration (e.g., Config E) that you want to add to your project. This will open the Save Configuration to Project dialog (see figure below). If you want this to be the active configuration in the project, check the checkbox. Then click **OK**.

Figure 9-10. Save Configuration to Project



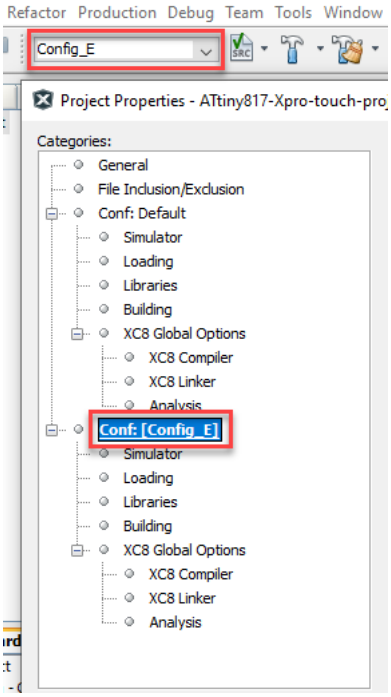
To open the Project Properties to see the added configuration, click the link in the Output window.

Figure 9-11. Open Project Properties from Output Window



The configuration is now added to the project. If the configuration was made active, it will also appear in the toolbar drop-down list.

Figure 9-12. Configuration Saved to Project



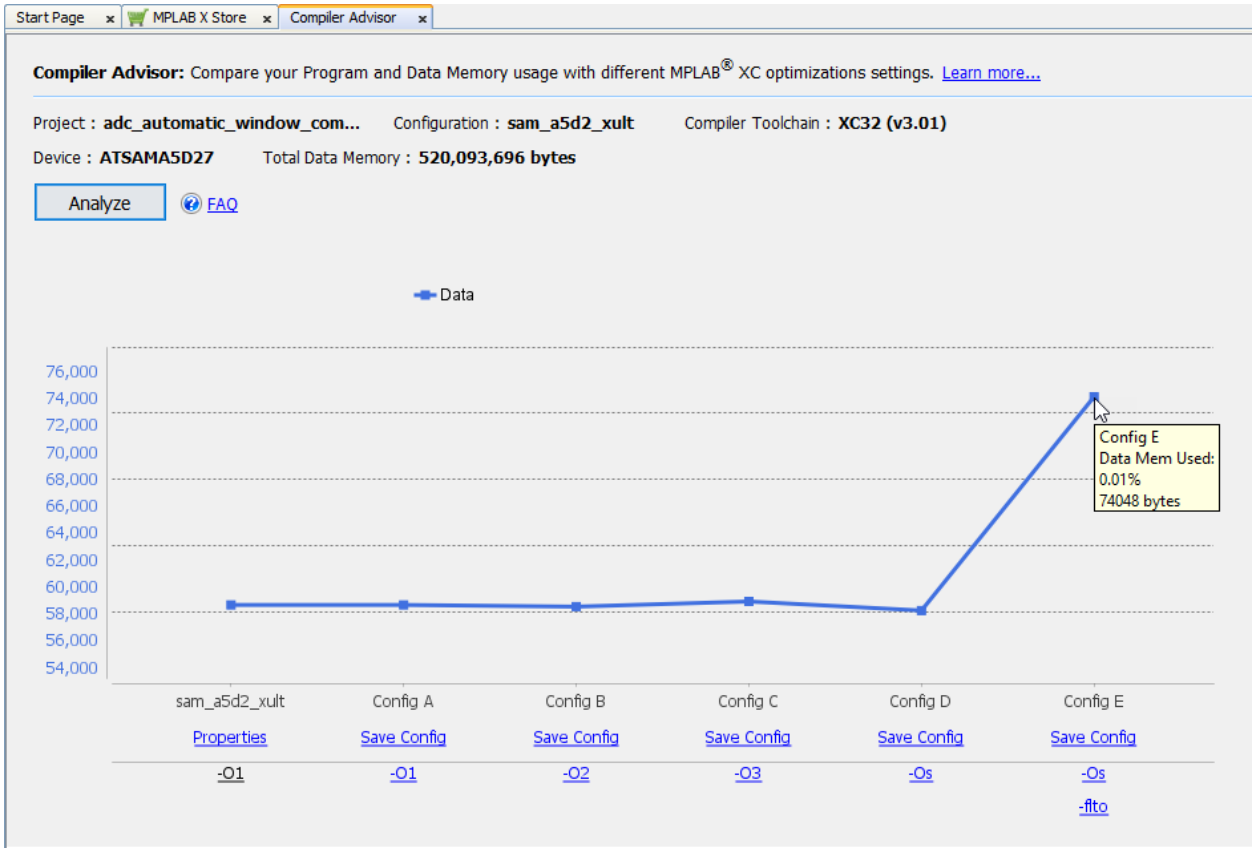
Note: Because the configuration has been added to the project, the Compiler Advisor will notice a change to the project properties and change **Analyze** to **Reload**.

9.1.3 Understand MPU Charts

The procedure to perform project analysis and the features of the resulting analysis chart are similar to those mentioned previously for MCU devices. Differences for MPU charts are:

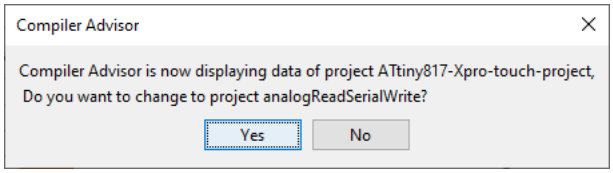
- MPU devices will only display information as data due to a combined program/data memory compiler output file.
- Data for each configuration may be seen by mousing over a data memory point.

Figure 9-13. MPU Chart from Analysis



9.1.4 Analyze Another Project

If you decide to analyze another project, select that project by making it active or main (see 9.1.1.1. Select Project for Analysis). Then reopen the Compiler Advisor (see 9.1.1.2. Open Compiler Advisor). A dialog will ask if you want to change from the existing project to the new project (see figure below). If you select **Yes**, then the Compiler Advisor window will be updated with the details of selected project.



9.2 Stack Guidance

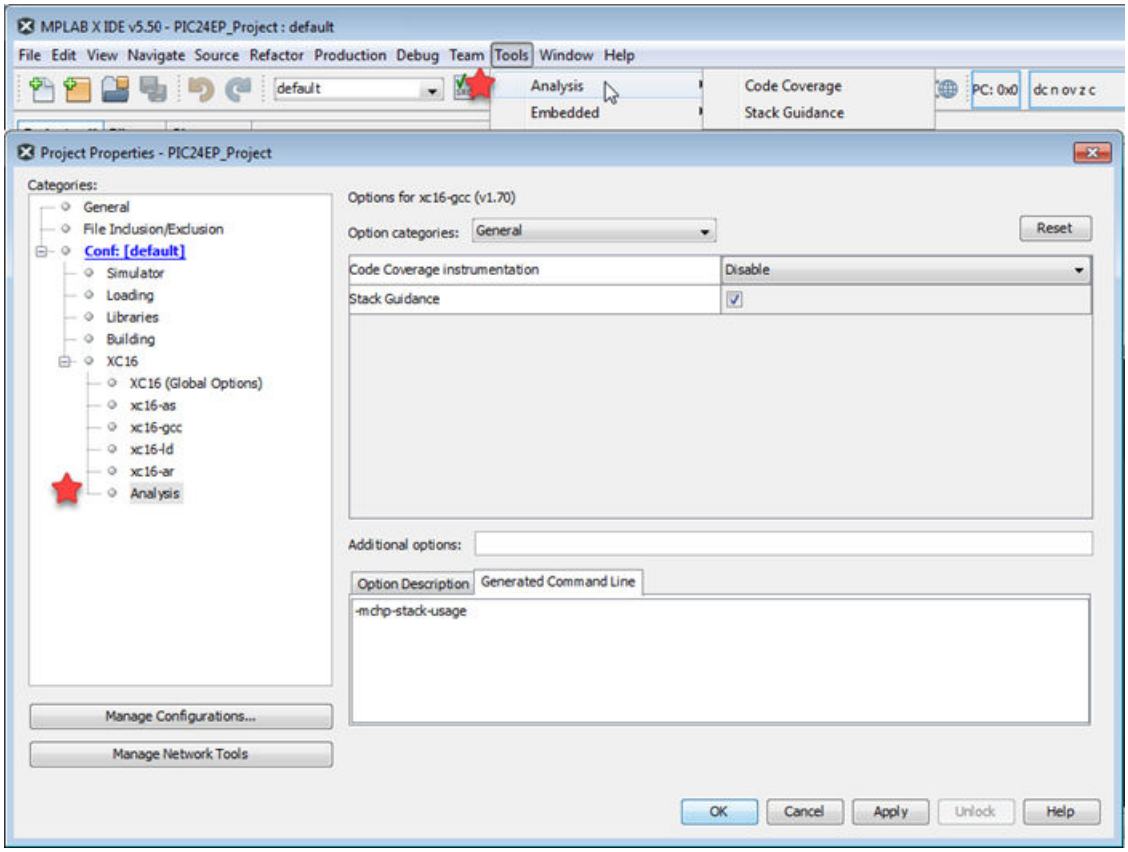
MPLAB XC C compilers provide a feature called 'Stack Usage Guidance'. This is a compile-time static analysis of the executable meant to guide you to an appropriate worst-case stack usage requirement. This guidance may need to be tempered with other facts that cannot be determined by a static analysis.

Support for Stack Guidance begins with these versions:

- MPLAB X IDE v5.50
- MPLAB XC8 v2.32
- MPLAB XC16 v1.70
- MPLAB XC32 v3.00

This feature requires a 'PRO' compiler license. When enabled, the compiler will provide some additional information pertaining to the linked executable to the terminal or Output window of MPLAB X IDE. Additionally, if a 'map' file is being specified, the language tool will emit more detailed information to the Map file for a permanent record.

Find more information on this feature in each compiler user's guide.



9.3 MPLAB Analysis Tool Suite

MPLAB Analysis Tool Suite provides tools for active and static analysis of application code. Purchasing the license enables the full features of suite tools:

- MPLAB Code Coverage for active analysis of running code.
- MPLAB MISRA Check for static analysis of code based on MISRA C rules.

9.3.1 MPLAB Code Coverage

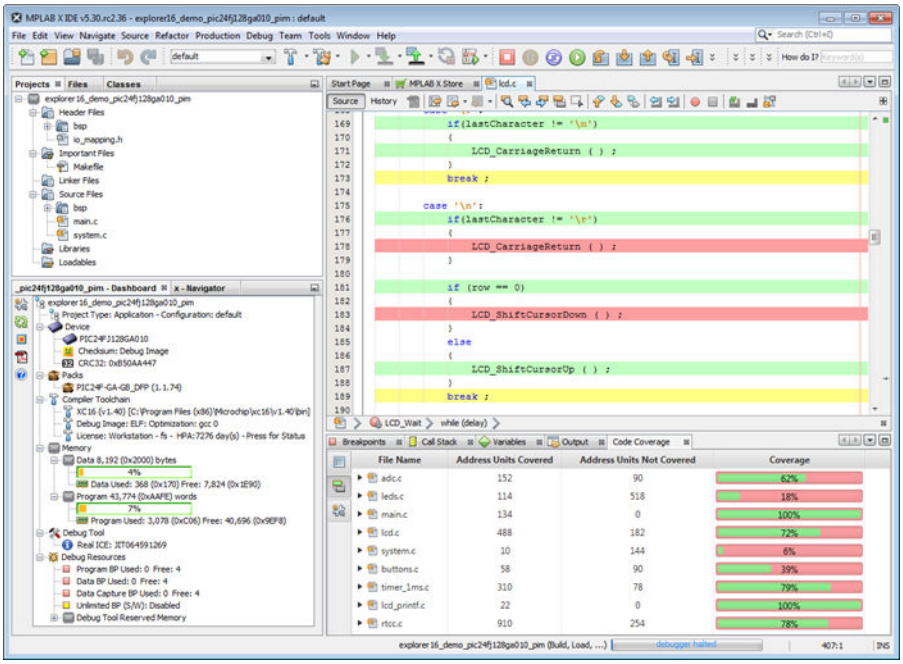
Code coverage provides visibility as to what portions of the code are being executed. To view code coverage output, an MPLAB XC C compiler that supports code coverage must be used with MPLAB X IDE. Support for code coverage begins with these versions:

- MPLAB X IDE v5.25
- MPLAB XC8 v2.10
- MPLAB XC16 v1.40
- MPLAB XC32 v2.30

In MPLAB X IDE, you can enable code coverage in the Project Properties window under a compiler category. Once enabled, debug your test code to execute it completely and then halt/pause. Select *Window>Debugging>Code Coverage* to see the percentage of covered code executed/total code covered in the **Code Coverage** tab window.

For more detailed coverage information, you can purchase an MPLAB Analysis Tool Suite license to activate the full code coverage features of your compiler to include:

- Editor text highlighted by colors representing coverage: green = executed, yellow = partially executed, and red = not executed.
- Program memory highlighted by colors representing coverage.
- A Code Coverage tab with a report displaying color percentages of code covered. This information may be written to an HTML Report for later viewing.



For more information, see:

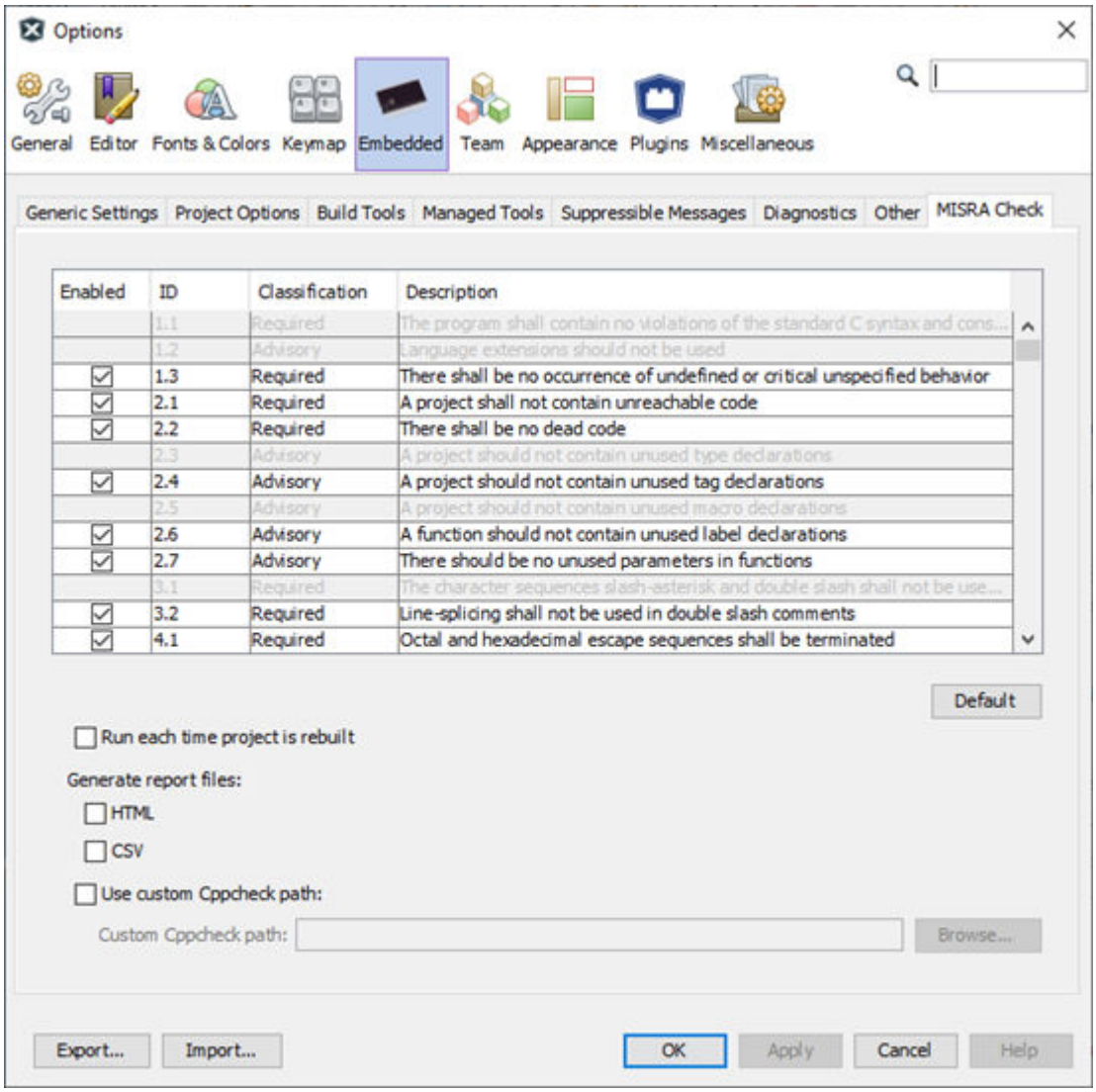
“MPLAB Analysis Tool Suite User’s Guide”

9.3.2 MISRA Check

MISRA® is a set of C and C++ coding standards developed by the Motor Industry Software Reliability Association (MISRA). MISRA guidelines help to ensure safe, secure, portable and reliable C/C++ code in embedded systems.

MISRA rules settings are located in *Tools>Options>Embedded>MISRA Check* (see figure).

To generate a detailed report, you can purchase an MPLAB Analysis Tool Suite license to activate this feature.



For more information, see:

“MPLAB Analysis Tool Suite User’s Guide”

10. Automation

Manually checking your application under different scenarios can be time consuming and error prone. Automating this process can save time and provide trusted results.

10.1 IDE Scripting (autoload.py)

MPLAB X IDE provides scripting capabilities to control the IDE behavior programmatically. 32-bit SAM microprocessor (MPU) projects in particular need to use scripts to support bootstrapping. However, scripts can be used with MCU projects as well.

The scripts are written in Jython. The scripts have access to an API that allows you to:

- Control the debugging session state (run, halt, etc.).
- Access memory in the device while debugging.
- Set breakpoints with callbacks to functions that determine if a halt should happen or not. This allows implementation of conditional breakpoints.
- Place actions in the Projects window which, when clicked, will run methods in a script.
- Provide hooks for debug events: a function will be called when a given event happens.

10.1.1 Default autoload.py File

If you create a project for a 32-bit SAM MPU, the project wizard will add a “Script Files” folder to your project and a default `autoload.py` file in that folder.

If you have an existing project to which you would like to add scripting features, follow the steps below:

1. Close the project in MPLAB X IDE.
2. Create a file named `autoload.py` or use the self-documenting example `autoload.py` that explains the APIs. Find the example file in `<MPLAB X IDE Install folder>\docs`.
3. Copy the file into your project directory. Edit the file here as necessary.
4. Re-open the project and you will see a “Script Files” folder in the project tree. The file `autoload.py` will be under this folder.

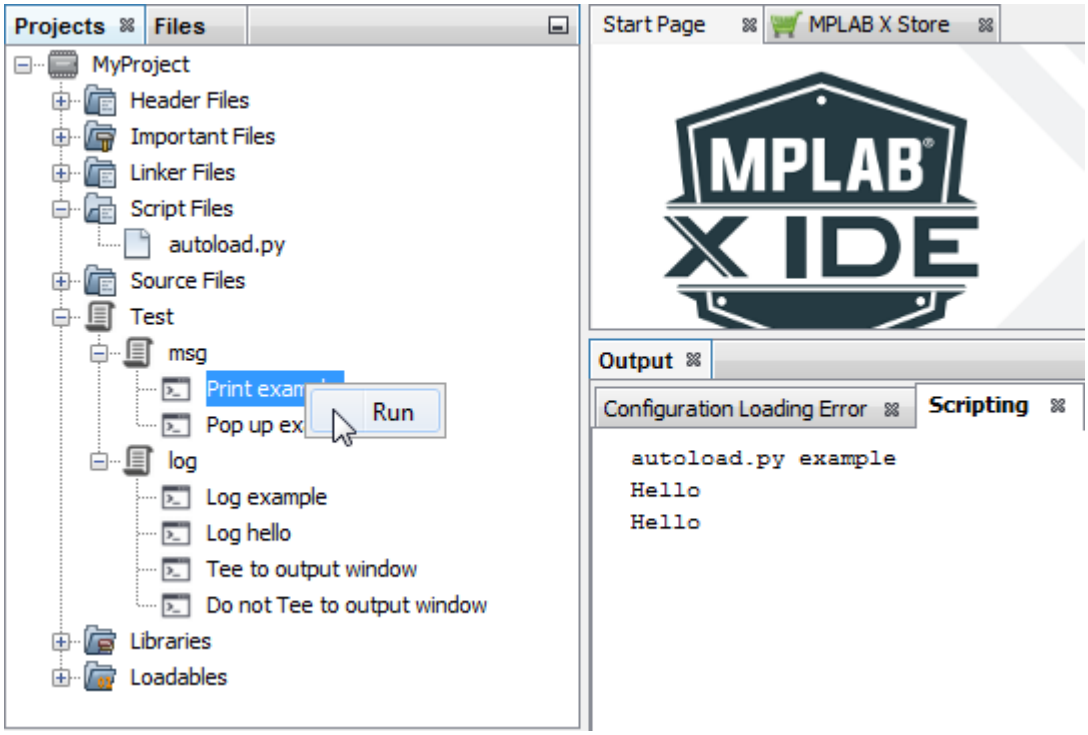
Note:
Whenever you edit the `autoload.py` file, you will have to shut down and reboot MPLAB X IDE before updates take affect.

For a 32-bit SAM MPU project, edit the `autoload.py` file for your project, shut down and reboot MPLAB X IDE, and debug your application.

For the example project, there is also a “Test” folder. Simply run the scripts under “Test” as needed.

© 2022 Microchip Technology Inc.
and its subsidiaries

Figure 10-1. Unedited Example autoload.py in Project



10.1.2 Debug Extensibility Using Python Scripts

A developer can use python scripts in `autoload.py` to customize the debugging experience in MPLAB X IDE.

Why use scripts:

- To automate complex setup scenarios (e.g. debug applications normally loaded by a bootloader.)
- To automates debug tasks (e.g. stop external processes when an MCU is halted in the debugger, such as robot motors.)
- To add actions to breakpoints (e.g. send mail on breakpoint, inspect and change memory during a halt, etc.)
- To read and write device memory, registers, set breakpoints, halt and resume device execution.

Scripts can use most of the Jython libraries, such as file access, mail access, regexp etc.

How to use the scripts:

- MPLAB X IDE will parse autoload.py from the project folder, if present.
- autoload.py can load other python files or modules. However, the variables injected by the IDE into the python namespace are not visible in files/modules loaded from autoload.py
- MPLAB X IDE calls functions with hardcoded names when a debugging event occurs. These functions can manipulate and alter the memories and registers of the device being debugged.

10.1.2.1 Invoking Python functions from the UI

The onload python hook allows you to add menu items to the project tree (in the Projects window). When selected, these items will invoke the registered callback function.

To persist user defined objects across multiple sessions, Python code can store and fetch strings in the **settings** map.

Here´s an example from the SAML11 scripts. In this example, the functions are meant to be called outside debug sessions. As such, when invoked, the code in the python function will ask the debugger to connect to the device and perform memory and register access to manipulate the device specific debug access level:

```
def onload(ide):
    ide.addCommand("Device actions|Set DAL 0", "set_dal_0")
```

```
    ide.addCommand("Device actions|Set DAL 1", "set_dal_1")

def set_dal_0():
  global dal
  special_action([],L11SDAL0)
  msg.msg("DAL is now " + str(dal),"Set DAL 0")

def set_dal_1():
  global dal
  special_action([],L11SDAL1)
  msg.msg("DAL is now " + str(dal),"Set DAL 1")

def special_action(key,cmd):
  if deb.Connected():
    r=msg.msg("Debugger is in use. Stop debugging and retry","Busy")
    return
  deb.Connect()
  try:
    SpecialOp(key,cmd)  # initiate bootrom cmd or change dal
  finally:
    deb.Disconnect()
```

10.1.2.2 Adding Board and Project Specific Properties

Sometimes a new project-level property needs to be added to support the feature set of a given board. For example, one might like to describe external memories, ranges, timings, etc., and use them during debug launch for proper setup.

Such properties can be declared in a function called `onShowProps` which needs to be placed inside `autoload.py`. They are edited using the tool properties tab of the Project properties dialog box.

The properties are persisted within the project file, making them available for future debug sessions.

```
def onShowProps(t):
    ide.addTextProperty(t,"x.ddram.timing" , "*|programoptions|
programoptions.eraseb4program","DDRAM frequency","Enter DDRAM frequency", "1200000")

    ide.addEnumProperty(t,   "x.memory.type", "*|programoptions|
programoptions.eraseb4program","External memory", "Select external memory", "ddram2",
                        "ddram2|DDRAM 2|ddram 2 memory\n"
                        "ddram3|DDRAM 3|ddram 3 memory\n"
                        "qspi|QSPI mem|QSPI  memory")
```

The properties are persisted in the **settings** object:

```
    timing  = settings.getNumeric("x.ddram.timing", 120000)
    extMem = settings.getString("x.memory.type","ddram2") # default to ddram2 if project
never configured
```

10.1.2.3 Debugging autoload.py

To debug your code you must you tracing statements or third party libraries. You can use `log.debug` or `log.info` statements to print to the output view, or `msg.msg` statements to pop up dialog boxes.

10.1.2.4 Adding custom Project Properties

The project properties dialog in MPLAB can display a custom JPanel , created using Java Swing objects from within jython.

- To enable MPLAB X IDE to discover the new JPanel, `autoload.py` must contain a function called `mplab_configure_XXXXXXX` , where `XXXXXX` is a user-defined string of characters:. An entry named `XXXXXXX` will show up under the configuration node of the Project properties.
- If present,`save_mplab_configure_XXXXXX` is called when the user closes the Project Properties dialog or presses **Apply**. At this time, the script should read user input from the Java Swing objects and save them to the settings object using `settings.setString()`.

Example

```
def mplab_configure_Bootstrap(confName):
    global bsOpt
    if bsOpt == None:
```

```
        bsOpt=BootstrapPane()
    return bsOpt.getPanel()

def save_mplab_configure_Bootstrap(confName):
    global bsOpt
    if bsOpt == None:
       return

    settings.setString("boot.path", bsOpt.binTxt.getText())
    settings.setString("boot.prj", bsOpt.prjTxt.getText())
    settings.setString("boot.load_adr", bsOpt.loadTxt.getText())
    settings.setString("boot.entry_adr", bsOpt.entryTxt.getText())
    settings.setString("boot.use", "true" if bsOpt.useBs.isSelected() else "false")
```

For a complete example, look in the `j-link_cortex-a5.py` file in the MPLAB pack folder . This script creates and declares a Bootstrap pane for MPU projects.

10.1.2.5 Function Reference

The following sections describe the available functions/methods available for scripting use.

10.1.2.5.1 IDE Object

Documentation for the `ide` object . Many functions are only available starting from MPLAB X IDE v5.40.

addTextProperty

`addTextProperty(t, id, location, name, desc, defaultVal)`

Declare a property with id `id`, to be inserted before the element described by `location`. The text property will be labeled with `name` and will be initialized with `defaultVal`. The description string `desc` is used to show help for this property.

Example

```
ide.addTextProperty(t, "x.favDrink" , "*|memories|AutoSelectMemRanges","Favourite
tea","Please enter favourite drink", "Matcha latte")
```

addTextPropertyExt

`addTextPropertyExt(t, id, location, name, desc, defaultVal, validator)`

Same as `addTextProperty` but with a validator function `validator`. The validator function is supposed to return `True` for accepted inputs, and `False` for invalid inputs. Invalid inputs will be flagged as red in the UI but are still accepted (dialog box can be closed). The reason for flagging an input as invalid must be set in the global variable `validationError`, and will be shown in the help area for the property pane.

Example

```
def ek_validator(s):
    global validationError
    if s != "0xC0FFEE,0xBABE,0xADDED,0xCACA0":
        validationError ="That's no fun !"
        return False
    return True

def onload(ide):
    ide.addTextPropertyExt(t,"x.eraseKey" , "*|programoptions|
programoptions.eraseb4program","Erase key with validation","Enter 4x32 bit numbers separated
by comma", "0xC0FFEE,0xBABE,0xADDED,0xCACA0", "ek_validator")
```

addEnumProperty

`addEnumProperty(t, id, location, name, desc, defaultVal, choicesString)`

Declare a property with id `id`, to be inserted in the dialog box before the element described by `location`. The property is labeled with `name` and will be initialized with the choice that has the `defaultVal` . The description string `desc` is used to show help for this property.

The user can choose one of the choices listed in `choicesString`. Different choices are separated by a carriage return character. Each choice is described by three fields separated by a bar. The first field is the `choiceId`, the second is a human readable description, and the third is a help string that is shown in the help area.

Example

```
def onShowProps(t):
     ide.addEnumProperty(t,"x.erase.type", "*|programoptions|
programoptions.eraseb4program","Chip erase type", "Select memories to erase", "ChipEraseAll",
            "ChipEraseAll|All Memories + NVM configuration rows (ChipErase_ALL)|Description
for ChipErase_All\n"
            "ChipEraseS|All Memories excluding BOOT region (ChipErase_S)|Description for
ChipErase_S\n"
            "ChipEraseNS|All Non Secure Memory Regions excludng BOOT region(ChipErase_NS)|
Description for ChipErase_NS")
```

addPropCategory

`addPropCategory(t, id, location, name)`

Declare a new category of options with `id`, to be inserted before the category described by `location`. The category will be available in the dropdown list of categories for the current tool.

Example

```
def onShowPropsDFP2(t):
    ide.addPropCategory(t,"diagnostics" , "*|toolpack", "Diagnostics")
    ide.addBooleanProperty(t,"script.show_output" , "*|diagnostics|*","Output diagnostic
logging","Show diagnostic text in the Output window", False)
    ide.addTextProperty(t,"script.log_level" , "*|diagnostics|*","Diagnostic log level","Set
diagnostic level (usually 0 1 or 3)", "0")
```

addBooleanProperty

`addBooleanProperty(t, id, location, name, desc, defaultVal)`

Declare a property with id `id`, to be inserted before the element described by `location`. The Boolean property will be labeled with `name` and will be initialized with the boolean value `defaultVal`. The description string `desc` is used to show help for this property.

addCommand

`addCommand(barSeparatedCmdPath , funcToCall)`

Insert a menu entry in the project view. The menu hierarchy is described by bar `|` separated submenus. The last string is shown as the command to invoke. When double clicked by the user, MPLAB X IDE will execute the function `funcToCall`.

Example

```
def onload(ide):
    ide.addCommand("Test|msg|Print example", "print_hello")
    ide.addCommand("Device actions|Set DAL 0", "set_dal_0")
    ide.addCommand("Device actions|Chip Erase", "chip_erase_l10")
def set_dal_0():
    if deb.Connected():
        r = msg.msg("Debugger is in use. Stop debugging and retry", "Busy")
        return
    deb.Connect()
    try:
       dev.Write32... # send commands to NVM controller to set DAL 0
    finally:
       deb.Disconnect()
```

Commands for the build process follow.

makeSafePath

`makeSafePath(windowsPathWithBakwardSlashes)`

Returns a file path with forward slashes only (replaces backslashes by forward slashes).

Example

```
fwdSlash = ide.makeSafePath("C:\\dev\\path\\file.txt")
```

expandProjectMacros

expandProjectMacros(project, stringToExpand)

Returns a string where the macros in `stringToExpand` are expanded in the context of the active configuration of the project, for debug mode.

Example

```
project_dir = ide.expandProjectMacros("${ProjectName}", "${ProjectDir}")
```

expandProjectMacrosEx
String expandProjectMacrosEx(project, conf, stringToExpand, debugMode)

Returns a string where the macros in `stringToExpand` are expanded in the context of the `conf` configuration of the project, in debug or production mode.

Example

```
project_dir = ide.expandProjectMacrosEx("${ProjectName}", "Default", "${ProjectDir}", False)
```

appendToMakefile
appendToMakefile(String t)

Appends strings to the generated Makefile.

Example

```
def on_project_build(e):
    ide.appendToMakefile("VERSION =" + str(version++))
```

setBuildVar
setBuildVar(o, key, val)

Creates an environment variable `key=val` visible to all processes invoked by the build process.

Example

```
def on_project_build(e):
    ide.setBuildVar(e,"THIS","ISaTest") #sets THIS=IsaTest in the build environment.
```

10.1.2.5.2 Callbacks into autoload.py

MPLAB X IDE will look up the following functions and call them at appropriate times during loading, building and debugging the project. This allows the user to customize many aspects of the process.

onShowPropsDFP
onShowPropsDFP(t)

Called to populate the tool properties dialog with custom project properties declared with `ide.addXXXXX` calls.

on_project_postbuild
on_project_postbuild(e)

Called after the project is built, during loading of the binary to internal MPLAB memory.

on_project_build
on_project_build(e)

Called before the project is built. This is a good place for calls to `ide.appendToMakefile` and `ide.setBuildVar`.

on_pre_program
on_pre_program()

Called before MPLAB programs the device.

Example 1

```
def on_pre_program():
  log.info("on_pre_program")
```

```
  labelAddress = deb.GetSymbolAddress("labelInSourceCode")
  deb.SetTempBP(labelAddress, "unused")

on_program_done()
```

called after the image is programmed

Example 2

```
def on_program_done():
    log.info("on_program_done")
```

on_pre_run
`on_pre_run()`

Called before MPLAB resumes execution after a halt.

Example

```
def on_pre_run():
    log.info("on_pre-run")
```

on_halted
`on_halted (pc)`

Called after execution just halted at PC = `pc`.

To tell MPLAB to resume execution without reflecting the halt in the UI, return `True` from this function. Otherwise, return `False`.

Example

```
def on_halted(pc):
    msg.print("Halted at " + hex(pc) + "\n")
    log.info("on_halted")
    return False # return True to continue running
```

on_session_ending
`on_session_ending()`

Called when MPLAB is about to stop session before connection to debug tool is terminated.

Example

```
def on_session_ending():
    log.info("on_session_ending")
```

on_session_ended
`on_session_ended()`

Called after MPLAB after the connection to debug tool is terminated

Example

```
def on_session_ended():
    log.info("on_session_ended")
```

onload
`onload (ide)`

Called when `autoload.py` is loaded for the first time (when the project is opened). It is typically used for adding menu commands to the project tree.

Example

```
def onload(ide):
    ide.addCommand("Test|msg|Print example", "print_hello")
```

10.1.2.5.3 Memory Objects

MPLAB declares one Python Memory object for each memory type:

`memPgm` , `memData`, `memRegs`, `memSFR`, `memUser`

These objects expose the following methods:

- **`Read8/Read16/Read32(address)`** - return a 1,2 or 4 byte little endian integer read at `address`
- **`Write8/Write16/Write32(address,val)`** - writes a 1, 2 or 4 bytes `val` in little endian to `address`
- **`Read( address, src, offset, count)`** - read `count` bytes from `offset` in `src` to `address`
- **`Write( address, dest, offset, count)`** - write `count` bytes from `address` into the byte array `dest` at offset `offset`

10.1.2.5.4 Debugger Object

Most functions can only be called when a debug session is in place and the device is halted.

Ready

```
boolean Ready()
```

Returns `True` if in ready state.

Connected

```
boolean Connected()
```

Returns `True` if `Connect` was successful. The IDE calls internally `Connect` when, for example, you press the refresh button in the Dashboard window.

After connection, depending on the device, the rest of the debug functions will be available.

For some devices the ready state needs to be arrived at by starting a debug session.

Connect

```
void Connect()
```

Does the equivalent of pressing the refresh button in the Dashboard window.

Disconnect

```
void Disconnect()
```

Terminates debug session and removes USB connection to debug tool.

GetPC

```
long GetPC()
```

If in the ready state, returns the current program counter.

IsHalted

```
boolean IsHalted()
```

Returns `True` if halted, `False` if running.

Run

```
void Run()
```

If in the ready state, start execution of program from current program counter.

Halt

```
void Halt()
```

If running, halt the program execution and enter the ready state.

StepIn

```
public void StepIn()
```

If in the ready state, step into the next line of source.

StepOverSourceLine

```
public void StepOverSourceLine()
```

If in the ready state, step over the next line of source. This might cause the debugger to set a breakpoint on the next line behind the scenes.

StepOut
`public void StepOut()`

If in the ready state, step out of the current C scope.

StepInstr
`public void StepInstr()`

If in the ready state, execute the opcode at program counter.

GetSymbolAddress
`long GetSymbolAddress(String symbol)`

This function requires that the image for a debug session has been loaded. This happens when you build for debug or when you press the debug-run button. Returns the value of the source level symbol.

GetSpecialSymbolAddress
`long GetSpecialSymbolAddress(String symbol)`

Returns the value of the special symbol after the image was loaded. The following special symbols are supported:

`__entry_address` - entry point to the executable as specified in the ELF header. Returns `0x7fffffffffffffffL` if no entry point was specified.

GetSymbolSize
`int GetSymbolSize(String symbol)`

This function requires that the image for a debug session has been loaded. This happens when you build for debug or when you click the Debug Project icon. Returns the size the source level symbol.

GetSpecialSymbolSize
`int GetSpecialSymbolSize(String symbol)`

Returns the byte size of the given special symbol.

UsingSWBP
`boolean UsingSWBP()`

Returns `True` if using software breakpoints.

UsingHWBP
`boolean UsingHWBP()`

Returns `True` if using hardware breakpoints.

UseSWBP
`void UseSWBP()`

Tell the IDE to use software breakpoints. All existing breakpoints will be converted to software breakpoints.

UseHWBP
`void UseHWBP()`

Tell the IDE to use hardware breakpoints. All existing breakpoints will be converted to hardware breakpoints. If the maximum number of hardware breakpoints is reached, the rest of the breakpoints will be disabled.

SetBP
`int SetBP(long address, String callback)`

Returns a handle to this breakpoint. The handle can be used to remove this breakpoint via `ClearBP()`.

If a breakpoint is available (if using hardware breakpoints), set a breakpoint at address. If callback is not `None`, then the function with the name callback will be called when this breakpoint is hit. If the callback function returns `False`, the debugger will halt and enter the ready state. If callback returns `True`, the debugger will continue execution as if

© 2022 Microchip Technology Inc. and its subsidiaries

this breakpoint had not happened. Setting a breakpoint with this function will create a breakpoint that will exist even after the debug session is expired.

Calling this function is the equivalent of setting a breakpoint using the IDE.

SetTempBP
`int SetTempBP(long address, String callback)`

Same as SetBP with this significant difference: the breakpoint will be automatically deleted with the debug session ends.

ClearBP
`void ClearBP(int handle)`

Takes the handle returned by `SetTempBP()` or `SetBP()` and clears that breakpoint.

GetNumMaxHWBP
`int GetNumMaxHWBP()`

Returns the maximum number of hardware breakpoints the device supports.

GetAddressesFromSourceLine
`PyList GetAddressesFromSourceLine(String fileName, long line)`

Returns a list of the addresses that contain code implementing the source line from `fileName` with line number `line`. It returns `None` if no matches are found. The reason this function returns a list is because the compiler may map a single sourceline into several addresses. This happens when using multiple statements on C line for example:

```
test();counter++;test2();
```

This also occurs when the compiler emits highly optimized code.

LoadImageFile
`void LoadImageFile(String fileName)`

Load an image (hex/elf) into the MPLAB X IDE memory objects. Overwrites existing image. Note that this does not mean the image will actually be programmed to the device memory.

This function is obsolete, so consider using `EnumCodeBlocks` instead.

EnumCodeBlocks
`boolean EnumCodeBlocks(String file, String callback)`

Calls the python function `callback` once for each code block in the binary file. A code block is a contiguous binary block that would be loaded to the device if `file` were to be debugged.

Example 1

```
def writeBlock(adr,bytes,name):
    log.info("Code block %s at %08x %d" % (name, adr,len(bytes)))
    dev.Write(adr,bytes,0,len(bytes)) # write block to device memory (ARM)

def load_file_and_pc(binary_path):
    deb.EnumCodeBlocks(binary_path, "writeBlock")

public String PeekFileType(String file)
```

Returns type of file as a string (either "hex", "coff", "ubrof8", or "nordic".)

Example 2

```
filetype = deb.PeekFileType(binary_path)
if "hex" == filetype or "elf" == filetype:
    deb.EnumCodeBlocks(binary_path, "writeBlock")
```

PeekSymbolAddress
`public long PeekSymbolAddress(String file, String sym, boolean specialSym)`

Returns the value of source level or special symbol `sym` , as loaded from the file executable. Note that the executable is not loaded into the debugger.

Example

```
entry = deb.PeekSymbolAddress(binary_path,"__entry_address",True)
```

10.1.2.5.5 The settings Object

The `settings` dictionary contains tool settings persisted in the project file `configurations.xml` . All properties are stored as ASCII strings and can be returned by a call to `getString`. The `getBool` and `getNumeric` functions convert the string to a Boolean or numeric value, respectively.

To save a property value (including declaring new props) , call `setString`.

getString

`String getString(key, defaultValueString)`

Returns the string value associated with `key`, or `defaultValueString` if the `key` does not exist.

getBool

`boolean getBool(key, defaultValueBool)`

Returns the boolean value (`True` or `False`) associated with `key`, or `defaultValueBool` if the `key` does not exist or the string was not "true" or "false".

getNumeric

`int getNumeric(key, defaultValueNum)`

Returns the numeric value associated with `key`, or `defaultValueNum` if the `key` does not exist or there was a conversion error.

setString

`void setString(key, valAsString)`

Assigns the string value associated with `key`, or `valAsString` if the `key` does not exist.

10.2 CI/CD Wizard

Note: This content is also in the “CICD Wizard in MPLAB X IDE User Guide” (DS-5000xxxx).

The *MPLAB X IDE CI/CD Wizard* helps you set up an automated build and test pipeline tailored to your MPLAB X IDE project.

CI/CD Requirements

The CI/CD Wizard is a feature of MPLAB X IDE, so the requirements found in “Release Notes for MPLAB X IDE” apply. In addition:

- An MPLAB XC **Workstation or Network Server license** is needed to enable the CI/CD Wizard for script generation.
- An MPLAB XC **Network Sever license** will be required to build your projects with PRO optimization levels in a generated Docker container.
- If you are using a **virtual machine network license**, ensure that this license and any Microchip **Network Server licenses** are placed in the same folder.
- If using the Jenkins Pipeline:
  - The MISRA Check (Static Code Analysis) and MPLAB Code Coverage (Hardware Testing) tools will require MPLAB X IDE v6.00 (or later) and an MPLAB Analysis Tool Suite **Network Server license** for tool execution in a generated Docker container.
  - Hardware Testing will require a tool with networking capabilities, which currently is only the MPLAB ICE 4 in-circuit emulator. This tool is supported in MPLAB X IDE v6.00 (or later).
- To use Jenkins and/or Docker, you will need to install these applications. For addition requirements when using Jenkins, see 10.2.3.1. Prerequisites.

Note: MPLAB X IDE CI/CD Wizard output execution relies on the third-party applications Jenkins and Docker. For issues with these tools, please see the corresponding website for support.

CI/CD Standard Use

The standard operating system used with Jenkins-Docker or Docker is Linux OS, and for the CI/CD Wizard output is specifically Debian Linux. To use other operating systems, please refer to the documentation for Jenkins and Docker. For example:

- JENKINS-60473: Docker pipeline: 'absolute path' error when running linux container under windows host

10.2.1 About CI/CD

CI/CD (or CICD) is a process for the continuous integration and continuous delivery or deployment of software. Instead of developing software modules fully and then trying to integrate them and deliver a product at the end (as has been done in the past), software is continually integrated and continuously delivered or deployed in small increments, ensuring that a product can be produced with minimal issues at any point.

Continuous integration moves the integration and testing activity early into the development cycle. This helps you reduce the project risks and increasing the likelihood of a successful product.

To support the CI/CD pipeline, containers can be used to build and test code using a consistent environment that the team can share. Developing the instruction files to create and run a container (e.g., Dockerfile) and run container contents (e.g., Jenkinsfile) is non-trivial. This is where the CI/CD Wizard can help with development.

10.2.1.1 Why Continuous Integration with build and test automation?

Continuous Integration (CI) is a developer practice of frequently committing your code changes to a source code repository, e.g., git. Each commit triggers an automated build and test cycle. If the build breaks or a test fails, you are notified. Doing Continuous Integration helps build quality into the product from the start, coordinates code changes with your team, and reduces the risks associated with late integration and late validation feedback.

The Continuous Integration developer practice works together with a **build automation** system. This system automatically starts jobs/processes like compile, test, package, and deploy. This helps you keep the code quality high by uncovering issues early. There are many different build automation tools available. Jenkins is a free and open-source software. It is a popular tool that we will use as an example. If you are using a different system, look at the Jenkinsfiles generated by the CI/CD Wizard for inspiration.

10.2.1.2 Docker and Containers

A key concept in build automation is **containerization**. When executing a build job, it should produce repeatable and consistent results independent of where and when the build was executed. **Docker containers** can help achieve this. Use of Docker containers enables repeatability, flexibility, and scalability. Every time a job is executed it has the same environment. Docker provides OS-level virtualization which enables you to separate your applications from your infrastructure.

For more information about how to install Docker and use Dockerfiles, see www.docker.com

10.2.2 CI/CD Wizard Feature Overview

1. Create tailor-made **Dockerfiles**. These files are recipes for generating Docker images. Docker images are used to create Docker containers. Docker containers contain all your MPLAB X IDE project settings needed to build, i.e., the version of MPLAB X IDE, MPLAB XC toolchain, XC license handling, Device File Pack (DFP) and tool pack.
2. Create **Jenkinsfiles**. These files are used to set up a Jenkins pipeline that describes how to build and test your MPLAB X IDE project on a Jenkins Build server using Docker containers.
3. Handle MPLAB XC PRO compiler licenses in a build server setting.
4. Create and launch a Docker container using a **Jenkins server**. This enables you to quickly test out build pipelines or serves as a base for a locally-hosted Jenkins server.
5. Set up the automated build pipeline to include steps like:
   - MISRA checking
   - Executing tests with the simulator
   - Executing tests on hardware
   - Collecting code coverage data

© 2022 Microchip Technology Inc. and its subsidiaries

– Generating documentation with Doxygen

10.2.3 Getting Started with Jenkins

Jenkins is an open source automation server which enables developers to automate build, test, and deploy their software. For more information go to Jenkins official site.

This section describes the steps needed to build an MPLAB X IDE project on a Jenkins server.

10.2.3.1 Prerequisites

10.2.3.1.1 Jenkins Server

- A Jenkins server needs to be available. If you do not have access to one you can set one up yourself by following the Installing Jenkins guide, or you can use the `cicdw jenkins-server` command to create a local test server (see 10.2.4. Getting Started with Jenkins Server).
- You need to have a user account that allows you to configure a new build job on the Jenkins server.
- The Jenkins server needs to have build nodes that supports Docker. If not see Jenkins and Docker.
- The Jenkins server must be setup with the following plugins:
  – Pipeline Aggregator View
  – Pipeline Utility Steps
  – Docker Pipeline
  – Workspace Cleanup
  – Warnings Next Generation

  Note: Additional plugins may be required for optional build steps, such as MISRA and Simulator.

10.2.3.1.2 Source Control System

- Your project source code needs to be hosted on a source control system (example: Subversion, CVS, or Git).
- The Jenkins server needs to support your source control system and have access to your project repository.

10.2.3.1.3 License Server

- An MPLAB XC license server needs to be configured and available on a given address and port. For more information see MPLAB XC License Server Manual
- Other license servers may be necessary to support other licensed options, such as the MPLAB Analysis Tool Suite (MISRA Check and MPLAB Code Coverage).

10.2.3.1.4 Testing Tools

In order to use either software (Simulator) or hardware (MPLAB ICE 4 in-circuit emulator) tools for testing within a container, you must consider your setup.

- Use Unity to perform unit tests and output results with `printf` or `putchar` using serial communications. For hardware tools, additional code is required at the target. To learn more about Unity, see 10.2.5. How to Write and Run Unit Tests with Unity. For information about the virtual COM port on the MPLAB ICE 4 in-circuit emulator, see the user's guide section "USB CDC Virtual COM Port."
- Use the Microchip Debugger (MDB) command-line interface to support basic debug commands. For hardware tools, use MDB to program the target. To learn more about MDB, see Microchip Debugger (MDB) User's Guide.

10.2.3.2 Step 1 - Generate Jenkinsfiles from MPLAB X IDE

1. Open your project in MPLAB X IDE and navigate to *Tools>CI/CD Wizard*.
2. Select **Jenkins Pipeline files** as output type.
3. Step through the wizard and select the tools you would like to run in the pipeline.
4. After generation, verify that a *Jenkinsfile* and *Dockerfile* have been added to your project base folder. Other files are may also be added depending on the selected options.

10.2.3.2.1 CI/CD Wizard Dialog 1

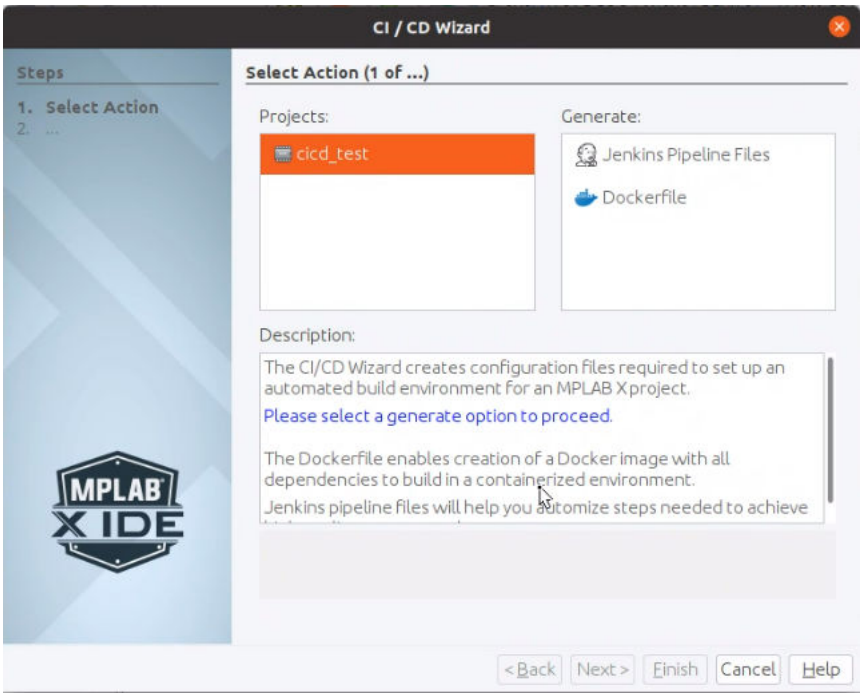
The first dialog of the wizard will display this note, and only the **Cancel** button will be active if you do not have an MPLAB XC C compiler license.

To generate files for use in a CICD pipeline, you need a licensed compiler.

See 10.2. CI/CD Wizard for supported licenses. You can purchase a license for your compiler under MySoftware. Click the **Configure Licenses** button to go to the Change Licensing Type dialog to set up your license.

If you have a compiler license, you will be able to select what file(s) you want to generate. Select Jenkins to create a Jenkinsfile and Dockerfile. Select Docker to create only a Dockerfile.

Figure 10-2. No Generate Selection



For this example, select “Jenkins Pipeline Files.” Read the text about this selection that now appears under “Description.”

Click **Next** to proceed through the dialogs.

Figure 10-3. Generate Selection

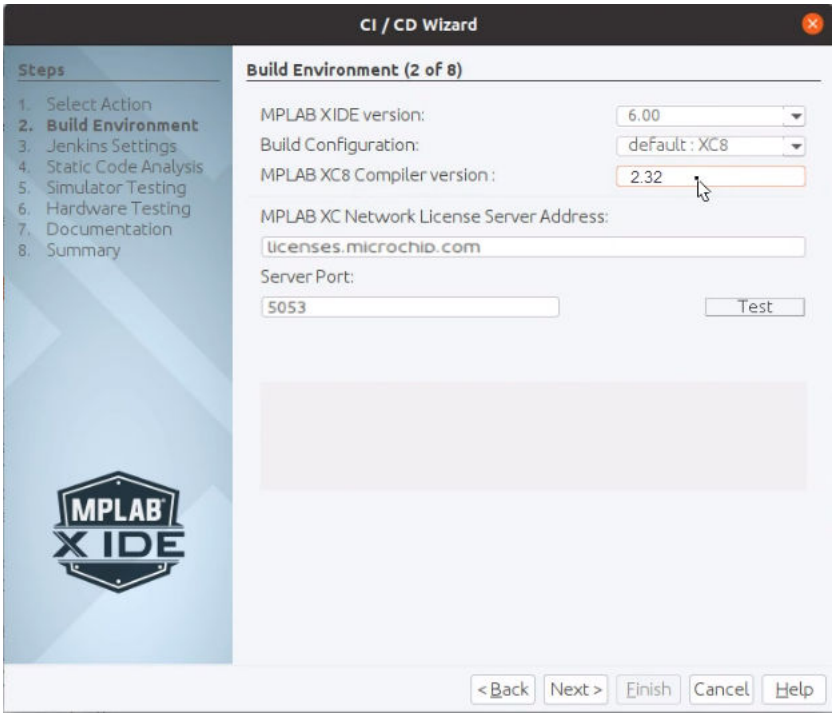


10.2.3.2.2 CI/CD Wizard Dialog 2

Set up the build environment to be used in a container. If you have a Network Server license, you can provide information on how to access it for the build. You can even test the connection (see figure below).

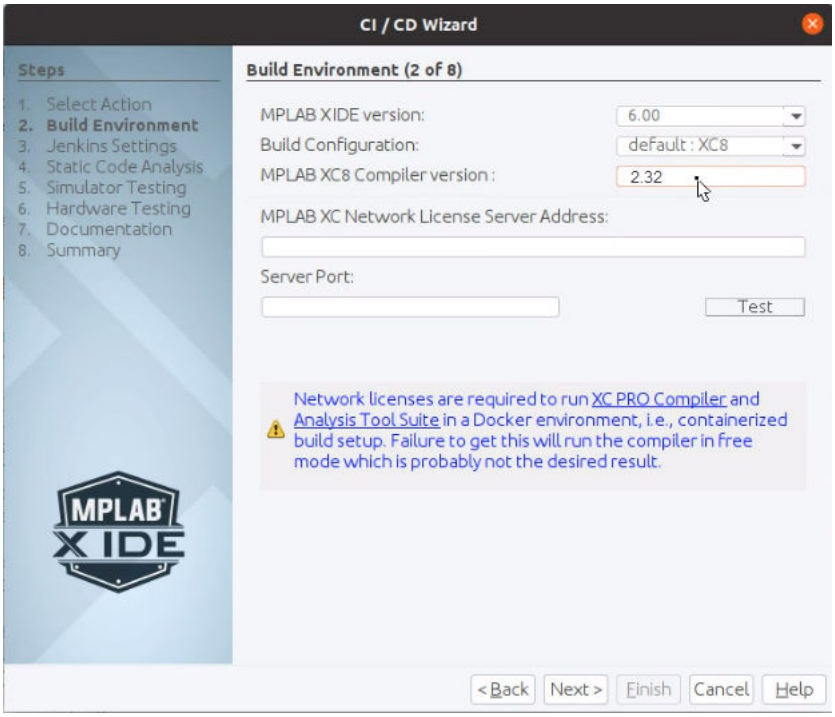
Note: If you have already set up the Network Server license using MPLAB X IDE *Tools>Licenses*, that information will auto-populate in this dialog.

Figure 10-4. Network Server License



Otherwise leave the text boxes blank (though a note will appear).

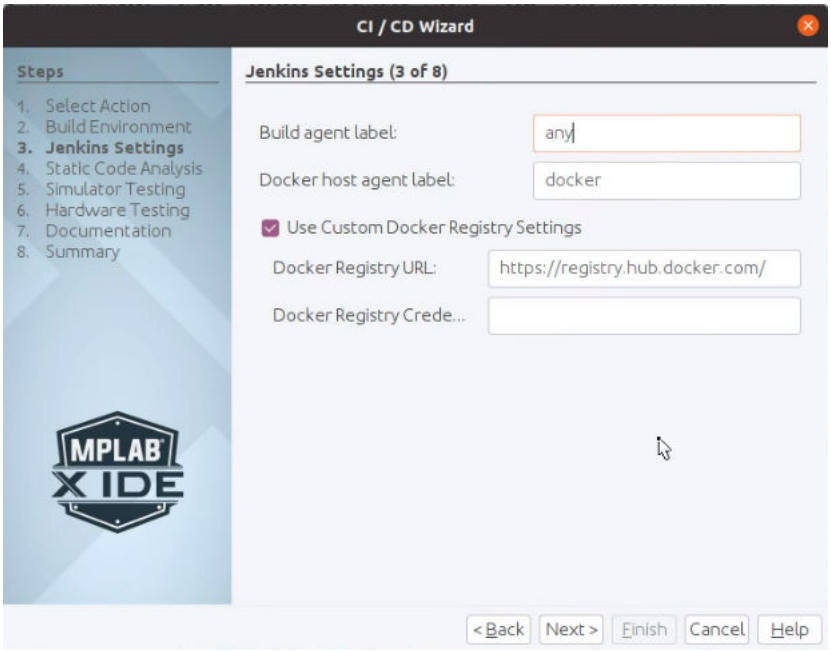
Figure 10-5. Workstation License



Option	Description
MPLAB IDE version	Select supported versions from the drop-down list. Default is the latest version.
Build Configuration	Select a project configuration for the build.
MPLAB XC Compiler version	Select a compiler version for the build.
MPLAB XC Network License Server Address	Enter the address of the server. For example: licenses.microchip.com.
Server Port	Enter the port number of the server. For example: 5053
Informational Note (conditional)	If you have a Workstation license, information on buying a Network Server license will be shown.

10.2.3.2.3 CI/CD Wizard Dialog 3

Determine the location from which Docker will download the image. Default Docker information is entered at the top. To use a custom docker registry, enter information below.



Option	Description
Build agent label	The default is set to “any,” but you may add a specific label. Please ensure your Jenkins agents are labeled accordingly in the Configure System settings on the Jenkins Server.
Docker host agent label	The Docker host agent label is the label that is placed on a Jenkins agent that can support the building and running of Docker containers. Please keep in mind there are standard naming syntax that may need to be followed. Consult the Jenkins documentation for these naming conventions.
Use Custom Docker Registry Settings	Use your own docker registry for the base image to download. For more on this, see How to use your own Registry. The base image will be downloaded from Docker Hub. You can specify a path for a customized base image if needed.
Docker Registry URL	Your registry location.
Docker Registry Credentials ID	Your credentials to push and pull. For details see Using Jenkins - Credentials.

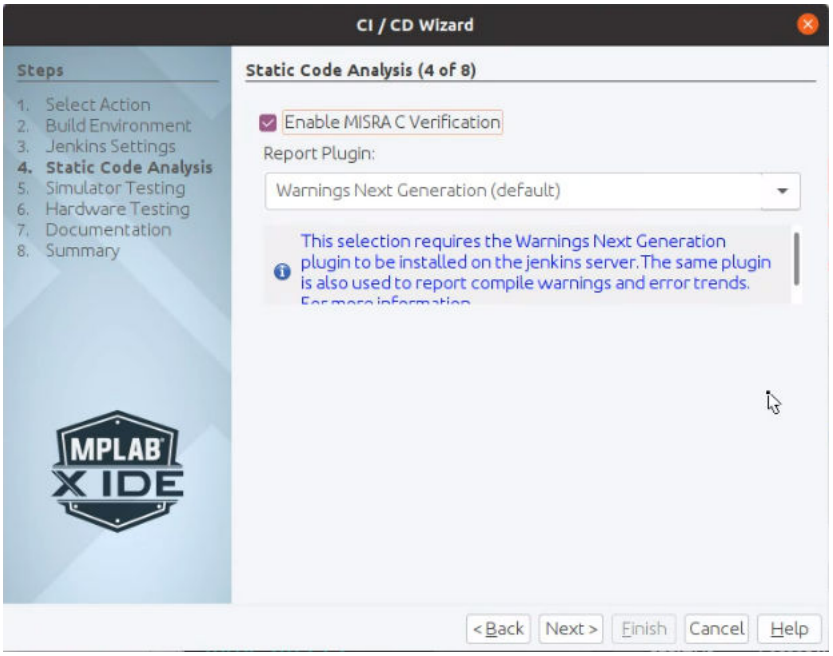
10.2.3.2.4 CI/CD Wizard Dialog 4

Check to enable MISRA C verification. For more on MISRA, go to www.misra.org.uk/.

Requirements for using this features are:

- A Jenkins plugin is required for either report selection. An informational note will provide a link.
- MISRA C verification is only available for MPLAB X IDE v6.00 and later. An informational note will be displayed if the version does not support this feature.
- An MPLAB Analysis Tool Suite Network Server license is required for tool execution in a generated Docker container.

Figure 10-6. Warnings Next Generation



Option	Description
Enable MISRA-C Verification	Check to enable MISRA verification.
Report Plugin	Select from: • Warnings Next Generation • Cppcheck

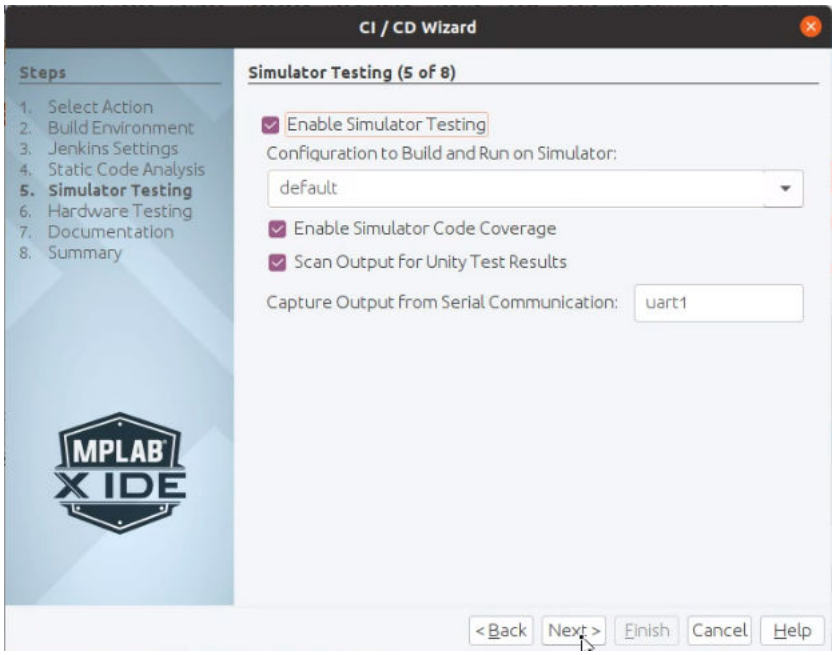
10.2.3.2.5 CI/CD Wizard Dialog 5

Check to enable the simulator for use as a test image and specify other options. This can be used to run unit tests.

A Microchip Debugger (MDB) script *mdb-simulator-script.txt* will be added to your project and this is executed during the simulator build step.

The default script will run the simulator for 10 seconds and capture output and coverage based on your settings. To change this, modify the script manually or regenerate using other options. Customize the input file as needed to fit your testing needs. Refer to the Microchip Debugger (MDB) User's Guide.

© 2022 Microchip Technology Inc. and its subsidiaries



Option	Description
Enable Simulator	Enable the simulator as the debug tool.
Configuration to Build and Run on Simulator	Select a project configuration that uses the simulator; either default or one dedicated to simulator use.
Enable Simulator Code Coverage	Enable Code Coverage feature built into the Simulator. **Note:** The Code Coverage API plugin must be available on your Jenkins server if coverage reporting is enabled.
Scan Output for Unity Test Results	Enable if the configuration builds Unity test runners, and the build job should create a report based on the resulting output. For more info on how to write Unity tests see 10.2.5. How to Write and Run Unit Tests with Unity.
Capture Output from Serial Communication	This option should be used together with the *Scan output for Unity test results* option. The simulator needs to know from which serial communication module instance to capture the output from. The default value is *uart1*, but this is dependent on which device you are using and how you have configured your test setup.

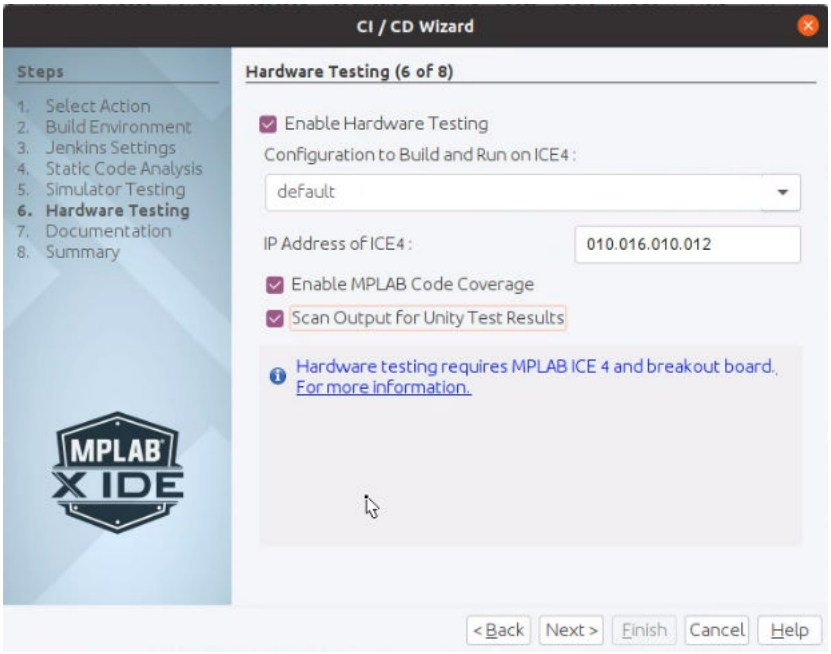
10.2.3.2.6 CI/CD Wizard Dialog 6

Check to enable a hardware tool for use as a test image and specify other options. This can be used to run unit tests.

Note: The only hardware tool available at this time with networking capabilities is the MPLAB ICE 4 in-circuit emulator. MPLAB ICE 4 is only supported by MPLAB X IDE v6.00 (or later.)

A Microchip Debugger (MDB) script *mdb-hardware-script.txt* will be added to your project and this is executed during the hardware build step.

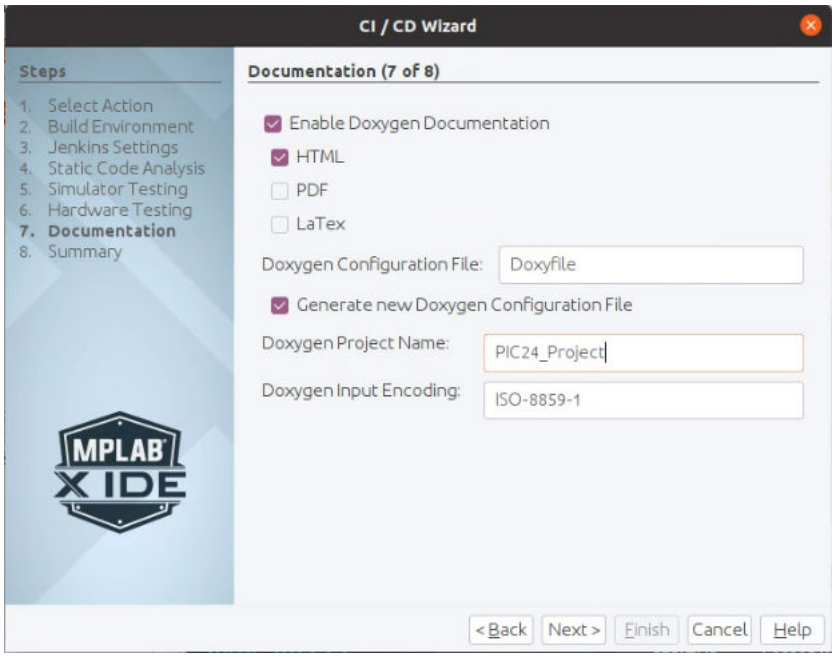
The default script will run for 10 seconds and capture output and coverage based on your settings. To change this, modify the script manually or regenerate using other options. Customize the input file as needed to fit your testing needs. Refer to the Microchip Debugger (MDB) User's Guide.



Option	Description
Enable Hardware Testing	Enable the emulator as a test tool.
Configuration to Build and Run on ICE4	Select a project configuration that uses the emulator; either default or one dedicated to hardware use.
IP Address of ICE4	Enter the IP address of the emulator you wish to use. If you do not know the IP address of the emulator, you can open *Tools>Manage Network Tools* and scan for network tools. This will list the emulator with its IP address.
Enable MPLAB Code Coverage	Enable the MPLAB Code Coverage feature. An MPLAB Analysis Tool Suite Network Server license is required for tool execution in a generated Docker container. **Note:** The Code Coverage API plugin must be available on your Jenkins server if coverage reporting is enabled.
Scan Output for Unity Test Results	Enable if the configuration builds Unity test runners, and the build job should create a report based on the resulting output. For more info on how to write Unity tests see 10.2.5. How to Write and Run Unit Tests with Unity.

10.2.3.2.7 CI/CD Wizard Dialog 7

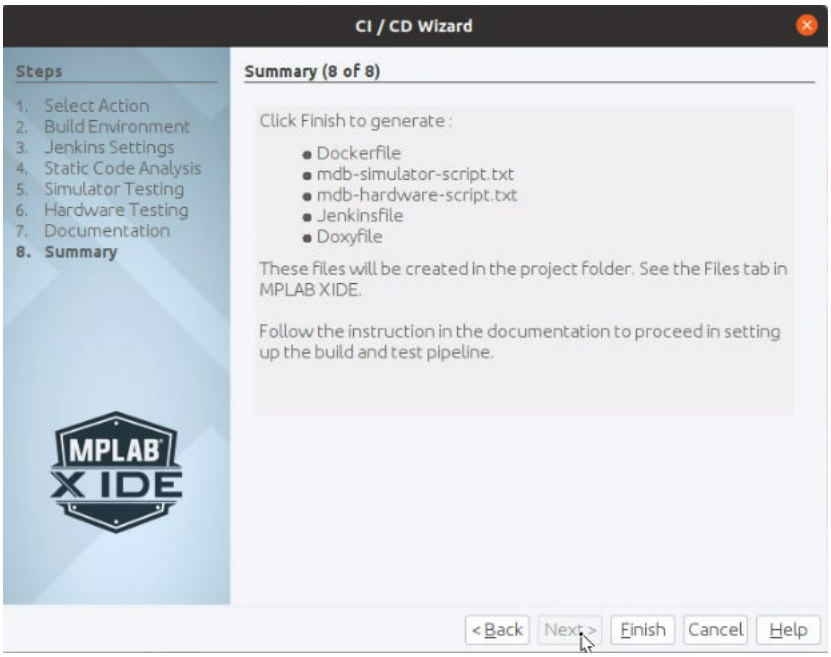
Check to enable Doxygen as a documentation tool and specify the output. For more info on Doxygen see Doxygen on GitHub.



Option	Description
Enable Doxygen Documentation	Enable documentation tool.
Documentation Format	Select one or several output formats.
Doxygen Configuration File	Specify configuration file.
Generate new Doxygen Configuration File	Check to create or overwrite an existing file. Uncheck to keep the existing file.
Doxygen Project Name	Specify project name.
Doxygen Input Encoding	Specify document encoding.

10.2.3.2.8 CICD Wizard Dialog 8

Review the expected output in the Summary screen. To make changes click **Back**.



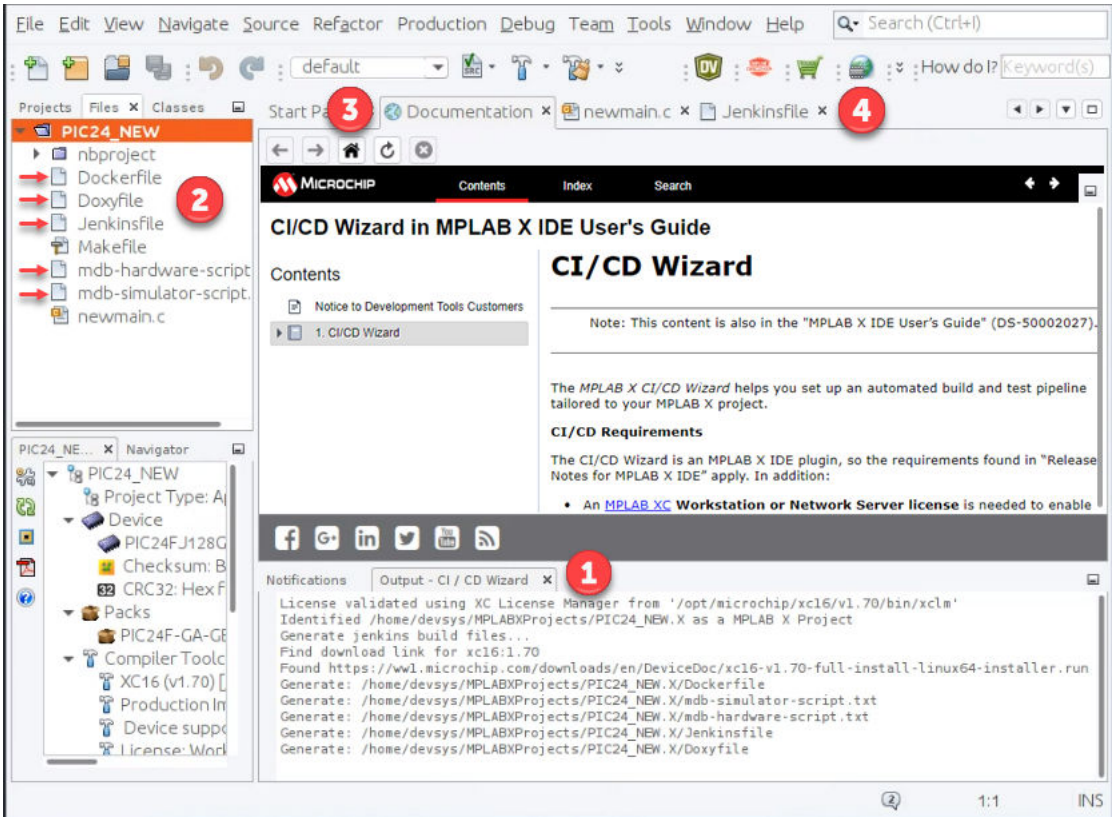
Click **Finish** to complete the setup. CI/CD Wizard will then generate files based on the setup information.

10.2.3.2.9 CI/CD Wizard Output

In MPLAB X IDE, you can see:

1. Files that are being generated in the Output window.
2. Resulting files added to the project folder.
3. WebHelp in its own window.
4. Jenkinsfile in its own window.

Figure 10-7. File Generation in MPLAB X IDE



Details of files generated:

Jenkinsfile

A text file that contains the definition of a Jenkins Pipeline. For more information, see Using a Jenkinsfile.

Dockerfile

A customized text file used to build the container. For more information, see Dockerfile reference.

mdb-simulator-script.txt

This file will be produced only if the simulator was selected in Step 5 of the assistant. It is a file to setup simulator usage in the Microchip Debugger (MDB), the GUI-less form of MPLAB X IDE. You can customize this file to your testing needs. For more information see the Microchip Debugger (MDB) User's Guide.

mdb-hardware-script.txt

This file will be produced only if the MPLAB ICE 4 in-circuit emulator was selected in Step 6 of the assistant. It is a file to setup emulator usage in the Microchip Debugger (MDB), the GUI-less form of MPLAB X IDE. You can customize this file to your testing needs. For more information see the Microchip Debugger (MDB) User's Guide.

Doxygen

This file will be produced only if documentation was selected in Step 7 of the assistant.

Doxygen is a standard tool for generating documentation from annotated source code. For details on the configuration file format, see Configuration.

10.2.3.3 Step 2 - Add Jenkinsfiles to Your Repository

Use your source control system and add the generated files to your project repository. Jenkins will add the project in this repository to the created container.

10.2.3.4 Step 3 - Configure Build Job on the Jenkins Server

This section gives a short summary of how to configure a new build job on a Jenkins server using the generated files. For more details see Using a Jenkinsfile on Jenkins.io.

Short summary:

1. Log in to the Jenkins server web interface and select **New Item**.
2. Enter a build job name and select **Pipeline** as project type.
3. In the new build job configuration, go to the **Pipeline** section and configure:
   a. Select **Pipeline script from SCM** and configure the Source Control Management (SCM) info required for the Jenkins server to check out your project. This may include *repository url*, *credentials* and also *branch specifier*.
   b. Specify *Jenkinsfile* as the **Script Path**. (This needs to match the name of the generated Jenkinsfile).
   c. Save the new build job configuration and start a new build job (**Build Now**).

The result should be a working build pipeline that first sets up a docker build environment before building and analyzing the MPLAB X IDE project.

10.2.3.5 Suggested Next Steps

The *Jenkinsfile* defines build steps that are run inside a docker image. The docker image is defined by the generated *Dockerfile* and provides a stable Linux environment where the required MPLAB XC toolchain, device support and MPLAB X IDE binaries are available.

You can modify the generated Jenkinsfile by following the Jenkinsfile documentation.

10.2.3.5.1 Gating

Once you have a stable build job you can implement a gated commit process where each commit is automatically tested before it is rejected or merged to your main branch. Such a process makes sure that the main branch remains stable even when there are many developers working on the same project.

Setting up a gated build process can be done in different ways depending on which source code management infrastructure you are using. Here is an example on how to use the Atlassian Bitbucket integration.

10.2.4 Getting Started with Jenkins Server

Jenkins is an open source automation server which enables developers to automate build, test, and deploy their software. For more information go to Jenkins official site.

The CI/CD Wizard offers a solution to run an automatically configured Jenkins server instance in a Docker container. The Jenkins server can be configured to build an existing MPLAB X IDE project from a local folder or from git. The server can be a starting point for further configuration and customization, or as a local test bed for the build pipelines generated by the wizard.

10.2.4.1 Prerequisites

Docker

Docker is a set of platform as a service products that use OS-level virtualization to deliver software in packages called containers. Containers are isolated from one another and bundle their own software, libraries and configuration files.

- Read more on docker.com
- Get started with Docker

10.2.4.2 Create and Start a Jenkins Server

To create and start a Jenkins server, `cicdw`, the command line interface for the CI/CD Wizard, will be used. For more information on this tool, see 10.2.7. CICDW Command Line Tool.

10.2.4.2.1 Create a “Plain” Jenkins Server

To create a plain Jenkins server without any preconfigured build jobs, type

```
cicdw jenkins-server create
```

10.2.4.2.2 Create a Jenkins Server with a Git-Based Build Job

To create a Jenkins server preconfigured with a build job to build against your git repository:

```
cicdw jenkins-server create --with-git-job origin
```

This will extract the git server information associated with the git remote named "origin" for your local git repository. For this to work, the project path (default is the current folder) must point to a valid MPLAB X IDE folder that contains a Jenkinsfile and is checked into a remote git repository.

Note: If you don't already have a Jenkinsfile associated with your project, use the CI/CD Wizard to generate the files for you. See 10.2.3. Getting Started with Jenkins.

10.2.4.2.3 Create a Jenkins Server with a Local Build Job

To create a Jenkins server preconfigured with a build job to build against your local folder:

```
cicdw jenkins-server create --with-local-job
```

When triggered, the build job will look for a Jenkinsfile on a local folder on your computer. Use the [`path`] setting when starting the server to provide the local folder path (this is not a production server mode).

Note: If you don't already have a Jenkinsfile associated with your project, use the CI/CD Wizard to generate the files for you. See 10.2.3. Getting Started with Jenkins.

10.2.4.2.4 Start the Jenkins Server

Once a Jenkins server is created, it can be started by entering the following command:

```
cicdw jenkins-server start
```

Note that a server configured with a local build job will use the project path (default is the current folder) given to the start command as its local folder.

Once started, the Jenkins server can be accessed through a browser.

The default location is http://localhost:8080/ - this can be changed using the `--server-name` and the `--server-port` parameters.

A default user is created with the credentials **mplabcicd/mplabcicd** - this can be changed using the `--admin-user` and the `--admin-password` parameters.

10.2.4.3 Credentials

When interacting with external resources like a git repository, various credentials will often be required to authenticate and authorize the build system. Jenkins credentials are handled by the credentials manager, and can be provided when a new server is created.

```
cicdw jenkins-server create --credentials [TYPE::]ID::USERNAME[::PASSWORD][::KEYFILE] ...
```

There are two types of credentials, `usr_pwd` or `sshkey`.

`usr_pwd` is the default, so the parameter `--credentials my_id::my_username::my_password` will be interpreted as `TYPE=usr_pwd`, `ID=my_id`, `USERNAME=my_username`, `PASSWORD=my_password`.

For the type `sshkey`, the fourth element will be interpreted as the path to a keyfile if only four elements are provided. With five elements, the fourth will be interpreted as a password for the keyfile, and the fifth as the path to the keyfile.

One or more credential parameters can be listed after the `--credentials` flag.

10.2.4.3.1 Examples

```
cicdw jenkins-server create --credentials httpuser::theuser::secret123
```

or

```
cicdw jenkins-server create --credentials usr_pwd::httpuser::theuser::secret123
```

will create a username/password credential with the username `theuser` and the password `secret123` as the credential id `httpuser`.

```
cicdw jenkins-server create --credentials sshkey::sshuser::theuser::path/to/.ssh/key
```

will create a ssh credential with the username `theuser` and the keyfile `key` from the folder `path/to/.ssh/` as the credential id `sshuser`.

```
cicdw jenkins-server create --credentials
sshkey::sshuser::theuser::keyfilepwd::path/to/.ssh/key
```

will be the same as the previous example, but will now use the passphrase `keyfilepwd` to unlock the keyfile.

10.2.4.3.2 Use Credentials with a Build Job

A git based build job will often require credentials. To use the credentials created by the `--credentials` parameter, add the `--git-job-credentials-id` parameter. `--gjci` for short.

```
cicdw jenkins-server create --credentials cred_id::user::pwd --git-job-credentials-id cred_id
```

Note that username/password authentication will not work for any ssh based remote url.

- ssh://git@some-git-server.com/projects/project.git
- ssh://user@some-git-server.com/~/projects/project.git
- git://some-git-server.com/projects/project.git
- git@some-git-server.com:projects/project.git

10.2.4.4 Edit the Jenkins Server Build Files

By default, the `cicdw jenkins-server create` command will write its configuration files to a temporary folder. By including the `--build-path` parameter, this can be changed to a more permanent location. Add the `--only-generate` flag to generate build files without creating a docker image.

```
cicdw jenkins-server create --only-generate --build-path my/build/files/location
```

This will generate three files inside that folder.

- Dockerfile - An instruction on how to build the docker image for the Jenkins server
- plugins.txt - A list of plugins to preinstall in the Jenkins server
- casc.yaml - Configuration instructions for the Jenkins server

Refer to the official JCasC documentation for information on how to edit these files.

To use the edited files, add the `--from-build-files` flag to the `cicdw jenkins-server create` command. Note that you will have to remove any previously created server before you can create a new one.

```
cicdw jenkins-server remove
cicdw jenkins-server create --build-path my/build/files/location --from-build-files
```

10.2.5 How to Write and Run Unit Tests with Unity

Unity is an MIT Licensed framework that makes it easy to write unit tests. The official getting started guide is available here: Unity - Getting Started.

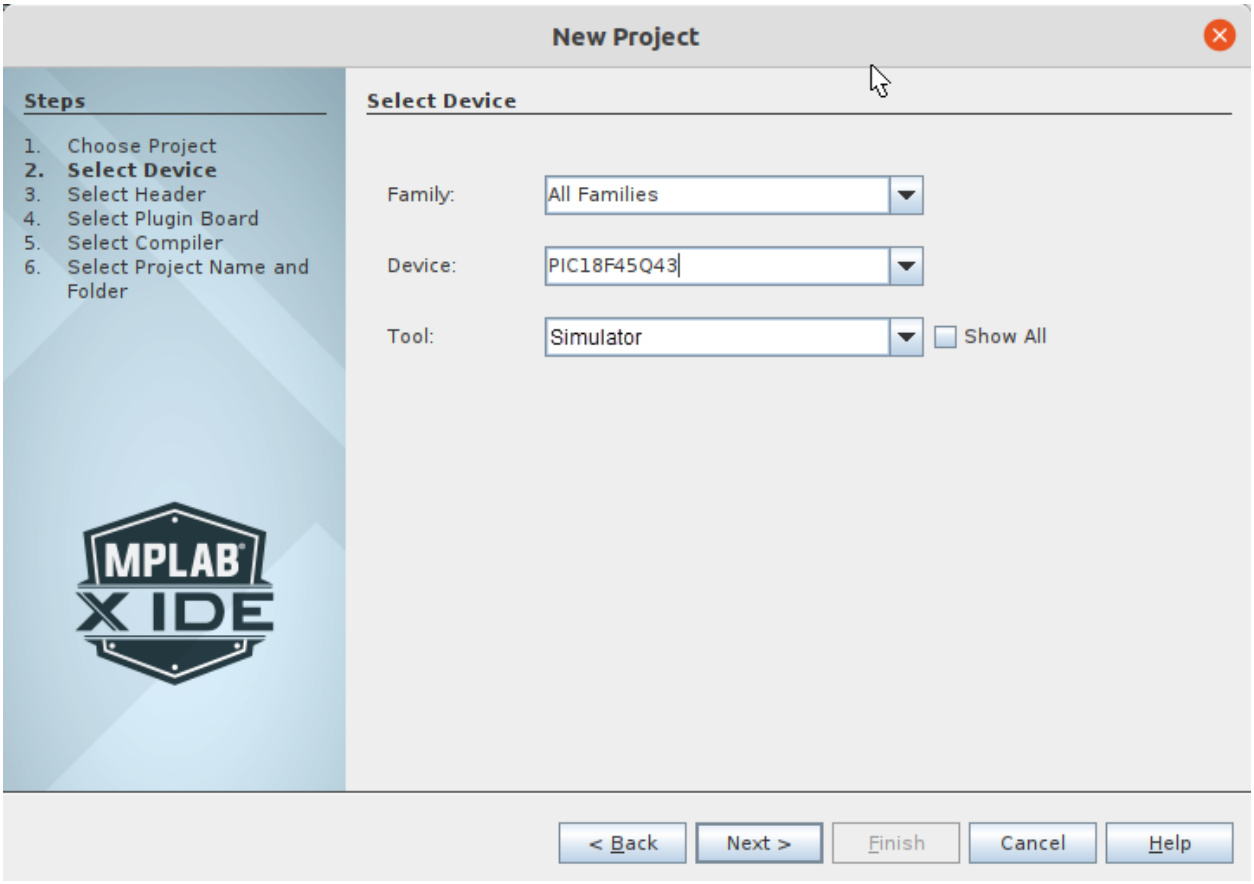
Examples of how to use this product in the MPLAB X IDE context are shown in the following sections.

10.2.5.1 Running Unit Tests on the Simulator

Follow the steps below for an example of how to use Unity with the MPLAB X Simulator.

10.2.5.1.1 Step 1. Create New Project

Create a new Standalone Project in MPLAB X IDE  , which includes selecting the device and toolchain.

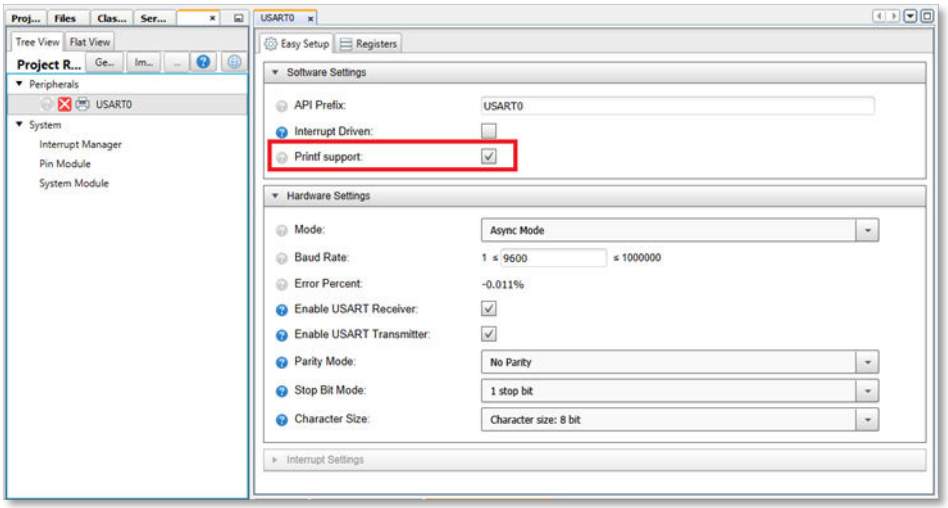


10.2.5.1.2 Step 2. Configure Serial Output

Unity requires support for *printf* to output test results. This can be done in several ways.

One approach is to install and use the MPLAB X IDE plugin *MPLAB Code Configurator* (MCC). Once installed, open MCC and add an USART (or UART) from the *Device Resources* view. Make sure to enable *Printf* support as part of the configuration.

The following example shows a configured USART for the *ATtiny817* device with *Printf* in MCC:



When you have configured the USART (or UART) in MCC remember to click “Generate” and verify that the configured files are added to your project.

10.2.5.1.3 Step 3. Verify Serial Output with MDB Script and the Simulator

You should verify that your serial output and `printf` works as expected before continuing with Unity.

Step 3.A – Test printf from main.c

The setup will differ from selected device and how `printf` was setup.

The following `main.c` example assumes that the device is ATtiny817 and that a configured USART is generated by MCC in the `mcc_generated_files` sub folder:

```
#define F_CPU 1000000UL

#include "mcc_generated_files/mcc.h"
#include <util/delay.h>

int main(void)
{
    // Initialize drivers from MCC
    SYSTEM_Initialize();
    _delay_ms(1000);

    printf("Hello world!\n");
}
```

Step 3.B – Create Simulator MDB Script

Create an MDB script that runs your program locally on the simulator and logs your serial communication to a local file.

An example of `mdb.script` using ATTiny817 is:

Note: You will need to modify the path to your elf file.

```
# Set tool and device
device ATtiny817
hwtool SIM

# Write serial communication from usart0 to file.
set usart0io.uartioenabled true
set usart0io.output file
set usart0io.outputfile ./sercom_output.txt

# Program and run your binary file
program ./dist/[INSERT_YOUR_BINARY_FILE_PATH_HERE].elf

run
# Wait 10 seconds before exiting
wait 10000
halt
quit
```

Step 3.C – Run MDB Script and Verify Output

Make sure that your project is built, then open a command line and navigate to your project. Run `mdb.sh` or `mdb.bat` with your mdb script as argument.

Here is an example:

```
me@mypc:/project_with_unittests.X$ "/opt/microchip/mplabx/6.00/mplab_platform/bin/mdb.bat"
mdb.sh

device ATtiny817

hwtool SIM

Resetting SFRs
Resetting peripherals

set usart0io.uartioenabled true
set usart0io.output file
set usart0io.outputfile ./sercom_output.txt
```

```
program /project_with_unittests.X/dist/default/production/
project_with_unittests.X.production.elf
Programming target...
Resetting SFRs
Resetting peripherals
Program succeeded.

run
Running

wait 10000
halt
Simulator halted
quit
```

If you now open `sercom_output.txt` you should get the output:

```
Hello world!
```

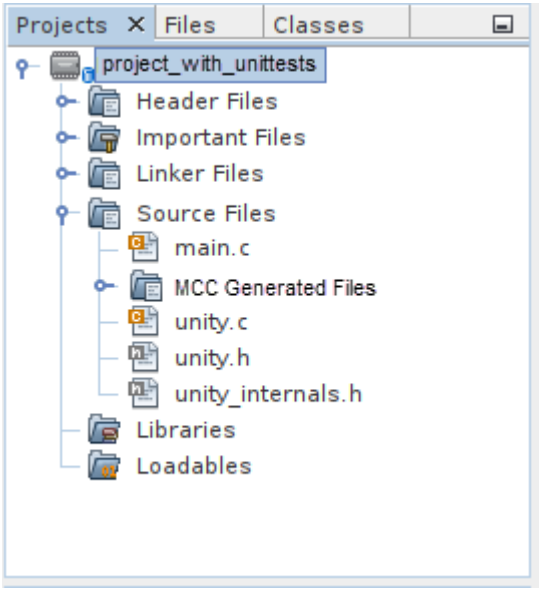
10.2.5.1.4 Step 4. Add Unity Test Framework

Now that you have a working serial output, you can add unit tests.

You need to manually download and add the Unity test framework to your project. The files you need are available from https://github.com/ThrowTheSwitch/Unity/tree/master/src.

Download them and add them to your project.

Your project should now look something like this:



10.2.5.1.5 Step 5. Write a Testrunner

Rewrite your `main.c` and add tests to it. For information on how to write more advanced tests see: https://github.com/ThrowTheSwitch/Unity/blob/master/docs/UnityGettingStartedGuide.md.

For example:

```
#define F_CPU 1000000UL

#include "mcc_generated_files/mcc.h"
#include <util/delay.h>

#include "unity.h"

void test_function1() {
    // Simple demo of working test
    TEST_ASSERT_TRUE(1);
```

```
}

void test_function2() {
    // Simple demo of failing test
    TEST_ASSERT_FALSE(1);
}

int run_unit_tests(void)
{
    UnityBegin("main.c");
    RUN_TEST(test_function1);
    RUN_TEST(test_function2);
    UnityEnd();
    return 0;
}

int main(void)
{
    // Initialize drivers from MCC
    SYSTEM_Initialize();
    _delay_ms(1000);

    run_unit_tests();
}

void setUp (void) {}
void tearDown (void) {}
```

10.2.5.1.6 Step 6. Run Tests on the Simulator

Rerun the `mdb` script that you have created (as described in Step 3.C).

This time the output should be:

```
main.c:21:test_function1: PASS
main.c:15:test_function2: FAIL: Expected FALSE Was TRUE

-----------------------
2  Tests 1  Failures 0 Ignored
 FAIL
```

The suggested `mdb` script only waits for 10 seconds before exiting. If your tests require more time, you can increase this in the `wait` command in `mdb.script`.

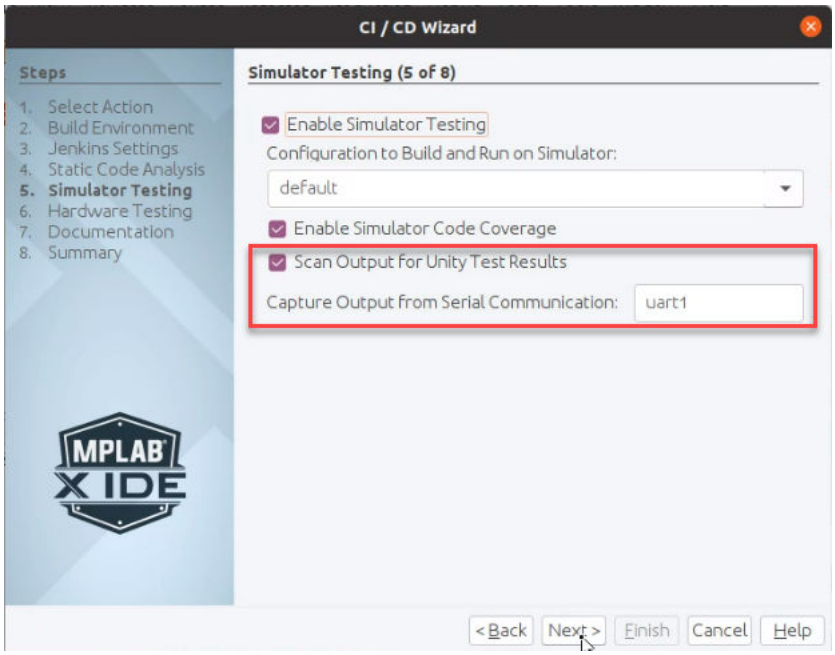
10.2.5.1.7 Step 7. Use the Simulator with CI/CD Wizard

When you have a stable setup that works locally, run the CI/CD Wizard and integrate your tests.

Enable the simulator test step in the CI/CD Wizard when generating Jenkinsfiles.

Enable “Scan Output for Unity Test Results” and fill in the name of the serial communication instance to capture output from:

© 2022 Microchip Technology Inc. and its subsidiaries

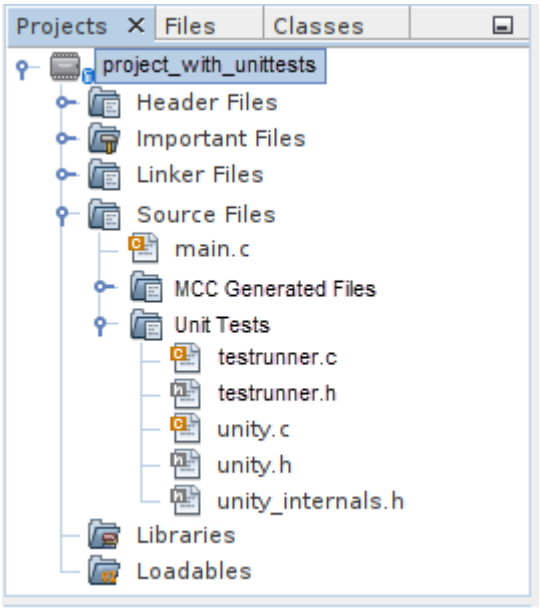


The wizard will generate its own `mdb` script and use this during Jeknins builds. Test results will be parsed from the output will be presented by Jenkins after builds.

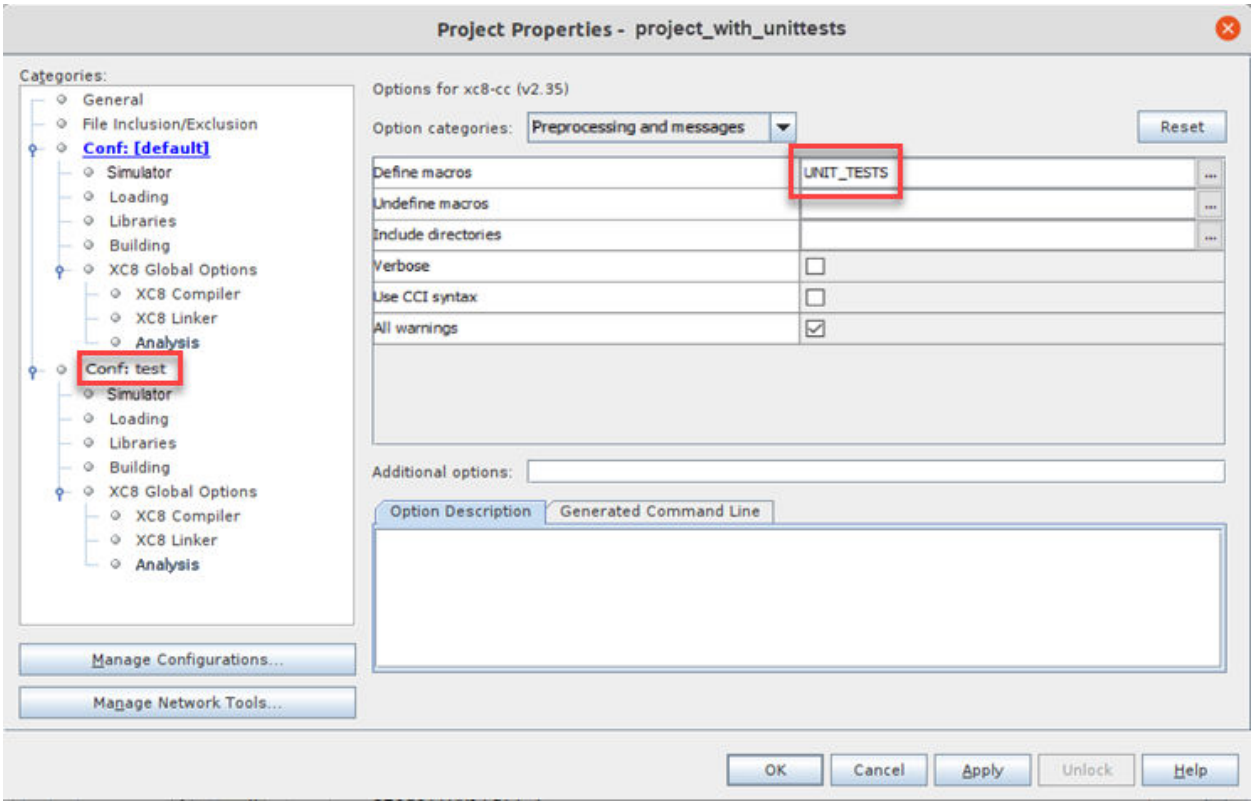
10.2.5.1.8 Optional Steps

Separate Test and Production Code

To better organize your project, you should move unit support files and testrunners into a separate subfolder:



You can also add a separate test configuration to your project:



Because you have a test configuration that defines UNIT_TESTS (see picture above) and you moved test running to a separate subfolder, you can modify your `main.c` to something like this:

```
#include "mcc_generated_files/mcc.h"
#include <util/delay.h>

#ifdef UNIT_TESTS
#include "unit_tests/testrunner.h"
#else
int do_normal_stuff();
#endif

int main(void)
{
    // Initialize drivers from MCC
    SYSTEM_Initialize();
    _delay_ms(1000);

#ifdef UNIT_TESTS
    printf("Starting unit tests\n");
    run_unit_tests();
#else
    printf("Starting normal execution\n");
    return do_normal_stuff();
#endif
}

int do_normal_stuff() {
    return 0;
}
```

This way you can have one common project but one configuration that optimizes the build for production, and one test configuration that runs the test.

Add Code Coverage

If you want to enable code coverage, you can add the following lines to your `mdb.script` file before the `program` command:

```
# Generate simulator coverage data and write it to a file
set codecoverage.enabled Enabled_Reset_on_run
set codecoverage.enableoutputtofile true
set codecoverage.outputtofile simulator_coverage.txt
```

This will write coverage data to `simulator_coverage.txt` while running the `mdb` script.

10.2.5.2 Running Unit Tests on MPLAB ICE 4

Follow the steps below for an example of how to use Unity with the hardware tool MPLAB ICE 4 In-Circuit Emulator.

10.2.5.2.1 Step 1. Setup a Project with Tests and Verify on Simulator

It is advised to make your tests work on the simulator before continuing with MPLAB ICE 4 in-circuit emulator. Therefore, begin by following the steps described in 10.2.5.1. Running Unit Tests on the Simulator. Make sure you have a project with test runner and a serial driver with `printf` support.

Once your tests are working on the simulator you can continue with the MPLAB ICE 4. For the serial output to be captured by the emulator, the communication settings in the table below need to be used.

Serial driver setting	Required values
Baud rate	115200
Transmission bits	8
Reception bits	8
Data parity	No parity

10.2.5.2.2 Step 2. Connect MPLAB ICE 4 to Target Using Breakout Board

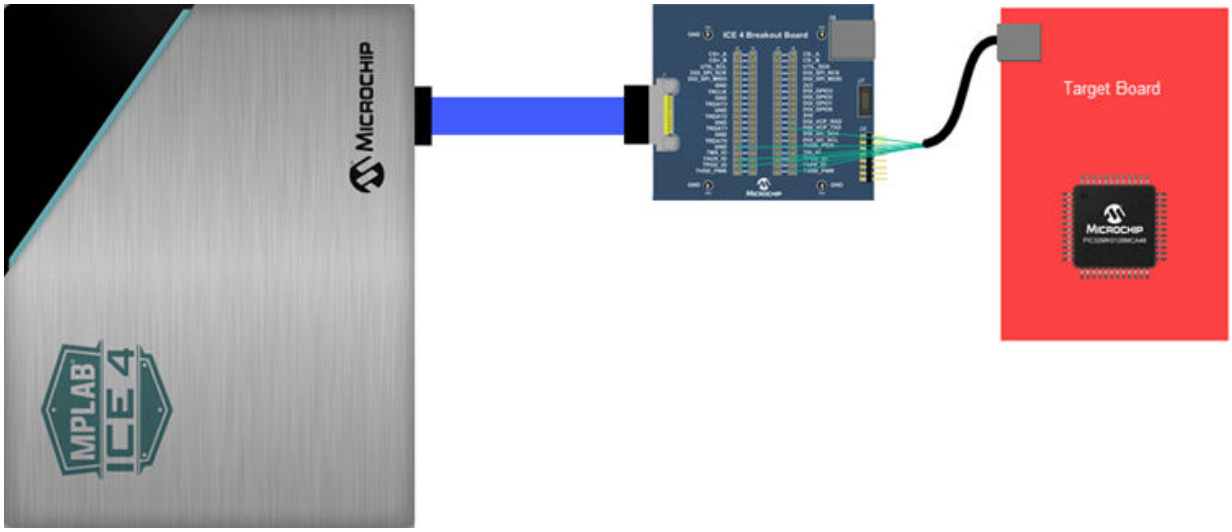
Connect the MPLAB ICE 4 to your hardware using the ICE 4 Breakout Board (AC244141). The ICSP pins are required to debug and program the device, and the DGI_VCP_TXD pin is required to read out the serial output. The DGI_VCP_TXD needs to be connected to the pin which you have configured for your serial driver to use.

Table 10-1. Connection Summary

ICE 4 Breakout Board pin name	Target
TVPP_IO	ICSP header 1
TVDD_PWR	ICSP header 2
GND	ICSP header 3
TPGD_IO	ICSP header 4
TPGC_IO	ICSP header 5
TAUX_IO	ICSP header 6
DGI_VCP_TXD	TX pin used by configured serial driver

© 2022 Microchip Technology Inc. and its subsidiaries

Figure 10-8. Hardware Connections



10.2.5.2.3 Step 3. Verify Connection Using Local ICE 4

Connect the MPLAB ICE 4 to your computer using USB. Open and use MPLAB X IDE to verify that you can program and run your test application on the target using the emulator and the ICE 4 breakout board.

If this fails, you need to review your breakout board connections.

Note: You will not be able to get the serial output yet.

10.2.5.2.4 Step 4. Configure MPLAB ICE 4 as Network Tool

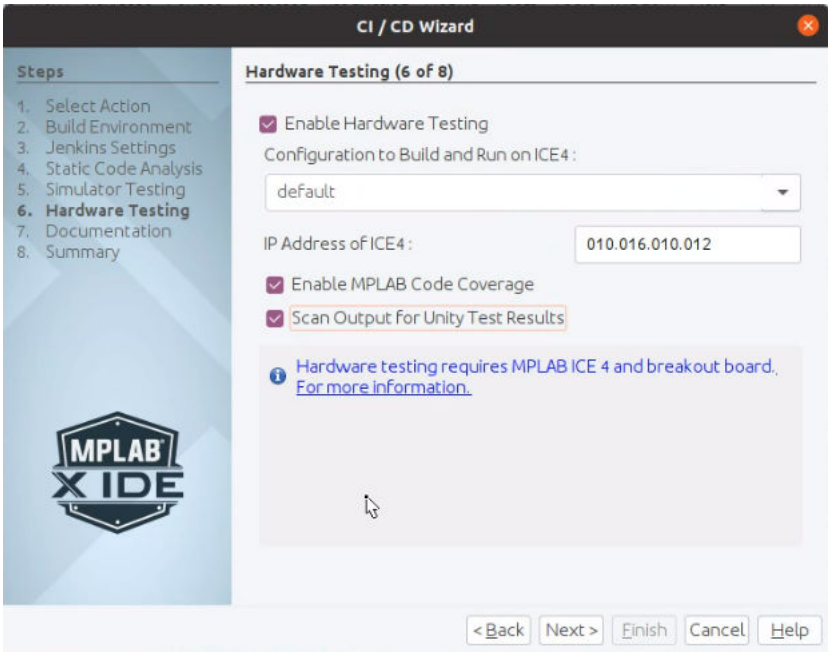
In MPLAB X IDE open *Tools>Manage Network Tools* and configure your connected MPLAB ICE 4 to use Ethernet instead of USB. The MPLAB ICE 4 needs to be connected to an Ethernet cable and you need to detach the USB cable and toggle the MPLAB ICE 4 power afterward. See the "*MPLAB ICE 4 In-Circuit Emulator User's Guide*" for details.

Once configured, the MPLAB ICE 4 should be discovered from the Manage Network Tools dialog when scanning for network tools and you should take a note of the IP address. To verify the Ethernet setup, you can now use the MPLAB ICE 4 as a remote tool in MPLAB X IDE to program and run your test application.

Note: If your network does not provide static IP addresses, you will need to repeat this step periodically.

10.2.5.2.5 Step 5. Add MPLAB ICE 4 in CI/CD Pipeline

From the CI/CD Wizard you should now be ready to enable the MPLAB ICE 4 hardware testing as part of the Jenkins build pipeline generation.



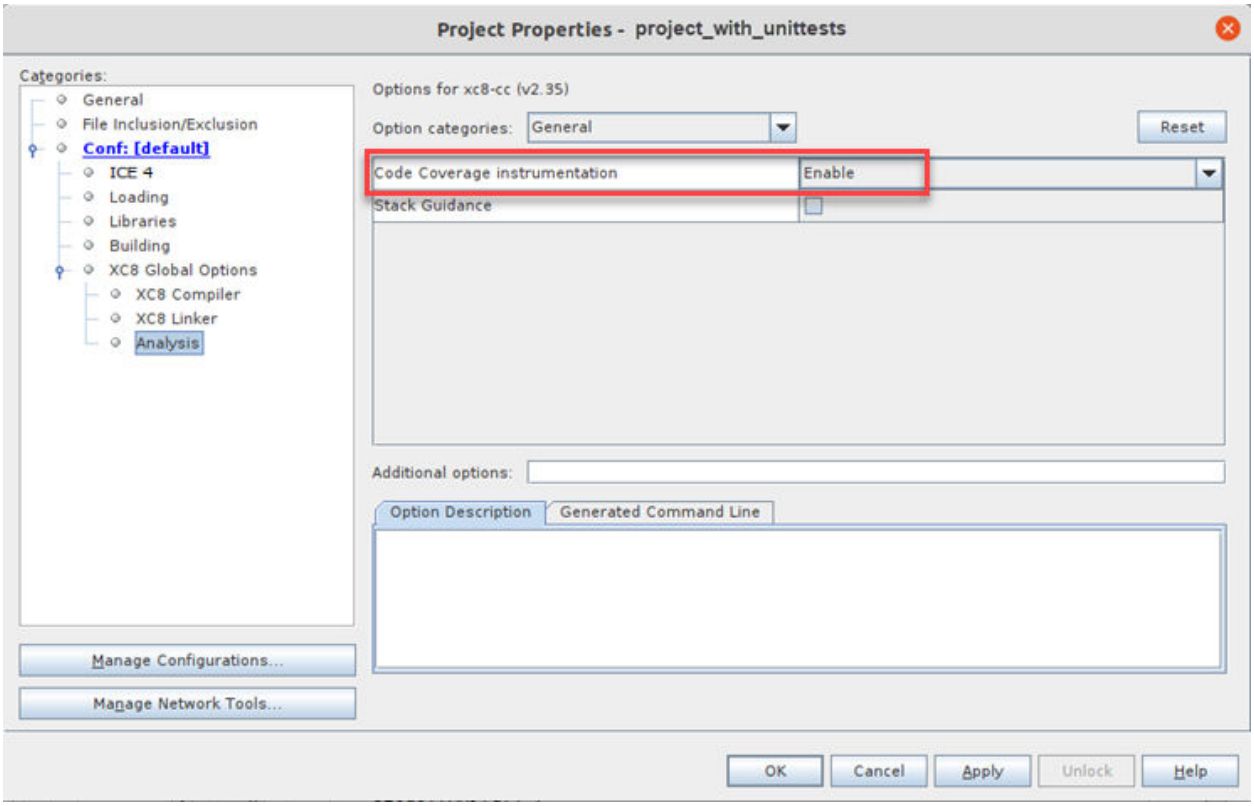
Select the build configuration you want to test.

Fill in the IP address that you noted when configuring the MPLAB ICE 4 on Ethernet.

Enable *MPLAB Code Coverage* if you want coverage data. But note that this requires that your project is set up to instrument your build with code coverage. To do this you need to open project properties and configure the Code Coverage instrumentation in the Analysis section (see figure below).

Enable *Scan Output for Unity Test Results* which will parse the output from your serial port and generate a test report in the Jenkins build job.

Figure 10-9. Enable Code Coverage



When everything is configured, the generated Jenkins build job will do the following when run:

- Build your project using the selected build configuration
- Communicate with the MPLAB ICE 4 and capture serial output
- Run your test application using a generated MDB script
- When done, parse the serial output and display test results
- Publish coverage data

10.2.5.2.6 Step 6. Update MPLAB ICE 4 IP address (optional)

If your MPLAB ICE 4 is later moved or your network uses dynamic IP addresses, you might need to update your build scripts with the new IP address.

In MPLAB X IDE select *Tools>Manage Network Tools* and scan for tools. Find the MPLAB ICE 4 and take a note of the new IP address.

In the *Jenkinsfile* update the *HARDWARE_TEST_TOOL_IP* field:

```
HARDWARE_TEST_TOOL_IP = '123.123.12.34'
```

In *mdb-hardware-script.txt* update the *addip* and *hwtool* commands:

```
# Add IP of tool so it can be detected
addip 123.123.012.034
# (Optional) List tools to verify that tool was detected
hwtool
# Connect to network configured tool
hwtool ICE4 <ipa>123.123.012.034
```

Note: The mdb scripts use a zero padded format, so use *123.123.012.034* instead of *123.123.12.34*

10.2.6 Getting Started with Dockerfile

Docker is an open platform for developing, shipping, and running applications. Docker provides OS-level virtualization which enables you to separate your applications from your infrastructure.

Docker will read the instructions from the Dockerfile and build a Docker image. A Docker image is like a template. The Docker image is used to build and run a Docker container. To do all this you must have Docker installed.

For more information about how to install Docker and use Dockerfiles, see www.docker.com.

10.2.6.1 Prerequisites

10.2.6.1.1 License Server

- To use the MPLAB XC PRO compilers in a setup with Docker containers, an MPLAB XC network license server must be accessible from the build node. MPLAB XC License Server Manual (DS50002334) describes how to set up this server.
- Other features may need a license server to verify their licenses, such as the MPLAB Analysis Tool Suite (for MISRA check and MPLAB Code Coverage).

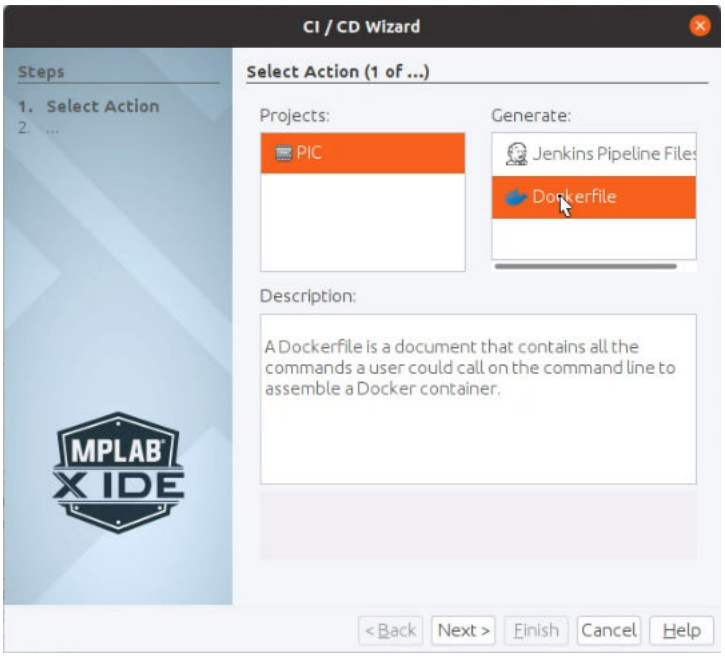
10.2.6.2 Generate Dockerfile from MPLAB X IDE

1. Open your project in MPLAB X IDE and open *Tools>CI/CD Wizard*.
2. Select **Docker File** as output type.
3. Fill in the remaining information as required in the wizard. See the following sections.
4. After generation, verify that a *Dockerfile* has been added to your project base folder.

10.2.6.2.1 CI/CD Wizard Dialog 1 - Docker Only

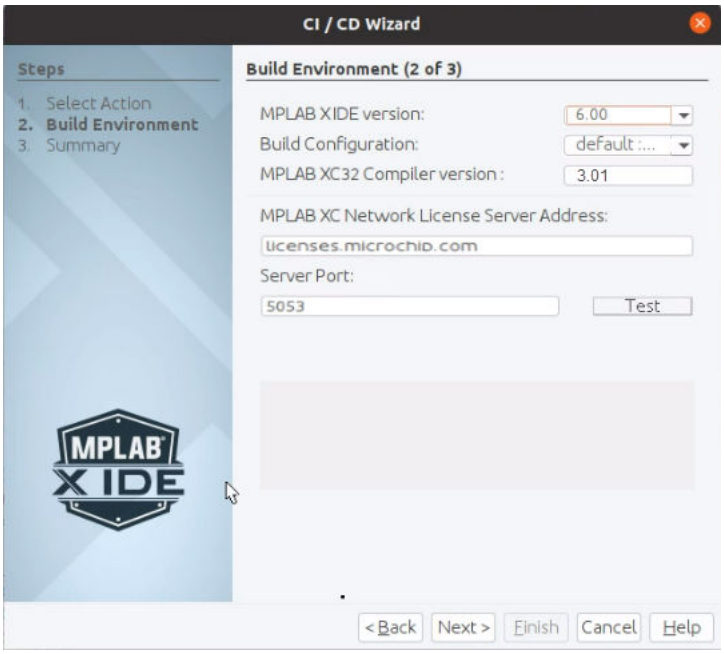
For this example, select “Docker File.” Click **Next** to proceed.

Note: You will need an MPLAB XC compiler license to create the Docker File.



10.2.6.2.2 CI/CD Wizard Dialog 2 - Docker Only

Set up the build environment to be used in a container. If you have a Network Server license, you can provide information on how to access it for the build. Otherwise leave the text boxes blank (though a note will appear).



Option	Description
MPLAB IDE version	Select supported versions from the drop-down list. Default is the latest version.
Build Configuration	Select a project configuration for the build
MPLAB XC Compiler version	Select a compiler version for the build
MPLAB XC Network License Server Address	Enter the address of the server. For example: licenses.microchip.com
Server Port	Enter the port number of the server. For example: 5053
Informational Note (conditional)	If you have a Workstation license, information on buying a Network Server license will be shown.

10.2.6.2.3 CI/CD Wizard Dialog 3 - Docker Only

View the summary to see that only the Dockerfile will be generated.

Click **Back** to change setup items. Click **Finish** to complete the setup. CI/CD Wizard will then generate Dockerfile based on the setup information.



10.2.6.2.4 CI/CD Wizard Output - Docker Only

In MPLAB X IDE, you can see:

1. Files that are being generated in the Output window.
2. Resulting files added to the project folder.
3. WebHelp in its own window.
4. Dockerfile in its own window.

This output is a subset of the Jenkins output.

10.2.6.3 Content of Generated Dockerfile

The generated Dockerfile is tailor-made for the specific MPLAB X IDE project.

The setup includes:

- Debian GNU/Linux 10
- Basic tools such as *curl*, *make* and *unzip*
- MPLAB X IDE installation including the binaries:
  - *mdb* - Microchip Debugger
  - *prjMakefilesGenerator* - CLI for generating Makefiles for MPLAB X IDE projects
  - *xclm* - XC Toolchain license manager
  - *IPE* - MPLAB Integrated Programming Environment
- MPLAB XC Toolchain installation. The installation uses the same version as specified in the MPLAB X IDE project
- The same Device Family Packs (DFPs) as specified in the MPLAB X IDE project

10.2.6.4 Suggested Next Steps

The generated Dockerfile can be used to set up a build environment for your MPLAB X IDE project. For more information, see Dockerfile reference.

10.2.6.4.1 Test Image Locally

The dockerfile makes it easy to create and test a docker image locally. If you do not have Docker installed locally, you can get it from www.docker.com.

Commands to build and run your image:

```
# Build an image based on the generated Dockerfile
docker build --tag mplabx-env --file Dockerfile .
```

```
# Check that the 'mplabx-env' image was created
docker images

# Start a docker container in interactive mode and map MPLAB X project source
docker run -it -v PATH_TO_MY_PROJECT_HERE:/usr/src/project mplabx-env bash
```

Once you have a started the container, you can run the following commands to build your mapped MPLAB X IDE project:

```
# Navigate to the mapped MPLAB X project
cd /usr/src/project

# Generate Makefiles
prjMakefilesGenerator .

# Build project
make

# List files generated in the dist folder
find dist
```

10.2.6.4.2 Use Dockerfile in a CI/CD Framework

The CI/CD Wizard has built-in support for **Jenkins**; you can generate support files for this framework. For other frameworks you need to refer to their documentation on how to use Docker.

10.2.6.4.3 Extending the Dockerfile with Additional Tools/Linux Packages

Additional Linux packages can be installed by modifying your dockerfile. To get started you can first start the image in interactive mode and then manually install the packages from command line:

```
apt-get update
apt-get install [packageName]
```

When you have verified that the packages are available and works, you can add them your Dockerfile:

```
RUN apt-get update -yq \
 && apt-get install -yq --no-install-recommends \
 [packageName] \
 && apt-get clean && rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*
```

Note: The last clean up line is used to keep the Docker image as small as possible.

10.2.7 CICDW Command Line Tool

If you would prefer to run the command-line version of the CI/CD Wizard, it can be found under `<MPLAB X IDE version install directory>mplab_platform/bin/cicdw.exe`.

Use `cicdw.exe -h` to get help on options (see below). For command specific help, use the command and `-h`; for example: `cicdw.exe generate-jenkins -h`.

Note: All command options must be on the same line, as in:

```
cicdw.exe generate-jenkins --misra --simulator ...
```

cicdw Help

```
usage: cicdw [-h] [-v] [-q] [--version]
             {generate-jenkins,generate-docker,generate-doxyfile,container,jenkins-server}
             ...

CI/CD Wizard for MPLAB X projects

optional arguments:
  -h, --help            show this help message and exit
  -v                    Enable verbose logging (Use -vv for extra verbose
                        logging)
  -q                    Show less output. (Use -qq to only show errors and
                        warnings and -qqq to only show errors)
```

```
  --version             Print version information

Commands:
  {generate-jenkins,generate-docker,generate-doxyfile,container,jenkins-server}
    generate-jenkins    Generate jenkins build files (Jenkinsfile and
                        Dockerfile)
    generate-docker     Generate dockerfile
    generate-doxyfile   Generate Doxyfile for doxygen documentation
    container           Manage local build container
    jenkins-server      Manage jenkins server

Type {command} -h for command specific help
```

10.2.7.1 CICDW Jenkins Example

Bolded text represents content specified by the developer. All content below must be on one line.

```
MPLABPath\mplab-nbplatform\nbbuild\netbeans\bin\cicdw.exe> generate-jenkins
--xclm ToolchainPath \xc8\v2.32\bin\xclm.exe
--configuration default
--mplabx-version 6.00
--toolchain-version 1.61
--license-server-name licenses.microchip.com
--license-server-port 5053
--docker-host-agent-label docker
--misra --use-cppcheck-trend
--simulate --simulate-with-coverage --simulate-with-unity --simulate-sercom-output uart1
 --simulate-build-configuration default
--doxygen --doxygen-configuration-file Doxyfile --doxygen-generate-doxyfile
--doxygen-project-name test --doxygen-input-encoding ISO-8859-1
--doc-artifact-html doc-html.zip --doc-artifact-latex doc-latex.zip --doc-artifact-pdf doc-
pdf.zip
--hardware-test --hardware-test-build-configuration default --hardware-test-tool-ip
192.168.16.3
--hardware-test-with-coverage --hardware-test-with-unity
ProjectPath
```

10.2.7.2 CICDW Docker Example

Bolded text represents content specified by the developer. All content below must be on one line.

```
MPLABPath\mplab-nbplatform\nbbuild\netbeans\bin\cicdw.exe> generate-docker
--xclm ToolchainPath\xc8\v2.32\bin\xclm.exe
--configuration default
--mplabx-version 6.00
--toolchain-version 2.32
--license-server-name licenses.microchip.com
--license-server-port 5053
ProjectPath
```

© 2022 Microchip Technology Inc.
and its subsidiaries

11. Troubleshooting

This section is designed to help you troubleshoot any problems, errors or issues you encounter while using MPLAB X IDE. If none of this information helps you, see “Support” for ways to contact Microchip Technology.

11.1 USB Driver Installation Issues

To install the correct USB drivers, see 3.3. Install the USB Device Drivers (for Hardware Tools).

To troubleshoot errors, see 3.3.3.5. Tool Communication Issues.

11.2 Operating System Issues

This section is focused on issues mainly regarding operating system issues.

11.2.1 Operating System Paths

Different operating systems have different issues concerning path, file and folder naming. For more information, see 6.5.5. Path, File and Folder Name Restrictions.

11.2.2 Windows Subsystem for Linux (WSL) and Projects

When working on projects that are accessed through the Windows® Subsystem for Linux® (WSL), the project must be stored on the Windows OS mounts inside WSL, and not inside the WSL image itself. An easy way to accomplish this is to map to a drive.

As an example, an WSL distribution called Ubuntu-20.04 can be made available on Windows OS by running this command:

```
net use Z: \\wsl$\Ubuntu-20.04
```

This will map the whole root file system of that WSL distribution into a new disk on your computer called `Z:`. Once the file system is mapped, then MPLAB X IDE can access the project on `Z:\home\...`.

11.3 NetBeans Platform Issues

The NetBeans platform may have issues concerning the platform release used for MPLAB X IDE. For more help, visit the Apache NetBeans website.

The following issues have a known impact in MPLAB X IDE.

11.3.1 Choosing “Inherited” as foreground color results in text for category being invisible in editor

When the Foreground color for Field is “Inherited” the text in popup completion window (tooltip) is invisible. The popup completion window shows unselected entries as white text on a white background. Scrolling down the list shows one entry at a time with white text on a blue background. Otherwise the feature seems to work normally.

This issue is recorded as a Netbeans Bugzilla issue for Fonts & Colors:

Options->Fonts & Colors. Choosing “Inherited” option as foreground color for any [Syntax] category causes the text to be invisible in the editor. Looking at the preferences xml file “org-netbeans-modules-editor-settings-CustomFontsColors-tokenColorings.xml” it is clear that there is no linkage to a valid color for the foreground attribute of the affected category like “Field” or “Identifier.”

Work-around:

1. Select *Tools>Options*, **Fonts & Colors** button, **Syntax** tab.
2. For “Language,” choose “C.”
3. Under “Category,” click “Field.”
4. To the left, see that “Foreground” is selected as “Inherited.” Select a color from the list.

© 2022 Microchip Technology Inc. and its subsidiaries

5. Click **OK**.

11.3.2 No visual indicators for debug

Although debugging appears to occur, there is no Pause arrow or line highlight, and breakpoints appear to be broken (broken icon in gutter). This can be caused by incorrect naming of file paths. See 6.5.5. Path, File and Folder Name Restrictions.

11.3.3 Unable to resolve identifier

Live parser shows unresolved identifier for structure nested in structure without name. This results from NetBeans native CND not supporting anonymous field declarations. The workaround is to specify an identifier for the field. The down side is that you will have to specify that field name everywhere you use it in code.

Example:

```
struct dados {
    unsigned char signature1;
    unsigned char signature2;
};
typedef union {
    unsigned char data [2];
    struct dados;         // unresolved ident.
} EEPROM_DATA;

void main(void) {
    EEPROM_DATA edata;
    edata.data [0] = 0xAA;    // this is ok
    edata.signature2 = 0xDD;  // unresolved ident.
}
```

11.3.4 SFR in Watches Window deselected when quickly stepping through code

When you step, the cursor updates in the source so the focus is removed from the Watches window. Netbeans then re-draws the whole Watches window, which removes any highlighting.

11.3.5 Sometimes menus disappear when navigating submenus

For Windows OS, sometimes when an MPLAB X IDE menu is selected and then a submenu of a menu item is selected, the menu will disappear. You may need to restart MPLAB X IDE to fix the problem.

11.4 MPLAB X IDE Issues

The following list are issues with MPLAB X IDE. If you do not see your issue here, look at documentation related to your project's debug and compiler tool.

11.4.1 Importing an MPLAB IDE v8 Project – Settings

Settings that were saved in a workspace in MPLAB IDE v8 (such as tool settings) will not be transferred to the new MPLAB X IDE project. Refer to the MPLAB IDE v8 help for what is stored in a workspace (*MPLAB IDE Reference>Operational Reference>Saved Information*).

11.4.2 Importing an MPLAB IDE v8 Project – Modified Linker Scripts

If you have modified your MPLAB IDE v8 project linker script so that it includes an object file, the linker will be unable to find the file when it is imported into MPLAB X IDE, because the build paths for MPLAB IDE v8 and MPLAB X IDE are different.

So you may see an error like:

```
<install path>ld.exe: cannot find file.o
```

since in MPLAB IDE v8 all build-related files are in one directory, whereas in MPLAB X IDE build files are in different subdirectories.

© 2022 Microchip Technology Inc. and its subsidiaries

You can edit your linker script to work with MPLAB X IDE by using wild cards. For example, change:

```
/* Global-namespace object initialization - MPLAB v8*/
.init   :
{
KEEP (crti.o(.init))
:
} >kseg0_program_mem
```

to:

```
/* Global-namespace object initialization - MPLAB X*/
.init   :
{
KEEP (*crti.o(.init))
:
} >kseg0_program_mem
```

Alternatively, you can use an address attribute that allows you to place functions in C code.

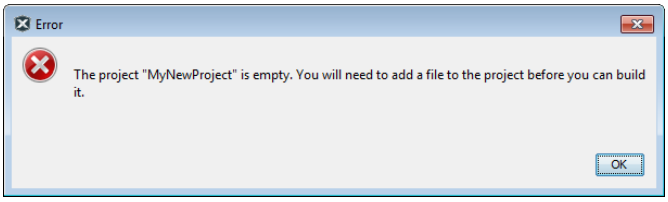
```
int __attribute__((address(0x9D001000))) myfunction (void) {}
```

This allows you to place a function at an address without modifying the default linker script.

11.4.3 Project is Empty/Project Files Grayed Out

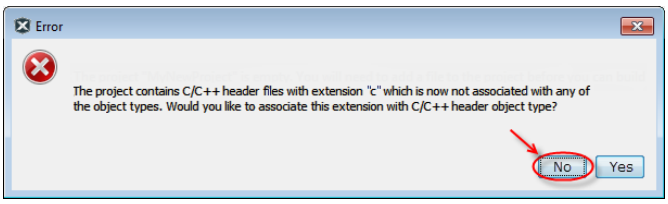
If you have loaded a corrupted project and answered a dialog question incorrectly, the Projects window could be grayed out and MPLAB X IDE will say the project cannot be built due to having no source files.

Figure 11-1. Project Empty Error



The problem occurs when the corrupted project is being loaded into MPLAB X IDE. A dialog will appear asking if C source files should be associated with header files. You can correct the issue at this point by clicking “No” (the default).

Figure 11-2. New Header Extension Error



However, if you say “Yes,” the source files are now seen as header files and the project looks empty. To fix the project, you can use either of the following methods:

- Exit from MPLAB X IDE.
- Go to the MPLAB X IDE User Directory (specified in the *Help>About* dialog) and delete the directory for your version of MPLAB X IDE. That is, delete the folder `vx.xx` from `C:\Users\UserName\AppData\Roaming\mplab_ide\dev\vx.xx`, where `x.xx` is your version number.
- Launch MPLAB X IDE again.

OR

- Go to *Tools>Options*, Embedded button, **Other** tab.
- Remove ‘c’ from the header files association.
- Add ‘c’ to the source files association.

- Click **OK**.

11.4.4 Code in Editor is gray after #error statement

See 7.8. C/C++ Code Error Directive.

11.4.5 Internet Connection is not stable enough to download latest toolchain

The feature covered in the link below is not available because your internet connection is determined to not be stable.

5.1.7.2. Download MPLAB XC Compilers

Alternately, find the latest MPLAB XC C compilers on the Microchip website:

www.microchip.com/en-us/development-tools-tools-and-software/mplab-xc-compilers

11.5 Errors

Errors can take many forms in the IDE, most commonly shown by icons in windows or messages in the Output window. Hovering over icons will pop up text that may explain the issue. For text messages, please refer to online help to search for the error.

Some common errors and resolutions for the IDE or Editor are listed in the following sections.

11.5.1 Command line too long (Windows® OS)

When a project's makefile is run, each command that is executed is passed to the local native shell. For Windows operating systems, there is a limit of 8191 characters. Therefore, for a project with hundreds of C files that link, the limit could be surpassed and this error will be displayed.

Linux OS and macOS can usually take very long command lines. If you want to find out the length limit for your system, you can type on the command line `getconf ARG_MAX`.

Response File Workaround

If you are using the MPLAB XC8 or MPLAB XC32 C compiler v1.01 and above, or MPLAB XC16 C compiler v1.22 and above, you may be able to use a response file when calling the linker to work around this issue.

- For MPLAB XC8, in the Project Properties* window, click in "Categories" on "XC8 linker." Under "Option categories" select "Additional Options." Select the check box next to "Use response file to link."
- For MPLAB XC16, in the Project Properties* window, click in "Categories" on "xc16-ld." Under "Option categories" select "General." Check the check box next to "Use response file to link."
- For MPLAB XC32, in the Project Properties* window, click in "Categories" on "xc32-ld." Under "Option categories" select "General." Check the check box next to "Use response file to link."

* To open the Project Properties window, see 5.3. Open Project Properties.

Library Workaround

For all other compilers, you can create an MPLAB X IDE library project and move some source files from your main project to the library project. Then add the library project to your main project.

To create a library project select *File>New Project*, click the "Microchip Embedded" category, and then select "Library Project" as the project.

To add the library to your main project, open the Project Properties window i.e., *File>Project Properties*), click in "Categories" under "Libraries," and then click the **Add Library Project** button.

Archive Checkbox Workaround

For MPLAB XC16 and MPLAB XC32, there is now a check box you can use when archiving long sets of files into libraries that might push the link line over the character limit. The "Break into multiple lines" option can be used to tell the compiler to break up the archive line into smaller lines to avoid this limitation.

Find this option in:

Project Properties window, "xc16-ar" or "xc32-ar" category, "General" option category, "Break into multiple lines" check box.

11.5.2 Destination Path Too Long/The filename or extension is too long (Windows® OS)

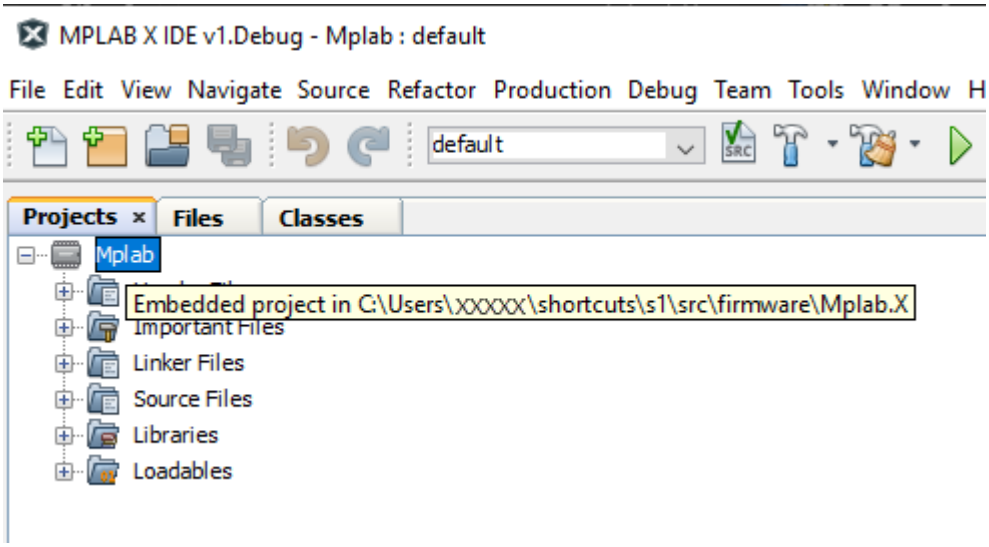
Windows OS has a maximum path length of 260 characters and a command-line limitation for Windows OS of 8191 characters. For details, see 6.5.5. Path, File and Folder Name Restrictions.

Workarounds for long paths are discussed below.

mklink Workaround

Use mklink to create shortcuts in a directory with a short path to projects files in directory with a long path.

1. Find the directory with the shortest path (from the root) that contains all the source files for a given project. For example, if the project is in `C:\Users\XXXXX\work\files\looooooooongDirectoryNamesHere\src\firmware\Mplab.X` but all project files, referenced via relative paths, are in `C:\Users\XXXXX\work\files\looooooooongDirectoryNamesHere`, then the later directory is the one with the shortest path. Call this **dirA**.
2. Create a new directory with a shorter path that will contain shortcuts to the actual files. How to link this directory with **dirA** will be explained in the next step. But to continue the example, consider the new directory is `C:\Users\XXXXX\shortcuts`. Call this **dirB**.
3. To make a link in **dirB** to point to **dirA**:
   a. Open a command (`cmd.exe`) or PowerShell® interface.
   b. Use the `cd` command to navigate to **dirB**.
   c. Make a link that points from **dirB** to **dirA**: `mklink /D s1 C:\Users\XXXXX\work\files\looooooooongDirectoryNamesHere`.
   d. Open the project in the IDE using **dirB** and `s1`. In this case the project directory would be shortened to: `C:\Users\XXXXX\shortcuts\s1\src\firmware\Mplab.X`.

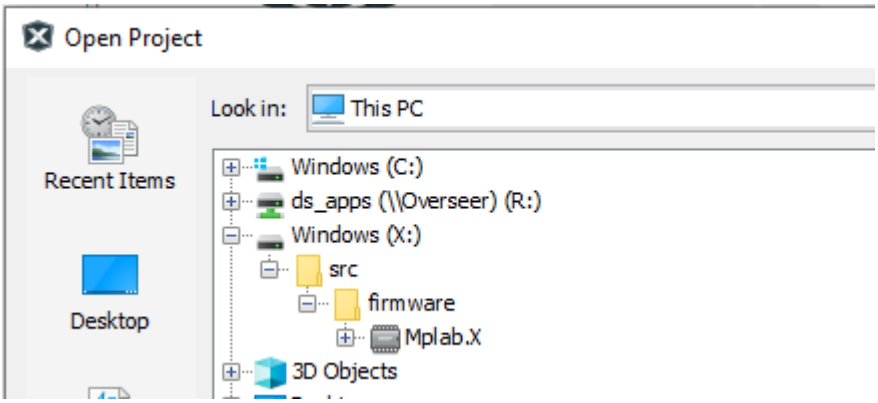


   e. Alternately in **dirB**, `cd s1\src\firmware\Mplab.X` and then run make.

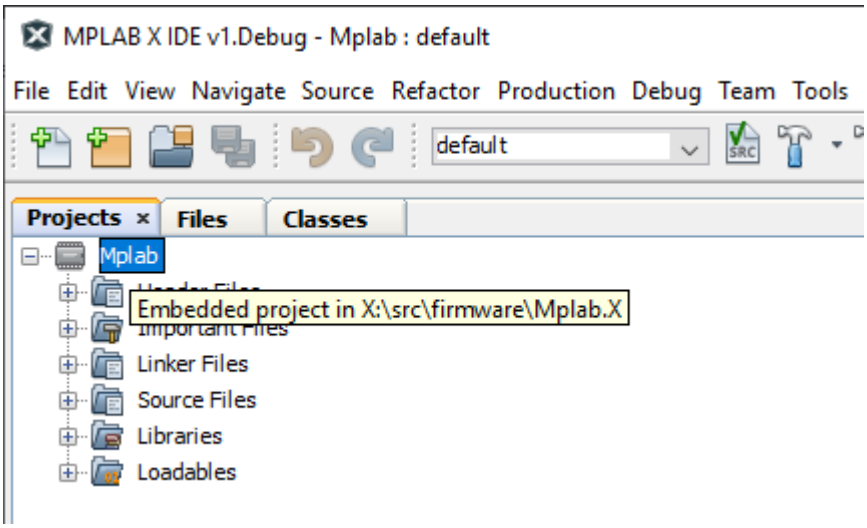
Virtual Drive Workaround

Use the same technique as in the previous section, but instead of a symbolic link, use a drive letter.

1. Select a drive letter not in use. For this example `X:`.
2. Open a command (`cmd.exe`) or PowerShell® interface.
3. Use the `cd` command to navigate to **dirA**.
4. Create a drive that starts in **dirB**. Run `subst x: .`
5. Open the project in the IDE using `X:`.



6. In this case the project would be shortened to `X:src\firmware\Mplab.X`.



7. Alternately in `X:`, `cd src\firmware\Mplab.X` and run make.

11.5.3 Couldn't reserve space for heap (Windows® OS)

MPLAB X IDE uses the MinGW in its make process. In Windows OS, you may have virtual memory allocation issues while GNU make is running.

Windows 7 or 8

To change the Virtual Memory settings, click *Start>Control Panel*. Then click System and then *Security>System>Advanced Systems Setting or System Protection*. On the **Advanced** tab, click on the **Performance Settings** button. In this dialog click the **Advanced** tab and then click on the **Change** button. Enter a custom size value as follows:

- Initial Size (MB) = Currently Allocated (shown at the bottom)
- Maximum Size (MB) = Recommended (shown at the bottom)

Click **Set**, click **OK**, and reboot your personal computer.

Windows 10

See:

www.geeksinphoenix.com/blog/post/2016/05/10/how-to-manage-windows-10-virtual-memory.aspx

11.5.4 The project ProjectName is empty

See 11.4.3. Project is Empty/Project Files Grayed Out.

11.5.5 For 8-bit code, #asm and #endasm cause red bangs in the Editor window

The `#asm` and `#endasm` statements are valid constructs for declaring in-line assembly code within an MPLAB XC8 C program. But in the IDE Editor, the C preparser does not see these as valid directives and will display a red bang next to these statements. However, this code will still work.

If you want to eliminate the red bangs in the Editor, you can use the `asm()` statement. This is the preferred method of using in-line assembly for all MPLAB XC C compilers.

Example:

```
// Inline Code that causes error
    #asm
        movlw 0x20;
        nop;
        nop;
    #endasm
// Workaround for inline assembly - that will not cause error
// (Multi line asm code)
    asm("\
    movlw 0x20;\
    nop;\
    nop;");
// Workaround for inline assembly - that will not cause Error

// (Single line asm code)
    asm("movlw 0x20; nop; nop");
```

11.5.6 For 16-bit code, MPLAB XC16 packed attribute statement causes red bangs in the Editor window

The following statement is marked as an error by the compiler parser but will build:

```
unsigned long long Bits8 : 8 __attribute__ ((packed));
```

The parser rules for qualifiers allow for any order, but the expectation is that the qualifiers are all specified before the identifier. Therefore, use one of the statements below to not produce an error:

```
unsigned long long __attribute__ ((packed)) Bits8 : 8;
```

OR

```
__attribute__ ((packed)) unsigned long long Bits8 : 8;
```

11.5.7 Hexmate Conflict Report Address Error

When Hexmate generates a conflict report, the address of the conflict in the hex file is sometimes not the same as the address shown in a memory window (where the data resides in memory.) For example:

Table 11-1. Hex Address vs. Memory Address

Device Family	Address in Hex File	Address in Memory
Baseline	0x100	0x80*
Midrange	0x100	0x80*
PIC18 MCUs	0x100	0x100
16-bit Devices	0x100	0x80*
32-bit Devices	0x100	0x100
* For some devices, the address shown in the conflict report is twice as large as the address of the data in memory.		

For more information on using Hexmate, see MPLAB XC8 documentation or *Hexmate User's Guide* (DS50003033) on the Microchip website.

11.5.8 Programming Errors

If you code protect a device and then attempt to program it, you will receive a programming failure error for an address region.

The “Erase Device Memory Main Project” function can help to determine if this is the programming issue.

1. Ensure code protection is off in your code. Also check the Configuration Bits window settings (*Window>Target Memory Views>Configuration Bits*.)
2. Under “Make and Program Device Project”  , select Erase device memory. This will erase the configuration bit settings.
3. Reprogram the device.

If programming succeeds, then code protect was the issue.

11.5.9 Programmer to Go Not Supported (PICkit 3)

A device that has too many configuration (config) words currently cannot support Programmer to Go (PTG). PTG is expecting config words to be sent all at once, but MPLAB X IDE sends each config word one at a time for devices with a large amount of config words.

11.5.10 Mouseover shows “Not Recognized” in Editor

When using mouseovers to reveal values in the Editor, ensure:

1. You are in debug mode (*Debug>Debug Project*). You cannot see symbols values unless you are debugging.
2. You are not mousing over a macro. You cannot see the value of a macro this way. Use the Macro Expansion Window by right clicking on the macro and selecting *Navigate>View Macro Expansion*.

11.5.11 Error Codes in -10000 Range

Error codes in the -10000 range represents Microsoft error codes. To determine the Microsoft error code, make the number positive (-10121 becomes 10121) and subtract 10000 (10121 - 10000 = 121). Microsoft error codes can be found at:

msdn.microsoft.com/en-us/library/windows/desktop/ms681382(v=vs.85).aspx

11.6 Forums

If you do not see your issue here, check out our forums at:

www.microchip.com/forums/f238.aspx

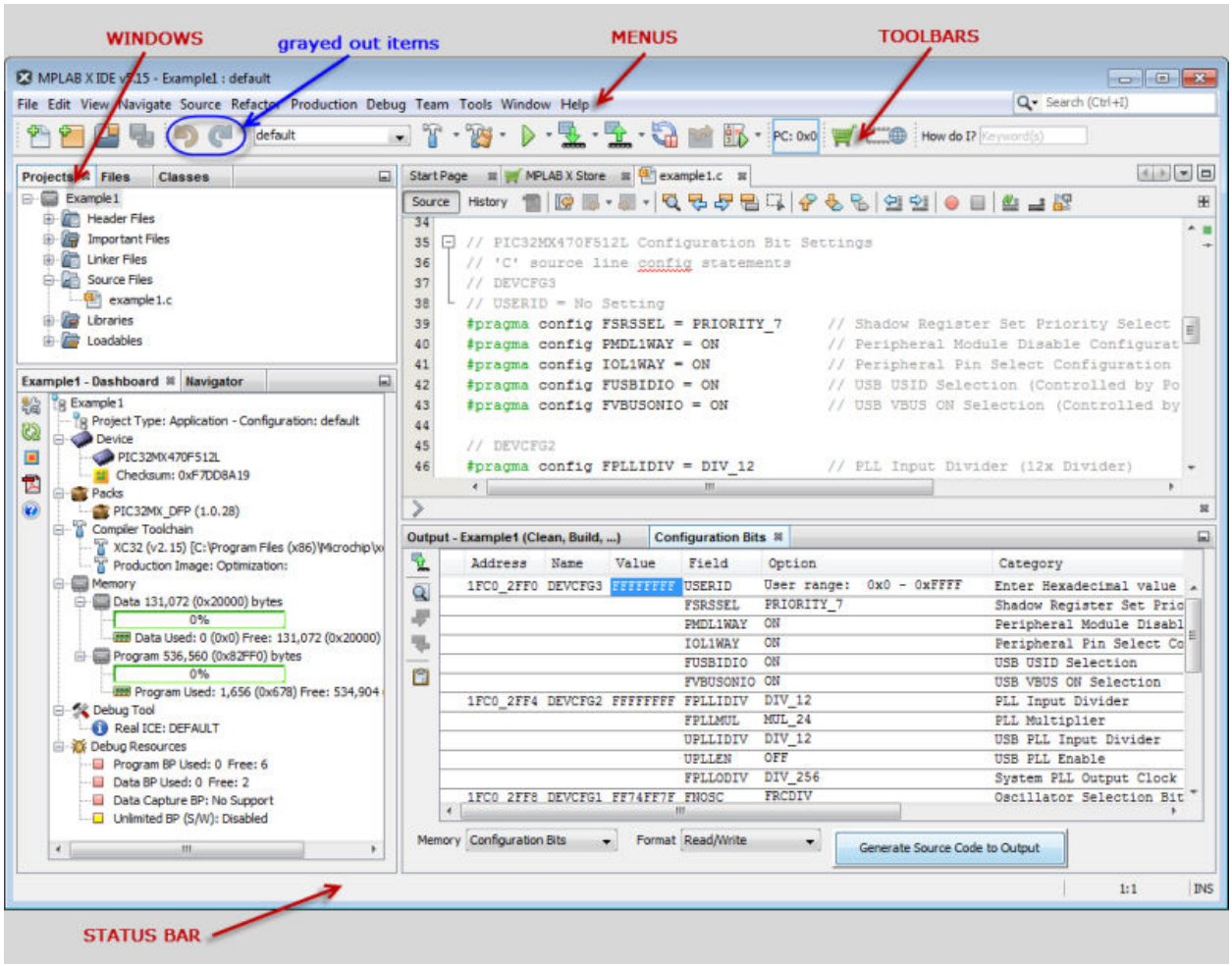
Here you will find discussions and recently posted solutions to problems.

12. Desktop Reference

The MPLAB® X IDE desktop is a resizable window that operates independent of the rest of the desktop items. The desktop consists of menus, toolbars, a status bar and tabbed windows. Menus, toolbars and the status bar are discussed here. Windows and dialogs are discussed in their own section.

Note: When the NetBeans help topic refers to a workspace, it is talking about a desktop. It is not referring to the MPLAB IDE v8 (and earlier) workspace.

Figure 12-1. MPLAB® X IDE Desktop



12.1 Menus

Many MPLAB® X IDE functions are accessible as menu items through the menu bar located across the top of the desktop. Menu items followed by ellipses (...) will open a dialog. For more on dialogs, see 13. MPLAB X IDE Windows and Dialogs.

Shortcut keys for menu items are listed next to the menu item.

Example: The shortcut keys for “New File” are Control-N (CTRL+N). More shortcut key information is available from *Help>Keyboard Shortcuts Card*.

Menu items may be grayed out for various reasons. See 12.4. Grayed Out or Missing Items and Buttons.

Additional context menus are available by right clicking in a window. For more on these menus, see 13.19. Projects Window.

Available menus are listed below.

Note: See the Start Page, My MPLAB IDE>Extend MPLAB>Selecting Simple or Full-Featured Menus to see all available menus and menu items. Not all of these items will be applicable to embedded development.

12.1.1 File Menu

This section lists the menu items in the File menu.

For keyboard shortcuts of some of these menu items, see *Help>Keyboard Shortcuts Card*.

Table 12-1. File Menu Options

Command	Action
New Project	Creates a new project with the New Project wizard.
New File	Creates a new file with the New File wizard.
Open Project	Opens an existing project.
Open Recent Project	Displays a list of all recently-opened projects for selection. This list can be cleared by deleting the file `/config/Preferences/org/netbeans/modules/projectui.properties` under the User directory (where the path may be found under *Help>About*).
Import	Imports one of the following: • Hex/ELF... (Prebuilt) File – built with another tool. • MPLAB IDE v8 Project – launch Import Legacy Project wizard. • Other Embedded – import from another embedded platform. • From ZIP – import from a zipped project, i.e., unzip and import.
Close Project	Closes the current project.
Close Other Projects	Closes all projects except for the main project.
Close All Projects	Closes all open projects.
Open File	Opens an existing file.
Open Recent File	Displays a list of all recently-opened files for selection.
Project Group	Associates the current project with a group.
Project Properties	Opens the Project Properties window.
Save	Saves the current file.
Save As	Saves the current file under a new path and/or name.
Save All	Saves all open files. If the “Compile on Save” feature is selected, this will also compile/build project files.
Page Setup	Sets up the page for printing.
Print	Shows print preview of current file and allows printing.
Print to HTML	Prints the current file to a new file in HTML format.
Exit	Quits MPLAB X IDE.

12.1.2 Edit Menu

This section lists the menu items in the Edit menu.

For keyboard shortcuts of some of these menu items, see *Help>Keyboard Shortcuts Card*.

© 2022 Microchip Technology Inc. and its subsidiaries

Table 12-2. Edit Menu Options

Command	Action
Undo	Reverses (one at a time) a series of editor actions, except Save.
Redo	Reverses (one at a time) a series of Undo commands.
Cut	Deletes the current selection and places it on the clipboard.
Copy	Copies the current selection to the clipboard.
Paste	Pastes the contents of the clipboard into the insertion point.
Paste Formatted	Pastes the formatted contents of the clipboard into the insertion point.
Paste from History	Pastes from Copy history.
Delete	Deletes the current selection.
Select All	Selects everything in the current document or window.
Select Identifier	Selects the word nearest the cursor.
Find Selection	Finds instances of the current selection.
Find Next	Finds next instance of found text.
Find Previous	Finds previous instance of found text.
Find	Finds a text string.
Replace	Finds a string of text and replaces it with the string specified.
Find Usages	Finds usages and subtypes of selected code.
Find in Projects	Finds specified text, object names, object types within projects.
Replace in Projects	Replaces text, object names, object types within projects.
Start Macro Recording	Starts recording keystrokes.
Stop Macro Recording	Stops recording keystrokes . To manage macros, select *Tools>Options*, **Editor** button, **Macros** tab.

12.1.3 View Menu

This section lists the menu items in the View menu.

For keyboard shortcuts of some of these menu items, see *Help>Keyboard Shortcuts Card*.

Table 12-3. View Menu Options

Command	Action
Editors	Selects an available editor to view your files. • History – Show the history of edits to this file. • Source – Show the source code in this file.
Split	Splits the current editor window and its contents into two windows, either vertically or horizontally oriented.
Code Folds>Collapse Fold	If the insertion point is in a foldable section of text, collapses those lines into one line.
Code Folds>Expand Fold	If the currently selected line in the Source Editor represents several folded lines, expands the fold to show all of the lines.

.....continued

Command	Action
Code Folds>Collapse All	Collapses all foldable sections of text in the Source Editor.
Code Folds>Expand All	Expands all foldable sections of text in the Source Editor.
Code Folds>Collapse Fold Tree	Collapses all folds in a nested group of folds.
Code Folds>Expand Fold Tree	Expand all folds in a nested group of folds.
Web Browser	Opens the default web browser to the NetBeans home page.
IDE Log	Opens the log file in a tab of the Output window.
Toolbars>File, etc.	Shows the associated toolbar.
Toolbars>Small Toolbar Icons	Uses small icons on the toolbars.
Toolbars>Reset Toolbars	Resets to the default toolbar setup.
Toolbars>Customize	Customizes the items on an existing toolbar and allows creation of a new one.
Show Editor Toolbar	Shows the editor toolbar on the File tab.
Show Line Numbers	Shows line numbers in left margin.
Show Non-printable Characters	Shows characters even if they are not printable (non-ASCII).
Show Breadcrumbs	Shows breadcrumbs, or the path to the file from the project file, on the bottom of the editor window.
Show Indent Guide Lines	Shows faint lines to represent indents.
Show Diff Sidebar	Shows the DIFF sidebar.
Show Versioning Labels	Shows version labels.
Synchronize Editor with Views	Synchronizes the editor with open views.
Show Only Editor	Shows editor window in full IDE window.
Full Screen	Expands window to full length and breadth of screen.

12.1.4 Navigate Menu

This section lists the menu items in the Navigate menu.

For keyboard shortcuts of some of these menu items, see *Help>Keyboard Shortcuts Card*.

Table 12-4. Navigate Menu Options

Command	Action
Go to File	Finds and opens a specific file.
Go to Type	Finds and opens a specific class or interface.
Go to Symbol	Finds the symbol name as specified.
Go to Previous Window	Gives focus to the previously selected window.
Go to Source	Displays the source file containing the definition of the selected class.
Go to Declaration	Jumps to the declaration of the item under the cursor.
Go to Super Implementation	Jumps to the super implementation of the item under the cursor.
Last Edit Location	Scrolls the editor to the last place where editing occurred.
Back	Navigates back.

© 2022 Microchip Technology Inc. and its subsidiaries

.....continued	
Command	**Action**
Forward	Navigates forward.
Go to Line	Jumps to the specified line.
Toggle Bookmark	Sets a bookmark on a line of code.
Bookmark History Popup Next	Goes to the next bookmark in the Bookmark window (*Window>IDE Tools>Bookmarks*).
Bookmark History Popup Previous	Goes to the previous bookmark in the Bookmark window (*Window>IDE Tools>Bookmarks*).
Next Error	Scrolls the Source Editor to the line that contains the next build error.
Previous Error	Scrolls the Source Editor to the line that contains the previous build error.
Select in Projects	Opens the Projects window and selects current document within.
Select in Files	Opens the Files window and selects current document within.
Select in Classes	Opens the Classes window and selects current document within.
Select in Favorites	Opens the Favorites window and selects current document within.

12.1.5 Source Menu

This section lists the menu items in the Source menu.

For keyboard shortcuts of some of these menu items, see *Help>Keyboard Shortcuts Card*.

Table 12-5. Source Menu Options

Command	Action
Format	Formats the selected code, or the entire file if nothing is selected.
Remove Trailing Spaces	Removes spaces at the end of the line.
Shift Left	Moves the selected line, or lines one tab to the left.
Shift Right	Moves the selected line, or lines one tab to the right.
Move Up	Moves the selected line, or lines one line up.
Move Down	Moves the selected line, or lines one line down.
Move Code Element Up	Moves the selected code element up one line.
Move Code Element Down	Moves the selected code element down one line.
Duplicate Up	Copies the selected line or lines one line up.
Duplicate Down	Copies the selected line or lines one line down.
Toggle Comment	Toggles the commenting out of the current line or selected lines.
Complete Code	Shows the code completion box.
Insert Code	Pops up a context aware menu that you can use to generate common structures such as constructors, getters, and setters.
Fix Code	Displays editor hints. The IDE informs you when a hint is available when the light bulb is displayed.
Show Method Parameters	Selects the next parameter. You must have a parameter selected (highlighted) for this shortcut to work.

© 2022 Microchip Technology Inc. and its subsidiaries

.....continued

Command	Action
Show Documentation	Shows documentation for item under the cursor.
Insert Next Matching Word	Generates the next word used elsewhere in your code, as you type its beginning characters.
Insert Previous Matching Word	Generates the previous word used elsewhere in your code, as you type its beginning characters.
Inspect	Runs the specified inspection on the selected file, package, or project.
Scan for External Changes	Scans file for changes made outside of MPLAB® X IDE.

12.1.6 Refactor Menu

This section lists the menu items in the Refactor menu. The items you see are dependent on the type of object (variable, function, etc.) you are refactoring. For more information, see 7.7. C Code Refactoring.

For keyboard shortcuts of some of these menu items, see *Help>Keyboard Shortcuts Card*.

Table 12-6. Refactor Menu Options

Command	Action
Rename	Enables you to change the name of a variable or function to something more meaningful. In addition, it updates all source code in your project to reference the element by its new name.
Move	Moves a class to another package or into another class In addition, all source code in your project is updated to reference the class in its new location.
Copy	Copies a class to the same or a different package.
Safely Delete	Checks for references to a code element and then automatically deletes that element if no other code references it.
Change Function Parameter	Changes the amount and name of parameters for the selected function.
Encapsulate Fields	**C++ Only:** Click in a C++ source file to see this option. Automatically generates a getter method and a setter method for a field and optionally updates all referencing code to access the field using the getter and setter methods. See also: wiki.netbeans.org/Refactoring

12.1.7 Production Menu

This section lists the menu items in the Production menu (previously Run menu).

Note: The Run action has been removed from this menu, but it still available as an icon on the toolbar: 

For keyboard shortcuts of some of these menu items, see *Help>Keyboard Shortcuts Card*.

Options displayed depend on device and tool.

Table 12-7. Production Menu Options

Command	Action
Build Project	Builds all the files in a project.
Clean and Build Project	Removes (cleans) previously generated project files and then rebuilds the files in a project.

.....continued	
Command	**Action**
Batch Build Project	Builds multiple configurations of a project (only embedded available).
Make and Program Device	Perform a Make and the program the device.
Program Device for Debugging	Program the device with a debug build.
Program Device for Production	Program the device with a production build.
Erase Device Memory	Erase all device memory.
Programmer To Go PICkit 3/PICkit 4	Program the device via PTG.
Set Project Configuration	Selects project configuration – Default.
Set Main Project	Sets the main project by selecting from a list of open projects.
Configuration Bits	Opens the Configuration Bits window.
Check File	Checks a file against a standard (XML related).
Validate File	Validates a file against a standard (XML related).
Repeat Build/Run	Runs again after halt.
Stop Build/Run	Ends run.

12.1.8 Debug Menu

Below are the menu items in the Debug menu.

Table 12-8. Debug Menu Options

Command	Action
Debug Project	Debugs the main or selected project.
Discrete Debugger Operation	Performs the following debug operations one step at a time (discretely): • Build for Debugging (build with debug executive). • Program Device for Debugging (program with debug build). • Launch Debugger. • Live Connect Debug. This is useful for changing the Memory window setting during debug and using starter kits.
Finish Debugger Session	Ends the debugging session.
Pause	Pauses debugging – use “Continue” to resume.
Continue	Resumes debugging after “Pause” until the next breakpoint or the end of the program is reached.
Step Over	Executes one source line of a program. If the line is a function call, executes the entire function and then stops.
Step Into	Executes one source line of a program. If the line is a function call, it executes the program up to the function’s first statement and then stops.
Step Out	Executes one source line of a program. Finishes execution of the current function and stops on the source line immediately following the call to that function.

© 2022 Microchip Technology Inc. and its subsidiaries

.....continued

Command	Action
Step Instruction	Executes one machine instruction If the instruction is a function call, it executes the function and returns control to the caller.
Run to Cursor	Runs the current project to the cursor's location in the file and stop program execution.
Reset	Resets the device.
Set PC at cursor	Sets the program counter (PC) value to the line address of the cursor.
Focus Cursor at PC	Moves the cursor to the current PC address and centers this address in the window.
Stack>Make Callee Current	Makes the method being called the current call . Only available when a call is selected in the Call Stack window.
Stack>Make Caller Current	Makes the calling method the current call. Only available when a call is selected in the Call Stack window.
Stack>Pop Topmost Call	Pops the call on top of the stack.
Stack>Pop To Current Stack Frame	Pops the current stack from the top of the stack.
Stack>Pop Last Debugger Call	Pops the last call from the debug tool from the top of stack.
Toggle Line Breakpoint	Adds a line breakpoint or removes the breakpoint at the cursor location in the program.
New Breakpoint	Sets a new breakpoint at the specified line, exception, or method.
New Watch	Adds the specified symbol to watch.
New Run Time Watch	Adds the specified symbol to watch that will change value as the program runs/ executes.
Disconnect from Debug Tool	Disconnects communications between MPLAB® X IDE and the debug tool. To reconnect, either Run or Debug.
Run Debugger/ Programmer Self Test	Performs a debug tool self-test. For tools that support a self test, follow the tool documentation to set up the hardware and then run this test to confirm proper operation.
Hardware Tool Emergency Boot Firmware Recovery	Run utility to restore hardware tool boot firmware to its factory state. See MPLAB ICD 4 or PICkit 4 Help for details.

12.1.9 Team Menu

This section lists the menu items in the Team menu.

For keyboard shortcuts of some of these menu items, see *Help>Keyboard Shortcuts Card*.

Table 12-9. Team Menu Options

Command	Action
Shelve Changes	Temporarily sets aside some not yet committed changes in a working directory as a patch file.
Git, Mercurial, Subversion	Displays submenus that are specific to each version management system. Please see product documentation for more on the submenu options.
History	Shows the history of a file or reverts file to history version.
Find Task	Finds an issue in a version control system.

.....continued	
Command	**Action**
Report Task	Reports an issue to a version control system.
Create Build Job	Creates a build using a version control system.

12.1.10 Tools Menu

This section lists the menu items in the Tools menu.

For keyboard shortcuts of some of these menu items, see *Help>Keyboard Shortcuts Card*.

Table 12-10. Tools Menu Options

Command	Action
Embedded	Visible if a plugin has been added – select the plugin from the submenu.
Licenses	Manage compiler licenses. For details, see the Documentation tab of: www.microchip.com/mplab/compilers • Roam Network Licenses. • Activate Workstation License. • License Status. • Change Licensing Type.
Packs	Open MPLAB Pack Manager to select a versioned device pack for your project device.
Templates	Opens the Template Manager.
DTDs and XML Schemas	Opens the DTDs and XML Schemas Manager.
Plugins	Opens the Plugins Manager . For details, see the NetBeans help topic: wiki.netbeans.org/InstallingAPlugin
Plugins Download	Opens a selection dialog, where you can: • Visit “Embedded Code Source” Website - Download a plugin to install later using the “Downloaded” tab of the Plugin Manager. • Go to MPLAB X Plugin Manager - Install a plugin immediately from the “Available Plugins” list in the Plugin Manager.
Options	Opens the Options dialog. For macOS: Use MPLAB X IDE>Preferences. See 13.20. Tools Options Window, Embedded.

12.1.11 Window Menu

This section lists the menu items in the Window menu.

For keyboard shortcuts of some of these menu items, see *Help>Keyboard Shortcuts Card*.

Table 12-11. Window Menu Options

Command	Action
Projects	Opens the Projects window. See 6.5.1. Projects Window View.
Files	Opens the Files window. See 6.5.2. Files Window View.

.....continued

Command	Action
Classes	Opens the Classes window. See 6.5.3. Classes Window View.
Favorites	Opens the Favorites window. See 6.5.4. Favorites Window View.
Services	Opens the Services window. See NetBeans help for more on this window.
Dashboard	Opens the Dashboard window. See 8.5. View the Dashboard Display.
Navigator	Opens the Navigator window. See NetBeans help for more on this window.
Acton Items	Opens the Action Items window. See 13.2. Action Items Window.
Tasks	Opens the Task List window.
Output	Opens or moves to front the Output window. See 13.16. Output Window.
Editor	Opens an empty editor window. See 7.1. Editor Usage.
Debugging>Output	Opens debugging output windows.
Debugging>Variables	Opens the Local Variables debugger window. See 5.17. Watch Local Variable Values Change.
Debugging>Watches	Opens the Watches debugger window. See 5.16. Watch Symbol Values Change.
Debugging>Call Stack	Opens the Call Stack debugger window. See 8.3. View the Call Stack.
Debugging>Breakpoints	Opens the Breakpoints window. See 5.14. Control Program Execution with Breakpoints.
Debugging>Sessions	Opens the Sessions window.
Debugging>Sources	Opens the Sources window.
Debugging>Disassembly	Opens the Disassembly window. See 13.8. Disassembly Window.
Debugging>PIC AppIO	Opens the Application In/Out window . Applies to the MPLAB® REAL ICE™ in-circuit emulator.
Debugging>Trace	Opens the Trace window . Applies to the Simulator or MPLAB REAL ICE in-circuit emulator.
Debugging>Stopwatch	Opens the Stopwatch window. See 8.1. Use the Stopwatch.
Debugging>PC Profiling	Open the PC Profiling window. Applies to the MPLAB® REAL ICE™ in-circuit emulator.

.....continued

Command	Action
Debugging>Code Coverage	Open the Code Coverage window. Enable code coverage under *Project Properties>MPLAB XC Compiler>Analysis*. A license is required for full code coverage features.
Debugging>Data Visualizer	Open the MPLAB Data Visualizer. See 8.6. Visualize Data.
Debugging>Power Monitoring	Open the MPLAB Data Visualizer for power monitoring.
Debugging>Triggers	Opens the Trigger In window. Applies to the MPLAB® REAL ICE™ in-circuit emulator.
Debugging>Debugger Console	Opens a console window for entering commands on the command line.
Web>Web Browser	Opens your default web browser.
Web>CSS	Opens the CSS Styles window. Enables you to edit the declarations of rules for HTML elements and selectors in a CSS file.
IDE Tools>Bookmarks	Opens the Bookmarks window. Displays a list of the bookmarks in your files in your open projects.
IDE Tools>Notifications	Opens the Notifications window. Displays a list of the IDE errors and warnings that occurred in the current IDE session.
IDE Tools>Terminal	Opens a Terminal window for a local or remote host.
IDE Tools>Processes	Opens the Processes window. You can attach a debug tool to a running process.
PIC Memory Views>Memory	Opens specified Memory window. Memories shown depend on the project device. See either: 13.12. Memory Windows - 8- and 16-Bit Devices 13.13. Memory Windows - 32-Bit Devices
Simulator>Stimulus	Opens the Simulator Stimulus window. See MPLAB X Simulator documentation for details.
Simulator>Analyzer	Opens the Simulator Analyzer window. See MPLAB X Simulator documentation for details.
Simulator>IOPin	Opens the Simulator IO Pin window. See MPLAB X Simulator documentation for details.
Simulator>Register Trace	Opens the Simulator Register Trace window. See MPLAB X Simulator documentation for details.
Configure Window> Maximize	Maximizes the active window to the full IDE window size.
Configure Window>Float	Opens a floating IDE window with a tab.
Configure Window> Float Group	Opens a floating IDE window group with tabs.
Configure Window>Minimize	Minimizes the active window to standard size.
Configure Window> Minimize Group	Minimizes the window group to standard size.
Configure Window>Dock	Restores a floating tab to the main window.

© 2022 Microchip Technology Inc.
and its subsidiaries

.....continued

Command	Action
Configure Window> Dock Group	Restores a floating tab group to the main window.
Configure Window> Clone Document	Opens a new tab for the same document.
Configure Window> Split Window	Splits the active document either vertically or horizontally.
Configure Window< New Document Tab Group	Splits the editor window into two groups of tabs and separates the selected tab into the new group.
Configure Window>Collapse Document Tab Group	Combines separate group of tabs into one group.
Reset Window	Resets windows to their default settings.
Close Window	Closes the current tab in the current window. If the window has no tabs, the whole window is closed.
Close All Documents	Closes all open documents in the Source Editor.
Close Other Documents	Closes all open documents except the active one.
Document Groups	Create named document group(s).
Documents	Opens the Documents dialog box, in which you can save and close groups of open documents.

12.1.12 Help Menu

This section lists the menu items in the Help menu.

Help Contents

Displays a help viewer with all installed help files, including some NetBeans help. Any plugin help file will appear in the viewer after the plugin is installed.

See the “Help on Help” topic for more information.

Tool Help Contents

Displays a single JavaHelp viewer with a single tool help file. This provides a way to search through content for just one tool.

Release Notes

Open a list of links to available Release Notes for MPLAB® X IDE supported tools. Other IDE support documentation links are also listed, including “User’s Guides”, “Device Support”, “Debug Features Support”, and “Reserved Resources”.

Online Docs and Support

Opens the NetBeans support web page.

Keyboard Shortcuts Card

Displays the keyboard shortcuts PDF document.

MPLAB X Store

Opens the MPLAB X Store tab. Use your Microchip account to order tools and software.

Start Page

Opens or moves the Start Page tab in front of any other open tabs.

About

© 2022 Microchip Technology Inc. and its subsidiaries

Displays a window about MPLAB X IDE. Information shown includes the IDE project version, Java and JRE version, operating system version, and User and Cache directories.

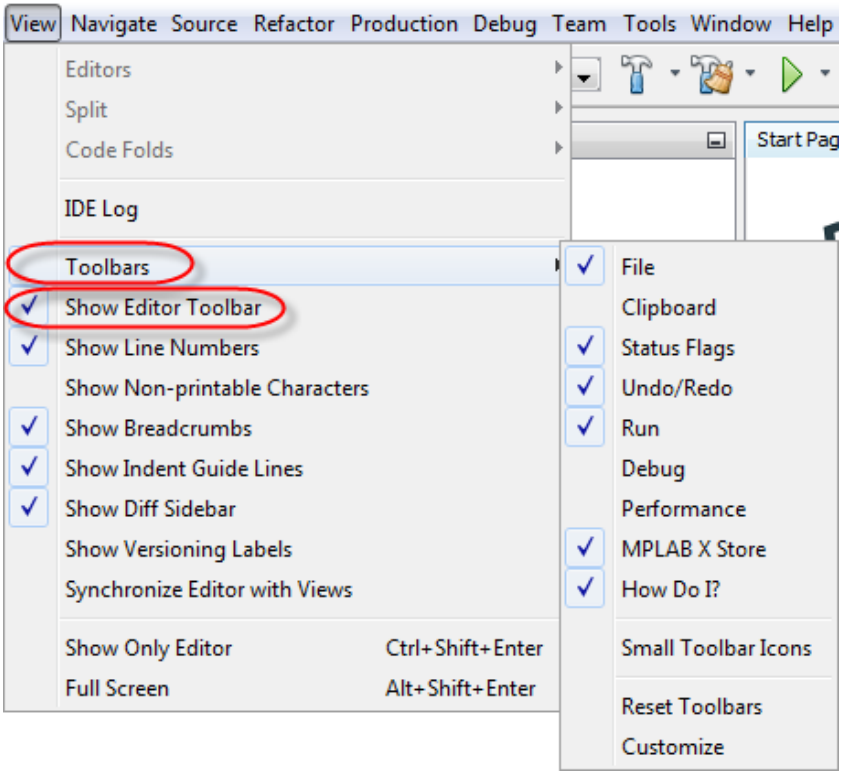
12.2 Toolbars

MPLAB X IDE displays different toolbars depending on which features or tools you are using. The icons in these toolbars provide shortcuts to routine tasks.

The following is useful when working with toolbars:

- Toolbar Features
- Customize Toolbars
- Grayed Out or Missing Items and Buttons

Figure 12-2. View Toolbars



12.2.1 File Toolbar

The File Toolbar currently contains button icons for the following functions. These functions are also on the File menu.

Icon	Icon Text	Function
	New File	Creates a new file with the New File wizard.
	New Project	Creates a new project with the New Project wizard.
	Open Project	Opens an existing project.

.....continued		
Icon	**Icon Text**	**Function**
	Save All Files	Saves all open files.

Related Links

12.2.11. Toolbar Features

12.2.12. Customize Toolbars

12.2.2 Clipboard Toolbar

The Clipboard Toolbar currently contains button icons for the following functions. These functions are also on the Edit menu.

Icon	Icon Text	Function
	Cut	Deletes the current selection and places it on the clipboard.
	Copy	Copies the current selection to the clipboard.
	Paste	Pastes the contents of the clipboard into the insertion point.

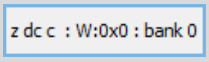
Related Links

12.2.11. Toolbar Features

12.2.12. Customize Toolbars

12.2.3 Status Flags Toolbar

The Status Flags Toolbar contains the following functions:

Image	Image Text	Description
	Program Counter	Program Counter (PC) current value.
	Status Flags	Device status flags. See your device data sheet for information on status bits and/or registers.

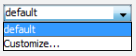
12.2.4 Undo/Redo Toolbar

The Undo/Redo Toolbar currently contains button icons for the following functions. These functions are also on the Edit menu.

Icon	Icon Name	Function
	Undo	Reverses (one at a time) a series of editor actions, except Save.
	Redo	Reverses (one at a time) a series of Undo commands.

12.2.5 Run Toolbar

The Run Toolbar currently contains button icons for the following functions. These functions are also on the Run, Debug and project context menus.

Icon	Icon Text	Function
	Set Project Configuration	Selects project configuration. Choose “default” or “Customize.” See Project Properties window for details.
	Build Project	Builds all the project files.
	>Build for Debugging	Builds all the project files for debug, which includes the debug executive. Step one of Discrete Debugger Operation.
	Clean and Build Project	Deletes files from previous builds and then builds the all project files.
	>Clean and Build for Debugging	Deletes files from previous builds and then Builds all the project files for debug, which includes the debug executive. Step one of Discrete Debugger Operation.
	Run Project	Builds, programs the target and Runs the selected project.
	Make and Program Device Project	Builds, programs the target and holds in Reset the selected project.
	>Program Device for Debugging	Programs the target with the debug build. Step two of Discrete Debugger Operation.
	>Program Device for Production	Programs the target with the production build.
	>Erase Device Memory	Erase device program memory.
	>Programmer To Go PICkit3	For the PICkit 3 as the project debug tool, program the target using Programmer To Go (PTG).
	Read Device Memory	Reads target device memory and loads into MPLAB X IDE.
	>Discrete Multi-Partition Read Operation	For multi-partition (dual partition) devices, read specified flash memory partition.
	>Read Device Memory to a File	Reads target device memory and saves to a file.
	>Read EE/Flash Data Memory to a File	Reads target device data memory and saves to a file.
	Hold in Reset/ Release from Reset	Alternately holds in or releases from Reset the selected project.
	Debug Project	Builds, programs the target and begins debugging the selected project.
	>Launch Debugger	Launch debugger and run target code. Step three of Discrete Debugger Operation.
	>Live Connect Debug	For MEC devices, connect to a running target and inspect the memory contents. Allows some debugging on a production device. For AVR devices (using JTAG if the OCDEN fuse is programmed), connect to a running target and debug. The only difference between debug and Live Connect Debug is the lack of a reset on entry.
> = Available from icon down arrow		

Related Links

12.2.11. Toolbar Features

12.2.12. Customize Toolbars

12.2.6 Debug Toolbar

The Debug Toolbar currently contains button icons for the following functions. These functions are also on the Debug menu.

Icon	Icon Text	Function
	Finish Debugger Session	Ends the debugging session.
	Pause	Pauses debugging. Use “Continue” to resume.
	Reset	Runs the current project to the cursor’s location in the file and stop program execution.
	Continue	Resumes debugging until the next breakpoint or the end of the program is reached.
	Step Over	Executes one source line of a program. If the line is a function call, executes the entire function then stops.
	Step Into	Executes one source line of a program. If the line is a function call, executes the program up to the function’s first statement and stops.
	Step Out	Executes one source line of a program. If the line is a function call, executes the functions and returns control to the caller.
	Run to Cursor	Runs the current project to the cursor’s location in the file and stop program execution.
	Set PC at cursor	Sets the program counter (PC) value to the line address of the cursor.
	Focus Cursor at PC	Moves the cursor to the current PC address and centers this address in the window.

Related Links

12.2.11. Toolbar Features

12.2.2. Clipboard Toolbar

12.2.7 Performance Toolbar

The Performance Toolbar contains:

Item	Item Name	Function
	Memory Usage Display	Click to force garbage collection. This may free up memory.
	Profile the Application	Click to begin profiling the application. Click again to stop profiling and save to a file (`.npss`). See MPLAB® REAL ICE™ in-circuit emulator documentation for more on PC sampling and profiling.

12.2.8 MPLAB X Store Toolbar

The MPLAB X Store Toolbar currently contains button icons for the following functions. The MPLAB X Store function is also on the Help menu.

Icon	Icon Text	Function
	MPLAB X Store	Click to open the **MPLAB X Store** tab in the IDE. Find out about products, services, and order devices.
	Programming Center	Click to order information on the Microchip Programming Center in a browser tab. Let Microchip do your device programming in a fast, cost-effective, secure and proven method.

12.2.9 How Do I Toolbar



This feature requires an Internet connection.

Search for information on the Developer Help. Enter your question in the text box. See also:

microchipdeveloper.com/

12.1.12. Help Menu

12.2.10 Editor Toolbar

The Editor Toolbar currently contains button icons for the following functions. These functions are also on the Edit and Source menus. This toolbar appears at the top of the tab containing the current file source code.

Figure 12-3. Editor Toolbar



Icon	Icon Text	Function
Source	Source	Views source code.
History	History	Views the history of source code edits.
	Online Data Sheet	Opens the project device data sheet to actively lookup Peripherals and SFR's selected (highlighted) in source code. Currently this functionality is for AVR devices only.
	Last Edited	Moves to the line that contains the last edit made.
	Back	Navigates back.
	Forward	Navigates forward.
	Find Selection	Finds the first occurrence of the selected text.
	Find Previous Occurrence	Finds the previous occurrence of the selected text.
	Find Next Occurrence	Finds the next occurrence of the selected text.
	Toggle Highlight Search	Turns on/off text selected for search.
	Toggle Rectangular Selection	Turns on/off text selected lines for search.

.....continued		
Icon	**Icon Text**	**Function**
	Previous Bookmark	Cycles backwards through the bookmarks.
	Next Bookmark	Cycles forward through the bookmarks.
	Toggle Bookmark	Sets a bookmark on a line of code.
	Shift Left	Moves the selected line or lines one tab to the left.
	Shift Right	Moves the selected line or lines one tab to the right.
	Start Macro Recording	Start recording keystrokes.
	Stop Macro Recording	Stop recording keystrokes.
	Comment	Makes selected line into a comment by adding “//”.
	Uncomment	Makes selected comment into a line by removing “//”.
	Go to Header/Source	Moves between the header and related source code.

Related Links

12.2.11. Toolbar Features

12.2.12. Customize Toolbars

12.2.11 Toolbar Features

Toolbars have the following features:

- **Hover** the cursor over an icon to pop up the icon function.
- **Click and drag** the toolbar to another location in the toolbar area.
- **Right click** in the toolbar area to show/hide a toolbar or change the contents of some toolbars.
- Select **View>Toolbars** to show/hide a toolbar, change the contents of some toolbars or create a custom toolbar.

12.2.12 Customize Toolbars

You can customize MPLAB X IDE toolbars using the Customize Toolbars window. Select *View>Toolbars>Customize* to open the window.

Available icons include Clean Only, Run, Set PC to Cursor, etc.

Add a function to a toolbar

- Drag an icon from the Customize Toolbars window to a toolbar.

Remove a function from a toolbar

- Drag an icon from a toolbar to the Customize Toolbars window.

Add your own toolbar

- Click “New Toolbar” and name the new toolbar.

Change toolbar icon size

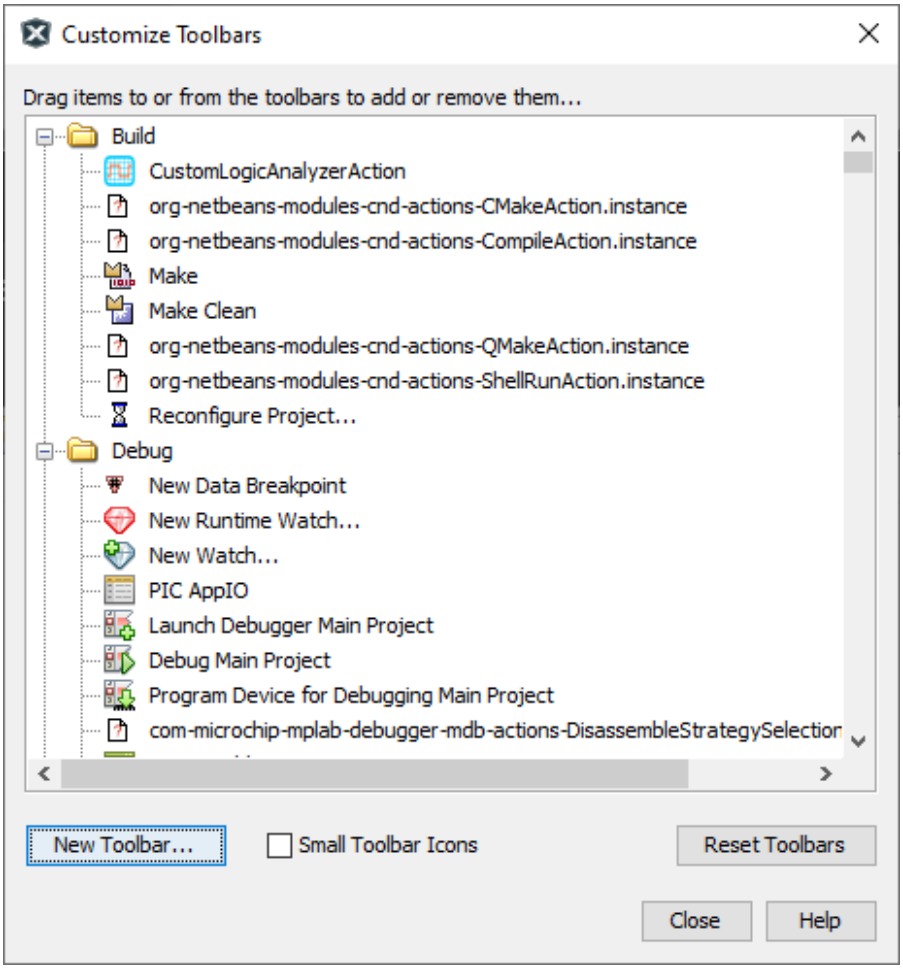
- Click the check box “Small Toolbar Icons” to make the icons smaller.
- Uncheck to make the icons larger.

© 2022 Microchip Technology Inc. and its subsidiaries

Revert to default toolbar

- Click “Reset Toolbars.”

Figure 12-4. Customize Toolbars Window



12.3 Status Bar

The status bar will provide up-to-date information on the status of your MPLAB IDE session. Editor information is also provided.

Figure 12-5. Status Bar During Debug



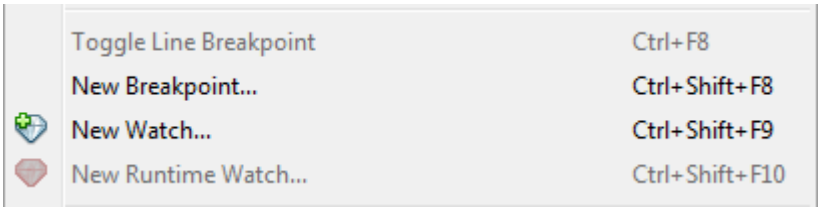
12.4 Grayed Out or Missing Items and Buttons

There are several reasons why a menu item, toolbar button or status bar item may be grayed out (unavailable) or missing:

- The item/button is related to a device feature that the selected device does not have; for example, not all devices support external memory.
- The item/button is related to a tool feature that the selected tool does not have; for example, “Step Out” is not available on all tools.

- The item/button is project-related and no project has been selected; for example, project build will not be available if there is no active project.
- The item/button is not supported for the selected device or tool; for example, hardware trace is only available when using an emulator and device that supports trace.
- The item/button is performing its function and so cannot be selected again; for example, “Debug Project” is grayed out when the program is already running code in debug mode.
- The item/button is mutually exclusive to another item; for example, “Pause” is available when the program is running while “Continue” is grayed out and “Continue” is available when the program is halted while “Pause” is grayed out.

Figure 12-6. Grayed Out Menu Items



13. MPLAB X IDE Windows and Dialogs

The MPLAB X IDE desktop has four sections, or panes, which contain tabbed windows. Windows may not be visible until a feature has been selected. Windows may have associated dialogs for entering specific information.

As an example, when you first open MPLAB X IDE, only the **Start Page** and **MPLAB X Store** tabbed windows are open. After you open a project, the basic windows open – namely Projects and Files in the top left pane, Dashboard and Navigator in the bottom left pane, and Output in the bottom right pane. The **Start Page** and **MPLAB X Store** tabbed windows move into the top right pane.

MPLAB X IDE windows are a combination of basic NetBeans Platform windows and MPLAB X IDE specific windows. Dialogs open when selected from menu items. As with windows, MPLAB X IDE dialogs are a combination of basic NetBeans Platform dialogs and MPLAB X IDE specific dialogs. For information on NetBeans windows and dialogs, see 14.1. NetBeans Specific Windows and Window Menus.

13.1 MPLAB X IDE Windows Behavior

MPLAB X IDE windows each have a specific function to help developers create and debug application code. However, all or some of the windows have features developers need to understand to fully make use each window.

13.1.1 MPLAB X IDE Windows Management

Information on managing IDE windows may be found in the NetBeans help topic:

Managing IDE Windows

Additional window information is provided below.

13.1.1.1 Window Data Updates

Open windows are updated on a program halt (except for Flash memory windows, which must be read). A program halt includes a halt after a run and stepping. Halt updates can have the following effects:

- Speed – Updating takes time. To decrease update time, close any unused windows.
- Data overwrites – The value of a file register displayed in an open window is read on halt. See your device data sheet for the register operation on read.

13.1.1.2 Window Data Changes

MPLAB X IDE window data may be edited as described below. If you cannot edit the data, then that information is not available for you to change.

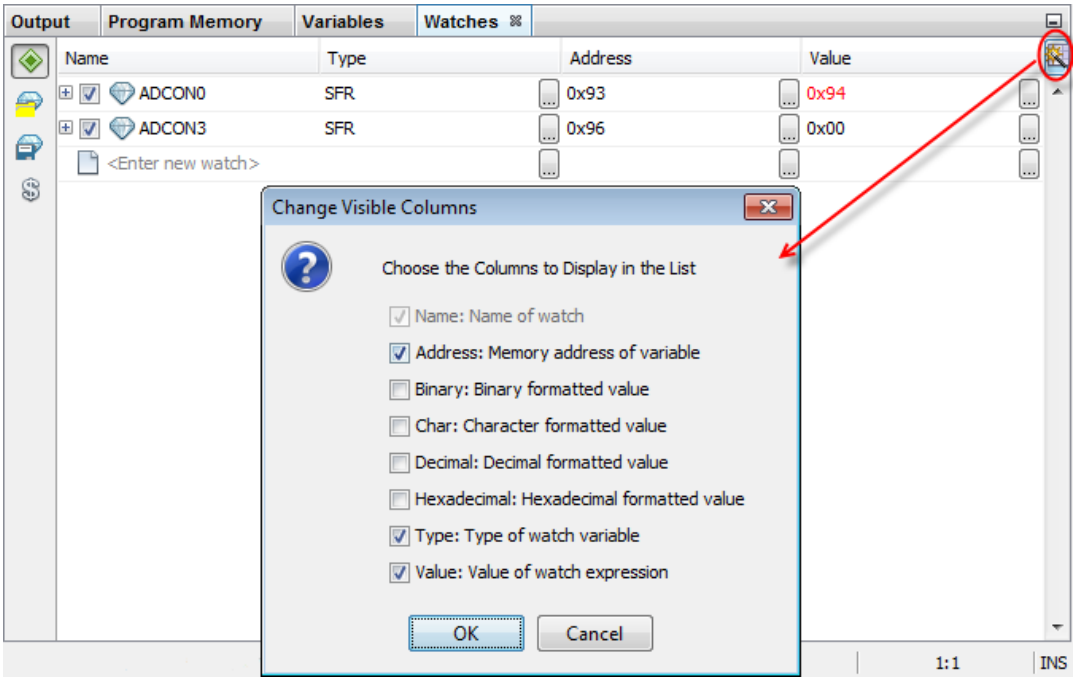
- Data may be edited “in place.” Either double click to select an item and then type in a new value, or click on the ellipsis (...) next to an item and type the new value in the window that pops up.
- Data may be chosen from a drop-down list when only certain choices are possible.

13.1.1.3 Window Column Display

For some windows, you can change the column display. Click on the icon in the window top right corner.

	Change column display.
	Change visible columns.

Figure 13-1. Change Visible Columns Example



13.1.1.4 Window Focus

To ensure that you have a window in focus, click not only on the window frame, but also on a button, a table cell, or a drop-down combo box.

13.1.2 Banked Data Memory and Values Displayed in Windows

This feature applies only to 8-bit PIC MCU devices that use banked Data memory (RAM).

In MPLAB X IDE v5.20 and earlier, a register address always includes the bank offset in the value displayed. However, this is not always correct as the actual value can only be accurately calculated when the instruction is at the program counter (PC).

In MPLAB X IDE v5.25 and later, a register address only includes the bank offset when the PC is on the line with the register address. When the PC is not on that line, the value defaults to the one shown in code (no bank offset or bank 0).

Example 13-1. Behavior Change from v5.20 to v5.25

Consider the example code snipet in the figure below. Display behavior for disassembly code in the Program Memory window (DisAssy colomn) or Disassembly window is shown in the table.

Figure 13-2. Example Code Snipet

```
14      asm("MOVLB 0x3A"); // change to bank 58
15      asm("CLRF 0x74"); // clear common RAM register
```

Table 13-1. Behavior Change from v5.20 to v5.25

MPLAB X IDE	Disassembly of Line 15 - Not at PC	Disassembly of Line 15 - At PC
v5.20 and earlier	`CLRF 0x1D74`	`CLRF 0x1D74`
v5.25 and later	`CLRF 0x74`	`CLRF 0x1D74`

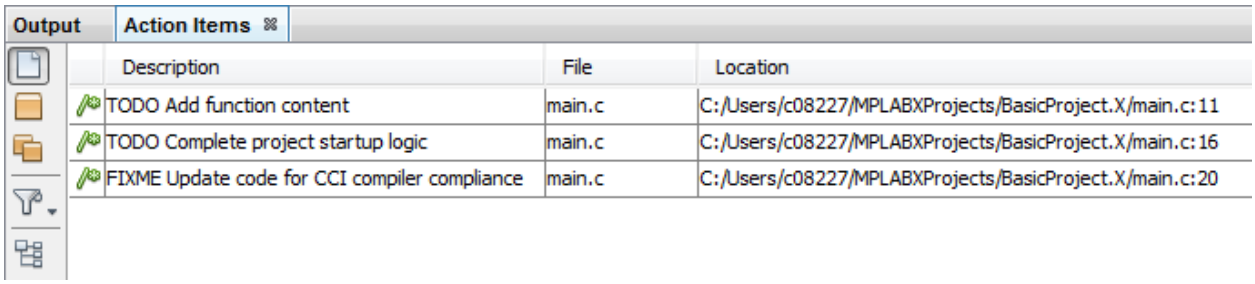
13.2 Action Items Window

Open the Action Item window from *Window>Action Items*.

The Action Items window displays a unified list of things that you need to resolve in your various files and projects, including compiler warnings and errors. Place comments in your code to specify action items. For example, in the figure below the following lines of code were used:

```
//TODO Add function content
        :
//TODO Complete project startup logic
        :
//FIXME Update code for CCI compiler compliance
```

Figure 13-3. Action Items Window Example



You can use filters to determine the entries that are displayed in the list and you can sort the list by clicking a column heading. The Action Items window icons enables you to modify and work with the displayed entries.

For more information, see: microchipdeveloper.com/mplabx:tasks-list

13.2.1 Action Items Window Display

This display format shows data in the following columns:

- Description – A description of the action required.
- File – The name of the file action.
- Location – The file location.

The Action Items window icons are displayed on the left side of the window.

Table 13-2. Action Items Window Icons

Icon	Icon Text	Description
	Show action items for currently edited file only	When selected, the IDE scans the file that is currently selected in the editor and displays a list of the action items found in that file. When you open a new file in the editor, the file is scanned and only the action items in that file are displayed.
	Show action items for the selected project	When selected, the IDE scans all the files in the currently selected project and displays all the action items in the project window.
	Show action items for all opened projects	When selected, the IDE scans all the files in all open projects and displays a list of all the action items in the window.
	Click here to select a filter	Click the Filter button to modify the filters for the action items or to create a new filter. You can click the drop-down list on this button to select a filter to apply to the list of action items.
	Group action items by category	When selected, the list of action items is grouped by category.

13.2.2 Action Items Window Menu

Select and right click on a row in the window to pop up this menu.

Table 13-3. Action Items Window Context Menu

Item	Description*
Show	Show the location in code of the error/warning.
Scope	Show the scope of the item - current file, current project, all projects.
Filter	Show how the item is filtered. Also change filtering.
Refresh	Refresh the window contents.
Sort By	Specify how the window contents should be arranged.

13.3 Breakpoints Window

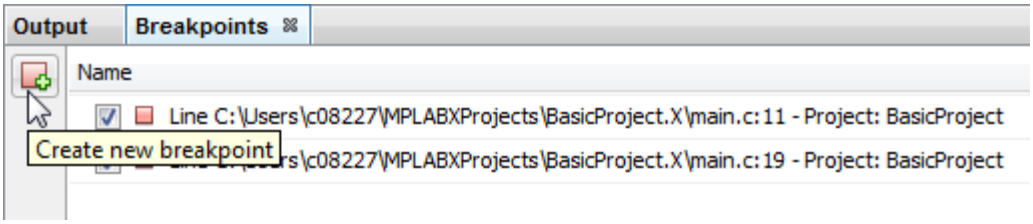
Open the window by selecting *Window>Debugging>Breakpoints*.

The Breakpoints window is used for managing breakpoints in code.

New Breakpoint Dialog

Click the **Create New Breakpoint** button on the Breakpoint window to open the Breakpoint dialog and create a new breakpoint. Types of breakpoints allowable for the project device are listed.

Figure 13-4. New Breakpoint Icon



Breakpoint Window Menus

Right click in a blank area of the window to pop the menu below.

Table 13-4. Breakpoints Window Menu - No Breakpoint Line Selected

Item	Description
New Breakpoint	Open the New Breakpoint dialog.
Enable All Disable All	Enable or disable all breakpoints.
Delete All	Delete all breakpoints.

Right click on an existing breakpoint row in the window to pop the menu below.

Table 13-5. Breakpoints Window Menu - Breakpoint Line Selected

Item	Description
Go to Source	Show the location in code of the breakpoint.
Complex Breakpoint	For a complex breakpoint, add this breakpoint to a new sequence or tuple. Also specify location in sequence or tuple.
Disable Enable	Disable or re-enable the operation of this breakpoint.
New Breakpoint	Open the New Breakpoint dialog.

© 2022 Microchip Technology Inc. and its subsidiaries

.....continued	
Item	**Description**
Enable All Disable All	Enable or disable all breakpoints.
Delete Delete All	Delete this or all breakpoints.
Customize	Customize the breakpoint behavior.

Related Links

5.14. Control Program Execution with Breakpoints

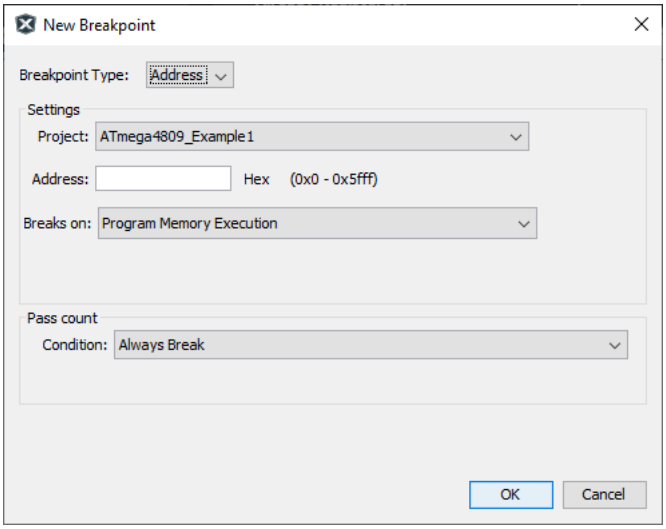
5.14.3. Set Breakpoints with the Breakpoint Dialog

13.4 New Breakpoint Dialog

Open this dialog from the Breakpoints Window (*Window>Debugging>Breakpoints*). Either click the "New Breakpoint" icon  or select "New Breakpoint" from a context menu.

The options available for the breakpoint are determined by the type of breakpoint selected and the selected device (not all options are available for all devices).

Figure 13-5. New Breakpoint Dialog - Address



13.4.1 Line Breakpoints

The following options are available for breakpoints specified on a line of code.

Table 13-6. Breakpoint Type: Line – Settings

Item	Description
Settings	Create a new Line breakpoint with a left mouse click on the Editor gutter next to the file line. Or, select "Toggle Line Breakpoint" from the Editor context menu.

13.4.2 Data Breakpoints

The following options are available for data memory breakpoints.

Table 13-7. Breakpoint Type: Data – Settings

Item	Description
Project	Select an open project from the drop-down list This is the project in which code will contain the breakpoint.
Symbols	Enter the name of a global symbol, or SFR, or browse to one by clicking **Symbols**. Accessing this symbol according to “Breaks on” triggers a pause in code execution.
Enable Range Address	Check to set a range breakpoint. Uncheck to set a single breakpoint.
Address **Address (Start)**	Depending on the selection of “Enable Range Address”, this item can be “Address” or “Address (Start)”. Enter a hexadecimal address in data memory Accessing this address according to “Breaks on” triggers a pause in code execution.
Address (End)	If “Enable Range Address” is checked, this box is enabled. Enter a hexadecimal address in data memory.
Breaks on	**Read, Write, Read or Write**: Break code execution when the symbol or address stated above is read, written, or either read or written. **Read Specific Value, Write Specific Value, Read or Write Specific Value**: Break code execution when the symbol or address stated above is read and has the value specified below, written with the value specified below, or either read or written according to the value specified below. **Note:** For dsPIC DSCs, there are read and write options for the X and Y buses.
Value	For the “Breaks on” selection of Read Specific Value, Write Specific Value, or Read or Write Specific Value, enter a hexadecimal value here.
Value Comparison	*For PIC16F1xxx devices only*- Compare to “Value” as specified: **= Value**: Equal to value **!= Value**: Not equal to value **> Value**: Greater than value **< Value**: Less than value
Data Value Mask	*For PIC16F1xxx devices only*- Use mask when comparing to “Value” Enter a value in the range 0x00 to 0xhh, where: 0x00: No bits compared 0xhh: All bits compared

Table 13-8. Breakpoint Type: Data – Pass Count

Item	Description
Condition	Determine when the break specified under “Breaks on” occurs. **Always Break**: Always break when the “Breaks on” condition is met. **Break occurs Count Instructions after Event**: After an event (“Breaks on” condition) occurs, execute Count Instructions before actually breaking. **Event must occur Count times**: An event (“Breaks on” condition) must occur Count times before actually breaking.

.....continued	
Item	**Description**
Count	According to the Condition specified, enter either a count for the number of instructions after an event or the number of events. See 13.4.4. Event Breakpoint.
Trigger Options	*For PIC16F1xxx devices only-* Select when to trigger, either: • Do not trigger out when breakpoint is reached • trigger out when breakpoint is reached
Interrupt Context	*For PIC16F1xxx devices only-* Interrupt Context qualifier for address/data breakpoints. Select from: • Always break (break in both ISR and main code) • Break in main line (non-interrupt) context only – break in main code only • Break in interrupt context only – break in ISR code only

13.4.3 Address Breakpoints

The following options are available for program/execution memory breakpoints.

Table 13-9. Breakpoint Type: Address – Settings

Item	Description
Project	Select an open project from the drop-down list. This is the project in which code will contain the breakpoint.
Enable Range Address	Check to set a range breakpoint. Uncheck to set a single breakpoint.
Address **Address (Start)**	Depending on the selection of “Enable Range Address,” this item can be “Address” or “Address (Start).” Enter a hexadecimal address in data memory. Accessing this address according to “Breaks on” triggers a pause in code execution.
Address (End)	If “Enable Range Address” is checked, this box is enabled. Enter a hexadecimal address in data memory.
Breaks on	**Program Memory Execution**: Break code execution when the address specified above is reached. **TBLRD Program Memory**: Break code execution when a table read to the address specified above occurs. **TBLWT Program Memory**: Break code execution when a table write to the address specified above occurs.

Table 13-10. Breakpoint Type: Data – Pass Count

Item	Description
Condition	Determine when the break specified under “Breaks on” occurs. **Always Break**: Always break when the “Breaks on” condition is met. **Break occurs Count Instructions after Event**: After an event (“Breaks on” condition) occurs, execute Count Instructions before actually breaking. **Event must occur Count times**: An event (“Breaks on” condition) must occur Count times before actually breaking.

.....continued	
Item	**Description**
Count	According to the Condition specified, enter either a count for the number of instructions after an event or the number of events. See 13.4.4. Event Breakpoint.
Trigger Options	*For PIC16F1xxx devices only-* Select when to trigger, either: • Do not trigger out when breakpoint is reached • trigger out when breakpoint is reached
Interrupt Context	*For PIC16F1xxx devices only-* Interrupt Context qualifier for address/data breakpoints. Select from: • Always break (break in both ISR and main code) • Break in main line (non-interrupt) context only – break in main code only • Break in interrupt context only – break in ISR code only
* See also Section “For some devices (PIC16F1xxx MCUs), enhanced event breakpoints actions are available.”	

13.4.4 Event Breakpoint

The following options are available for event breakpoints.

Table 13-11. Breakpoint Type: Event

Item*	Description
Project	Select an open project from the drop down list. This is the project whose code will contain the breakpoint.
Break on clock mode switch	Break when the clock mode switches.
Break on Reset instruction	Break when a device Reset instruction occurs.
Break on Sleep	Break when Sleep is entered.
Break on stack over/underflow	Break when the stack either overflows or underflows.
Break on wake up	Break when the device wakes up from sleep.
Break when watchdog timer has expired	Break when the watchdog timer period has ended.
Break on execution out of bounds	Break when the program attempts to move out of normal program memory/space.
Break on MCLR Reset	Break on a Master Clear (MCLR) Reset.
Break on trigger in signal	Break when a Trigger In pulse is detected.
* Select event(s) from the list that will cause executing code to pause (break). Some events may not be available for your device.	

For some devices (PIC16F1xxx MCUs), enhanced event breakpoints actions are available.

Action	Description
Break	Break (halt) execution per option specified.
Trigger out	Emit a trigger out pulse per option specified.
Break and trigger out	Break (halt) execution and emit a trigger out pulse per option specified.

13.4.5 Function Breakpoints

The following options are available for function breakpoints.

Table 13-12. Breakpoint Type: Function – Settings

Item	Description
Symbol/Hex Address	Enter or select either a symbol representation or address for a function.

Table 13-13. Breakpoint Type: Function – Conditions

Item	Description
Condition	Check to enable. Then enter a break condition.
Break when hit count exceeds	Check to enable. Then enter an integer number to represent the number of times the function is to be entered, after which a break occurs.

Table 13-14. Breakpoint Type: Function – Actions

Item	Description
Suspend	On break, suspend: **No thread (continue)**: No program suspension. **Breakpoint thread**: Only suspend the function thread. **All threads**: Suspend all threads (not main program).
Thread ID	Set ID of breakpoint thread.
Print Text	Enter text to be displayed in the Output window when a break occurs.

13.4.6 Pass Count Operation

Using a pass count allows you to delay breaking until after a specified count.

Break occurs Count Instructions after Event

Count is the number of instructions that execution passes after the breakpoint but before stopping.

For example,

- 0 specifies that execution stops immediately
- 1 specifies that execution stops after one additional instruction
- 10 specifies that execution stops after ten additional instructions

Event must occur Count times

Count is the number of times that execution passes the event until stopping.

For example,

- 0 specifies that execution stops immediately
- 1 specifies that execution passes the event one time, then stops the next time (the second time the event occurs)
- 10 specifies that execution passes the event ten times, then stops the next time (the eleventh time the event occurs)

13.5 Call Stack Window

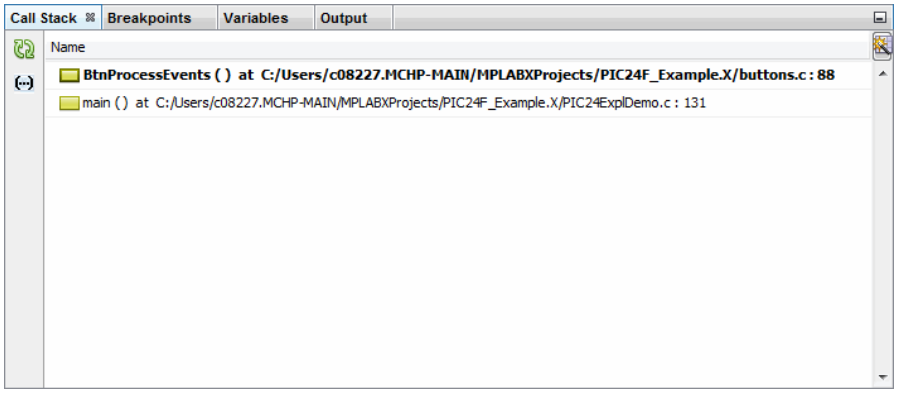
Select *Window>Debugging>Call Stack* to open the Call Stack window. You must be in debug mode to view content.

The Call Stack window displays C functions and their arguments listed in the order in which they were called in the executing program. C code that is not optimized works best.

The Call Stack is available for 16- and 32-bit devices. The window is updated on Debug Pause or Halt.

© 2022 Microchip Technology Inc. and its subsidiaries

Figure 13-6. Call Stack Window



The following icons are available on the window on Pause or Halt.

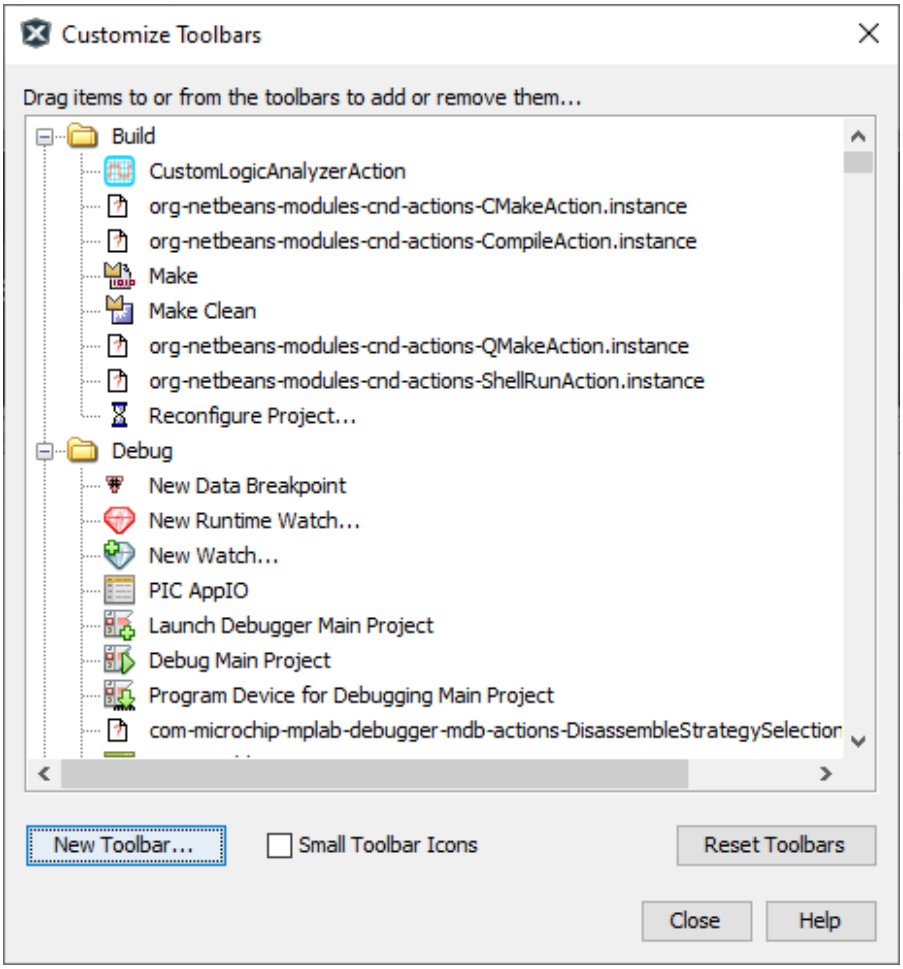
Table 13-15. Call Stack Icons

Icon	Description
	Auto or Manual Refresh of Window Contents. Depends on the value of "Disable auto refresh for call stack view during debug sessions" under *Tools>Options>Embedded>Generic Settings* – Unchecked = false (default), checked = true. See 13.20.1. Generic Settings Tab. False = Auto Refresh (Green Icon): The window contents are automatically updated on Pause or Halt. If the window is closed or not focused, selecting the window and clicking this button will display the updates without having to run and pause/halt again. True = Manual Refresh (Orange Icon): The window contents are not automatically updated on Pause or Halt. This button must be clicked to update window contents on a pause/halt.
	Select to disable/enable evaluation of function parameter variables.
	Change visible columns. For this window, show or hide location of call stack frame.

13.6 Customize Toolbars Window

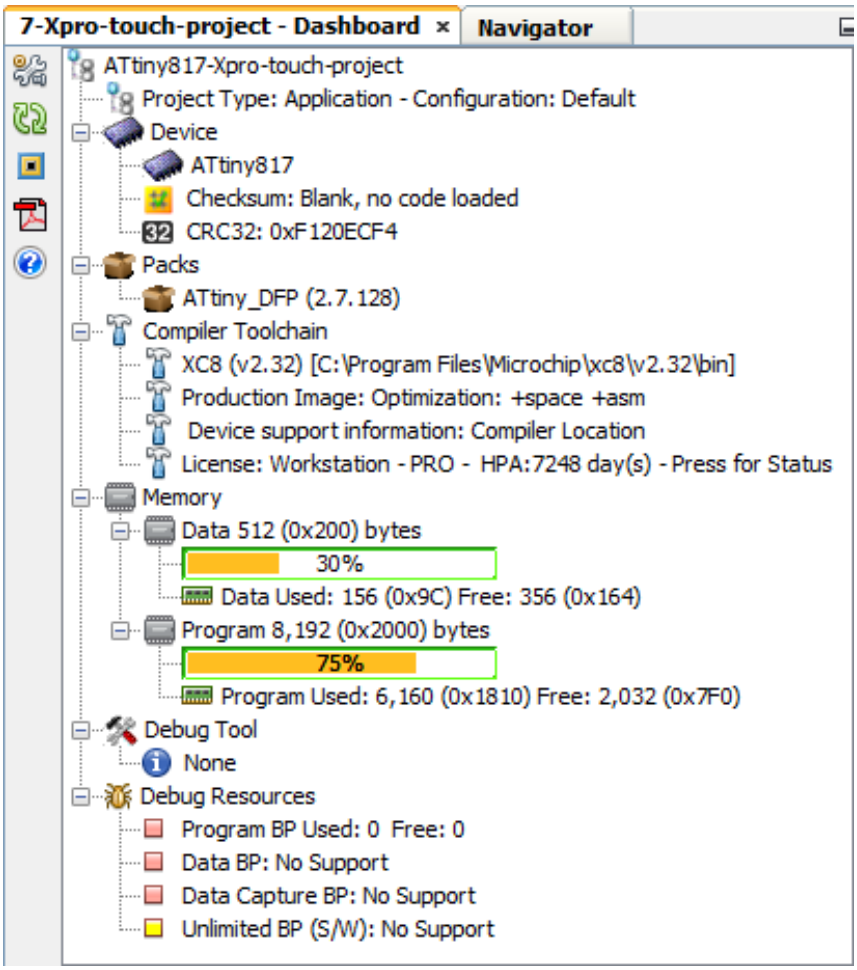
You can customize MPLAB X IDE toolbars using the Customize Toolbars window. Select *View>Toolbars>Customize* to open the window.

For customization instructions, see 12.2.12. Customize Toolbars.



13.7 Dashboard Window

The Dashboard window contains general project information, such as checksums, compiler versions, memory usage, and breakpoint resources. For more information, see: 8.5. View the Dashboard Display.



13.8 Disassembly Window

View disassembled C code in the Disassembly window. Select *Window>Debugging>Disassembly* to open this window. You must be in debug mode.

When debugging, the "Step Over" function works in the source code context. Disassembly debugging is done using the "Step In" function. If during Disassembly debugging "Step In" is not needed - for example in cases where a CALL instruction needs to be stepped over and not stepped in - this can be achieved by using the "Run to Cursor" feature. Place the cursor at the line where PC needs to be run to and hit the "Run to Cursor"/F4 button.

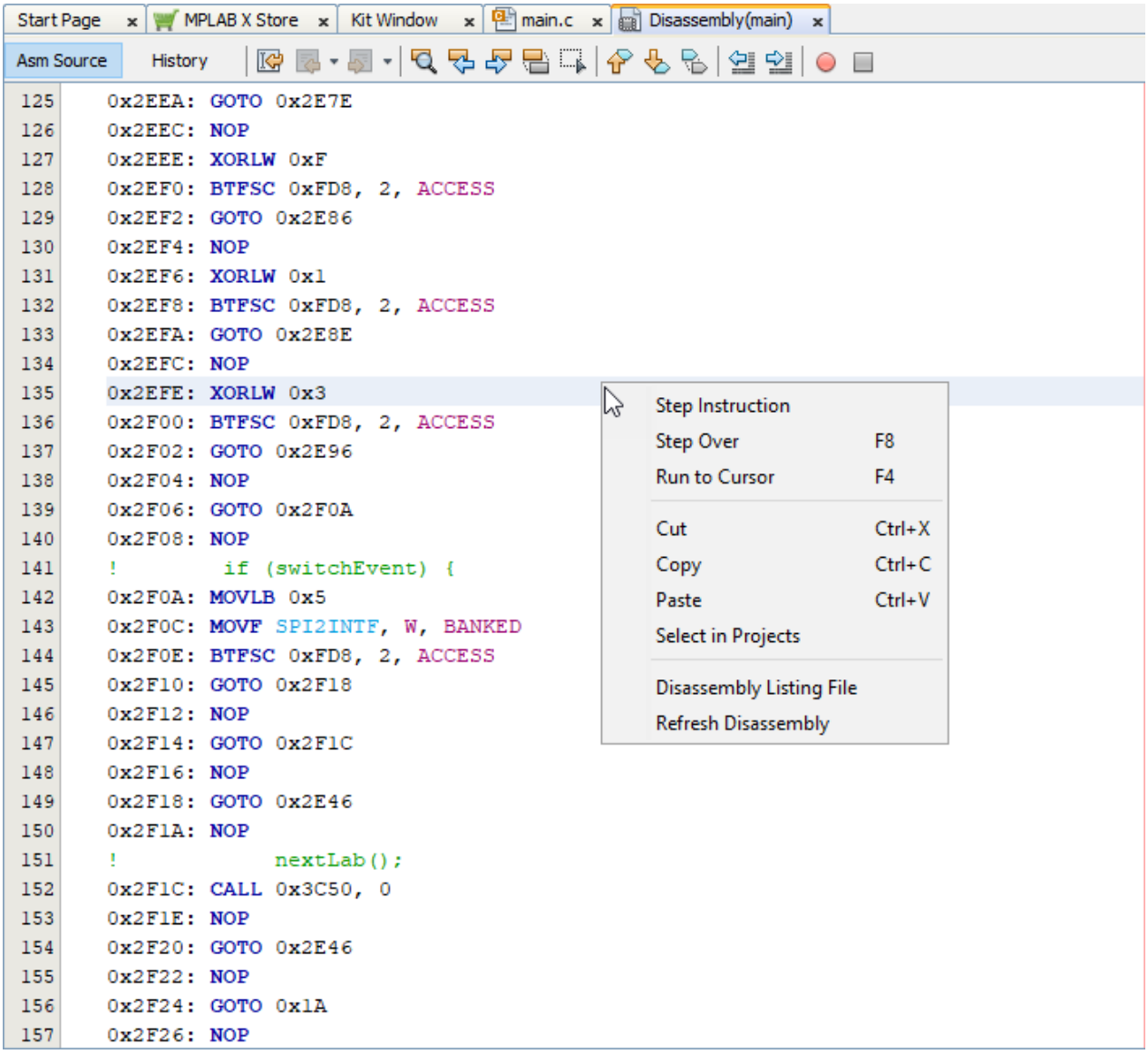
Right click in the window to see the context menu.

A quick way to view the entire disassembly file is to select “Disassembly Listing File.” This file is created by the compiler assembler and contains the mapping between the original source code and the generated assembly code. There is one list file produced for the entire program, including library code.

If your project device uses RAM banking, see also:

13.1.2. Banked Data Memory and Values Displayed in Windows

Figure 13-7. Disassembly Window



13.9 I/O View Window

Note: Currently this window is only available for

Use the I/O View window (*Window>Debugging>IO View*) to see an overview of registers of the target device for the current project. For an overview of operation, see:

5.18. View Registers for the Project (I/O View)

I/O View Window Sections

The default view of the window is vertically split with peripheral groups in the top section and registers in the bottom section. Each peripheral typically has defined settings and value enumerations which can be displayed by expanding a register in the peripheral view (top section). The register view (bottom section) will display all registers which belong to a selected peripheral group. If no peripheral is selected, the view is empty. Each register can also be expanded to display the predefined value groupings that belong to the register.

© 2022 Microchip Technology Inc. and its subsidiaries

Figure 13-8. I/O View Window with Content

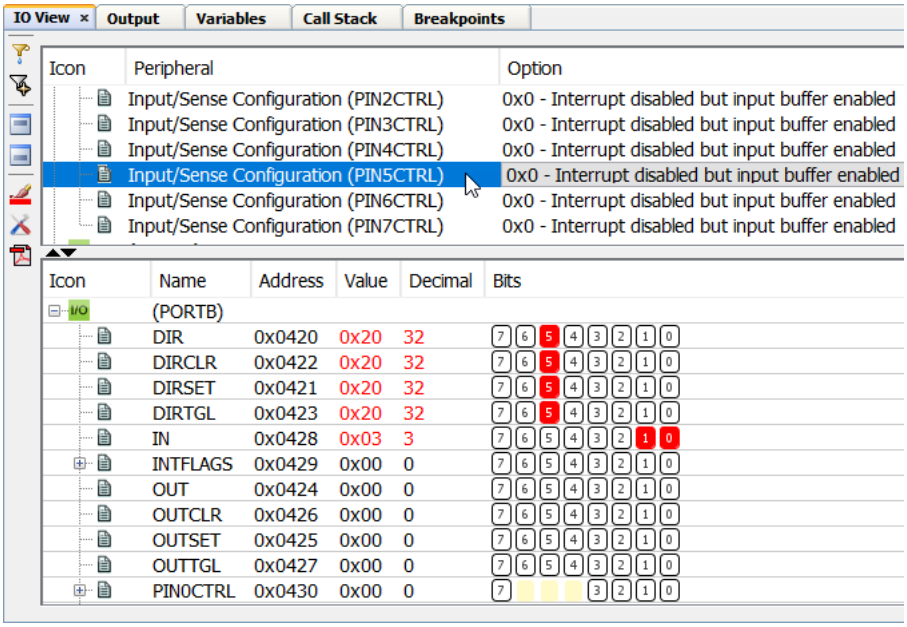


Table 13-16. I/O View Icons

Icon	Function
	Filter Peripherals. Filter by groups of peripherals available in view. Click icon to open the Select dialog. Select peripherals using Shift-click (range) or Ctrl-click (individually). Alternately you can specify a Register Text Filter file.
	Select All Peripherals. If peripherals have previously been filtered, this will undo that filter.
	Toggle Peripheral Pane. Click to show/hide peripheral (top) pane.
	Toggle Register Pane. Click to show/hide register (bottom) pane.
	Blue icon: Show Standard Bits. Standard bits (no highlighting) are displayed for modified bits. Red icon: Highlight Modified Bits. Modified bits are highlighted in display. Click on one icon to toggle to the other icon.
	Settings. Click to open the IO View Settings dialog.
	Online Datasheet. Click to open the online data sheet for the project device and selected register. If the icon is grayed out, ensure you have selected a register in either pane. If a register is selected and the icon is still gray, the data sheet may be unavailable online.

Table 13-17. I/O View Context Menus

Item	Function	Peripheral Pane	Register Pane
Expand Row	Expand collapsed content in selected row.	x	
Expand All	Expand all collapsed content in pane/section.	x	x

.....continued			
Item	**Function**	**Peripheral Pane**	**Register Pane**
Collapse All	Collapse all expanded content in pane/section.	x	x
Online Datasheet	Online Datasheet. Click to open the online data sheet for the project device.	x	x
Filter Peripherals	Click to open the Select dialog. Select peripherals using Shift-click (range) or Ctrl-click (individually). Alternately you can specify a Register Text Filter file.	x	
Select All Peripherals	If peripherals have previously been filtered, this will undo that filter.	x	
Copy Address	Copy the address of the selected register.		x
Adjust Table Columns	Register Pane Only. Adjust width of table columns.	x	x

13.10 Licenses Windows

To manage compiler licenses, select *Tools>Licenses* and select an action from the pop up menu.

For more on licensing compilers, see the MPLAB® XC Compilers webpage, Documentation tab, Documentation section.

13.10.1 Roam Network License

Select *Tools>Licenses>Roam Network License* to roam a network license out to your computer (Get) and than to roam a network license back in to the network server (Release).

See also the “MPLAB XC License Server Manual” (DS-50002334).

Figure 13-9. Roam License Out

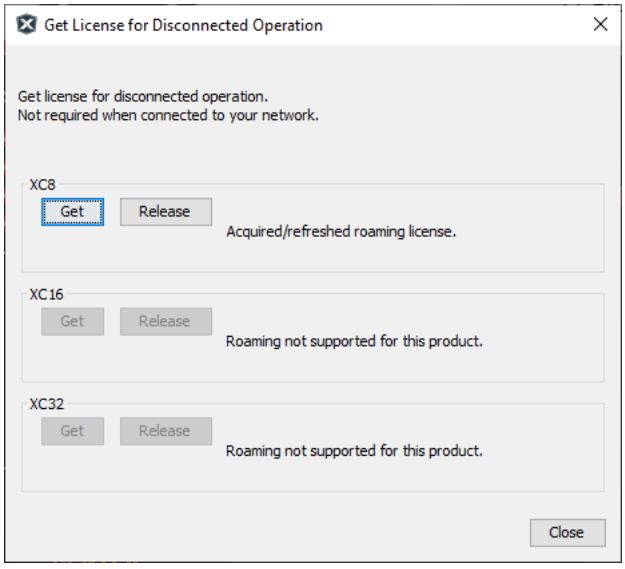
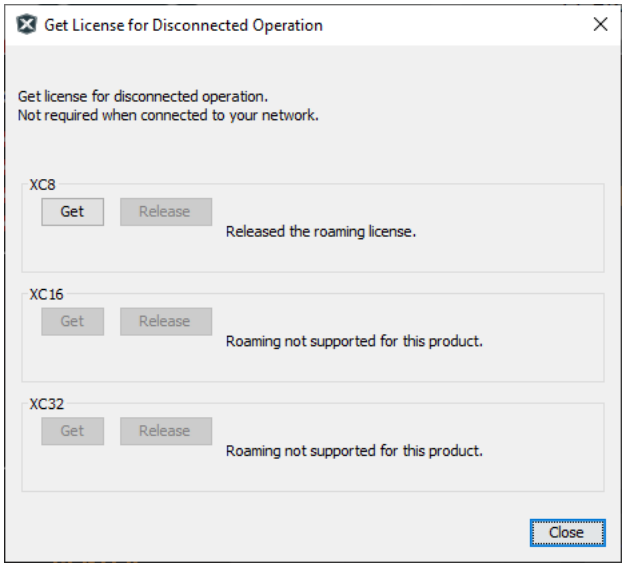


Figure 13-10. Roam License In



13.10.2 Activate Workstation License

Select *Tools>Licenses>Activate Workstation License* to go to the MySoftware webpage where you can log in, locate a purchased Workstation license, and activate it. For details, see "*Installing and Licensing MPLAB XC C Compilers*" (DS-50002059).

13.10.3 Change Licensing Type

Select *Tools>Licenses>Change Licensing Type* to change the type of license for a compiler. To change to another license type, you must already have purchased, installed and activated that license. To see the types of licenses currently available, see the "License status" view.

Note: If using CI/CD Wizard, Network information entered here will auto-populate in the build wizard dialog.

© 2022 Microchip Technology Inc. and its subsidiaries

Figure 13-11. Change Licensing Type from Workstation

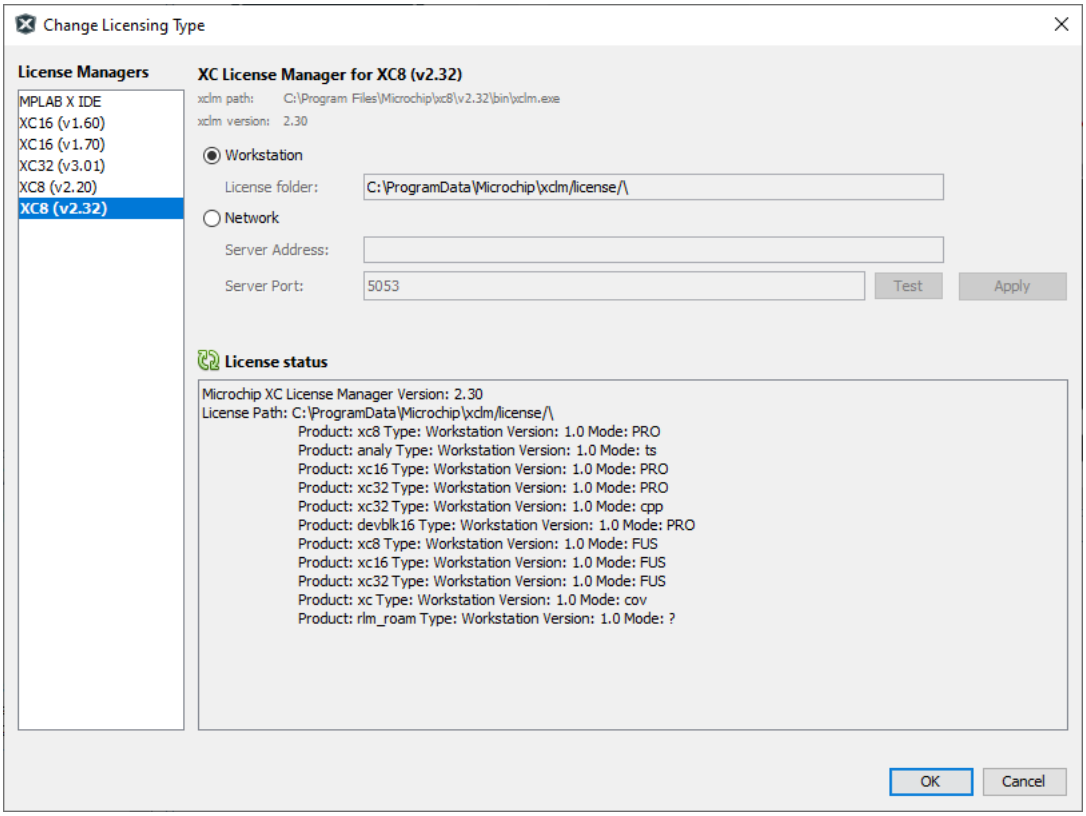
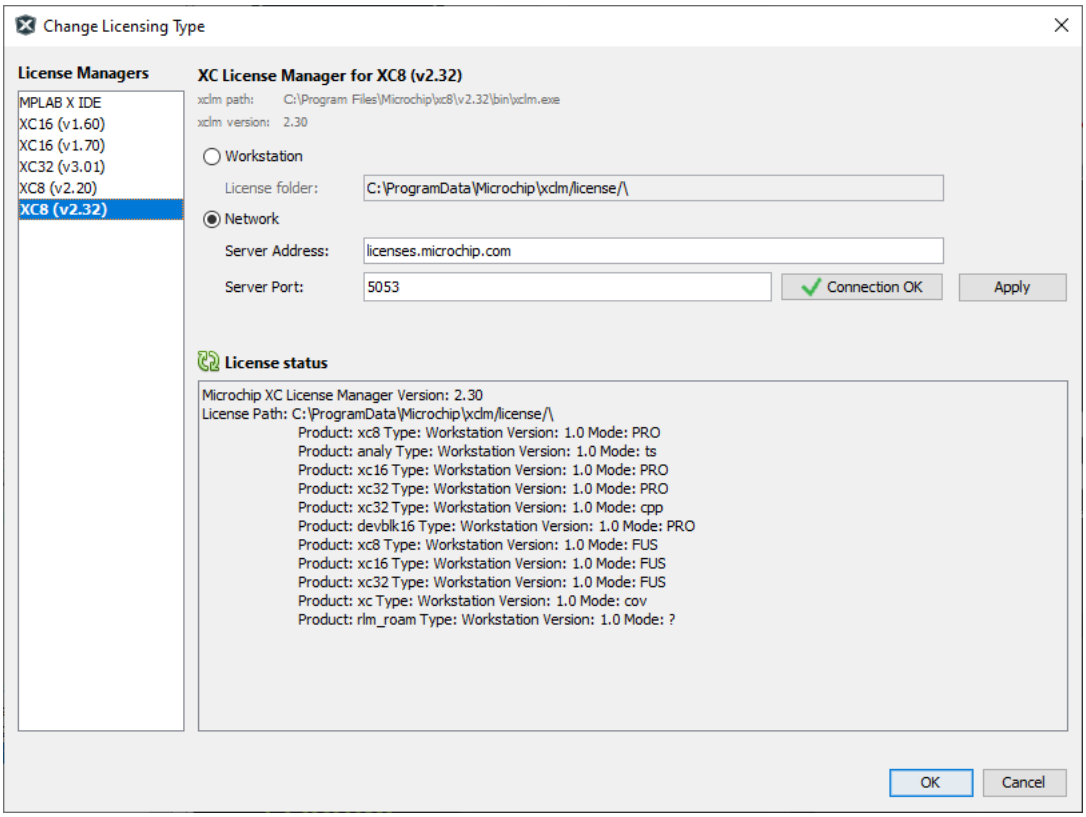


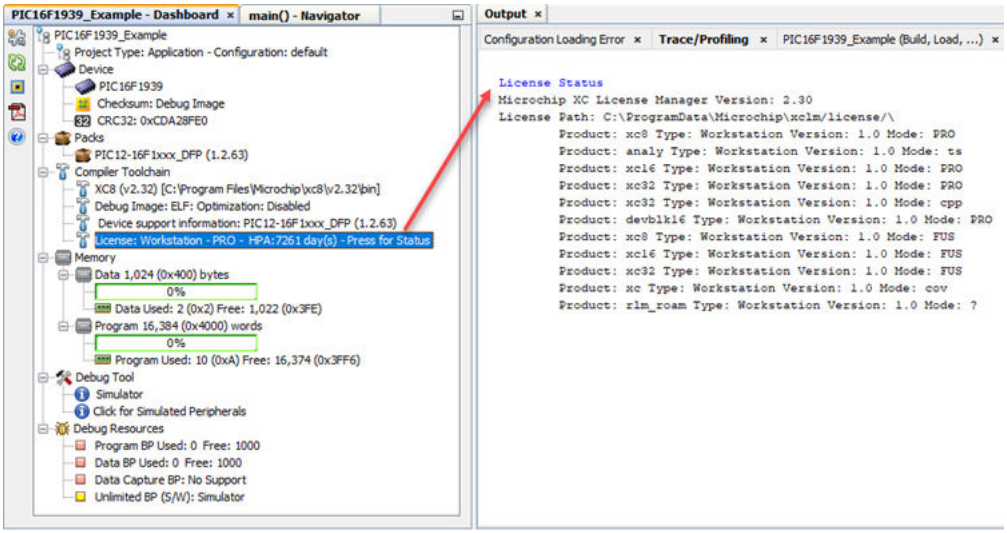
Figure 13-12. Change Licensing Type to Network



13.10.4 License Status

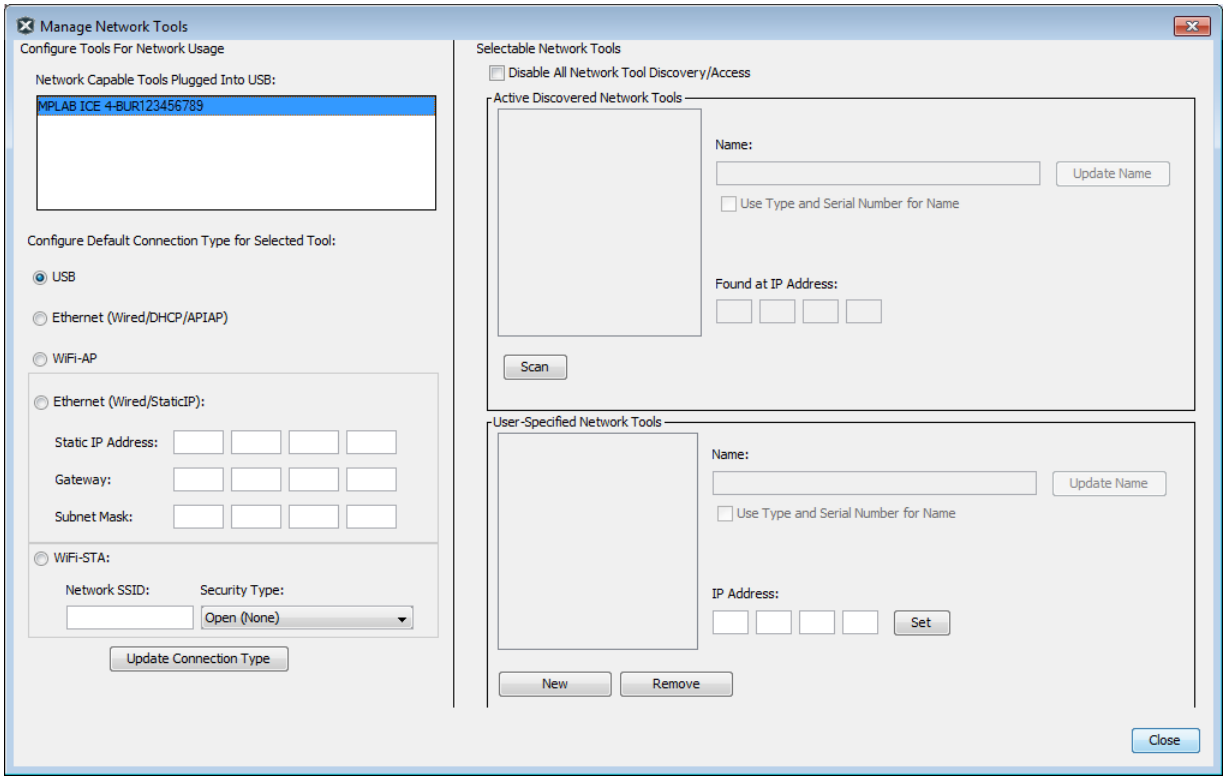
Another way to check compiler license status is through the Dashboard.

Figure 13-13. License Status Through Dashboard



13.11 Manage Network Tools Dialog

The Manage Network Tools (MNT) dialog is used for development tools with network capabilities, currently only MPLAB ICE 4 in-circuit emulator. Please see MPLAB ICE 4 documentation for more on how to use this dialog.



13.12 Memory Windows - 8- and 16-Bit Devices

Memory windows (*Window>Target Memory Views*) display the many types of device memory, such as SFRs and Configuration bits. Use the “Memory” and “Format” drop-down boxes to customize your window.

Some memory windows have additional functionality through associated dialogs.

Figure 13-14. Window>Target Memory Views - PIC16F1939

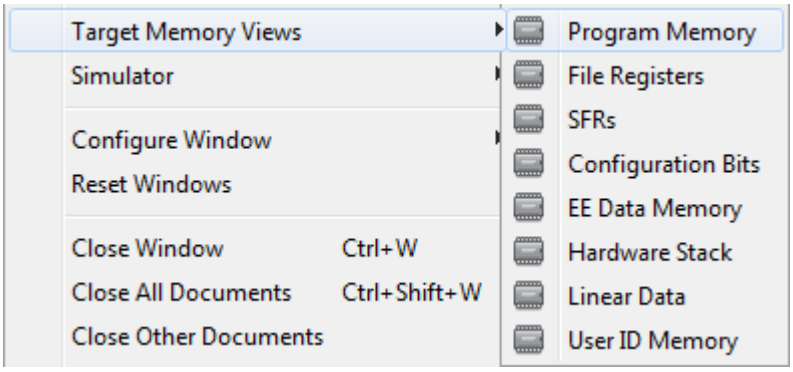
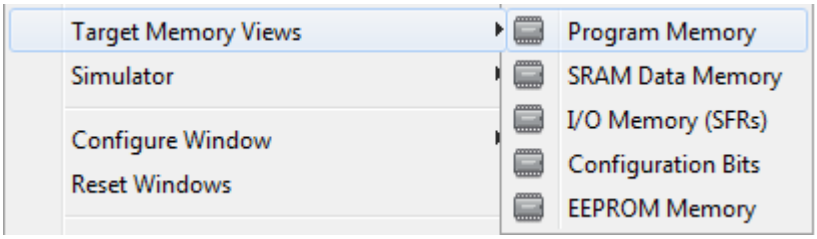


Figure 13-15. Window>Target Memory Views - ATtiny817



Related Links

13.14. Memory Windows Associated Dialogs

5.19. View or Change Device Memory

13.12.1 Program Memory Window

The Program Memory window displays locations in the range of program memory for the currently selected processor. If external program memory is supported by the selected device and enabled, it will also appear in the Program Memory window.

For the MPLAB X Simulator, when a program memory value changes or the processor is halted, the data in the Program Memory window is updated.

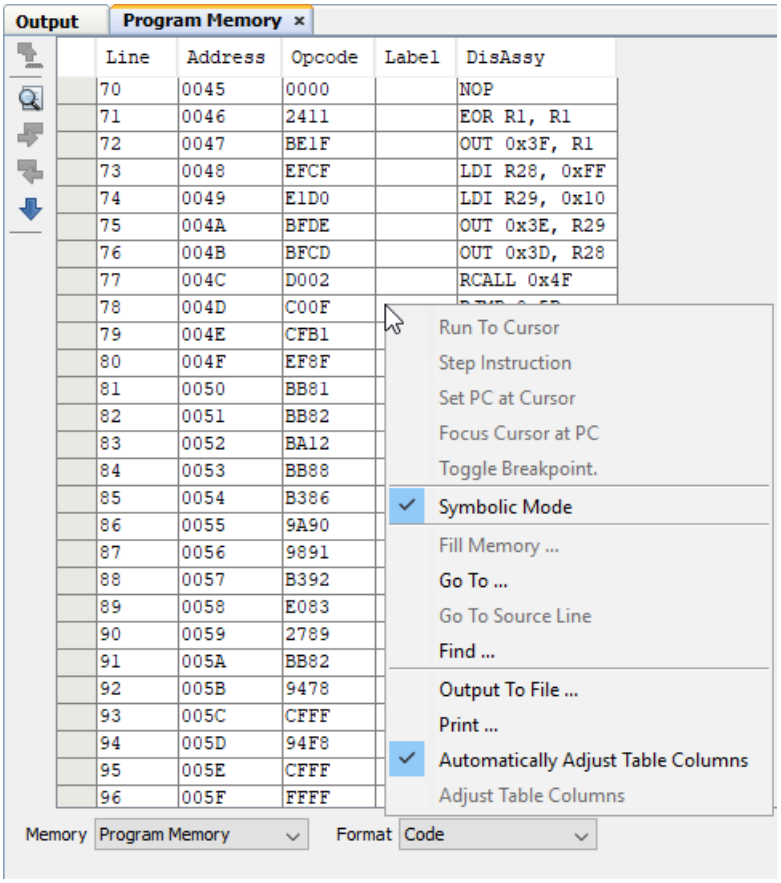
For any Microchip hardware debug tool (e.g., MPLAB ICD 4 in-circuit debugger), when a program memory value changes or the processor is halted, the data in the Program Memory window is **not** updated; you must do a Read of device memory.

When using a hardware tool for debug, some registers may show an “R” for each nibble of data to represent a reserved resource.

If your project device uses RAM banking, see also:

13.1.2. Banked Data Memory and Values Displayed in Windows

Figure 13-16. Program Memory Window with Content



Related Links

13.1.2. Banked Data Memory and Values Displayed in Windows

13.12.1.1 Program Memory Window Displays

You may specify how memory is displayed in the window by selecting from the Format drop-down box on the bottom of the window.

When using a hardware tool for debug, some registers may show an “R” for each nibble of data to represent a reserved resource.

Code Format

Code format displays disassembled hex code with symbols. The window will have the following columns:

- Debug Info – Information useful for debugging
  . A pointer shows the current location of the program counter.
- Line – Reference line number.
- Address – Opcode hexadecimal address.
- Opcode – Hexadecimal opcode, shown in 2- or 3-byte blocks
  For most PIC MCUs, these blocks represent words.
  For PIC18CXXX devices, the blocks represent 2 bytes.
  For dsPIC DSC devices, the blocks represent 3 bytes.
- Label (Symbolic Only) – Opcode label in symbolic format.
- Disassembly – A disassembled version of the opcode mnemonic.

Hex Format

This format displays program memory information as hex code. The window will have the following columns:

- Address – Hexadecimal address of the opcode in the next column.
- Opcode Blocks – Hexadecimal opcode, shown in 2- or 3-byte blocks
  For most PIC MCUs these blocks represent words. For PIC18CXXX devices, the blocks represent 2 bytes. For dsPIC DSC devices, the blocks represent 3 bytes.
  The opcode block that is highlighted represents the current location of the program counter.
- ASCII – ASCII representation of the corresponding line of opcode.

PSV Mixed (dsPIC DSC/PIC24 devices only)

This format displays program memory as opcode and the PSV area (CORCON register, PSV bit set). The window will have the following columns:

- Debug Info – Information useful for debugging. A pointer shows the current location of the program counter.
- Line – Reference line number.
- Address – Opcode hexadecimal address.
- PSV Address – Data space hexadecimal address of the opcode.
- Data – Opcode formatted as data.
- Opcode – Hexadecimal opcode, shown in 3-byte blocks.
- Label – Opcode label in symbolic format.
- Disassembly – A disassembled version of the opcode mnemonic.

For more information, see the dsPIC30F Family Reference Manual (DS70046).

PSV Data (dsPIC DSC/PIC24 devices only)

This format displays program memory as file registers, for when program space is visible in data space (CORCON register, PSV bit set). The window will have the following columns:

- Address – Program space hexadecimal address of the data.
- PSV Address – Data space hexadecimal address of the data.
- Data Blocks – Hexadecimal data, shown in 3-byte blocks
  . The highlighted data block represents the current location of the program counter.
- ASCII – ASCII representation of the corresponding line of data.

For more information, see the dsPIC30F Family Reference Manual (DS70046).

13.12.1.2 Program Memory Window Icons

Icons are located on the left side of the window.

Table 13-18. Program Memory Window Icons

Icon	Icon Text	Function
	Refresh by Read Device Memory	Same function as the Debug toolbar “Read Device Memory” icon - uploads device memory to the MPLAB X IDE.
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.
	Go To	Go to specified line number or address.

13.12.1.3 Program Memory Window Menus

Right click in the memory window data area set to CODE OR PSV MIXED FORMAT to pop up this menu. Not all items may be visible for all devices.

Table 13-19. Program Memory Window Menu - Code/PSV Mixed

Item	Description
Run to Cursor	Run the program to the current cursor location.
Step Instruction	Perform a step instruction.
Set PC at Cursor	Set the Program Counter (PC) to the cursor location.
Focus Cursor at PC	Move the cursor to the current PC address and centers this address in the window.
Toggle Breakpoint	Toggle (on/off) existing breakpoint.
Symbolic Mode	Display disassembled hex code with symbols.
Verbose Labels	**PSV Mixed Format Only** Show internal compiler labels.
Fill Memory	Fill memory from Start Address to End Address with the value in Data. Specify other options in the Fill Memory dialog.
Go To	Go to the address/function specified using the Go To dialog.
Go To Source Line	Go to the corresponding line in source code in the editor.
Find	Find text specified using the Find dialog.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

Right click in the memory window data area set to HEX OR PSV DATA FORMAT to pop up this menu. Not all items may be visible for all devices.

Table 13-20. Program Memory Window Menu - Hex/PSV Data

Item	Description*
Hex Display Width	**Hex Format Only** Set the hexadecimal display width (Options depend on the device selected).
Fill Memory	Fill memory from Start Address to End Address with the value in Data. Specify other options in the Fill Memory dialog.
Go To	Go to the address/function specified using the Go To dialog.
Find	Find text specified using the Find dialog.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Import Table	Import tabular data from a file into a Memory window using the Import Table dialog.
Export Table	Export tabular data from a Memory window into a file using the Export Table dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, General tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

© 2022 Microchip Technology Inc. and its subsidiaries

13.12.2 File Registers Window

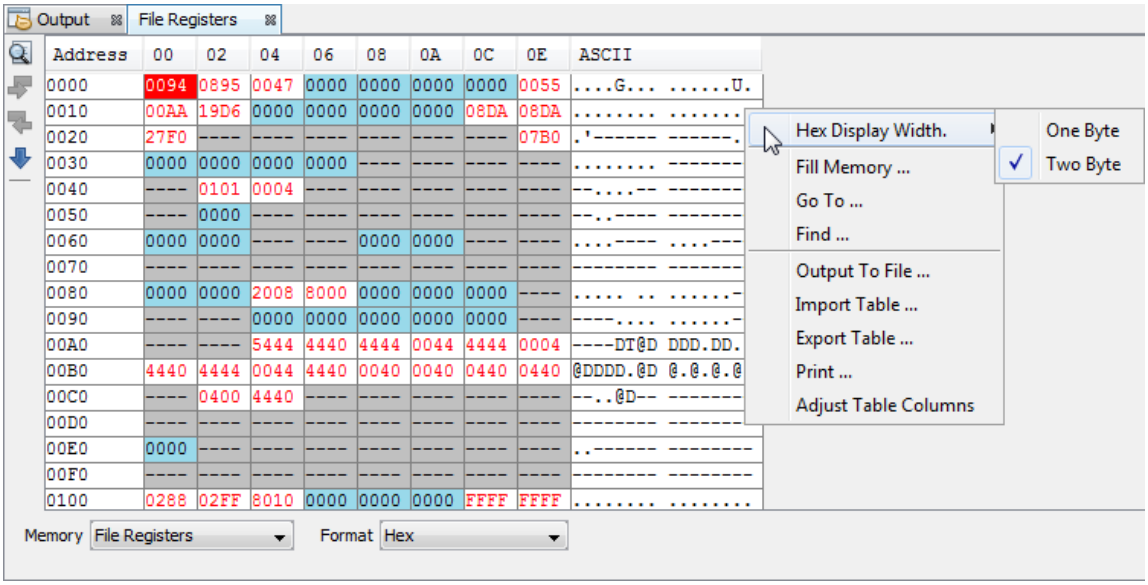
The File Registers window displays all the file registers of the selected device. When a file register value changes, or the processor is interrogated, the data in the File Registers window is updated (values updated shown in red).

Some data has colored cells:

- Blue cells represent SFRs.
- Green cells represent project variables.
- Gray cells represent reserved memory.

Note: To speed up debugging with certain hardware tools, close this window. Use the SFR or Watches window instead.

Figure 13-17. File Register Window with Content



13.12.2.1 File Registers Window Displays

You may specify how memory is displayed in the window by selecting from the Format drop-down box on the bottom of the window.

When using a hardware tool for debug, some registers may show an “R” for each nibble of data to represent a reserved resource.

Hex

This format displays file register information as hex data. The window has the following columns:

- Address – Hexadecimal address of the data in the next column.
- Data Blocks – Hexadecimal data, shown in 1- or 2-byte blocks.
- ASCII – ASCII representation of the corresponding line of data.

Symbol

This format displays each file register symbolically with corresponding data in hex, decimal, binary and character formats. The window has the following columns:

- Address – Data hexadecimal address.
- Symbol Name – Symbolic name for the data.
- Radix Information – Hex, Decimal, Binary, Char
  Radix information is displayed in these four columns. Hex is shown in 1- or 2-byte blocks.

Dual Port (dsPIC33FJ DSC/PIC24HJ MCU devices only)

This format displays file register information as hex data. The window has the following columns:

- Address – Data hexadecimal address.
- DMA Address – Offset from the silicon DMA address.
- Data Blocks – Hexadecimal data, shown in 1- or 2-byte blocks.
- ASCII – ASCII representation of the corresponding line of data.

For information on dsPIC33F DSC and PIC24H MCU devices, see the Microchip web site for device data sheets and dsPIC33F and PIC24H Reference Manual sections.

XY Data (dsPIC DSC devices only)

This format displays file register information as hex data. The window has the following columns:

- Address – X hexadecimal address of the data.
- Y Bus – Y hexadecimal address of data, if supported.
- Data Blocks – Hexadecimal data, shown in 2-byte blocks.
- ASCII – ASCII representation of the corresponding line of data.

For more information on dsPIC DSC devices, see “dsPIC30F Family Reference Manual” (DS70046).

13.12.2.2 File Registers Window Icons

Icons are located on the left side of the window.

Table 13-21. File Registers Icons

Icon	Icon Text	Function
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.
	Go To	Go to specified line number or address.

13.12.2.3 File Registers Window Menu

Right click in the memory window data area to pop up this menu. Not all items may be visible for all devices.

Table 13-22. File Registers Window Context Menu

Item	Description
Hex Display Width	**Hex Format Only** Set the hexadecimal display width (options depend on the device selected).
Fill Memory	Fill memory from Start Address to End Address with the value in Data. Specify other options in the Fill Memory dialog.
Go To	Go to the address/function specified using the Go To dialog.
Find	Find text specified using the Find dialog.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Import Table	**Hex, Dual Port or XY Data Formats Only** Import tabular data from a file into a Memory window using the Import Table dialog.
Export Table	**Hex, Dual Port or XY Data Formats Only** Export tabular data from a Memory window into a file using the Export Table dialog.

© 2022 Microchip Technology Inc.
and its subsidiaries

.....continued	
Item	**Description**
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

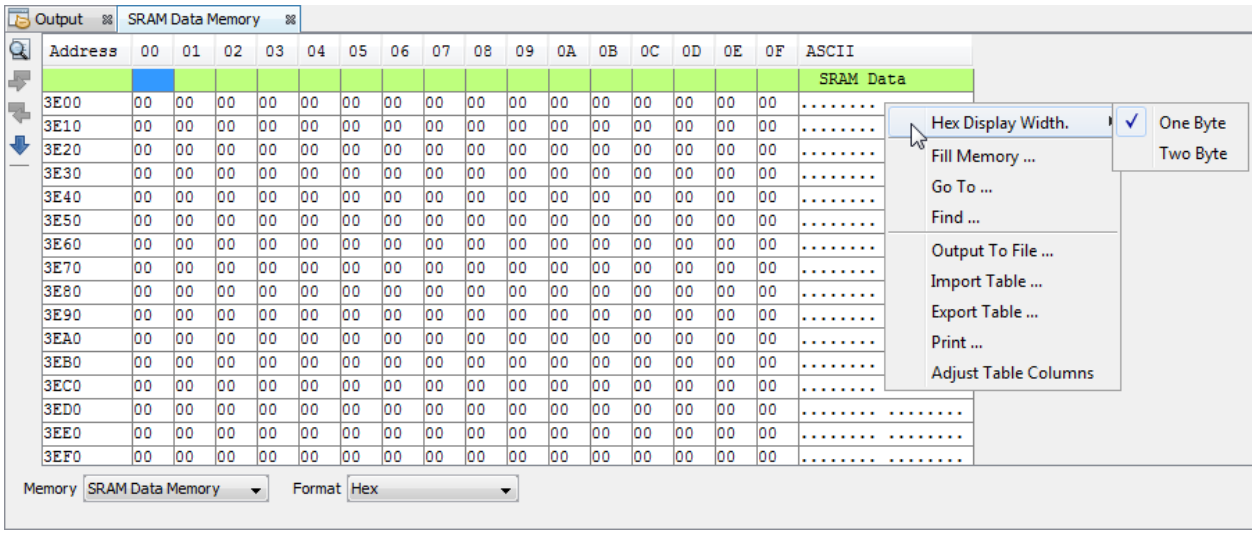
13.12.3 SRAM Data Memory Window

The SRAM Data Memory window displays all the SRAM of the selected AT device. When a register value changes, or the processor is interrogated, the data in this window is updated (values updated shown in red).

When using a hardware tool for debug, some registers may show an “R” for each nibble of data to represent a reserved resource.

Note: To speed up debugging with certain hardware tools, close this window. Use the I/O Memory (SFRs) Window or Watches Window instead.

Figure 13-18. SRAM Data Memory Window with Content



13.12.3.1 SRAM Data Memory Window Displays

You may specify how memory is displayed in the window by selecting from the Format drop-down box on the bottom of the window.

Hex

This format displays file register information as hex data. The window has the following columns:

- Address – Hexadecimal address of the data in the next column.
- Data Blocks – Hexadecimal data, shown in 1- or 2-byte blocks.
- ASCII – ASCII representation of the corresponding line of data.

Symbol

This format displays each file register symbolically with corresponding data in hex, decimal, binary and character formats. The window has the following columns:

- Address – Data hexadecimal address.
- Symbol Name – Symbolic name for the data.
- Radix Information – Hex, Decimal, Binary, Char.

Radix information is displayed in four columns. Hex is shown in 1- or 2-byte blocks.

13.12.3.2 SRAM Data Memory Window Icons

Icons are located on the left side of the window.

Table 13-23. SRAM Data Memory Window Icons

Icons	Icon Text	Function
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.
	Go To	Go to specified line number or address.

13.12.3.3 SRAM Data Memory Window Menu

Right click in the memory window data area to pop up this menu. Not all items may be visible for all devices.

Table 13-24. SRAM Data Memory Window Context Menu

Item	Description
Hex Display Width	**Hex Format Only** Set the hexadecimal display width (options depend on the device selected).
Fill Memory	Fill memory from Start Address to End Address with the value in Data. Specify other options in the Fill Memory dialog.
Go To	Go to the address/function specified using the Go To dialog.
Find	Find text specified using the Find dialog.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Import Table	**Hex Format Only** Import tabular data from a file into a Memory window using the Import Table dialog.
Export Table	**Hex Format Only** Export tabular data from a Memory window into a file using the Export Table dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

13.12.4 SFRs Window

The Special Function Registers (SFRs) window displays the contents of the SFRs for the selected processor. The format provided by this window is more useful for viewing the SFRs than the normal file register window, since each SFR name is included and several number formats are presented. To view only a few SFRs, you might want to use a Watches window, which may help with speed issues when using hardware debug tools (i.e., faster window update rate).

Whenever a break occurs, the contents of the Special Function Registers are updated.

When using a hardware tool for debug, some registers may show an “R” for each nibble of data to represent a reserved resource.

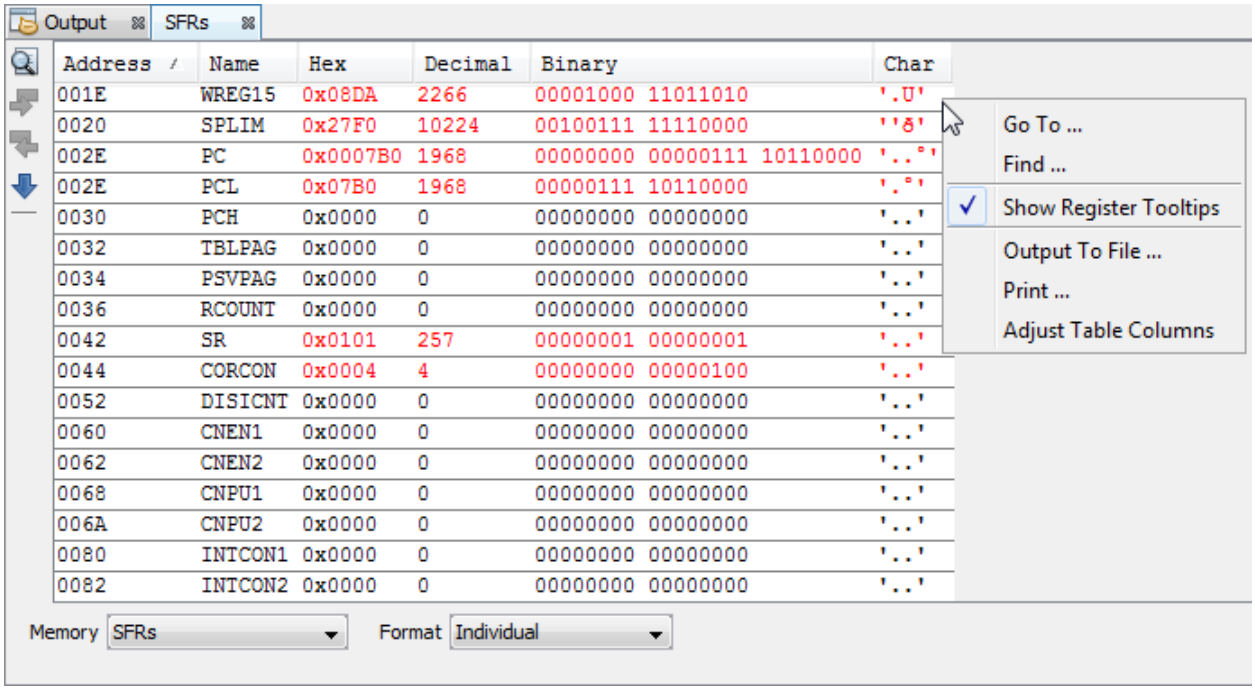
Visible Registers

If a data memory register is not physically implemented on a device, it may not appear in the SFR list. Some tools, such as simulators, may allow you to see registers that do not exist on the actual device, such as prescalers.

Single Stepping

If “Freeze Peripherals On Halt” is selected, the I/O port bits in the SFR or the Watches windows will not update when single stepping. The pin will be modified, but the read request to retrieve the new value is blocked by the freeze and cannot be updated until the next step or run command.

Figure 13-19. SFRs Window with Content



13.12.4.1 SFRs Window Displays

You may specify how memory is displayed in the window by selecting from the Format drop-down box on the bottom of the window. Choices are:

- Individual - View all SFR registers together.
- Peripherals - View SFR registers in functional groups.

Individual

In this display, SFRs are listed by address.

Data is displayed in the following columns.

- Address – SFR hexadecimal address.
- Name – Symbolic name for the SFR.
- Radix Information – Hex, Decimal, Binary
  Radix information is displayed in these four columns. Hex is shown in 1-byte blocks.

Peripheral

In this display, SFRs are grouped according to their related device peripherals.

Data is displayed in the following columns:

- Address – SFR hexadecimal address.
- Name – Name of peripheral or symbolic name for the SFR.
- Radix Information – Hex, Decimal, Binary, Char
  Radix information is displayed in these four columns. Hex is shown in 1-byte blocks.

Click the **Filter Peripheral** button to view the SFRs for only the peripherals selected.

13.12.4.2 SFRs Window Icons

Icons are located on the left side of the window.

Table 13-25. SFRs Window Icons

Icon		Function
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.
	Go To	Go to specified line number or address.
	Filter Peripherals	**Peripheral Format Only** Filter by peripherals available in view. Click icon to open the Select dialog. Select peripherals using Shift-click (range) or Ctrl-click (individually).
	Filter Select All Peripherals.	**Peripheral Format Only** Filter Select All Peripherals.

13.12.4.3 SFRs Window Menu

Right click in the memory window data area to pop up this menu.

Table 13-26. SFRs Window Context Menu

Item	Description*
Find	Find text specified using the Find dialog.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

13.12.5 I/O Memory (SFRs) Window

The I/O Memory window displays the contents of the Special Function Registers (SFRs) for the selected AT processor. The format provided by this window is more useful for viewing the SFRs than the normal SRAM Data Memory Window, since each SFR name is included and several number formats are presented. To view only a few SFRs, you might want to use a Watches Window, which may help with speed issues when using hardware debug tools (i.e., faster window update rate).

Whenever a break occurs, the contents of the Special Function Registers are updated.

When using a hardware tool for debug, some registers may show an “R” for each nibble of data to represent a reserved resource.

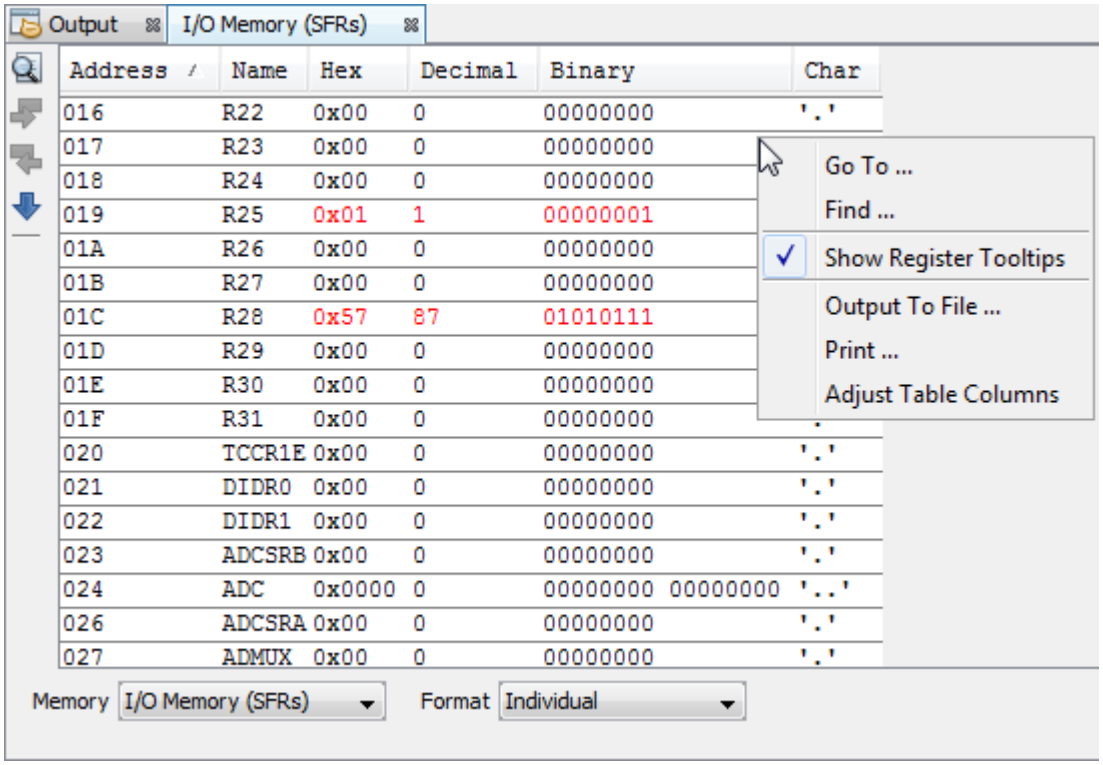
Visible Registers

If a data memory register is not physically implemented on a device, it may not appear in the SFR list. Some tools, such as simulators, may allow you to see registers that do not exist on the actual device.

Single Stepping

If “Freeze Peripherals On Halt” is selected, the I/O port bits in the SFR or the Watches windows will not update when single stepping. The pin will be modified, but the read request to retrieve the new value is blocked by the freeze and cannot be updated until the next step or run command.

Figure 13-20. I/O Memory Window with Content



13.12.5.1 I/O Memory Window Displays

You may specify how memory is displayed in the window by selecting from the Format drop-down box on the bottom of the window.

Individual

In this display, SFRs are listed by address.

Data is displayed in the following columns.

- Address – SFR hexadecimal address.
- Name – Symbolic name for the SFR.
- Radix Information – Hex, Decimal, Binary, Char.
  Radix information is displayed in these four columns. Hex is shown in 1-byte blocks.

Peripheral

In this display, SFRs are grouped according to their related device peripherals.

Data is displayed in the following columns:

- Address – SFR hexadecimal address.
- Name – Name of peripheral or symbolic name for the SFR.
- Radix Information – Hex, Decimal, Binary, Char.
  Radix information is displayed in these four columns. Hex is shown in 1-byte blocks.

Click the **Filter Peripheral** icon to view the SFRs for only the peripherals selected.

CPU Registers

In this display, CPU registers (PC and working registers Rn) are listed by address.

Data is displayed in the following columns:

- Address – SFR hexadecimal address.
- Name – Name of peripheral or symbolic name for the SFR.
- Radix Information – Hex, Decimal, Binary, Char.
  Radix information is displayed in these four columns. Hex is shown in 1-byte blocks.

13.12.5.2 I/O Memory Window Icons

Icons are located on the left side of the window.

Table 13-27. I/O Memory Window Icons

Icon		Function
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.
	Go To	Go to specified line number or address.
	Filter Peripherals	**Peripheral Format Only** Filter by peripherals available in view. Click icon to open the Select dialog. Select peripherals using Shift-click (range) or Ctrl-click (individually).
	Filter Select All Peripherals.	**Peripheral Format Only** Filter Select All Peripherals.

13.12.5.3 I/O Memory Window Menu

Right click in the memory window data area to pop up this menu.

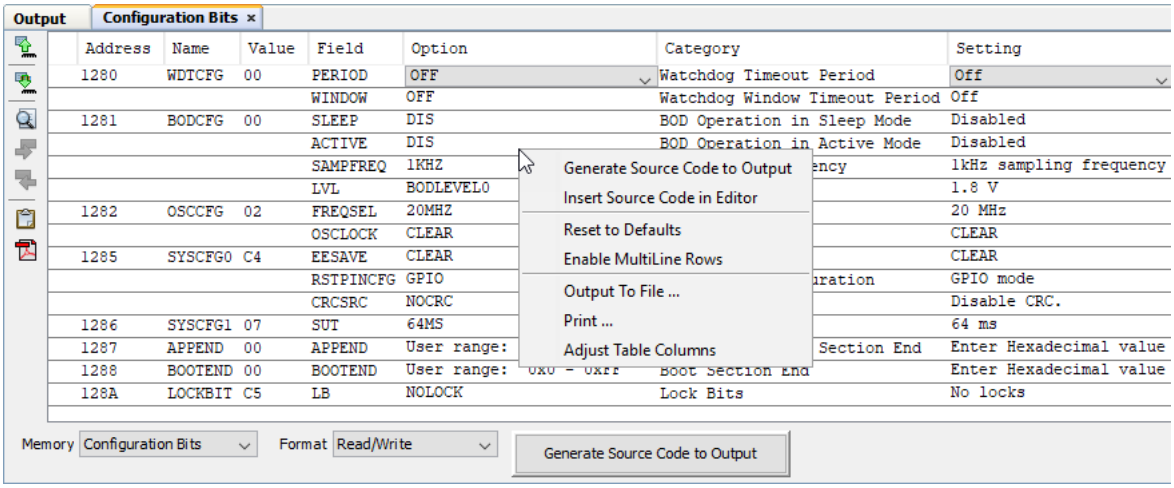
Table 13-28. I/O Memory Window Context Menu

Item	Description
Go To	Go to the address/function specified using the Go To dialog.
Find	Find text specified using the Find dialog.
Show Register Tooltips	Show or hide tooltips for a register. Hover over the register name for pop-up information.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

13.12.6 Configuration Bits Window

Use the Configuration Bits window to temporarily change configuration settings or generate configuration code to add to your application.

Figure 13-21. Configuration Bits Window with Content



Related Links

5.20. Set Configuration Values in the Configuration Bits Window

5.20. Set Configuration Values in the Configuration Bits Window

13.12.6.1 Configuration Bits Window Display

You may specify how memory is displayed in the window by selecting from the Format drop-down box on the bottom of the window. Available formats depend on your device.

When you are happy with the configuration bits settings in the window, you can click the **Generate Source Code to Output** button to write device-specific configuration code to the Output window. You may then cut-and-paste this code into your application in the editor window.

Table 13-29. Configuration Bits Window Display

Column Head	Definition
Address	Address of the configuration word/byte.
Name	Name of the Configuration Register.
Value	Current value of the configuration word/byte.
Field*	For configuration bits set in code, the field portion of the macro. As an example, WDTE is the field portion of the macro _WDTE_OFF.
Option*	For configuration bits set in code, the option portion of the macro. As an example, OFF is the option portion of the macro _WDTE_OFF.
Category	Name of the Configuration bit in the corresponding configuration word/byte.
Setting	Current setting of the Configuration bit. Use the drop-down list to change the setting. The Value of the configuration word/byte will change accordingly.
* Not all devices supported.	

13.12.6.2 Configuration Bits Window Icons

Icons are located on the left side of the window.

Table 13-30. Configuration Bits Window Icons

Icon	Icon Text	Function
	Read Configuration Bits	Uploads device configuration bit memory to the MPLAB X IDE.

.....continued		
Icon	**Icon Text**	**Function**
	Program Configuration Bits	Downloads configuration bit memory from the MPLAB X IDE to the device.
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.
	Paste	Paste Generated Source code into Editor at cursor. If Editor not in focus, Output window shows a warning and Generated Source code placed into this window.
	Open Device Datasheet	Open the online device datasheet.

13.12.6.3 Configuration Bits Window Menu

Right click in the memory window data area to pop up this menu. Not all items may be visible for all MCUs.

Table 13-31. Configuration Bits Window Menu

Item	Description
Generate Source Code to Output	Set the configuration bits in the window, as desired. Then select this option to generate code in the Output window, which you can cut-and-paste into your program to set configuration bits. Same function as the button on the window.
Insert Source Code to Editor	Set the configuration bits as desired in the window. Then select this option to generate code into an Editor window that you can cut-and-paste into your program to set configuration bits.
Reset to Defaults	Reset the values of all configuration bits back to their MPLAB X IDE defaults.
Enable Multiline Rows	Allow wrapping of contents if column is shorter than length of contents (e.g., Category). Otherwise contents will be truncated, signified by ellipsis (...).
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

13.12.7 EE Data Memory Window

The EEPROM window displays EEPROM data for any microcontroller device that has EEPROM data memory (e.g., PIC16F1829). Data/opcode hex information of the selected device is shown.

For the MPLAB X Simulator, when an EEPROM register value changes or the processor is halted, the data in the EEPROM window is updated.

For any Microchip hardware debug tool (e.g., MPLAB PICkit 4 in-circuit debugger), when an EEPROM register value changes or the processor is halted, the data in the EEPROM window is **not** updated; you must either do a Debug Read of device memory, if the device/header supports this, or you must exit from the debug session and then read the device data.

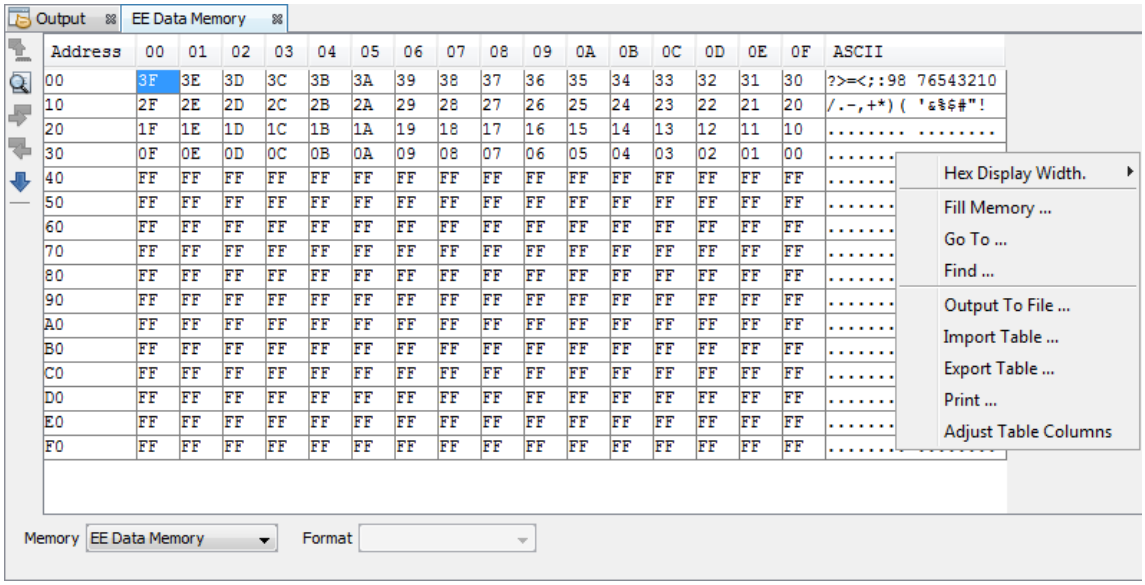
© 2022 Microchip Technology Inc. and its subsidiaries

The start of EEPROM data memory needs to be specified for use with programmers. The table below shows some generic values, but please check the programming specification for your selected device to determine the correct address.

Table 13-32. Programmer – Data EEPROM Start Address

Device	Generic Start Address
Midrange MCUs	0x2100
Enhanced Midrange MCUs	0x1E000
PIC18F MCUs	0xF00000
PIC24 MCUs, dsPIC DSCs	0x7FFE00

Figure 13-22. EE Data Memory Window with Content



13.12.7.1 EE Data Memory Window Display

This display format shows data in the following columns:

- Address – Hexadecimal address of the data in the next column.
- Data Blocks – Hexadecimal data, shown in 1-, 2- or 4-byte blocks, selectable from the menu.
- ASCII – ASCII representation of the corresponding line of data.

13.12.7.2 EE Data Memory Window Icons

Icons are located on the left side of the window.

Table 13-33. EE Data Memory Window Icons

Icon	Icon Text	Function
	Refresh by Read Device Memory	Same function as the Debug toolbar “Read Device Memory” icon - uploads device memory to the MPLAB X IDE.
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.

.....continued		
Icon	**Icon Text**	**Function**
	Go To	Go to specified line number or address.

13.12.7.3 EE Data Memory Window Menu

Right click in the memory window data area to pop up the menu as in shown below.

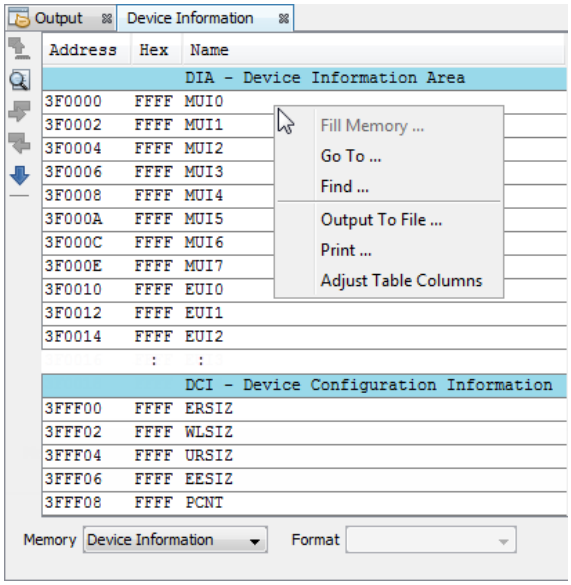
Table 13-34. EE Data Memory Window Context Menu

Item	Description*
Hex Display Width	**Hex Format Only** Set the hexadecimal display width. (Options depend on the device selected).
Fill Memory	Fill memory from Start Address to End Address with the value in Data. Specify other options in the Fill Memory dialog.
Go To	Go to the address/function specified using the Go To dialog.
Find	Find text specified using the Find dialog.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Import Table	Import tabular data from a file into a Memory window using the Import Table dialog.
Export Table	Export tabular data from a Memory window into a file using the Export Table dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, General tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

13.12.8 Device Information Window

The Device Information window displays the contents of the Device Information Area of memory for devices that support this memory area (currently PIC18F24K42 and many PIC16Fxxxxx devices). The DIA contains the calibration data for the internal temperature indicator module, stores the Microchip Unique Identifier words and the Fixed Voltage Reference voltage readings measured in mV. See your device data sheet for details.

Figure 13-23. Device Information Window



13.12.8.1 Device Information Window Display

This display format shows data in the following columns:

- Address – Hexadecimal address of the data in the next column.
- Hex – Hexadecimal data, shown in 1-, 2- or 4-byte blocks, selectable from the menu.
- Name – Description of the corresponding line of data.

13.12.8.2 Device Information Window Icons

Icons are located on the left side of the window.

Table 13-35. Device Information Window Icons

Icon	Icon Text	Function
	Refresh by Read Device Memory	Same function as the Debug toolbar “Read Device Memory” icon - uploads device memory to the MPLAB X IDE.
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.
	Go To	Go to specified line number or address.

13.12.8.3 Device Information Window Menu

Right click in the memory window data area to pop up this menu. Not all items may be visible for all devices.

Table 13-36. Device Information Window Context Menu

Item	Description
Fill Memory	Fill memory from Start Address to End Address with the value in Data. Specify other options in the Fill Memory dialog.
Go To	Go to the address/function specified using the Go To dialog.

.....continued

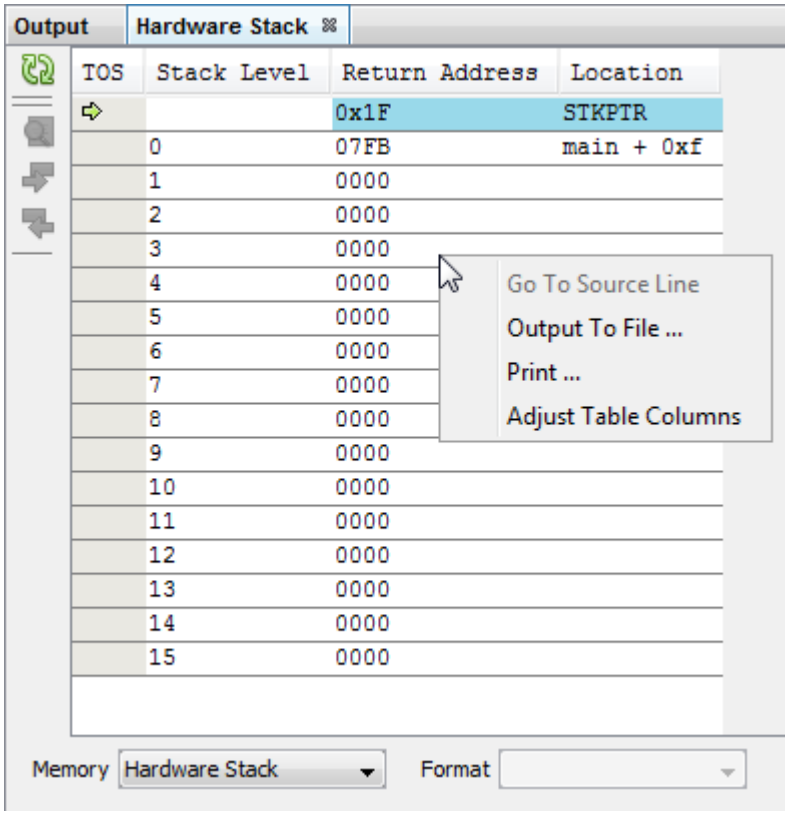
Item	Description
Find	Find text specified using the Find dialog.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

13.12.9 Hardware Stack Window

View the contents of the hardware stack on halt.

To open the hardware stack window, select *Window>Target Memory Views>Hardware Stack*. The first stack level is denoted by “0”.

Figure 13-24. Hardware Stack Window with Content



13.12.9.1 Hardware Stack Window Display

The number of levels of stack depend on the device. This display format shows data in the following columns:

- TOS – Top of Stack current location.
- Stack Level – Location of items on the stack.
- Return Address – Hexadecimal address for return from stack.
- Location – Disassembly of return address code.

When using a hardware tool for debug, some registers may show an “R” for each nibble of data to represent a reserved resource.

13.12.9.2 Hardware Stack Window Icons

The following icons are available on the window on Pause or Halt.

Table 13-37. Hardware Stack Window Icons

Icon	Icon Text	Description
	Auto Refresh Manual Refresh	Auto or Manual Refresh of Window Contents. Depends on the value of “Disable auto refresh for call stack view during debug sessions” under *Tools>Options>Embedded>Generic Settings* – Unchecked = false (default), checked = true. False = Auto Refresh (Green Icon): The window contents are automatically updated on Pause or Halt. If the window is closed or not focused, selecting the window and clicking this button will display the updates without having to run and pause/halt again. True = Manual Refresh (Orange Icon): The window contents are not automatically updated on Pause or Halt. This button must be clicked to update window contents on a pause/halt.
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.

13.12.9.3 Hardware Stack Window Menu

Right click in the memory window data area to pop up this menu..

Table 13-38. Hardware Stack Window Context Menu

Item	Description*
Go To Source Line	Go to the corresponding line in source code in the editor.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

13.12.10 Linear Data Window

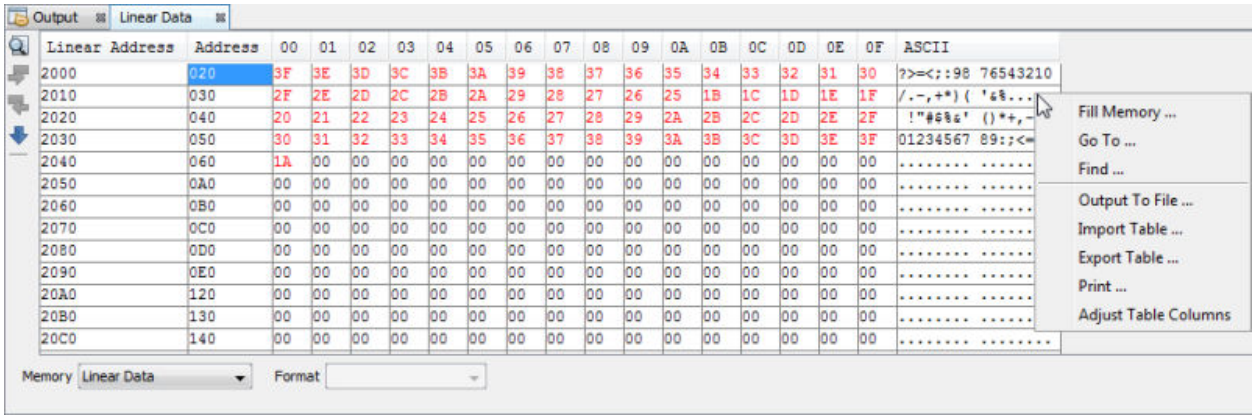
The Linear Data Window displays Flash data for any microcontroller device that has this type of memory. Data/opcode hex information of the selected device is shown.

For the MPLAB X Simulator, when a Linear Data register value changes or the processor is halted, the data in the Linear Data window is updated.

For any Microchip hardware debug tool (e.g., MPLAB ICD 4 in-circuit debugger), when a Linear Data register value changes or the processor is halted, the data in the Linear Data window is **not** updated; you must either do a Debug Read of device memory, if the device/header supports this, or you must exit from the debug session and then read the device data.

© 2022 Microchip Technology Inc. and its subsidiaries

Figure 13-25. Linear Data Window with Content



13.12.10.1 Linear Data Window Display

This display format shows data in the following columns:

- Address – Hexadecimal address of the data in the next column.
- Data Blocks – Hexadecimal data, shown in 1-, 2- or 4-byte blocks, selectable from the menu.
- ASCII – ASCII representation of the corresponding line of data.

13.12.10.2 Linear Data Window Icons

Icons are located on the left side of the window.

Table 13-39. Linear Data Window Icons

Icon	Icon Text	Function
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.
	Go To	Go to specified line number or address.

13.12.10.3 Linear Data Window Menu

Right click in the memory window data area to pop up the menu as in the table below.

Table 13-40. Linear Data Window Context Menu

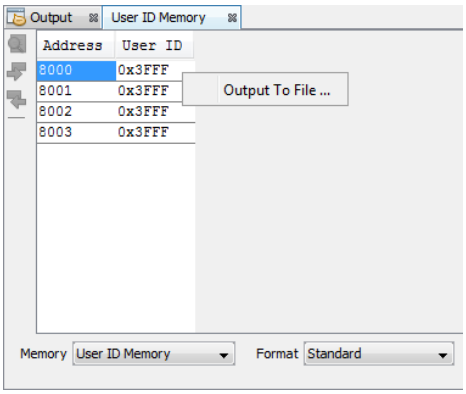
Item	Description*
Fill Memory	Fill memory from Start Address to End Address with the value in Data. Specify other options in the Fill Memory dialog.
Go To	Go to the address/function specified using the Go To dialog.
Find	Find text specified using the Find dialog.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Import Table	Import tabular data from a file into a Memory window using the Import Table dialog.
Export Table	Export tabular data from a Memory window into a file using the Export Table dialog.

.....continued	
Item	**Description***
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

13.12.11 User ID Memory Window

Some devices have memory locations where you can store checksum or other code identification (ID) numbers. These locations are readable and writable during program/verify. Depending on the device, they also may be accessible during normal execution through the TBLRD and TBLWT instructions.

Figure 13-26. User ID Memory Window with Content

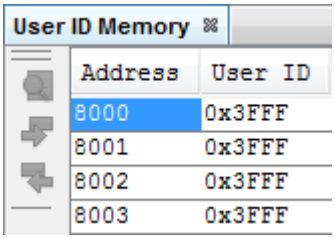


13.12.11.1 User ID Memory Window Display

Consult your device programming specification to determine what values may be entered here. For most devices, this sets the low nibble of the device ID word; the high nibble is set to ‘0’. The high nibble can be only be written to programmatically, such as by using Table Writes.

You may specify how memory is displayed in the window by selecting from the Format drop-down box on the bottom of the window.

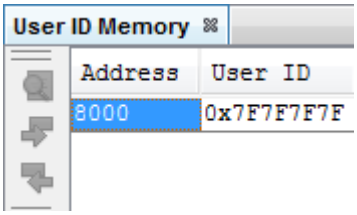
Figure 13-27. Standard Display



In the Standard display, data is shown in the following columns.

- Address – User ID hexadecimal address(es).
- User ID – Contents (in hex) of User ID memory.

Figure 13-28. Legacy Display



In the Legacy display, data is shown in the following columns.

- Address – User ID hexadecimal starting address.
- User ID – Contents (in hex) of User ID memory.

13.12.11.2 User ID Memory Window Icon

Icons are located on the left side of the window.

Table 13-41. User ID Memory Window Icons

Icon	Icon Text	Function
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.

13.12.11.3 User ID Memory Window Menu

Right click in the memory window data area to pop up the menu, as in the table below.

Table 13-42. User ID Memory Window Context Menu

Item	Description*
Output To File	Write the displayed window contents to a text file using the Output to File dialog.

13.13 Memory Windows - 32-Bit Devices

Memory windows (*Window>Target Memory Views*) display the many types of device memory, such as SFRs and Configuration bits. Use the “Memory” and “Format” drop-down boxes to customize your window.

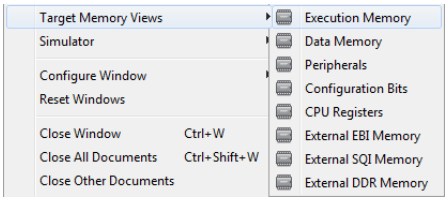
Some memory windows have additional functionality through associated dialogs.

Note: The Configuration Bits window and the User ID window are similar for 8-, 16- and 32-bit devices. The only difference is that the values in the windows reflect the program memory available for the device.

Figure 13-29. Window>Target Memory Views - PIC32MX795F512L



Figure 13-30. Window>Target Memory Views - ATSAME70Q21B



Related Links

5.19. View or Change Device Memory

13.14. Memory Windows Associated Dialogs

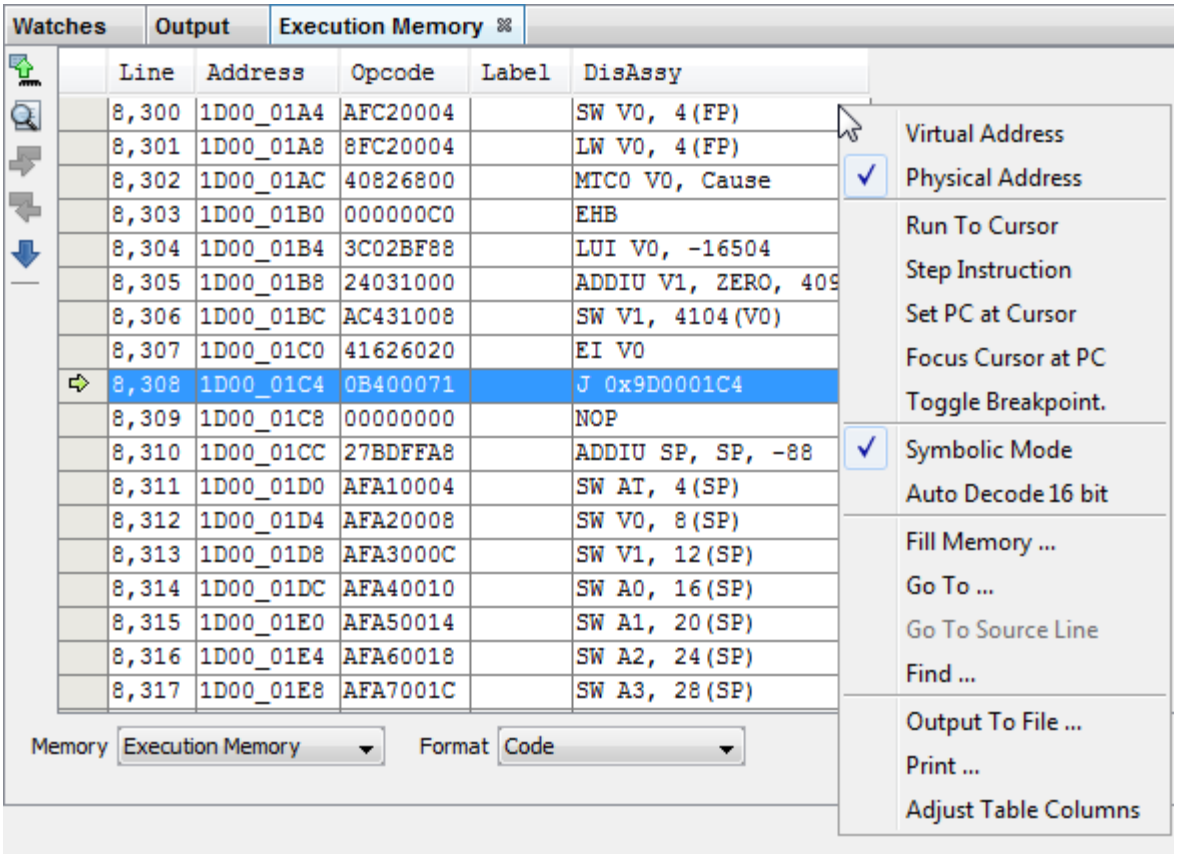
13.13.1 Execution Memory Window

The Execution Memory window displays locations in the range of program and/or data memory for the currently selected 32-bit device.

For the MPLAB X Simulator, when an execution memory value changes or the processor is halted, the data in the Execution Memory window is updated.

For any Microchip hardware debug tool (e.g., MPLAB ICD 4 in-circuit emulator), when an execution memory value changes or the processor is halted, the data in the Execution Memory window is **not** updated; you must do a Read of device memory.

Figure 13-31. Execution Memory Window with Content



13.13.1.1 Execution Memory Window Display

You may specify how memory is displayed in the window by selecting from the Format drop-down box on the bottom of the window.

When using a hardware tool for debug, some registers may show an “R” for each nibble of data to represent a reserved resource.

Code Format

This format displays execution memory information as hex code. The window will have the following columns:

- Line – Reference line number corresponding to memory address.
- Address – Physical hexadecimal address of the opcode.
- Opcode – Hexadecimal opcode, shown in 4-byte blocks. The opcode that is highlighted represents the current location of the program counter.
- Label – Opcode label in symbolic format.
- Disassembly – A disassembled version of the opcode mnemonic.

Data Format

This window will have the following columns:

- Address – Hexadecimal address of the data in the next column.
- Data Blocks – Hexadecimal data, shown in 4-byte blocks.
- ASCII – ASCII representation of the corresponding line of data.

13.13.1.2 Execution Memory Window Icons

Icons are located on the left side of the window.

Table 13-43. Execution Memory Window Icons

Icon	Icon Text	Function
	Refresh by Read Device Memory	Same function as the Debug toolbar “Read Device Memory” icon - uploads device memory to the MPLAB X IDE.
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.
	Go To	Go to specified line number or address.

13.13.1.3 Execution Memory Window Menu

Right click in the memory window data area set to CODE FORMAT to pop up this menu. Not all items may be visible for all 32-bit MCUs.

Table 13-44. Execution Memory Window Content Menu - Code Format

Item	Description
Virtual Address (KSEG[0:1])	Select virtual addresses for display. See your device data sheet for details.
Physical Address	Select physical addresses for display.
Run to Cursor	Run the program to the current cursor location.
Step Instruction	Perform a step instruction.
Set PC at Cursor	Set the Program Counter (PC) to the cursor location.
Focus Cursor at PC	Move the cursor to the current PC address and centers this address in the window.
Toggle Breakpoint	Toggle (on/off) existing breakpoint.
Symbolic Mode	Display disassembled hex code with symbols.
Auto Decode 16 bit	Use MIP16 / microMIPS instruction decoding. See your device data sheet for details.

.....continued

Item	Description
Fill Memory	Fill memory from Start Address to End Address with the value in Data. Specify other options in the Fill Memory dialog.
Go To	Go to the address/function specified using the Go To dialog.
Go To Source Line	Go to the corresponding line in source code in the editor.
Find	Find text specified using the Find dialog.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

Right click in the memory window data area set to DATA FORMAT to pop up the menu. Not all items may be visible for all 32-bit MCUs.

Table 13-45. Execution Memory Window Content Menu - Data Format

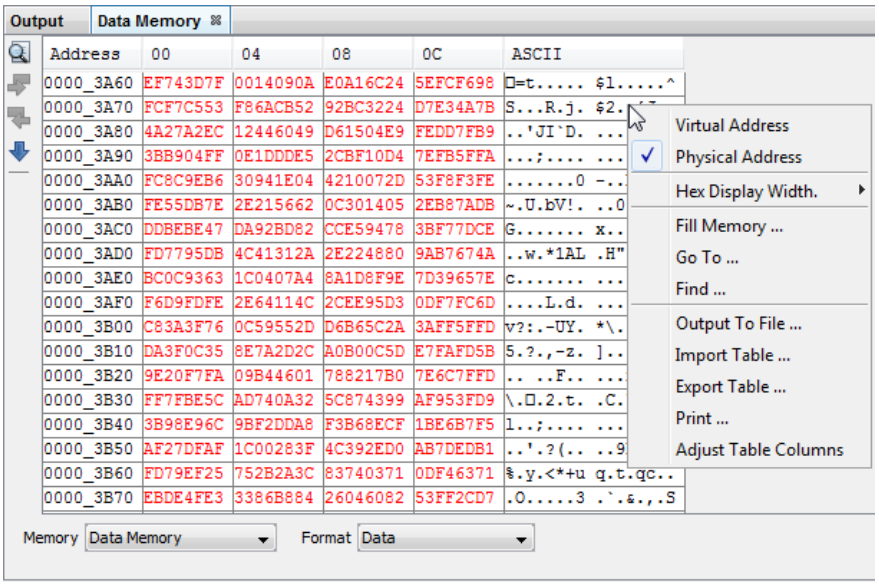
Item	Description
Virtual Address (KSEG[0:1])	Select virtual addresses for display. See your device data sheet for details.
Physical Address	Select physical addresses for display.
Hex Width Display	Set the hexadecimal display width. (Options depend on the device selected). Example: One byte, e.g., 00 01 02 ... 0E 0F Two bytes, e.g., 00 02 04 ... 0C 0E Four bytes, e.g., 00 04 08 0C
Fill Memory	Fill memory from Start Address to End Address with the value in Data. Specify other options in the Fill Memory dialog.
Go To	Go to the address/function specified using the Go To dialog.
Find	Find text specified using the Find dialog.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Import Table	Import tabular data from a file into a Memory window using the Import Table dialog.
Export Table	Export tabular data from a Memory window into a file using the Export Table dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

© 2022 Microchip Technology Inc.
and its subsidiaries

13.13.2 Data Memory Window

The Data Memory window displays locations in the range of data and/or program memory for the currently selected 32-bit device.

Figure 13-32. Data Memory Window with Content



13.13.2.1 Data Memory Window Display

You may specify how memory is displayed in the window by selecting from the Format drop-down box on the bottom of the window.

When using a hardware tool for debug, some registers may show an “R” for each nibble of data to represent a reserved resource.

Data Format

This window will have the following columns:

- Address – Hexadecimal address of the data in the next column.
- Data Blocks – Hexadecimal data, shown in 4-byte blocks.
- ASCII – ASCII representation of the corresponding line of data.

Code Format

This window will have the following columns:

- Line – Reference line number corresponding to memory address.
- Address – Physical hexadecimal address of the opcode.
- Opcode – Hexadecimal opcode, shown in 4-byte blocks
  The opcode that is highlighted represents the current location of the program counter.
- Label – Opcode label in symbolic format.
- Disassembly – A disassembled version of the opcode mnemonic.

13.13.2.2 Data Memory Window Icons

Icons are located on the left side of the window.

Table 13-46. Data Memory Window Icons

Icon	Icon Text	Function
	Refresh by Read Device Memory	Same function as the Debug toolbar “Read Device Memory” icon - uploads device memory to the MPLAB X IDE.

.....continued		
Icon	**Icon Text**	**Function**
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.
	Go To	Go to specified line number or address.

13.13.2.3 Data Memory Window Menu

Right click in the memory window data area to pop up this menu. Not all items may be visible for all 32-bit MCUs.

Table 13-47. Data Memory Window Content Menu

Item	Description
Virtual Address	Select virtual addresses for display. See your device data sheet for details.
Physical Address	Select physical addresses for display.
Symbolic Mode	**Code Format Only** Display disassembled hex code with symbols.
Hex Width Display	**Data Format Only** Set the hexadecimal display width. (Options depend on the device selected). Example: One byte, e.g., 00 01 02 ... 0E 0F Two bytes, e.g., 00 02 04 ... 0C 0E Four bytes, e.g., 00 04 08 0C
Fill Memory	Fill memory from Start Address to End Address with the value in Data. Specify other options in the Fill Memory dialog.
Go To	Go to the address/function specified using the Go To dialog.
Go To Source Line	**Code Format Only** Go to the corresponding line in source code in the editor.
Find	Find text specified using the Find dialog.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Import Table	Import tabular data from a file into a Memory window using the Import Table dialog.
Export Table	Export tabular data from a Memory window into a file using the Export Table dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

13.13.3 Peripherals Window

The Peripherals window displays the contents of the SFRs that relate to the device peripherals. To view only a few SFRs, you may prefer to use a Watch window, which may help with speed issues when using hardware debug tools (i.e., faster window update rate).

Whenever a break occurs, the contents of the SFRs are updated.

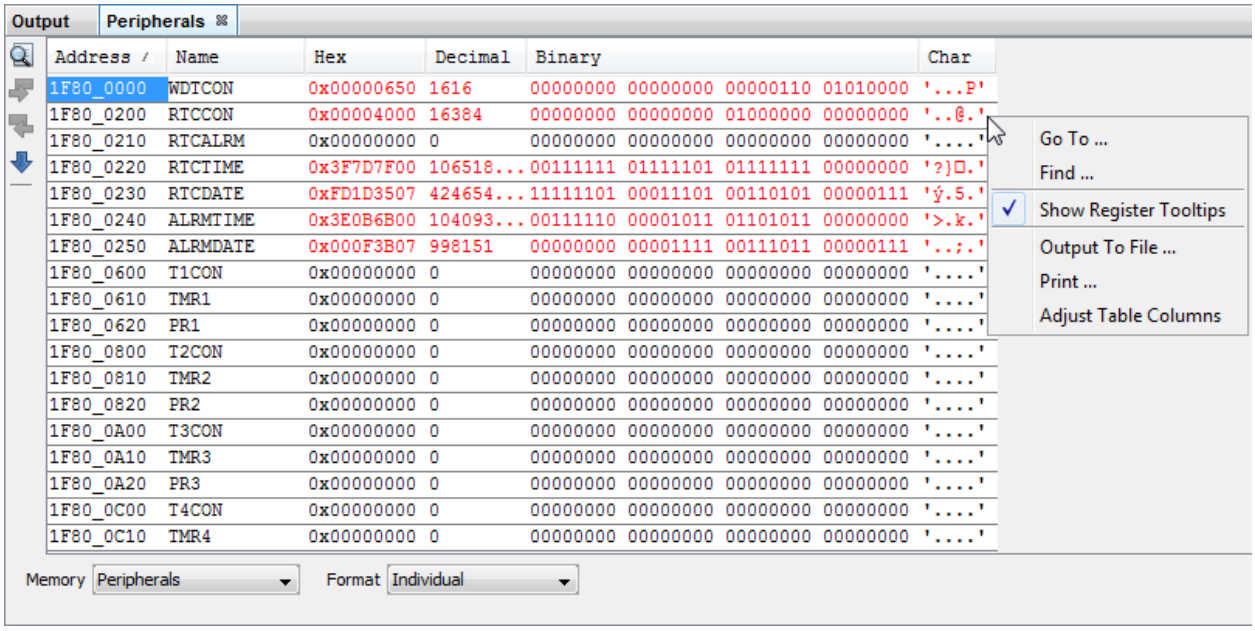
Visible Registers

If a data memory register is not physically implemented on a device, it may not appear in the SFR list. Some tools, such as simulators, may allow you to see registers that do not exist on the actual device, such as prescalers.

Single Stepping

If “Freeze Peripherals On Halt” is selected, the I/O port bits in the SFR or the Watches windows will not update when single stepping. The pin will be modified, but the read request to retrieve the new value is blocked by the freeze and cannot be updated until the next step or run command.

Figure 13-33. Peripherals Window with Content



13.13.3.1 Peripherals Window Display

You may specify how memory is displayed in the window by selecting from the Format drop-down box on the bottom of the window.

When using a hardware tool for debug, some registers may show an “R” for each nibble of data to represent a reserved resource.

- **Individual** - View all peripheral registers together.
- **Groups** - View peripheral registers in functional groups.

Data is displayed for both formats in the following columns.

- Address – SFR physical hexadecimal address.
- Virtual – SFR virtual hexadecimal address as defined by the Bus Matrix.
- Name – Symbolic name for the SFR.
- Radix Information – Hex, Decimal, Binary, Char.
  You may add radix information to the display by right clicking on the column header bar. Hex is shown in 4-byte blocks.

13.13.3.2 Peripherals Window Icons

Icons are located on the left side of the window.

Table 13-48. Data Memory Window Icons

Icon	Icon Text	Function
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.
	Go To	Go to specified line number or address.

13.13.3.3 Peripherals Window Menu

Right click in the memory window data area to pop up the menu as in the table below.

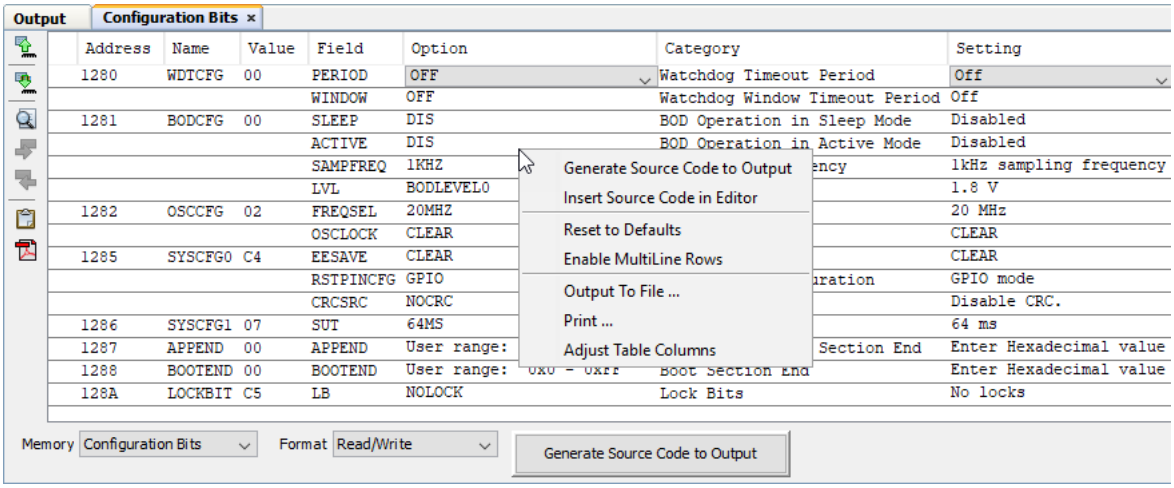
Table 13-49. Peripherals Window Context Menu

Item	Description
Find	Find text specified using the Find dialog.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

13.13.4 Configuration Bits Window

Use the Configuration Bits window to temporarily change configuration settings or generate configuration code to add to your application.

Figure 13-34. Configuration Bits Window with Content



Related Links

5.20. Set Configuration Values in the Configuration Bits Window

5.20. Set Configuration Values in the Configuration Bits Window

13.13.4.1 Configuration Bits Window Display

You may specify how memory is displayed in the window by selecting from the Format drop-down box on the bottom of the window. Available formats depend on your device.

When you are happy with the configuration bits settings in the window, you can click the **Generate Source Code to Output** button to write device-specific configuration code to the Output window. You may then cut-and-paste this code into your application in the editor window.

Table 13-50. Configuration Bits Window Display

Column Head	Definition
Address	Address of the configuration word/byte.
Name	Name of the Configuration Register.
Value	Current value of the configuration word/byte.
Field*	For configuration bits set in code, the field portion of the macro. As an example, WDTE is the field portion of the macro _WDTE_OFF.
Option*	For configuration bits set in code, the option portion of the macro. As an example, OFF is the option portion of the macro _WDTE_OFF.
Category	Name of the Configuration bit in the corresponding configuration word/byte.
Setting	Current setting of the Configuration bit. Use the drop-down list to change the setting. The Value of the configuration word/byte will change accordingly.
* Not all devices supported.	

13.13.4.2 Configuration Bits Window Icons

Icons are located on the left side of the window.

Table 13-51. Configuration Bits Window Icons

Icon	Icon Text	Function
	Read Configuration Bits	Uploads device configuration bit memory to the MPLAB X IDE.
	Program Configuration Bits	Downloads configuration bit memory from the MPLAB X IDE to the device.
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.
	Paste	Paste Generated Source code into Editor at cursor. If Editor not in focus, Output window shows a warning and Generated Source code placed into this window.
	Open Device Datasheet	Open the online device datasheet.

13.13.4.3 Configuration Bits Window Menu

Right click in the memory window data area to pop up this menu. Not all items may be visible for all MCUs.

Table 13-52. Configuration Bits Window Menu

Item	Description
Generate Source Code to Output	Set the configuration bits in the window, as desired. Then select this option to generate code in the Output window, which you can cut-and-paste into your program to set configuration bits. Same function as the button on the window.
Insert Source Code to Editor	Set the configuration bits as desired in the window. Then select this option to generate code into an Editor window that you can cut-and-paste into your program to set configuration bits.
Reset to Defaults	Reset the values of all configuration bits back to their MPLAB X IDE defaults.
Enable Multiline Rows	Allow wrapping of contents if column is shorter than length of contents (e.g., Category). Otherwise contents will be truncated, signified by ellipsis (...).
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

13.13.5 CPU Registers Window

The CPU Registers window displays the contents of the SFRs that relate to the device CPU. To view only a few CPU SFRs, you may prefer to use a Watches window, which may help with speed issues when using hardware debug tools (i.e., faster window update rate).

Whenever a break occurs, the contents of the CPU Registers are updated.

Visible Registers

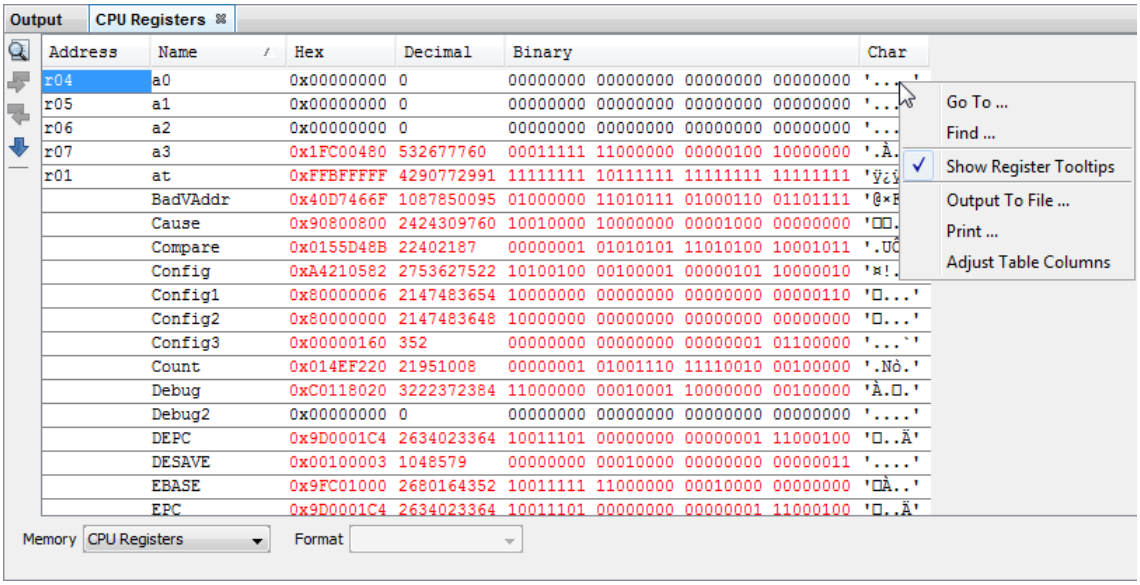
If a data memory register is not physically implemented on a device, it may not appear in the SFR list. Some tools, such as simulators, may allow you to see registers that do not exist on the actual device, such as prescalers.

Single Stepping

If “Freeze Peripherals On Halt” is selected, the I/O port bits in the SFR or the Watches windows will not update when single stepping. The pin will be modified, but the read request to retrieve the new value is blocked by the freeze and cannot be updated until the next step or run command.

© 2022 Microchip Technology Inc.
and its subsidiaries

Figure 13-35. CPU Registers Window with Content



13.13.5.1 CPU Registers Window Display

You may specify how memory is displayed in the window by selecting from the Format drop-down box on the bottom of the window. Choices are:

- Blank - Only one view.
- Devices with FPU - For PIC32 devices with a Floating-Point Unit (FPU), the Format options are:
  - All Registers
  - Only CPU Registers
  - Only FPU Registers.

Blank, All Registers or Only CPU Registers Format

Data is displayed in the following columns.

- Address – SFR physical hexadecimal address.
- Name – Symbolic name for the SFR.
- Radix Information – Hex, Decimal, Binary, Char.
- Virtual – SFR virtual hexadecimal address as defined by the Bus Matrix.

Only FPU Registers Format

Data is displayed in the following columns.

- Address – SFR physical hexadecimal address.
- Name – Symbolic name for the SFR.
- Radix Information – Hex
- Floating Point – Float, Double

13.13.5.2 CPU Registers Window Icons

Icons are located on the left side of the window.

Table 13-53. Data Memory Window Icons

Icon	Icon Text	Function
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.

.....continued

Icon	Icon Text	Function
	Find Previous	Find previous instance of string from Find.
	Go To	Go to specified line number or address.

13.13.5.3 CPU Registers Window Menu

Right click in the memory window data area to pop up the menu as in the table below.

Table 13-54. CPU Registers Window Context Menu

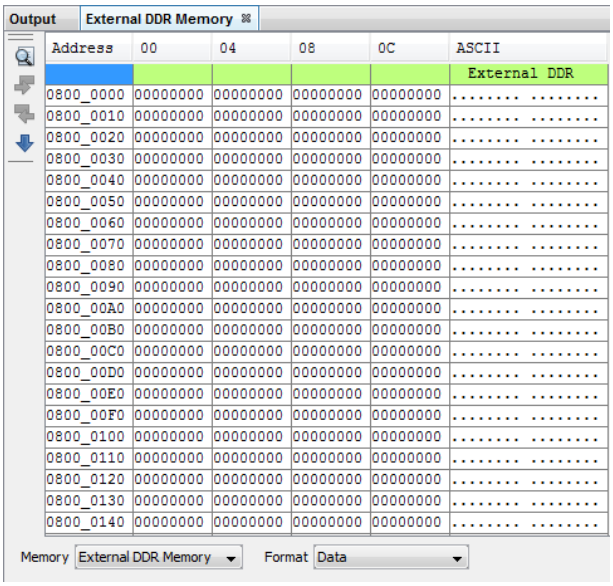
Item	Description
Go To	Go to the address/function specified using the Go To dialog.
Find	Find text specified using the Find dialog.
Show Register Tooltip	Show or hide tooltips for a register. Hover over the register name for pop-up information.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, "Print to File" check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

13.13.6 External EBI, SQI, or DDR Memory Window

The External Memory window displays the contents of external memory by bus type:

- External Bus Interface (EBI)
- Serial Quad Interface (SQI)
- Double Data Rate (DDR)

Figure 13-36. External DDR Memory Window



13.13.6.1 External EBI, SQI, or DDR Memory Window Display

To see external memory contents in this window:

- Allocate functions/variables to external memory in your code - see the "*MPLAB XC32/XC32++ C Compiler User's Guide*" (DS50001686) for details.
- Load symbols - *File>Properties*, "Loading" category, check "Load symbols when programming or building for production (slows process)."
- Compile your code - Build the project.

You may specify how memory is displayed in the window by selecting from the Format drop-down box on the bottom of the window:

Code Format

This window will have the following columns:

- Line – Reference line number corresponding to memory address.
- Address – Physical hexadecimal address of the opcode.
- Opcode – Hexadecimal opcode, shown in 4-byte blocks
  The opcode that is highlighted represents the current location of the program counter.
- Label – Opcode label in symbolic format.
- Disassembly – A disassembled version of the opcode mnemonic.

Data Format

This window will have the following columns:

- Address – Hexadecimal address of the data in the next column.
- Data Blocks – Hexadecimal data, shown in 4-byte blocks.
- ASCII – ASCII representation of the corresponding line of data.

13.13.6.2 External EBI, SQI, or DDR Memory Window Icons

Icons are located on the left side of the window.

Table 13-55. External Memory Window Icons

Icon	Icon Text	Function
	Refresh by Read Device Memory	Same function as the Debug toolbar "Read Device Memory" icon - uploads device memory to the MPLAB X IDE.
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.
	Go To	Go to specified line number or address.

13.13.6.3 External EBI, SQI, or DDR Memory Window Menu

Right click in the memory window data area set to CODE FORMAT to pop up this menu. Not all items may be visible for all 32-bit MCUs.

Table 13-56. External Memory Window Context Menu - Code Format

Item	Description
External Address (KSEG[2:3])	Select addresses for display. See your device data sheet for details.
Physical Address	Select physical addresses for display.

.....continued

Item	Description
Run to Cursor	Run the program to the current cursor location.
Step Instruction	Perform a step instruction.
Set PC at Cursor	Set the Program Counter (PC) to the cursor location.
Focus Cursor at PC	Move the cursor to the current PC address and centers this address in the window.
Toggle Breakpoint	Toggle (on/off) existing breakpoint.
Symbolic Mode	Display disassembled hex code with symbols.
Auto Decode 16 bit	Use MIP16 / microMIPS instruction decoding. See your device data sheet for details.
Fill Memory	Fill memory from Start Address to End Address with the value in Data. Specify other options in the Fill Memory dialog.
Go To	Go to the address/function specified using the Go To dialog.
Go To Source Line	Go to the corresponding line in source code in the editor.
Find	Find text specified using the Find dialog.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

Right click in the memory window data area set to DATA FORMAT to pop up the menu. Not all items may be visible for all 32-bit MCUs.

Table 13-57. External Memory Window Context Menu - Data Format

Item	Description
External Address (KSEG[2:3])	Select addresses for display. See your device data sheet for details.
Physical Address	Select physical addresses for display.
Hex Width Display	Set the hexadecimal display width (Options depend on the device selected.) Example: One byte, e.g., 00 01 02 ... 0E 0F Two bytes, e.g., 00 02 04 ... 0C 0E Four bytes, e.g., 00 04 08 0C
Fill Memory	Fill memory from Start Address to End Address with the value in Data. Specify other options in the Fill Memory dialog.
Go To	Go to the address/function specified using the Go To dialog.
Find	Find text specified using the Find dialog.
Output To File	Write the displayed window contents to a text file using the Output to File dialog.
Import Table	Import tabular data from a file into a Memory window using the Import Table dialog.

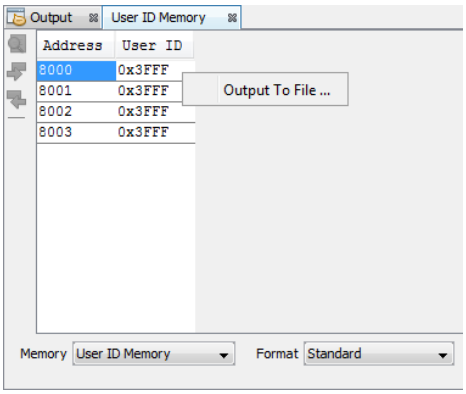
© 2022 Microchip Technology Inc.
and its subsidiaries

.....continued	
Item	**Description**
Export Table	Export tabular data from a Memory window into a file using the Export Table dialog.
Print	Print the contents of this window using the Print dialog. **Note:** If you have a large memory device, the number of pages printed can become very large. A suggestion is to print the window contents to a file (Print dialog, **General** tab, “Print to File” check box) and then select which pages from the file you need to print.
Adjust Table Columns	Adjust the columns automatically.

13.13.7 User ID Memory Window

Some devices have memory locations where you can store checksum or other code identification (ID) numbers. These locations are readable and writable during program/verify. Depending on the device, they also may be accessible during normal execution through the TBLRD and TBLWT instructions.

Figure 13-37. User ID Memory Window with Content

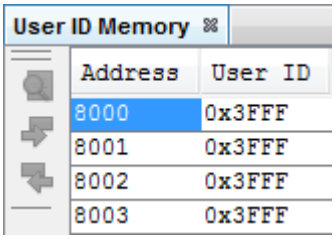


13.13.7.1 User ID Memory Window Display

Consult your device programming specification to determine what values may be entered here. For most devices, this sets the low nibble of the device ID word; the high nibble is set to ‘0’. The high nibble can be only be written to programmatically, such as by using Table Writes.

You may specify how memory is displayed in the window by selecting from the Format drop-down box on the bottom of the window.

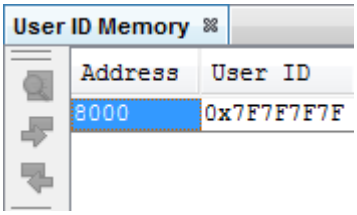
Figure 13-38. Standard Display



In the Standard display, data is shown in the following columns.

- Address – User ID hexadecimal address(es).
- User ID – Contents (in hex) of User ID memory.

Figure 13-39. Legacy Display



In the Legacy display, data is shown in the following columns.

- Address – User ID hexadecimal starting address.
- User ID – Contents (in hex) of User ID memory.

13.13.7.2 User ID Memory Window Icon

Icons are located on the left side of the window.

Table 13-58. User ID Memory Window Icons

Icon	Icon Text	Function
	Find	Specify a string to find in the window. Select to match whole word or case.
	Find Next	Find next instance of string from Find.
	Find Previous	Find previous instance of string from Find.

13.13.7.3 User ID Memory Window Menu

Right click in the memory window data area to pop up the menu, as in the table below.

Table 13-59. User ID Memory Window Context Menu

Item	Description*
Output To File	Write the displayed window contents to a text file using the Output to File dialog.

13.14 Memory Windows Associated Dialogs

Memory windows have associated dialogs to support formatting, management and output of window data. These dialogs are accessed from content menus.

13.14.1 Memory Windows Go To Dialog

Go to a location in target memory. Access this dialog from some target memory window context menus.

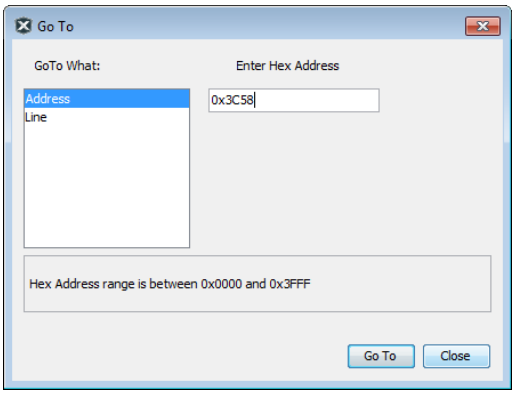


Table 13-60. Go To Dialog

Item	Description
Go To What	Enter what you want to go to. Click on the item to see a description below. **Address**: Enter a hexadecimal address. **Symbol**: Enter a program symbol. **Line**: Enter a line number. **Label**: Select a program label. **Function**: Select a program function.
Go to description	Select a “Go To What” item to see a description.

13.14.2 Memory Windows Find Dialog

Find a location in target memory. Access this dialog from some target memory window context menus.

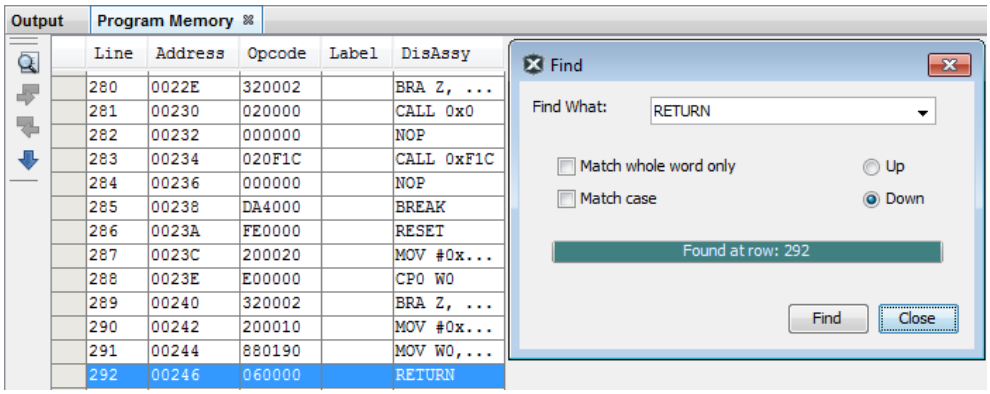


Table 13-61. Find Dialog

Item	Description
Find What	Enter the text you want to find.
Match	Select to “Match whole word only” and/or “Match case.”
Direction	Select to search “Up” or “Down” the window.
Found At bar	When an item is found, the location is shown on this bar.

13.14.3 Memory Windows Output to File Dialog

Output specified memory range to a file. Access this dialog from some target memory window context menus.

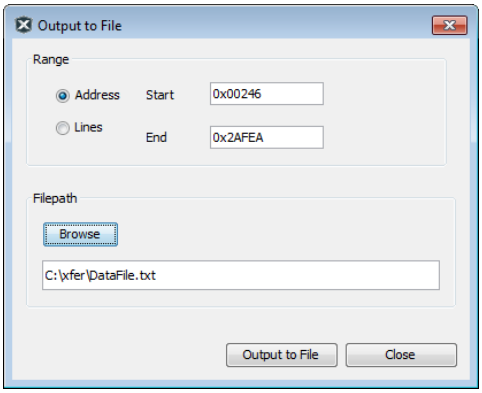


Table 13-62. Output to File Dialog

Item	Description
Range	Enter a range of data to output to a file. **Address**: Enter an Address range - Start to End Address. **Lines**: Enter a range of Lines - Start line to End line.
Filepath	Browse to a location for the output file or enter in the text box.

13.14.4 Memory Windows Import/Export Table Dialog

Import from or export to a file target memory in tabular format. Access this dialog from some target memory window context menus.

Figure 13-40. Export Table Dialog

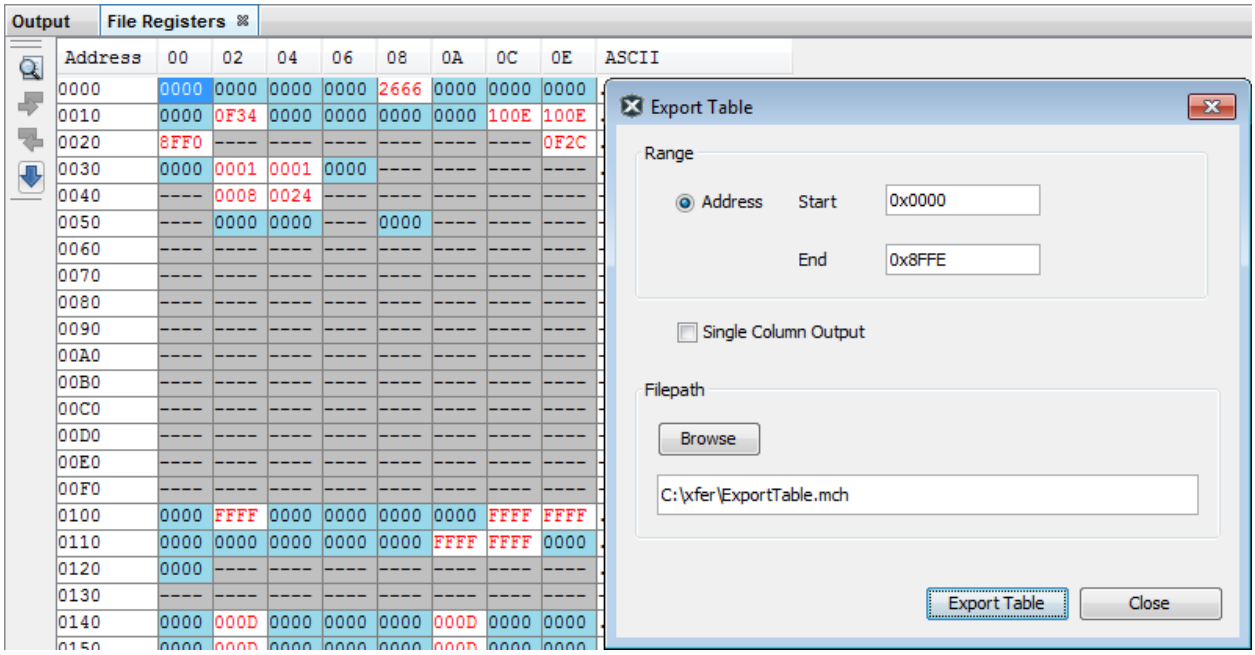


Figure 13-41. Export Table File

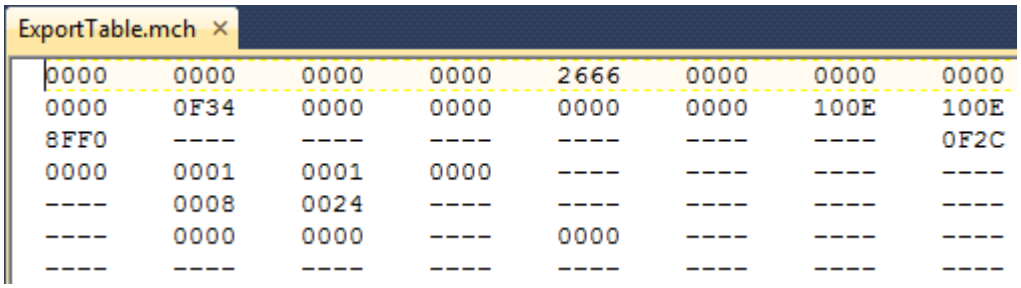


Table 13-63. Import/Export Table Dialog

Item	Description
Range	Enter a range of data to import or export in tabular format to a file. **Address**: Enter an Address range - Start to End Address.
Single Column Format	**Export** table as a single column.
Filepath	Browse to a location for the input or output file or enter in the text box.

13.14.5 Memory Windows Print Dialog

Print out the memory window contents. Access this dialog from some target memory window context menus.

Print options available may depend on printer selected.

Figure 13-42. Print Dialog

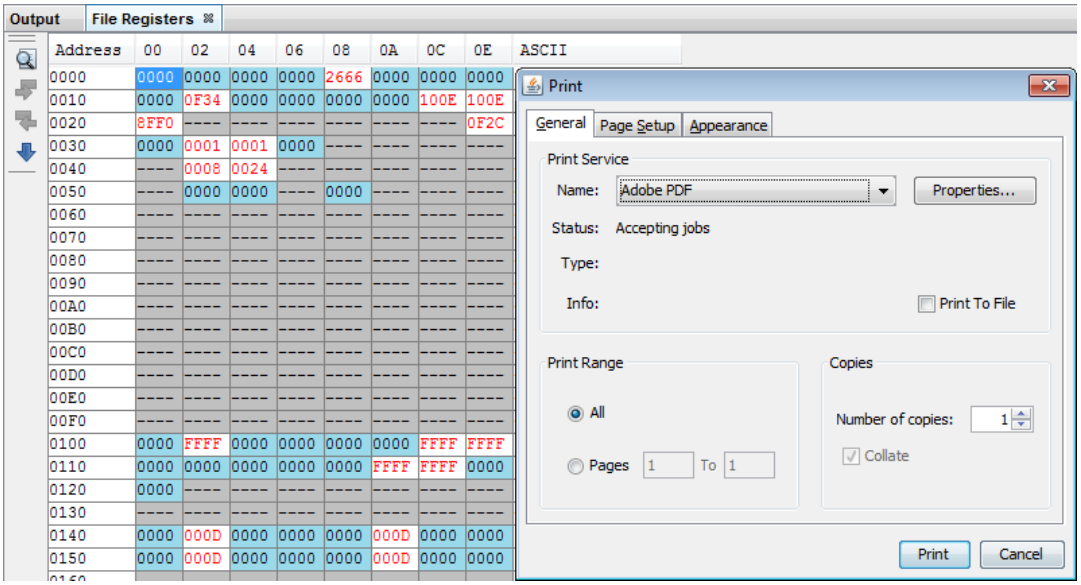


Figure 13-43. Print Dialog Output

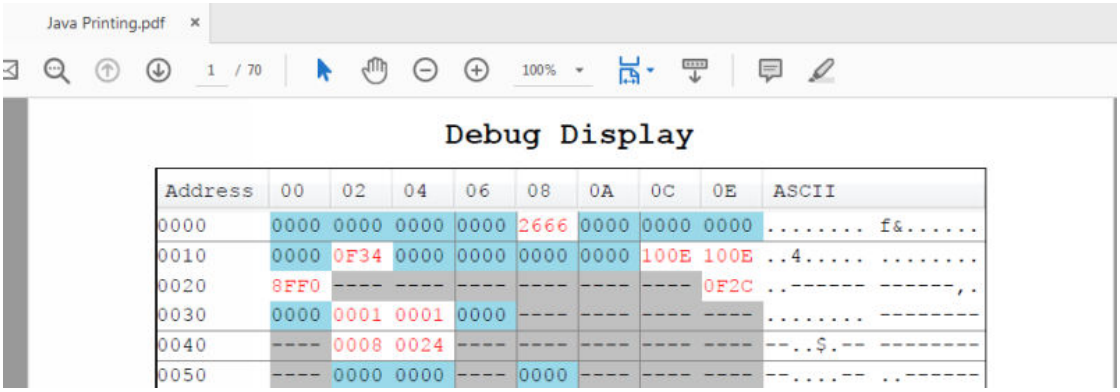


Table 13-64. Print Dialog

Item	Description
General Tab	Set up general print options. **Print Service**: Select a printer/print service. Click Properties to set up the print service. Also, check to print to a file. **Print Range**: Range of pages to print. **Copies**: Number of copies to print.
Page Setup	Set up page options. **Media**: Specify page size and source. **Orientation**: Specify page orientation. **Margin**: Specify page margins.

.....continued	
Item	**Description**
Appearance	Set up appearance of print out. **Color appearance**: Specify color or monochrome. **Quality**: Specify draft, normal or high. **Sides**: Specify sides and multi-side orientation. **Job Attributes**: Print a banner, specify job name and user.

13.15 Message Center

Open the Message Center window from *Window>Debugging>Output>Message Center*.

The Message Center is a window that collects all the output window messages and then presents them chronologically into one view. Different filters can be selected to display different relevant messages.

Figure 13-44. Message Center Window

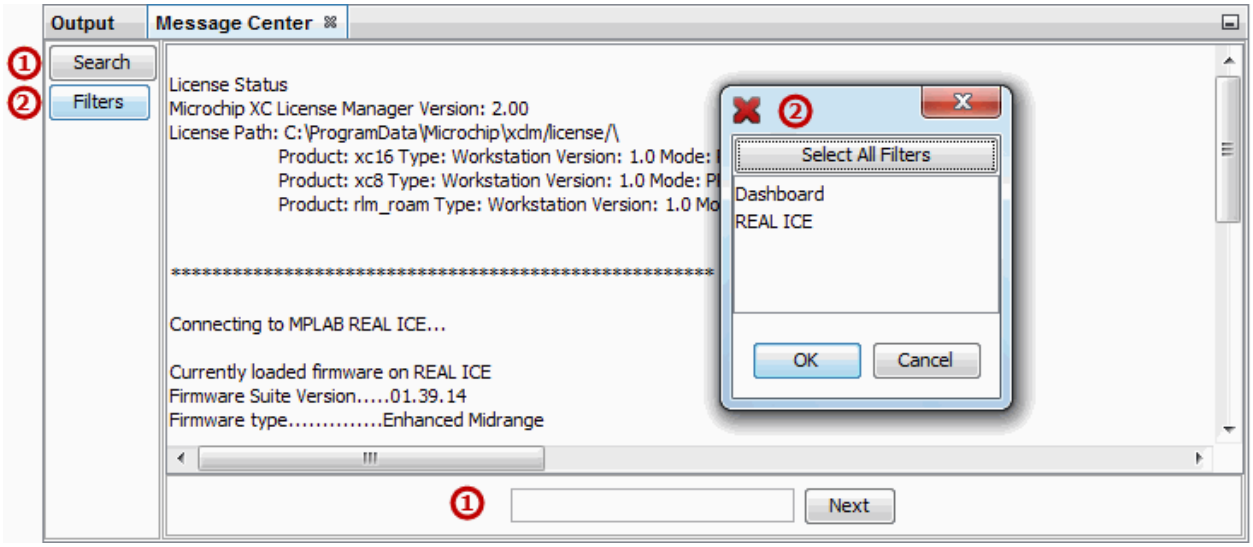


Table 13-65. Message Center Buttons

Number	Button	Function
1	Search	Toggle the search text box. When the search box is visible, enter text to search for that text string in the Message Center window.
2	Filters	By default, all messages that appear in the Output window are displayed in the Message Center window. Click this button to open a window with a list of Output window tabbed items. Filter messages displayed by selecting one or more from the list.

13.16 Output Window

Open this window by selecting *Window>Output*.

The Task pane contains many windows, some inherited from the NetBeans platform and some that are specific to MPLAB X IDE. The Output window contains the MPLAB X IDE output information. It is shown on tabs within the window.

Table 13-66. Output Window Tab Items

Item	Description
Debugger Console	Shows main debug actions, such as “User program running.”
Tool-specific	Shows tool firmware version, device ID, and action status.
Build, Load	Shows information and status on the build, and program loading.
Clean, Build, Load	Shows information and status on the clean, build, and program loading.
Peripheral Output	Shows technical output from peripherals such as the UART, with the Simulator as the debug tool.

13.16.1 Content Scrolling

Programming actions taking place will be shown and scrolled in an Output window. The last action executed will be shown at the bottom of the window.

If you scroll up to view the content and then program again, the cursor will not move from your current location. This is to keep you from losing your place even though more content is filing the window below. To see the new content, you must manually scroll down.

13.16.2 Context Menu

Right clicking in the Output window will display various options as shown below.

Table 13-67. Output Window Context Menu

Item	Description
Copy	Copies selected text from the Output window to the clipboard.
Paste	Pastes selected text from the clipboard to the Output window.
Find	Finds the selected text, or enters other text to find, in the Output window. You may use regular expressions and match case.
Find Next	Finds the next occurrence of the Find text.
Find Previous	Finds the previous occurrence of the Find text.
Filter	Filters the output by text or regular expression.
Wrap Text	Wraps the text in the Output window.
Larger Font	Makes the font larger.
Smaller Font	Makes the font smaller.
Settings	Selects font size, color, etc.
Save As	Saves the selected text to a file.
Clear	Clears all text in the Output window tab.
Close	Closes the Output window.
Output Folds	Shows options for Collapse/Expand Fold, Collapse/Expand All or Collapse/Expand Fold Tree.

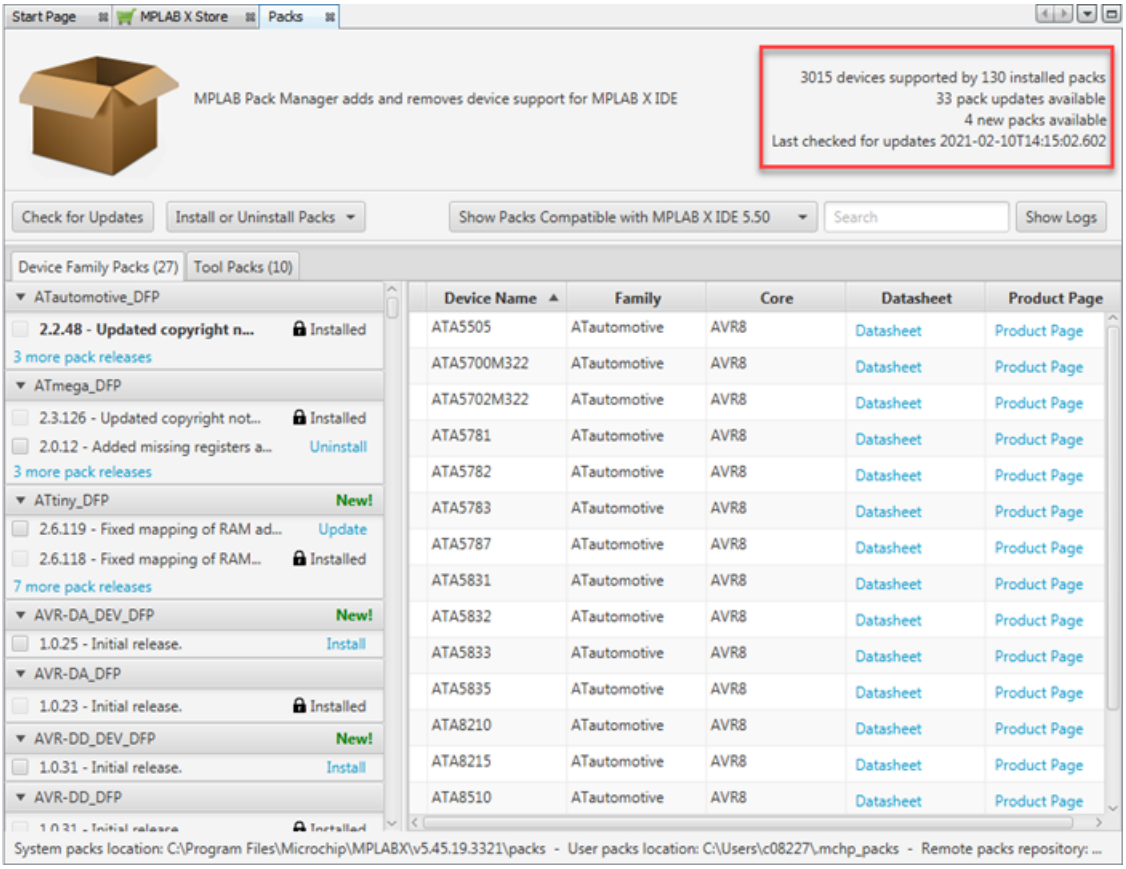
13.17 Pack Manager Window

Select *Tools>Packs* to open the MPLAB Pack Manager window.

On opening, MPLAB X IDE will look for updates to and installations of packs. The results are shown in the upper right corner of the window.

© 2022 Microchip Technology Inc. and its subsidiaries

Figure 13-45. Pack Manager Window



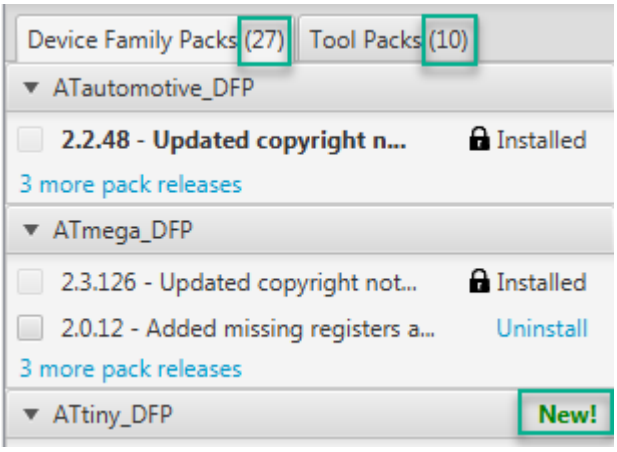
Related Links

6.1.3. Select Packs after MPLAB X IDE Install

13.17.1 Device Family Packs and Tool Packs

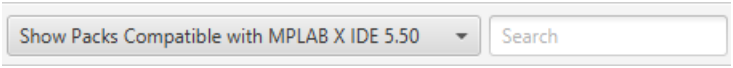
Device Family Packs (DFPs) contain device information files (.PIC, etc) for all devices in a family. Tool Packs (TPs) contain information files for using a development tool, such as MPLAB PICkit 4.

Packs for each type are displayed in tabbed windows. The number of new packs for each is also displayed on the tab and next to the name of the new pack.



13.17.2 Show Packs

Initially the Pack Manger window is set to “Show Packs Compatible with MPLAB X IDE vx.xx” where “x.xx” is the current version of the IDE. To change this filter, click on the down arrow to see more options (shown in the table below.) Use the Search box to enter a specific device and find the selected pack(s) that provide support.



Packs selected for display are shown in the Device Family Packs and Tool Packs tabs.

Table 13-68. Show Packs Options

Selection	Description
Show Packs Compatible with MPLAB X IDE vx.xx	Show all packs that have been tested to work with the current IDE version.
Show All Packs	Show all available packs regardless of compatibility with the current IDE version.
Show Only New Packs	Show only new packs, i.e., packs released after the release of the current IDE.
Show Only Pack Updates	Show only packs that have updates, i.e., updated packs released after the release of the current IDE.
Show Deprecated Packs	Show packs that have been deprecated, i.e., packs that should no longer be used for development in the current IDE.

13.17.3 Individual Pack Actions

For individual packs:

1. Actions available for each pack version are shown on the right. Click on an action in blue to perform it. Actions are described in the table below.
2. To see more pack versions, click on “*N* more pack releases.”
3. For the “Device Family Pack” tab, information about the devices in a selected pack version (in bold) is displayed in another pane to the right. To select a pack version, click on the checkbox next to the version name.

© 2022 Microchip Technology Inc.
and its subsidiaries

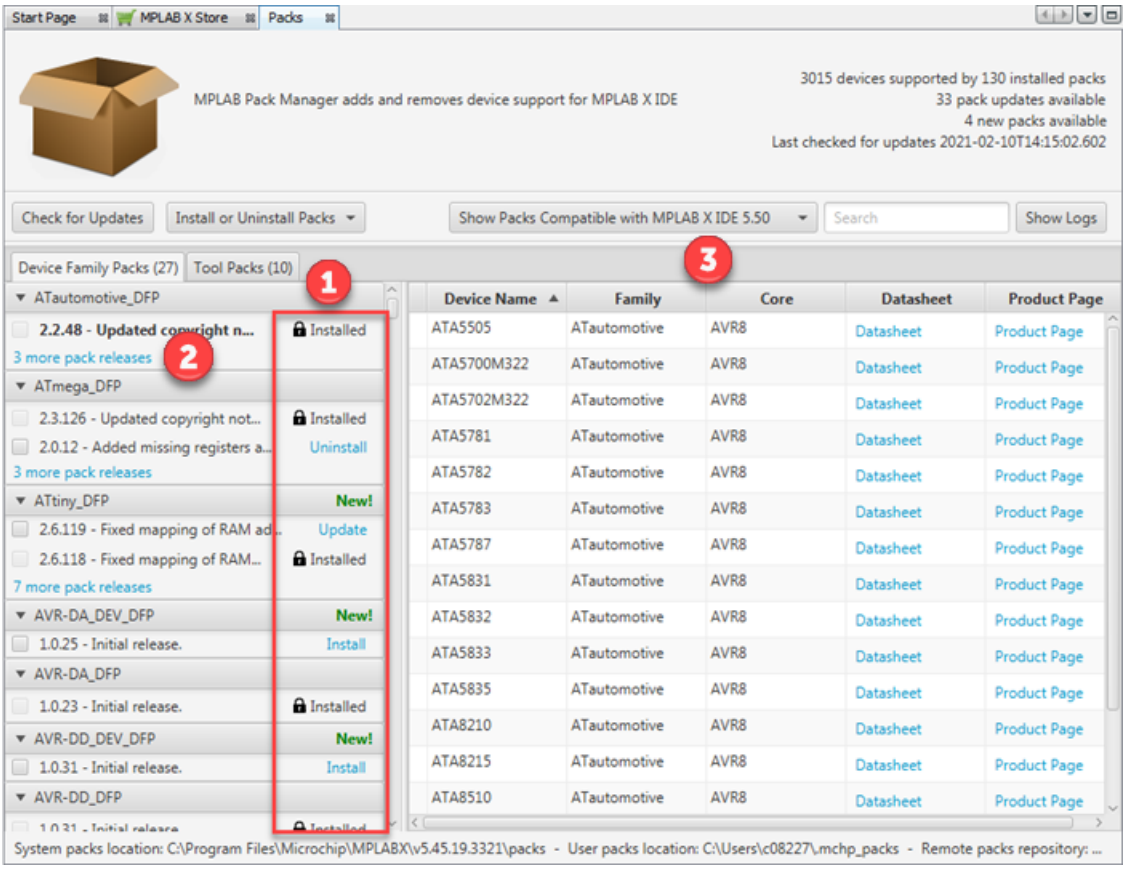


Table 13-69. Pack Actions

Action	Description
Installed	Pack version was installed with the MPLAB X IDE version to a system area and cannot be removed.
Installed	Pack version available for install.
Uninstall	Uninstall pack version. To reinstall, see 6.1.5. Packs on the Web.
Update	Pack version has an update available.

If a new device has a feature that requires support from one or more tools, then installing or updating the device-related DFP will automatically install or update any necessary TPs as well.

13.17.4 Multiple Pack Actions

To perform actions on multiple packs:

1. Check the checkbox next to pack versions to select multiple packs.
2. Click on the “Install or Uninstall Packs” button to install or uninstall selected packs.
   Information on this and other actions, visible by clicking the down arrow, can be found in the table below.
3. Click on “Check for Updates” to find all the latest updated packs.

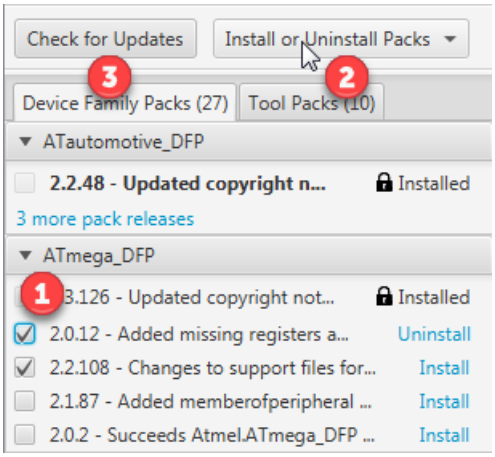
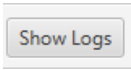


Table 13-70. Multiple Pack Actions

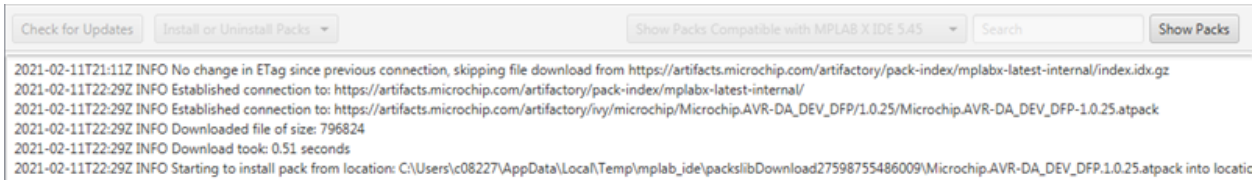
Action	Affected Packs
Install or Uninstall Packs	Pack(s) checked
Install Selected Packs	Pack(s) checked
Uninstall Selected Packs	Pack(s) checked
Install Pack Updates	All Packs
Install Pack from Local Source	Pack checked. Selecting will open a file browser window to locate the local source pack file.

13.17.5 Show Logs

To see how an action has progressed, click “Show Logs” next to the Search box.



View the log messages. The “Show Logs” button will change to “Show Packs” so you can return to pack view.



13.17.6 Offline Actions

Some customers may want to use packs offline, that is, disable checking the pack database from the internet. This is accomplished by adding the option `-J-Dmicrochip.connect=false` to the `mplab_ide.conf` file or as an argument to MPLAB X IDE startup.

In this mode some actions are **not available**:

- “Install” (except for “Install Pack from Local Source”)
- “Update” or “Check for Updates”
- For DFPs, device “Datasheet” and “Product Page” links

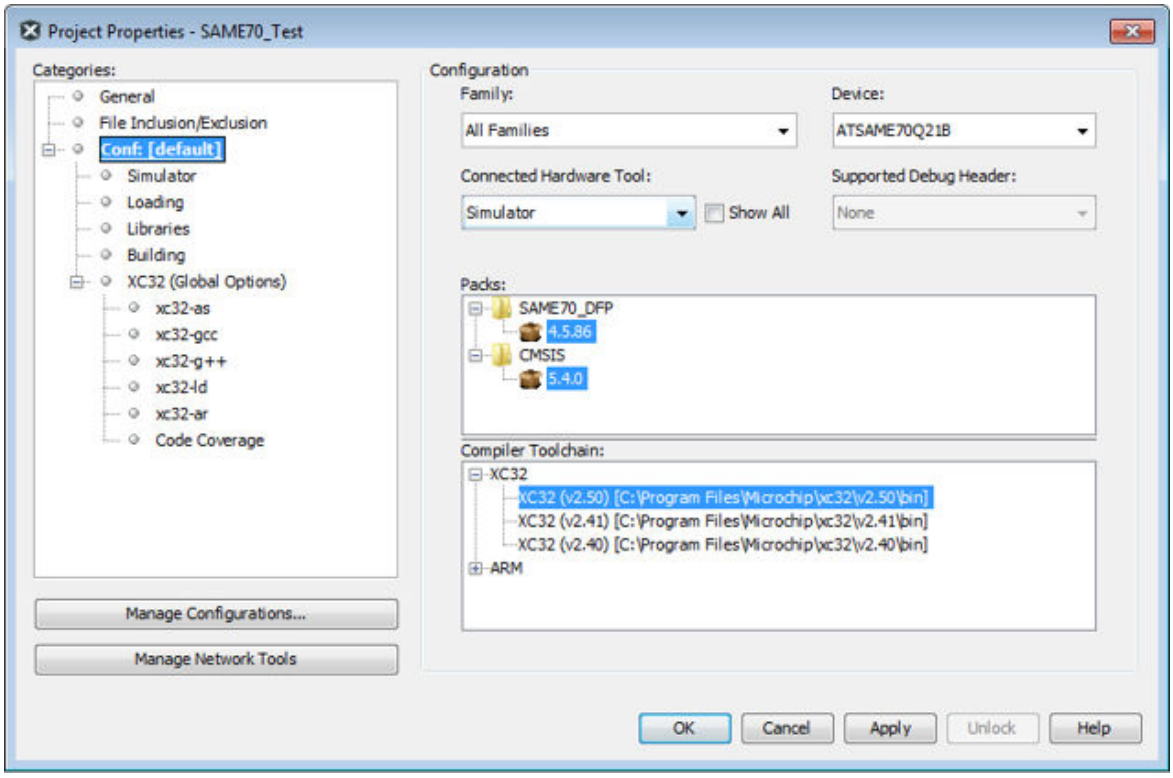
13.18 Project Properties Window

There are several ways to open this window. See 5.3. Open Project Properties.

This window is used to view or change the project device, tools and tool settings. For more information, see:

- 5.4. View or Make Changes to Project Properties
- 5.5. Set Up or Change Debugger/Programmer Tool Options
- 5.6. Set Up or Change Language Tool Options
- 6.2.5. Loadable Projects, Files and Symbols
- 5.9.5. Add and Set Up Library and Object Files
- 5.10. Set Build Properties

Figure 13-46. Project Properties Window



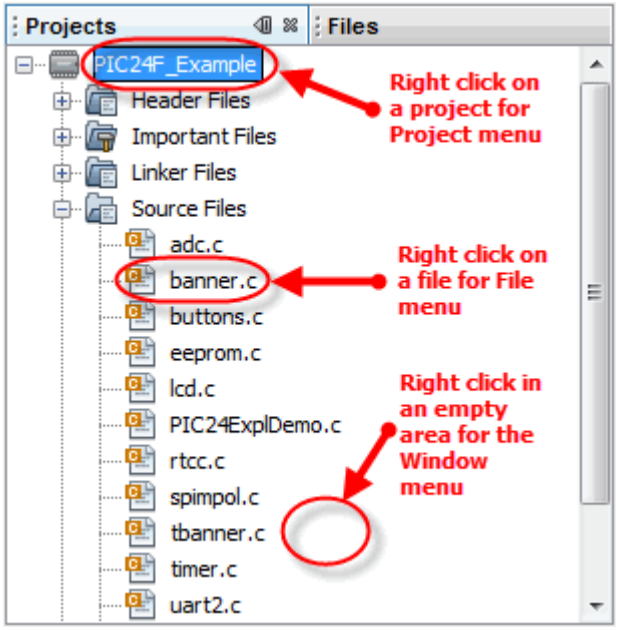
13.19 Projects Window

Open this window by selecting *Window>Projects*.

The Projects window is a NetBeans platform window. However, it has been customized to show the relevant logical (virtual) folders for an MPLAB X IDE project. Also, the context (right click) menus have been customized with items that are specific to MPLAB X IDE.

- Projects Window – Logical Folders
- Projects Window – Project Menu
- Projects Window – File Menu
- Projects Window – Window Menu

Figure 13-47. Projects Window Context Menus



Projects Window – Logical Folders

Additional logical folders have been added that apply to MPLAB X projects. Therefore, all folders displayed can be categorized as one of the following types of files:

- **Header Files** – MPLAB X IDE does not use this category when building. Consider it as a means to document a project's dependency on a header file and a convenient method to access these files. You can double click on a file in the Projects window to open the file in an editor.
- **Important Files** – Any file that does not fit into any of the other categories will end up in this folder.
  You can add project-specific data sheets (PDFs) to this location in the Projects window. Then, you can double click on the PDF to launch the data sheet (this requires that a PDF reader is installed).
- **Linker Files** – You do not need to add a linker script to your project, because the project language tools will find the appropriate generic linker script for your device. So this folder will be empty, unless you have created your own linker script.
  There should be only one file in this folder. If you have more than one linker script, only the first one has any effect – it is the linker script that the tool will use in the link step.
- **Source Files** – These are the only files that the toolchain will accept as input to its file commands.
- **Libraries** – The toolchain should take all of the files in this folder, as well as the object files, and include them in the final link step.
- **Loadables** – Projects and files to combine with or replace your project hex files.

Projects Window – Project Menu

Right click on a project name in the Projects window to pop up the Project menu. Menu items are shown in the table below.

Table 13-71. Project Context Menu Items

Menu Item	Description
New	Adds a new item For MPLAB® X IDE; embedded file types are available.
Add Existing Item	Adds an existing file to the project. The path to the file may be auto (selected by MPLAB X IDE), absolute or relative. Additionally, you may choose to copy the file into the project folder.

.....continued

Menu Item	Description
Add Existing Items from Folders	Adds all files contained in the specified folder(s). You may also specify a pattern to ignore certain folders from inclusion. Click **Default** to see the default pattern. See also *Tools>Options*, **Miscellaneous** button, **Files** tab, “Files ignored by the IDE.”
New Logical Folder	Creates a new logical folder. To view actual folders, see the Files window.
Locate Headers	Locates all the header files (.h) called out in the C code project files and presents them in a checklist window so they may be added to the project There are several reasons why you might want to add headers to your project: • The files can be opened from the Projects window under “Header Files” instead of hunting for them on your computer. • The project created by “Save Project As” will not contain header files unless they are added to the project; therefore this project may not build if any user-generated header files are required. • The zipped project created by “Package” will not contain header files unless they are added to the project; therefore the unzipped project may not build if any user-generated header files are required. This feature is able to locate user-generated header files for all Microchip toolchains.
Add Item to Important Files	Adds an existing item to the project. The path to the file may be auto (selected by MPLAB X IDE), absolute or relative. Additionally, you may choose to copy the file into the project folder. This is useful for adding simulator files, data sheet PDFs, and other files to the project for your reference.
Export Hex	Exports the project build as a Hex file. In MPLAB IDE v8, export is an extraction of memory objects. In MPLAB X IDE, export is the hex file result of a build; therefore, it needs to include all necessary auxiliary memory settings for configuration and EEPROM in code.
Build	Builds the project according to the options selected in the Project Properties window. **Note:** When you Debug or Run, your project is built automatically.
Clean and Build	Cleans and then builds the project according to the options selected in the Project Properties window. **Note:** When you Debug or Run, your project is built automatically.
Clean	Cleans the project by deleting the outputs of previous builds.
Build Batch	Builds one or more configurations.
Package	Packages the current project into a .zip file.
Set Configuration	Opens the Project Properties window so you can set the project configuration.
Run	Executes the project code.
Debug	Executes the project code in a debug environment.
Step Into	Steps into Paused code running in the debug environment.
Learn more about scripting	Opens the WebHelp topic “Use IDE Scripting (autoload.py)”.
Make and Program Device	Programs the target device and holds in Reset (do not run).

© 2022 Microchip Technology Inc. and its subsidiaries

.....continued

Menu Item	Description
Set as Main Project	Sets this project as the main project. This is useful when you are working with multiple projects.
Open Required Projects	Loads all other projects that are required for the selected project to run.
Close	Closes the selected project.
Rename	Renames the project.
Move	Moves the project to another location. Along with the project, it moves the source files that are inside the project directory. Files outside the project directory are not moved. However, the project still maintains reference to the files outside the project directory – this is by design.
Copy	Creates a copy of the project. If the source files are within the project directory, the source files are also copied to the new location. Files outside the project directory are not copied. However, the project still maintains reference to the files outside the project directory – this is by design.
Delete	Deletes the project files. The project file is deleted but not the contents under the project directory. Only on selecting “Also delete sources” are all the source files deleted.
Code Assistance	Selects assistance in creating code: code folding, code completion, etc.
Find	Finds specific text in files in this project.
Analysis	Displays list of analysis tools for selection.
Show Code Coverage Summary	Opens Code Coverage window to display summary.
Versioning	Controls the version of the project by using a version control system.
History	To view, or revert to, the local history for the project.
Properties	Sets project properties. For MPLAB X IDE, the Project Properties window is specific to embedded development.

Projects Window – File Menu

Right click on a file name in the Projects window to pop up the File menu. The table below shows the specific menu items.

Table 13-72. File Context Menu Items

Menu Item	Description
Open	Opens this file in a tabbed Editor window.
Cut	Removes the file from the project, but places a copy of it on the clipboard.
Copy	Places a copy of the file on the clipboard.
Paste	To paste the clipboard copy of a file into the project.
Exclude file(s) from current configuration	Select a file or files to exclude from the current project configuration. Once a file has been excluded, right clicking on it again will show “Include file(s) from the current configuration.” The same as opening the Project Properties window, selecting “File Inclusion/Exclusion” and including/excluding the file.
Compile File	Compiles only the current file.

© 2022 Microchip Technology Inc.
and its subsidiaries

.....continued

Menu Item	Description
Remove from Project	Removes the file from the project. This does not delete the file from the PC. To delete the file from the project and the computer, use the Delete key.
Rename	Renames the file.
Save as Template	Saves the current file as a template file.
History	To view, or revert to, the local history for the file.
Tools	File tools include: • **Apply Diff Patch**: applies an existing patch created by Diff. • **Diff to**: shows the differences between this file and the one specified here. • **Add to Favorites**: adds this file to the Favorites window.
Properties	Sets file properties differently from the project properties. Select to exclude the file from the build or override the project build options by selecting a different configuration.

Projects Window – Window Menu

Right click in an empty area in the Projects window to pop up the Window menu. The table below shows the specific menu items.

Table 13-73. Window Context Menu Items

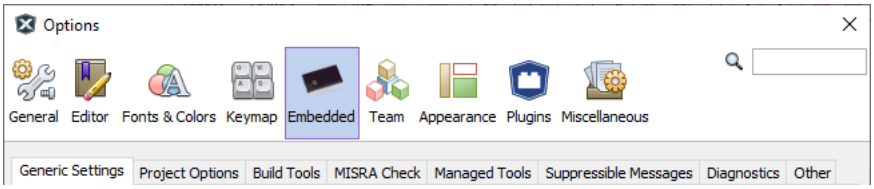
Menu Item	Description
New Project	Launches the New Project wizard.
New File	Launches the New File wizard.
Open Project	Opens an existing project.
Open Recent Project	Opens an existing project for the list of recent projects.
Project Groups	Opens the Manage Groups dialog to select, create, remove or view project groups.
Build Project (*Project Name*)	Builds the selected project shown in parentheses.
Clean and Build Project (*Project Name*)	Cleans and Builds the selected project shown in parentheses.
Set Main Project	Sets the main project from the list of open projects.
Collapse All	Collapses all tree items.
Show Selected Node(s) Project Owner	See NetBeans Help, Projects Window.

13.20 Tools Options Window, Embedded

Select *Tools>Options* (*MPLAB X IDE>Preferences* for macOS) and then click the **Embedded** button to see the tabs and options listed below.

The Options window is a NetBeans platform window. However, it has been customized for MPLAB X IDE projects.

Figure 13-48. Tools Options Embedded Tabs



13.20.1 Generic Settings Tab

Select *Tools>Options* (*MPLAB X IDE>Preferences* for macOS) and then click the **Embedded** button, **Generic Settings** tab, to see the options listed below.

Table 13-74. Generic Settings Tab Items

Item	Description
Open source file and locate line in editor when debugger halts	Jump to a line of code on halt, such as caused by a breakpoint, to examine the code. Disable to simply halt execution.
Clear tool output window on new session (debug, program, upload)	Clear out the contents of the Output window when you begin a Run, Debug or upload.
Halt build on first failure	When building, halt the process on the first failure. The selected project language tool can be set up to determine which errors are produced. Go to the Project Properties window and select the language tool under “Categories.” Then look for through the Option Categories to find one that allows you to set up errors, warnings, and/or messages.
Maintain active connection to hardware tool	If selected, keep hardware tool connected always, not just at runtime (MPLAB® IDE v8 behavior). When switching projects with this option selected (e.g., when developing bootloading applications), ensure tool and device are the same to avoid error messages. **Note:** This option applies only to the current project and configuration. If you switch projects or change items in the Project Properties, a reconnect will occur.
Read Device Memory to File: Export only memory used	For the “Read Device Memory to File” icon/function, save only device memory used to a file.
Silent build	Build without generating messages in the Output window.
Enable alternate watch list views during debug session	Display three watch view diamonds in the Watches window. Associate a watch view with a watch variable. When you click on a watch view diamond, only the variables associated with that view will be displayed. So, this feature works like a filter.
Disable auto refresh for call stack view during debug sessions	Enable to automatically update contents on Pause or Halt. If the window is closed or not focused, selecting the window and clicking this button will display the updates without having to run and pause/halt again. Disable to manually update contents on Pause or Halt. This button must be clicked to update window contents on a pause/halt.
On mouse over structure and array expressions during a debug session, evaluate integral members only	Checked shows only integral members when mousing over structures or array expressions in code. Unchecked shows all members when mousing over structures or array expressions in code.

.....continued

Item	Description
Show unresolved variable names in watch window during debug session	Check to make unresolved variables visible in Watches window.
Enable gathering of compiler symbols	Run the compiler behind-the-scenes, requesting information on compiler built-ins and private macros. This option is disabled by default. Enabling it will increase the time it takes the IDE to gather symbol information. **Note:** If you do not rely on compiler's internal information, you do not need to enable this feature.
On mouseover source lines in editor, evaluate break point status	Select what triggers evaluation. **Disable**: No evaluation. **Mouse Only, Alt+Mouse, Shirt+Mouse, Shift+Alt+Mouse**: Evaluate values at a breakpoint based on the one of these keystrokes.
Hold-off period before memory view synchronization: Give priority to debug events (step over, step in, continue)	Specify a delay between when the device halts and when the device data is synced with a memory window. A longer delay can be set so that more single step executions can occur before an update. This is only applicable to general PIC Memory Views like file registers and data memory.
Debug Reset @ (following reset action during paused debug session)	Select action for a debug reset, i.e., *Debug>Reset*: **Main**: Stop at `main()` on Reset. **Reset Vector**: Stop at the Reset vector on Reset. **Note:** This is not related to a hardware reset of the device.
Debug start-up (following debug project action)	Specify code execution for *Debug>Debug Project*: **Run**: Start execution immediately. **Main**: Stop at `main()`. **Reset Vector**: Stop at the Reset vector.
Default Charset	Select the default character set for the project.
Exclude device checks for kits	Do not check kits for currently-supported devices.
MPLAB Code Coverage Report Type	**None**: Do not generate a report. **HTML**: HTML format **GCOV**: GCOV output format, i.e., a copy of the source code annotated with execution frequencies. **All Types**: Create multiple reports of types above.

13.20.2 Project Options Tab

Select *Tools>Options* (*MPLAB X IDE>Preferences* for macOS) and then click the **Embedded** button, **Project Options** tab, to see the options listed below.

Table 13-75. Project Options Tab Items

Item	Description
Make Options	Enter make options to use when building projects. These options are toolchain dependent. See your language tool documentation.

.....continued

Item	Description
File Path Mode	Specifies how to store file path information in a project: **Auto**: paths to files inside project folder stored as relative; paths to files outside project folder stored as absolute. **Always Relative**: all paths stored as relative to project folder. **Always Absolute**: all paths stored as absolute (full path).
Save All Modified Files Before Running Make	If selected, saves all unsaved files in the IDE before running make. It is recommended to leave this property selected because modifications to files in the IDE are not recognized by make unless they are first saved to disk.
Reuse Output Tabs from Finished Process	When selected, output messages are displayed in the same tab in the Output window. When deselected, a new tab is opened for each new process.
Use parallel make (`make -j 2n`)	If selected, make will execute several processes at a time, where `-j` (or `--jobs`) is the option to run in parallel and 2n is the number of processes, where n is the number of processors available on your computer. If your computer does not support parallel processing, parallel make will be disabled. If you wish, you can specify more processes by using “Make Options.” Example: `-j 10`. **Note:** For MPLAB XC16 or MPLAB C30, Procedural Abstraction needs to be turned off to use parallel make (*File>Project Properties*, compiler category, “Optimizations” option category: uncheck “Unlimited procedural abstraction”).
Force makefile regeneration when opening a project	Uncheck to allow MPLAB X IDE to determine whether or not a makefile should be regenerated when a project has opened (e.g., opening a project on a different computer). This is the default. Check to force regeneration of the makefile on project open. Use this if you suspect the makefile is not being regenerated when it needs to be. **Note:** Regeneration can take some time.
Use “clean” target from Makefile	Uncheck to use the standard clean, i.e., delete all build artifacts. This is faster. Check to use make clean, i.e., use the make file when performing clean. This gives more control to what is cleaned.
Minimize rebuilding on makefile changes	Check this option to rebuild only the minimum amount even when the makefiles are regenerated.

13.20.3 Build Tools Tab

Select *Tools>Options* (*MPLAB X IDE>Preferences* for macOS) and then click the **Embedded** button, **Build Tools** tab, to see the options listed below.

Ensure that you have INSTALLED THE LANGUAGE TOOLCHAIN or it will not show up on the “Toolchain” list. If you know you have installed it but it is not on the list, click **Scan for Built Tools**. If it is still not found click **Add** to add the tool to the list.

Other tools may be obtained from the Microchip web site (www.microchip.com) or third parties.

Table 13-76. Build Tools Tab Items

Item	Description
Toolchain	Shows a list of language tools installed on your computer. Select the tool for your project and ensure that the paths (that apply) to the right are correct. **Base directory**: Path to the tool's main folder. **C compiler**: Path to the C compiler (if available). **Assembler**: Path to the assembler (if available). **Make command**: Name of the make command generated by MPLAB® X IDE.
Add	Add a new language tool item to the list. Also consider using **Scan for Build Tools**.
Add Custom Compiler	Add a customized compiler. In this case, you must specify Microchip devices supported. **Note:** f the compiler is not currently supported by MPLAB X IDE, you may be able to use it by editing or creating your own makefile outside of the IDE. See: microchipdeveloper.com/mplabx:work-outside-create-makefiles
Remove	Remove a language tool item from the list. This does not remove the language tool from the computer.
Default	Click on a tool and then click **Default** to make this tool the default compiler/assembler for the selected device.
Scan for Build Tools	Scan the computer for installed compilers/assemblers in various default locations, **not** the whole system. If you install in a different location, add the compiler manually.

13.20.4 MISRA Check Tab

The MISRA® Check tab below supports checking your application code against these standards. For more on how to use MISRA Check and make selection on this tab, see the "MPLAB Analysis Tool Suite User's Guide".

13.20.5 Managed Tools Tab

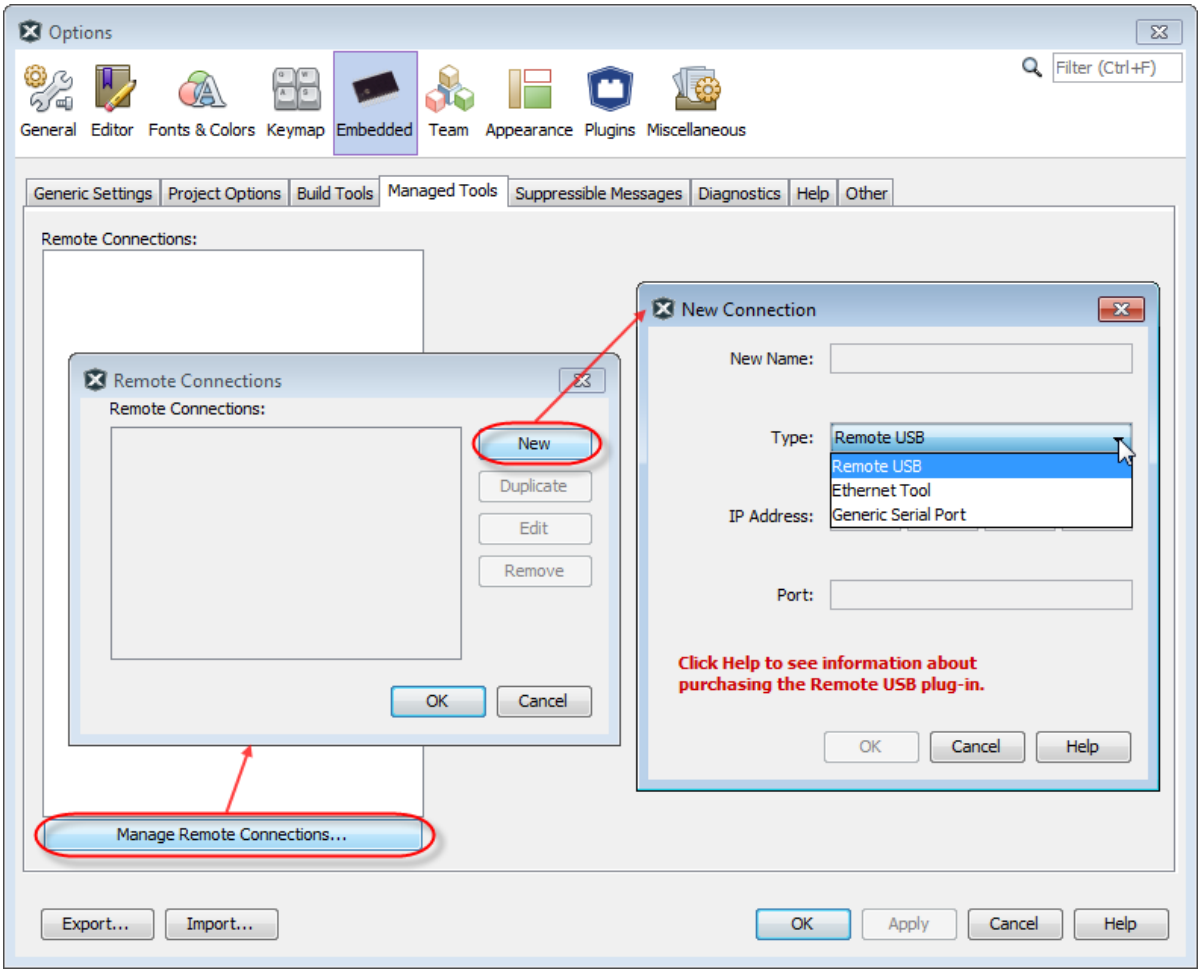
Select *Tools>Options* (*MPLAB X IDE>Preferences* for macOS) and then click the **Embedded** button, **Managed Tools** tab, to see the options listed below.

Select remote connections to use. Add a new connection for a:

- Remote USB Tool (Plugin required)
- Remote Ethernet Tool
- Generic Serial Port

© 2022 Microchip Technology Inc. and its subsidiaries

Figure 13-49. New Managed Tool

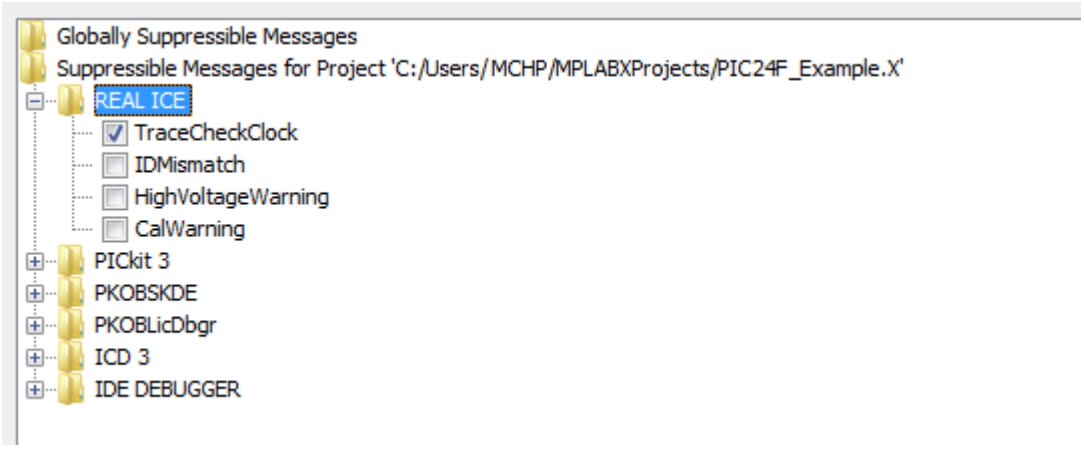


13.20.6 Suppressible Messages Tab

Select *Tools>Options* (*MPLAB X IDE>Preferences* for macOS) and then click he **Embedded** button, **Suppressible Messages** tab, to see the options listed below.

Select error and warming messages to suppress. Double click on available folders to drill down to available items to suppress.

Figure 13-50. Suppressible Messages List



13.20.7 Diagnostics Tab

Select *Tools>Options* (*MPLAB X IDE>Preferences* for macOS) and then click the **Embedded** button, **Diagnostics** tab, to see the options listed below.

Set up the log file and other diagnostic features.

Table 13-77. Diagnostics Tab Items

Item	Description
Logging Level	Set the message logging level: **OFF**: no logging. **SEVERE**: log severe (error) messages only. **WARNING**: log warning messages only. **INFO**: log informational messages only. **CONFIG**: log configuration information only. **FINE**: log some module to module communication. **FINER**: log more module to module communication. **FINEST**: log all module to module communication.
Clear Existing Logs	Click to delete existing log file contents.
Log File	The path and name of log file.
USB Circular Log	Applies only to MPLAB REAL ICE in-circuit emulator, MPLAB ICD 3 and PICkit 3.
Start New Logging Session/ Pause Logging	Click buttons to begin a new logging session or pause the logging session in progress.
Circular USB log file location	Specify the path for the log file.
Circular USB log file max size in KBs	Specify the size of the log file.

13.20.8 Other Tab

Select *Tools>Options* (*MPLAB X IDE>Preferences* for macOS) and then click the **Embedded** button, **Other** tab, to see the options listed below.

Edit the lists of accepted file extensions for C/C++ and assembler source files and header files. Also, set the default extension for each type (displayed as bold).

Table 13-78. Other Tab Items

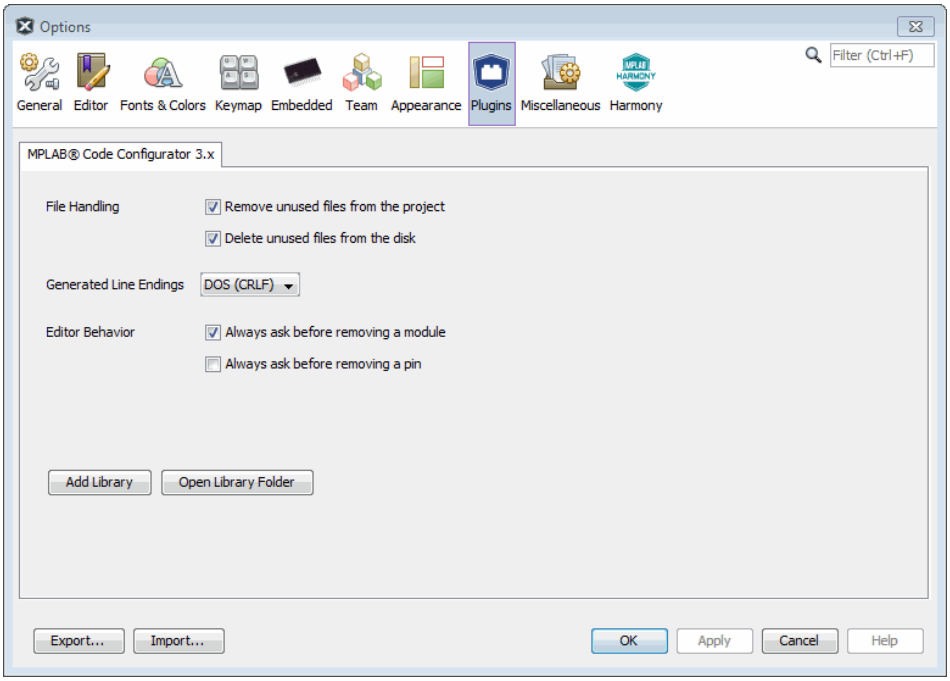
File Extensions	Default Extensions
C/C++ Header	H, SUNWCCh, **h**, hpp, hxx, tcc
C++	C, c++, cc, **cpp**, cxx, mm
C	**c**, i, m
Fortran	Not applicable. MPLAB X IDE does not support Fortran programming.
Assembler	AS, ASM, S, as, asm, **s**
Assembler Include	INC, **inc**

13.21 Tools Options Window, Plugins

Open this window using *Tools>Options* (*MPLAB X IDE>Preferences* for macOS) and then click the **Plugins** button.

The Options window is a NetBeans platform window. However, it has been customized for MPLAB X IDE projects. If plugin(s) have been installed (using *Tools>Plugins*) and they have plugin options, go to this window to see the available plugin options.

Figure 13-51. Plugin Options Example



13.22 Trace Window

Open the Trace window by selecting *Window>Debugging>Trace*. You must be in debug mode to see trace data.

Tracing allows you to record the step-by-step execution of your code and examine this recording in the Trace window. Trace is currently available for the following tools:

- Simulator
- MPLAB REAL ICE in-circuit emulator

Right clicking on a trace column in the window will pop up the context menu (first table). Depending on the tool you are using, you may or may not see all menu items.

Dialogs associated with trace are defined in the second table.

Table 13-79. Trace Window Context Menu

Menu Item	Description
Symbolic Mode	For the “Instruction” column, toggle between displaying literal register addresses (e.g., 0x5) and symbolic register macros (e.g., PORTA).
Go To	Opens a Go To dialog. **Trigger**: move to the trigger line (0). **Top**: move to the top trace line. **Bottom**: move to the bottom trace line. **Trace Line**: specifies and goes to a trace line location.
Go To Source Line	Selects a trace line and then selects this option to go to the corresponding line in source code.

.....continued	
Menu Item	**Description**
Display Time	For the “Cycle” column (will display if not previously visible): **As Hex Cycle Count**: displays cycle count as hexadecimal. **As Decimal Cycle Count**: displays cycle count as decimal. **In Seconds Elapsed**: displays cycle count in seconds elapsed. **In Engineering Format**: displays cycle count in the appropriate engineering format (powers of 10^3).
Clear Trace File	Clears the data in the trace display.
Find	Finds items in trace display. Opens a Find dialog.
Output To File	Saves the trace data to a file. Opens Define Range dialog, which opens a Save dialog.
Print	Print the data. Opens a Print dialog.
Adjust Table Columns	Adjusts the columns in the trace display to fit the data automatically.
Reload View	Reloads the original data view for the trace display at pause.

Table 13-80. Trace Dialogs

Dialog	Description
Go To	Specifies a trace line to go to.
Find	Finds a line number or other data in the trace display.
Output to File Range	Specifies a range of lines to output to a file. Click **OK** to proceed to the Save dialog to save the data to a text file.
Save	Saves data to a text file.
Print	Specifies the printer, page setup, and appearance before printing.

13.23 Variables Window

To open the Variables window, select *Window>Debugging>Variables*. You must be in debug mode to watch local variable and watches values change.

Variables Display
The following display features are discussed in the sections below: icons, columns and actions (buttons).

Icons

Icons are found to the left of the name object in the Name column:

Table 13-81. Name Column Icons

Icon	Description
	Local variable
	Local pointer variable

.....continued	
Icon	**Description**
	Static local variable
	Static local pointer variable
	Field
	Pointer field
	Static field
	Static pointer field

Columns

You can change the columns displayed in the window by:

- Right clicking on a heading and checking/unchecking a heading to be displayed.
- Clicking on the “Change Visible Columns” icon in the top right of the window  . In the Change Visible Columns dialog, check/uncheck a heading to be displayed.

Actions

Actions are on buttons on the left side of the window:

Table 13-82. Action Button Icons

Button	Description
 	Show only local variables near the PC. Show all local variables.
	**Selected**: Make watches visible in the Variables window and enables creation of a new watch. **Not selected**: Hide watches in the Variables window.
	Create a New Watch in the New Watch dialog.

Variables Menu

If watches are hidden in the Variables window, the following menu is visible by clicking in a row.

If watches are visible, additional watch functions will also be visible by clicking in the window or a row.

Table 13-83. Variable Window Menu – Row

Menu Item	Description
Display Value Column as	Display the symbol value in the selected row in one of the formats listed.
Export Data	Export watch data to a CSV file or as a literal string.
Show Only Rows Where	Show only those rows with the specified criteria. Criteria available depends on column content.

Related Links

13.24. Watches Window
5.17. Watch Local Variable Values Change

13.24 Watches Window

To open the Watches window, select *Window>Debugging>Watches*. You must be in debug mode to watch symbol values change.

Select global symbols and SFRs and watch values change on program halt or during runtime (if supported).

For information on using the Watches window, see 5.16. Watch Symbol Values Change.

- Watches Display
- Watches Menu
- Fractional Integer Properties Dialog

Related Links

5.16. Watch Symbol Values Change

13.23. Variables Window

13.24.1 Watches Display

The following display features are discussed in the sections below: icons, columns and actions (buttons).

Icons

Icons are found to the left of the name object in the Name column:

Table 13-84. Name Column Icons

Icon	Description
	Watch object
	Watch object in Program Memory
	Static field of an object
	Non-static field of an object

Columns

You can change the columns displayed in the window by:

- Right clicking on a heading and checking/unchecking a heading to be displayed.
- Clicking on the “Change Visible Columns” icon in the top right of the window  . In the Change Visible Columns dialog, check/uncheck a heading to be displayed.

Actions

Actions are on buttons on the left side of the window:

Table 13-85. Action Button Icons

Button	Action
	Toggle button: • Show verbose (qualified) names of member fields. • Show brief (relative) names of member fields.
	Import watches from list file. Right click for options.
	Export all watches to list file.

.....continued

Button	Action
	Set the default numeric format for the Value field.

13.24.2 Watches Menus

Right click in a blank area of a Watches window (not a row) to open a Watches window menu with basic functions.

Table 13-86. Watches Window Menu – Window

Menu Item	Description
New Watch*	Add a new symbol to watch.
New Runtime Watch*	**Only visible if debug tool supports runtime watches.** Add a new runtime watch for the selected symbol.
Export All Watches to List File	Export information about the symbols to be watched to a file (.xwatch).
Delete All	Remove all the watched symbols from the Watches window.
* You can also right click in a symbol in code to add a New Watch or New Runtime Watch.	

Right click on a row to open a Watches menu with additional functions that depend on whether a symbol is in the row or what debug tool you are using.

Table 13-87. Watches Window Menu – Row

Menu Item	Description
New Watch	Add a new symbol to watch.
New Runtime Watch	**Only visible if debug tool supports runtime watches.** Add a new runtime watch for the selected symbol.
Run Time Update Interval	**Only visible if debug tool supports runtime watches.** Specify the rate at which the symbol value will be updated. "No Delay" will update as quickly as your personal computer is able. If you are seeing errors in the runtime data, you can add a delay to decrease the update speed.
Export Selected Watches to List File	Export information about the symbol(s) to be watched to a file (`.xwatch`). You can select more than one row using Shift click (select several rows in order) or Ctrl click (select several rows individually).
Export Data	Export watch data to a CSV file or as a literal string.
Delete	Delete selected symbol row(s) from the watch list. You can also select a row and hit the <Delete> key.
Delete All	Remove all the watched symbols from the Watches window.
Display Value Column as	Display the symbol value in the selected row in one of the formats listed. See also 13.24.3. Fractional Integer Properties Dialog.
User Defined Size	Specify the size of the program memory symbol from the list.

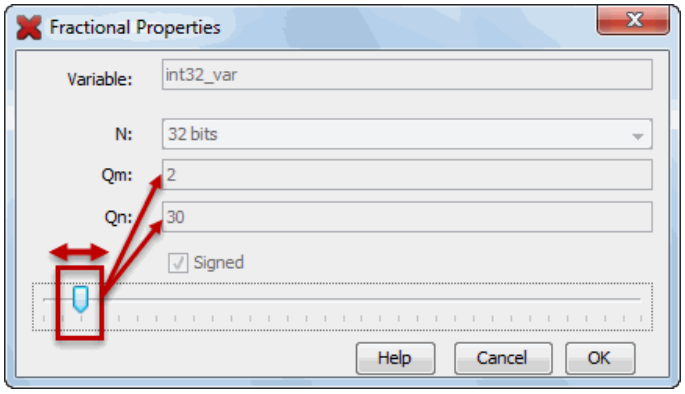
13.24.3 Fractional Integer Properties Dialog

This dialog is intended to improve support for evaluating all fixed-point operation modes supported by dsPIC and PIC32 devices. View this dialog from the pop-up menu item *Display Value Column as>Fractional Integer* to select this feature and *Display Value Column as>Fractional Properties* optionally afterwards.

Table 13-88. Fractional Properties Dialog Items

Dialog Item	Description
Variable	The name of the variable, for reference.
N*	The total number of bits available to represent the integer number.
Qm	The whole part of the fixed data format. 'm' represents the number bits used to specify the whole part.
Qn	The fractional part of the fixed data format. 'n' represents the number of bits used to specify the fractional part.
Signed*	Check to use a bit to represent a signed value.
Slider	Click and drag the slider to select the fixed point position (Qm vs. Qn) to be applied to the calculation of the fixed point fractional value.
* If the variable type is unknown, these items are made editable.	

Figure 13-52. Fractional Properties Dialog

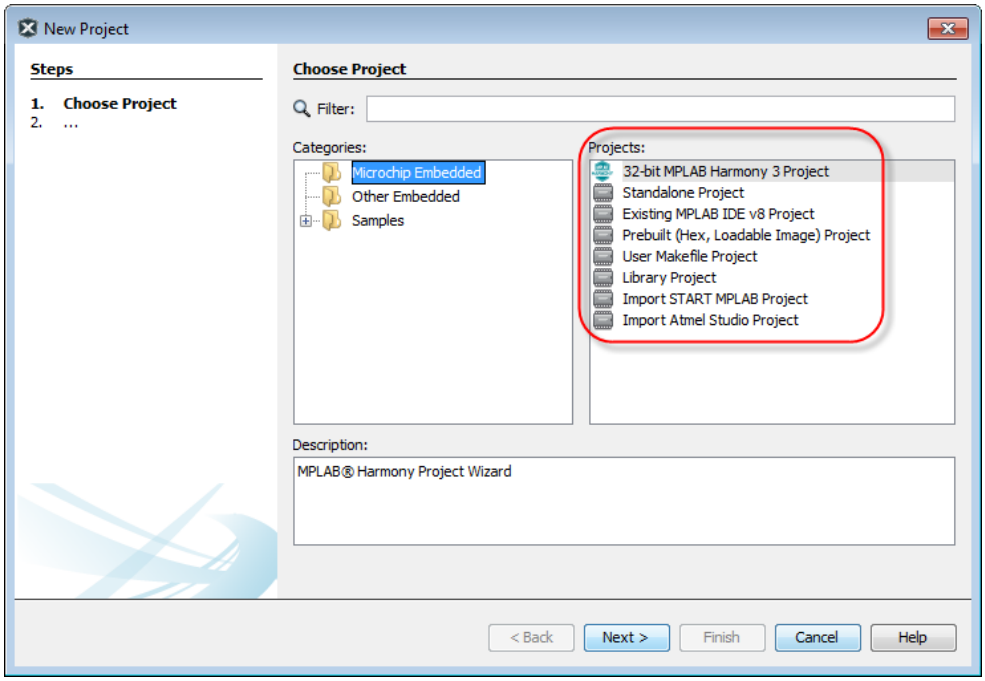


13.25 Wizards

A wizard is a sequential progression of dialog windows that guide you through a process. Two common wizards in MPLAB X IDE are:

- The New Project wizard (see Create a New Project).
- The New File wizard (see Create a New File).

Figure 13-53. New Project Types



14. NetBeans Windows and Dialogs

MPLAB X IDE windows and dialogs are a combination of basic NetBeans windows and dialogs and MPLAB X IDE specific windows and dialogs. Help topics are displayed with a white background for NetBeans items and a yellow background for MPLAB X IDE items.

NetBeans windows and dialogs are discussed here. For information on MPLAB X IDE specific windows and dialogs see 13. MPLAB X IDE Windows and Dialogs.

14.1 NetBeans Specific Windows and Window Menus

NetBeans windows are discussed in NetBeans help or web documentation (see either “About This Help File” or “Preface”). To open help about a window, click on the window tab to select it and then hit the <F1> key. If no help can be found, select *Help>Help Contents* and click the help file’s **Search** tab to search for information on that window, or see the table of contents for the NetBeans help topic “Managing IDE Windows.”

To open most windows, see the 12.1.11. Window Menu.

Windows may be docked and undocked (right click on the window tab) and have window-specific pop-up, or context, menus with items such as:

- Fill
- Goto
- Edit Cells

Right clicking in a window or on an item in the window will pop up a context menu. Most content menu items are also located on menus on the desktop menu bar (see 12.1. Menus).

Set up window properties by selecting *Tools>Options* (*MPLAB X IDE>Preferences* for macOS), **Miscellaneous** button, **Appearance** tab.

14.2 NetBeans Specific Dialogs

NetBeans dialogs are discussed in NetBeans help. To open help about a dialog, click the **Help** button on the dialog or, if there is no Help button, press the <F1> key. If no help can be found, select *Help>Help Contents* and click the help file’s **Search** tab to search for information on that dialog.

To open most dialogs, see 12.1. Menus.

© 2022 Microchip Technology Inc. and its subsidiaries

15. Configuration Settings Summary

Examples of how to set configuration bits in code for different language tools and related devices are shown below. For more information on how to set configuration bits, see your language tool documentation. For some language tools, a configurations settings document is available listing all configuration settings for a device. Otherwise, consult your device header file for macros.

Another option is to use the Configuration Memory window to set bits and then click “Generate Source Code to Output.” See 5.20. Set Configuration Values in the Configuration Bits Window.

The MPLAB Code Configurator (MCC) for 8- and 16-bit devices and MPLAB Harmony for 32-bit devices are tools that can be used to generate configuration bits, as well as project code. See the Microchip website for more information on these tools.

15.1 AVR GCC Toolchain

Most configuration bits are contained in device Fuse bytes. To program the fuse bits, the Fuse API is used. To use this API, include the `<avr/io.h>` header file, which provides everything necessary to set the AVR fuses.

To program code protection configuration bits, or lock bits, the Lockbit API is used. To use this API, include the `<avr/io.h>` header file.

An example using the `FUSES` and `LOCKBITS` macros is provided below. For more information on the Fuse and Lockbit APIs, see:

- AVR Fuses - Developer Help
- Use of Lock Bits in AVR devices

Example: ATtiny861 MCU

```
// ATtiny817 Configuration Bit Settings

// 'C' source line config statements

#include <avr/io.h>

FUSES = {
    0x00, // WDTCFG{PERIOD=OFF, WINDOW=OFF}
    0x02, // BODCFG{SLEEP=SAMPLED, ACTIVE=DIS, SAMPFREQ=1KHz, LVL=BODLEVEL0}
    0x00, // OSCCFG{OSCLOCK=CLEAR}
    0x00, // Reserved
    0xC4, // TCD0CFG{CMPA=CLEAR, CMPB=CLEAR, CMPC=SET, CMPD=CLEAR, CMPAEN=CLEAR,
CMPBEN=CLEAR, CMPCEN=SET, CMPDEN=SET}
    0xF6, // SYSCFG0{EESAVE=CLEAR, RSTPINCFG=UPDI, CRCSRC=NOCRC}
    0x00, // SYSCFG1{SUT=0MS}
    0x00, // APPEND
    0x00, // BOOTEND
};

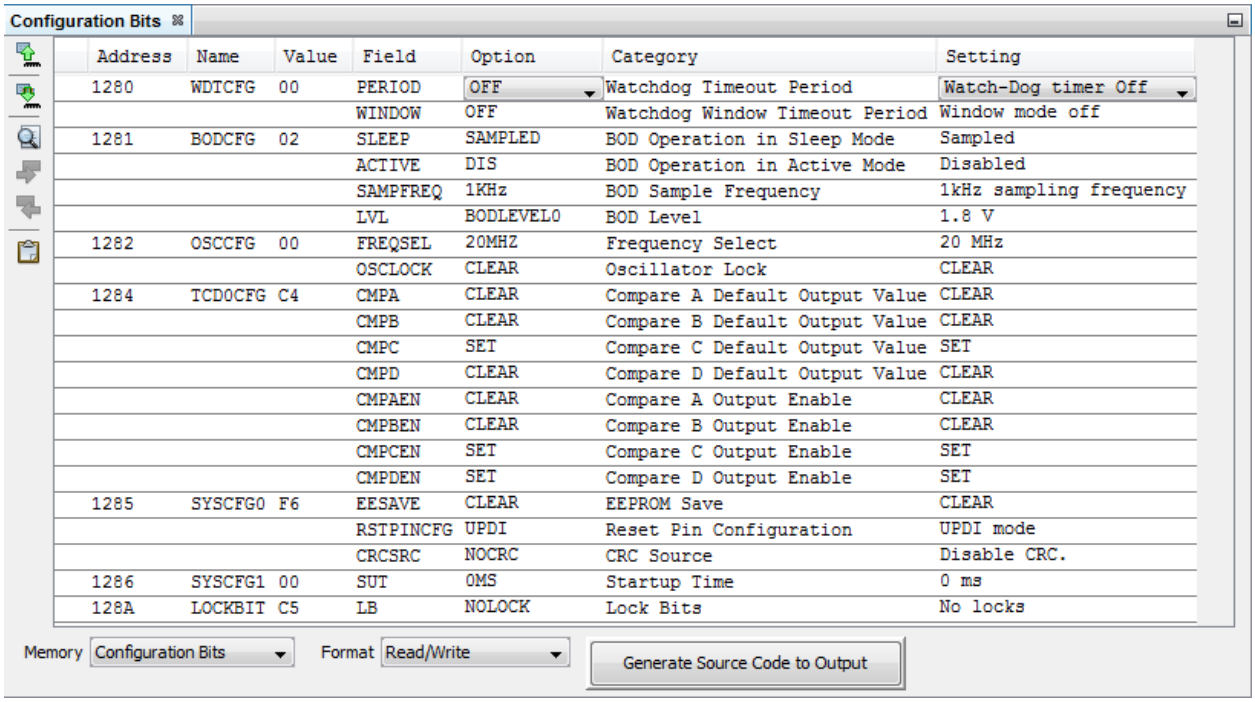
LOCKBITS = {
    0xC5, // LOCKBIT{LB=NOLOCK}
};
```

The example above was generated from the Configuration Bits window (*Window>Target Memory Views>Configuration Bits*).

When you first open the Configuration Bits window, the data will be red, meaning you need to read from device memory. Use the “Read Configuration Bits” icon  . Make any changes to the data in the window and then write changes back to the device by using the “Program Configuration Bits” icon  .

To preserve your settings in code, click either the “Generate Source Code to Output” button, to put the code in the Output window for copy-and-paste into your code, or the “Insert Source Code into Editor” icon , to place the code at the cursor in an editor window.

Figure 15-1. Example: ATtiny817 MCU



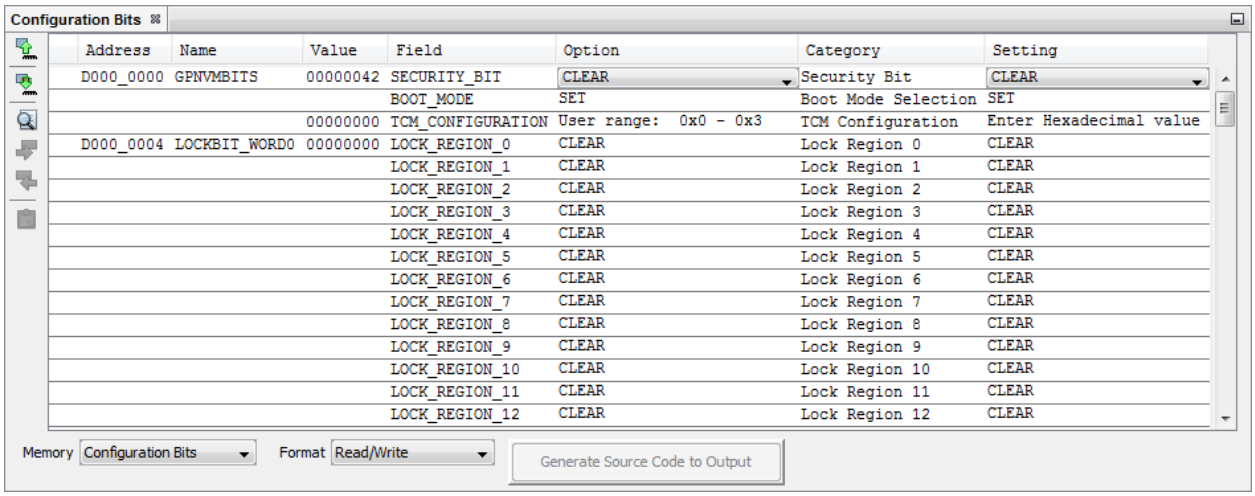
15.2 Arm GCC Toolchain

Most configuration bits are contained in device GPNVM bits. Specifically, the GPNVM0 represents the Security bit.

To program code protection configuration bits, Lock bits are used.

To program configuration bits, use the Configuration Bits window (*Window>Target Memory Views>Configuration Bits*). When you first open the Configuration Bits window, the data will be red, meaning you need to read from device memory. Use the “Read Configuration Bits” icon . Make any changes to the data in the window and then write changes back to the device by using the “Program Configuration Bits” icon .

Example: ATSAME70Q21 MCU



15.3 XC Toolchains

To create code that is as portable as possible, refer to the “Common Compiler Interface (CCI)” chapter, `config` macro, in each MPLAB XC C compiler document.

- *MPLAB® XC8 C Compiler User’s Guide for PIC MCU* (DS50002737) or *MPLAB® XC8 C Compiler User’s Guide for AVR MCU* (DS50002750)
- *MPLAB® XC16 C Compiler User’s Guide* (DS50002071)
- *MPLAB® XC32 C/C++ Compiler User’s Guide for PIC32M MCUs* (DS50002799) or *MPLAB® XC32 C/C++ Compiler User’s Guide for PIC32C/SAM MCUs* (DS50002895)

To read, change, and program configuration bits, see 5.20. Set Configuration Values in the Configuration Bits Window.

16. Revision History

The following is a list of changes by version to this document.
Note: Some revision letters are not used - the letters `I` and `O` - as they can be confused for numbers in some fonts.

16.1 Revision F (January 2022)

- Document structure updated to follow Development Tools Ecosystem flow.
- MPLAB ICE 4 in-circuit emulator added to development tools list where appropriate.
- **Chapter 3**. “Before You Begin”: Added and updated several links to more content in sections.
- **Chapter 4**. “Tutorial”: Updated tutorial with a new example from MPLAB Discover.
- **Chapter 5**. “Basic Development and Debug Flow”: Renamed from “Basic Tasks”. Link fixes, Macro updates.
- **Chapter 6**. “Projects”: New chapter made from project-related sections of previous chapters. **Section 6.1**. “Work with Packs”: from “Additional Tasks”. **Section 6.2**. “Project Types”: from “Additional Tasks”, “Advanced Tasks & Concepts”, and “Project Files and Folders”. Updated “Import Atmel Studio 7 or Atmel START Project” screens. **Section 6.3**. “Project Files”: from “Additional Tasks” and “Advanced Tasks & Concepts”. **Section 6.4**. “Work with Projects”: from “Advanced Tasks & Concepts”. Updated `mplab_ide.conf` contents in Section 6.4.2. “Speed Up MPLAB X IDE”.**Section 6.5**. “Project Reference”: from “Project Files and Folders”.
- **Chapter 7**. “Editor”: Updated links.
- **Chapter 8**. “Development and Debug Features”: from “Additional Tasks” and “Advanced Tasks & Concepts”. Clarifications in “View the Disassembly Window”.
- **Chapter 9**. “Analysis”: Renamed from “Advanced Tasks & Concepts>Use Analysis Features”. **Section 9.1**. Compiler Advisor: New feature. **Section 9.2**. “Stack Guidance”: from “Use Analysis Features”.**Section 9.3**. “MPLAB Analysis Tool Suite”: New license covering MPLAB Code Coverage and MISRA check.
- **Chapter 10**. “Automation”: New chapter. **Section 10.1**. “IDE Scripting (autoload.py)”: from “Advanced Tasks & Concepts>IDE Scripting (autoload.py)”. **Section 10.2**. “CI/CD Wizard”: New feature.
- **Chapter 11**. “Troubleshooting”: Updated/fixed links. **Section 10.5.3**. “Couldn’t reserve space for Cygwin™ heap (Windows® OS”): Updated. **Section 10.5.6**. “For 16-bit code, MPLAB XC16 packed attribute statement causes red bangs in the Editor window”: Corrected code.
- **Chapter 12**. “Desktop Reference”: Minor updates to **Section 12.1**. “Menus”.
- **Chapter 13**. “MPLAB X IDE Windows and Dialogs”: Added “MPLAB X IDE Windows Behavior” chapter to beginning of this chapter. Minor updates to windows and content menu items. **Section 13.10**. “Licenses Windows”: Added content on additional windows. **Section 13.11**. “Manage Network Tools Window”: Added for MPLAB ICE 4.
- **Chapter 14**. “NetBeans Windows and Dialogs”: No updates.
- **Chapter 15**. “Configuration Settings Summary”: Added/fixed links.

WebHelp Only (May 2021)

- Updated user’s guide for the impact of MPLAB X IDE changing to a 64-bit only application: Removed content pertaining to MPASM assembler and other legacy language tools that are not fully 64-bit, updated USB driver information, and updated Windows OS paths from "Program Files (x86)" to "Program Files".
- Changed references from Oracle Java to OpenJDK.
- **Chapter 1**: Added Preface. Other chapter numbering affected.
- Removed references to support lights for debug tools and compilers in chapters 5 and 6.
- Added reference to Hexmate User’s Guide instead of MPLAB XC8 User’s Guide: **Section 5.10.4** “Normalize hex file” and **Section 6.5** “Loadable Projects, Files and Symbols”.
- AVR GCC issues: **Section 6.19.1** “Dashboard Groups”, Memory, compiler does not provide an accurate report of memory usage, and **Section 3.5** “Install the Language Tools”, for a compiler on a mounted drive, version information not discovered for a Linux OS.
- **Chapter 2** updates: 2.7 “Components of MPLAB X IDE” updated from webpage content; 2.8 “MPLAB X IDE Help” and 2.9 “Other MPLAB Documentation” merged and updated; 2.9 “MPLAB Discover” created; 2.13 “MPLAB X IDE Updates” added pack information.

- **Chapter 4**. “Tutorial”: Added section referencing “Other Examples”; In 4.13 “Viewing Device Memory (including Configuration Bits)” added image and description of “Read Device Memory” button on a memory window.
- **Section 5.1** “Create a New Project”: Combined Device and Tool steps into single dialog. Updated subsections to reflect changes for DFPs and TPs. Added **Section 5.1.10** “Changing the Project Configuration Type” as a link to another relevant section.
- **Section 5.18** “View or Change Device Memory”: Added image and description of “Read Device Memory” button on a memory window.
- **Section 6.1** “Work with Packs”. Subsections rearranged for better flow and content updated for DFPs and Tool packs.
- **Section 6.3** “Use a Compatible Device Project” added to explain some example projects that include a compatible device list on opening.
- **Section 6.4.2** “Import MPLAB IDE v8 Project Wizard: Combined Device and Tool steps into single dialog.
- **Section 6.10** “Other Embedded Projects”: Added information on IAR Embedded Workbench® import.
- **Section 6.23.1** “Saving Project Files”: Added “.generated_files directory” to table.
- **Section 7.9.2** “User Makefile Project Folder and Working Folder”: Added a note that package project is not supported for makefile projects.
- **Section 7.10** “Log Data”: Updated information.
- **Section 7.13** “Use IDE Scripting”: Updated information. Added section regarding “Debug Extensibility using Python Scripts”.
- **Section 11.1.1** “File Menu”: How to clear the list of recent projects.
- **Section 11.1.7** “Production Menu”: Added “Erase Device Memory”.
- **Section 11.2.5** “Run Toolbar”: Added “Erase Device Memory” to “Make and Program Device Project” description. Added AVR support to Live Connect Debug.
- **Section 13.14** “Pack Manager Window”: Added details on using the Pack Manager.
- **Section 13.16** “Projects Window”: Updated table “File Context Menu Items” under “Projects Window - Window Menu.”
- **Chapter 16**. “Revision History”: Added this chapter.

16.2 Revision E (September 2019)

- Overall updated screen images and related text for the latest IDE.
- **Chapter 1**. “What is MPLAB X IDE?” updated to add more device types supported. 1.13 “Programming Center” added.
- **Chapter 2**. “Before You Begin” updated. 2.2 “Installation Requirements” added.
- **Chapter 3**. “Tutorial”. This chapter has been updated with new tools and references on-line code.
- **Chapter 4**. “Basic Tasks” updated. 4.2.1 “MPLAB X IDE v5 New Project Format” added. 4.23 “Set Configuration Bits in the Configuration Bits Window”. Split out this information from 4.22 “View/Change Device Memory”.
- **Chapter 6**. “Advanced Tasks and Concepts” had many updates.
  - 6.3 “Speed Up Build Times”. 5.13 moved to here.
  - 6.6 “Create Dual Core Projects”. Added section.
  - 6.6 “Package an MPLAB X IDE Project”. Moved to 6.12.
  - 6.8 “Use Linked Resources for Source File Folders”. Added section.
  - 6.10 “Use Code Coverage”. Added section.
  - 6.13 “Hardware Tool Connections and Debugging”. Added section
  - 6.14 “Use IDE Scripting”. Added section.
- **Chapter 7**. “Editor” updated. 7.4 “Hyperlinks in Code” taken from “Editor Usage”. 7.8 “C/C++ Best Code Practices” added.
- **Chapter 8**. “Project Files and Folders” updated. 8.6 “File or Folder Name Restrictions” updated name and content to “Path, File and Folder Name Restrictions. Also added 8.7 “Path, File and Folder Project Recommendations”.
- **Chapter 9**. “Troubleshooting” updated. 9.3 “Cross-Platform (Operating System) Issues” and 9.4 “Windows Operating System Issues” combined into 9.3 “Operating System Issues”.

- **Chapter 10**. “MPLAB X IDE Migration” renamed from “MPLAB X IDE vs. MPLAB ICE v8”.
- **Chapter 12**. “MPLAB X IDE Windows Behavior” added. 12.1 “MPLAB X IDE Windows Management” taken from “MPLAB X IDE Windows and Dialogs” chapter. 12.2 “Banked Data Memory and Values Displayed in Window” added.

16.3 Revision D (September 2015)

- Updated images for MPLAB X IDE v3.00 (NetBeans 8.1 platform)
- Added information about the MPLAB X Store for the tab, icon and Help menu item.
- Added information about *Help>Tool Help Contents*, individual help files.
- **Chapter 2**: 2.6 “Access Information from the Start Page”: Updated information.
- **Chapter 2**: 2.9 “Launch Multiple Versions of the IDE”: Clarifies version execution.
- **Chapter 4**: 4.13 “Build Properties”: Added “Change project Configuration type”.
- **Chapter 4**: 4.2.2 “Step 2: Select Device”: Added info on LF devices.
- **Chapter 5**: 5.6 “Library Projects”: Added link to section “Change project Configuration type”.
- **Chapter 6**: Changed name to “Advanced Tasks & Concepts”. Added section groupings. Added sections “Work with Third-Party Hardware Tools” and “Configurations”. Updated section “Create User Makefile Projects” content and added examples
- **Chapter 9**: 9.6 “Errors”: Added “HEXMATE Conflict Report Address Error” and “Programming Errors”.
- **Chapter 12**: 12.4 “Watches Window”: Swapped location of Watches Display and Watches Menu sections. Added more content to menu section. Added section “Fractional Integer Properties Dialog”.
- **Chapter 12**: 12.7.5 “EE Data Memory Window”: Updated text for read during debug or after end of debug session. Updated start address table.
- **Chapter 12**: 12.10 “Message Center” added.
- **Appendix B**: Reordered existing sections and added the section “Creating Makefiles Outside of MPLAB X IDE”.

16.4 Revision C (March 2014)

- Overall Changes: Removed references to old tools. Added information on Windows 8. Updated screens and text to match the latest GUI.
- **Chapter 1**. “What is MPLAB X IDE?”: Updated figure references. Mentioned stand-alone tool help and the Microchip wiki as resources.
- **Chapter 2**. “Before You Begin”: Updated information in 2.3.2 “USB Driver Installation for Windows® XP/7/8 Operating Systems”. Added compiler licensing information under 2.5 “Install the Language Tools”. Added 2.7.1 “Setting Up Hardware Tools to Work with Multiple Instances”.
- **Chapter 4**. “Basic Tasks”: Removed 4.1 “Introduction”. Added 4.6.1 “Add or Change a Toolchain” and 4.6.2 “About Toolchain Paths”. Added how to exclude file/folders from build under 4.12 “Set File and Folder Properties”. Expanded Table 4-3 into sections under 4.13 “Set Build Properties”. Added 4.15.2 “Run Considerations”. Added 4.16.2 “Debug Macros Generated” and 4.16.3 “Debug Considerations”. Added symbol information in 4.19 “Watch Symbol Values Change”. Updated 4.21.2 “Change Device Memory”. Added 4.23.1 “Set Project Programming Properties”.
- **Chapter 5**. “Additional Tasks”: Removed 5.1 “Introduction”. Updates 5.4 “Loadable Projects and Files” and created a new section, 5.5 “Loadable Projects and Files – Bootloaders”. Added 5.12 “Modify Project Folders and Encoding” and 5.13 “Speed Up Build Times”. Updated text to match new display options in 5.17 “View the Dashboard Display”. Fixed using version control steps and updated project files that need to be saved into a repository under 5.19 “Control Source Code”. Updated options for 5.20 “Collaborate on Code Development and Error Tracking”. Reordered sections under 5.21 “Add Plug-In Tools” and added 5.21.4 “Plug-In Code Location”.
- **Chapter 6**. Advanced Tasks: Added 6.1 “Speed Up MPLAB X IDE” and 6.5 “Create User MakeFile Projects”. Added information about working without setting an active project and grouping projects under 6.3 “Work with Multiple Projects”. Removed statement that debug configuration needed (it is not) from 6.4 “Work with Multiple Configurations”. Log file requirements update under 6.6 “Log File”.
- **Chapter 7**. “Editor”: “Editor Features of Note” moved to 7.2.4 under 7.2 “Editor Usage”. Also added 7.2.1. “Desktop Controls”, 7.2.2 “Hyperlinks in C Code” and 7.2.3 “Hyperlinks in ASM Code”. Macros definition

expanded in Table 7-6 “Macros Tab”. Rearranged 7.4 “Code Folding” into 7.4.1 “Code Folding Usage” (includes MPLAB C18 and assembly folding issue) and 7.4.2. “Custom Code Folding”.

- **Chapter 8**. “Project Files and Folders”: This chapter moved from Chapter 13. Added 8.4 “Favorites Window View”, 8.5 “Classes Window View”, 8.6 “Viewing User Configuration Data” and 8.9 “Deleting a Project”. Changed “Moving a Project” to 8.8 “Moving, Copying or Renaming a Project”. Moved “Building a Project Outside of MPLAB X IDE” to Appendix B. “Working Outside the IDE”.
- **Chapter 9**. “Troubleshooting”: Added information to 9.6 “Errors”.
- **Chapter 10**. “MPLAB X IDE vs. MPLAB IDE v8”: Clarified under 10.2 “Major Differences”, item 3. “MPLAB X IDE allows multiple tool selection” and item 9. “MPLAB X IDE uses configuration bits set in code”. Other minor updates for MPLAB X IDE additional selections.
- **Chapter 11**. “Desktop Reference”: 11.2.10 “Tools Menu” updated for plug-ins and compiler licensing. Other minor updates for MPLAB X IDE additional selections.
- **Chapter 12**. “MPLAB X IDE Windows and Dialogs”: Added 12.2 “MPLAB X IDE Windows Management”. In 12.3 “MPLAB X IDE Windows with Related Menus and Dialogs”, added new windows to table and moved windows subsections to their own sections. Added 12.4 “Breakpoints Window” and 12.6 “Licenses Window”. Extensive updates to add descriptions of different memory windows under 12.8 “Memory Windows”. 12.11 “Projects Window” menus updates. 12.12 “Tools Options Embedded Window” options update, including the addition of 12.12.5 “Diagnostics Tab”. 12.14 “Watches Window” content broken up into three sections.
- **Appendix A**: “Configuration Settings”: Change from Chapter 14 to an appendix. Added information about the Configuration Memory window.
- **Appendix B**: “Working Outside the IDE”: B.1 “Building a Project Outside of MPLAB X IDE” from Chapter 8 “Project Files and Folders”. Added B.2 “Compiling for Debug Outside of MPLAB X IDE”.
- **Appendix C**: “Revision History”: This chapter updated for major changes from version B to C.

16.5 Revision B (October 2012)

- JRE installation now automatic with Windows and Linux operating systems. Mac operating system instructions provided. Discussed in **Chapter 2**. “Before You Begin”, 2.2 “Install JRE and MPLAB X IDE”.
- Corrected information on device driver names and paths in **Chapter 2**.
- Added description of meaning of the two lights next to a debug tool in **Chapter 3**. “Tutorial” and Chapter 4. “Basic Tasks”.
- Updated Icon definitions in **Chapter 3**. “Tutorial” and Chapter 4. “Basic Tasks”.
- Added a list of language toolchain abbreviations in **Chapter 4**. “Basic Tasks”, 4.3 “Create a New Project”, 4.3.5 “Step 5: Select Compiler”.
- Multiple dialogs updated to show projects using the new MPLAB XC C compiler.
- Added information on the Libraries category in the Project Properties dialog in **Chapter 4**. “Basic Tasks”, 4.12 “Add and Setup Library and Object Files”.
- Added information on normalizing a hex file under **Chapter 4**. “Basic Tasks”, 4.14 “Set Build Properties”.
- Added information on how to add a literal value to a Watches window in **Chapter 4**. “Basic Tasks”, 4.20 “Watch Symbol Values Changes”.
- Updated several sections for recent support of C++ (currently for the MPLAB XC32++ Compiler).
- Added section on loadable projects in **Chapter 5**. “Additional Tasks”, 5.5 “Loadable Projects and Files”.
- Discussed how to import embedded projects from other applications into MPLAB X IDE in **Chapter 5**. “Additional Tasks”, 5.7 “Other Embedded Projects”.
- Added information on working with samples projects in **Chapter 5**. “Additional Tasks”, 5.8 “Sample Projects”.
- Described how to work with other files, like XML, in **Chapter 5**. “Additional Tasks”, 5.9 “Work with Other Types of Files”.
- Added information on how to work with log files in **Chapter 6**. Advanced Tasks”, 6.4 “Log Data”.
- Described how to add functions to a toolbar in **Chapter 6**. Advanced Tasks”, 6.5 “Customize Toolbars”.
- Added **Chapter 7**. Editor, that discusses MPLAB X IDE Editor usage, options and features.
- Described an error message you might receive if you modify or move the default linker script, and how to avoid this. See **Chapter 8**. “Troubleshooting”, 8.4 “MPLAB X IDE Issues”.

- Combined “Major Differences” and “Feature Differences” into **Chapter 9**. “MPLAB X IDE vs. MPLAB IDE v8”, 9.2 “Major Differences”. Explained NetBeans platform is open source, but MPLAB X IDE is proprietary.
- In **Chapter 9**. “MPLAB X IDE vs. MPLAB IDE v8”, updated 9.3 “Menu Differences” to reflect new functions in MPLAB X IDE and 9.4 “Tool Support Differences” to reflect new plug-in support.
- In **Chapter 10**. “Desktop Reference”, updated 10.2 “Menus” to reflect new functions in MPLAB X IDE.
- Split “Windows and Dialogs” into **Chapter 11**. “MPLAB X IDE Windows and Dialogs” and Chapter 12. “NetBeans Windows and Dialogs” to better delineate the different windows. Updated window menus.
- Updated project file structure in **Chapter 13**. “Project Files and Folders”, 13.2 “Files Window View”. Added information on Makefiles.
- Updated discussion of MPLAB XC C compiler configuration bits to reference the common C interface (CCI) in **Chapter 14**. “Configuration Settings Summary”, 14.8 “XC Toolchains”.

16.6 Revision A (November 2011)

Initial release of this document.

17. Glossary

Absolute Section
A GCC compiler section with a fixed (absolute) address that cannot be changed by the linker.

Absolute Variable/Function
A variable or function placed at an absolute address using the OCG compiler's @ address syntax.

Access Memory
PIC18 Only – Special registers on PIC18 devices that allow access regardless of the setting of the Bank Select Register (BSR).

Access Entry Points
Access entry points provide a way to transfer control across segments to a function which may not be defined at link time. They support the separate linking of boot and secure application segments.

Address
A value that identifies a location in memory.

Alphabetic Character
Alphabetic characters are those characters that are letters of the Roman alphabet (a, b, …, z, A, B, …, Z).

Alphanumeric
Alphanumeric characters are comprised of alphabetic characters and decimal digits (0,1, …, 9).

ANDed Breakpoints
Set up an ANDed condition for breaking, i.e., breakpoint 1 AND breakpoint 2 must occur at the same time before a program halt. This can only be accomplished if a data breakpoint and a program memory breakpoint occur at the same time.

Anonymous Structure
16-bit C Compiler – An unnamed structure.

PIC18 C Compiler – An unnamed structure that is a member of a C union. The members of an anonymous structure may be accessed as if they were members of the enclosing union. For example, in the following code, hi and lo are members of an anonymous structure inside the union caster.

```
union castaway
int intval;
struct {
char lo; //accessible as caster.lo
char hi; //accessible as caster.hi
};
} caster;
```

ANSI
The American National Standards Institute is an organization responsible for formulating and approving standards in the United States.

Application
A set of software and hardware that may be controlled by a PIC® microcontroller.

Archive/Archiver
An archive/library is a collection of relocatable object modules. It is created by assembling multiple source files to object files, and then using the archiver/librarian to combine the object files into one archive/library file. An archive/library can be linked with object modules and other archives/libraries to create executable code.

ASCII
The American Standard Code for Information Interchange is a character set encoding that uses 7 binary digits to represent each character. It includes upper and lower case letters, digits, symbols and control characters.

Assembly/Assembler
Assembly is a programming language that describes binary machine code in a symbolic form. An assembler is a language tool that translates assembly language source code into machine code.

Assigned Section
A GCC compiler section which has been assigned to a target memory block in the linker command file.

Asynchronously
Multiple events that do not occur at the same time. This is generally used to refer to interrupts that may occur at any time during processor execution.

Asynchronous Stimulus
Data generated to simulate external inputs to a simulator device.

Attribute
GCC Characteristics of variables or functions in a C language program, which are used to describe machine-specific properties.

Attribute, Section
GCC Characteristics of sections, such as “executable”, “read-only”, or “data” that can be specified as flags in the assembler `.section` directive.

AVR MCUs
AVR® microcontrollers (MCUs) refer to all Microchip AVR 8-bit microcontroller families.

Binary
The base two numbering system that uses the digits 0-1. The rightmost digit counts ones, the next counts multiples of 2, then $2^2 = 4$, etc.

Bookmarks
Use bookmarks to easily locate specific lines in a file.

Select Toggle Bookmarks on the Editor toolbar to add/remove bookmarks. Click other icons on this toolbar to move to the next or previous bookmark.

C/C++
C is a general-purpose programming language which features economy of expression, modern control flow and data structures, as well as a rich set of operators. C++ is the object-oriented version of C.

Calibration Memory
A special function register or registers used to hold values for calibration of a PIC microcontroller on-board RC oscillator or other device peripherals.

© 2022 Microchip Technology Inc.
and its subsidiaries

Central Processing Unit
The part of a device that is responsible for fetching the correct instruction for execution, decoding that instruction, and then executing that instruction. When necessary, it works in conjunction with the arithmetic logic unit (ALU) to complete the execution of the instruction. It controls the program memory address bus, the data memory address bus, and accesses to the stack.

Clean
Clean removes all intermediary project files, such as object, hex and debug files, for the active project. These files are recreated from other files when a project is built.

COFF
Common Object File Format. An object file of this format contains machine code, debugging and other information.

Command Line Interface
A means of communication between a program and its user based solely on textual input and output.

Compiled Stack
A region of memory managed by the compiler in which variables are statically allocated space. It replaces a software or hardware stack when such mechanisms cannot be efficiently implemented on the target device.

Compiler
A program that translates a source file written in a high-level language into machine code.

Conditional Assembly
Assembly language code that is included or omitted based on the assembly-time value of a specified expression.

Conditional Compilation
The act of compiling a program fragment only if a certain constant expression, specified by a preprocessor directive, is true.

Configuration Bits
Special-purpose bits programmed to set PIC MCU and dsPIC DSC modes of operation. A Configuration bit may or may not be preprogrammed.

Control Directives
Directives in assembly language code that cause code to be included or omitted based on the assembly-time value of a specified expression.

CPU
See *Central Processing Unit*.

Cross Reference File
A file that references a table of symbols and a list of files that references the symbol. If the symbol is defined, the first file listed is the location of the definition. The remaining files contain references to the symbol.

Data Directives
Data directives are those that control the assembler’s allocation of program or data memory and provide a way to refer to data items symbolically; that is, by meaningful names.

Data Memory
On Microchip MCU and DSC devices, data memory (RAM) is comprised of General Purpose Registers (GPRs) and Special Function Registers (SFRs). Some devices also have EEPROM data memory.

Data Monitor and Control Interface (DMCI)
The Data Monitor and Control Interface, or DMCI, is a tool in MPLAB X IDE. The interface provides dynamic input control of application variables in projects. Application-generated data can be viewed graphically using any of 4 dynamically-assignable graph windows.

Debug/Debugger
See *ICE/ICD*.

Debugging Information
Compiler and assembler options that, when selected, provide varying degrees of information used to debug application code. See compiler or assembler documentation for details on selecting debug options.

Deprecated Features
Features that are still supported for legacy reasons, but will eventually be phased out and no longer used.

Device Programmer
A tool used to program electrically programmable semiconductor devices such as microcontrollers.

Digital Signal Processing\Digital Signal Processor
Digital signal processing (DSP) is the computer manipulation of digital signals, commonly analog signals (sound or image) which have been converted to digital form (sampled). A digital signal processor is a microprocessor that is designed for use in digital signal processing.

Directives
Statements in source code that provide control of the language tool's operation.

Download
Download is the process of sending data from a host to another device, such as an emulator, programmer or target board.

dsPIC DSCs
dsPIC® digital signal controllers (DSCs) refer to the Microchip family of microcontrollers with digital signal processing capability.

DWARF
Debug With Arbitrary Record Format. DWARF is a debug information format for ELF files.

EEPROM
Electrically Erasable Programmable Read Only Memory. A special type of PROM that can be erased electrically. Data is written or erased one byte at a time. EEPROM retains its contents even when power is turned off.

ELF
Executable and Linking Format. An object file of this format contains machine code. Debugging and other information is specified in with DWARF. ELF/DWARF provide better debugging of optimized code than COFF.

Emulation/Emulator
See *ICE/ICD*.

Endianness
The ordering of bytes in a multi-byte object.

Environment
MPLAB PM3 – A folder containing files on how to program a device. This folder can be transferred to a SD/MMC card.

Epilogue
A portion of compiler-generated code that is responsible for deallocating stack space, restoring registers and performing any other machine-specific requirement specified in the runtime model. This code executes after any user code for a given function, immediately prior to the function return.

EPROM
Erasable Programmable Read Only Memory. A programmable read-only memory that can be erased usually by exposure to ultraviolet radiation.

Error/Error File
An error reports a problem that makes it impossible to continue processing your program. When possible, an error identifies the source file name and line number where the problem is apparent. An error file contains error messages and diagnostics generated by a language tool.

Event
A description of a bus cycle which may include address, data, pass count, external input, cycle type (fetch, R/W) and time stamp. Events are used to describe triggers, breakpoints and interrupts.

Executable Code
Software that is ready to be loaded for execution.

Export
Send data out of the MPLAB X IDE in a standardized format.

Expressions
Combinations of constants and/or symbols separated by arithmetic or logical operators.

Extended Microcontroller Mode
In extended microcontroller mode, on-chip program memory as well as external memory is available. Execution automatically switches to external if the program memory address is greater than the internal memory space of the PIC18 device.

Extended Mode (PIC18 MCUs)
In Extended mode, the compiler will utilize the extended instructions (i.e., `ADDFSR`, `ADDULNK`, `CALLW`, `MOVSF`, `MOVSS`, `PUSHL`, `SUBFSR`, and `SUBULNK`) and the indexed with literal offset addressing.

External Label
A label that has external linkage.

External Linkage
A function or variable has external linkage if it can be referenced from outside the module in which it is defined.

External Symbol
A symbol for an identifier which has external linkage. This may be a reference or a definition.

External Symbol Resolution
A process performed by the linker in which external symbol definitions from all input modules are collected in an attempt to resolve all external symbol references. Any external symbol references which do not have a corresponding definition cause a linker error to be reported.

© 2022 Microchip Technology Inc. and its subsidiaries

External Input Line
An external input signal logic probe line (TRIGIN) for setting an event based upon external signals.

External RAM
Off-chip Read/Write memory.

Fatal Error
An error that halts compilation immediately. No further messages will be produced.

File Registers
On-chip data memory, including General Purpose Registers (GPRs) and Special Function Registers (SFRs).

Filter
Determine by selection what data is included/excluded in a trace display or data file.

Fixup
The process of replacing object file symbolic references with absolute addresses after relocation by the linker.

Flash
A type of EEPROM where data is written or erased in blocks instead of bytes.

FNOP
Forced No Operation. A forced NOP cycle is the second cycle of a two-cycle instruction. Since the PIC microcontroller architecture is pipelined, it prefetches the next instruction in the physical address space while it is executing the current instruction. However, if the current instruction changes the program counter, this prefetched instruction is explicitly ignored, causing a forced NOP cycle.

Frame Pointer
A pointer that references the location on the stack that separates the stack-based arguments from the stack-based local variables. Provides a convenient base from which to access local variables and other values for the current function.

Free-Standing
An implementation that accepts any strictly conforming program that does not use complex types and in which the use of the features specified in the library clause (ANSI ‘89 standard clause 7) is confined to the contents of the standard headers `<float.h>`, `<iso646.h>`, `<limits.h>`, `<stdarg.h>`, `<stdbool.h>`, `<stddef.h>`, and `<stdint.h>`.

GPR
General Purpose Register. The portion of device data memory (RAM) available for general use.

Halt
A stop of program execution. Executing Halt is the same as stopping at a breakpoint.

Heap
An area of memory used for dynamic memory allocation where blocks of memory are allocated and freed in an arbitrary order determined at run-time.

Hex Code/Hex File
Hex code is executable instructions stored in a hexadecimal format code. Hex code is contained in a hex file.

© 2022 Microchip Technology Inc. and its subsidiaries

Hexadecimal
The base 16 numbering system that uses the digits 0-9 plus the letters A-F (or a-f). The digits A-F represent hexadecimal digits with values of (decimal) 10 to 15. The rightmost digit counts ones, the next counts multiples of 16, then $16^2 = 256$, etc.

High Level Language
A language for writing programs that is further removed from the processor than assembly.

ICE/ICD
In-Circuit Emulator/In-Circuit Debugger: A hardware tool that debugs and programs a target device. An emulator has more features than an debugger, such as trace.

In-Circuit Emulation/In-Circuit Debug: The act of emulating or debugging with an in-circuit emulator or debugger.

-ICE/-ICD: A device (MCU or DSC) with on-board in-circuit emulation or debug circuitry. This device is always mounted on a header board and used to debug with an in-circuit emulator or debugger.

ICSP
In-Circuit Serial Programming. A method of programming Microchip embedded devices using serial communication and a minimum number of device pins.

IDE
Integrated Development Environment, as in MPLAB X IDE.

Identifier
A function or variable name.

IEEE
Institute of Electrical and Electronics Engineers.

Import
Bring data into the MPLAB X IDE from an outside source, such as from a hex file.

Initialized Data
Data which is defined with an initial value. In C,

```
int myVar=5;
```

defines a variable, which will reside in an initialized data section.

Instruction Set
The collection of machine language instructions that a particular processor understands.

Instructions
A sequence of bits that tells a central processing unit to perform a particular operation and can contain data to be used in the operation.

Internal Linkage
A function or variable has internal linkage if it can not be accessed from outside the module in which it is defined.

International Organization for Standardization
An organization that sets standards in many businesses and technologies, including computing and communications. Also known as ISO.

© 2022 Microchip Technology Inc. and its subsidiaries

Interrupt
A signal to the CPU that suspends the execution of a running application and transfers control to an Interrupt Service Routine (ISR) so that the event may be processed. Upon completion of the ISR, normal execution of the application resumes.

Interrupt Handler
A routine that processes special code when an interrupt occurs.

Interrupt Service Request (IRQ)
An event which causes the processor to temporarily suspend normal instruction execution and to start executing an interrupt handler routine. Some processors have several interrupt request events allowing different priority interrupts.

Interrupt Service Routine (ISR)
Language tools: A function that handles an interrupt.

MPLAB X IDE: User-generated code that is entered when an interrupt occurs. The location of the code in program memory will usually depend on the type of interrupt that has occurred.

Interrupt Vector
Address of an interrupt service routine or interrupt handler.

L-value
An expression that refers to an object that can be examined and/or modified. An l-value expression is used on the left-hand side of an assignment.

Latency
The time between an event and its response.

Library/Librarian
See *Archive/Archiver*.

Linker
A language tool that combines object files and libraries to create executable code, resolving references from one module to another.

Linker Script Files
Linker script files are the command files of a linker. They define linker options and describe available memory on the target platform.

Listing Directives
Listing directives are those directives that control the assembler listing file format. They allow the specification of titles, pagination and other listing control.

Listing File
A listing file is an ASCII text file that shows the machine code generated for each C source statement, assembly instruction, assembler directive, or macro encountered in a source file.

Little Endian
A data ordering scheme for multi-byte data, whereby the least significant byte is stored at the lower addresses.

Local Label
A local label is one that is defined inside a macro with the LOCAL directive. These labels are particular to a given instance of a macro's instantiation. In other words, the symbols and labels that are declared as local are no longer accessible after the ENDM macro is encountered.

© 2022 Microchip Technology Inc.
and its subsidiaries

Machine Code
The representation of a computer program that is actually read and interpreted by the processor. A program in binary machine code consists of a sequence of machine instructions (possibly interspersed with data). The collection of all possible instructions for a particular processor is known as its “instruction set.”

Machine Language
A set of instructions for a specific central processing unit, designed to be usable by a processor without being translated.

Macro
Macro instruction. An instruction that represents a sequence of instructions in abbreviated form.

Macro Directives
Directives that control the execution and data allocation within macro body definitions.

Makefile
Export to a file the instructions to Make the project. Use this file to Make your project outside of MPLAB X IDE, i.e., with a make.

Make Project
A command that rebuilds an application, recompiling only those source files that have changed since the last complete compilation.

MCU
Microcontroller Unit. An abbreviation for microcontroller. Also uC.

Memory Model
For C compilers, a representation of the memory available to the application. For the PIC18 C compiler, a description that specifies the size of pointers that point to program memory.

Message
Text displayed to alert you to potential problems in language tool operation. A message will not stop operation.

Microcontroller
A highly integrated chip that contains a CPU, RAM, program memory, I/O ports and timers.

Microcontroller Mode
One of the possible program memory configurations of PIC18 microcontrollers. In microcontroller mode, only internal execution is allowed. Thus, only the on-chip program memory is available in microcontroller mode.

Microprocessor Mode
One of the possible program memory configurations of PIC18 microcontrollers. In microprocessor mode, the on-chip program memory is not used. The entire program memory is mapped externally.

Mnemonics
Text instructions that can be translated directly into machine code. Also referred to as opcodes.

Module
The preprocessed output of a source file after preprocessor directives have been executed. Also known as a translation unit.

© 2022 Microchip Technology Inc.
and its subsidiaries

MPLAB X IDE
Microchip's Integrated Development Environment. MPLAB X IDE comes with an editor, project manager and simulator.

MPLAB X Simulator
Microchip's simulator that works with MPLAB X IDE in support of Microchip MCU, DSC and MPU devices.

MPLAB XC C Compilers
Microchip's family of C and C++ compilers comprising of the MPLAB XC8 C compiler (8-bit device support), MPLAB XC16 C compiler (16-bit device support) and MPLAB XC32 C/C++ compiler (32-bit support.)

MPLAB Xpress IDE
Microchip's Integrated Development Environment in the Cloud. MPLAB Xpress comes with an editor, project manager and simulator.

MPU
Microprocessor Unit. An abbreviation for microprocessor.

MRU
Most Recently Used. Refers to files and windows available to be selected from MPLAB X IDE main pull down menus.

Native Data Size
For Native trace, the size of the variable used in a Watches window must be of the same size as the selected device's data memory: bytes for PIC18 devices and words for 16-bit devices.

Nesting Depth
The maximum level to which macros can include other macros.

Node
MPLAB X IDE project component.

Non-Extended Mode (PIC18 MCUs)
In Non-Extended mode, the compiler will not utilize the extended instructions nor the indexed with literal offset addressing.

Non Real Time
Refers to the processor at a breakpoint or executing single-step instructions or MPLAB X IDE being run in simulator mode.

Non-Volatile Storage
A storage device whose contents are preserved when its power is off.

NOP
No Operation. An instruction that has no effect when executed except to advance the program counter.

Object Code/Object File
Object code is the machine code generated by an assembler or compiler. An object file is a file containing machine code and possibly debug information. It may be immediately executable or it may be relocatable, requiring linking with other object files, e.g., libraries, to produce a complete executable program.

Object File Directives
Directives that are used only when creating an object file.

© 2022 Microchip Technology Inc.
and its subsidiaries

Octal
The base 8 number system that only uses the digits 0-7. The rightmost digit counts ones, the next digit counts multiples of 8, then 8^2 = 64, etc.

Off-Chip Memory
Off-chip memory refers to the memory selection option for the PIC18 device where memory may reside on the target board, or where all program memory may be supplied by the emulator. The Memory tab accessed from Options>Development Mode provides the Off-Chip Memory selection dialog box.

Opcodes
Operational Codes. See *Mnemonics*.

Operators
Symbols, like the plus sign '+' and the minus sign '-', that are used when forming well-defined expressions. Each operator has an assigned precedence that is used to determine order of evaluation.

OTP
One Time Programmable. EPROM devices that are not in windowed packages. Since EPROM needs ultraviolet light to erase its memory, only windowed devices are erasable.

Pass Counter
A counter that decrements each time an event (such as the execution of an instruction at a particular address) occurs. When the pass count value reaches zero, the event is satisfied. You can assign the Pass Counter to break and trace logic, and to any sequential event in the complex trigger dialog.

PC
Personal Computer or Program Counter.

PC Host
Any PC running a supported Windows operating system.

Persistent Data
Data that is never cleared or initialized. Its intended use is so that an application can preserve data across a device Reset.

Phantom Byte
An unimplemented byte in the dsPIC architecture that is used when treating the 24-bit instruction word as if it were a 32-bit instruction word. Phantom bytes appear in dsPIC hex files.

PIC MCUs
PIC® microcontrollers (MCUs) refers to all Microchip PIC 8-, 16-, and 32-bit microcontroller families.

Plug-ins
The MPLAB X IDE has both built-in components and plug-in modules to configure the system for a variety of software and hardware tools. Several plug-in tools may be found under the Tools menu.

Pod
The enclosure for an in-circuit emulator or debugger. Other names are *Puck*, if the enclosure is round, and *Probe*, not be confused with logic probes.

Power-on-Reset Emulation
A software randomization process that writes random values in data RAM areas to simulate uninitialized values in RAM upon initial power application.

© 2022 Microchip Technology Inc. and its subsidiaries

Pragma
A directive that has meaning to a specific compiler. Often a pragma is used to convey implementation-defined information to the compiler.

Precedence
Rules that define the order of evaluation in expressions.

Production Programmer
A production programmer is a programming tool that has resources designed in to program devices rapidly. It has the capability to program at various voltage levels and completely adheres to the programming specification. Programming a device as fast as possible is of prime importance in a production environment where time is of the essence as the application circuit moves through the assembly line.

Profile
For MPLAB X Simulator, a summary listing of executed stimulus by register.

Program Counter
The location that contains the address of the instruction that is currently executing.

Program Counter Unit
16-bit assembler – A conceptual representation of the layout of program memory. The program counter increments by 2 for each instruction word. In an executable section, 2 program counter units are equivalent to 3 bytes. In a read-only section, 2 program counter units are equivalent to 2 bytes.

Program Memory
MPLAB X IDE: The memory area in a device where instructions are stored. Also, the memory in the debugger, emulator or simulator containing the downloaded target application firmware.

16-bit assembler/compiler: The memory area in a device where instructions are stored.

Project
A project contains the files needed to build an application (source code, linker script files, etc.) along with their associations to various build tools and build options.

Prologue
A portion of compiler-generated code that is responsible for allocating stack space, preserving registers and performing any other machine-specific requirement specified in the run-time model. This code executes before any user code for a given function.

Prototype System
A term referring to a user's target application, or target board.

Psect
The OCG equivalent of a GCC section, short for program section. A block of code or data which is treated as a whole by the linker.

PWM Signals
Pulse Width Modulation Signals. Certain PIC MCU devices have a PWM peripheral.

Qualifier
An address or an address range used by the Pass Counter or as an event before another operation in a complex trigger.

© 2022 Microchip Technology Inc.
and its subsidiaries

Radix
The number base, hex, or decimal, used in specifying an address.

RAM
Random Access Memory (Data Memory). Memory in which information can be accessed in any order.

Raw Data
The binary representation of code or data associated with a section.

Read Only Memory
Memory hardware that allows fast access to permanently stored data but prevents addition to or modification of the data.

Real Time
When an in-circuit emulator or debugger is released from the halt state, the processor runs in Real Time mode and behaves exactly as the normal chip would behave. In Real Time mode, the real time trace buffer of an emulator is enabled and constantly captures all selected cycles, and all break logic is enabled. In an in-circuit emulator or debugger, the processor executes in real time until a valid breakpoint causes a halt, or until the user halts the execution.

In the simulator, real time simply means execution of the microcontroller instructions as fast as they can be simulated by the host CPU.

Recursive Calls
A function that calls itself, either directly or indirectly.

Recursion
The concept that a function or macro, having been defined, can call itself. Great care should be taken when writing recursive macros; it is easy to get caught in an infinite loop where there will be no exit from the recursion.

Re-entrant
A function that may have multiple, simultaneously active instances. This may happen due to either direct or indirect recursion or through execution during interrupt processing.

Relaxation
The process of converting an instruction to an identical, but smaller instruction. This is useful for saving on code size. MPLAB XC16 currently knows how to relax a CALL instruction into an RCALL instruction. This is done when the symbol that is being called is within +/- 32k instruction words from the current instruction.

Relocatable
An object whose address has not been assigned to a fixed location in memory.

Relocatable Section
16-bit assembler – A section whose address is not fixed (absolute). The linker assigns addresses to relocatable sections through a process called relocation.

Relocation
A process performed by the linker in which absolute addresses are assigned to relocatable sections and all symbols in the relocatable sections are updated to their new addresses.

ROM
Read Only Memory (Program Memory). Memory that cannot be modified.

© 2022 Microchip Technology Inc.
and its subsidiaries

Run
The command that releases the emulator from halt, allowing it to run the application code and change or respond to I/O in real time.

Run-time Model
Describes the use of target architecture resources.

Run-time Watch
A Watches window where the variables change in as the application is run. See individual tool documentation to determine how to set up a run-time watch. Not all tools support run-time watches.

SAM MCUs/MPUs
SAM microcontrollers (MCUs) and microprocessors (MPUs) refer to all Microchip SAM 32-bit microcontroller and microprocessor families.

Scenario
For MPLAB X simulator, a particular setup for stimulus control.

Section
The GCC equivalent of an OCG psect. A block of code or data which is treated as a whole by the linker.

Section Attribute
A GCC characteristic ascribed to a section (e.g., an access section).

Sequenced Breakpoints
Breakpoints that occur in a sequence. Sequence execution of breakpoints is bottom-up; the last breakpoint in the sequence occurs first.

Serialized Quick Turn Programming
Serialization allows you to program a serial number into each microcontroller device that the Device Programmer programs. This number can be used as an entry code, password or ID number.

Shell
The MPASM assembler shell is a prompted input interface to the macro assembler. There are two MPASM assembler shells: one for the DOS version and one for the Windows operating system version.

Simulator
A software program that models the operation of devices.

Single Step
This command steps though code, one instruction at a time. After each instruction, MPLAB X IDE updates register windows, watch variables, and status displays so you can analyze and debug instruction execution. You can also single step C compiler source code, but instead of executing single instructions, MPLAB X IDE will execute all assembly level instructions generated by the line of the high level C statement.

Skew
The information associated with the execution of an instruction appears on the processor bus at different times. For example, the executed opcodes appear on the bus as a fetch during the execution of the previous instruction; the source data address, value, and destination data address appear when the opcodes are actually executed; and the destination data value appears when the next instruction is executed. The trace buffer captures the information that is on the bus at one instance. Therefore, one trace buffer entry will contain execution information for three instructions. The number of captured cycles from one piece of information to another for a single instruction execution is referred to as the skew.

© 2022 Microchip Technology Inc.
and its subsidiaries

Skid
When a hardware breakpoint is used to halt the processor, one or more additional instructions may be executed before the processor halts. The number of extra instructions executed after the intended breakpoint is referred to as the skid.

Source Code
The form in which a computer program is written by the programmer. Source code is written in a formal programming language which can be translated into machine code or executed by an interpreter.

Source File
An ASCII text file containing source code.

Special Function Registers (SFRs)
The portion of data memory (RAM) dedicated to registers that control I/O processor functions, I/O status, timers or other modes or peripherals.

SQTP
See *Serialized Quick Turn Programming*.

Stack, Hardware
Locations in PIC microcontroller where the return address is stored when a function call is made.

Stack, Software
Memory used by an application for storing return addresses, function parameters, and local variables. This memory is dynamically allocated at run-time by instructions in the program. It allows for re-entrant function calls.

Stack, Compiled
A region of memory managed and allocated by the compiler in which variables are statically assigned space. It replaces a software stack when such mechanisms cannot be efficiently implemented on the target device. It precludes re-entrancy.

Static RAM or SRAM
Static Random Access Memory. Program memory you can read/write on the target board that does not need refreshing frequently.

Status Bar
The Status Bar is located on the bottom of the MPLAB X IDE window and indicates such current information as cursor position, development mode and device, and active tool bar.

Step Into
This command is the same as Single Step. Step Into (as opposed to Step Over) follows a CALL instruction into a subroutine.

Step Over
Step Over allows you to debug code without stepping into subroutines. When stepping over a `CALL` instruction, the next breakpoint will be set at the instruction after the `CALL`. If for some reason the subroutine gets into an endless loop or does not return properly, the next breakpoint will never be reached. The Step Over command is the same as Single Step except for its handling of `CALL` instructions.

Step Out
Step Out allows you to step out of a subroutine which you are currently stepping through. This command executes the rest of the code in the subroutine and then stops execution at the return address to the subroutine.

© 2022 Microchip Technology Inc. and its subsidiaries

Stimulus
Input to the simulator, i.e., data generated to exercise the response of simulation to external signals. Often the data is put into the form of a list of actions in a text file. Stimulus may be asynchronous, synchronous (pin), clocked and register.

Stopwatch
A counter for measuring execution cycles.

Storage Class
Determines the lifetime of the memory associated with the identified object.

Storage Qualifier
Indicates special properties of the objects being declared (e.g., const).

Symbol
A symbol is a general purpose mechanism for describing the various pieces which comprise a program. These pieces include function names, variable names, section names, file names, struct/enum/union tag names, etc. Symbols in MPLAB X IDE refer mainly to variable names, function names and assembly labels. The value of a symbol after linking is its value in memory.

Symbol, Absolute
Description

System Window Control
The system window control is located in the upper left corner of windows and some dialogs. Clicking on this control usually pops up a menu that has the items “Minimize,” “Maximize,” and “Close.”

Target
Refers to user hardware.

Target Application
Software residing on the target board.

Target Board
The circuitry and programmable device that makes up the target application.

Target Processor
The microcontroller device on the target application board.

Template
Lines of text that you build for inserting into your files at a later time. The MPLAB Editor stores templates in template files.

Term
Represents an immediate value such as a definition through the assembly .equ directive.

Toolbar
A row or column of icons that you can click on to execute MPLAB X IDE functions.

Trace
An emulator or simulator function that logs program execution. The emulator logs program execution into its trace buffer which is uploaded to the MPLAB X IDE trace window.

© 2022 Microchip Technology Inc.
and its subsidiaries

Trace Memory
Trace memory contained within the emulator. Trace memory is sometimes called the trace buffer.

Trace Macro
A macro that will provide trace information from emulator data. Since this is a software trace, the macro must be added to code, the code must be recompiled or reassembled, and the target device must be programmed with this code before trace will work.

Trigger Output
Trigger output refers to an emulator output signal that can be generated at any address or address range, and is independent of the trace and breakpoint settings. Any number of trigger output points can be set.

Trigraphs
Three-character sequences, all starting with ??, that are defined by ISO C as replacements for single characters.

Unassigned Section
A section which has not been assigned to a specific target memory block in the linker command file. The linker must find a target memory block in which to allocate an unassigned section.

Uninitialized Data
Data which is defined without an initial value. In C,

```
int myVar;
```

defines a variable which will reside in an uninitialized data section.

Upload
The Upload function transfers data from a tool, such as an emulator or programmer, to the host computer or from the target board to the emulator.

USB
Universal Serial Bus. An external peripheral interface standard for communication between a computer and external peripherals over a cable using bi-serial transmission. Currently supported USB versions for Microchip hardware tools are:

USB	Speed Descriptor	Maximum Speed (Megabits/Gigabits per second)
USB 2.0	High Speed	480 Mbps
USB 3.0	SuperSpeed	5 Gbps

Vector
The memory locations that an application will jump to when either a Reset or interrupt occurs.

Volatile
A variable qualifier which prevents the compiler applying optimizations that affect how the variable is accessed in memory.

Warning
Warning
MPLAB X IDE: An alert that is provided to warn you of a situation that would cause physical damage to a device, software file, or equipment.

16-bit assembler/compiler: Warnings report conditions that may indicate a problem, but do not halt processing.

Watch Variable
A variable that you may monitor during a debugging session in a Watches window.

Watches Window
Watches windows contain a list of watch variables that are updated at each breakpoint.

Watchdog Timer (WDT)
A timer on a PIC microcontroller that resets the processor after a selectable length of time. The WDT is enabled or disabled and set up using Configuration bits.

Workbook
For MPLAB X Stimulator, a setup for generation of SCL stimulus.

© 2022 Microchip Technology Inc.
and its subsidiaries

The Microchip Website

Microchip provides online support via our website at www.microchip.com/. This website is used to make files and information easily available to customers. Some of the content available includes:

- **Product Support** – Data sheets and errata, application notes and sample programs, design resources, user's guides and hardware support documents, latest software releases and archived software
- **General Technical Support** – Frequently Asked Questions (FAQs), technical support requests, online discussion groups, Microchip design partner program member listing
- **Business of Microchip** – Product selector and ordering guides, latest Microchip press releases, listing of seminars and events, listings of Microchip sales offices, distributors and factory representatives

Product Change Notification Service

Microchip's product change notification service helps keep customers current on Microchip products. Subscribers will receive email notification whenever there are changes, updates, revisions or errata related to a specified product family or development tool of interest.

To register, go to www.microchip.com/pcn and follow the registration instructions.

Customer Support

Users of Microchip products can receive assistance through several channels:

- Distributor or Representative
- Local Sales Office
- Embedded Solutions Engineer (ESE)
- Technical Support

Customers should contact their distributor, representative or ESE for support. Local sales offices are also available to help customers. A listing of sales offices and locations is included in this document.

Technical support is available through the website at: www.microchip.com/support

Microchip Devices Code Protection Feature

Note the following details of the code protection feature on Microchip products:

- Microchip products meet the specifications contained in their particular Microchip Data Sheet.
- Microchip believes that its family of products is secure when used in the intended manner, within operating specifications, and under normal conditions.
- Microchip values and aggressively protects its intellectual property rights. Attempts to breach the code protection features of Microchip product is strictly prohibited and may violate the Digital Millennium Copyright Act.
- Neither Microchip nor any other semiconductor manufacturer can guarantee the security of its code. Code protection does not mean that we are guaranteeing the product is "unbreakable". Code protection is constantly evolving. Microchip is committed to continuously improving the code protection features of our products.

Legal Notice

This publication and the information herein may be used only with Microchip products, including to design, test, and integrate Microchip products with your application. Use of this information in any other manner violates these terms. Information regarding device applications is provided only for your convenience and may be superseded by updates. It is your responsibility to ensure that your application meets with your specifications. Contact your local Microchip sales office for additional support or, obtain additional support at www.microchip.com/en-us/support/design-help/client-support-services.

THIS INFORMATION IS PROVIDED BY MICROCHIP "AS IS". MICROCHIP MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND WHETHER EXPRESS OR IMPLIED, WRITTEN OR ORAL, STATUTORY OR OTHERWISE, RELATED TO THE INFORMATION INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A PARTICULAR PURPOSE, OR WARRANTIES RELATED TO ITS CONDITION, QUALITY, OR PERFORMANCE.

IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE, INCIDENTAL, OR CONSEQUENTIAL LOSS, DAMAGE, COST, OR EXPENSE OF ANY KIND WHATSOEVER RELATED TO THE INFORMATION OR ITS USE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S TOTAL LIABILITY ON ALL CLAIMS IN ANY WAY RELATED TO THE INFORMATION OR ITS USE WILL NOT EXCEED THE AMOUNT OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THE INFORMATION.

Use of Microchip devices in life support and/or safety applications is entirely at the buyer's risk, and the buyer agrees to defend, indemnify and hold harmless Microchip from any and all damages, claims, suits, or expenses resulting from such use. No licenses are conveyed, implicitly or otherwise, under any Microchip intellectual property rights unless otherwise stated.

Trademarks

The Microchip name and logo, the Microchip logo, Adaptec, AnyRate, AVR, AVR logo, AVR Freaks, BesTime, BitCloud, CryptoMemory, CryptoRF, dsPIC, flexPWR, HELDO, IGLOO, JukeBlox, KeeLoq, Kleer, LANCheck, LinkMD, maXStylus, maXTouch, MediaLB, megaAVR, Microsemi, Microsemi logo, MOST, MOST logo, MPLAB, OptoLyzer, PIC, picoPower, PICSTART, PIC32 logo, PolarFire, Prochip Designer, QTouch, SAM-BA, SenGenuity, SpyNIC, SST, SST Logo, SuperFlash, Symmetricom, SyncServer, Tachyon, TimeSource, tinyAVR, UNI/O, Vectron, and XMEGA are registered trademarks of Microchip Technology Incorporated in the U.S.A. and other countries.

AgileSwitch, APT, ClockWorks, The Embedded Control Solutions Company, EtherSynch, Flashtec, Hyper Speed Control, HyperLight Load, IntelliMOS, Libero, motorBench, mTouch, Powermite 3, Precision Edge, ProASIC, ProASIC Plus, ProASIC Plus logo, Quiet- Wire, SmartFusion, SyncWorld, Temux, TimeCesium, TimeHub, TimePictra, TimeProvider, TrueTime, WinPath, and ZL are registered trademarks of Microchip Technology Incorporated in the U.S.A.

Adjacent Key Suppression, AKS, Analog-for-the-Digital Age, Any Capacitor, AnyIn, AnyOut, Augmented Switching, BlueSky, BodyCom, CodeGuard, CryptoAuthentication, CryptoAutomotive, CryptoCompanion, CryptoController, dsPICDEM, dsPICDEM.net, Dynamic Average Matching, DAM, ECAN, Espresso T1S, EtherGREEN, GridTime, IdealBridge, In-Circuit Serial Programming, ICSP, INICnet, Intelligent Paralleling, Inter-Chip Connectivity, JitterBlocker, Knob-on-Display, maxCrypto, maxView, memBrain, Mindi, MiWi, MPASM, MPF, MPLAB Certified logo, MPLIB, MPLINK, MultiTRAK, NetDetach, NVM Express, NVMe, Omniscient Code Generation, PICDEM, PICDEM.net, PICkit, PICtail, PowerSmart, PureSilicon, QMatrix, REAL ICE, Ripple Blocker, RTAX, RTG4, SAM-ICE, Serial Quad I/O, simpleMAP, SimpliPHY, SmartBuffer, SmartHLS, SMART-I.S., storClad, SQI, SuperSwitcher, SuperSwitcher II, Switchtec, SynchroPHY, Total Endurance, TSHARC, USBCheck, VariSense, VectorBlox, VeriPHY, ViewSpan, WiperLock, XpressConnect, and ZENA are trademarks of Microchip Technology Incorporated in the U.S.A. and other countries.

SQTP is a service mark of Microchip Technology Incorporated in the U.S.A.

The Adaptec logo, Frequency on Demand, Silicon Storage Technology, Symmcom, and Trusted Time are registered trademarks of Microchip Technology Inc. in other countries.

GestIC is a registered trademark of Microchip Technology Germany II GmbH & Co. KG, a subsidiary of Microchip Technology Inc., in other countries.

All other trademarks mentioned herein are property of their respective companies.

© 2022, Microchip Technology Incorporated and its subsidiaries. All Rights Reserved.

ISBN: 978-1-5224-9573-4

Quality Management System

For information regarding Microchip's Quality Management Systems, please visit www.microchip.com/quality.

Worldwide Sales and Service

AMERICAS	ASIA/PACIFIC	ASIA/PACIFIC	EUROPE
Corporate Office 2355 West Chandler Blvd. Chandler, AZ 85224-6199 Tel: 480-792-7200 Fax: 480-792-7277 Technical Support: www.microchip.com/support Web Address: www.microchip.com **Atlanta** Duluth, GA Tel: 678-957-9614 Fax: 678-957-1455 **Austin, TX** Tel: 512-257-3370 **Boston** Westborough, MA Tel: 774-760-0087 Fax: 774-760-0088 **Chicago** Itasca, IL Tel: 630-285-0071 Fax: 630-285-0075 **Dallas** Addison, TX Tel: 972-818-7423 Fax: 972-818-2924 **Detroit** Novi, MI Tel: 248-848-4000 **Houston, TX** Tel: 281-894-5983 **Indianapolis** Noblesville, IN Tel: 317-773-8323 Fax: 317-773-5453 Tel: 317-536-2380 **Los Angeles** Mission Viejo, CA Tel: 949-462-9523 Fax: 949-462-9608 Tel: 951-273-7800 **Raleigh, NC** Tel: 919-844-7510 **New York, NY** Tel: 631-435-6000 **San Jose, CA** Tel: 408-735-9110 Tel: 408-436-4270 **Canada - Toronto** Tel: 905-695-1980 Fax: 905-695-2078	**Australia - Sydney** Tel: 61-2-9868-6733 **China - Beijing** Tel: 86-10-8569-7000 **China - Chengdu** Tel: 86-28-8665-5511 **China - Chongqing** Tel: 86-23-8980-9588 **China - Dongguan** Tel: 86-769-8702-9880 **China - Guangzhou** Tel: 86-20-8755-8029 **China - Hangzhou** Tel: 86-571-8792-8115 **China - Hong Kong SAR** Tel: 852-2943-5100 **China - Nanjing** Tel: 86-25-8473-2460 **China - Qingdao** Tel: 86-532-8502-7355 **China - Shanghai** Tel: 86-21-3326-8000 **China - Shenyang** Tel: 86-24-2334-2829 **China - Shenzhen** Tel: 86-755-8864-2200 **China - Suzhou** Tel: 86-186-6233-1526 **China - Wuhan** Tel: 86-27-5980-5300 **China - Xian** Tel: 86-29-8833-7252 **China - Xiamen** Tel: 86-592-2388138 **China - Zhuhai** Tel: 86-756-3210040	**India - Bangalore** Tel: 91-80-3090-4444 **India - New Delhi** Tel: 91-11-4160-8631 **India - Pune** Tel: 91-20-4121-0141 **Japan - Osaka** Tel: 81-6-6152-7160 **Japan - Tokyo** Tel: 81-3-6880- 3770 **Korea - Daegu** Tel: 82-53-744-4301 **Korea - Seoul** Tel: 82-2-554-7200 **Malaysia - Kuala Lumpur** Tel: 60-3-7651-7906 **Malaysia - Penang** Tel: 60-4-227-8870 **Philippines - Manila** Tel: 63-2-634-9065 **Singapore** Tel: 65-6334-8870 **Taiwan - Hsin Chu** Tel: 886-3-577-8366 **Taiwan - Kaohsiung** Tel: 886-7-213-7830 **Taiwan - Taipei** Tel: 886-2-2508-8600 **Thailand - Bangkok** Tel: 66-2-694-1351 **Vietnam - Ho Chi Minh** Tel: 84-28-5448-2100	**Austria - Wels** Tel: 43-7242-2244-39 Fax: 43-7242-2244-393 **Denmark - Copenhagen** Tel: 45-4485-5910 Fax: 45-4485-2829 **Finland - Espoo** Tel: 358-9-4520-820 **France - Paris** Tel: 33-1-69-53-63-20 Fax: 33-1-69-30-90-79 **Germany - Garching** Tel: 49-8931-9700 **Germany - Haan** Tel: 49-2129-3766400 **Germany - Heilbronn** Tel: 49-7131-72400 **Germany - Karlsruhe** Tel: 49-721-625370 **Germany - Munich** Tel: 49-89-627-144-0 Fax: 49-89-627-144-44 **Germany - Rosenheim** Tel: 49-8031-354-560 **Israel - Ra'anana** Tel: 972-9-744-7705 **Italy - Milan** Tel: 39-0331-742611 Fax: 39-0331-466781 **Italy - Padova** Tel: 39-049-7625286 **Netherlands - Drunen** Tel: 31-416-690399 Fax: 31-416-690340 **Norway - Trondheim** Tel: 47-72884388 **Poland - Warsaw** Tel: 48-22-3325737 **Romania - Bucharest** Tel: 40-21-407-87-50 **Spain - Madrid** Tel: 34-91-708-08-90 Fax: 34-91-708-08-91 **Sweden - Gothenberg** Tel: 46-31-704-60-40 **Sweden - Stockholm** Tel: 46-8-5090-4654 **UK - Wokingham** Tel: 44-118-921-5800 Fax: 44-118-921-5820

© 2022 Microchip Technology Inc.
and its subsidiaries